

Frontiers
in
Artificial
Intelligence
and
Applications

NEURO-SYMBOLIC ARTIFICIAL INTELLIGENCE: THE STATE OF THE ART

Edited by
Pascal Hitzler
Md Kamruzzaman Sarker



IOS Press

NEURO-SYMBOLIC ARTIFICIAL INTELLIGENCE: THE STATE OF THE ART

Frontiers in Artificial Intelligence and Applications

The book series Frontiers in Artificial Intelligence and Applications (FAIA) covers all aspects of theoretical and applied Artificial Intelligence research in the form of monographs, selected doctoral dissertations, handbooks and proceedings volumes. The FAIA series contains several sub-series, including 'Information Modelling and Knowledge Bases' and 'Knowledge-Based Intelligent Engineering Systems'. It also includes the biennial European Conference on Artificial Intelligence (ECAI) proceedings volumes, and other EurAI (European Association for Artificial Intelligence, formerly ECCAI) sponsored publications. The series has become a highly visible platform for the publication and dissemination of original research in this field. Volumes are selected for inclusion by an international editorial board of well-known scholars in the field of AI. All contributions to the volumes in the series have been peer reviewed.

The FAIA series is indexed in ACM Digital Library; DBLP; EI Compendex; Google Scholar; Scopus; Web of Science: Conference Proceedings Citation Index – Science (CPCI-S) and Book Citation Index – Science (BKCI-S); Zentralblatt MATH.

Series Editors:

Joost Breuker, Nicola Guarino, Pascal Hitzler, Joost N. Kok, Jiming Liu,
Ramon López de Mántaras, Riichiro Mizoguchi, Mark Musen, Sankar K. Pal,
Ning Zhong

Volume 342

Recently published in this series

- Vol. 341. A.J. Tallón-Ballesteros (Ed.), Modern Management based on Big Data II and Machine Learning and Intelligent Systems III – Proceedings of MMBD 2021 and MLIS 2021
- Vol. 340. A.J. Tallón-Ballesteros (Ed.), Fuzzy Systems and Data Mining VII – Proceedings of FSDM 2021
- Vol. 339. M. Villaret, T. Alsinet, C. Fernández and A. Valls (Eds.), Artificial Intelligence Research and Development – Proceedings of the 23rd International Conference of the Catalan Association for Artificial Intelligence
- Vol. 338. C. Frasson, K. Kabassi and A. Voulodimos (Eds.), Novelties in Intelligent Digital Systems – Proceedings of the 1st International Conference (NIDS 2021), Athens, Greece, September 30 – October 1, 2021
- Vol. 337. H. Fujita and H. Perez-Meana (Eds.), New Trends in Intelligent Software Methodologies, Tools and Techniques – Proceedings of the 20th International Conference on New Trends in Intelligent Software Methodologies, Tools and Techniques (SoMeT_21)
- Vol. 336. A. Biere, M. Heule, H. van Maaren and T. Walsh (Eds.), Handbook of Satisfiability – Second Edition

ISSN 0922-6389 (print)
ISSN 1879-8314 (online)

Neuro-Symbolic Artificial Intelligence: The State of the Art

Edited by

Pascal Hitzler

Kansas State University, USA

and

Md Kamruzzaman Sarker

University of Hartford, USA



IOS Press

Amsterdam • Berlin • Washington, DC

© 2022 The authors and IOS Press.

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, without prior written permission from the publisher.

ISBN 978-1-64368-244-0 (print)

ISBN 978-1-64368-245-7 (online)

Library of Congress Control Number: 2021952106

doi: 10.3233/FAIA342

Publisher

IOS Press BV

Nieuwe Hemweg 6B

1013 BG Amsterdam

Netherlands

fax: +31 20 687 0019

e-mail: order@iospress.nl

For book sales in the USA and Canada:

IOS Press, Inc.

6751 Tepper Drive

Clifton, VA 20124

USA

Tel.: +1 703 830 6300

Fax: +1 703 830 2300

sales@iospress.com

LEGAL NOTICE

The publisher is not responsible for the use which might be made of the following information.

PRINTED IN THE NETHERLANDS

Preface:

The 3rd AI Wave Is Coming, and It Needs a Theory

Frank van Harmelen, Vrije Universiteit Amsterdam

The history of an entire scientific field as dynamic and permeable as AI can only be captured in metaphors. But even though they are metaphors, such sketches of history do sometimes manage to capture the essence of how a debate in an entire scientific field evolved. One such useful metaphor about the history of AI is that of the “two towers”, the “two tribes”, or even the “two religions”. These two tribes are known under a variety of names: symbolic vs. subsymbolic, reasoning vs. learning, logic vs. data, model-based vs. function-based, but I prefer to use the terms “knowledge driven” vs. “data driven”. For decades, AI was indeed divided into these two tribes, whose members adhered to different religions: either AI systems were based on manipulating symbols (as succinctly expressed through the “physical symbol systems hypothesis” by Newel and Simon in their 1976 Turing Award paper), or AI systems were based on statistical patterns to be learned from large amounts of data (an approach that earned Bengio, Hinton and Lecun the 2018 Turing Award). And as with any two proper religions, for many years their proponents did not read each other’s books, did not go to each other’s meetings, and did not take each other’s ideas very seriously.

Another, somewhat different metaphor about the history of AI is the picture of two waves. In this myth, the “first wave” of AI consisted of investigations into knowledge-driven AI, with successes in knowledge-based systems (1980s and 1990s), the world-championship chess (1997) and conversational assistants such as Siri (from 2010 onwards). The “second wave” of AI consisted of the rise of Deep Learning systems with impressive successes in machine translations, image recognition and game playing, among many others. If Google Trends is to be believed, the current wide-spread interest in Deep Learning started to take off in early 2015, although its roots of course stretch back deep into AI history, with the current frenzied levels of interest only reached from early 2018 onwards. The more general term “machine learning” shows a similar trend curve.

But things are moving fast in the field of AI. A generous flow of public funding, a keen interest from industry, and a culture of sharing data and code are all contributing factors to the fast pace of progress. And this fast pace of progress is rapidly shattering both of the above metaphors, both the two-tribes myth and the two-waves myth. Increasingly, the two tribes are now converging on the view that neither purely data-driven nor purely knowledge-driven systems alone hold the key to further progress in AI. And increasingly, the first knowledge-driven wave and the second data-driven wave are being rapidly joined by a third wave of systems that are, in their wide variety, known under the common term of “neuro-symbolic systems”.

However, although the proponents of such neuro-symbolic systems are united in what they agree that AI systems should not be (namely: they should be neither purely

knowledge-driven nor purely data-driven), these researchers are not at all united in their view on what AI should be. The literature on neuro-symbolic systems (sometimes also known as “hybrid” systems) is rapidly growing, with dozens of new (and often unreviewed) manuscripts appearing on ArXiv every week, but these contributions cover a wide variety of different architectural and algorithmic proposals, with precious little agreement on which architectures suit which purpose, which algorithms should be used when, what the strengths and weaknesses of the different approaches are, how the results should be evaluated, or even what the theoretical limits are of the different proposals.

Neuro-symbolic systems are the offspring of two very different parents, roughly speaking the fields of “knowledge representation” and “machine learning”, and each of these parents has its own strengths and weaknesses. The first of these parents is characterised by strong theoretical foundations, with analyses on expressivity, on computational tractability, on the trade-offs between the two, and with a compositional theory on how to combine multiple representations. The second parent is characterised by a strong emphasis on empirical results, measured on widely accepted benchmarks and challenge tasks. Conversely, the Knowledge Representation field has never managed to agree on widely shared benchmarks (although exceptions exist in subdisciplines), while much of the classical machine learning theory, founded on the statistical learning theory from the ‘90s, no longer suffices to explain the “unreasonable effectiveness” of deep learning (with [1] a recent attempt at addressing the situation).

The promising “3rd wave of AI”, the neuro-symbolic systems to which this book is devoted, should aim to combine the strengths of both parents. Coming, as I do, from a Knowledge Representation background, I am particularly concerned that we should strive for a good theoretical understanding of neuro-symbolic systems, beyond purely empirically measuring their capabilities. We should start to make some foundational distinctions: what are the possible interactions between knowledge and learning? Can reasoning be used as a symbolic prior for learning? Can such symbolic priors be used to make data-driven systems more sample efficient? More explainable? More generalisable outside the training set? Should such symbolic priors be domain-specific, or should they be in the form of very general knowledge capturing only the abstract structure of time, space, causality and other such general principles? Can symbolic constraints be enforced on data-driven systems to make them more safe? Or less biased? Or can, vice versa, learning be used to yield symbolic knowledge? And if so, how to manage the inherent uncertainty that comes with such learned knowledge? And how to avoid the inevitable bias seeping from the data into the resulting knowledge base? And can such learned knowledge ever capture abstract principles of time, space and causality, or will it only be limited to the domain-specific patterns that arise from a given dataset? Or perhaps, and hopefully, the answer is: both directions (knowledge-driven methods supporting data-driven methods and vice versa) are possible and even required.

However, neuro-symbolic systems currently lack a theory that even begins to ask these questions, let alone answer them. All too often, new conference papers and ArXiv manuscripts simply propose a new neuro-symbolic architecture, or a new algorithm, without even discussing which of the above questions (or any others, for that matter) they aim to address. Systematic surveys such as [2] are a good start for such a theory. Our own work on an informal “boxology” of neuro-symbolic systems ([3] and [4]) is an attempt at such a “theory of neuro-symbolic systems”, but it is highly incomplete, and entirely informal. It remains a challenge to put such a theory on a firm mathematical ba-

sis (for example, through pre- and post-conditions on the components of neuro-symbolic systems). Can such a theory be closely coupled with fragments of executable code? And most importantly: is the assumption of loosely coupled components perhaps too strong, and will much more tightly coupled neuro-symbolic systems in the end turn out to be the way forward?

But whatever the answers to these questions will be, we will only make truly scientific and long-lasting progress on AI in general, and on hybrid, neuro-symbolic systems of the “3rd wave of AI” in particular, if we go beyond building systems. We must, in the end, develop a theory of such systems that allows us to predict and understand what neuro-symbolic systems can and can’t do.

References

- [1] Berner J, Grohs P, Kutyniok G, et al. The modern mathematics of deep learning. CoRR. 2021; abs/2105.04026. Available from: <https://arxiv.org/abs/2105.04026>.
- [2] Marra G, Dumancic S, Manhaeve R, et al. From statistical relational to neural symbolic artificial intelligence: a survey. CoRR. 2021;abs/2108.11451. Available from: <https://arxiv.org/abs/2108.11451>.
- [3] van Harmelen F, ten Teije A. A boxology of design patterns for hybrid learning and reasoning systems. J Web Eng. 2019;18(1-3):97–124.
- [4] van Bekkum M, de Boer M, van Harmelen F, et al. Modular design patterns for hybrid learning and reasoning systems. Appl Intell. 2021;51(9):6528–6546.

Introduction

Pascal Hitzler, Kansas State University

Md Kamruzzaman Sarker, University of Hartford

Neuro-symbolic AI is an emerging subfield of Artificial Intelligence that brings together two of its prominent approaches in its field, as indicated by its name: "neuro" (sometimes "neural") refers to artificial neural networks, prominent in machine learning, and in the guise of deep learning the main driver for the current increase in research and development in AI; "symbolic" refers to algorithmic processing on the level of meaningful symbols, prominent in knowledge representation, and often expressed by means of formal logics and corresponding logical reasoning algorithms.

The neural and the symbolic worlds in AI have been rather disjoint in the past, as has been eloquently elaborated on in the Preface [1]. The two approaches also differ significantly in terms of their strengths and weaknesses, and in terms of the types of applications they are most adequate for. Neural systems are made for machine learning, for being trained on raw data in order to achieve a particular objective like classification or forecasting. They are robust against mistakes or noise in the data, and they are very quick once a system is trained. On all these aspects, in contrast, symbolic systems in general do not do so well. However they thrive if an application depends on exactly encoded knowledge, on provable correctness, or on being able to inspect an algorithm run in order to understand reasons for algorithm behavior beyond a mere statistical assessment of output correctness. On all these aspects, neural approaches are generally very lacking. So, from a technical perspective, there is the hope – and by now there is also some evidence, as presented throughout this book – that a combination of neural and symbolic approaches may provide a bit of a best-of-two-worlds scenario, with each component being able to compensate for the others' weaknesses.

There is also another, more foundational, motivation for pursuing neuro-symbolic AI research. Neural approaches can be understood as mathematical or computational abstractions of the human neural system. Likewise, Symbolic approaches can be understood as mathematical or computational abstractions of human cognitive reasoning as we may perceive it introspectively. From a Cognitive Science perspective, thus, there is a natural fundamental question how a neural system can perform symbolic reasoning, and how the significant representational differences between these two approaches – symbolic representations in a discrete space for symbolic systems, and sub-symbolic representations in a continuous space for symbolic systems – can be bridged.

The first chapter [2] in fact provides a historic (pre-2017) perspective of the field, primarily from a Cognitive Science perspective. It is in fact a 2016 manuscript that has received significant attention, but had not been properly published. We are including this

as a chapter in mostly original form, i.e. it has not been updated, and serves as a historic overview. An update would have required a significant reworking of the chapter.

Indeed, in the last few years activity on neuro-symbolic themes has seen very rapid growth, not only in terms of manuscripts and publications, but also in terms of research events dedicated to the topic [3]. The neuro-symbolic field also seemed to pivot in terms of topics, fueled by the influx of deep learning results and opportunities. It was exactly this pivot which prompted us to pursue this book, i.e., it is based on our own desire to understand the current state of the art with its apparently quite different emphasis from the pre-2017 state discussed in [2].

In order to gather the contributions for this book, we thus began with a survey of recent prominent publications, which led us to a list of authors that have made significant contributions in the past few years. We invited these authors to contribute a chapter giving an overview of their recent contributions to the topic, with co-authors of their choice. Not all those invited chose or were able to contribute, but the chapters which constitute this book do reflect the breadth and depth of the current state of the art in neuro-symbolic AI. It is the book that we wanted to have, the one we can give to our own students and tell them to read it, to get an overview of what's going on in the research area. Chapters were lightly reviewed for feedback.

We thank all the authors, not only for their contributions but also for the quick turn-around which made it possible to produce it – from inviting authors to sending the camera-ready copy to the publisher – within only seven months.

References

- [1] van Harmelen F. Preface: The 3rd AI wave is coming, and it needs a theory. In: Sarker MK, Hitzler P, editors. Neuro-symbolic artificial intelligence: The state of the art. IOS Press; 2021. Studies on the Semantic Web. In this volume.
- [2] Besold TR, d'Avila Garcez S, Bader S, et al. Neural-symbolic learning and reasoning: A survey and interpretation. In: Sarker MK, Hitzler P, editors. Neuro-symbolic artificial intelligence: The state of the art. IOS Press; 2021. Studies on the Semantic Web. In this volume.
- [3] Sarker MK, Zhou L, Eberhart A, et al. Neuro-symbolic artificial intelligence: Current trends. AI Communications. 2021;To appear.

Contents

Preface: The 3rd AI wave is coming, and it needs a theory <i>Frank van Harmelen</i>	v
Introduction <i>Pascal Hitzler and Md Kamruzzaman Sarker</i>	ix
Chapter 1. Neural-Symbolic Learning and Reasoning: A Survey and Interpretation <i>Tarek R. Besold, Artur d'Avila Garcez, Sebastian Bader, Howard Bowman, Pedro Domingos, Pascal Hitzler, Kai-Uwe Kühnberger, Luis C. Lamb, Priscila Machado Vieira Lima, Leo de Penning, Gadi Pinkas, Hoifung Poon and Gerson Zaverucha</i>	1
Chapter 2. Symbolic Reasoning in Latent Space: Classical Planning as an Example <i>Masataro Asai, Hiroshi Kajino, Alex Fukunaga and Christian Muise</i>	52
Chapter 3. Logic Meets Learning: From Aristotle to Neural Networks <i>Vaishak Belle</i>	78
Chapter 4. Graph Reasoning Networks and Applications <i>Qingxing Cao, Wentao Wan, Xiaodan Liang and Liang Lin</i>	103
Chapter 5. Answering Natural-Language Questions with Neuro-Symbolic Knowledge Bases <i>Haitian Sun, Pat Verga and William W. Cohen</i>	126
Chapter 6. Tractable Boolean and Arithmetic Circuits <i>Adnan Darwiche</i>	146
Chapter 7. Neuro-Symbolic AI = Neural + Logical + Probabilistic AI <i>Robin Manhaeve, Giuseppe Marra, Thomas Demeester, Sebastijan Dumančić, Angelika Kimmig and Luc De Raedt</i>	173
Chapter 8. A Constraint-Based Approach to Learning and Reasoning <i>Michelangelo Diligenti, Francesco Giannini, Marco Gori, Marco Maggini and Giuseppe Marra</i>	192
Chapter 9. Spike-Based Symbolic Computations on Bit Strings and Numbers <i>Ceca Krašniković, Wolfgang Maass and Robert Legenstein</i>	214
Chapter 10. Explainable Neuro-Symbolic Hierarchical Reinforcement Learning <i>Daoming Lyu, Fangkai Yang, Hugh Kwon, Bo Liu, Wen Dong and Levent Yilmaz</i>	235
Chapter 11. Neuro-Symbolic Semantic Reasoning <i>Bassem Makni, Monireh Ebrahimi, Dagmar Gromann and Aaron Eberhart</i>	253
Chapter 12. Learning Reasoning Strategies in End-to-End Differentiable Proving <i>Pasquale Minervini, Sebastian Riedel, Pontus Stenetorp, Edward Grefenstette and Tim Rocktäschel</i>	280

Chapter 13. Generalizable Neuro-Symbolic Systems for Commonsense Question Answering	294
<i>Alessandro Oltramari, Jonathan Francis, Filip Ilievski, Kaixin Ma and Roshanak Mirzaee</i>	
Chapter 14. Combining Probabilistic Logic and Deep Learning for Self-Supervised Learning	311
<i>Hoifung Poon, Hai Wang and Hunter Lang</i>	
Chapter 15. Human-Centered Concept Explanations for Neural Networks	337
<i>Chih-Kuan Yeh, Been Kim and Pradeep Ravikumar</i>	
Chapter 16. Abductive Learning	353
<i>Zhi-Hua Zhou and Yu-Xuan Huang</i>	
Chapter 17. Logic Tensor Networks: Theory and Applications	370
<i>Luciano Serafini, Artur d'Avila Garcez, Samy Badreddine, Ivan Donadello, Michael Spranger and Federico Bianchi</i>	
Author Index	395

Chapter 1

Neural-Symbolic Learning and Reasoning: A Survey and Interpretation

Tarek R. Besold, Philosophy & Ethics, Faculty of IE/IS, Eindhoven University of Technology

Artur d'Avila Garcez, Department of Computer Science, City, University London

Sebastian Bader, Department of Computer Science, University of Rostock

Howard Bowman, School of Computing, University of Kent

Pedro Domingos, Department of Computer Science & Engineering, University of Washington

Pascal Hitzler, Center for Artificial Intelligence and Data Science, Kansas State University

Kai-Uwe Kühnberger, University of Osnabrück

Luis C. Lamb, Universidade Federal do Rio Grande do Sul

Priscila Machado Vieira Lima, PESC/COPPE & NCE, Universidade Federal do Rio de Janeiro

Leo de Penning, Illuminoo B.V.

Gadi Pinkas, Afeka College of Engineering and Afeka Center for Language Processing, Tel-Aviv, Israel

Hoifung Poon, Microsoft Research

Gerson Zaverucha, COPPE, Universidade Federal do Rio de Janeiro

Note: This chapter was originally distributed via arXiv¹ on 10th of November 2017. We have left it mostly untouched, indeed we only made minor corrections and updated the references that were previously cited as pre-prints. As such, the chapter provides a

¹arXiv:1711.03902v1

historic perspective, pre-2017, which complements the other chapters in this volume that primarily focus on the time period past 2017.

Abstract. The study and understanding of human behaviour is relevant to computer science, artificial intelligence, neural computation, cognitive science, philosophy, psychology, and several other areas. Presupposing cognition as basis of behaviour, among the most prominent tools in the modelling of behaviour are computational-logic systems, connectionist models of cognition, and models of uncertainty. Recent studies in cognitive science, artificial intelligence, and psychology have produced a number of cognitive models of reasoning, learning, and language that are underpinned by computation. In addition, efforts in computer science research have led to the development of cognitive computational systems integrating machine learning and automated reasoning. Such systems have shown promise in a range of applications, including computational biology, fault diagnosis, training and assessment in simulators, and software verification. This joint survey reviews the personal ideas and views of several researchers on neural-symbolic learning and reasoning. The article is organised in three parts: Firstly, we frame the scope and goals of neural-symbolic computation and have a look at the theoretical foundations. We then proceed to describe the realisations of neural-symbolic computation, systems, and applications. Finally we present the challenges facing the area and avenues for further research.

1.1. Overview

The study of human behaviour is an important part of computer science, artificial intelligence (AI), neural computation, cognitive science, philosophy, psychology, and other areas. Presupposing that behaviour is generally determined and guided by cognition and mental processing, among the most prominent tools in the modelling of behaviour are computational-logic systems mostly addressing high-level reasoning and thought processes (classical logic, nonmonotonic logic, modal and temporal logic), connectionist models of cognition and the brain mostly addressing lower-level dynamics and emergent processes (feedforward and recurrent networks, symmetric and deep networks, self-organising networks), and models of uncertainty addressing the often vague or probabilistic nature of many aspects of cognitive processing (Bayesian networks, Markov decision processes, Markov logic networks, probabilistic inductive logic programs).

Recent studies in cognitive science, artificial intelligence, and psychology have produced a number of cognitive models of reasoning, learning, and language that are underpinned by computation [1, 2, 3]. In addition, recent efforts in computer science have led to the development of cognitive computational systems integrating machine learning and automated reasoning [4, 5, 6]. Such systems have shown promise in a range of applications, including fault diagnosis, computational biology, training and assessment in simulators, and software verification [7, 8].

Forestalling the presentation of the theoretical foundations in Section 1.2, the intuition motivating neural-symbolic integration as an active field of research is the following: In neural computing, it is assumed that the mind is an emergent property of the brain, and that computational cognitive modelling can lead to valid theories of cogni-

tion and offer an understanding of certain cognitive processes [3]. From this it is in turn assumed that connectionism should be able to offer an appropriate representational language for artificial intelligence as well. In particular, a connectionist computational theory of the mind should be able to replicate the parallelism and kinds of adaptive learning processes seen in neural networks, which are generally accepted as responsible for the necessary robustness and ultimate effectiveness of the system in dealing with common-sense knowledge. As a result, a purely symbolic approach would not be sufficient, as argued by Valiant in [9].

On the other hand, logic is firmly established as a fundamental tool in the modelling of thought and behaviour [10, 11] and by many has been viewed generally as the “calculus of computer science”. In this context, often also nonclassical logics play an important role: Temporal logic, for instance, has had significant impact in both academia and industry [12], and different modal logics have become a lingua franca for, among others, the specification and analysis of knowledge and communication in multi-agent and distributed systems [13]. Research on practical reasoning in AI has been dominated by nonmonotonic formalisms. Intuitionistic logic can provide an adequate logical foundation for several core areas of theoretical computer science, including type theory and functional programming [14]. Finally, description logics—which are similar to Kripke models—have been instrumental in the study of the semantic web [15].

However, when building models that combine learning and reasoning, one has to conciliate the methodologies of distinct areas—namely predominantly statistics and logic—in order to combine the respective advantages and circumvent the shortcomings and limitations. For instance, the methodology of neural-symbolic systems aims to transfer principles and mechanisms between (often nonclassical) logic-based computation and neural computation. In particular, it considers how principles of symbolic computation can be implemented by connectionist mechanisms and how subsymbolic computation can be described and analysed in logical terms. Here, connectionism provides the hardware upon which different levels of abstraction can be built according to the needs of the application. This methodology—looking at principles, mechanisms, and applications—has proven a fruitful way of progressing the research in the area of neural-symbolic integration for more than two decades now as evidenced by, for instance, the results summarised in the overview by [16], collected in the books by [17] and [5], and reported in the present survey.

For example in [5], the described approach has led to a prototypical connectionist system for nonclassical reasoning in an attempt to find an adequate balance between complexity and expressiveness. In this framework—known as a neural-symbolic system—artificial neural networks (ANNs) provide the machinery for parallel computation and robust learning, while logic provides the necessary explanation for the network models, facilitating the necessary interaction with the world and other systems. In the integrated model, no conflict arises between a continuous and a discrete component of the system. Instead, a tightly-coupled hybrid system exists that is continuous by nature (the ANN), but that has a clear discrete interpretation (its logic) at various levels of abstraction.

From a more practical perspective, rational agents are often conceptualised as performing concept acquisition (generally unsupervised and statistical) and concept manipulation (generally supervised and symbolic) as part of a permanent cycle of perception and action. The question of how to reconcile the statistical nature of learning with the logical nature of reasoning, aiming to build such robust computational models integrat-

ing concept acquisition and manipulation, has been identified as a key research challenge and fundamental problem in computer science [18]. Against this backdrop we see neural-symbolic integration as a way of addressing stated challenge through the mechanisms of knowledge translation and knowledge extraction between symbolic logic systems and subsymbolic networks.

There are also important applications of neural-symbolic integration with high relevance for industry applications. The merging of theory (known as background knowledge in machine learning) and data learning (i.e. learning from examples) in ANNs has been shown more effective than purely symbolic or purely connectionist systems, especially in the case of real-world, noisy, unstructured data [7, 8, 19]. Here, successfully addressed application scenarios include business process modelling, service-oriented computing (trust management and fraud prevention in e-commerce), synchronisation and co-ordination in large multi-agent systems, and multimodal processing and integration.

In multimodal processing, for example, there are several forms of reasoning: a scene classification can be achieved by the well-trained network giving an immediate answer following a number of assumptions. A change in the scene, however, may require more specific temporal, nonmonotonic reasoning and learning from data (based, for example, on the amount of change in the scene). Some assumptions may need to be revised, information from an image annotation may provide a different context, abduction and similarity reasoning by intersecting network ensembles may be needed, probability distributions may have to be reasoned about, and so on. The integrated system will need to respond quickly, revise its answers in the presence of new information, and control the inevitable accumulation of errors derived from real-world data (i.e. prove its robustness). This provides an excellent opportunity for the application of neural-symbolic systems.

The remainder of the paper is structured as follows. In Section 1.2, we describe the principles of neural-symbolic computation, revisit the theoretical underpinnings of the endeavour, and give a prototypical example of how one can combine learning and reasoning in an integrated fashion. In Section 1.3 we illustrate the application of the methodology using NSCA, a neural-symbolic agent endowed with learning and reasoning capabilities, as a first detailed example. Section 1.4 relates concepts underpinning the theories of mind in psychology and cognitive science and their counterparts in neural-symbolic computation, before Section 1.5 outlines work addressing the binding problem (introduced in the preceding section), thus enabling first-order inference computed by neural-symbolic systems. Section 1.6 then highlights the more technical foundations of first-order (predicate) logic learning in connectionist systems, followed in Section 1.7 by an introduction to Markov logic and corresponding networks as a combination between logic and graphical models. This leads into the conceptual considerations given in Section 1.8 which relate neural-symbolic computation to recent developments in AI and the reinvigorated interest in (re-)creating human-level capacities with artificial systems, notably recently for instance in the area of language modelling and processing. Finally, Section 1.9 summarises selected currently ongoing and widely recognised approaches to solving core questions of neural-symbolic integration arising from neighbouring research efforts and disciplines, before Sections 1.10 and 1.11 present suggested directions for further research and conclude the survey.

1.2. Prolegomena of Neural-Symbolic Computation

The goals of neural-symbolic computation are to provide a coherent, unifying view for logic and connectionism, to contribute to the modelling and understanding of cognition and, thereby, behaviour, and to produce better computational tools for integrated machine learning and reasoning. To this end, logic and network models are studied together as integrated models of computation. Typically, translation algorithms from a symbolic to a connectionist representation and vice-versa are employed to provide either (i) a neural implementation of a logic, (ii) a logical characterisation of a neural system, or (iii) a hybrid learning system that brings together features from connectionism and symbolic artificial intelligence.

From a theoretical perspective, these efforts appear well-founded. According to our current knowledge and understanding, both symbolic/cognitive and sub-symbolic/neural models—especially when focusing on physically-realizable and implementable systems (i.e. physical finite state machines) rather than strictly abstract models of computation, together with the resulting physical and conceptual limitations—seem formally equivalent in a very basic sense: notwithstanding partially differing theoretical arguments such as given by [20], both paradigms are considered in practice equivalent concerning computability [21]. Also from a tractability perspective, for instance in [22], equivalence in practice with respect to classical dimensions of analysis (i.e. interchangeability except for a polynomial overhead) has been established, complementing and supporting the prior theoretical suggestion of equivalence in the widely accepted Invariance Thesis of [23]. Finally, [24] provided an *in principle* existence result, showing that there is no substantial difference in representational or problem-solving power between dynamical systems with distributed representations and symbolic systems with non-monotonic reasoning capabilities.

But while these findings provide a solid foundation for attempts at closing the gap between connectionism and logic, many questions nonetheless remain unanswered especially when crossing over from the realm of theoretical research to implementation and application, among others switching from compositional symbols denoting an idealised reality to virtually real-valued vectors obtained from sensors in the real world: Although introducing basic connections and mutual dependencies between the symbolic and the subsymbolic paradigm, the levels of analysis are quite coarse and almost all results are only existential in character. For instance, while establishing the *in principle* equivalence described above, [24] does not provide constructive methods for how to actually obtain the corresponding symbolic counterpart to a sub-symbolic model and vice versa.

Still, over the last decades several attempts have been made at developing a general neural-symbolic framework, usually trying to apply the most popular methods of their respective time—such as currently modular deep networks. Growing attention has been given recently to deep networks where it is hoped that high-level abstract representations will emerge from low-level unprocessed data [25]. Most modern neural-symbolic systems use feedforward and recurrent networks, but seminal work in the area used symmetric networks [26] of the kind applied in deep learning, and recent work starts to address real applications of symmetric neural-symbolic networks [7]. There, in general each level of a neural-symbolic system represents the knowledge evolution of multiple agents over time. Each agent is represented by a network in this level encoding commonsense (non-monotonic) knowledge and preferences. The networks/agents at different levels can be

combined upwards to represent relational knowledge and downwards to create specialisations, following what is known as a network-fibring methodology [27].

Fibring—which will continue to serve as general example throughout the remainder of this section—offers a principled way of combining networks and can be seen as one of the general methodologies of neural-symbolic integration. The main idea of network fibring is simple: fibred networks may be composed of interconnected neurons, as usual, but also of other networks, forming a recursive structure. A fibring function defines how this network architecture behaves; it defines how the networks should relate to each other. Typically, the fibring function will allow the activation of neurons in one network *A* to influence the change of weights in another network *B*. Intuitively, this may be seen as training network *B* at the same time that network *A* is running. Albeit being a combination of simple and standard ANNs, fibred networks can approximate any polynomial function in an unbounded domain, thus being more expressive than standard feedforward networks.

Fibring is just one example of how principles from symbolic computation (in this case, recursion), can be used by connectionism to advance the research in this area. In the remainder of this section, we discuss in more detail—also in way of a manifesto summarising our view(s) on neural-symbolic integration—the principles, mechanisms and applications that drive the research in neural-symbolic integration.

1.2.1. Principles of Neural-Symbolic Integration

From the beginning of connectionism [28]—arguably the first neural-symbolic system for Boolean logic—most neural-symbolic systems have focused on representing, computing, and learning languages other than classical propositional logic [29, 30, 4, 5, 31, 2], with much effort being devoted to representing fragments of classical first-order logic. In [5], a new approach to knowledge representation and reasoning has been proposed, establishing connectionist nonclassical logic (including connectionist modal, intuitionistic, temporal, nonmonotonic, epistemic and relational logic). More recently, it has been shown that argumentation frameworks, abductive reasoning, and normative multi-agent systems can also be represented by the same network framework. This is encouraging to the extent that a variety of forms of reasoning can be realised by the same, simple network structure that specialises in different ways.

A key characteristic of many neural-symbolic systems is modularity. One way of building neural-symbolic networks is through the careful engineering of network ensembles, where modularity then serves an important role for comprehensibility and maintenance. Each network in the ensemble can be responsible for a specific task or logic, with the overall model being potentially very expressive despite its relatively simple components. Still, although being fairly common, modularity is not a strict necessity: alternatively, as described in Section 1.5, one can start with an unstructured network (i.e. not modular) and let weight changes shape its ability to process symbolic or subsymbolic representations.

Another common organisational property of neural-symbolic networks is—similar to deep networks—their generally hierarchical organisation. The lowest-level network takes raw data as input and produces a model of the dataset. The next-level network takes the first network's output as its input and produces some higher-level representation of the information in the data. The next-level network then further increases the level of

abstraction of the model, and so on, until some high-level representation can be learned. The idea is that such networks might be trained independently, possibly also combining unsupervised and supervised learning at different levels of the hierarchy. The resulting parallel model of computation can be very powerful as it offers the extra expressiveness required by complex applications at comparatively low computational costs [5].

1.2.2. Mechanisms of Neural-Symbolic Integration

We subscribe to the view that representation initially precedes learning. Neural-symbolic networks can represent a range of expressive logics and implement certain important principles of symbolic computation. However, neural-symbolic computation is not just about representation. The mechanisms of propagation of activation and other message passing methods, gradient-descent and other learning algorithms, reasoning about uncertainty, massive parallelism, fault tolerance, etc. are a crucial part of neural-symbolic integration. Put simply, neural-symbolic networks are efficient computational models, not representational tools. It is the mechanisms in place in the form of efficient algorithms that enable the computational feasibility of neural-symbolic systems.

Returning to the example of fibring, also fibred networks are computational models, not just graphical models or mathematical abstractions like graphs or networks generally. The neural-symbolic networks can be mapped directly onto hardware which promises, for instance, an implementation in a Very-large-scale integration (VLSI) chip to be straightforward and cost effective. The main architectural constraint, which here is brain-inspired, is that neural-symbolic systems should replicate and specialise simple neuronal structures to which a single algorithm can be applied efficiently at different levels of abstraction, with the resulting system being capable of exhibiting emergent behaviour.

It is precisely this emergence as common characterising feature of connectionist approaches which prompts the need for mechanisms of knowledge extraction. The cycle of neural-symbolic integration therefore includes (i) translation of symbolic (background) knowledge into the network, (ii) learning of additional knowledge from examples (and generalisation) by the network, (iii) executing the network (i.e. reasoning), and (iv) symbolic knowledge extraction from the network. Extraction provides explanation, and facilitates maintenance and incremental or transfer learning.

In a general neural-symbolic system, a network ensemble A (representing, for example, a temporal theory) can be combined with another network ensemble B (representing, for example, an agent's epistemic state). Again using fibring as exemplary mechanism, meta-level knowledge in one network can be integrated with object-level knowledge in another network. For example, one may reason (in the meta-level) about the actions that are needed to be taken (in the object-level) to solve inconsistencies in a database. Relational knowledge can also be represented in the same way, with relations between concepts encoded in distinct (object-level) networks potentially being represented and learned through a meta-level network. More concretely, if two networks denote concepts $P(X, Y)$ and $Q(Z)$ containing variables X , Y and Z , respectively, a meta-level network can be used to map a representation of P and Q onto a new concept, say $R(X, Y, Z)$, such that, for example, the relation $P(X, Y) \wedge Q(Z) \rightarrow R(X, Y, Z)$ is valid [5].

Figure 1 illustrates a prototypical neural-symbolic system. The model, in its most general form, allows a number of network ensembles to be combined at different levels of abstraction through, for instance, fibring. In the figure, each level is represented by

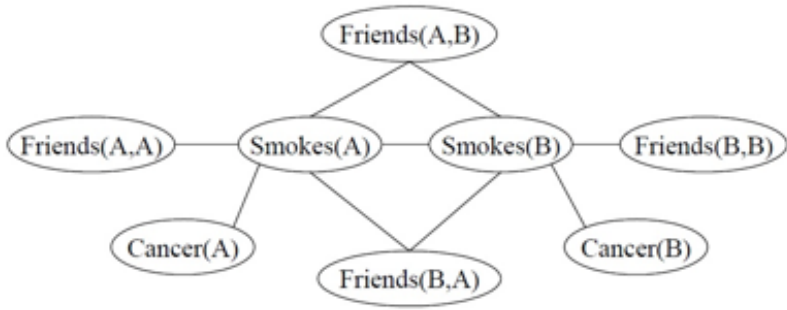


Figure 1. General conceptual overview of a neural-symbolic system [5].

a network ensemble in a horizontal plane, while network fibring takes place vertically among networks at different ensembles. Specialisation occurs downwards when a neuron is fibred onto a network. Relational knowledge is represented upwards when multiple networks are combined onto a meta-level network. Knowledge evolution through time occurs at each level, as do alternative outcomes, and nonmonotonic and epistemic reasoning for multiple, interacting agents. Modular learning takes place inside each network, but is also applicable across multiple networks in the ensemble. The same brain-inspired structure is replicated throughout the model so that a single algorithm is applied at each level and across levels.

1.2.3. Applications of Neural-Symbolic Integration

As so often, it will ultimately be through successful implementations and real-world applications that the usefulness and importance of neural-symbolic integration will be widely acknowledged. Practical applications are a crucial ingredient and have been a permanent feature of the research on neural-symbolic computation. On the theory side—in parallel to the quest for understanding the interplay and integration between connectionism and symbolism as the basis of cognition and intelligence—we are interested in finding the limits of representation and developing better machine learning methods which may open up new applications for consideration. In practice, already at the current state of the research, real applications are possible in areas with societal relevance and/or potentially high economic impact, like bioinformatics, semantic web, fault diagnosis, robotics, software systems verification, business process modelling, fraud prevention, multimodal processing, noisy text analytics, and training and assessment in simulators.

All these different application areas have something in common: a computational system is required that is capable of learning from experience and which may reason about what has been learned [29, 18]. For this learning-reasoning process to be successful, the system must be robust (in a way so that the accumulation of errors resulting from the intrinsic uncertainty associated with the problem domain can be controlled [18]). The method used by the research on neural-symbolic integration to enable some of the above applications has been (i) to apply translation algorithms between logic and networks (making use of the associated equivalence proofs), (ii) to study the systems empirically through case studies (following the practical motivation from statistical machine learning and neural computation), and (iii) to focus on the needs of the application

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

(noting that some potentially interesting applications require just even rudimentary logical representations, e.g. [32]). An example of a neural-symbolic system that is already providing a contribution of this type to problems in bioinformatics and fault diagnosis is the Connectionist Inductive Learning and Logic Programming (CILP) System [33, 4]. Similarly, the NSCA system discussed in Section 1.3 has successfully been applied, for instance, to behaviour modelling in training simulators.

1.2.4. Neural-Symbolic Integration in a Nutshell

In summary, neural-symbolic computation encompasses the high-level integration of cognitive abilities (including induction, deduction, and abduction) and the study of how brains make mental models [34], among others also covering the modelling of emotions and attention/utility. At the computational level, it addresses the study of the integration of logic, probabilities, and learning, and the development of new models of computation combining robust learning and efficient reasoning. With regard to applications, successes have been achieved in various domains, including simulation, bioinformatics, fault diagnosis, software engineering, model checking, visual information processing, and fraud prevention.

In the following section, in order to further familiarise the reader with the overall mindset we will have a detailed look at one such system—namely a neural-symbolic cognitive agent architecture called NSCA—aligning the presentation with the three categories principles, methods, and applications.

1.3. NSCA as Application Example for Neural-Symbolic Computing

It is commonly agreed that an effective integration of automated learning and cognitive reasoning in real-world applications is a difficult task [18]. Usually, most applications deal with large amounts of data observed in the real-world containing errors, missing values, and inconsistencies. Even in controlled environments, like training simulators, integrated learning and reasoning is not very successful [35, 36]. Although the use of simulated environments simplifies the data and knowledge acquisition, it is still very difficult to construct a cognitive model of an (intelligent) agent that is able to deal with the many complex relations in the observed data. For example, when it comes to the assessment and training of high-level complex cognitive abilities (e.g. leadership, tactical manoeuvring, safe driving, etc.) training is still guided or done by human experts [37]. The reason is that expert behaviour on high-level cognition is too complex to model, elicit, and represent in an automated system. Among others, there can be many temporal relations between low- and high-order aspects of a training task, human behaviour is often non-deterministic and subjective (i.e. biased by personal experience and other factors like stress or fatigue), and what is known is often described vaguely and limited to explicit (i.e. “explainable”) behaviour.

Several attempts have been made to tackle these problems. For instance [38] describes a number of systems that use machine learning to learn the complex relations from observations of experts and trainees during task execution. Although these systems are successful at learning and generalisation, they lack the expressive power of symbolic systems and are therefore difficult to interpret and validate [39]. Alternatively, one could

add probabilistic reasoning to logic-based systems [36]. These systems perform better in expressing their internal knowledge as they use explicit symbolic representations and are able to deal with many of the most common types of inconsistencies in the data by reasoning with probabilities. Unfortunately, when it comes to knowledge representation and modelling, these systems still require either statistical analyses of large amounts of data or knowledge representation by hand. Therefore, both approaches are time expensive and are not appropriate for use in real-time applications, which demand online learning and reasoning.

1.3.1. Principles of Neural-Symbolic Integration Exemplified

The construction of effective cognitive agent models is a long-standing research endeavour in artificial intelligence, cognitive science, and multi-agent systems [18, 40]. One of the main challenges toward achieving such models is the provision of integrated cognitive abilities, such as learning, reasoning, and knowledge representation. Neural-symbolic systems seek to do so within the neural computation paradigm, integrating inductive learning and deductive reasoning (cf. [5, 41] for examples). In such models, ANNs are used to learn and reason about (an agent's) knowledge about the world, represented by symbolic logic. In order to do so, algorithms map logical theories (or knowledge about the world) T into a network N which computes the logical consequences of T . This provides also a learning system in the network that can be trained by examples using T as background knowledge. In agents endowed with neural computation, induction is typically seen as the process of changing the weights of a network in ways that reflect the statistical properties of a dataset, allowing for generalisations over unseen examples. In the same setting, deduction is the neural computation of output values as a response to input vectors (encoding stimuli from the environment) given a particular set of weights. Such network computations have been shown equivalent to a range of temporal logic formalisms [42]. Based on this approach, an agent architecture called NEURAL SYMBOLIC COGNITIVE AGENT (NSCA) has been proposed in [8]. NSCA uses temporal logic as theory T and a Restricted Boltzmann Machine (RBM) as ANN N . An RBM is a partially connected ANN with two layers, a visible layer V and a hidden layer H , and symmetric connections W between these layers [43].

An RBM defines a probability distribution $P(V = v, H = h)$ over pairs of vectors v and h encoded in these layers, where v encodes the input data in binary or real values and h encodes the posterior probability $P(H|v)$. Such a network can be used to infer or reconstruct complete data vectors based on incomplete or inconsistent input data and therefore implement an auto-associative memory or an autoencoder. It does so by combining the posterior probability distributions generated by each unit in the hidden layer with a conditional probability distribution for each unit in the visible layer. Each hidden unit constrains a different subset of the dimensions in the high-dimensional data presented at the visible layer and is therefore called an expert on some feature in the input data. Together, the hidden units can form a "Product of Experts" model that constrains all the dimensions in the input data.

NSCA is capable of (i) performing learning of complex temporal relations from uncertain observations, (ii) reasoning probabilistically about the knowledge that has been learned, and (iii) representing the agent's knowledge in a logic-based format for validation purposes. This is achieved by taking advantage of neural learning to perform ro-

bust learning and adaptation, and symbolic knowledge to represent qualitative reasoning. NSCA was validated in a training simulator employed in real-world scenarios, illustrating the effective use of the approach. The results show that the agent model is able to learn to perform automated driver assessment from observation of real-time simulation data and assessments by driving instructors, and that this knowledge can be extracted in the form of temporal logic rules [8].

1.3.2. Mechanisms of Neural-Symbolic Integration Exemplified

Using a Recurrent Temporal Restricted Boltzmann Machine (RTRBM) [44] as specialised variant of the general RBM concept, NSCA encodes temporal rules in the form of hypotheses about beliefs and previously-applied rules. This is possible due to recurrent connections between hidden unit activations at time t and the activations at time $t - 1$ in the RTRBM. Based on the Bayesian inference mechanism of the RTRBM, each hidden unit H_j represents a hypothesis about a specific rule R_j that calculates the posterior probability that the rule implies a certain relation in the beliefs b being observed in the visible layer V , given the previously applied rules r^{t-1} (i.e. $P(R|B = b, R^{t-1} = r^{t-1})$). From these hypotheses the RTRBM selects the most applicable rules r using random Gaussian sampling of the posterior probability distribution (i.e. $r \propto P(R|B = b, R^{t-1} = r^{t-1})$) and calculates the conditional probability or likelihood of all beliefs given the selected rules are applied (i.e. $P(B|R = r)$). The difference between the observed and inferred beliefs can be used by NSCA to train the RTRBM (i.e. update its weights) in order to improve the hypotheses about the observed data. For training, the RTRBM uses a combination of Contrastive Divergence and backpropagation through time.

In the spirit of [45]’s Belief, Desire, Intention (BDI) agents, the observed data (e.g. simulation data or human assessments) are encoded as beliefs and the difference between the observed and inferred beliefs are the actual implications or intentions of the agent on its environment (e.g. adapting the assessment scores in the training simulator). The value of a belief represents either the probability of the occurrence of some event or state in the environment (e.g. *Raining = true*), or a real value (e.g. *Speed = 31.5*). In other words, NSCA deals with both binary and continuous data, for instance, by using a continuous stochastic visible layer [46]. This improves the agent’s ability to model asymmetric data, which in turn is very useful since measured data coming from a simulator is often asymmetric (e.g. training tasks typically take place in a restricted region of the simulated world).

Due to the stochastic nature of the sigmoid activation functions, the beliefs can be regarded as fuzzy sets with a Gaussian membership function. This allows one to represent vague concepts like *fast* and *slow*, as well as approximations of learned values, which is useful when reasoning with implicit and subjective knowledge [47].

The cognitive temporal logic described in [42] is used to represent domain knowledge in terms of beliefs and previously-applied rules. This logic contains several modal operators that extend classical modal logic with a notion of past and future. To express beliefs on continuous variables, this logic is extended with the use of equality and inequality formulae (e.g. *Speed < 30*, *Raining = true*). As an example, consider a task where a trainee drives on an urban road and approaches an intersection. In this scenario the trainee has to apply a *yield-to-the-right* rule. Using the extended temporal logic, one can describe rules about the conditions, scenario, and assessment related to this task. In

rules (1) to (4) in Table 1, $\diamond A$ denotes “A is true sometime in the future” and ASB denotes “A has been true since the occurrence of B”.

Conditions: (1) (*Weather* > *good*)

meaning: the weather is at least good

Scenario: (2) $\text{ApproachingIntersection} \wedge \diamond(\text{ApproachingTraffic} = \text{right})$

meaning: the car is approaching an intersection and sometime in the future traffic is approaching from the right

(3) $((\text{Speed} > 0) \wedge \text{HeadingIntersection})S(\text{DistanceIntersection} < x) \rightarrow \text{ApproachingIntersection}$
 meaning: if the car is moving and heading towards an intersection since it has been deemed close to the intersection, then the car is approaching the intersection.

Assessment: (4) $\text{ApproachingIntersection} \wedge (\text{DistanceIntersection} = 0) \wedge (\text{ApproachingTraffic} = \text{right}) \wedge (\text{Speed} = 0) \rightarrow (\text{Evaluation} = \text{good})$

meaning: if the car is approaching an intersection and arrives at the intersection when traffic is coming from the right and stops then the trainee gets a good evaluation

Table 1. Situations and assessments from a driving simulator scenario.

Rule (4) is an example of an uncertain notion that is highly subjective (the distance x at which a person is regarded as approaching an intersection is dependent on the situation and personal experience). When this rule is encoded in an RTRBM, it becomes possible to learn a more objective value for x based on the observed behaviour of different people in various scenarios. This exemplifies a main objective of combining reasoning and learning.

The temporal logic rules are represented in NSCA by setting the weights of the connections in the RTRBM. Therefore the rules need to be translated to a form that relates only to the immediately previous time step (denoted by the temporal operator $\bullet$). A transformation algorithm for this is also described in [42]. Then we can encode any rule as a stochastic relation between the hidden unit that represents the rule, the visible units that represent the beliefs, and the previous hidden unit activations that represent the applied rules in the previous time step. For example, the rule $\alpha S\beta$ can be translated to the following rules: $\beta \rightarrow \alpha S\beta$ and $\alpha \wedge \bullet(\alpha S\beta) \rightarrow \alpha S\beta$, where α and β are modelled by visible units, $\alpha S\beta$ by a hidden unit, and $\bullet(\alpha S\beta)$ is modelled by a recurrent connection to the same hidden unit. [48] shows how to map these logic-based propositions into the energy function of a symmetric network and how to deal with uncertainty by introducing a notion of confidence, called a penalty, as discussed further below.

1.3.3. Applications of Neural-Symbolic Integration Exemplified

NSCA has been developed as part of a three-year research project on assessment in driving simulators. It is implemented as part of a multi-agent platform for Virtual Instruction [49] and was used for an experiment on a driving simulator. In this experiment five students participated in a driving test consisting of five test scenarios each. For each attempt all data from the simulator (i.e. 43 measurements, like relative positions and orientations of all traffic, speed, gear, and rpm of the student’s car) and numerical assessments scores on several driving skills (i.e. vehicle control, economic driving, traffic flow, social-, and safe driving) that were provided by three driving instructors present during the attempts were observed by NSCA in real-time. NSCA was able to learn from these observations and infer assessment scores that are similar to those of the driving instructors [7, 8].

The NSCA system has also been applied as part of a Visual Intelligence (VI) system, called CORTEX, in DARPA's Mind's Eye program [50]. This program pursues the capacity to learn generally-applicable and generative representations of actions between entities in a scene (e.g. persons, cars, objects) directly from visual inputs (i.e. pixels), and reason about the learned representations. A key distinction between Mind's Eye and the state-of-the-art in VI is that the latter has made large progress in recognising a wide range of entities and their properties (which might be thought of as the nouns in the description of a scene). Mind's Eye seeks to add the ability to describe and reason about the actions (which might be thought of as the verbs in the description of the scenes), thus enabling a more complete narrative of the visual experience. This objective is supported by NSCA as it seeks to learn to describe an action in terms of entities and their properties, providing explanations for the reasoning behind it. Results have shown that the system is able to learn and represent the underlying semantics of the actions from observation and use this for several VI tasks, like recognition, description, anomaly detection, and gap-filling [51].

Most recently, NSCA has been included in an Intelligent Transport System to reduce CO2 emissions [52]. Results show that NSCA is able to recognise various driving styles from real-time in-car sensor data and outperforms state-of-the-art in this area. In addition, NSCA is able to describe these complex driving styles in terms of multi-level temporal logic-based rules for human interpretation and expert validation.

1.3.4. NSCA in a Nutshell

In summary, NSCA is an example for a cognitive model and agent architecture that offers an effective approach integrating symbolic reasoning and neural learning in a unified model. This approach allows the agent to learn rules about observed data in complex, real-world environments (e.g. expert behaviour for training and assessment in simulators). Learned behaviour can be extracted to update existing domain knowledge for validation, reporting, and feedback. Furthermore the approach allows domain knowledge to be encoded in the model and deals with uncertainty in real-world data. Results described in [7, 8] show that the agent is able to learn new hypotheses from observations and extract them into a temporal logic formula. But although results are promising, the model requires further evaluation by driving experts. This will allow further validation of the model in an operational setting with many scenarios, a large trainee population, and multiple assessments by driving instructors. Other on-going work includes research on using Deep Boltzmann Machines [53, 54] to find higher-level rules and the application of an RTRBM to facilitate adaptive training. Overall, this work illustrates an application model for knowledge representation, learning, and reasoning which may indeed lead to realistic computational cognitive agent models, thus addressing the challenges put forward in [18, 40].

1.4. Neural-Symbolic Integration in and for Cognitive Science: Building Mental Models

Thus far, we elaborated on the general theoretical and conceptual foundations of neural-symbolic integration and discussed NSCA as example of a successful implementation

and application deployment of a neural-symbolic system. In what follows, we now return in more detail to some of the most pressing theoretical and applied core questions—including the binding problem, connectionist first-order logic (FOL) learning, the combination of probabilities and logic in Markov Logic Networks, and the relationship between neural-symbolic reasoning and human-level AI—and describe our current state of knowledge, as well as potential future developments, in the following sections. We start with an introduction to the intimate relation between neural-symbolic integration and core topics from cognitive science.

To fully explain human cognition, we need to understand how brains construct and manipulate mental models. Clarification of this issue would inform the development of (artificially) intelligent systems and particularly neural-symbolic systems. The notion of mental model has a long history, perhaps most notably being used by Johnson-Laird to designate his theory of human reasoning [55]. Taking partial inspiration from [56] and [57], we interpret a mental model as a cognitive representation of a real or imagined situation, the relationship amongst the situation's parts and, even perhaps, how those parts can act upon one another. These models will necessarily preserve the constraints inherent in what is represented, and their evaluation and manipulation has been argued to be key to human reasoning [55]. For example, sight of an empty driveway and a scattering of broken glass, might conjure a mental representation (i.e. a model) of a felon breaking the window of your car, leading you to reason that your car had been stolen. As a more abstract example, when pondering on why the Exclusive Or problem is linearly inseparable in the context of network learning, one might visualise a two dimensional plane containing four regularly-placed crosses (two for true, two for false) and mentally consider placement of classification boundary lines. By exploring such a mental model, one might convince oneself that a line partitioning true and false outputs cannot be found.

It has particularly been argued that construction and inference on mental models might play a major role in abductive reasoning and, even, creativity [34], our car theft example being a case of the former. Exactly how mental models and their manipulation are neurally realised, though, remains largely uncertain. Central to construction of a mental model is the formation of a combined (superordinate) whole representation from sets of (subordinate) part representations. For example, in the car theft example, a composite of your car and a hooded figure breaking a window might be created from memory and imagination of car and figure in isolation.

The most basic requirement of a neural realisation of such representation combination is a solution to the so called “binding problem”, i.e. identification of a general neural mechanism to represent which individual assemblies of active neurons (e.g. those representing red Ford and those representing hooded figure) are associated and which are not (also see Section 1.5.2). Many proposals have been made for the brain's binding mechanism, e.g. temporal synchrony [58], conjunctive codes [59, 60, 61] and even convolution-based approaches [34]. Of these, conjunctive codes seem to involve the smallest step from traditional rate-coded ANNs, e.g. spiking and oscillatory dynamics are not assumed, and would thus seem most naturally integrable with neural-symbolic systems as currently formulated.

Under the conjunctive codes approach, it is assumed that units that respond selectively to the coactivation of multiple item representations are available. Such conjunctive units might reside in a widely accessible binding resource, such as, say, the Binding Pool

in [60]. Two challenges to conjunctive code binding approaches (and, by extension, to neural representation of mental models) are, (i) scalability, and (ii) novel conjunctions:

- A binding resource that exhaustively enumerates all possible combinations of representations as unique (localist) units does not scale. Thus, realistic conjunctive codes typically assume that the binding resource employs distributed representations, which provide more compact, and indeed scalable, codings of binding associations, see for example, the explorations of this issue in [61] and [62].
- We often experience representation combinations that we have never experienced before. A standard example is the proverbial blue banana or, in the context of the car theft example, it is likely that a mental model conjoining a hooded figure, a red Ford, and a particular driveway would never have been previously experienced. (Indeed, the fact that every experience must—necessarily—have a first occurrence makes the point.) As a result, conjunctive coding approaches propose randomly preconfigured binding resources with such discrimination capacity that they can effectively conjoin any combination of item representations that might arise in the future; the model of Frontal lobe function in [61] is such an approach.

Neural conjunctive coding methods of this kind would, then, seem prerequisite to brain-based theories of mental models.

Clearly, a capacity to construct mental models is of limited value unless it helps us to make decisions and, in general, to reason, e.g. to determine that a crime has been committed and that the Police should be called. Two ways in which modern cognitive neuroscience suggests such reasoning could arise is through either statistical inference, or deliberative exploration. The former of these may employ statistical inference based upon pattern matching, in the manner of classic ANNs, as reflected in use of the descriptor “associative learning” in this context. In contrast, the latter would be more explicitly deliberative in nature and, accordingly, would involve the serial exploration of a space of hypothetical possibilities. Such a process might most naturally be framed in terms of classic symbolic computation and, perhaps, search in production system architectures [63]. In addition, the statistical inference process might be viewed as more opaque to conscious awareness than deliberation.

Dual process theories of reasoning, e.g. [64], have argued that a subdivision of this kind is fundamental to human reasoning. With regard to our main interest here, though, the second of these forms of reasoning, deliberative exploration, seems most naturally to embrace the notion of mental model, with its connotation of forming conscious hypothetical representations.

Modern cognitive neuroscience also emphasises the role of emotion/body-state evaluations in reasoning. In particular, emotion may be viewed as providing an assessment of the evaluative quality of a situation or alternative, much in the manner of [65]’s somatic markers. But importantly, theories of affect, such as, say, appraisal theory [66], suggest a multidimensional space of emotionally-charged feelings, richer than that required of simple reward and punishment or utility function. For instance, both disgust and fear are negatively valenced, marking, if you like, punishing experiences; however, the aversive responses to the two are typically quite different. In particular, only fear is likely to engage highly urgent flight or fight responses, associated with immediate physical risk. Indeed, it could be argued that a key function of emotions is to mark out response types and, at least in that sense, the distinctions between emotions on many dimensions

are cognitively significant. This is relevant to cognitive and neuroscientific accounts of neural-symbolic integration in that mental models might, then, be viewed as “stamped” with affective evaluations, which would be bound (in the manner previously discussed) to nonaffective portions of a mental model, yielding a rich affect-coloured high dimensional representation.

1.4.1. Cognitive Neuroscience and Neural-Symbolic Methods

Cognitive neuroscience (Gazzaniga, Ivry et al. 1998) is a major current scientific project, with the explanation of cognitive capacities (such as, perception, memory, attention, and language) in terms of their implementation in the brain as central objective. Computational modelling (especially when ANN-based) is a key ingredient of the research programme. Conceptually, such modelling provides the “glue” between cognition and the brain: it enables concrete (disprovable) explanations to be formulated of how cognitive capacities could be generated by the brain. In this sense, models that explain cognitive behaviour as well as neuroimaging data do a particularly good job of bridging between the cognitive and the neural. Examples of such models include (Bowman 2006; Chennu, Craston et al. 2009; Cowell and Cottrell 2009; Craston, Wyble et al. 2009), and the whole Dynamic Causal Modelling project (Friston, Harrison et al. 2003) can be seen to have this objective.

This then brings to the fore the suitability of different cognitive modelling paradigms in Cognitive Neuroscience; that is, how should computational models in Cognitive Neuroscience be formulated? If we consider the cognitive and brain sciences in very broad definition, although computational models of many varieties have been employed, there are two dominant traditions. The first of these is symbolic modelling, which would most often be formulated in Production System architectures, such as SOAR [63], EPIC [67] and ACT-R [68, 69]. In contrast, the second tradition is network modelling, ranging from abstract connectionist, e.g. [70], to neurophysiologically detailed approaches, e.g. [71, 72]. From the earliest computation-based proposals for the study of mind and brain, which probably date to the 1950s, pre-eminence has oscillated between symbolic and neural network-based. However, modern interest in the brain and mapping cognition to it, has led to a sustained period of neural network pre-eminence and, certainly in the cognitive neuroscience realm, symbolic modelling is now rare; the most prominent exceptions being [69, 73].

There does though remain a minority who dissent from the neural networks emphasis. Some have taken a very strongly anti-neural networks perspective, e.g. [74], arguing that thought is fundamentally symbolic—indeed linguistic, c.f. [75]—and that it should thus necessarily be studied at that level. Computer systems are often taken as a justification for this position. It is noted that software is the interesting level, which governs a computer’s functional behaviour at any moment. In contrast, it is argued that the mapping to hardware implementation, e.g. compilation down to assembly code, is a fixed predefined transformation that does not need to be reprogrammed when the algorithm being run is changed. That is, the algorithm being executed is determined by software, not the mapping to hardware, or to paraphrase, in computer systems, software is where the functional complexity resides. The key point being that software is symbolic. If one views cognition as marked by its functional richness, one might then conclude that cognition is most appropriately studied symbolically and that the mapping to the brain is a

fixed uninteresting implementation step, which does not reflect the richness and variety of cognitive experience and behaviour.

This position does, of course, hinge upon an analogy between the structure of computer systems and the structure of the mind-brain. Many would reject this analogy. Indeed, the majority of those that express dissent from network modelling, take a less extreme line. While acknowledging the relevance of brain implementation, they argue that ANNs are expressively limited as a cognitive modelling method, or—at least—emphasise that ANNs do not lead to natural modelling of certain cognitive functions. The following are cases in point.

(i) Rule-guided problem solving: Duncan has explored the human capacity to encode and apply sequences of rules governing how to perform mental tasks [76]. Such *task rules* state that fulfilment of a set of properties mandates a response of a particular kind. Furthermore, these properties and the relationships between them could be quite complex, e.g. “if all the letters on one side of the screen are in capitals and an arrow points diagonally up to the right on the other side then respond in the direction indicated by the big triangle”. Duncan and co-workers have shown that performance on their task rule experiments is strongly correlated with fluid intelligence and thus, with IQ. Performance is also associated with activation of brain areas believed to be involved in effortful task-governed behaviour [77]. Thus, these task rule experiments seem to be revealing the neurocognitive underpinnings of intelligence and problem solving; a key concern for science. A natural way to think about these task rules is as symbolic operations with logic preconditions and actions that define a particular response. In this context, participants are seeking to correctly evaluate preconditions against presented stimuli and apply the corresponding (correct) action. Clearly, this perspective could be directly reflected in a production system model. Finally, one of the key questions for task rule research is characterising how errors are made. To a large extent, errors seem to arise from preconditions and/or actions migrating between operations (i.e. between rules). Again, such errors could naturally be modelled in a production system formulation in which preconditions and actions could misbind to one another.

(ii) Central Executive Function: The notion of a centralised control system that guides thought (a Central Executive) is common in information processing theories of cognition, c.f. Baddeley and Hitch’s working memory model [78], Shallice’s Supervisory Attentional System [79] and the Central Engine in Barnard’s Interacting Cognitive Subsystems [80]. The Central Executive is, for example, hypothesised to direct sensory, perceptual, and motor systems, particularly overriding prepotent (stereotyped) responses. Accordingly, it would be involved in mapping the organism’s current goals to a strategy to realise those goals (a, so called, task set). In addition, it would impose that task set upon the brain’s processing through centralised control. Indeed, the Central Executive would be involved in encoding and applying task rules of the kind just discussed, and also conscious exploration of sequences of alternatives in search of a strategy to obtain a goal, i.e. what in AI terms would be called planning. Neurophysiological localization of the Central Executive has focused on the Frontal lobe and particularly problem solving deficits arising from frontal lobe damage [79]. A number of the most prominent workers in this field have formulated their models in symbolic production systems, rather than ANNs. One such approach employs the COGENT high-level modelling notation [81], in which components, coded in production systems, execute in parallel, subject to inter-component interaction. In particular, symbolic AI methods seem well suited to modelling

some classic Central Executive (frontal lobe) tasks such as, Towers of Hanoi, Towers of London, and planning in general [81].

(iii) Syntactic structures: A long running debate in psycholinguistics has contrasted rule-based and association-based explanations of language morphology, with Pinker typically carrying the flag for the former and McClelland and co-advocates for the latter [82]. The hottest debate has been focused on a particularly distinctive (U-shaped) profile of developmental data arising when children learn the English past tense inflection; i.e. when they determine that addition of an “ed” suffix enforces a past tense interpretation (e.g. “stay” to “stayed”), subject to many exceptions (e.g. “run” to “ran”). Although focused on one specific aspect of language, this debate serves as a sounding-board for a broader consideration: are the regular syntactic and morphological aspects of language in general, best viewed as symbolic rules or as emergent from the training of a Parallel Distributed Processing (PDP)-style network. This debate is specific to regular aspects of language; there is more agreement that exceptions could be naturally handled by PDP networks. The critical point for the connectionists is that, in the PDP case, there is no sense to which any particular rule or, indeed, an abstract notion of rule, is built into the network a priori; the network, in a sense, “discovers” regularity; indeed, if you like, the concept of regularity. Although the rules vs. associations debate will surely run and run, in terms of our focus here, it demonstrates an—at the least perceived—mismatch between classic connectionist approaches and the characteristics of cognitive behaviour. Furthermore, the proposed response to this mismatch is a symbolic modelling metaphor, which could naturally be realised in classical AI methods, such as, production systems, logic programming, etc.

(iv) Compositionality: One characterisation of the problem connectionism suffers is that it does not naturally reflect the representational compositionality inherent to many cognitive capacities [74]. For example, when parts are combined into wholes, those parts, at least to a large extent, carry their interpretation with them, e.g. “John” in “John loves Jane” has, ostensibly, the same meaning as it does in “Jane loves John”. Although different classes of ANNs adhere to this characteristic to different extents, it certainly seems that classic PDP-style models do not automatically generate representational compositionality. At least when taken on face value, PDP models exhibit position-specific coding schemes; that is, the same item (e.g. “John”) would have to be separately learned in position X and position Y, with no automatic interpretational carry-over from one position to the other. Consequently, the item may have a very different interpretation in the two positions and, indeed, what has been learned about “John” from its appearance at position X (where it might have been seen frequently) would not automatically inform the understanding of “John” at position Y (where it might have been seen rarely, if at all). This does not fit with subjective experience: the first time we see “John” in a particular grammatical position, having seen it elsewhere frequently, does not, it would seem, manifest as a complete unfamiliarity with the concept of “John”. Although not always explicitly framed in terms of compositionality, the intermittent attacks on PDP approaches by proponents of localist neural coding schemes have a similar flavour [83]. Indeed, localists’ criticism of the slot-coding used in PDP models of word reading [84] is very much a critique of position-specific coding.

This dissatisfaction with connectionism suggests a symbolic perspective, since symbolic models are foundationally compositional in the fashion ANNs are argued not to be. If one accepts this weight of argument, one is left with the perspective that while the

link to the brain is certainly critical (indeed, the brain does only have neurons, synapses, etc., to process with), it may not be practical to map directly to neural implementation. Rather, cognitive modelling at an intermediate symbolic level may be more feasible.

In particular, ANNs have certainly made a profound contribution to understanding cognition. This though has been most marked for a specific set of cognitive faculties. For example, compelling models of sensory processes have been developed, e.g. stages of the “what” visual processing pathway (i.e. the ventral stream), c.f. [85, 86, 87]; and face-specific processing within the what pathway, cf. [88, 89]. Mature neural models of visual attention through both space and time have also been proposed, e.g. [90, 91, 92, 60]. There are also sophisticated neural models of memory systems and learning in general, e.g. [93, 94, 95]. Furthermore, many of these models are compelling in their neurophysiological detail.

However, as justified above, there are areas where neural modelling has had less success. In general terms, architectural-level neural models are scarce, i.e. models that are broad-scope, general-purpose, and claim relevance beyond a specific cognitive phenomenon. There are, though, many symbolic architecture-level models of mind, e.g. SOAR [63]—possible reasons for this situation have been discussed, among others, in [96]. It may also be the case that neural explanations become less compelling as cognitive functions become more high-level; that is, they are well suited to characterising “peripheral” systems, such as, vision, audition, and motor action, but they do less well as one steps up the processing hierarchy away from input and output. In particular, what Sloman would call deliberative processing [97], which might embrace Duncan’s task rule experiments, planning and, at least elements of, Central Executive function, are less naturally modelled with ANNs. To emphasise again, we are not saying that such cognitive capacities cannot in some fundamental sense be neurally modelled, but rather we are suggesting it might not be the most natural method of description.

As already suggested, a possible response to this situation is to consider whether mapping to a neural substrate could be expedited by breaking the modelling problem into two steps: the first a symbolic modelling of cognitive behaviour and the second a mapping from symbolic to neural. From a philosophical perspective, one would be arguing that the symbolic intermediary more directly reflects the functional characteristics of the cognitive capacity in question, leading the modeller to a, somehow, more faithful model at the cognitive level. Furthermore, the suggestion would be that such a functional characterisation would also strongly constrain the neural implementation and vice-versa, via a direct mapping to ANNs, i.e., effectively a compilation step (which may nevertheless be necessary computationally for the sake of efficiency).

The first, cognitive behaviour to symbolic model, step might very naturally yield some form of computational logic model, perhaps formulated in a production systems architecture, such as SOAR [63], ACT-R [68] or EPIC [67], or as a logic program. Either way, the second mapping would involve some form of computational logic to ANN transformation—exactly the topic of the neural-symbolic project. Thus, in this respect, application of the results of neural-symbolic research could make a significant contribution to Cognitive Neuroscience.

However, such an application raises a number of research topics. Perhaps the most significant of these is to develop mappings from computational logic to neurophysiologically plausible neural networks. This requires consideration of more complex activation equations that directly model ion channels and membrane potential dynamics (e.g.

based upon Hodgkin-Huxley equations [98]); it also suggests approaches that treat excitation and inhibition distinctly and employ more biologically-realistic learning, such as contrastive Hebbian learning, as arises in, say, O'Reilly and Munakata's Generalised Recirculation Algorithm. A good source for such neural theories is [99].

1.5. Putting the Machinery to Work: Binding and First-Order Inference in a Neural-Symbolic Framework

While it should have become clear that it seems plausible to assume that human cognition produces and processes complex combinatorial structures using neural networks, on the computer science and systems engineering side such systematic and dynamic creation of these structures presents challenges to ANNs and for theories of neuro-cognition in general [74, 100, 101, 102]. We now will zoom in on concrete work relating to the problem of representing such complex combinatorial structures in ANNs—with emphasis on FOL representations—while efficiently computationally learning and processing them in order to perform high-level cognitive tasks. Similar to the presentation of NSCA in Section 1.3 as application example for the theoretical and conceptual considerations from Section 1.2, the following shall—besides presenting a relevant body of work from neural-symbolic integration—provide a practical counterpart and grounding for some of the cognition and cognitive neuroscience-centred discussion of the previous Section 1.4.

Specifically, this section describes how predicate logic can efficiently be represented and processed in a type of recurrent network with symmetric weights. The approach is based on a (variable) binding mechanism which encodes compactly first-order logic expressions in activation. Processing is then done by encoding symbolic constraints in the weights of the network, thus forcing a solution to an inference problem to emerge in activation.

1.5.1. A Computational View on Fodor, Pylyshyn, and the Challenge(s) for Modelling Cognition

One of the computationally most influential and relevant positions relating cognition and neural-symbolic computation has been developed by [74]. There, two characteristics are deemed essential to any paradigm that aims at modelling cognition: (i) combinatorial syntax and semantics for mental representations and (ii) structure sensitivity of processes. The first characteristic allows for representations to be recursively built from atomic ones in a way that the semantics of a non-atomic structure is a function of the semantics of its syntactic components. The second characteristic refers to the ability to base the application of operations on a structured representation, on its syntactic structure.

Several works partially answered their call, yet none could capture the full expressive power of FOL: Even with limited expressiveness, most attempts to address Fodor's and Pylyshyn's challenges are extremely localist, have no robustness to neural damage, have limited learning ability, have large size-complexity, and require ad hoc network engineering rather than using a general purpose mechanism which would be independent of the respective knowledge base (KB) [103, 104, 5, 105, 106, 107, 101]. Additionally, only few systems used distributed representations—and even those which do typically suffer from information loss, and little processing abilities [108, 109, 110].

1.5.2. The Binding Problem Computationally Revisited

Despite the large body of work (non-exhaustively) summarised in Section 1.4 and the corresponding many attempts to approach Fodor’s and Pylyshyn’s criticism, computational neural modelling of high-level cognitive tasks and especially language processing is still considered a hard challenge. Many of the problems in modelling are taken to very likely be related to the already previously described “binding problem”. Following the definition by [102], in short the general binding problem concerns how items, which are encoded in distinct circuits of a massively parallel computing device, can be combined in complex ways, for various cognitive tasks. In particular, binding becomes extremely challenging in language processing and in abstract reasoning [111, 112, 100, 101], where one must deal with a more demanding, special case: variable binding. The underlying question is narrowed down to: How are simple constituents glued to variables and then used by several and distinct parts of a massively parallel system, in different roles? Following [112], the binding problem can be further decomposed into four constitutive challenges: (i) the massiveness of the binding problem, (ii) the problem of multiple instances of a fact, (iii) the problem of associating values to variables and (iv) the relation between bindings in short-term working memory (WM) as opposed to bindings in long-term memory (LTM). When jointly looking at all four, the binding mechanism seems to be the fundamental obstacle as each individual item depends on it. But even if a binding mechanism should exist and enables the representation of complex structures in neuron-like units, it is not clear how they are processed in order to achieve a goal and how the procedural knowledge needed for such processing is encapsulated in the synapses. Specifically, in the computational case, when FOL expressions shall be processed using just neural units with binary values, how should the respective formulae be encoded? How are neural-ensembles, representing simple constituents, glued together to form complex structures? And how are these structures manipulated for unification and ultimately for reasoning?

As a first step, a binding mechanism is required that will allow to “glue” together items in a KB, such as predicates, functions, constants, and variables, and then apply it in manipulating these structures. Correspondingly, several attempts have been made to tackle the variable binding problem [103, 111, 113, 114, 101]. But while the respective approaches have greatly contributed to our understanding of possible mechanisms at work, it turned out that they each still have significant shortcomings related to limited expressiveness, high space requirements, or central control demands.

1.5.3. Inference Specifications as Fixed Points of ANNs

Instead of developing a bottom-up, modular solution to performing reasoning with ANNs, let us examine the opposite approach: specifying what should be the possible results of a reasoning process as fixed points (or stable states) of an ANN. In every ANN that reaches equilibrium, converging onto a stable state (instead of oscillating), the fixed points of the network—given clamped inputs—can be associated with solutions for the problem at hand. In particular, ANNs with a symmetric matrix of weights such as, for instance, the already discussed Boltzmann machines from Section 1.3, perform gradient descent in an energy function, whose global minima can be associated with solutions to the problem. Although symmetric (energy minimisation) networks are the straight-

forward architecture choice, other (non-symmetric) architectures may be used as long as stable states emerge and those stable states correspond to solutions of the problem at hand. When using symmetric networks, the general strategy is to associate the problem-solutions with the global minima of the energy equation that corresponds to the network. In particular, as the “problem” is FOL inference, valid inference chains are the solutions, which are mapped into the global minima of the energy function.

It has been shown that high-order symmetric ANNs (with multiplicative synapses) are equivalent to “standard” symmetric ANNs (with pairwise synapses) with hidden units [26]. In the corresponding simulations, high-order variants of Boltzmann Machines are therefore used, as they are faster to simulate and have a smaller search space than the “standard” Boltzmann machines with hidden units and only pair-wise connections. The declarative specifications (constraints) characterising valid solutions are compiled (or learned) into the weights of a higher-order Boltzmann machine. The first-order KB is either compiled into weights, or it is clamped onto the activation of some of the visible neurons. A query, short-term facts, and/or optional cues may also be clamped onto the WM. Once the KB is in its place (stored as long-term weights) and queries or cues are clamped onto the WM, the ANN starts performing its gradient descent algorithm, searching for a solution for the inference problem that satisfies the constraints stored as weights. When the network settles onto a global minimum, such a stable state is interpreted as a FOL inference chain proving queries using resolution-based inference steps [115, 116, 117, 118].

1.5.3.1. ANN Specifications Using Weighted Boolean Constraints

The first step in the process is the specification of the problem (e.g., FOL inference) as a set of weighted propositional logic formulae. These formulae represent Boolean constraints that enforce the neurons to obtain zero/one assignments that correspond exactly to the valid FOL inference solutions.

Satisfying a propositional formula is the process of assigning truth-values (true/false) to the atomic variables of the formula, in such a way that the formula turns out to be true. Maximally satisfying a weighted set of such formulae is finding truth-values that minimise the sum of the weights of the violated formulae. By associating the truth-value true with 1 and false with 0, a quadratic energy function is created, which basically calculates the weighted sum of the violated constraints. Conjunctions are expressed as multiplicative terms, disjunctions as cardinality of union, and negations are expressed as subtraction from 1. In order to hint the network about how close it is to a solution, the conjunction of weighted formulae is treated as a sum of the weights of Boolean constraints that are not satisfied. It is possible to associate the problem of maximal-weighted-satisfiability of the original set of weighted formulae with finding a global minimum for the energy equation that corresponds to the problem specification [26]. In other words, not satisfying a clause would incur an increase of the system’s energy. One interesting and powerful side effect of such a mapping is the possibility of partially satisfying an otherwise unsatisfiable set of formulae. This phenomenon is used for specifying optimal and preferred solutions as in: finding a shortest explanation or proof (parsimony), finding most general unifications, finding the most probable inference or most likely explanation, etc. When different levels of weights are associated with the formulae of the KB, a non-monotonic inference engine is obtained based on Penalty Logic. In fact, in [48] equivalence has been shown between minimising the energy of symmetric ANNs and

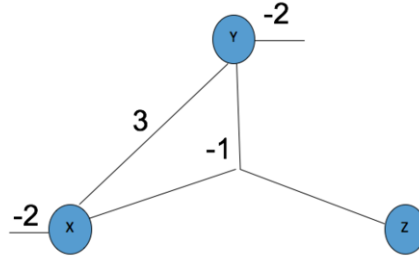


Figure 2. A symmetric 3-order network, characterised by a 3-order energy function: $XYZ - 3XY + 2X + 2Y$. Minima of this energy function are fixed points of the network. This network searches for satisfying solutions for the weighted conjunctive normal form (CNF): $(\neg X \vee \neg Y \vee Z) \wedge (X \vee Y)$. Note that the clauses of the weighted CNF are augmented by penalties reflecting the importance of each constraint.

satisfiability of penalty logic, i.e., every penalty logic formula can be efficiently translated into a symmetric ANN and vice versa. Figure 2 gives an example for a network instantiation of a satisfiability problem.

Based on these results, a compiler has been built that accepts solution specifications as a form of index-quantified penalty logic formulae and generates ANNs that seek for maximally satisfying solutions for the specified formulae. This compiler can then be used to generate ANNs for a variety of FOL retrieval, unification, and inference problems, involving thousands of neural units and tens of thousands of synapses. Alternatively to compilation, these specification constraints could also be learned (unsupervised) using an anti-Hebbian learning rule, whenever a fixed point is found that does not satisfy some constraints [48].

1.5.3.2. ANN Specifications of FOL Inference Chains

There are two possible approaches to logical reasoning, one that uses Model Theory and the other Proof Theory. Informally, Model Theory confirms that a statement could be inferred by checking all possible models that satisfy a KB, while Proof Theory applies inference steps to generate a proof for the statement in question. Most attempts at doing FOL inferences in ANNs—or using propositional satisfiability in general—employ model-theoretic techniques based on grounded KBs [119, 4, 120, 106]. However, grounding results in either exponential explosion of the number of boolean variables (neurons) or severe limitations to the expressive power of the FOL language (e.g. no function symbols and no existential quantifiers). Proof Theory techniques may use ungrounded FOL formulae and are more efficient in size. Another consequence of having a proof for a query statement is that this proof can be seen as an explanation for that statement, which is not always possible to obtain when model-theoretic methods are used. A proof-theoretic approach was, for instance, used by [115] and allows for a variety of inference rules (e.g. Resolution, Modus Ponens, etc.) with either clamped KB or with the KB stored in weights. In [116], a FOL theorem prover has been studied with resolution and a knowledge base that is clamped entirely in activation. In [118] most general unification with occurrence checking is implemented using a more recent binding mechanism with a reduced size complexity.

After the query and/or cues had been clamped onto the WM, the network is set to run until a solution emerges as a global minimum of the network's energy function. This stable state may be interpreted as a proof or a chain of FOL clauses. Each clause is obtained by either retrieving it from the LTM, by copying it from previous steps (optionally with collapse), or by resolving two previous clauses in the chain. In every step, the need to unify resolved literals may cause literals to become more specific by variable binding. It is possible to construct an ANN that searches for proofs by refutation (a complete strategy that derives a contradiction when the negated query is added to the KB) or directly infer a statement. Directly inferring the conclusion, though not a complete strategy, bears more resemblance to human reasoning, and seems more cognitively plausible.

Variations of the above basic inference mechanism use competition among proofs and can be used for inferring the most likely explanation for a query, most probable explanation, most probable inference, least expensive plan or most preferable action (as in Asimov's robotic laws).

1.5.4. A Fault Tolerant Mechanism for Dynamic Binding

A compact binding mechanism is at the foundation of the described FOL encoding and of the general purpose inference engine. The binding technique is the basic mechanism upon which the WM is constructed and it is this mechanism that enables the retrieval of the needed knowledge from LTM and then unification and inference.

Contrary to temporal binding [114], it uses spatial (conjunctive) binding which captures compositionality in the stable state of the neurons and is not sensitive to timing synchronisation or time slot allocation. Although several conjunctive (non-temporal) binding mechanisms have been proposed, the one in [117] compactly uses only few ensembles of neurons, each of which can bind any of the many knowledge items stored in the LTM. This property has three side effects: (i) reduction in the number of neurons (units), as binders are only needed to represent a proof (and not to capture all the KB), (ii) fault tolerance, as each binding ensemble is general purpose and may take the place of any other failed binding ensemble, and finally (iii) high expressive power, as any labelled directed graph structure may be represented by this mechanism (FOL as a special case).

This binding technique applies crossbars of neurons to bind FOL objects (clauses, literals, predicates, functions, and constants) using special neural ensembles called General Purpose (GP) binders. These GP-binders act as indirect pointers, i.e., two objects are bound if and only if a GP-binder points to both of them. Since GP-binders can point to other GP-binders, a nested hierarchy of objects may be represented using such binders pointing at the same time to objects/concepts and to other binders. In fact, arbitrary digraphs may be represented and, as special cases, nested FOL terms, literals, clauses, and even complete resolution-based proofs [115, 116]. Manipulation of such compound representations is done using weighted Boolean constraints (compiled or learned into the synapses), which cause GP-binders to be dynamically allocated to FOL objects, retrieved from the LTM. Since the GP-binders can be used to bind any two or more objects, the number of binders that is actually needed is proportional to the size of the proof. As a result only few such binders are actually needed for cognitively-plausible inference chains, causing the size complexity of the WM to be drastically reduced to $O(n \cdot \log(k))$ where n is the proof length and k is the size of the KB [118]. Interestingly, the fault tolerance of this FOL WM is the result of the generic nature of the binders, which is due to redun-

dancy in synapses rather than to redundancy in units. GP-binders happen to address most of the challenges stated by [112] and summarised above.

1.5.5. Examples of Inferences, Cues, and Deductive Associations

To illustrate the GP-binder approach, assume a KB consisting of the following rules:

- For every u , there exists a mother of u who is also a parent of u : $Parent(mother(u), u)$
- For all x, y, z : $Parent(x, y) \wedge Parent(y, z) \rightarrow GrandParent(x, z)$
- More KB items: facts and/or rules optionally augmented by weights

A general purpose resolution-based inference engine and a WM implemented in a variation of Boltzmann Machines is compiled (or learned) with the above KB stored in its synaptic connections. When clamping the query: $GrandParent(V, Anne)$ onto some of the visible units in the WM, while settling onto a global energy minimum, the network is capable of binding the variable V to the term representing “the mother of the mother of Anne” and finding the complete inference chain as a proof for that query:

1. Retrieving and unifying rule 1: $Parent(mother(Anne), Anne)$
2. Retrieving and unifying rule 2: $\neg Parent(mother(mother(Anne)), mother(Anne)) \vee \neg Parent(mother(Anne), Anne) \vee GrandParent(mother(mother(Anne)), Anne)$
3. Resolving 1 and 2: $\neg Parent(mother(mother(Anne)), mother(Anne)) \vee GrandParent(mother(mother(Anne)), Anne)$
4. Retrieving rule 1 again but with different unification (multiple instances): $Parent(mother(mother(Anne)), mother(Anne))$
5. QED by resolving 3 and 4: $GrandParent(mother(mother(Anne)), Anne)$

FOL clauses, literals, atoms, and terms are represented as directed acyclic graphs while GP-binders act as nodes and labeled arcs. The GP-binders are dynamically allocated without central control and only the needed rules are retrieved from the KB, in order to infer the query (the goal). Other rules that are stored in the KB (in the LTM) are not retrieved into the WM because they are not needed for the inference.

The query may be considered as a kind of cue in the WM, that directs the network’s “attention” to searching for the grandparent of Anne; yet, other cues may be provided instead or in addition to the query: for example, we can accelerate the system’s convergence to the correct solution by specifying in the WM that rule 1 and/or rule 2 should be used, or, even better, by clamping these rules onto the WM activation prior to the network operation.

This ANN inference mechanism could also be interpreted as an intelligent associative memory because cues generate a chain of deduction steps that leads to complex associations. For example, instead of a query, if just “Anne” and “mother” are clamped as cues in the WM this amounts to asking the system to infer something about the person “Anne” and the concept “mother”. In this case the system will converge to one of several possible inferences (each with a known probability). For instance, the system may find that Anne has a parent who is her mother. Alternatively, it may find that Anne has a grandparent (the mother of her mother), or that Anne’s mother has a mother and that mother also has a mother. The likelihood of the possible emerging associations is determined by such factors as the weights of the LTM (i.e. penalties of the FOL KB items, stored in the synapses) and the depth of the inference chains.

1.5.6. Can Neurons Be Logical? Fundamental Negative and Positive Results

The just described work provides evidence that unrestricted (but memory-bound) FOL inference can be represented and processed by relatively compact, attractor-based ANNs such as the previously introduced Boltzmann machines. Nevertheless, in [121] it has also been shown that distributed neural mechanisms—among which also Boltzmann Machines count—may have pathological oscillations in ANNs that have cycles and may never find a global solution. The related positive result is that when the ANN has no cycles, it is possible to have a neural activation function that guarantees finding a global solution. Once a cycle is added to the network, no activation function is able to eliminate all the pathological scenarios above. Unfortunately, the outlined FOL networks have many cycles and are therefore doomed to be incomplete even with a bound on the proof length. Luckily with some plausible assumptions on the daemon, one can guarantee that such pathological scenarios will not last forever. Another major obstacle for achieving good performance is that Boltzmann Machines and other symmetric networks use stochastic local search algorithms to search for a global minimum of their energy function. Theoretically and practically, local minima may plague the search space, so such ANNs are not guaranteed to find a solution in a finite time. Current research is trying to solve this problem by using many variations of Hebbian learning methods and variations of Boltzmann machines to change the synapses of an ANN in a way that local minima are eliminated and a global minimum is found faster. As it turns out, the recent success of deep learning mostly also emerges from the same variations but from a more practical perspective. Learning styles of this type are called “speed-up practicing” as they aim to use learning as a strategy for self-improvement.

1.6. Connectionist First-Order Logic Learning Using the Core Method

As has already become clear in the previous sections, one of the central problems at the core of neural-symbolic integration is the representation and learning of first-order rules (i.e. relational knowledge) within a connectionist setting. But there is a second motivation besides the incentives coming out of cognitive science and cognitive neuroscience and aiming at the modelling and (re-)creation of high-level cognitive processes in a plausible neural setting: The aim to create a new, neurally-inspired computational platform for symbolic computation which would, among others, be massively distributed with no central control, robust to unit failures, and self-improving with time.

In this and the following section we take a more technical perspective—to a certain extent contrasting the strongly cognitively-motivated Sections 1.4 and 1.5—and focus on two prototypical examples for work on the computer science side of neural-symbolic integration, namely (i) the approximation of the immediate consequence operator T_P associated to a FOL program P in a neural learning context, and (ii) Markov Logic as probabilistic extension of FOL combining logic and graphical models. While relegating elaborations on the latter topic to the following Section 1.7, we start with the question of the consequence operator.

Assume a FOL program containing the fact $p(X)$. Applying the associated T_P -operator once (to an arbitrary initial interpretation), leads to a result containing infinitely many atoms, namely all $p(X)$ -atoms for every X within the underlying Herbrand uni-

verse U_L . Thus, one has to approximate the operator because even a single application can lead to infinite results. And while in this simple example one might still be able to define a finite representation, the problem might become arbitrary complex for other programs. So-called rational models have been developed to tackle this problem [122].

Unfortunately, there is no way to compute an upper bound on the size of the resulting rational representation. Because we are not aware of any other finite representation, we will in the following concentrate on the standard representation using Herbrand interpretations. In principle, there are two options to approximate a given operator. On the one hand, one can develop an approximating connectionist representation for a given accuracy. This approximation in space leads to an increasing number of hidden layer units in the resulting networks [123, 124]. Alternatively, a connectionist system can be constructed that approximates a single application of T_P , and feeding its output back as input to itself, thus resulting in an iteration. This approximation in time was used in [125] and [126]. Networks constructed along the approximation in space approach are more or less standard architectures with many hidden layer units, while the others are usually very small but use non-standard architectures and units.

1.6.1. Feasibility of the First-Order Core Method and the Embedding of First-Order Rules into Neural Networks

It is well-known that multilayer feed-forward networks are universal approximators for all continuous functions on compact subsets [127]. If a suitable way to represent first-order interpretations by (finite vectors of) real numbers could be found, then feed-forward networks may be used to approximate the meaning functions of such programs. It is necessary that such a representation is compatible with both, the logic programming and the ANN paradigm. For this, one defines a homeomorphic embedding from the space of interpretations into a compact subset of the real numbers. [128] showed the existence of a corresponding metric homeomorphism, and [129] subsequently expanded this result to the characterisation of a topological isomorphism. Following these insights, one can use level mappings to realise this embedding. For some Herbrand interpretation I , a bijective level mapping $|\cdot| : B_L \rightarrow \mathbb{N}^+$, and $b > 2$, the embedding function $\eta : B_L \rightarrow \mathbb{R}$ and its extension $\eta : I_L \rightarrow \mathbb{R}$ are defined as follows:

$$\eta : B_L \rightarrow \mathbb{R} : A \mapsto b^{-|A|} \quad \eta : I_L \rightarrow \mathbb{R} : I \mapsto \sum_{A \in I} \eta(A)$$

Here, $C_b := \{\eta(I) | I \in L\} \subset \mathbb{R}$ is used to denote the set of all embedded interpretations. This results in a “binary” representation of a given interpretation I in the number system with base b . After embedding an interpretation and looking at its representation within this number system one finds a 1 at each position $|A|$ for all $A \in I$. As mentioned above, the embedding is required to be homeomorphic (i.e. to be continuous, bijective and to have a continuous inverse), because being homeomorphic ensures that η is at least a promising candidate to bridge the gap between logic programs and connectionist networks. One can construct a real valued version f_P of the immediate consequence operator T_P as follows:

$$f_P : C_b \rightarrow C_b : X \mapsto \eta(T_P(\eta^{-1}(X)))$$

Because η is a homeomorphism, f_P is a correct embedding of T_P in the sense that all structural information is preserved. Furthermore, η is known to be continuous which allows to conclude that f_P is continuous for a continuous T_P . Based on these insights, one can construct connectionist networks to approximate T_P . As mentioned above there are in principle two options for the approximation—in space and in time. The embedding η introduced above allows for both, as discussed in detail in [130, 124, 125, 126]. In [131], methods have been described in full detail to construct standard sigmoidal networks, as well as radial basis function networks for any given accuracy of approximation. That is, if an interpretation I is fed into the network, the resulting output can be interpreted as an interpretation J and the distance between J and $T_P(I)$ is limited by a given ε .

So far we have been concerned with the approximation of a single application of the immediate consequence operator only. More interesting is whether the repeated application of the approximating function converges to a fixed point relating to the model of the underlying program. Convergence can be demonstrated by showing that a function is contractive and by employing the Banach contraction mapping principle. Unfortunately, it turns out that the approximating functions described above are not contractive on R in general, but they are contractive if restricted to C_b . Therefore, one can conclude that not only the meaning function T_P can be approximated using a connectionist network, but also the iteration of the approximation converges to a state corresponding to a model of the underlying program provided that T_P itself is contractive.

1.6.2. Learning in a First-Order Setting

Within a propositional setting, learning has been shown to be possible [4] and to be of advantage [132]. As in the propositional case, training data is used to adapt the weights and structure of randomly initialised networks. An approach built upon vector-based networks has first been proposed in [124] along with a specifically tailored learning method. To show the applicability of that approach, a network had randomly been initialised and pairs of interpretations and corresponding consequences (i.e. pairs of I and $T_P(I)$), had been used as training samples. After convergence of the training process the network was evaluated by feeding random interpretations into the network and iterating the computation by feeding the outputs back into the input layer. The resulting sequence of interpretations was found to converge to the model of the underlying program P . Hence it can be concluded that the network indeed managed to learn the immediate consequence operator to sufficient accuracy.

From a more practical perspective (acknowledging the above difficulties, yet recognising their scientific, representational value), some have opted for connecting neural computation with the work done in Inductive Logic Programming [133, 134]. Initial results, that take advantage of ILP methods for efficient FOL learning called propositionalisation [135], have only started to emerge, but with promising results (cf., among others, the Connectionist ILP system by [136], or the work by [137, 138]). Also within a more practical approach, an extension of the above fixed-point method to multiple networks, as shown in Figure 1, has been proposed in [5]. Such an extension, called connectionist modal logic, is of interest here in relation to FOL learning because it has proven capable of representing and learning propositional modal logics, which in turn are equivalent to the fragment of FOL with two variables. Propositional modal logic has been shown to be robustly decidable [139] and it can be considered, as a result, a candidate for offer-

English	First-Order Logic	Weight
Smoking causes cancer	$\forall x : Sm(x) \rightarrow Ca(x)$	1.5
Friends share smoking habits	$\forall x \forall y : Fr(x,y) \wedge Sm(x) \rightarrow Sm(y)$	1.1

Table 2.

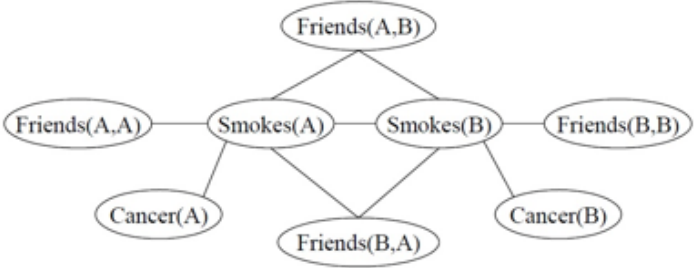


Figure 3. A Markov network with two constants Anna (A) and Bob (B).

ing an adequate middle ground between the requirements of representation capacity and learning efficiency.

1.7. Markov Logic Networks Combining Probabilities and First-Order Logic

As a second prototypical example for the more technical side of neural-symbolic integration besides the connectionist first-order learning application from the previous section, we now turn our attention to Markov logic networks. Markov logic is a probabilistic extension of FOL [140, 141]. Classical logic can compactly model high-order dependencies. However, at the same time it is deterministic and therefore brittle in the presence of noise and uncertainty. The key idea of Markov logic is to overcome this brittleness by combining logic with graphical models. To that end, logical formulae are softened with weights, and effectively serve as feature templates for a Markov network or Markov Random Field (MRF) [142], i.e., an undirected graphical model that allows arbitrary high-order potentials, similar to (but potentially much more compact than) a high-order Boltzmann distribution.

Concretely, a Markov logic network (MLN) is a set of weighted FOL formulae. Together with a set of constants, it defines a Markov network with one node per ground atom (a predicate with all variables replaced by a constant) and one feature per ground formula. The weight of a feature is the weight of the first-order formula that originated it. The probability of a state x in such a network is given by $P(x) = \frac{1}{Z} \exp(\sum_i w_i f_i(x))$, where Z is a normalisation constant, w_i is the weight of the i -th formula, $f_i = 1$ if the i -th formula is true, and $f_i = 0$ otherwise.

As an example, Table 2 shows an MLN with two weighted formulae, and Figure 3 shows the corresponding Markov network with two constants, Anna (A) and Bob (B).

Exact and approximate inference methods for MRFs and other graphical models can also be applied to MLNs. Popular choices include variable elimination or junction trees for exact inference and message-passing or Markov chain Monte Carlo methods for approximate inference. However, compared to standard MRFs, MLNs introduce new

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

challenges and opportunities for efficient reasoning. One challenge is that even modestly sized domains can yield exceptionally large and challenging ground Markov networks. For example, in a social network domain such as Table 2, the number of friendships grows quadratically with the number of people, since any pair of people can be friends. Worse, simple concepts such as transitivity (friends of your friends are likely to be your friends as well) lead to a cubic number of ground formulae. Thus, a domain of 1000 people has 1 million possible friendships and 1 billion possible transitivity relations. Standard inference methods do not scale well to these sizes, especially when formula weights are large, representing strong interactions among the variables. The opportunity is that these large models have a great deal of symmetry, structure that can be exploited by “lifted” inference algorithms (cf. [143, 144]). Lifted inference works by grouping variables or variable configurations together, leading to smaller, simpler inference problems. For example, when there is no evidence to differentiate Anna and Bob, the probability that each smokes must be identical. Depending on the MLN structure and evidence, lifted inference can compute exact or approximate probabilities over millions of variables without instantiating or reasoning about the entire network. Another technique for scaling inference is to focus on just the subset of the network required to answer a particular query or set of queries [145]. Related methods include lazy inference [146, 147], which only instantiates the necessary part of the network, and coarse-to-fine inference [148], which uses more complex models to refine the results of simpler models.

The parameter values in an MLN are critical to its effectiveness. For some applications, simply setting the relative weights of different formulae is sufficient; for example, a domain expert may be able to identify some formulae as hard constraints, others as strong constraints, and others as weak constraints. In many cases, however, the best performance is obtained by carefully tuning these to fit previously observed training data. As with other probabilistic models, MLN weight learning is based on maximizing the (penalized) likelihood of the training data. Given fully-observed training data, this is a convex optimization problem that can be solved with gradient descent or second-order optimization methods. However, computing the gradient requires running inference in the model to compute the expected number of satisfied groundings of each formula. Furthermore, the gradient is often very ill-conditioned, since some formulae have many more satisfying groundings than others. The standard solution is to approximate the gradient with a short run of Markov chain Monte Carlo, similar to persistent contrastive divergence methods used to train restricted Boltzmann machines [149], and to precondition the gradient by dividing by its variance in each dimension [150]. A popular alternative is optimizing pseudolikelihood [151], which does not require running inference but may handle long chains of evidence poorly.

Most recently, a new paradigm has emerged by tightly integrating learning with inference, as exemplified by sum-product networks (SPNs) [152]. The development of SPNs is motivated by seeking the general conditions under which the partition function is tractable. SPNs are directed acyclic graphs with variables as leaves, sums and products as internal nodes, and weighted edges. Figure 4 (left) shows an example SPN implementing a junction tree with clusters $(X_1 \text{ and } X_2)$ and $(X_1 \text{ and } X_3)$ and separator X_1 , and Figure 4 (right) depicts a Naive Bayes model with variables X_1 and X_2 and three components.

To compute the probability of a partial or complete variable configuration, set the values of the leaf variables and compute the value of each sum and product node, starting

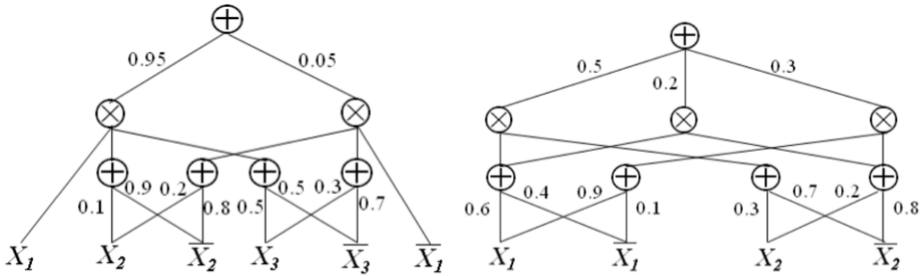


Figure 4. (left) Example SPNs implementing a junction tree with clusters $(X_1 \text{ and } X_2)$ and $(X_1 \text{ and } X_3)$ and separator X_1 . (right) Naive Bayes model with variables X_1 and X_2 and three components.

at the bottom of the SPN. For example, to compute $P(X_1 = \text{true}, X_3 = \text{false})$ in the SPN in Figure 4 (left), set the value of $\bar{X}_1$ and X_3 to 0, since these variable values contradict the specified configuration, and set all other leaves to 1. Each of the four lower sum nodes is a weighted sum of two leaf values; from left to right, they evaluate to 1, 1, 0.5, and 0.3. The product nodes above them evaluate to 0.5 and 0.3, and the overall root equals $0.95 \cdot 0.5 + 0.05 \cdot 0.3 = 0.49$. Thus, inference follows the structure of the SPN, allowing arbitrary probability queries to be answered in linear time.

It has been shown that if an SPN satisfies very general conditions called completeness and consistency, then it represents the partition function and all marginals of some graphical model. Essentially all tractable graphical models can be cast as SPNs, but SPNs are also strictly more general. SPNs achieve this by leveraging the key insight of dynamic programming, where intermediate results are captured (by internal sums and products) and reused. In this aspect, SPNs are similar to many existing formalisms such as AND/OR graphs and arithmetic circuits. However, while these formalisms are normally used as compilation targets for graphical models, SPNs are a general probabilistic model class in its own right, with efficient learning methods developed based on backpropagation and EM. Experiments show that inference and learning with SPNs can be both faster and more accurate than with standard deep networks. For example, SPNs perform image completion better than many state-of-the-art deep networks for this task. SPNs also have intriguing potential connections to the architecture of the cortex (cf. [152]).

Relational sum-product networks (RSPNs) extend SPNs to handle relational domains such as social networks and cellular pathways [153]. Like MLNs, an RSPN represents a probability distribution over the attributes and relations among a set of objects. Unlike MLNs, inference in an RSPN is always tractable. To achieve this, RSPNs define a set of object classes. A class specifies the possible attributes of each object, the classes of its component parts, and the possible relations among the parts. For example, Figure 5 shows a partial class specification for a simple political domain. A ‘Region’ may contain any number of nations, and two nations within a region may be adjacent, in conflict, or both. A ‘Nation’ contains a government object and any number of person objects, each of which may or may not support the government. A class also specifies a tractable distribution over the attributes, parts, and relations. The structure and parameters of SPNs can be learned from data. RSPNs have been applied to social network and automated debugging applications, where they were much faster and more accurate than MLNs.

In summary, due to the achieved combination between symbolic representations and subsymbolic (in this case, graphical) features and forms of reasoning, MLNs and related

```

class Region:
    exchangeable part Nation
    relation Adjacent(Nation,Nation)
    relation Conflict(Nation,Nation)

class Nation:
    unique part Government
    exchangeable part Person
    attribute HighGDP
    relation Supports(Person,Government)

```

Figure 5. Partial class definition for a relational sum-product network in a simple political domain.

formalisms have found a growing number of applications and gained popularity in AI in general. They can therefore be seen as success stories of symbolic-numerical integration closely related to the original domain of neurally-plausible or -inspired implementations of high-level cognitive reasoning, exemplifying valuable theory work with high relevance to computer science and, thus, complementing the aforementioned examples—among others given by NSCA and Penalty Logic—as relevant neural-symbolic applications on the systems level.

1.8. Relating Neural-Symbolic Systems to Human-Level Artificial Intelligence

Following the quite technical exposition of the last two sections, we now return to more conceptually-motivated considerations focusing on the relation between neural-symbolic integration and research in Human-Level Artificial Intelligence (HLAI), before identifying corresponding challenges and research opportunities.

HLAI, understood as the quest for artificially-created entities with human-like abilities, has been pursued by humanity since the invention of machines. It has also been a driving force in establishing artificial intelligence as a discipline in the 1950s. 20th-century AI, however, has developed into a much narrower direction, focussing more and more on special-purpose and single-method driven solutions for problems which were once (or still are) considered to be challenging, like game playing, problem solving, natural language understanding, computer vision, cognitive robotics, and many others. In our opinion, 20th-century AI can therefore be perceived as expert AI, producing and pursuing solutions for relevant specific tasks by using specialised methodologies designed for the tasks in question. We do not aim to say that this is a bad development—quite to the contrary, we think that this was (and still is) a very worthwhile and very successful endeavour with ample (and in some cases well-proven) scope for considerable impact on society.

However—although having been one of the core ideas in the early days of AI—the pursuit of HLAI has been declining in the 20th century, presumably because the original vision of establishing systems with the desired capabilities turned out to be much harder to realise than expected in the beginning. Nevertheless, in recent years a rejuvenation of the original ideas has become apparent, driven on the one hand by the insight that certain complex tasks are outside the scope of specialised systems, and on the other hand

by the discovery of new methodologies appropriate to address hard problems of general intelligence. Some examples for such new methodologies are the numerous approaches for machine learning, non-classical logic frameworks, or probabilistic reasoning, just to mention some of them. Furthermore, rapid developments in the neurosciences based on the invention of substantially refined means of recording and analysing neural activation patterns in the brain influenced this development. These are accompanied by interdisciplinary efforts within the cognitive science community, including psychologists and linguists with similar visions.

It is apparent that the realisation of HLAI requires the cross-disciplinary integration of ideas, methods, and theories. Indeed we believe that disciplines such as (narrow) artificial intelligence, neuroscience, psychology, and computational linguistics will have to converge substantially before we can hope to realise human-like artificially intelligent systems. One of the central questions in this pursuit is thus a meta-question: What are concrete lines of research which can be pursued in the immediate future in order to advance in the right direction and to avoid progress towards—currently prominently discussed, albeit highly speculative—less desirable technological or social scenarios? The general vision does not give any answers to this, and while it is obvious that we require some grand all-encompassing interdisciplinary theories for HLAI, we cannot hope to achieve this in one giant leap. For practical purposes—out of pure necessity since we cannot, and for principled reasons should not even attempt to, shed our scientific inheritance [154]—we require the identification of next steps, of particular topics which are narrow enough so that they can be pursued, but general enough so that they can advance us into the right direction. This brings us back to the topic of this survey article, neural-symbolic integration.

The proposal for neural-symbolic integration as such a research direction starts from two observations. These correspond to the two principled perspectives intersecting and combining in neural-symbolic integration:

(i) The physical implementation of our mind is based on the neural system, i.e., on a network of neurons as identified and investigated in the neurosciences. If we hope to achieve HLAI, we cannot ignore this neural or subsymbolic aspect of biological intelligent systems.

(ii) Formal modelling of complex tasks and human—in particular, abstract—thinking is based on symbol manipulation, complex symbolic data structures (like graphs, trees, shapes, and grammars) and symbolic logic. At present, there exists no viable alternative to symbolic approaches in order to encode complex tasks.

These two perspectives however—the neural and the symbolic—are substantially orthogonal to each other in terms of the state-of-the-art in the corresponding disciplines. As also evidenced by the previous sections, symbolically understanding neural systems is hard and requires significant amounts of refined theory and engineering while still falling short of providing general solutions. Furthermore, it is quite unclear at present how symbolic processing at large emerges from neural activity of complex neural systems. Therefore, symbolic knowledge representation and the manipulation of complex data structures used for representing knowledge at the level required for HLAI is way outside the scope of current artificial neural approaches.

At the same time humans—using their neural wetware, i.e. their brains—are able to deal successfully with symbolic tasks, to manipulate symbolic formalisms, to represent knowledge using them, and to solve complex problems based on them. In total, there is

a considerable mismatch between human neurophysiology and cognitive capabilities as role models for HLAI on the one hand, and theories and computational models for neural systems and symbolic processing on the other hand.

It is our belief that significant progress in HLAI requires the reconciliation of neural and symbolic approaches in terms of theories and computational models. We believe that this reconciliation is as much central for the advancement of HLAI as it is for cognitive science and cognitive neuroscience (as described in Section 1.4). We also believe that the pursuit of this reconciliation is timely and feasible based on the current state of the art, and will continue by briefly mentioning some recent developments in neural-symbolic integration which we consider to be of particular importance for the HLAI discussion. Further pointers can be found, e.g., in [126, 17, 5].

The line of investigation we want to mention takes its starting point from computational models in (narrow) AI and machine learning. It attempts to formally define and practically implement systems based on ANNs which are capable of learning and dealing with logic representations of complex domains and inferences on these representations. While this can be traced back to McCulloch & Pitts' landmark paper on the relation between propositional logic and binary threshold ANNs [28], it has been largely dormant until the 1990s, where (as already mentioned in previous sections) first neural-symbolic learning systems based on these ideas were realised—cf., among others, [19, 33, 4]. While these initial systems were still confined to propositional logics, in recent years systems with similar capabilities based on first-order logic have been realised—cf., for instance, [155] or Section 1.6. It is to be noted, however, that these systems—despite the fact that they provide a conceptual breakthrough in symbol processing by ANNs—are still severely limited in their scope and applicability, and improvements in these directions do not appear to be straightforward at all. In order to overcome such problems, we also require new ideas borrowed from other disciplines, in order to establish neural-symbolic systems which are driven by the HLAI vision. Results from cognitive psychology on particularities of human thinking which are usually not covered by standard logical methods need to be included. Recent paradigms for ANNs which are more strongly inspired from neuroscience—as discussed, for example, in [156]—need to be investigated for neural-symbolic integration. On top of this, we require creative new ideas borrowed, among others, from dynamical systems theory or organic computing to further the topic.

The presented selection is considered to be exemplary and there are several other efforts which could be mentioned. However, we selected this line of research for presentation in the last paragraph as to us it appears to be typical and representative in that it is driven by computer science, machine learning, and classical AI research augmented with ideas from cognitive science and neuroscience.

1.9. (Most) Recent Developments and Work in Progress from the Neural-Symbolic Neighbourhood

Before concluding this opinionated survey with two sections dedicated to future challenges and directions (Section 1.10) and to some short overarching remarks (Section 1.11), in the following we will highlight recent work surfacing outside the traditional core field but in the topical proximity of neural-symbolic integration with high relevance and potential impact for the corresponding core questions.

1.9.1. Other Paradigms in Computation and Representation Narrowing the Neural-Symbolic Gap

We first want to direct the reader's attention to a recently proposed neuro-computational mechanism called "conceptors" [157]. Broadly speaking, conceptors are a proposed mathematical, computational, and neural model combining two basic ideas, namely that the processing modes of a recurrent neural network (RNN) can be characterised by the geometries of the associated state clouds, and that the associated processing mode is selected and stabilised if the states of the respective RNN are filtered to remain within a certain corresponding state cloud. When taken together, this basically amounts to a possibility to control a multiplicity of processing modes of one single RNN, introducing a form of top-down control to the bottom-up connectionist network. While the underlying mathematical considerations are fairly elaborate, the overall approach can be conceptualised as a two-step approach. First, the different ellipsoids enveloping the differently shaped state clouds a driven RNN exhibits when exposed to different dynamical input patterns each give a different conceptor. Once the driving patterns have been stored in the network (i.e., the network has learned to replicate the pattern-driven state sequences in the absence of the driver stimuli, performing a type of self-simulation), they can be selected and stably re-generated by inserting the corresponding conceptor filters (i.e., the geometry of the ellipsoid serves as decision mechanism of the filter) in the network's update loop. This partially geometric nature of the approach also offers another advantage in that conceptors can be combined by operations akin (and partially co-extensional) to Boolean logic, thus introducing an additional dimension of semantic interpretation especially catering to the symbolic side of neural-symbolic integration.

While conceptors are an approach to equipping sub-symbolic models in form of RNNs with certain symbolic capacities and properties, the second recent research result we want to mention aims at a connectionist implementation of Von Neumann's computing architecture [158] as the prototypical symbolic computer model. "Neural Turing Machines" (NTMs), introduced in [159], basically couple RNNs with a large, addressable memory yielding the end-to-end differentiable and gradient-descent trainable analog of Turing's memory-tape enriched finite-state machines. In terms of architecture, an NTM is constituted by a neural network controller and a memory bank (see Figure 6). The controller manages the interaction with the environment via input and output vectors and interacts with a memory matrix using selective read and write operations. The main feature of NTMs is their end-to-end differentiability which allows for training with gradient-descent approaches. In order to introduce this capacity, read and write operations have been defined in a way which allows them to interact to different degrees with all elements in the memory (modulated by a focus mechanism limiting the actual range of interaction), resulting in a probabilistic mode of reading and writing (different from the highly specific discrete read/write operations a Turing machine performs). Also, interaction with the memory tends to be highly sparse, biasing the model towards interference-free data storage. Which memory location to attend to is determined by specialised network outputs parameterising the read and write operations over the memory matrix, defining a normalised weighting over the rows of the matrix. These network outputs thereby can focus attention to a sharply defined single memory location or spread it out to weakly cover the memory at multiple adjacent locations. Several experiments also reported in [159] demonstrate that NTMs are generally capable of learning simple

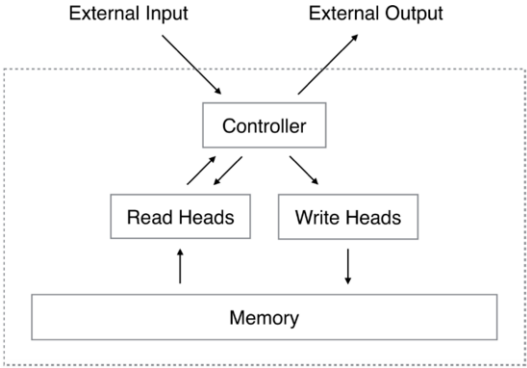


Figure 6. The basic structure of a Neural Turing Machine (taken from [159]): The controller network receives external inputs and emits outputs during each update cycle. Simultaneously, it reads to and writes from a memory matrix via parallel read and write heads.

algorithms from example data and subsequently generalise these algorithms outside of the training regime.

Another approach to combining the learning capacities of connectionist networks with a form of read- and writable memory, as well as inference capacities, has been presented in the form of so called “memory networks” [160]. A memory network consists of an array of indexed objects (such as arrays or vectors) m constituting the memory component, an input feature map I converting incoming input to the respective internal feature representation, a generalisation G updating (and/or possibly compressing and generalising) old memories given the new input, an output feature map O producing a new output given the new input and the current memory state, and finally a response R converting the output into the final format. Except for m , the remaining four components I , G , O , and R could potentially be learned. Against this backdrop, if an input x arrives at the memory network, x is converted to an internal feature representation $I(x)$ before updating the memories m_i within m given x : $\forall i : m_i = G(m_i, I(x), m)$. Subsequently, output features $o = O(I(x), m)$ are computed given the new input and the memory, and the output features are decoded into a final response $r = R(O)$. The flow of the model remains the same during both training and testing, with the only difference being that the model parameters I , G , O , and R stay unchanged at test time. Due to this very general characterisation, memory networks can be flexible concerning the possible implementation mechanisms for I , G , O , and R , generally allowing the use of standard models from machine learning (such as RNNs, SVMs, or decision trees). In [160] also an account of an instantiation of the general model is given, describing an implementation of a memory network for application in a text-based question answering (QA) setting using neural networks as components. The resulting system is then applied and evaluated separately in a large-scale QA and a simulated QA setting (the latter also including some QA with previously unseen words), before combining both setups in a final hybrid large-scale and simulated QA system serving as proof of concept for the possible future capacities of the approach.

1.9.2. Other Application Systems Narrowing the Neural-Symbolic Gap

Following the previous discussion of neural-symbolically relevant advances on the architectural and methodological level, in this subsection we will focus on several recent application systems which are partially or fully based on connectionist approaches but are successfully applied to tasks of predominantly symbolic nature.

The first application area which recently has seen rapid development and systems operating on a previously unachieved, qualitatively new level of performance is the domain of vision-based tasks such as the semantic labelling of images according to their pictorial content. In [161] and [162] two approaches to related problems have been presented. While the generative, deep RNN model in [161] generates natural language sentences describing the content of input images, [162] introduces an approach to recognition and labelling tasks for the content of different image regions (corresponding to different objects and their properties in the pictures). The model from [161] combines an input convolutional neural network, used for encoding the image (the network is pre-trained for an image classification task, and the last hidden layer serves as interface to the following natural language decoder), with an output RNN, generating natural language sentences based on the processed data of the input network. The result is a fully stochastic gradient descent-trainable neural network combining, in its sub-networks a vision and a language model, and—when put in application—significantly outperforming previous approaches and narrowing the gap to human performance on the used test sets. In [162], a region convolutional neural network is applied to the identification of objects (and the corresponding regions) in images and their subsequent encoding in a multimodal embedding space, together with a bidirectional RNN which computes word representations from input sentences and also embeds them in the same space. Once this mapping of images and sentences into the embedding space has been achieved, an alignment objective in terms of an image-sentence score (a function of the individual region-word scores) is introduced, tying together both parts of the model. The resulting system again performs better than previous approaches, showing state of the art performance in image-sentence ranking benchmarks and outperforming previous retrieval baselines in fullframe and region-level experiments. Both models and the corresponding implementation systems are remarkable in that classically tasks involving semantic descriptions had been associated with databases containing some form of background knowledge, and the relevant picture processing/computer vision approaches had for a long time also relied on rule-based techniques. The described work gives proof that this type of task—which clearly conceptually still operates on a symbolic level—generally can also be addressed in practice in purely connectionist architectures.

Another closely related type of task is visual analogy-making as, for instance, exemplified in transforming a query image according to an example pair of related images. In [163], the authors describe a way how to tackle this challenge again applying deep convolutional neural networks. In their approach a first network learns to operate as a deep encoder function mapping images to an embedding space (which due to its internal structure allows to perform some forms of reasoning about analogies via fairly standard mathematical operations), before a second network serves as a deep decoder function mapping from the embedding back to the image space. The resulting models are capable of learning analogy-making based on appearance, rotation, 3D pose, and several object attributes, covering a variety of tasks ranging from analogy completion, to animation

transfer between video game characters, and 3D pose transfer and rotation. Also here, in general analogy models previously have mostly been symbol-based, and generative models of images had to encode significant prior background knowledge and restrict the allowed transformations, making the general task domain mostly symbolic in nature.

A different domain is addressed by the work described in [164]. There, a deep distributed recurrent Q-network is successfully applied to learning communication protocols for teams of agents in coordination tasks in a partially observable multi-agent setting. This type of task requires agents to coordinate their behaviour maximising their common payoff while dealing with uncertainty about both, the hidden state of the environment and the information state and future actions of the members of their team. In order to solve this challenge, so called “deep distributed recurrent Q-networks” (DDRQN) have been introduced. These networks are based on independent deep Q-network agents with Long Short-Term Memory (LSTM) networks [165]—a currently very popular type of RNN for tasks involving the processing of sequential data—augmented by (i) supplying each agent with its previous action as input on the following time step (enabling agents to approximate their own action-observation history), (ii) sharing a single network’s weights between all agents and conditioning on the agent’s ID (enabling fast learning and diverse behaviour), and (iii) disabling experience replay (avoiding problems with non-stationarity arising from the simultaneous learning of multiple agents). The resulting system proves capable of solving (for limited numbers of agents) two classical logic riddles necessarily requiring collaborative communication and information exchange between players in their optimal solution strategies. While future work still will have to prove the scalability of the approach, the work in [164] nonetheless is a proof of concept for the ability of connectionist networks to learn communication strategies in multi-agent scenarios—which is another instance of a genuinely symbolic capacity that has successfully been implemented in a neural network architecture.

The final example of a successful application system addressing aspects of neural-symbolic integration which we want to mention is the AlphaGo Go-playing system presented in [166]. While Go had for a long time proven to be computationally unfeasible to solve due to the quickly growing game tree and corresponding search space, the authors introduce an architecture equipped with deep neural network-based move selection and position evaluation functions (learned using a combination of supervised learning from human expert games and reinforcement learning from massively parallel self-play). Integrating these, in the form of policy and value networks, with a Monte Carlo tree search lookahead approach, the resulting high-performance tree search engine allows to efficiently explore the game tree and play the game at a level on par with the world’s best human players. Different from the previous examples from the visual domain, the resulting architecture is closer to being genuinely neural-symbolic in its combination between trained connectionist networks for pruning and narrowing down the search space, and the subsequent Monte Carlo algorithm selecting actions by lookahead tree search. With its previously unachieved performance AlphaGo constitutes a prime example of the possibilities introduced by combining connectionist and symbolic approaches and techniques.

1.9.3. Other Research Efforts and Programs Narrowing the Neural-Symbolic Gap

In this final subsection we want to focus on three different lines of work which stand out among the usual work in neural-symbolic integration in that they either start out

from a different perspective than neural computation or logic-based modelling, or apply techniques and approaches which are not commonly used in the field. Examples for such “non-classical neural-symbolic research programs” can be found, for instance, in certain investigations into methods and mechanisms of multi-agent systems, in empirical attempts at understanding RNNs by using language models as testbed providing data for qualitative analyses, and in theoretical computer science and complexity theory projects targeting the practical differences between symbolic and connectionist approaches with theoretical means. We describe three recent suggestions as prototypical cases in point.

(i) Anchoring Knowledge in Interaction in Multi-Agent Systems: The typical setting in multi-agent systems encompasses an agent exploring the environment, learning from it, reasoning about its internal state given the input, acting and, in doing so, changing the environment, exploring further, and so on as part of a permanent cycle [167]. One of the biggest challenges in this context is the (bidirectional) integration between low-level sensing and interacting as well as the formation of high-level knowledge and subsequent reasoning. In [168] an interdisciplinary group of researchers spanning from cognitive psychology through robotics to formal ontology repair and reasoning sketch conceptually (what amounts to) a strongly cognitively-motivated neural-symbolic architecture and model of a situated agent’s knowledge acquisition through interaction with the environment in a permanent cycle of learning through experience, higher-order deliberation, and theory formation and revision.

In the corresponding research program—further introduced and actually framed in terms of neural-symbolic integration in [169]—different topics also mentioned in the present survey article are combined for that purpose. Starting out from computational neuroscience and network-level cognitive modelling (as represented, e.g., by the conceptors framework described above) combined with psychological considerations on embodied interaction as part of knowledge formation, low-level representations of the agent’s sensing are created as output of the ground layer of the envisioned architecture/model. This output then is fed into a second layer performing an extended form of anchoring [170] not only grounding symbols referring to perceived physical objects but also dynamically adapting and repairing acquired mappings between environment and internal representation. The enhanced low-level representations as output of the anchoring layer are then in turn fed to the—in terms of neural-symbolic integration most classical—lifting layer, i.e., to an architectural module repeatedly applying methods combining neural learning and temporal knowledge representation in stochastic networks (as, e.g., already discussed in the form of RBMs in the context of the NSCA framework in Section 1.3) to continuously further increase the abstraction level of the processed information towards a high-level symbolic representation thereof. Once a sufficiently general level has been reached, a final layer combining elaborate cognitively-inspired reasoning methods on the symbolic level (such as, for instance, analogy-making, concept blending, and concept reformation) then is used for maintaining a model of the agent and the world on the level of explicit declarative knowledge [171], which nonetheless is intended to feed back into the lower layers—among others by serving as guidance for the augmented anchoring process.

(ii) Visualising and Understanding Recurrent Networks: LSTMs stand out between current deep network models due to their capacity to store and retrieve information over prolonged time periods using built-in constant error carousels governed by explicit gating mechanisms. Still, while having been used in several high-profile application stud-

ies, this far only little is known about the precise properties and representation capacities of LSTMs. In [172], a first step towards clarifying some of the relevant questions is described and results are presented which are also relevant from a neural-symbolic perspective. The authors empirically shed light on aspects of representation, predictions and error types in LSTM networks, uncover the existence of interpretable cells within the network which keep track of long-range dependencies in the respective input data, and suggest that the remarkable performance of LSTMs might be due to long-range structural dependencies.

In order to gain insight into the inner workings of LSTM networks, character-level language models are used as an interpretable testbed for several computational experiments. In theory, by making use of its memory cells an LSTM should be capable to remember long-range information and keep track of different attributes of the respective input it is just processing. Also, using text processing as an example, manually setting up a memory cell in such a way that it keeps track of whether it is inside a quoted string or not is not overly challenging. Still, whether LSTMs actually resort to this type of internal structure (i.e., forming interpretable cells with dedicated, more abstract tasks) was not as clear. The empirical findings in [172] indicate that such “task cells” indeed exist, in the case of processing program code as textual input ranging from cells checking for parenthesis after an if statement to cells acting as line length counter or tracking the indentation of blocks of code. In a second step, using finite horizon n -gram models as comparandum, it is shown that LSTM networks are remarkably good at keeping track of long-range interactions and dependencies within the input data.

From the neural-symbolic point of view, the work reported in [172] is important in at least two ways. On the one hand, the findings relating to the existence of interpretable long-range interactions and associated cells in the network setup actually take LSTMs one step closer to the level of symbolic interpretability and away from the completely distributed and non-localised classical connectionist setting. On the other hand, the overall approach and the set of methods applied in the corresponding empirical studies and analyses (including qualitative visualisation experiments, statistical evaluations of cell activations, and the comparison to the n -gram models) suggest potential techniques which could be used in more detailed follow-up studies about knowledge extraction from LSTMs and similar assessments of other types of recurrent neural networks in general.

(iii) Identifying and Exploring Differences in Complexity: Contrasting the multi-agent approach described in (i) and the empirical analysis of LSTM networks in (ii), the second research program proposed in [169] approaches the differences between ANNs and logics from the angle of theoretical computer science and complexity theory. In the suggested line of work recent advances in the modelling and analysis of connectionist networks and new developments and tools for investigating previously unconsidered properties of symbolic formalisms shall be combined in an attempt at providing an explanation for the empirical differences in terms of applicability and performance between neural and logical approaches very quickly showing up in application scenarios (which seem to harshly contradict the formal equivalence between both described in Section 1.2).

To this end, it is suggested that the form and nature of the the polynomial overhead as computational-complexity difference between paradigms should be examined in more detail. The underlying hope is that a closer look at this overhead might shed

light on previously unconsidered factors as hitherto most complexity results have been established using exclusively *TIME* and *SPACE* as classical resources for this form of analysis, and most analyses have left out more precise investigations of the remaining polynomial overhead after establishing tractability. Against this background, the working hypotheses for the program are that for a fully informative analysis other resources might have to be taken into account on the connectionist side (such as the number of spikes a spiking network requires during computation, or the number of samples needed for convergence from an initial network state to the stationary distribution) and that additional formal tools and techniques will have to be applied on the symbolic side (such as parameterised-complexity methods which take into account problem- and application-specific properties of problem classes, or descriptive-complexity theory which also considers the polynomial-time and the logarithmic-time hierarchy).

The results of these efforts promise to contribute to resolving some of the basic theoretical and practical tensions arising when comparing the suitability and feasibility of an application of certain formalisms to different types of tasks and domains. Which, in turn, then is expected to also successively provide information on general challenges (and their potential remedies) which would have to be overcome for advancing towards neural-symbolic integration on the formal and theoretical side.

1.10. Challenges and Future Directions

In this second-to-last section of our survey we give an overview of what we consider the currently most pressing and/or—if addressed successfully—potentially most rewarding theoretical and practical questions and topics on the way towards bridging the gap between connectionist computation and high-level reasoning.

As described throughout this article, if connectionism is to be an alternative paradigm to logic-based artificial intelligence, ANNs must be able to compute symbolic reasoning efficiently and effectively, combining the fault-tolerance of the connectionist component with the “brittleness and rigidity” of its symbolic counterpart. On the more architecturally-oriented side, by integrating connectionist and symbolic systems using some of the previously sketched or similar approaches, hybrid architectures [173] seek to tackle this problem and offer a principled way of computing and learning with knowledge of various types and on different levels of abstraction. Starting out from multi-modular approaches such as, for instance, the CLARION architecture [174] in cognitive modelling or the vision of symbolic/sub-symbolic multi-context systems [175] in data integration and reasoning, they have the potential to open up the way and serve as foundation for the convergence between the paradigms and the development of fully-integrated monolithic neural-symbolic systems. Nonetheless, the development of such systems is currently still in its infancy and often is motivated much more by concrete engineering imperatives on a case-by-case basis than by coordinated and targeted research efforts aiming at testing general approaches and understanding the general mechanisms and constraints at work.

On a more positive note, from a more formally-centred perspective it is possible to identify several current points of approximation and potential confluence: deep belief networks are based on Boltzmann machines as an early symmetric ANN model, and related to Bayesian networks. Certain neural-symbolic recurrent networks are very similar

to dynamic Bayesian networks. Connectionist modal logic uses modal logic as a language for reasoning about uncertainty in a connectionist framework. Markov logic networks combine logical symbols and probability. In summary it can, thus, be observed that the objectives indeed seem to be converging. Still, also here the current state of affairs is not yet satisfactory as too often too little consideration is given to an integration on the level of mechanisms, allowing them to remain varied instead of triggering convergence also along this (potentially even more relevant) dimension.

In general, many questions and limits to integration hitherto are unsolved and sometimes even unaddressed. Currently, the focus still predominantly lies on either learning or reasoning, with efforts aiming to improve one using state-of-the-art models from the other, rather than truly integrating both in a principled way with contribution to both areas. In addition to, for instance, [176] we now turn to some of the (many) remaining challenges: we have seen that neural-symbolic systems are composed of (i) translations from logic to network, (ii) machine learning and reasoning, (iii) translation from network to logic. Among the main challenges are in (i) finding the limits of representation, in (ii) finding representations that are amenable to integrated learning and reasoning, and in (iii) producing effective knowledge extraction from very large networks.

What follows is a list of research issues related to challenges (i) to (iii) each of which could serve as basis for a research program in its own right:

- Reconciling first-order logic learning and first-order logic reasoning.
- Embracing semi-supervised and incremental learning.
- Evaluating and analysing large-scale gains of massive parallelism.
- Implementing learning-reasoning-acting cycles in cognitive agents.
- Developing representations for learning which learn the ensemble structure.
- Rule extraction and interpretability for networks with thousands of neurons.
- Applying fibring in practice and actually learning the fibring function.
- Theoretically understanding the differences in application behaviour between connectionist and symbolic methods.
- Developing a proof theory and type theory for ANNs.
- Developing analogical and abductive neuro-symbolism with the aim of automating the scientific method.
- Investigating neural-symbolic models of attention focus and modelling and contrasting emotions and utility functions.

While some of the former recently have started to attract active attention from researchers—examples are the work on neural-symbolic multi-agent systems, the proposal for a research program addressing the learning-reasoning-acting cycle, or the effort to better theoretically understand the theoretical basis of application differences between neural and logical approaches outlined in Section 1.9—our knowledge about these issues is only limited and many questions still have to be asked and answered.

Also in terms of approach, several remarks seem in place: while seeing it as highly relevant concerning motivations and as source of inspiration, we agree that the direct bottom-up approach (i.e., brain modelling) is not productive given the current state of the art. Also, we agree that an engineering approach is valid but specific and will not answer the big questions asked, for instance, in more cognitively- or HLAI-oriented lines of research (including some of the bullet points above). Therefore, a foundational approach

is needed combining all the different directions touched upon in this survey article, i.e., encompassing (at least) different elements of computer science (in particular, knowledge representation and machine learning), cognitive science, and cognitive neuroscience.

Finally, in terms of applications, the following can be counted among the potential “killer applications” for neural-symbolic systems: ANNs have been very effective at image processing and feature extraction as evidenced, for instance, by [177] and the work reported in the previous section. Yet, large-scale symbolic systems have been the norm for text processing (as most approaches currently in use rely on large ontologies, are inefficient to train from scratch, and heavily dependent on data pre-processing). Even if networks could be trained without providing starting knowledge to perform as well as, for example, WordNet, the networks would be very difficult to maintain and validate. Neural-symbolic systems capable of combining network models for image processing and legacy symbolic systems for text processing seem therefore ideally suited for multi-modal processing. Here concepts such as fibring (our running example from Section 1.2) seem important so that symbolic systems and network models can be integrated loosely at the functional level. In this process, inconsistencies may arise, making the resolution of clashes a key issue. One approach is to see inconsistency as a trigger for learning, with new information in either part of the combined system serving to adjudicate the conflict (similar to the mechanisms envisioned within the learning-reasoning-acting framework described in the previous section). The immediate impact of this application would be considerable in many areas including the web, intelligent applications and tools, and security.

1.11. Concluding Remarks

This survey summarises initial links between the fields of computer science (and more specifically AI), cognitive science, cognitive neuroscience, and neural-symbolic computation. The links established are aimed at answering a long-standing dichotomy in the study of human and artificial intelligence, namely the perceived dichotomy of brain and mind, characterised within computer science by the symbolic approach to automated reasoning and the statistical approach of machine learning.

We have discussed the main characteristics and some challenges for neural-symbolic integration. In a nutshell:

neural-symbolic systems = connectionist machine + logical abstractions

The need for rich, symbol-based knowledge representation formalisms to be incorporated into learning systems has explicitly been argued at least since [178]’s seminal paper, and support for combining logic and connectionism, or logic and learning was already one of Turing’s own research endeavours [11].

Both the symbolic and connectionist paradigms embrace the approximate nature of human reasoning, but when taken individually have different virtues and deficiencies. Research into the integration of the two has important implications that can benefit computing, cognitive science, and even cognitive neuroscience. The limits of effective integration can be pursued through neural-symbolic methods, following the needs of different applications where the results of principled integration must be shown advantageous in practice in comparison with purely symbolic or purely connectionist systems.

The challenges for neural-symbolic integration today emerge from the goal of effective integration, expressive reasoning, and robust learning. Computationally, there are challenges associated with the more practical aspects of the application of neural-symbolic systems in areas such as engineering, robotics, semantic web, etc. These challenges include the effective computation of logical models, the efficient extraction of comprehensible knowledge and, ultimately, striking of the right balance between tractability and expressiveness.

In summary, by paying attention to the developments on either side of the division between the symbolic and the sub-symbolic paradigms, we are getting closer to a unifying theory, or at least promoting a faster and principled development of cognitive and computing sciences and AI. To us this is the ultimate goal of neural-symbolic integration together with the associated provision of neural-symbolic systems with expressive reasoning and robust learning capabilities.

References

- [1] Pinker S, Nowak MA, Lee JJ. The logic of indirect speech. *Proceedings of the National Academy of Sciences USA*. 2008;105:833–838.
- [2] Shastri L. Shruti: A neurally motivated architecture for rapid, scalable inference. In: Hammer B, Hitzler P, editors. *Perspectives of neural-symbolic integration*. (Studies in Computational Intelligence; Vol. 77). Springer; 2007. p. 183–203.
- [3] Sun R. Theoretical status of computational cognitive modeling. *Cognitive Systems Research*. 2009; 10(2):124–140.
- [4] Garcez A, Broda K, Gabbay DM. *Neural-symbolic learning systems: Foundations and applications*. Springer; 2002. *Perspectives in Neural Computing*.
- [5] Garcez A, Lamb LC, Gabbay DM. *Neural-symbolic cognitive reasoning*. Springer; 2009. *Cognitive Technologies*.
- [6] Valiant LG. A neuroidal architecture for cognitive computation. *Journal of the ACM (JACM)*. 2000; 47(5):854–882.
- [7] de Penning L, d'Avila Garcez A, Lamb LC, et al. An integrated neural-symbolic cognitive agent architecture for training and assessment in simulators. In: *Proceedings of the 6th International Workshop on Neural-Symbolic Learning and Reasoning, NeSy10, held at AAAI-2010*; 2010.
- [8] de Penning L, d'Avila Garcez A, Lamb LC, et al. A neural-symbolic cognitive agent for online learning and reasoning. In: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI'11)*; 2011.
- [9] Valiant LG. Knowledge infusion: In pursuit of robustness in artificial intelligence. In: *FSTTCS*; Cite-seer; 2008. p. 415–422.
- [10] Kowalski R. *Computational logic and human thinking: How to be artificially intelligent*. Cambridge University Press; 2011.
- [11] Pereira LM. Turing is among us. *Journal of Logic and Computation*. 2012;22(6):1257–1277.
- [12] Pnueli A. The temporal logic of programs. In: *Foundations of Computer Science, 1977., 18th Annual Symposium on*; 1977. p. 46–57.
- [13] Fagin R, Halpern J, Moses Y, et al. *Reasoning about knowledge*. MIT Press; 1995.
- [14] Van Dalen D. Intuitionistic logic. In: *Handbook of philosophical logic*. Springer; 2002. p. 1–114.
- [15] Baader F, Calvanese D, McGuinness DL, et al., editors. *The description logic handbook: Theory, implementation, and applications*. Cambridge University Press; 2003.
- [16] Bader S, Hitzler P. Dimensions of neural-symbolic integration: A structured survey. In: Artemov S, Barringer H, d'Avila Garcez A, et al., editors. *We will show them: Essays in honour of dov gabbay* (volume 1). King's College Publications; 2005. p. 167–194.
- [17] Hammer B, Hitzler P, editors. *Perspectives of neural-symbolic integration*. Springer; 2007.
- [18] Valiant LG. Three problems in computer science. *Journal of the ACM (JACM)*. 2003;50(1):96–99.
- [19] Towell GG, Shavlik JW. Knowledge-based artificial neural networks. *Artificial intelligence*. 1994; 70(1):119–165.

- [20] Tabor W. A dynamical systems perspective on the relationship between symbolic and non-symbolic computation. *Cognitive Neurodynamics*. 2009;3(4):415–427.
- [21] Siegelmann H. Neural networks and analog computation: Beyond the turing limit. Birkhäuser; 1999.
- [22] van Rooij I. The tractable cognition thesis. *Cognitive Science*. 2008;32:939–984.
- [23] van Emde Boas P. Machine models and simulations. In: *Handbook of Theoretical Computer Science* A. Elsevier; 1990.
- [24] Leitgeb H. Interpreted dynamical systems and qualitative laws: From neural networks to evolutionary systems. *Synthese*. 2005;146:189–202.
- [25] Hinton GE, Osindero S, Teh YW. A fast learning algorithm for deep belief nets. *Neural Computation*. 2006 Jul;18(7):1527–1554. Available from: <http://dx.doi.org/10.1162/neco.2006.18.7.1527>.
- [26] Pinkas G. Symmetric neural networks and propositional logic satisfiability. *Neural Computation*. 1991; 3:282–291.
- [27] Garcez A, Gabbay DM. Fibring neural networks. In: *Proceedings of 19th National Conference on Artificial Intelligence (AAAI-2004)*. AAAI Press; 2004. p. 342–347.
- [28] McCulloch WS, Pitts W. A logical calculus of the ideas immanent in neural nets. *Bulletin of Mathematical Biophysics*. 1943;5:115–133.
- [29] Browne A, Sun R. Connectionist inference models. *Neural Networks*. 2001;14:1331–1355.
- [30] Cloete I, Zurada J, editors. *Knowledge-based neurocomputing*. MIT Press; 2000.
- [31] Hölldobler S, Kalinke Y. Toward a new massively parallel computational model for logic programming. In: *Proceedings of the Workshop on Combining Symbolic and Connectionist Processing*, held at ECAI-94; 1994. p. 68–77.
- [32] Becker M, Mackay J, Dillaway B. Abductive authorization credential gathering. In: *Policies for Distributed Systems and Networks, 2009. POLICY 2009*. IEEE International Symposium on; 2009. p. 1–8.
- [33] Garcez A, Zaverucha G. The connectionist inductive learning and logic programming system. *Applied Intelligence*. 1999;11(1):59–77.
- [34] Thagard P, Stewart TC. The aha! experience: Creativity through emergent binding in neural networks. *Cognitive science*. 2011;35(1):1–33.
- [35] Sandercock J. Lessons for the construction of military simulators: a comparison of artificial intelligence with human-controlled actors. DSTO; 2004. TR-1614.
- [36] Heuvelink A. Cognitive models for training simulations (dissertation) [dissertation]. Amsterdam, The Netherlands: Dutch Graduate School for Information and Knowledge Systems (SIKS), Vrije Universiteit Amsterdam; 2009. Available from: <http://nbn-resolving.de/urn:nbn:de:bsz:14-qucosa-25468>.
- [37] van den Bosch K, Riemersma JB, Schifflett S, et al. Reflections on scenario-based training in tactical command. In: Schifflett S, Elliott L, E S, et al., editors. *Scaled worlds: Development, validation and applications*. Ashgate; 2004. p. 1–21.
- [38] Fernlund HK, Gonzalez AJ, Georgiopoulos M, et al. Learning tactical human behavior through observation of human performance. *Systems, Man, and Cybernetics, Part B: Cybernetics*, IEEE Transactions on. 2006;36(1):128–140.
- [39] Smith E, Kosslyn S. *Cognitive psychology: Mind and brain*. Upper Saddle River, NJ: Prentice-Hall Inc; 2006.
- [40] Wooldridge M. *An introduction to multiagent systems*. John Wiley & Sons; 2009.
- [41] Lehmann J, Bader S, Hitzler P. Extracting reduced logic programs from artificial neural networks. *Applied Intelligence*. 2010;32(3):249–266.
- [42] Lamb LC, Borges RV, Garcez A. A connectionist cognitive model for temporal synchronisation and learning. In: *Proceedings of the 22nd National Conference on Artificial Intelligence (AAAI-2007)*. AAAI Press; 2007. p. 827–832.
- [43] Smolensky P. Information processing in dynamical systems: Foundations of harmony theory. In: Rumelhat DE, McClelland JL, editors. *Parallel distributed processing: Volume 1: Foundations*. MIT Press; 1986.
- [44] Sutskever I, Hinton GE, Taylor GW. The recurrent temporal restricted boltzmann machine. In: *Advances in Neural Information Processing Systems*; 2009. p. 1601–1608.
- [45] Bratman M. *Intention, plans, and practical reason*. Cambridge University Press; 1999.
- [46] Chen H, Murray AF. Continuous restricted boltzmann machine with an implementable training algo-

- rithm. Vision, Image and Signal Processing, IEE Proceedings. 2003;150(3):153–158.
- [47] Sun R. A neural network model of causality. Neural Networks, IEEE Transactions on. 1994;5(4):604–611.
- [48] Pinkas G. Reasoning, nonmonotonicity and learning in connectionist networks that capture propositional knowledge. Artificial Intelligence. 1995;77(2):203–247.
- [49] de Penning L, Boot E, Kappé B. Integrating training simulations and e-learning systems: the simscorm platform. In: Proceedings: Interservice/Industry Training, Simulation, and Education Conference (IIT-SEC); 2008.
- [50] Donlon J. DARPA's mind's eye program: Broad agency announcement. Arlington, USA; 2010.
- [51] de Penning L, den Hollander R, Bouma H, et al. A neural-symbolic cognitive agent with a mind's eye. In: Proceedings of the AAAI Workshop on Neural-Symbolic Learning and Reasoning (NeSy) 2012; 2012.
- [52] de Penning L, d'Avila Garcez AS, Lamb LC, et al. Applying neural-symbolic cognitive agents in intelligent transport systems to reduce co2 emissions. In: 2014 International Joint Conference on Neural Networks (IJCNN); 2014. p. 55–62.
- [53] Salakhutdinov R, Hinton G. Deep boltzmann machines. In: Proceedings of the International Conference on Artificial Intelligence and Statistics; 2009. p. 448–455.
- [54] Tran S, Garcez A. Logic extraction from deep belief networks. In: In Proceedings of the ICML Workshop on Representation Learning, held at ICML-2012; 2012.
- [55] Johnson-Laird PN. Mental models: Towards a cognitive science of language, inference, and consciousness. Harvard University Press; 1983. (Cognitive Science Series; 6).
- [56] Thagard P. How brains make mental models. In: Model-based reasoning in science and technology. Springer; 2010. p. 447–461.
- [57] Nersessian N. Creating scientific concepts. MIT press; 2010.
- [58] Engel AK, Singer W. Temporal binding and the neural correlates of sensory awareness. Trends in cognitive sciences. 2001;5(1):16–25.
- [59] O'Reilly R, Busby R, Soto R. Three forms of binding and their neural substrates: Alternatives to temporal synchrony. In: Cleeremans A, editor. The unity of consciousness: Binding, integration, and dissociation. Oxford University Press Oxford; 2003. p. 168–192.
- [60] Bowman H, Wyble B. The simultaneous type, serial token model of temporal attention and working memory. Psychological review. 2007;114(1):38.
- [61] Rigotti M, Rubin DBD, Wang XJ, et al. Internal representation of task rules by recurrent dynamics: the importance of the diversity of neural responses. Frontiers in computational neuroscience. 2010;4.
- [62] Wyble B, Bowman A. A neural network account of binding discrete items into working memory using a distributed pool of flexible resources. Journal of Vision. 2006;6(6):33–33.
- [63] Newell A. Unified theories of cognition. Harvard University Press; 1994.
- [64] Evans JSB. In two minds: dual-process accounts of reasoning. Trends in cognitive sciences. 2003; 7(10):454–459.
- [65] Damasio A. Descartes' error: Emotion, reason and the human brain. Random House; 2008.
- [66] Scherer K. Appraisal theory. In: Dalglish T, Power M, editors. Handbook of cognition and emotion. John Wiley & Sons; 1999. p. 637–663.
- [67] Kieras D, Meyer D, Mueller S, et al. Insights into working memory from the perspective of the epic architecture for model skilled perceptual-motor and cognitive human performance. In: Miyake A, Shah P, editors. Models of working memory. Cambridge University Press; 1999. Available from: <http://dx.doi.org/10.1017/CB09781139174909.012>.
- [68] Anderson JR, Lebiere CJ. The atomic components of thought. Psychology Press; 2014.
- [69] Anderson JR, Fincham JM, Qin Y, et al. A central circuit of the mind. Trends in cognitive sciences. 2008;12(4):136–143.
- [70] Rumelhart DE, McClelland JL, PDP Research Group, editors. Parallel distributed processing: Explorations in the microstructure of cognition (vol. 1 & 2). MIT Press; 1986.
- [71] Hasselmo ME, Wyble BP. Free recall and recognition in a network model of the hippocampus: simulating effects of scopolamine on human memory function. Behavioural brain research. 1997;89(1):1–34.
- [72] Fransén E, Tahvildari B, Egorov AV, et al. Mechanism of graded persistent cellular activity of entorhinal cortex layer v neurons. Neuron. 2006;49(5):735–746.
- [73] Taatgen NA, Juvina I, Schipper M, et al. Too much control can hurt: A threaded cognition model of the attentional blink. Cognitive psychology. 2009;59(1):1–29.

- [74] Fodor JA, Pylyshyn ZW. Connectionism and cognitive architecture: A critical analysis. *Cognition*. 1988;28(1):3–71.
- [75] Fodor JA. Language, thought and compositionality. *Mind & Language*. 2001;16(1):1–15.
- [76] Duncan J, Parr A, Woolgar A, et al. Goal neglect and spearman's g: competing parts of a complex task. *Journal of Experimental Psychology: General*. 2008;137(1):131.
- [77] Dumontheil I, Thompson R, Duncan J. Assembly and use of new task rules in fronto-parietal cortex. *Journal of Cognitive Neuroscience*. 2011;23(1):168–182.
- [78] Baddeley A. The episodic buffer: a new component of working memory? *Trends in cognitive sciences*. 2000;4(11):417–423.
- [79] Norman DA, Shallice T. Attention to action. In: Davidson R, Schwartz G, Shapiro D, editors. *Consciousness and self-regulation*. Springer; 1986. p. 1–18.
- [80] Barnard PJ. Interacting cognitive subsystems: Modeling working memory phenomena within a multi-processor architecture. In: Miyake A, Shah P, editors. *Models of working memory*. Cambridge University Press; 1999. p. 298–339. Available from: <http://dx.doi.org/10.1017/CB09781139174909.012>.
- [81] Shallice T, Cooper R. *The organisation of mind*. Oxford University Press; 2011.
- [82] Pinker S, Ullman MT. The past and future of the past tense. *Trends in cognitive sciences*. 2002;6(11):456–463.
- [83] Page M. Connectionist modelling in psychology: A localist manifesto. *Behavioral and Brain Sciences*. 2000;23(04):443–467.
- [84] Bowers JS. Challenging the widespread assumption that connectionism and distributed representations go hand-in-hand. *Cognitive Psychology*. 2002;45(3):413–445.
- [85] Li Z. Computational design and nonlinear dynamics of a recurrent network model of the primary visual cortex. *Neural computation*. 2001;13(8):1749–1780.
- [86] Raizada RD, Grossberg S. Towards a theory of the laminar architecture of cerebral cortex: Computational clues from the visual system. *Cerebral Cortex*. 2003;13(1):100–113.
- [87] Serre T, Oliva A, Poggio T. A feedforward architecture accounts for rapid categorization. *Proceedings of the National Academy of Sciences*. 2007;104(15):6424–6429.
- [88] Burton AM, Bruce V, Johnston RA. Understanding face recognition with an interactive activation model. *British Journal of Psychology*. 1990;81(3):361–380.
- [89] Cowell R, Cottrell G. Virtual brain reading: A connectionist approach to understanding fmri patterns. *Journal of Vision*. 2009;9(8):472–472.
- [90] Itti L, Koch C, Niebur E. A model of saliency-based visual attention for rapid scene analysis. *IEEE Transactions on Pattern Analysis & Machine Intelligence*. 1998;20(11):1254–1259.
- [91] Heinke D, Humphreys GW. Attention, spatial representation, and visual neglect: Simulating emergent attention and spatial memory in the selective attention for identification model (SAIM). *Psychological Review*. 2003;110(1):29–87.
- [92] Bundesen C, Habekost T, Kyllingsbæk S. A neural theory of visual attention: bridging cognition and neurophysiology. *Psychological review*. 2005;112(2):291.
- [93] Bogacz R, Brown MW. Comparison of computational models of familiarity discrimination in the perirhinal cortex. *Hippocampus*. 2003;13(4):494–524.
- [94] Norman KA, O'Reilly RC. Modeling hippocampal and neocortical contributions to recognition memory: a complementary-learning-systems approach. *Psychological review*. 2003;110(4):611.
- [95] Davelaar E, Usher M. An extended buffer model for active maintenance and selective updating. In: *Proceedings of the 8th Neural Computatoin and Psychology Workshop*; Vol. 15. World Scientific; 2004. p. 3–14.
- [96] Bowman H, Su L. Cognition, concurrency theory and reverberations in the brain: in search of a calculus of communicating (recurrent) neural systems. In: Voronkov A, Korovina MV, editors. *HOWARD-60: A Festschrift on the Occasion of Howard Barringer's 60th Birthday*. EasyChair; 2014. p. 66–84.
- [97] Sloman A, Logan B. Building cognitively rich agents using the sim.agent toolkit. *Communications of the ACM*. 1999;42(3):71–ff.
- [98] Hodgkin AL, Huxley AF. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of physiology*. 1952;117(4):500–544.
- [99] O'Reilly RC, Munakata Y. *Computational explorations in cognitive neuroscience: Understanding the mind by simulating the brain*. MIT Press; 2000.
- [100] Marcus GF. *The algebraic mind: Integrating connectionism and cognitive science*. MIT press; 2003.

- [101] Van der Velde F, De Kamps M. Neural blackboard architectures of combinatorial structures in cognition. *Behavioral and Brain Sciences*. 2006;29(01):37–70.
- [102] Feldman J. The neural binding problem(s). *Cognitive neurodynamics*. 2013;7(1):1–11.
- [103] Anandan P, Letovsky S, Mjolsness E. Connectionist variable-binding by optimization. In: *Proceedings of the 11th Annual Conference of the Cognitive Science Society (CogSci-89)*; 1989.
- [104] Ballard DH. Parallel logical inference and energy minimization. In: *Proceedings of the Fifth National Conference on Artificial Intelligence (AAAI-1986)*. AAAI Press; 1986. p. 203–208.
- [105] Hölldobler S. A structured connectionist unification algorithm. In: *Proceedings of the 8th National Conference of the American Association on Artificial Intelligence (AAAI-1990)*; Vol. 90; 1990. p. 587–593.
- [106] Hölldobler S, Kurfess F. *Chcl: A connectionist inference system*. Springer; 1992.
- [107] Stolcke A. Unification as constraint satisfaction in structured connectionist networks. *Neural Computation*. 1989;1(4):559–567.
- [108] Plate T. Holographic reduced representations. *Neural networks, IEEE transactions on*. 1995;6(3):623–641.
- [109] Pollack JB. Recursive distributed representations. *Artificial Intelligence*. 1990;46(1):77–105.
- [110] Stewart TC, Eliasmith C. Building production systems with realistic spiking neurons. In: *Proceedings of the 30th annual meeting of the Cognitive Science Society*. Austin, TX: Cognitive Science Society; 2008.
- [111] Barrett L, Feldman JA, Dermed ML. A (somewhat) new solution to the binding problem. *Neural Computation*. 2008;20:2361–2378.
- [112] Jackendoff R. *Foundations of language: Brain, meaning, grammar*. Oxford University Press; 2002.
- [113] Browne A, Sun R. Connectionist variable binding. *Expert Systems*. 1999;16(3):189–207.
- [114] Shastri L, Ajanagadde V. From simple associations to systematic reasoning: A connectionist representation of rules, variables and dynamic bindings using temporal synchrony. *Behavioral and brain sciences*. 1993;16(03):417–451.
- [115] Pinkas G. Constructing syntactic proofs in symmetric networks. In: *Advances in Neural Information Processing Systems (NIPS-91)*; 1991. p. 217–224.
- [116] Lima P. Resolution-based inference on artificial neural networks [dissertation]. London, UK: Imperial College London; 2000. Available from: <http://nbn-resolving.de/urn:nbn:de:bsz:14-qucosa-25468>.
- [117] Pinkas G, Lima P, Cohen S. Dynamic binding mechanism for retrieving and unifying complex predicate-logic knowledge. In: *Proceedings of the International Conference on Artificial Neural Networks (ICANN) 2012*; 2012.
- [118] Pinkas G, Lima P, Cohen S. Representing, binding, retrieving and unifying relational knowledge using pools of neural binders. *Journal of Biologically Inspired Cognitive Architectures (BICA)*. 2013;6:87–95.
- [119] Clarke E, Biere A, Raimi R, et al. Bounded model checking using satisfiability solving. *Formal Methods in System Design*. 2001;19(1):7–34.
- [120] Domingos P. Markov logic: a unifying language for knowledge and information management. In: *Proceedings of the 17th ACM conference on Information and knowledge management*; ACM; 2008. p. 519–519.
- [121] Pinkas G, Dechter R. Improving connectionist energy minimization. *Journal of Artificial Intelligence Research*. 1995;3:223–248.
- [122] Bornscheuer SE. Rational models of normal logic programs. In: Götz G, Hölldobler S, editors. *Ki-96: Advances in artificial intelligence*. (Lecture Notes in Computer Science; Vol. 1137). Springer Berlin Heidelberg; 1996. p. 1–4.
- [123] Bader S. Extracting propositional rules from feed-forward neural networks by means of binary decision diagrams. In: Garcez A, Hitzler P, editors. *Proceedings of the 5th International Workshop on Neural-Symbolic Learning and Reasoning, NeSy'09, held at IJCAI-09*; 2009. p. 22–27. Available from: <http://ceur-ws.org/Vol-481/paper-5.pdf>.
- [124] Bader S, Hitzler P, Hölldobler S, et al. A fully connectionist model generator for covered first-order logic programs. In: Veloso MM, editor. *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI-07)*. AAAI Press; 2007. p. 666–671. Available from: <http://www.ijcai.org/papers07/Papers/IJCAI07-106.pdf>.
- [125] Bader S, Hitzler P. Logic programs, iterated function systems, and recurrent radial basis function net-

- works. *Journal of Applied Logic*. 2004;2(3):273–300.
- [126] Bader S, Hitzler P, Garcez A. Computing first-order logic programs by fibring artificial neural networks. In: Russell I, Markov Z, editors. *Proceedings of the 18th International Florida Artificial Intelligence Research Symposium Conference, FLAIRS05*; 2005.
 - [127] Funahashi KI. On the approximate realization of continuous mappings by neural networks. *Neural Networks*. 1989;2(3):183–192.
 - [128] Hölldobler S, Kalinke Y, Störr HP. Approximating the semantics of logic programs by recurrent neural networks. *Applied Intelligence*. 1999;11(1):45–58.
 - [129] Hitzler P, Hölldobler S, Seda AK. Logic programs and connectionist networks. *Journal of Applied Logic*. 2004;2(3):245 – 272.
 - [130] Bader S, Hitzler P, Witzel A. Integrating first-order logic programs and connectionist systems - a constructive approach. In: Garcez A, Elman J, Hitzler P, editors. *Proceedings of the 1st International Workshop on Neural-Symbolic Learning and Reasoning, NeSy'05, held at IJCAI-05*; 2005.
 - [131] Bader S. *Neural-symbolic integration (dissertation) [dissertation]*. Dresden, Germany: Technische Universität Dresden; 2009. Available from: <http://nbn-resolving.de/urn:nbn:de:bsz:14-qucosa-25468>.
 - [132] Bader S, Hölldobler S, Marques NC. Guiding backprop by inserting rules. In: Garcez A, Hitzler P, editors. *Proceedings of the 4th International Workshop on Neural-Symbolic Learning and Reasoning, NeSy'08, held at ECAI-08; (CEUR Workshop Proceedings; Vol. 366)*; 2008. Available from: <http://ceur-ws.org/Vol-366/paper-5.pdf>.
 - [133] Muggleton S, de Raedt L. Special issue: Ten years of logic programming inductive logic programming: Theory and methods. *The Journal of Logic Programming*. 1994;19:629–679.
 - [134] Basilio R, Zaverucha G, Barbosa VC. Learning logic programs with neural networks. In: *11th International Conference on Inductive Logic Programming (ILP) 2001; (Lecture Notes in Artificial Intelligence; Vol. 2157)*; 2001. p. 15–26.
 - [135] Krogel MA, Rawles S, Zelezny F, et al. Comparative evaluation of approaches to propositionalization. In: Horvath T, Yamamoto A, editors. *Inductive logic programming. (Lecture Notes in Computer Science; Vol. 2835)*. Springer; 2003. p. 197–214.
 - [136] Franca M, Zaverucha G, Garcez A. Fast relational learning using bottom clause propositionalization with artificial neural networks. *Machine Learning*. 2014;94(1):81–104. Available from: <http://dx.doi.org/10.1007/s10994-013-5392-1>.
 - [137] Sourek G, Aschenbrenner V, Zelezny F, et al. Lifted relational neural networks. In: *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2015; (CEUR Workshop Proceedings; Vol. 1583)*. CEUR-WS.org; 2015.
 - [138] de Raedt L, Kersting K, Natarajan S, et al. *Statistical relational artificial intelligence: Logic, probability, and computation*. Morgan and Claypool Publishers; 2016. *Synthesis Lectures on Artificial Intelligence and Machine Learning*.
 - [139] Vardi MY. Why is modal logic so robustly decidable? In: *Descriptive Complexity and Finite Models, Proceedings of a DIMACS Workshop 1996, Princeton, New Jersey, USA, January 14-17, 1996*; 1996. p. 149–184.
 - [140] Richardson M, Domingos P. Markov logic networks. *Machine Learning*. 2006;62:107–136.
 - [141] Domingos P, Lowd D. Markov logic: An interface layer for artificial intelligence. Morgan & Claypool; 2009.
 - [142] Taskar B, Chatalbashev V, Koller D. Learning associative markov networks. In: *Proceedings of the 21st International Conference on Machine Learning (ICML-2004)*. ACM; 2004. p. 102–111.
 - [143] Braz R, Amir E, Roth D. Lifted first-order probabilistic inference. In: *Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (IJCAI-2005)*; Edinburgh, UK. Morgan Kaufmann; 2005. p. 1319–1325.
 - [144] Singla P, Domingos P. Lifted first-order belief propagation. In: *Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI-2008)*. AAAI Press; 2008. p. 1094–1099.
 - [145] Riedel S. Improving the accuracy and efficiency of MAP inference for Markov logic. In: *Proceedings of the Twenty-Fourth Conference on Uncertainty in Artificial Intelligence (UAI-2008)*; Helsinki, Finland. AUAI Press; 2008. p. 468–475.
 - [146] Singla P, Domingos P. Memory-efficient inference in relational domains. In: *Proceedings of the 21st National Conference on Artificial Intelligence (AAAI-2006)*. AAAI Press; 2006. p. 488–493.
 - [147] Poon H, Domingos P, Sumner M. A general method for reducing the complexity of relational inference

- and its application to mcmc. In: Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI 2008); 2008.
- [148] Kiddon C, Domingos P. Coarse-to-fine inference and learning for first-order probabilistic models. In: Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence (AAAI-2011); San Francisco, CA. AAAI Press; 2011.
 - [149] Tieleman T. Training restricted Boltzmann machines using approximations to the likelihood gradient. In: Proceedings of the Twenty-Fifth International Conference on Machine Learning (ICML-2008); ACM; 2008. p. 1064–1071.
 - [150] Lowd D, Domingos P. Efficient weight learning for Markov logic networks. In: Proceedings of the Eleventh European Conference on Principles and Practice of Knowledge Discovery in Databases; Warsaw, Poland. Springer; 2007. p. 200–211.
 - [151] Besag J. Statistical analysis of non-lattice data. *The Statistician*. 1975;24:179–195.
 - [152] Poon H, Domingos P. Sum-product networks: A new deep architecture. In: Proceedings of the Twenty-Seventh Conference on Uncertainty in Artificial Intelligence (UAI) 2011; 2011. p. 337–346.
 - [153] Nath A, Domingos P. Learning relational sum-product networks. In: Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence (AAAI-2015); Austin, TX. AAAI Press; 2015.
 - [154] Besold TR. Human-level artificial intelligence must be a science. In: Kühnberger KU, Rudolph S, Wang P, editors. *Artificial general intelligence*. (Lecture Notes in Computer Science; Vol. 7999). Springer; 2013. p. 174–177.
 - [155] Gust H, Kühnberger KU, Geibel P. Learning models of predicate logical theories with neural networks based on topos theory. In: *Perspectives of neural-symbolic integration*. Springer; 2007. p. 233–264.
 - [156] Maass W. Paradigms for computing with spiking neurons. In: *Models of neural networks iv*. Springer; 2002. p. 373–402.
 - [157] Jaeger H. Controlling recurrent neural networks by conceptors; 2014. arXiv. 1403.3369v1 [cs.NE], 13 Mar 2014.
 - [158] Von Neumann J. First draft of a report on the EDVAC; 1945.
 - [159] Graves A, Wayne G, Danihelka I. Neural turing machines; 2014. arXiv. 1410.5401v2 [cs.NE], 10 Dec 2014.
 - [160] Weston J, Chopra S, Bordes A. Memory networks. In: Bengio Y, LeCun Y, editors. 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings; 2015.
 - [161] Vinyals O, Toshev A, Bengio S, et al. Show and tell: A neural image caption generator. In: IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015. IEEE Computer Society; 2015. p. 3156–3164.
 - [162] Karpathy A, Fei-Fei L. Deep visual-semantic alignments for generating image descriptions. *IEEE Trans Pattern Anal Mach Intell*. 2017;39(4):664–676.
 - [163] Reed SE, Zhang Y, Zhang Y, et al. Deep visual analogy-making. In: Cortes C, Lawrence N, Lee D, et al., editors. *Advances in neural information processing systems* 28. Curran Associates, Inc.; 2015. p. 1252–1260. Available from: <http://papers.nips.cc/paper/5845-deep-visual-analogy-making.pdf>.
 - [164] Foerster JN, Assael YM, de Freitas N, et al. Learning to communicate with deep multi-agent reinforcement learning. In: Lee DD, Sugiyama M, von Luxburg U, et al., editors. *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016*, December 5-10, 2016, Barcelona, Spain; 2016. p. 2137–2145.
 - [165] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Computation*. 1997;9:1735–1780.
 - [166] Silver D, Huang A, Maddison C, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*. 2016;529:484–489.
 - [167] Cutsuridis V, Hussain A, Taylor JG, editors. *Perception-action cycle: Models, architectures, and hardware*. Springer; 2011.
 - [168] Besold TR, Kühnberger KU, Garcez Ad, et al. Anchoring knowledge in interaction: Towards a harmonic subsymbolic/symbolic framework and architecture of computational cognition. In: Bieger J, Goertzel B, Potapov A, editors. *Artificial general intelligence*. (Lecture Notes in Computer Science; Vol. 9205). Springer; 2015. p. 35–45.
 - [169] Besold TR, Kühnberger KU. Towards integrated neural-symbolic systems for human-level ai: Two research programs helping to bridge the gaps. *Biologically Inspired Cognitive Architectures*. 2015; 14:97–110.

- [170] Coradeschi S, Saffiotti A. Anchoring symbols to sensor data: Preliminary report. In: Proceedings of the 17th AAAI Conference. AAAI Press; 2000. p. 129–135.
- [171] Bundy A. The interaction of representation and reasoning. In: Proceedings of the Royal Society A; Vol. 469; 2013. p. 1–18.
- [172] Karpathy A, Johnson J, Fei-Fei L. Visualizing and understanding recurrent networks; 2015. arXiv. 1506.02078v2 [cs.LG], 17 Nov 2015.
- [173] Sun R. Hybrid systems and connectionist implementationism. In: Encyclopedia of cognitive science. Nature Publishing Group (MacMillan); 2002. p. 697–703.
- [174] Sun R. Duality of the mind: A bottom-up approach towards cognition. Lawrence Erlbaum Associates; 2003.
- [175] Besold TR, Mandl S. Integrating logical and sub-symbolic contexts of reasoning. In: Filipe J, Fred ALN, Sharp B, editors. Proceedings of the International Conference on Agents and Artificial Intelligence (ICAART) 2010. INSTICC Press; 2010. p. 494–497.
- [176] Garcez A, Besold TR, de Raedt L, et al. Neural-Symbolic Learning and Reasoning: Contributions and Challenges. In: AAAI Spring 2015 Symposium on Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches; (AAAI Technical Reports; Vol. SS-15-03). AAAI Press; 2015.
- [177] Ji S, Xu W, Yang M, et al. 3d convolutional neural networks for human action recognition. Pattern Analysis and Machine Intelligence, IEEE Transactions on. 2013;35(1):221–231.
- [178] Valiant LG. A theory of the learnable. Communications of the ACM. 1984;27(11):1134–1142.

Chapter 2

Symbolic Reasoning in Latent Space: Classical Planning as an Example

Masataro Asai, MIT-IBM Watson AI Lab, IBM Research Cambridge

Hiroshi Kajino, IBM Research Tokyo

Alex Fukunaga, Graduate School of Arts and Sciences, University of Tokyo

Christian Muise, School of Computing, Queen's University

Recently, significant advances have been made in neural network (NN) deep learning approaches for perceptually-based cognitive tasks including image classification [1], object recognition [2], speech recognition [3], machine translation as well as NN-based problem-solving systems [4, 5]. However, the current state-of-the-art, pure NN-based systems do not yet provide guarantees provided by symbolic systems, such as deterministic completeness and solution optimality, suffer from sample efficiency issues where a large amount of training data is required.

Meanwhile, recent advances in the theoretical and empirical understanding of various combinatorial problems led to highly-scalable, industrial-strength state-of-the-art combinatorial solvers in a variety of fields, including Answer Set Programming (ASP) [6], Constraint Programming (CP) [7], Boolean Satisfiability (SAT) or Maximum Satisfiability (MAXSAT) [8], Satisfiability Modulo Theories (SMT) [9], General Game Playing [10], or Classical Planning [11].

However, inputs to these symbolic systems need to be provided in a structured, symbolic representation, and in general, such symbolic models need to be provided by a human, either directly in a modeling language designed for each paradigm, or via a compiler which transforms some other symbolic problem representation into the target language. For example, General Game Playing requires a Game Description Language (GDL) input, and Classical Planning requires a Planning Domain Description Language (PDDL) input. This results in the *knowledge-acquisition bottleneck*, where the modeling step is sometimes the bottleneck in the problem-solving cycle. In addition, the requirement for symbolic input poses a significant obstacle to using symbolic systems in *new, unforeseen* situations where no human is available to create such a model or a generator, e.g., autonomous spacecraft exploration. In particular, this first requires generating symbols from raw sensor input, i.e., the *symbol grounding problem* [12]. Using a

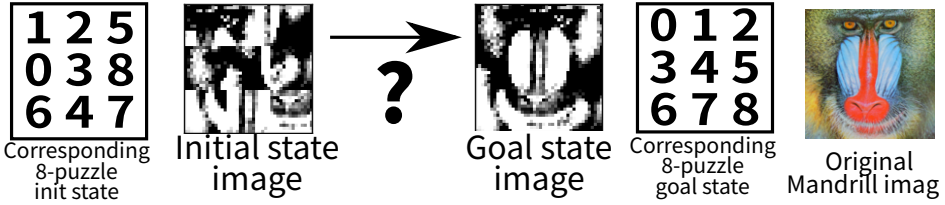


Figure 1. An image-based 8-puzzle.

NN-based perceptual system to *automatically* provide input models for symbolic systems could greatly expand the applicability of symbolic solvers and offer the benefits of both paradigms. *We consider the problem of robustly, automatically map between such subsymbolic representations and the formal specifications required by symbolic systems.*

This chapter demonstrates the feasibility of this approach in Classical Planning problems. Figure 1 (left) shows a scrambled, 3x3 tiled version of the photograph on the right, i.e., an image-based instance of the 8-puzzle. Even for humans, this photograph-based task is arguably more difficult to solve than the standard 8-puzzle (leftmost) because of the distracting visual aspects. We seek a domain-independent system which, given only a set of unlabeled images showing the valid moves for this image-based puzzle, finds an optimal solution to the puzzle (Figure 2). Although the 8-puzzle is trivial for symbolic planners, solving this image-based problem with a domain-independent system which (1) *has no prior assumptions/knowledge* (e.g., “sliding objects”, “tile arrangement”), and (2) *must acquire all knowledge from the images*, is nontrivial. Such a system should not make assumptions about the image (e.g., “a grid-like structure”). The only assumption allowed about the nature of the task is that it can be modeled as a classical planning problem (deterministic and fully observable).

We introduce Latent-space Planner (Latplan) [13, 14, 15], an architecture that automatically generates a symbolic problem representation that can be used as the input for a classical planner. Latplan is a *deep discrete generative model* trained on a set of subsymbolic inputs (*unlabeled images* of the environment) in an unsupervised manner. Then, given a planning problem instance as a pair of initial and goal images such as Figure 1, it maps the problem to a symbolic planning instance, invokes a planner, then visualizes the plan execution as a sequence of images.

A major takeaway in this chapter is that a naive machine learning model that is not aimed at neuro-symbolic integration will suffer from the *Symbol Stability Problem* (SSP) and *non-conformance to the syntactical restrictions in the target formal language*. Unlike typical Machine Learning tasks, **merely having a model that achieves high accuracy is insufficient for Neuro-Symbolic tasks**. We show how these issues can be addressed using the *Cube-Space AutoEncoder* neural network.

2.1. Backgrounds

2.1.1. Processes Required for Obtaining a Classical Planning Model

Let $\mathcal{F}(V)$ be a propositional formula consisting of logical operations $\{\wedge, \neg\}$, constants $\{\top, \perp\}$, and a set of propositional variables V . We define a grounded (propositional) STRIPS Planning problem as a 4-tuple $\langle P, A, I, G \rangle$ where P is a set of propositions, A

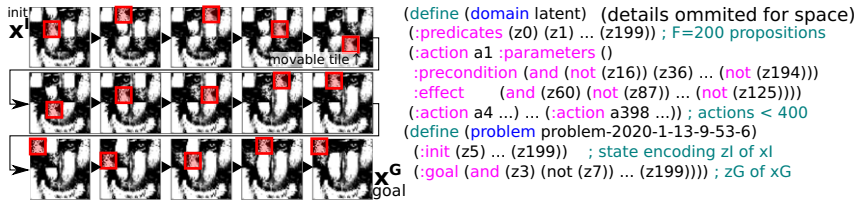


Figure 2. (left) A 14-step optimal plan for a 15-puzzle instance generated by our system using off-the-shelf Fast Downward with h^{LMcut} heuristics using the PDDL generated by our system. (right) The intermediate PDDL output from our NN-based system.

is a set of actions, $I \subseteq P$ is the initial state, and $G \subseteq P$ is a goal condition. Each action $a \in A$ is a 3-tuple $\langle \text{PRE}(a), \text{ADD}(a), \text{DEL}(a) \rangle$ where $\text{PRE}(a) \in \mathcal{F}(P)$ is a precondition and $\text{ADD}(a), \text{DEL}(a)$ are the add-effects and delete-effects, where $\text{ADD}(a) \cap \text{DEL}(a) = \emptyset$. A complete state (or just state) $s \subseteq P$ is a set of true propositions where those in $P \setminus s$ are presumed to be false. A partial state is similarly represented (i.e. a subset of P), but those propositions not mentioned may be either true or false. The initial state (I) is a complete state while the goal condition (G) and effect sets ($\text{PRE}(a), \text{ADD}(a)$, and $\text{DEL}(a)$) are partial states. We say that a state s entails a partial state ps when $ps \subseteq s$ – intuitively, every proposition that must be true in ps is true in the state s . An action a is *applicable* when s entails $\text{PRE}(a)$, and applying an action a to s yields a new successor state $a(s) = (s \setminus \text{DEL}(a)) \cup \text{ADD}(a)$. The task of classical planning is to find a *plan* $(a_1, \dots, a_n)$ which satisfies G by the repeated application of applicable actions, i.e., $G \subseteq a_n \circ \dots \circ a_1(I)$.

Traditionally, STRIPS models are implemented by humans, and are encoded in modeling languages such as PDDL [11]. The need for human involvement results in the so-called Knowledge Acquisition Bottleneck [16], which refers to the high cost of human involvement in encoding real-world problems into inputs that can be processed by symbolic AI systems. Ideally, STRIPS models should be learned/generated by the machine itself, eliminating the Knowledge Acquisition Bottleneck. In order to fully automatically acquire symbolic models for Classical Planning, **Symbol Grounding** and **Action Model Acquisition** (AMA) are necessary.

Symbol Grounding is an unsupervised process of establishing a mapping from noisy, continuous and/or unstructured inputs to a set of discrete, identifiable entities, i.e., symbols. For example, PDDL has six kinds of symbols: Objects, predicates, propositions, actions, problems, and domains (Table 1). Each type of symbol requires its own mechanism for grounding. For example, the large body of work in the image processing community on recognizing objects (e.g., individual faces) and their attributes (male, female) in images, or scenes in videos (e.g., cooking), can be viewed as corresponding to grounding the object, predicate and action symbols, respectively.

In contrast, an **Action Model** is a symbolic data structure representing the causality in the transitions of the world. In models expressible in the STRIPS subset of PDDL, this consists of preconditions and effects. In more expressive subsets of PDDL and related modeling languages, the action model may further contain, e.g., temporally extended conditions/effects [17], numeric conditions/effects [18], or probability of each outcome [19]. Action models have been referred to by other names in other fields. In the field of control theory / optimal control, where “control” is synonymous with “action” in AI literature, such a model is called a control-input model [20]. In the Model-based Reinforcement Learning literature, such a model is just called a “model” or sometimes “dynamics”.

Types of symbols	
Object symbols	panel7, x₀, y₀ ...
Predicate symbols	(empty ?x ?y) (up ?y₀ ?y₁)
Propositions	empty₅ = (empty x₂ y₁)
Action symbols	(slide-up panel7 x₀ y₁)
Problem symbols	eight-puzzle-instance1504, etc.
Domain symbols	eight-puzzle, hanoi

Table 1. 6 types of symbols in a PDDL definition.

The process of obtaining the action model through learning or other automated means is called Action Model Acquisition (AMA) [21, 22], Planning Operator Acquisition [23], or Action Model Learning [24].

2.1.2. Symbol Grounding

The Physical Symbol Systems Hypothesis [25, 26, 27] states that “[a] physical symbol system has the necessary and sufficient means for general intelligent action,” i.e., symbols are necessary for performing intelligent actions, and symbols are all that are required for performing intelligent actions. In planning, symbolic manipulation enables the encoding of powerful, *domain-independent knowledge* that can be easily applied to multiple tasks without training data. For example, given a problem instance from a previously unseen domain D , a planning algorithm can often solve an instance of D much faster than blind search by using domain-independent heuristic functions (e.g., [28, 29]) that exploit D based purely on the symbolic structure of the action model of D .

In contrast, current learning-based approaches to planning such as AlphaZero [30] achieve impressive performance by using or generating massive amounts of data in order to learn task-dependent evaluation functions and policies that result in high performance on a given task. MuZero [31] goes further and learns the action model. Transferring domain-independent strategies across tasks remains a challenge for learning-based approaches, as they currently lack a convenient, symbolic representation (formal language) for expressing and exchanging task-independent knowledge.

A system that exploits task-independent symbolic knowledge but operates in a real-world (e.g., physical) environment while avoiding the Knowledge Acquisition Bottleneck requires that the agent must be able to define and generate the symbolic representation of the environment by itself, without human input. This *symbol grounding problem* [32, 12, 33] is one of the key challenges in achieving autonomous symbolic systems for real-world environments and in Artificial Intelligence.

To further clarify what we mean by the term “symbol grounding,” which we use to describe each component of our system, we must first provide a clear definition of what we mean by “symbols.” We adopt a traditional, implementation-oriented definition of symbols such as those seen in the LISP family of programming languages:

Definition 1. A symbol is a structure that contains a pointer to a name and an object, and can be uniquely looked up in a knowledge base using the name as a key.

A name must be a literal from a set over which a *computable* equivalence relation is defined, i.e., for any element $i, j \in S$, $(i = j) \vee (i \neq j)$ and $\neg((i = j) \wedge (i \neq j))$, which guarantees that the symbols are always *identifiable*. This allows a name to be an element

of any subset of countable, potentially infinite sets, such as mathematical integers and ASCII strings (finite sequence of characters, where the character set is finite).

An *object* is a structured memory location in the programming languages / computer systems sense, as in “in-memory object” / “persistent object store”, rather than objects in STRIPS/PDDL. Conceptually, an object represents a meaning of the symbol. Objects are sometimes also called *referents* and the fact that a name is associated to the objects by a symbol is called *designation* [25].

A *knowledge base* is a namespace (e.g., implemented as a hash table). Symbols in different namespaces (such as the predicate namespace and the action namespace) may point to a different object even though they share the same name.

For example, an *action symbol* *move* can be looked up from the knowledge base (namespace) of actions *A* using the string “move” as the name. The symbol points to a tuple object that contains the preconditions and the effects. Similarly, a *predicate symbol* *on* can be looked up by the name “on” in the predicate knowledge base *P*, and points to a function object of two arguments, $\text{on}(?x, ?y)$, which takes two object representations and returns a truth/boolean value. We do not distinguish whether such a function object is hand-coded or learned; thus such a function can be a binary classifier learned by a machine learning system. Finally, at the same time, a predicate symbol *move* may refer to a static unary predicate (a PDDL type) representing a chess move, such as $\text{move}(\text{pawn-move-two-squares}) = \top$ without conflicting the action symbol *move*.

Discreteness is one of the characteristic aspects of symbols. However, being *apparently* discrete does not automatically qualify something as a proper name of a symbol, and this is encoded into the requirement of identifiability of names. To illustrate the significance of this requirement, we use an interesting phenomenon that some mathematical reals cannot be used as a name of a symbol, while it can be used as an object it points to. This is because the comparison between real numbers in infinite precision is undecidable in Turing Machine [34]. Imagine a symbol representing a mathematical constant π . The symbol π is named by a Greek letter “ π ”, while its actual instance, which cannot be printed out, could be represented in several implicit discrete forms, such as **D1**: a string “half of the length of a circumference of a unit circle,” or **D2**: an x86 binary of a compiled function/procedure that computes its value. Although these implicit forms are discrete, there is no general algorithm that computes the high-level equivalence of **D1** and **D2** being the same mathematical constant because such an algorithm can encode a halting problem (*functional equivalence is undecidable*), thus **D1** and **D2**, which represents the constant itself, cannot be used as names.

Inability to identify the equivalence between two keys relates to *Symbol Stability Problem* (Section 2.2.1), where the same real-world state may result in a different symbolic representation each time the agent performs an observation. For example, if both **D1** and **D2** are used as names of two symbols, a perception system that observed a mathematical value of π may return either **D1** or **D2** — its behavior becomes unpredictable. As we see later, this significantly affects the efficiency of algorithms such as A^* .

Symbol grounding comprises two tasks: *Grounding* of symbols, which deals with assigning an object (“meaning”) to an existing symbol, and *generation* of symbols (GENSYM), which deals with symbol creation. Although a symbol grounding system may perform both grounding and generation in one pass, the distinction between grounding and generation has been recognized since the early literature on symbol grounding. For example, [32] claims that “symbolic functions would *emerge* [emphasis added] as

an intrinsically ‘dedicated’ symbol system [i.e., generation] as a consequence of bottom-up grounding of category names in their sensory representations.” [33] claim that systems that can ground but cannot generate symbols by themselves are insufficient because the knowledge learned for a given symbol (e.g., on) are said to be *parasitic* to human knowledge.

2.1.3. Autoencoders, Variational Autoencoders, Discrete Variational Autoencoders

An Autoencoder (AE) is a type of neural network that learns a pair of an encoder and a decoder whose composition becomes an identity function [35]. Autoencoders consist of an input $\mathbf{x}$, a *latent vector* $\mathbf{z}$, an output $\hat{\mathbf{x}}$, an encoder f , and a decoder g , where $\mathbf{z} = f(\mathbf{x})$ and $\hat{\mathbf{x}} = g(\mathbf{z}) = g(f(\mathbf{x}))$. The encoder f and decoder g are trained so that $\mathbf{x} \approx g(f(\mathbf{x}))$ holds for any $\mathbf{x}$. The output is also called a *reconstruction*. The dimension of $\mathbf{z}$ is typically smaller than the input; thus $\mathbf{z}$ is considered a compressed representation of $\mathbf{x}$.

A Variational Autoencoder (VAE) is an autoencoder that additionally assumes that the latent variable $\mathbf{z}$ follows a certain distribution $p(\mathbf{z})$, typically called a *prior distribution*. It is chosen arbitrarily by a model designer, such as Normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{1})$. A VAE is trained by minimizing a sum of a reconstruction loss $-\log p(\mathbf{x}|\mathbf{z})$ and a KL divergence $D_{\text{KL}}(q(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$ between $q(\mathbf{z}|\mathbf{x})$ and $p(\mathbf{z})$, where $q(\mathbf{z}|\mathbf{x})$ is a distribution of the latent vector $\mathbf{z}$ obtained by the encoder at hand.

A negative sum of the reconstruction loss and the KL divergence is called an *Evidence Lower BOund* (ELBO), a lower bound of the log likelihood $\log p(\mathbf{x})$, also known as the evidence [36]. ELBO is derived by the *variational method* which treats q as a *variational distribution*, sees $q(\mathbf{z}|\mathbf{x})$ as an approximation of $p(\mathbf{z}|\mathbf{x})$, and applies Jensen’s inequality:

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \langle \log p(\mathbf{x}|\mathbf{z}) \rangle - D_{\text{KL}}(q(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})). \quad (1)$$

VAEs are theoretically more appealing than AEs because of this lower bound. Since the loss function of an AE (= reconstruction loss $\log p(\mathbf{x}|\mathbf{z})$) does not have this lower bound property, maximizing $\log p(\mathbf{x}|\mathbf{z})$ has no guarantee that it maximizes $\log p(\mathbf{x})$.

While typically $p(\mathbf{z})$ is a Normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{1})$ for continuous latent variables, there are multiple VAE methods for discrete latent distributions. One example is Gumbel-Softmax (GS) VAE [37], which was independently discovered by [38] as Concrete VAE. The binary special case of Gumbel Softmax is called Binary-Concrete (BC) VAE [38]. These VAEs use a discrete, uniform categorical/Bernoulli(0.5) distribution as the prior $p(\mathbf{z})$, and further approximate it with a continuous relaxation by introducing a temperature parameter τ that is annealed down to 0. In this section, we briefly summarize the information necessary for implementing GS and BC.

For Gumbel Softmax and Binary Concrete, we denote corresponding activation functions in the latent space as $\text{GS}_\tau(\mathbf{l})$ and $\text{BC}_\tau(l)$, where $\mathbf{l} \in \mathbb{R}^C$ and $l \in \mathbb{R}$ where C denotes the number of classes represented by Gumbel Softmax. $\mathbf{l}$ and l can be an arbitrary vector/scalar produced by the encoder neural network f . Both functions use a temperature parameter τ which is annealed during the training in a fixed schedule such as an exponential schedule $\tau(t) = Ar^{-(t-t_0)}$, where t is a training epoch.

GS and BC are defined as

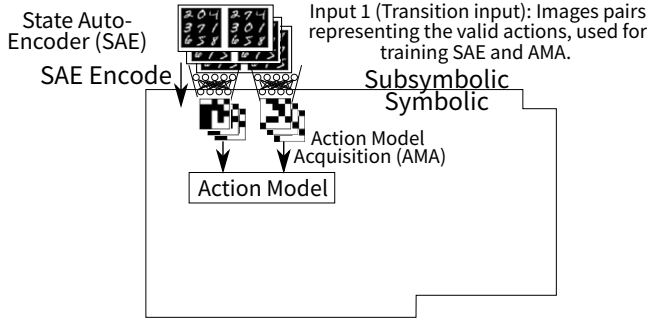


Figure 3. The Training phase of Latplan. We use the learned State Autoencoder (Section 2.2.1) to convert pairs of images (*pre, suc*) first to symbolic transitions, from which the AMA component generates an action model.

$$\text{GS}_\tau(l) = \text{SOFTMAX}\left(\frac{l + \text{GUMBEL}^C(0, 1)}{\tau}\right),$$

$$\text{BC}_\tau(l) = \text{SIGMOID}\left(\frac{l + \text{LOGISTIC}(0, 1)}{\tau}\right)$$

where $\text{GUMBEL}^C(0, 1) = -\log(-\log u)$ and $u \sim \text{UNIFORM}^C(0, 1) \in \mathbb{B}^C$, as well as $\text{LOGISTIC}(0, 1) = \log u - \log(1 - u)$ and $u \sim \text{UNIFORM}(0, 1) \in \mathbb{B}$. Both functions converge to discrete functions at the limit $\tau \rightarrow 0$: $\text{GS}_\tau(l) \rightarrow \arg\max(l)$ and $\text{BC}_\tau(l) \rightarrow \text{STEP}(l) = (l < 0) ? 0 : 1$. Note that we assume $\arg\max$ returns a one-hot representation rather than the index of the maximum value in order to maintain dimensional compatibility with SOFTMAX function. In practice, GS-VAEs and BC-VAEs contain multiple latent vectors / latent scalars to model complex inputs, which we call n -way Gumbel-Softmax / Binary-Concrete. The latent space is denoted as $\mathbf{z} \in \mathbb{B}^{n \times C}$ for Gumbel-Softmax, and $\mathbf{z} \in \mathbb{B}^n$ for Binary Concrete.

2.2. Latplan: Overview of the System Architecture

Latplan is a framework for *domain-independent classical planning in environments where symbolic input (PDDL models) are unavailable and only subsymbolic sensor input (e.g., images) are accessible*. Latplan operates in two phases: The training phase and the planning phase. In the former, it learns state representations and transition rules of the environment using deep neural networks entirely from subsymbolic data (e.g., image-based observations). In the latter, it receives subsymbolic representations (images) of the start and goal states as input and solves the problem using a classical planner. While we present an image-based implementation (“data” = images), the architecture itself does not make such assumptions and could be applied to the other formats such as audio/text.

In the training phase (Figure 3), Latplan is trained on *transition input* X : a set of pairs of raw data randomly sampled from the environment. The i -th pair in the dataset $\mathbf{x}^i = (\mathbf{x}^{i,0}, \mathbf{x}^{i,1}) \in X$ is a transition from an observation $\mathbf{x}^{i,0}$ to another observation $\mathbf{x}^{i,1}$ as a result of some high-level action. These high-level actions are *unknown* to Latplan, in that no symbolic representation of the action is available to Latplan.

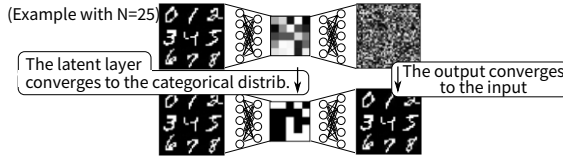


Figure 4. Train the State Autoencoder by minimizing the sum of the reconstruction loss and the variational loss of Gumbel-Softmax. As the training continues, the output of the network converges to the input images. Also, as the Gumbel-Softmax temperature τ decreases during training, the latent values approach either 0 or 1.

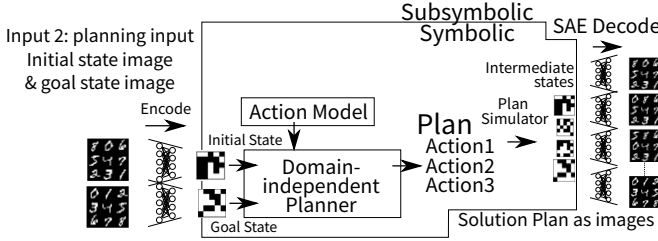


Figure 5. The planning phase of Latplan: We encode the initial and goal state images into symbolic initial/goal states. A classical planner finds the symbolic solution plan. Finally, intermediate states in the plan are decoded back to a human-comprehensible image sequence.

Latplan uses a *State AutoEncoder* (SAE) (Figure 4) neural network that learns a bidirectional mapping between subsymbolic raw data $\mathbf{x}$ (e.g., images) and propositional states $\mathbf{z} \in \{0, 1\}^F$, i.e., F -dimensional bit vectors. To make $\mathbf{z}$ binary, the SAE is implemented as a Binary Concrete (BC) VAE.

The mapping ENCODE from $\{\dots \mathbf{x}^{i,0}, \mathbf{x}^{i,1} \dots\}$ to $\{\dots \mathbf{z}^{i,0}, \mathbf{z}^{i,1} \dots\}$ provides a set of propositional transitions $\mathbf{z}^i = (\mathbf{z}^{i,0}, \mathbf{z}^{i,1}) \in Z$. Using these transitions, Latplan performs *Action Model Acquisition*, which identifies the action symbols from Z and learns the transition rules of each action, both in an unsupervised manner.

In the planning phase (Figure 5), Latplan takes as input a *planning input* $(\mathbf{x}^I, \mathbf{x}^G)$, a pair of raw data corresponding to an initial and a goal state of the environment. The output of Latplan is a data sequence representing the plan $(\mathbf{x}^I, \dots, \mathbf{x}^G)$ that reaches $\mathbf{x}^G$ from $\mathbf{x}^I$. For example, an 8-puzzle problem instance consists of an image of the start (scrambled) configuration of the puzzle ($\mathbf{x}^I$), and an image of the solved state ($\mathbf{x}^G$).

Latplan generates a planning problem instance $(\mathbf{z}^I, \mathbf{z}^G)$ from the planning input $(\mathbf{x}^I, \mathbf{x}^G)$ using SAE’s encoder, then invokes a domain-independent planner to solve the problem. As a result, Latplan obtains a propositional state trace $(\mathbf{z}^I, \dots, \mathbf{z}^G)$. However, these outputs are not interpretable for a human observer. This is primarily because the states in the state trace are generated by SAE neural network which was trained unsupervised — each bit of the state has a “meaning” in the latent space determined by the neural network, which does not necessarily directly correspond to human-interpretable notions. Furthermore, the “actions” in the plan trace are transitions according to the AMA, but are not necessarily directly interpretable by a human. In order to make the plan generated by Latplan understandable to a human (or other agents outside Latplan), we generate a step-by-step visualization of the plan execution (e.g. Figure 2) by decoding the latent bit vectors for each intermediate state in $(\mathbf{z}^I, \dots, \mathbf{z}^G)$. This results in a sequence of images

representing the plan which can be understood and executed by a human (or other agent outside of Latplan).

Latplan is a high-level planner designed to solve puzzle domains such as the 8-puzzle, and mapping a high-level action to low-level actuation sequences via a motion planner is beyond its scope. Physically “executing” the plan is not necessary, as finding the solution to the puzzles is the objective, so a “mental image” of the solution (i.e., the image sequence visualization) is sufficient.

In the following, we introduce two key building blocks of Latplan, a State AutoEncoder (SAE) and an Action AutoEncoder (AAE) in Section 2.2.1 and Section 2.2.2 respectively, and present a Vannila Space AutoEncoder as a naive implementation of Latplan, which jointly trains SAE and AAE in Section 2.2.3.

2.2.1. Propositional Symbol Grounding with State AutoEncoder

In order to perform high-level task planning using PDDL models generated from on a messy real-world input such as images, one of the key requirements is to obtain a propositional representation of the environment on which a planning solver can perform logical reasoning. We believe such a representation must satisfy the following three properties to be practical:

- It should be able to describe unseen world states using the same symbols,
- Similar images for the “same” world state should map to the same representation,
- The ability to map symbolic states back to images.

For example, one may come up with a trivial method that simply discretizes the pixel values of an image array or that computes an image hash function. Such a trivial representation lacks the ability to generalize and recover original images.

Latplan’s key observation is that the categorical variables modeled by discrete VAEs (Figure 4) are compatible with symbolic reasoning systems. Thus we implement SAEs as Binary-Concrete VAEs, whose binary latent variables can be used as propositional symbols in STRIPS language, providing a solution to the propositional symbol grounding, i.e., generation and grounding of propositional symbols.

To map an image to a latent state with sufficient accuracy, the latent layer requires a certain capacity. The lower bound of the number of propositional variables F is the encoding length of the states, i.e., $F \geq \log_2 |S|$ for a state space S . In practice, however, obtaining the most compact representation is not only difficult but also unnecessary, and we use a hyperparameter F which could be significantly larger than $\log_2 |S|$.

2.2.2. Action Symbol Grounding with Action AutoEncoder

To perform planning in the latent space, simply having a state representation is insufficient. Action AutoEncoder (AAE, Figure 6) was thus developed as a network which jointly grounds the action symbols and acquires the action model from the subset of examples, in an unsupervised manner.

Acquiring a descriptive action model (e.g., PDDL) from a set of unlabeled propositional state transitions consists of three steps.

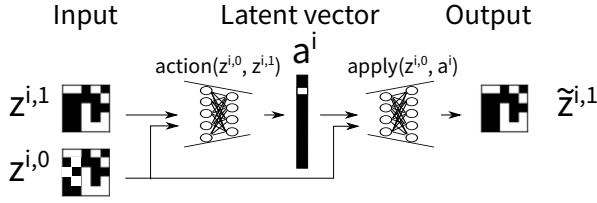


Figure 6. Action Autoencoder.

- (Step 1)** Identify the “types” of transitions, where each “type” is identified as an *action symbol*. A hand-coded “slide-up-8-at-position-1-2” in 8-puzzle would be an example of action symbols, although an action symbol grounding system should ground anonymous symbols without human-provided labels.¹
- (Step 2)** Identify the preconditions and the effects of each action and store the information in an action model.
- (Step 3)** Represent the model in a modeling language (e.g., PDDL).

Addressing this entire process is a daunting task. Existing AMA methods typically handle only Steps 2 and 3, skipping Step 1 (action symbols are given *a priori*). Without step 1, however, an agent lacks the ability to learn in an unknown environment where it does not know *what is even possible*. Note that even if the agent has the full knowledge of its low-level actuator capabilities, it does not know its own high-level capabilities, e.g., sliding a tile. AAE thus focuses on Step 1 and 2, skipping Step 3. It grounds the action symbols (Step 1) and finds a successor function that can be used for forward state space search (Step 2), but maintains its implicit, black-box representation. Step 3 is addressed by Cube-Space AE later (Section 2.4).

Consider a simple, linear search space with no branches. In this case, grounding the action symbol is not necessary and the AMA task reduces to predicting the next state $\mathbf{z}^{i,1}$ from the current state $\mathbf{z}^{i,0}$. This applies to much of the work on scene prediction from videos such as [39]. However, when the current state has multiple successors, as in planning problems, such a network is not helpful. One might consider training a separate NN for each action, but (1) it is unknown how many types of transitions are available, (2) the number of transitions depends on the current state, and (3) it does not know which transition belongs to which action. Although a single NN could learn a multi-modal distribution, then it lacks the ability to *enumerate* the successors (successor function), a crucial requirement for completeness and efficiency of a search algorithm.

AAE avoids these problems by reformulating the transitions as $\text{APPLY}(\mathbf{a}^i, \mathbf{z}^{i,0}) = \tilde{\mathbf{z}}^{i,1}$ which makes the action label as a trainable discrete latent variable. The AAE learns a bidirectional mapping between $\mathbf{z}^{i,1}$ and $\mathbf{a}^i$, both conditioned by $\mathbf{z}^{i,0}$:

- The encoder $\text{ACTION}(\mathbf{z}^{i,0}, \mathbf{z}^{i,1}) = \mathbf{a}^i$ returns an action label for a state transition $(\mathbf{z}^{i,0}, \mathbf{z}^{i,1})$.

¹In typical hand-coded STRIPS domains, actions are described in a lifted form (first order logic language). While actions found by AAEs are not lifted/parameterized, they still provide abstraction. For example, the same “slide-up-8-at-position-1-2” action, which slides the tile 8 at position $(x,y) = (1,2)$ upward, applies to many states (each state being a permutation of tiles 1-7).

- The decoder $\text{APPLY}(\mathbf{a}^i, \mathbf{z}^{i,0}) = \tilde{\mathbf{z}}^{i,1}$ applies $\mathbf{a}^i$ to $\mathbf{z}^{i,0}$ and returns a successor $\tilde{\mathbf{z}}^{i,1}$. Thus the decoder network can be seen as modeling the *effect* of each action, which represents what change should be made to $\mathbf{z}^{i,0}$ due to the application of the action.

2.2.3. Vanilla Space AutoEncoder

Overall, SAEs and AAEs can be jointly trained in a single end-to-end network, which is widely known to improve the performance in Machine Learning literature. We call the resulting network a *Vanilla Space AutoEncoder*, a world model [40] based on a discrete generative model. In the following, we always omit the dataset index i .

A Vanilla Space AE (Figure 7) is a network representing a function of two inputs $\mathbf{x}^0, \mathbf{x}^1$ and three outputs $\tilde{\mathbf{x}}^0, \tilde{\mathbf{x}}^1, \tilde{\mathbf{x}}^2$. It consists of several subnetworks: ENCODE, DECODE, ACTION, APPLY, APPLICABLE. These subnetworks are defined as a result of a graphical model for representation learning (Figure 8, left). This generative model has two ELBOs depending on how we derive the lower bound. We therefore use the average of two ELBOs for optimization, which results in a loss function defined in equation 2.

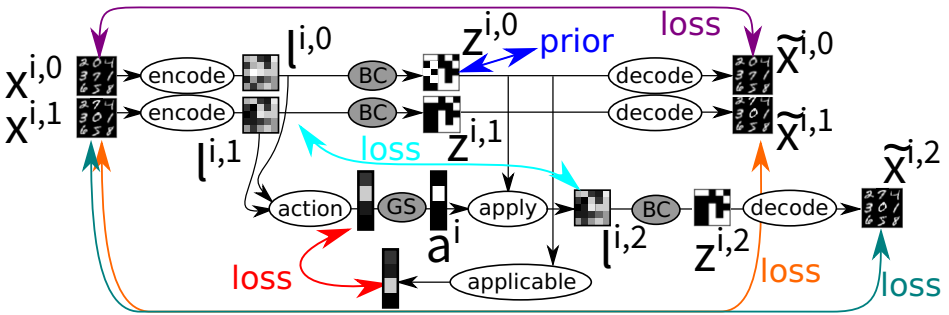


Figure 7. A Vanilla Space AutoEncoder architecture.

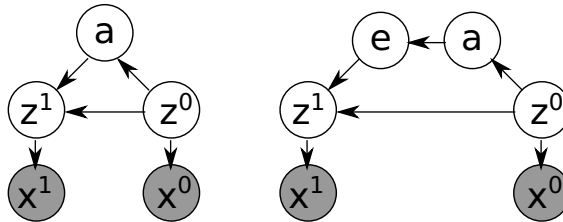


Figure 8. Graphical Model of Vanilla Space AE (left) and Cube-Space AE (right).

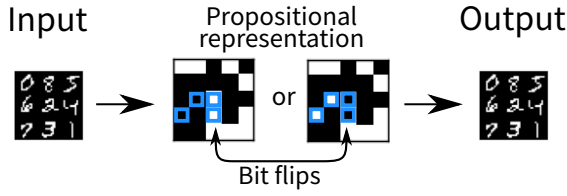


Figure 9. Propositions found by SAEs may contain uninformative random bits that do not affect the output.

$$\begin{aligned}
 \log p(\mathbf{x}^0, \mathbf{x}^1) &\geq -D_{\text{KL}}(q(\mathbf{z}^0 | \mathbf{x}^0) \| p(\mathbf{z}^0)) \\
 &\quad + \mathbb{E}_{q(\mathbf{z}^0 | \mathbf{x}^0)} [-D_{\text{KL}}(q(\mathbf{a} | \mathbf{x}^0, \mathbf{x}^1) \| p(\mathbf{a} | \mathbf{z}^0))] \\
 &\quad + \frac{1}{2} \mathbb{E}_{q(\mathbf{z}^0 | \mathbf{x}^0)q(\mathbf{a} | \mathbf{x}^0, \mathbf{x}^1)} [-D_{\text{KL}}(q(\mathbf{z}^1 | \mathbf{x}^1) \| p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}))] \\
 &\quad + \mathbb{E}_{q(\mathbf{z}^0 | \mathbf{x}^0)} \log p(\mathbf{x}^0 | \mathbf{z}^0) + \frac{1}{2} \mathbb{E}_{q(\mathbf{z}^1 | \mathbf{x}^1)} \log p(\mathbf{x}^1 | \mathbf{z}^1) \\
 &\quad + \frac{1}{2} \mathbb{E}_{q(\mathbf{z}^0 | \mathbf{x}^0)q(\mathbf{a} | \mathbf{x}^0, \mathbf{x}^1)p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a})} \log p(\mathbf{x}^1 | \mathbf{z}^1). \tag{2}
 \end{aligned}$$

Vanilla Space AE defines a sound generative model of the environment and provides means to end-to-end training. However, unlike typical Machine Learning tasks, merely having a highly accurate model is not sufficient for Neuro-Symbolic integration. The following sections describe and address two issues in the Vanilla Space AE, namely *Symbol Stability Problem* (SSP) and *non-conformance to the syntactical restrictions in the target formal language*.

2.3. The Symbol Stability Problem: Addressing the Issues of Unstable Propositional Symbols with Closed World Assumption

Propositional representations learned by Binary Concrete VAEs in the original form proposed and evaluated by [37, 38] have a problematic behavior that makes them less suitable for propositional reasoning. While the SAE can reconstruct the input with high accuracy, the learned latent representations are not “stable,” i.e., propositional values (true/false) may flip at random given identical or nearly identical image inputs (Figure 9). This gives rise to the *first* issue of a naive machine learning model which does not consider neuro-symbolic integration, the *Symbol Stability Problem* (SSP).

The core issue with the vanilla Binary Concrete VAEs is that the class probability for the class “true” and the class “false” could be neutral (e.g., around 0.5) at some neuron, causing the value of the neuron to change frequently due to the stochasticity of the system. The source of stochasticity is twofold:

- *Systemic/epistemic uncertainty*: Random sampling in the network introduces stochasticity and causes the propositions to change values.
- *Statistical/aleatoric uncertainty*: Stochastic observation of the environment causes the propositions to change values. When the class probabilities are almost neutral, a small noise in the input image may cause the activation to cross the decision

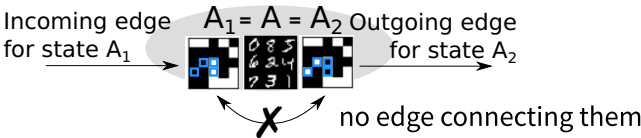


Figure 10. Random variations of the propositional encoding could disconnect the search space.

boundary for each neuron, causing bit flips. In contrast, humans still regard the corrupted image as the “same” image.

Unstable symbols are harmful to symbolic reasoning because they break the identity assumption built into the recipient symbolic reasoning algorithms such as classical planners. It causes several issues: Firstly, the performance of search algorithms (e.g., A^*) that run on the state space generated by the propositional vectors are degraded by multiple propositional states which correspond to the same real-world state. As standard symbolic duplicate detection techniques would treat such spurious representations of the same state as different states, the search algorithm can unnecessarily re-expand the “same” real-world state several times, slowing down the search.

Secondly, the state space could be disconnected due to such random variations (Figure 10). Some states may be reachable only via a single variation of the real-world state and are not connected to another propositional variation of the same real-world state. One way to circumvent this phenomenon is *state augmentation*, which samples the propositional states at every node by decoding and encoding the current state several times. This is inefficient, and is also infeasible in the standard PDDL-based classical planners that operate completely in the propositional space.

Thirdly, in order to reduce the stochasticity of the propositions, we encounter a hyperparameter tuning problem which is costly when we train a large NN. The neurons that behave randomly for the same or perturbed input do not affect the output, i.e., they are unused and uninformative. Unused neurons appear because the network has an excessive capacity to model the entire state space, i.e., they are surplus neurons. Therefore, a straightforward solution to address this issue is to reduce the size of the latent space F . On the other hand, if F is too small, it lacks the capacity to represent the state space, and the SAE no longer learns to reconstruct the real-world image. Thus we need to find an *ideal* value of F , a costly and difficult hyperparameter tuning problem. While this issue is similar to *posterior collapse* [41, 42], which is also caused by ignored latent variables, an important difference is that the latter degrades the reconstruction (makes it blurry) because there are too many ignored variables. In contrast, the symbol stability issue is significant even if the reconstruction is successful.

These unstable propositional symbols indicate that the Binary Concrete VAE learns the mapping between images and binary representations as a many-to-many relationship. While this property has not been considered as a major issue in the machine learning community where accuracy is the primary consideration, its unstable latent representation poses a significant impediment to the reasoning ability of the planners. Unlike machine learning tasks, symbolic planning requires a mapping that abstracts many images into a single symbolic state, i.e., many-to-one mapping. To this end, it is necessary to develop an autoencoder that learns a many-to-one relationship between images and binary representations.

Fundamentally, the first two harmful effects are caused by the fact that the representation learned by a standard Binary Concrete VAE lacks a critical feature of symbols, *designation* [25], that each symbol uniquely refers to an entity (referent, concept, meaning), e.g., the referents of the symbols grounded by SAEs are the truth assignments. If the grounding (meaning) of a symbol changes frequently and unexpectedly, the entire symbolic manipulation is fruitless because the underlying symbols are not tied to any particular concept and do not represent the real-world.

Thus, for a symbol grounding procedure to produce a set of symbols for symbolic reasoning, it is not sufficient to find a set of symbols that can be mapped to real states in the environment; It should find a *stable* symbolic representation that *uniquely* represents the environment.

Definition 2. *A symbol is stable when its referents are identical for the same environment, under some equivalence relation (e.g., invariance or noise threshold on the observation).*

Example 1. *A propositional symbol points to a boolean value. The value should not change under the same real-world state and its noisy observations.*

Example 2. *A symbolic state ID points to a set of propositions whose values are true. The content of the set should be the same under the same real-world state and its observations.*

Example 3. *A symbolic action label points to a tuple containing an action definition (e.g., preconditions). It should point to the same action definition each time it observes the same state transition.*

Example 4. *Say that a real-world state s_1 transitions to another state s_2 by performing a certain symbolic action a . The representation obtained by applying the symbolic action definition of a to the symbolic representation of s_1 should be equivalent to the symbolic representation directly observed from s_2 .*

The stability of the representation obtained by a NN depends on its inherent (systemic, epistemic) stochasticity of the system during the runtime (as opposed to the training time) as well as the stochasticity of the environment (statistical, aleatoric). Thus, *any* symbol grounding system potentially suffers from the symbol stability problem. As for the stochasticity of the environment, in many real-world tasks, it is common to obtain stochastic observations due to external interference, e.g., vibrations of the camera caused by the wind. As for the stochasticity of the network, both VAEs [36, 37, 43] used in Latplan and GANs (Generative Adversarial Networks) [44] used in Causal InfoGAN [45] rely on sampling processes.

To address the Symbol Stability Problem in the propositional symbols generated by the standard BC-VAE, we proposed two modifications. The first improvement we made from the original GS-VAE/BC-VAE is that we can disable the systemic stochasticity of the network in the test time. After the training, we replace the Gumbel-Softmax/Binary-Concrete activation with a pure argmax of class probabilities, which makes the network fully deterministic.



Figure 11. Autoencoding results of an MNIST 8-puzzle state using a vanilla BC-VAE and a proposed, non-uniform Bernoulli prior with 100 propositions. The latter obtains a sparse/compact representation with fewer true bits.

$$\text{GS}_\tau(\mathbf{I}) \rightarrow \arg \max(\mathbf{I}). \quad (\mathbf{I} \in \mathbb{R}^{F \times C}) \quad (3)$$

$$\text{BC}_\tau(\mathbf{I}) \rightarrow \text{STEP}(\mathbf{I}). \quad (\mathbf{I} \in \mathbb{R}^F) \quad (4)$$

The next improvement is to select the appropriate prior that results in a representation suited for symbolic computation. The prior distribution of the original Gumbel-Softmax VAE is a uniform random categorical distribution $\text{Cat}(\mathbf{1}/C)$, and the prior distribution of the original Binary-Concrete VAE is a uniform random Bernoulli distribution $\text{Bernoulli}(0.5)$. By optimizing the ELBO with stochastic gradient descent algorithms, the KL divergence $D_{\text{KL}}(q(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$ in the ELBO moves the encoder distribution $q(\mathbf{z}|\mathbf{x})$ closer to the target distribution (prior distribution) $p(\mathbf{z}) = \text{Bernoulli}(0.5)$. Observe that this objective encourages the latent representation to be *more random*, as $\text{Bernoulli}(0.5)$ is the most random distribution among $\text{Bernoulli}(p)$ for all p .

To tackle the instability of the propositional representation, we propose a different prior distribution $p(\mathbf{z})$ for the latent random variables: $p(\mathbf{z}) = \text{Bernoulli}(0)$ (Figure 11). Its key insight is to penalize the latent propositions for unnecessarily becoming true while preserving the propositions that are absolutely necessary for maintaining the reconstruction accuracy. In practice, however, since computing the KL divergence with $p(\mathbf{z}) = \text{Bernoulli}(0)$ causes a division-by-zero error in the logarithm, we instead use $p(\mathbf{z}) = \text{Bernoulli}(\varepsilon)$ for a small ε (e.g., 0.1, 0.01).

The KL divergence with $p(\mathbf{z}) = \text{Bernoulli}(\varepsilon)$ results in a form

$$q \log \frac{q}{\varepsilon} + (1-q) \log \frac{(1-q)}{(1-\varepsilon)} = -H(q) + \alpha \cdot q - \log(1-\varepsilon)$$

for $\alpha = \log(1-\varepsilon) - \log \varepsilon$. This form shows that minimizing the KL divergence has an additional regularization pressure to move q toward 0 because $\alpha > 0$ ($\varepsilon < \frac{1}{2}$).

Intuitively, $p(\mathbf{z}) = \text{Bernoulli}(0)$ prior assumes that a proposition is false by default when the encoder fails to find a combination of complex visual patterns that makes the proposition true. This resembles *closed-world assumption* [46] where propositional variables are assumed to be false unless it is explicitly declared or proven to be true (negation as failure). The difference is whether the declaration / proof is implicit (performed by the encoder using the input data) or explicit (encoded manually by human, or proved by a proof system). In contrast, the optimization objective made by the original prior distribution $p(\mathbf{z}) = \text{Bernoulli}(0.5)$ makes a latent proposition highly random when the input image contains no visual patterns that either support or disprove the proposition. Since a uniform binary distribution $\text{Bernoulli}(0.5)$ carries no information about either the truthfulness or the falsifiability of the proposition, a uniform random binary variable can be seen as in an “unknown” state. This resembles an open-world assumption where a propositional variable that is neither proved true or false are put in an unknown state.

2.4. Descriptive Action Model Acquisition with Cube-Space AutoEncoder

One major drawback of AAE is that it used a black-box neural model as the successor generator. Translating/compiling such a black-box model into a PDDL model may not be feasible because the compilation could be exponential [47]. The issue is caused by the ability of AAE to learn an arbitrarily complex state transition model. The traditional STRIPS progression $\text{APPLY}(s, a) = (s \setminus \text{DEL}(a)) \cup \text{ADD}(a)$ *disentangles* the effects from the current state s , i.e., the effect $\text{ADD}(a)$ and $\text{DEL}(a)$ are defined entirely based on action a . In contrast, AAE's black-box progression $\text{APPLY}(\mathbf{a}^i, \mathbf{z}^{i,0})$ does not offer such a separation, allowing the model to learn arbitrarily complex conditional effects. This gives rise to the *second* major issue of a naive machine learning model which does not consider neuro-symbolic integration, the *non-conformance to the syntactical restrictions in the target formal language*.

The final installment to Latplan architecture is *Cube-Space AutoEncoder* (CSAE) which directly learns a compact STRIPS model by regularizing / restricting the action model in a Vanilla Space AE. The action model searched by the NN, specifically the APPLY in the AAE, is restricted to a graph class called *cube-like graph* [48] that corresponds precisely to the STRIPS semantics, thus is able to extract the PDDL definitions of each action that is immediately usable by off-the-shelf planners.

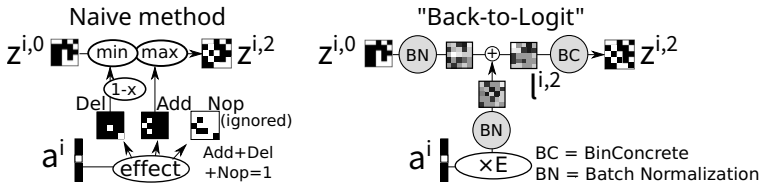


Figure 12. A naive and a Back-to-Logit implementation of the APPLY module of the CSAE.

Cube-Space AE modifies the APPLY network so that it directly predicts the effects without taking the current state as the input and logically computes the successor state based on the predicted effect and the current state. To illustrate the idea, we first present an ad-hoc, naive and impractical implementation of such a network in Figure 12 (left). The EFFECT network predicts a binary tensor of shape $F \times 3$ using F -way Gumbel-Softmax of 3 categories. Each Gumbel Softmax corresponds to one bit in the F -bit latent space and 3 classes correspond to the add effect, delete effect, and NOP, only one of which is selected by the one-hot vector. The effects are applied to the current state by max/min operations. Formally, the naive CSAE is formulated as follows:

$$\begin{aligned} \{0, 1\}^F \ni \mathbf{z}^{i,2} &= \max(\min(\mathbf{z}^{i,0}, 1 - \text{DEL}(\mathbf{a}^i)), \text{ADD}(\mathbf{a}^i)), & \text{where} \\ \text{EFFECT}(\mathbf{a}^i) &= \text{GS}_{\tau}(\text{MLP}(\mathbf{a}^i)) \in \{0, 1\}^{F \times 3}, & \text{ADD}(\mathbf{a}^i) = \text{EFFECT}(\mathbf{a}^i)_0, \\ \text{DEL}(\mathbf{a}^i) &= \text{EFFECT}(\mathbf{a}^i)_1, & \text{NOP}(\mathbf{a}^i) = \text{EFFECT}(\mathbf{a}^i)_2. \end{aligned} \quad (5)$$

Although this approach explicitly models the STRIPS transition rule $s' = (s \setminus \text{DEL}(a)) \cup \text{ADD}(a)$, we cannot compute the KL divergence $D_{\text{KL}}(q(\mathbf{z}^1 | \mathbf{x}^1) \| p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}))$ from equation 5 because its $\mathbf{z}^{i,2}$ is not produced by Binary Concrete, and thus we do not have the logit $\mathbf{l}$ for computing $p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}) = \text{SIGMOID}(\mathbf{l})$. Note that $D_{\text{KL}}(q(\mathbf{z}^1 |$

$\mathbf{x}^1 \parallel p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}))$, a KL divergence that represents successor state prediction accuracy, maintains the symbol stability in Example 4 by making two distributions $q(\mathbf{z}^1 | \mathbf{x}^1)$ and $p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a})$ identical. Without this KL term, latent vectors encoded by the encoder (i.e., $q(\mathbf{z}^1 | \mathbf{x}^1)$) and by the AAE (i.e., $p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}))$ may diverge, despite them being the symbolic representations for the same observation $\mathbf{x}^1$ of the successor state.

Cube-Space AE addresses this issue with *Back-to-Logit* (BtL, Figure 12, right) formalized in equation 6. It re-encodes a binary vector back to a continuous logit by a monotonic function m . This monotonicity preserves the order between true (1) and false (0) even after transformed into real numbers. We then apply the given action by adding a *continuous effect vector* $\text{EFFECT}(\mathbf{a}^i)$ to the continuous current state. After adding the continuous vectors, we re-discretize the result with Binary Concrete. In practice, we can implement m as a *Batch Normalization* [49] layer, and EFFECT as a linear embedding, i.e., $\text{EFFECT}(\mathbf{a}^i) = \mathbf{E}\mathbf{a}^i$, where $\mathbf{E} \in \mathbb{R}^{F \times A}$. Applying Batch Normalization to the effect vector also helps the training. Therefore, a recommended implementation is equation 7.

$$\mathbf{z}^{i,2} = \text{BC}_\tau(\text{APPLY}(\mathbf{z}^{i,0}, \mathbf{a}^i)) = \text{BC}_\tau(m(\mathbf{z}^{i,0}) + \text{EFFECT}(\mathbf{a}^i)). \quad (6)$$

$$\mathbf{z}^{i,2} = \text{BC}_\tau(\text{APPLY}(\mathbf{z}^{i,0}, \mathbf{a}^i)) = \text{BC}_\tau(\text{BN}(\mathbf{z}^{i,0}) + \text{BN}(\mathbf{E}\mathbf{a}^i)). \quad (7)$$

BTL injects a new assumption into $p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a})$ by introducing a random variable $\mathbf{e}$ and removing the dependency from $\mathbf{e}$ to $\mathbf{z}^0$ as in equation 8, where $p(\mathbf{e} | \mathbf{a})$ is modeled by EFFECT , and $p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{e})$ is modeled by m and an addition. This leads to a graphical model (Figure 8, right) where a dependency from $\mathbf{z}^0$ to $\mathbf{e}$ is removed.

$$p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}) = \sum_{\mathbf{e}} p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{a}, \mathbf{e}) p(\mathbf{e} | \mathbf{z}^0, \mathbf{a}) := \sum_{\mathbf{e}} p(\mathbf{z}^1 | \mathbf{z}^0, \mathbf{e}) p(\mathbf{e} | \mathbf{a}) \quad (8)$$

Finally, the embedding learned by BTL can be shown to be compatible with STRIPS as follows. Consider an ideal training result where $D_{\text{KL}}(q(\mathbf{z}^1 = \mathbf{z}^1 | \mathbf{x}^1) \parallel p(\mathbf{z}^1 = \mathbf{z}^2 | \mathbf{z}^0, \mathbf{a})) = 0$, i.e., $\forall i; \mathbf{z}^1 \equiv \mathbf{z}^2$. States learned by BTL have the following property:

Theorem 1. *For each action $\mathbf{a}$, the transitions $\{(\mathbf{z}^0, \mathbf{z}^1) \in Z \mid \mathbf{a} = \text{ACTION}(\mathbf{z}^0, \mathbf{z}^1)\}$ are bitwise monotonic, deterministic, and restricted to three mutually exclusive modes, i.e., for each bit j , it is either: (add:) $\forall i; (\mathbf{z}_j^{i,0}, \mathbf{z}_j^{i,1}) \in \{(0, 1), (1, 1)\}$, or (del:) $\forall i; (\mathbf{z}_j^{i,0}, \mathbf{z}_j^{i,1}) \in \{(1, 0), (0, 0)\}$, or (nop:) $\forall i; (\mathbf{z}_j^{i,0}, \mathbf{z}_j^{i,1}) \in \{(0, 0), (1, 1)\}$.*

This theorem guarantees that each action deterministically sets a certain bit on and off in the binary latent space. Therefore, the actions and the transitions satisfy the STRIPS state transition rule $s' = (s \setminus \text{DEL}(\mathbf{a})) \cup \text{ADD}(\mathbf{a})$, thus enabling a direct translation from neural network weights to PDDL modeling language. The proof is straightforward from the monotonicity of m and Binary Concrete and a step function.

To extract the effects of an action $\mathbf{a}$ from CSAE, we compute $\text{ADD}(\mathbf{a}) = \text{APPLY}(\mathbf{a}, \mathbf{0})$ and $\text{DEL}(\mathbf{a}) = 1 - \text{APPLY}(\mathbf{a}, \mathbf{1})$ for each action $\mathbf{a}$, where $\mathbf{0}, \mathbf{1} \in \mathbb{B}^F$ are vectors filled by zeros/ones and has the same size as the binary embedding. Since APPLY deterministically sets values to 0 or 1, feeding these vectors is sufficient to see which bit it turns on and off. For each j -th bit that is 1 in each result, a corresponding proposition is added to the add/delete-effect, respectively.

Subnetworks	Layers
ENCODE	GaussianNoise(0.2), BN, {Conv(5,32), ReLU, BN, Dropout(0.2),}*2, Conv(5,32)
DECODE	BN, {Conv(5,32), ReLU, BN, Dropout(0.2),}*2, Conv(5,32)
ACTION	SIGMOID, fc(1000), ReLU, BN, Dropout(0.2), fc(6000)
APPLICABLE	fc(6000)

Table 2. Detailed overview of the subnetworks of Cube-Space AE. “Conv(k, c)” denotes a convolutional layer with kernel size k and channel width c , and “fc(w)” denotes a fully-connected layer with width w . We omit reshaping operators such as “flatten”. To maintain the notational consistency with Figure 7, this table does not include BC_τ activation in ENCODE and GS_τ activation in ACTION.

To extract the preconditions of an action $\mathbf{a}$, we propose a simple ad-hoc method (very similar to [50, 51]) that looks for bits that always take the same value when $\mathbf{a}$ is used. Let $Z^0(\mathbf{a}) = \{z^0 \mid \text{ACTION}(z^0, z^1) = \mathbf{a}\}$. Then the positive and the negative preconditions of an action $\mathbf{a}$ are defined as follows:

$$\text{PRE}(\mathbf{a}) = \{f \mid \forall z^0 \in Z^0(\mathbf{a}); z_f^0 = 1\} \cup \{\neg f \mid \forall z^0 \in Z^0(\mathbf{a}); z_f^0 = 0\}. \quad (9)$$

2.5. Output Examples and Experimental Evaluation

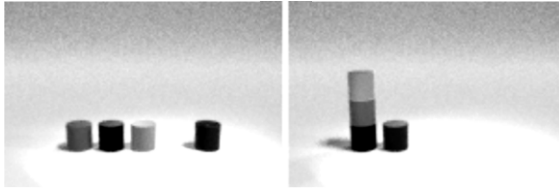


Figure 13. An example of problem instances (initial and goal images) for Blocksworld with 4 blocks in red, green, blue and black. The actual training and planning experiments are performed on smaller resized images.

Latplan works on a wide variety of domains without modifying the main training code, although it currently needs a separate training for each domain and mixed multi-domain training is an avenue of future work (e.g., same general propositions are reused for a variety of domains). While we verified that Latplan works on 5 more visual domains (MNIST 8-Puzzle [13], Mandrill 15-Puzzle [52], Sokoban [53, 54], and two visual versions of LightsOut game [55]), due to space we focus on providing the details of **Photo-realistic Blocksworld** [56] dataset (Figure 13). It consists of 100x150 RGB images rendered by Blender 3D engine containing complex visual elements such as shadows and reflections from neighboring blocks. We resized the images to 30x45 pixels and normalized the mean and the variance to 0/1. We sampled 20000 transitions / 40000 states and divided them into 90%, 5%, 5% for a training set, a validation set, and a test set, respectively. We evaluated 30 hyperparameter configurations using grid search, varying the latent space size F and the β_1, β_3 coefficients of KL divergence terms [43]. Details of the network are specified in Table 2 and parameter ranges are listed in Table 3.

To validate the correctness of the model, we ran an off-the-shelf state-of-the-art classical planner Fast Downward [58] on a PDDL domain file and a PDDL problem file,

Optimizer, Learning rate, Gradient norm clipping	Rectified Adam [57], 10^{-3} , 0.1
Training Epochs, Batch size	2000,400
Annealing schedule $\tau(t)$ for epoch t	$\tau_{\max} \left(\frac{\tau_{\min}}{\tau_{\max}} \right)^{\frac{\min(t,1000)}{1000}}$, $\tau_{\max} = 5$, $\tau_{\min} = 0.5$
Latent space dimension F	$F \in \{50, 100, 300\}$
Maximum number of actions A	$A = 6000$
β_1 for $D_{\text{KL}}(q(\mathbf{z}^0 \mathbf{x}^0) \parallel p(\mathbf{z}^0))$	$\beta_1 \in \{1, 10\}$
β_2 for $D_{\text{KL}}(q(\mathbf{a} \mathbf{x}^0, \mathbf{x}^1) \parallel p(\mathbf{a} \mathbf{z}^0))$	1
β_3 for $D_{\text{KL}}(q(\mathbf{z}^1 \mathbf{x}^1) \parallel p(\mathbf{z}^2 \mathbf{z}^0, \mathbf{a}))$	$\beta_3 \in \{1, 10, 100, 1000, 10000\}$

Table 3. List of hyperparameters.

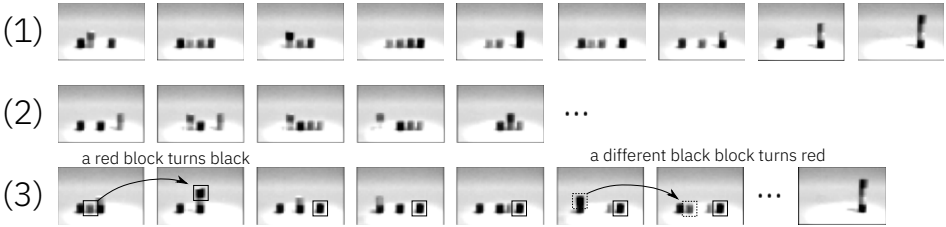


Figure 14. Outputs of Latplan with Cube-Space AE on different problem instances. Sequence (1) is a valid suboptimal plan generated by A^* with LMcut heuristics. Sequence (2,3) are invalid plans. In (2), floating objects can be observed, and in (3), a red block turns black and as a result, the black block is duplicated.

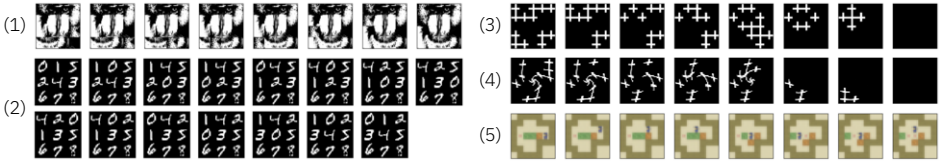


Figure 15. Cherry-picked examples of Latplan on (1) MNIST 8-puzzle, (2) Mandrill 15-puzzle, (3) Lights Out, (4) Twisted Lights Out, (5) Sokoban instances. All plans are optimal (shortest) for the problem instance.

which are generated by the process described in Section 2.4 and Section 2.2, respectively. The initial and the goal images ($\mathbf{x}^I, \mathbf{x}^G$) are generated randomly.

As Latplan uses an off-the-shelf planner, it can leverage a variety of search configurations (heuristic function and search algorithms) already available in the planner. For example, we are able to run A^* with blind heuristics (i.e., Dijkstra search), A^* with Landmark-Cut (LMcut) heuristics [29], Merge-and-Shrink (M&S) heuristics [59], or the first iteration of a satisficing planner LAMA [60], a complex configuration of Greedy Best First Search that alternates between 4 search queues. Experiments are run with the 10 minutes time limit and 8GB memory limit, and Figure 14 shows a successful case and failure cases. The failure cases provide intriguing examples of how a neural network sees the world and makes errors. Figure 15 shows successful cases (optimal plans) found in other 5 domains.

A rather surprising discovery made by this system is that domain-independent heuristics in the planning literature managed to reduce the search effort in solving the PDDL instances generated by Latplan. Domain-independent heuristics in the planning literature are traditionally evaluated on International Planning Competition instances

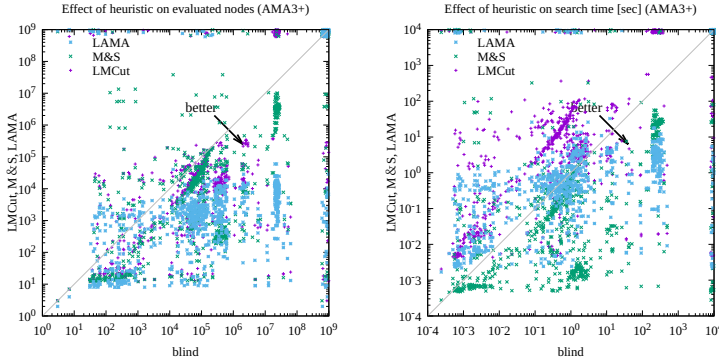


Figure 16. The left column shows the number of node evaluations for h^{blind} heuristics (x-axis) and h^{LMCut} , $h^{\text{M\&S}}$ heuristics as well as LAMA (y-axis), for instances solved and validated by both. Unsolved instances are placed on the border. State-of-the-Art heuristics developed in classical planning community significantly reduces the search effort. The right column shows the search time. M\&S still outperforms blind. Due to the higher cost-per-node, LMCut tends to consume more search time compared to blind. Similarly, LAMA seems to struggle in easy instances. However, it tends to behave better in more difficult instances.

[11], which contain a broad variety of domains and are regarded as a representative subset of the real-world tasks. Since they are hand-coded benchmarks, they may accidentally contain certain general structure such as serializable subgoals and abstraction [61], which could have originated from the way humans perceive the world. However, the latent space domains, derived from raw pixels and neural networks, may have completely different characteristics that could render them useless. In fact, blind heuristic can sometimes outperform well-known heuristics on some IPC domains (e.g., floortile) specifically designed to defeat their efficacy [62]. Note that even though the image-based domains we evaluate *correspond* to the domains used in the combinatorial search and classical planning literature, the latent representations may be unlike any standard PDDL encoding of the domain written by humans. Thus, there is little *a priori* reason to believe that the standard heuristics will be effective until we evaluate them.

Figure 16 compares node evaluations and search time between blind search and LMCut, M\&S, and LAMA on 240 problem instances for the 6 domains we evaluated (40 instances each). For each domain, we selected the hyperparameter configuration which found the largest number of valid solutions. Node evaluation plots confirm the overall reduction in the search effort. In contrast, the results on search time are mixed. M\&S reduces the search time by an order of magnitude, while LMCut tends to be slow due to the large evaluation cost per node. LAMA tends to be faster than a blind search in more difficult instances, though a blind search is faster in easier instances that require less search (< 10 seconds). Despite the increased runtime, the reduced node evaluation strongly indicates that either the models we are inferring contain similar properties as the human-designed PDDL models, or that powerful, domain-independent planning heuristics can induce effective search guidance even in less human-like PDDL models.

2.6. Related Work

A variety of action model acquisition approaches have been proposed already [63, 64]. However, these traditional action learners tend to require a certain type of human domain knowledge and have been situating itself merely as an additional assistance tool for humans, rather than a system that builds knowledge from the scratch, e.g., from unstructured images. Many systems require structured input representations (i.e., First Order Logic) that are partially hand-crafted and exploit the symmetry and the structures provided by the representation [24, 65, 66, 67, 68, 69, 70, 71, 72, 73] although the requirements of the systems may vary. For example, some systems require state sequences [24] which contain predicate symbols (such as *on*, *clear* for blocksworld), while others require the sample action sequences [68, 65, 71, 72, 73] or a state space graph labeled by actions [74] which contain action symbols (such as *board*, *depart* in miconic domain, or *go-left/right*, *up-ladder* in Treasure Game domain). They may also require object symbols and type symbols, such as *block-A/B* and *block* in blocksworld domain. Some are robust against noisy inputs [69, 70], partial observations in a state and/or missing state/actions in a plan trace [66], a disordered plan trace [67]. There is also a line of work that extracts a PDDL action model from a natural language corpus. Framer [75] uses a CoreNLP language model while EASDRL [76] uses Deep Reinforcement Learning [4]. The difference from our approach is that they are reusing the symbols found in the corpus, i.e., *parasitic* [32], while we generate the discrete propositional symbols from the scratch, using only the visual perception which completely lacks a predefined set of symbols.

Despite relying on symbolic, discrete information that are given *a priori* as input, some of these existing work may claim that they do not use symbolic information. For example, [71] limits the scope of symbols to propositional symbols, and [74] claims the state space graph is not symbolic information although it contains state IDs and action symbols. Our definition of symbols in Section 2.1.2 helps avoid such confusions.

Learning from Observation [77, 78, 79, LfO] in robotics literature could produce propositions from the observations (unstructured input). However, LfO is seen as a form of supervised learning [79] because they typically assume labels or various forms of external help from humans, thus they do not eliminate knowledge acquisition bottlenecks. [80] proposed a method that obtains a PDDL representation for a manipulator robot using the data collected through exploration from its actuators and several hand-crafted visual features. They discover “object categories” which are type symbols (unary static predicate symbols) in PDDL, and “effect categories” which resemble propositional symbols, but the system requires predefined action labels. A follow-up work [81] replaces hand-crafted extractors with discrete variational autoencoders. A related field of Activity Recognition [82, 83, 84, 85] learns to map a sequence of observations to a certain action symbol, which is either fixed (i.e., there is only one type of motions in the dataset) or is manually assigned and is made available in the dataset. While some recent work has unsupervised learning capability from the sensor data or documents [86, 87, 88, 89, 90, 91], they do not generate representations for planning.

Model-based RL [31, 92] and world-model literature [40] has deep connection with Latplan because they also learn a transition function in the latent space. However, the learned transition function is typically a black-box function that is not compatible with symbolic model analysis. Vanilla Space AE (Section 2.2.3) and Causal InfoGAN [45] are instances of black-box world models.

2.7. Discussion and Conclusion

We proposed Latplan, an integrated architecture for learning and planning which, given only a set of unlabeled images and no prior knowledge, generates a classical planning problem, solves it with a symbolic planner, and presents the plan as a human-comprehensible sequence of images. Only unlabeled image pairs representing action start/end states are available, and there is no simulator, expert trace, or reward function. The only key assumptions about the input domain we make are that it is fully observable and deterministic. Thus, we have shown that different domains can all be solved by the same system, without modifying any code or the NN architecture. In other words, *Latplan is a domain-independent, image-based classical planner.*

Much work is still required to determine the applicability and scalability of this approach. For example, Latplan requires uniform sampling from the environment which is nontrivial in many scenarios, and is also limited to tasks where a single goal state is specified. However, we believe Latplan is an important step in bridging the gap between symbolic and subsymbolic reasoning and opens many avenues for future research.

One natural direction for future research is to generate other types of combinatorial search problems (e.g., Boolean Satisfiability, Answer Set Programming) from subsymbolic data (e.g., images, text) in an unsupervised manner. This differs from neural networks that solve existing symbolic search problems [93, 94], or map subsymbolic entities to known symbolic elements with supervised learning (e.g., classification).

Some conceptual contributions of Latplan may facilitate symbol grounding for those tasks. First, we generalized the problematic behavior of unstable propositions into an abstract notion of *Symbol Stability Problem* (SSP), a subproblem of symbol grounding. In Latplan, the SSP is addressed by a KL-divergence loss that resembles Closed-World Assumption (Section 2.3) and a KL-divergence loss that enforces accurate successor prediction (Section 2.2.3). The new notion of the SSP opens directions for future work in other types of neuro-symbolic systems operating on real world input.

Next, Cube-Space AE showcased an example of encoding *a syntactic constraint in the target formal language* in a neural network. Back-to-Logit in Cube-Space AE managed to encode the restriction in STRIPS formalism that the effects must be independent from the current state. Similarly, various formal languages have its own restrictions and semantics, which would have to be properly translated into a neural architecture. For example, *alldifferent* constraints in an input language for Constraint Programming (e.g., Minizinc [7]) or various logics in Satisfiable Modulo Theories would require appropriate architectures for learning a symbolic model as well as for exporting the learned weights into the target language.

References

- [1] Deng J, Dong W, Socher R, et al. ImageNet: A Large-Scale Hierarchical Image Database. In: Proc. of IEEE Conference on Computer Vision and Pattern Recognition; IEEE; 2009. p. 248–255.
- [2] Ren S, He K, Girshick R, et al. Faster R-CNN: Towards Real-time Object Detection with Region Proposal Networks. In: Advances in Neural Information Processing Systems; 2015. p. 91–99.
- [3] Deng L, Hinton GE, Kingsbury B. New Types of Deep Neural Network Learning for Speech Recognition and Related Applications: An Overview. In: Proc. of IEEE Conference on Acoustics, Speech and Signal Processing; IEEE; 2013. p. 8599–8603.

- [4] Mnih V, Kavukcuoglu K, Silver D, et al. Human-Level Control through Deep Reinforcement Learning. *Nature*. 2015;518(7540):529–533.
- [5] Graves A, Wayne G, Reynolds M, et al. Hybrid Computing using a Neural Network with Dynamic External Memory. *Nature*. 2016;538(7626):471–476.
- [6] Gebser M, Kaufmann B, Kaminski R, et al. Potassco: The Potsdam Answer Set Solving Collection. *AI Communications*. 2011;24(2):107–124.
- [7] Wallace M. Building Decision Support Systems: Using MiniZinc. Springer Nature; 2020.
- [8] Davies J. Solving MAXSAT by decoupling optimization and satisfaction [dissertation]. University of Toronto; 2013.
- [9] Barrett C, Fontaine P, Tinelli C. The Satisfiability Modulo Theories Library (SMT-LIB) [WWW.SMT-LIB.ORG]; 2016.
- [10] Love N, Hinrichs T, Haley D, et al. General Game Playing: Game Description Language Specification. 2008;.
- [11] McDermott DV. The 1998 AI Planning Systems Competition. *AI Magazine*. 2000;21(2):35–55.
- [12] Steels L. The Symbol Grounding Problem has been Solved. So What's Next? In: de Vega M, Glenberg A, Graesser A, editors. Symbols and Embodiment. Oxford University Press; 2008.
- [13] Asai M, Fukunaga A. Classical planning in deep latent space: Bridging the subsymbolic-symbolic boundary. In: McIlraith SA, Weinberger KQ, editors. Proc. of AAAI Conference on Artificial Intelligence. AAAI Press; 2018. p. 6094–6101.
- [14] Asai M, Kajino H. Towards Stable Symbol Grounding with Zero-Suppressed State AutoEncoder. In: Proc. of the International Conference on Automated Planning and Scheduling(ICAPS); July; 2019.
- [15] Asai M, Muise C. Learning Neural-Symbolic Descriptive Planning Models via Cube-Space Priors: The Voyage Home (to STRIPS). In: Proc. of International Joint Conference on Artificial Intelligence (IJCAI); 2020.
- [16] Cullen J, Bryman A. The Knowledge Acquisition Bottleneck: Time for Reassessment? *Expert Systems*. 1988 August;5(3).
- [17] Fox M, Long D. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *J Artif Intell Res(JAIR)*. 2003;20:61–124.
- [18] Fox M, Long D. Modelling Mixed Discrete-Continuous Domains for Planning. *J Artif Intell Res(JAIR)*. 2006;27:235–297.
- [19] Younes HL, Littman ML. PPDDL1.0: An Extension to PDDL for Expressing Planning Domains with Probabilistic Effects. *Techn Rep CMU-CS-04-162*. 2004;2:99.
- [20] Bertsekas DP. Reinforcement learning and optimal control. Athena Scientific Belmont, MA; 2019.
- [21] Zhuo HH, Yang Q. Action-Model Acquisition for Planning via Transfer Learning. *Artificial Intelligence*. 2014;212:80–103.
- [22] Zhuo HH. Crowdsourced Action-Model Acquisition for Planning. In: Proc. of AAAI Conference on Artificial Intelligence; Vol. 29; 2015.
- [23] Wang X. Learning by Observation and Practice: An Incremental Approach for Planning Operator Acquisition. In: Proc. of the International Conference on Machine Learning. Elsevier; 1995. p. 549–557.
- [24] Yang Q, Wu K, Jiang Y. Learning Action Models from Plan Examples using Weighted MAX-SAT. *Artificial Intelligence*. 2007;171(2-3):107–143.
- [25] Newell A, Simon HA. Computer Science as Empirical Inquiry: Symbols and Search. *Commun ACM*. 1976;19(3):113–126.
- [26] Newell A. Physical Symbol Systems. *Cognitive science*. 1980;4(2):135–183.
- [27] Nilsson NJ. The Physical Symbol System Hypothesis: Status and Prospects. In: 50 Years of Artificial Intelligence. Springer; 2007. p. 9–17.
- [28] Hoffmann J, Nebel B. The FF Planning System: Fast Plan Generation through Heuristic Search. *J Artif Intell Res(JAIR)*. 2001;14:253–302.
- [29] Helmert M, Domshlak C. Landmarks, Critical Paths and Abstractions: What's the Difference Anyway? In: Proc. of the International Conference on Automated Planning and Scheduling(ICAPS); 2009.
- [30] Silver D, Hubert T, Schrittwieser J, et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *CoRR*. 2017;abs/1712.01815.
- [31] Schrittwieser J, Antonoglou I, Hubert T, et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *CoRR*. 2019;abs/1911.08265.
- [32] Harnad S. The Symbol Grounding Problem. *Physica D: Nonlinear Phenomena*. 1990;42(1-3):335–346.
- [33] Taddeo M, Floridi L. Solving the Symbol Grounding Problem: A Critical Review of Fifteen Years of

- Research. *Journal of Experimental & Theoretical Artificial Intelligence*. 2005;17(4):419–445.
- [34] Kawamura A, Ziegler M. Invitation to Real Complexity Theory: Algorithmic Foundations to Reliable Numerics with Bit-Costs. *CoRR*. 2018;abs/1801.07108.
- [35] Hinton GE, Salakhutdinov RR. Reducing the Dimensionality of Data with Neural Networks. *Science*. 2006;313(5786):504–507.
- [36] Kingma DP, Welling M. Auto-Encoding Variational Bayes. In: *Proc. of the International Conference on Learning Representations*; 2013.
- [37] Jang E, Gu S, Poole B. Categorical Reparameterization with Gumbel-Softmax. In: *Proc. of the International Conference on Learning Representations*; 2017.
- [38] Maddison CJ, Mnih A, Teh YW. The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables. In: *Proc. of the International Conference on Learning Representations*; 2017.
- [39] Srivastava N, Mansimov E, Salakhutdinov RR. Unsupervised Learning of Video Representations using LSTMs. In: *Proc. of the International Conference on Machine Learning*; 2015. p. 843–852.
- [40] Ha D, Schmidhuber J. World Models. *CoRR*. 2018;abs/1803.10122.
- [41] Dinh L, Dumoulin V. Training Neural Bayesian Nets ; 2014.
- [42] Bowman S, Vilnis L, Vinyals O, et al. Generating Sentences from a Continuous Space. In: *Proceedings of The 20th SIGNLL Conference on Computational Natural Language Learning*; 2016. p. 10–21.
- [43] Higgins I, Matthey L, Pal A, et al. β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In: *Proc. of the International Conference on Learning Representations*; 2017.
- [44] Goodfellow IJ, Pouget-Abadie J, Mirza M, et al. Generative Adversarial Nets. In: *Advances in Neural Information Processing Systems*; 2014. p. 2672–2680.
- [45] Kurutach T, Tamar A, Yang G, et al. Learning Plannable Representations with Causal InfoGAN. In: *Advances in Neural Information Processing Systems*; 2018.
- [46] Reiter R. On Closed World Data Bases. In: *Readings in Artificial Intelligence*. Elsevier; 1981. p. 119–140.
- [47] Asai M. Neural-Symbolic Descriptive Action Model from Images: The Search for STRIPS. In: *Proc. of the ICAPS Workshop on Knowledge Engineering for Planning and Scheduling (KEPS)*; 2020.
- [48] Payan C. On the Chromatic Number of Cube-Like Graphs. *Discrete mathematics*. 1992;103(3).
- [49] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: Bach FR, Blei DM, editors. *Proc. of the International Conference on Machine Learning; (JMLR Workshop and Conference Proceedings; Vol. 37)*. JMLR.org; 2015. p. 448–456.
- [50] Wang X. Learning planning operators by observation and practice. In: Hammond KJ, editor. *Proceedings of the Second International Conference on Artificial Intelligence Planning Systems*. AAAI; 1994. p. 335–340.
- [51] Amado L, Pereira RF, Aires JP, et al. Goal Recognition in Latent Space. In: *Proc. of International Joint Conference on Neural Networks (IJCNN)*; 2018.
- [52] Asai M, Tang Z. Discrete Word Embedding for Logical Natural Language Understanding. *CoRR*. 2020; abs/1412.6980.
- [53] Culberson J. Sokoban is PSPACE-complete. In: *Proceedings in Informatics 4, International Conference on Fun with Algorithms*; June. Carleton Scientific; 1998. p. 65–76.
- [54] Junghanns A, Schaeffer J. Sokoban: Enhancing General Single-Agent Search Methods using Domain Knowledge. *Artificial Intelligence*. 2001;129(1):219–251.
- [55] Anderson M, Feil T. Turning lights out with linear algebra. *Mathematics Magazine*. 1998;71(4):300–303.
- [56] Asai M. Photo-realistic blocksworld dataset. *CoRR*. 2018;abs/1812.01818.
- [57] Liu L, et al. On the Variance of the Adaptive Learning Rate and Beyond. *CoRR*. 2019;abs/1908.03265.
- [58] Helmert M. The Fast Downward Planning System. *J Artif Intell Res (JAIR)*. 2006;26:191–246.
- [59] Helmert M, Haslum P, Hoffmann J, et al. Merge-and-Shrink Abstraction: A Method for Generating Lower Bounds in Factored State Spaces. *Journal of ACM*. 2014;61(3):16:1–16:63.
- [60] Richter S, Westphal M. The LAMA Planner: Guiding Cost-Based Anytime Planning with Landmarks. *J Artif Intell Res (JAIR)*. 2010;39(1):127–177.
- [61] Korf RE. Macro-Operators: A Weak Method for Learning. *J Artif Intell Res (JAIR)*. 1985;26(1):35–77.
- [62] Edelkamp S. PDB or not PDB? - that's the Question. A Tribute to Blind Search Planning. In: *Festivus in the 22nd International Conference on Automated Planning and Scheduling (ICAPS)*; 2012.
- [63] Jiménez S, de la Rosa T, Fernández S, et al. A Review of Machine Learning for Automated Planning. *Knowledge Engineering Review*. 2012;27(4):433.

- [64] Arora A, Fiorino H, Pellier D, et al. A Review of Learning Planning Action Models. *Knowledge Engineering Review*. 2018;33.
- [65] Cresswell S, McCluskey TL, West MM. Acquiring planning domain models using *LOCM*. *Knowledge Engineering Review*. 2013;28(2):195–213.
- [66] Aineto D, Jiménez S, Onaindia E. Learning STRIPS Action Models with Classical Planning. In: *Proc. of the International Conference on Automated Planning and Scheduling(ICAPS)*; 2018.
- [67] Zhuo HH, Peng J, Kambhampati S. Learning Action Models from Disordered and Noisy Plan Traces. *CoRR*. 2019;abs/1908.09800.
- [68] Cresswell S, Gregory PJ. Generalised Domain Model Acquisition from Action Traces. In: *Proc. of the International Conference on Automated Planning and Scheduling(ICAPS)*; 2011.
- [69] Mourão K, Zettlemoyer LS, Petrick RPA, et al. Learning STRIPS Operators from Noisy and Incomplete Observations. In: *Proc. of the International Conference on Uncertainty in Artificial Intelligence*; 2012. p. 614–623.
- [70] Zhuo HH, Kambhampati S. Action-Model Acquisition from Noisy Plan Traces. In: *Proc. of International Joint Conference on Artificial Intelligence (IJCAI)*; 2013.
- [71] Konidaris G, Kaelbling LP, Lozano-Pérez T. Constructing Symbolic Representations for High-Level Planning. In: *Proc. of AAAI Conference on Artificial Intelligence*; 2014. p. 1932–1938.
- [72] Konidaris G, Kaelbling LP, Lozano-Pérez T. Symbol Acquisition for Probabilistic High-Level Planning. In: *Proc. of International Joint Conference on Artificial Intelligence (IJCAI)*; 2015. p. 3619–3627.
- [73] Andersen G, Konidaris G. Active Exploration for Learning Symbolic Representations. In: *Advances in Neural Information Processing Systems*; 2017. p. 5009–5019.
- [74] Bonet B, Geffner H. Learning First-Order Symbolic Representations for Planning from the Structure of the State Space. In: *Proc. of European Conference on Artificial Intelligence*; 2020.
- [75] Lindsay A, Read J, Ferreira JF, et al. Framer: Planning Models from Natural Language Action Descriptions. In: *Proc. of the International Conference on Automated Planning and Scheduling(ICAPS)*; 2017.
- [76] Feng W, Zhuo HH, Kambhampati S. Extracting Action Sequences from Texts Based on Deep Reinforcement Learning. In: *Proc. of International Joint Conference on Artificial Intelligence (IJCAI)*; 2018.
- [77] Barbu A, Narayanaswamy S, Siskind JM. Learning Physically-Instantiated Game Play through Visual Observation. In: *Proc. of IEEE International Conference on Robotics and Automaton (ICRA)*; 2010. p. 1879–1886.
- [78] Kaiser L. Learning Games from Videos Guided by Descriptive Complexity. In: *Proc. of AAAI Conference on Artificial Intelligence*; 2012.
- [79] Argall B, Chernova S, Veloso MM, et al. A Survey of Robot Learning from Demonstration. *Robotics and Autonomous Systems*. 2009;57(5):469–483.
- [80] Ugur E, Piater JH. Bottom-up Learning of Object Categories, Action Effects and Logical Rules: From Continuous Manipulative Exploration to Symbolic Planning. In: *Proc. of IEEE International Conference on Robotics and Automaton (ICRA)*; IEEE; 2015. p. 2627–2633.
- [81] Ahmetoglu A, Seker MY, Sayin A, et al. DeepSym: Deep Symbol Generation and Rule Learning from Unsupervised Continuous Robot Interaction for Planning. *CoRR*. 2020;abs/2012.02532.
- [82] Mitra S, Acharya T. Gesture Recognition: A Survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*. 2007;37(3):311–324.
- [83] Riley M, Cheng G. Extracting and generalizing primitive actions from sparse demonstration. In: 2011 11th IEEE-RAS International Conference on Humanoid Robots; IEEE; 2011. p. 630–635.
- [84] Dong S, Williams BC. Learning and Recognition of Hybrid Manipulation Motions in Variable Environments using Probabilistic Flow Tubes. *International Journal of Social Robotics*. 2012;4(4):357–368.
- [85] Lee SU, Hofmann A, Williams BC. A Model-Based Human Activity Recognition for Human–Robot Collaboration. In: *Proc. of IEEE International Workshop on Intelligent Robots and Systems (IROS)*; IEEE; 2019. p. 736–743.
- [86] Wyatt D, Philipose M, Choudhury T. Unsupervised Activity Recognition using Automatically Mined Common Sense. In: *Proc. of AAAI Conference on Artificial Intelligence*; Vol. 5; 2005. p. 21–27.
- [87] Maekawa T, Watanabe S. Unsupervised Activity Recognition with User’s Physical Characteristics Data. In: 2011 15th Annual International Symposium on Wearable Computers; IEEE; 2011. p. 89–96.
- [88] Vahdatpour A, Amini N, Sarrafzadeh M. Toward unsupervised activity discovery using multi dimensional motif detection in time series. In: *Proc. of International Joint Conference on Artificial Intelligence (IJCAI)*; Citeseer; 2009.
- [89] Trabelsi D, Mohammed S, Chamroukhi F, et al. An Unsupervised Approach for Automatic Activity

Recognition based on Hidden Markov Model Regression. *IEEE Transactions on automation science and engineering*. 2013;10(3):829–835.

- [90] Duckworth P, Alomari M, Gatsoulis Y, et al. Unsupervised Activity Recognition using Latent Semantic Analysis on a Mobile Robot. In: *Proc. of European Conference on Artificial Intelligence*; Vol. 285; 2016. p. 1062–1070.
- [91] Duckworth P, Alomari M, Charles J, et al. Latent Dirichlet Allocation for Unsupervised Activity Analysis on an Autonomous Mobile Robot. In: *Proc. of AAAI Conference on Artificial Intelligence*; Vol. 31; 2017.
- [92] Kaiser L, Babaeizadeh M, Milos P, et al. Model Based Reinforcement Learning for Atari. In: *Proc. of the International Conference on Learning Representations*; 2020.
- [93] Bieszczad A, Kuchar S. Neurosolver Learning to Solve Towers of Hanoi Puzzles. In: *Proc. of International Joint Conference on Computational Intelligence (IJCCI)*; Vol. 3; IEEE; 2015. p. 28–38.
- [94] Agostinelli F, McAleer S, Shmakov A, et al. Solving the Rubik's Cube with Deep Reinforcement Learning and Search. *Nature Machine Intelligence*. 2019;1(8):356–363.

Chapter 3

Logic Meets Learning: From Aristotle to Neural Networks

Vaishak Belle, University of Edinburgh, UK

The tension between deduction and induction is perhaps the most fundamental issue in areas such as philosophy, cognition and artificial intelligence. In this chapter, we survey work that provides evidence for the long-standing and deep connections between logic and learning. After a brief historical prelude, our narrative is then structured in terms of three strands of interaction: logic versus learning, machine learning for logic, and logic for machine learning, but with ample overlap.

3.1. Introduction

Deduction is the process of drawing conclusions from acquired or specified knowledge. So we focus on questions about the expressiveness of formal languages for representing knowledge, together with proof systems for reasoning from that knowledge. Induction is the process of extracting knowledge from observations. So we focus on the semantics of generalization from partial descriptions, together with the target language for representing and the algorithms for computing that generalization. Consequently, frameworks for integrating the two mechanisms in service of artificial or natural cognition is of fundamental theoretical and practical import. Conceptually, the matter is clearly relevant to areas such as philosophy, cognitive science and artificial intelligence, but on a practical level, it impacts science broadly as it attempts to balance expert knowledge with empirical data.

Somewhat surprisingly, historically, approaches to reasoning and learning in AI have attempted to make progress on the topic of cognition almost independently of each other. However, advances in cross-over areas such as *statistical relational learning* [1, 2], *high-level control* [3, 4] and most recently, *neuro-symbolic systems* [5, 6, 7, 8] have illustrated that the dichotomy is not very constructive, and perhaps even ill-formed. Such areas correctly argue that logic emphasizes high-level reasoning, and encourages structuring the world in terms of objects, properties, and relations. In contrast, much of the inductive machinery assume random variables to be independent and identically distributed, which can be problematic when attempting to reason about (physical and social) relationships between groups of objects.

The need to integrate logic and learning can also be motivated without resorting to the apparent flexibility that logical syntax offers for modeling relations and hierarchies. To tackle artificial cognition, it seems we might need to consider approaches that symbolic logic and machine learning struggle to duplicate in functionality individually. For example, although there is much debate about what precisely commonsense knowledge might look like, it is widely acknowledged that concepts such as time, space, abstraction and causality are essential [9, 10]. Low-level sensory data can be processed by machine learning, but beyond that, (classical, or perhaps non-classical) logic can provide the formal machinery to reason about such concepts in a rigorous way. Indeed, it is worth noting that attempts to logically formalize generalizations of sensory data (vision, audio, and so on) have not been successful – they have either proven to be too coarse or too brittle to deal with variations seen in the real-world [11]. So a logical theory for unifying top-down and bottom-up processing also seems infeasible.

In more immediate terms, despite the success of methods such as deep learning, it is now increasingly recognized that owing to a number of reasons, including model re-use, transferability, explainability and data efficiency, those methods need to be further augmented with logical, symbolic and/or programmatic artifacts [12, 13, 14]. Likewise, for building intelligent agents, low-level, data-intensive, reactive computations needs to be tightly integrated with high-level, deliberative computations [3, 4, 15], the latter possibly also engaging in hypothetical and counterfactual reasoning. Here, a parallel is often drawn to Kahneman’s so-called *System 1* versus *System 2* processing in human cognition [16], in the sense that experiential and reactive processing (learned behavior) needs to be coupled with cogitative processing (reasoning, deliberation and introspection) for sophisticated machine intelligence. (Of course, this is not to suggest that developments in one equates to insights in the other, or even that abstract models can easily imbibe cultural and sociopolitical norms in a straightforward manner.)

The purpose of this chapter is not to resolve this debate, but rather briefly survey work and distill representative ideas that provide evidence for the deep connections between logic and learning. We begin with historical developments before turning to the ways we will distill those ideas.

3.2. Lineage

Our goal here is to trace a very rough lineage of ideas. It would be impractical to discuss the intricate philosophical and mathematical issues that arise in logic, learning and their integration. The historical developments of inductive reasoning alone is significant and complex, surely worthy of a book in itself. Likewise, there are various book-length treatments on subfields such as statistical relational learning [2, 1], and of course, the current book on neuro-symbolic learning.

So we are organized as follows. We begin with the type of formal reasoning that distinguishes this enterprise from conventional machine learning: the presence of logical artifacts, their use for postulating hypotheses by arranging symbols, and deriving conclusions in a truth-preserving manner. Formal logic, in the vast majority of applications, is concerned with *deduction* after all. This then provides the background for introducing *induction*, some of the modern flavors of which are the focus of the rest of the chapter. We also briefly position *abduction*, a reasoning approach for most likely explanations.

We remark that this article is undoubtedly a biased view, as the body of related work is large, touching on multiple sub-areas of AI, computer science, philosophy and cognitive science [17]. Readers are encouraged to refer to discussions in our references, among others, to get a sense of the breadth of the area.

3.2.1. Deduction

Deductive reasoning goes back at least to 4th century BC. In a collection of works, Aristotle suggested the use of syllogisms as a means to arrive at *new* and *necessary* truths. New, because the conclusion is not present in the premise, and necessary, because the conclusions are inescapable – provided the rules of inference are irrefutable. As a mechanism for reasoning about discourse and arguments, it is clearly attractive to carefully work out what to infer from a set of premises and it ultimately influenced philosophy considerably [18, 19]. Leibniz, in particular, felt that just as the rules of arithmetic manipulate abstract numbers symbolically, the rules of logic would manipulate abstract ideas symbolically too [20]. Thus, logical reasoning had long been identified with rational thought [21, 22].

It was only much later, in the 19th century primarily, that efforts to algebraize and equationalize logical reasoning by Boole, Frege and others led to formal systems like the ones in use today in computer science and AI. Nowadays, we associate deductive reasoning with at least the following three rules of inference [23, 24, 25]:

- **Modus Ponens.** Suppose p gives q . Assume p . Therefore q .
- **Modus Tollens.** Suppose p gives q . Assume $\neg q$. Therefore $\neg p$.
- **Syllogisms.** Suppose p gives q . Suppose q gives r . Assume p . Therefore r .

(Syllogisms could also take the form of Modus Ponens.) These could be further combined with quantifiers, which allow rules such as:

- **Universal instantiations.** Assume $\forall x P(x)$. Therefore $P(t)$ for any term t . Terms are constants, or variables, or functions applied to constants and variables.

It is the application of universal instantiations with syllogisms that allows Aristotelean arguments such as: all humans are mortal; Socrates is human; therefore, Socrates is mortal.

Given a set of sentences, or theory, T in a logic L , we are interested in the following question: assume T ; is $\alpha \in L$ a logical consequence of T by way of deduction? We could arrive at α either by substituting sentences from T in the rules of inference – i.e., proof-theoretically, or by trying out truth value substitutions to the atoms in T – i.e., semantically. In the latter case, every substitution where T evaluates to true must also be one where α evaluates to true. The celebrated Gödel completeness theorem established the correspondence between these two views: for any T, α in first-order logic, if α is a semantic consequence of T , written $T \models \alpha$, then there is a proof of α from T , written $T \vdash \alpha$.

In the 1960s, *resolution*, which compacts syllogistic reasoning, was established as a single rule of inference that is complete in the above sense for first-order logic. Together with its straightforward application in Horn clausal logic, the development of logic programming and Prolog burgeoned [26]. While all of this is proof-theoretic machinery, se-

mantic solvers have also matured. SAT (Satisfiability) and SMT (Satisfiability Modulo Theory) solvers attempt to find satisfying assignments to propositional and fragments of first-order logic [27]. Like resolution, semantic approaches determine $T \models \alpha$ by refutation: $T \models \alpha$ if and only if $T \wedge \neg\alpha$ has no satisfying assignment. Early SAT solvers used backtracking to search the space of all possible truth value substitutions; modern solvers use a range of advanced techniques from local search (sound but not complete), conflict analysis and parallelization [27].

It is interesting to note that Gödel's other ground-breaking result, the incompleteness theorem, might have led Turing to believe that intelligent machines need a mechanism for induction [28]. The incompleteness theorem states that any first-order logical theory T , which includes Peano's axioms for natural numbers, is either inconsistent or incomplete.

We now briefly review induction, i.e., learning by generalizing from observations. Note that this is not to be confused with mathematical induction, which is used to prove properties for an infinite sequence of objects, and is a form of deduction.

3.2.2. Induction

Not surprisingly, inductive reasoning has been a core issue for the logical worldview, as we need a mechanism for obtaining axiomatic knowledge. Here too, discussions date back to the early Greek philosophers: Aristotle suggested that we need the means for generalizing an argument *from the particular to the universal*. That is, deduction produces *new* knowledge in the form of conclusions not explicitly stored in the knowledge base as beliefs, and induction produces *new* knowledge via statements evidenced only in individual instances. But this knowledge is not *necessary* in the sense of being irrefutable. If a counterexample were provided, the induced statement would have to be amended or in the worst case retracted. The often used example here is that of believing that swans are white until one encounters the less common black swan. It is this fragility of induced beliefs that Hume alluded to in his criticisms of inductive reasoning [29].

However, the usefulness of learning from observations as a mechanism for producing knowledge has led to numerous accounts of induction. For example, through the development of the mathematical area of Statistics, assessing hypotheses has a rigorous regime in many scientific fields. But prominent philosophers, such as De Finetti and Carnap, were also interested in formal accounts of induction [30, 31, 32, 33]. Carnap in particular considered how such statistical notions of evaluating hypotheses could be formulated in first-order logic. Indeed, rather than seeking a generalization (or hypothesis) H that explains all the observations O , we might construct a hypothesis H' such that $\text{Pr}(O \mid H')$ is maximized, where the probability function Pr captures the degree of support for H' .

Since the birth of the field of AI, Plotkin [34], Michalski [35], Shapiro [36], Mugleton [28], De Raedt [37] and others established practical inductive reasoning systems in first-order logic based on generalizing clauses that entail the examples, but variant accounts were developed too, such as the information-theoretic approach of Quinlan [38]. These could be contrasted with much of early (but also many modern) machine learning frameworks (or learners) that are propositional. They also lack a general solution for the provision of axiomatic background knowledge.

3.2.3. Abduction

Although we solely focus on the interaction between deduction and induction here, one other type of logical reasoning is very relevant in our context. Developed by Charles Sanders Pierce [39], *abduction* allows us to infer a hypothesis as a plausible explanation for an observation. The “deductive” capabilities that the fictional detective Sherlock Holmes was most famous for are, in fact, instances of abduction [40]. From a logical perspective, abductive hypotheses might appear to have a similar semantical requirement as inductive hypotheses, in the sense of constructing a sentence such that together with the background knowledge, the examples are entailed. However, while in induction we seek a clause that generalizes examples, in abduction, given a clause that makes a general claim and observations that agree with the conclusion of that claim, we offer the premise of the clause as a plausible explanation. For example, if all swans from Australia are black, and we observe a swan that is (unusually) black as opposed to the white swans we usually encounter, we conclude that the swan must be from Australia.

We will not dwell on abduction anymore, but note that, for example, in [41, 42], a case is made for neuro-symbolic integration as an abductive process. Even earlier to this, Poole [43, 44] established a semantic foundation for a probabilistic logic programming language called ICL (independent choice logic) via abduction. ICL is a precursor of many probabilistic logical modeling languages seen in statistical relational learning [1].

3.2.4. A Modern Desiderata

In the recent years, there has been tremendous development on the interface between learning and logic, and symbolic structures more generally [45]. The impetus for these developments stems from the fact that symbolic logic served as the calculus for modeling information in artificial intelligence and computer science for several decades [46, 47]. But the sheer intricacy and noise in the real-world coupled with the incompleteness of specifications and requirements at the start of any problem solving endeavor has motivated modelers to look to data for completeness and robustification.

This is not without its challenges, however. A great deal of effort is needed to adapt conventional learners based on labelled-data to the involved syntactic structures widely used in computer science and artificial intelligence, including exotic logics and programs. In fact, just the extension of learning schemes from atomic random variables to clausal logic and probabilistic extensions of clausal logic has led to the subfields of inductive logic programming and statistical relational learning. These subfields continue to enjoy active development [48, 49].

But leaving such syntactic matters aside, there are a number of fundamental questions that probe at the heart of the logic-learning enterprise, such as:

- *What knowledge does a system need to have in advance – i.e., provided by the modeler – versus what can be acquired by observations?*

That is, are we only learning the probabilities of a given knowledge base, or are we also learning the knowledge base itself? Do we need to assume the provision of some background knowledge before the learning process is initiated (i.e., the question of what is innate to the agent before it can effectively learn from its environment)? Is this determined on a case-by-case basis, or can we arrive at broader conditions for this delineation?

- *What is the language for representing and reasoning with the background knowledge and observations?*

That is, is it propositional, first-order, modal, or some probabilistic extension thereof? Do we assume the provision of labeled training data denoting the observations, or are we considering systems that purposefully act in an environment, and then receive observations and rewards as a result of those actions?

- *What kind of semantics governs the updating of a priori knowledge given new and possibly conflicting observations?*

That is, do we assume all observations are provided before the hypothesis is generated? What is the appropriate mechanism for learning the rules and repairing them as more, and possibly, conflicting observations are made? Do we have a means to quantify the generalization capabilities of the learner with respect to unknown ground truth?

- *What are the rules of inference that apply to the resulting knowledge base?*

That is, is the query language unrestricted wrt logical connectives? Do we permit proofs of arbitrary depth? What is the mechanism for requesting “shallow” versus “deep” reasoning (e.g., quickly reacting to a tiger versus reflecting on the nature of the universe)?

- *How does the system generalize from low-level observations to high-level structured knowledge?*

That is, do we assume the vocabulary of the high-level language is available a priori, or do we need to discover a (possibly hierarchical) mapping between high-level predicates and low-level atomic observations? Is there a means to generate examples to support or refute a claim made using high-level predicates?

To a large extent, much of the recent work on logic and learning, including deep learning, are attempting to address such questions by formulating methodological or algorithmic solutions. It would be impossible to report on every such development, of course, so in this chapter we mainly look at some representative ideas. Our narrative is structured in terms of three strands, inspired by [50]:

1. *Logic versus Machine Learning*: the study of problems that can be solved using either logical techniques or via machine learning;
2. *Machine Learning for Logic*: the learning of logical formulas and logic programs, but also the use of machine learning to guide search; and
3. *Logic for Machine Learning*: the use of logic to enhance machine learning, identify the boundary between tractable and intractable learning problems, and as a declarative language for expressing machine learning problems.

At the outset, we note that the focus is on logical languages, in the sense of a formal framework embodying inference rules in the manner discussed earlier, and not simply the induction of symbols lacking a formal semantics. In a way, symbolic induction is much larger and more ambiguous problem space: after all, even conventional neural network outputs can be interpreted symbolically.

Through the course of our discussion, we also focus on the following sore point: there is a common misconception that logic is for discrete properties, whereas probability theory and machine learning, more generally, is for continuous properties. It is true that logical formulas are discrete structures, but they can very easily also express properties

about countably infinite or even uncountably many objects. Consequently, in this article we survey some recent results that tackle the integration of logic and learning in infinite domains.

It is worth remarking that the importance of *logic programming* to the thrust of this article *cannot be overstated*. Indeed, it is logic programming that set the stage for numerous deduction-based programming languages, from declarative modeling [26, 51] to reasoning about choices and probabilities [43, 1] to reasoning about actions and plans [52]. Likewise, inductive logic programming set the stage for major practical advances on learning theories and programs [28, 49]. Probabilistic extensions to logic programming and inductive logic programming [1], which we also touch upon in the below subsection, continue to invite new integrations of relational and graph-based modeling with learning. Many neural learning schemes to integrate logical reasoning do so via logic programming [14]. Thus, although various other languages are being developed too, logic programming provides a convenient and fairly accessible apparatus to integrate reasoning and learning, and study questions of the sort discussed earlier.

3.2.5. Logic & Probability

Before turning to the three strands, let us briefly reflect on a very related enterprise, that of bridging logic and probability. In so much as one would view induction as a probabilistic process, our discussions above already suggest deep connections between logic and probability. But there is much more to be said about the role of probabilities for logic, and vice versa.

Philosophers from the 19th century, including John Stuart Mill and George Boole, felt (human) reasoning would involve both logic and probability. In the earliest days of AI, McCarthy [53] put forward a profound idea: he posited that what the system needs to know could be represented in a *formal language*, and a general-purpose algorithm would then conclude the necessary actions needed to solve the problem at hand. The main advantage is that the representation can be scrutinized and understood by external observers, and the system's behavior could be improved by making statements to it. All of this was ultimately grounded in logic, of course. But pervasive uncertainty in almost every domain of interest led to alternative formalisms for representing knowledge. Uncertainty could range from measurement errors (e.g. readings from a thermometer) to the absence of categorical assertions (e.g. smoking may be a factor for cancer, but cancer is not an absolute consequence for smokers) to “latent” factors that the modeler may simply not have taken into account, all of which question the legitimacy of the expert knowledge. Moreover, classical logic is seen as being “rigid” (sentences always evaluate to true or false) and “brittle” (sentences in the knowledge base must be true in all possible worlds). Although this in turn led to probabilistic formalisms such as Bayesian and Markov networks, as argued the relational structure of the world has meant we have now come a full circle: probabilistic logics are logical languages that admit probabilistic assertions [54, 55, 56]. Semantically, in classical logic, $T \models \alpha$ implies that every truth assignment where T is true is also one where α is true. In probabilistic logics, logical and probabilistic assertions in T mean that there is a measure on the set of truth assignments, and we are often interested in the probability of α being true.

Currently, the most popular flavors of such probabilistic logics are syntactic extensions to probabilistic formalisms, as seen in statistical relational learning [1]. That is,

the use of logical syntax allows one to define an involved probabilistic model, but it can often be interpreted at a propositional level with probabilities over propositional interpretations. For example, the weighted formula [1]:

$$0.9 \quad \forall x, y. (Smokers(x) \wedge Friends(x, y) \supset Smokers(y))$$

over a finite universe corresponds to a fully interconnected Markov network; the random variables in the network are obtained from all the ground atoms. This sentence says that the likelihood of friends of smokers being smokers themselves is very high. Likewise, when logic programs are decorated with probabilities, it enables reasoning about possible worlds and can capture intricate probabilistic models involving deterministic and stochastic constraints [43, 57]. For a slightly more complex version of the friends-smokers example from above in ProbLog [57], one such logic programming language, consider the below program:

```
0.1::stress(X) :- person(X).
0.2::friend(X,Y) :- person(X), person(Y).
smokes(X) :- stress(X).
smokes(X) :- friend(X,Y), smokes(Y).
```

This program says that the likelihood of a person being stressed is low, as is the likelihood of two people taken at random being friends. However, stress categorically leads to smoking, and friends of smokers are certainly smokers too.

Interestingly, such logical languages for probabilistic modeling also enables the use of logic-based inference technology, which we will be review further in the subsection below.

Formulating probabilistic graphical models as weighted logical formulas is an elegant approach to decouple the constraints and dependencies from the probabilistic parameters. However, at a conceptual level, there is little gained as we are still embedded in the framework of standard probability theory. When the representation language is thought of as a model of a system's mental state, we need to be able to reason about probabilistic events in a more general way. For example, we may need to compare the probabilities of hypothetical outcomes, or analyze the behavior of non-terminating action sequences. In some cases, we may even be ignorant about the actual probabilities of events. Such concerns were raised already by McCarthy and Hayes [58], who argued that assigning probabilities to quantifiers and logical connectives is not straightforward, and it should not be required that every statement be accorded a probability. This then leads to extensions of first-order logic that allow classical first-order sentences to be declared alongside weighted assertions, in addition to any dynamic and causal laws about how objects and actions interact in the world [59, 55, 60, 61]. For example, such languages can express statements such as:

- $\Pr(p) > \Pr(q)$ and $\Pr(p) < [a] \Pr(p)$
- $\Pr(\forall x Q(x)) \neq 0$
- $\Box(\Pr(p \vee q) = \Pr(p) + \Pr(q) - \Pr(p \wedge q))$

These say that (respectively): the probability of p is more than that of q , that of p becomes lesser after doing action a , the probability ascribed to all instances of Q being true is

non-zero, and after any number of actions, the axiom of probability regarding the union of events necessarily holds. Consider, for example, p to denote that a robot is close to the wall; if a is the action of moving away from the wall then surely $\Pr(p) < [a]\Pr(p)$.

Although such logics are often intractable when taken in their full generality, the view taken is that as a language, it should be as general as possible: we are then in a position to investigate fragments that enjoy reasonable computational properties.

Before concluding this interlude, it is worth remarking about two classes of development. First, there are a number of results that allude to the statistical properties of logic. For example, an influential result by Fagin [62] established the Zero-One Law for first-order logic. For any first-order formula ϕ , consider the probability $\Pr_n$ of ϕ being true in first-order interpretations of size n . As n tends to infinity, $\Pr_n(\phi)$ will either be 0 or 1. Likewise, the discovery of a phase transition phenomena for propositional satisfiability [63] also implies profound probabilistic properties underlying logical reasoning. Second, probabilities are but one approach to quantify uncertainty, perhaps the most common and an obvious choice for many modeling situations. Of course, there are other approaches for specifying priors and defining posteriors [64]. One proposal that attempts to redefine logical truth altogether is fuzzy logic and related proposals [65]. These relax the truth value to atoms: in classical logic, we assign either a value of 1 or 0; in these logics, they can be any real number in the interval $[0,1]$. Consequently, the truth value of non-atomic formulas is then defined as an arithmetic function over such real numbers. As processing functions over such real numbers is simpler in some situations than reasoning about possible worlds as required by probability theory, real-valued logics are very popular in both statistical relational learning [66] and neuro-symbolic learning [67].

In what follows, we mostly restrict our attention to either classical or probabilistic logical languages, but as the above discussion indicates, other approaches are available, and they come with their own distinctive features.

3.3. Logic versus Machine Learning

To appreciate the role and impact of logic-based solvers for machine learning systems, it is perhaps useful to consider the core computational problem underlying (probabilistic) machine learning: the problem of inference, including evaluating the partition function and conditional probabilities of a probabilistic graphical model such as a Bayesian network.

When leveraging Bayesian networks for machine learning tasks [68], the networks are often learned using local search to maximize a likelihood or a Bayesian quantity. For example, given data D and the current guess for the network N , we might estimate the “goodness” of the guess by means of a score: $score(N, D) \propto \log \Pr(D | N) - size(N)$. That is, we want to maximize the fit of the data wrt the current guess, but we would like to penalize the model complexity, to avoid overfitting. Then, we would opt for a second guess N' only if $score(N', D) > score(N, D)$. Note that in as much as N represents a (scientific) hypothesis, and D represents the observations, we are very much following in the spirit of early work by Carnap and others on evaluating hypotheses via $\Pr(O | H)$.

3.3.1. Weighted Model Counting

Needless to say, even with a reasonable local search procedure for guessing hypotheses, the most significant computational effort here is that of probabilistic inference. Reasoning in such networks becomes especially challenging with logical syntax. Intuitively, where previously we wished to draw samples $z_1, \dots, z_k$ from a distribution, we would now need to ensure that these samples are consistent with logical constraints that might need to be always true. These constraints can be thought of as making arbitrarily complex regions of the probability space invalid, and so can very challenging to efficiently sample from.

The prevalence of large-scale social networks, machine reading domains, and other types of relational knowledge bases has led to numerous formalisms that borrow the syntax of predicate logic for probabilistic modeling [69, 70, 71, 72]. This has led to a large family of solvers for the *weighted model counting* (WMC) problem [73, 74]. The idea is this: given a Bayesian network, a relational Bayesian network, a factor graph, or a probabilistic program [75], one considers an encoding of the formalism as a *weighted propositional theory*, consisting of a propositional theory Δ and a weight function w that maps literals in Δ to $\mathbb{R}^+$. Recall that SAT is the problem of finding an assignment to such a Δ , whereas #SAT counts the number of assignments for Δ . WMC extends #SAT by computing the sum of the weights of all assignments: that is, given a set of models that satisfy Δ , we evaluate the quantity $W(\Delta) = \sum_{M \models \Delta} w(M)$ where $w(M)$ is factorized in terms of the literals true at M . To obtain the conditional probability of a query q against evidence e (wrt the theory Δ), we define $\Pr(q \mid e) = W(\Delta \wedge q \wedge e) / W(\Delta \wedge e)$. (Although widely practiced, it turns out that factorization of the weight function in terms of literals is limiting: not only does it necessitate the need for additional variables and clauses, but it also constraints the space of discrete probability distributions that can be captured by WMC. See [76, 77] for discussions on alleviating this restriction.)

The popularity of WMC can be explained as follows. Its formulation elegantly decouples the logical or symbolic representation from the numeric representation, which is encapsulated in the weight function. When building solvers, this allows us to reason about logical equivalence and reuse SAT solving technology (such as constraint propagation and clause learning). WMC also makes it more natural to reason about deterministic, hard constraints in a probabilistic context [74]. For ideas on generating such representations randomly to assess scalability and compare inference algorithms, see [78], for example.

Exact WMC solvers are based on knowledge compilation [79, 80], exhaustive DPLL search [81] or pseudo-Boolean function manipulation [82, 76, 77]. Approximate WMC algorithms use local search [83] or sampling [84]. Here, knowledge compilation is worth particularly highlighting, where the logical theory is transformed to a data structure called a *circuit* [74]. Knowledge compilation [85] arose as a way to represent logical theories in a manner where certain kinds of computations (e.g., checking satisfiability) is significantly more effective, often polynomial in the size of the circuit. In the context of probabilistic inference, the idea was to then position probability estimation to also be computable in time polynomial in the size of the circuit [74, 86]. In the recent years, knowledge compilation has also been extended to probabilistic relational models (but in a finite domain setting) [87], referred to as *lifted inference*.

In sum, the discussion above suggests using WMC to compute $\Pr(D \mid N)$, especially if N has logical syntax. Implicit in the suggestion is that the probabilistic estimation

machinery of WMC could also be used for tasks such as *parameter learning*. Roughly, if the hypothesis is a probabilistic representation, as is the case in this section, the *structure* determines the conditional dependencies between random variables, and the *parameters* determines the probability function associated with those variables. Parameter learning involves computing the likelihood of a guessed weight against the data, and so WMC can be used for that probabilistic estimation. But, of course, it is also possible to consider *structure learning* while using WMC for appropriate probabilistic estimation tasks. In particular, if N is logical artifact, then this is the case of using machine learning for logic – our next section – where we will consider this task in more detail. In the first instance, for example, [88] provides an end-to-end treatment of parameter and structure learning with probabilistic logical representations.

3.3.2. Beyond Propositional Languages

Lifted reasoning techniques attempt to leverage the relational structure of probabilistic logical models so as to avoid the exponentially large propositional theory that may result if the variables are substituted by constants. For example, the formula $\forall x(P(x) \vee Q(x))$ has 3^n models (out of the 4^n possible assignments for a domain of size n leading to $2n$ propositions in the ground theory). But still n is a fixed natural number. Thus, there is a computational benefit, but logically, we might as well assume a propositional language, and the formulation is limited to discrete random variables. At least on the logical front, a similar observation can be made for SAT, which for the longest time could only be applied in discrete domains. This changed with the increasing popularity of *satisfiability modulo theories* (SMT) [89], which enable us to, for example, reason about the satisfiability of linear constraints over the reals. Extending earlier insights on piecewise-polynomial weight functions [90, 91], the formulation of *weighted model integration* (WMI) was proposed in [92]. WMI extends WMC by leveraging the idea that SMT theories can represent mixtures of Boolean and continuous variables: for example, a formula such as $p \wedge (x > 5)$ denotes the logical conjunction of a Boolean variable p and a real-valued variable x taking values greater than 5. For every assignment to the Boolean and continuous variables, the WMI problem defines a weight. The total WMI is computed by integrating these weights over the domain of solutions to Δ , which is a mixed discrete-continuous (or simply *hybrid*) space. Consider, for example, the special case when Δ has no Boolean variables, and the weight of every model is 1. Then, the WMI simplifies to computing the volume of the polytope encoded in Δ . When we additionally allow for Boolean variables in Δ , this special case becomes the hybrid version of #SAT, known as #SMT [93]. In particular, previously we assumed that w mapped literals in Δ to $\mathbb{R}^+$, but now we will allow the mapping to any arithmetic expression to capture density functions. For example, mapping an interval $0 < x < 5$ to a number 0.3 would capture a piecewise-constant density, but if mapped to x^3 , we would be admitting piecewise-polynomial densities. WMI is defined in much the same way as WMC as the sum of weights of models, except that we would need to additionally integrate the density function.

Since that proposal, numerous advances have been made on building efficient WMI solvers (e.g., [94, 95, 96]), including the development of knowledge compilation strategies for the extended language [97, 98, 99]. In [100], a lifted strategy for WMI is considered.

WMI proposes an extension of WMC for uncountably infinite (i.e., continuous) domains. What about countably infinite domains? The latter type is particularly useful for

reasoning in quantified first-order settings, where we may say that a property such as $\forall x, y, z (\text{parent}(x, y) \wedge \text{parent}(y, z) \supset \text{grandparent}(x, z))$ applies to every possible x, y and z . Of course, in the absence of the finite domain assumption, reasoning in the first-order setting suffers from undecidability properties, and so various strategies have emerged for reasoning about an *open universe* [101]. One popular approach is to perform *forward reasoning*, where samples needed for probability estimation are obtained from the facts and declarations in the probabilistic model [101, 102]. Each such sample corresponds to a possible world. But there may be (countably or uncountably) infinitely many worlds, and so exact inference is usually sacrificed. A second approach is to restrict the model wrt the query and evidence atoms and define estimation from the resulting finite sub-model [103, 104, 105], which may also be substantiated with exact inference in some cases [106, 107]. For example, in [106, 107], the following property is proved: given a first-order clausal theory Δ where the universal quantifier ranges over a countably infinite set and a ground query α , $\Delta \models \alpha$ can be determined via refutation in a propositional language with finitely many propositional variables. In general, if we are able to establish results where first-order properties in a restricted fragment can be reduced to propositional reasoning, we might look to consider reasoning and learning with probabilistic extensions to such fragments.

In the next section, we will be interested in the learning of logical artifacts. Given the successes of logic-based solvers for inference and probability estimation, the next section will also feature the application of such solvers to learning models with relational features and deterministic constraints.

3.3.3. Neurally-guided Search

Before turning to the next section, it is worth briefly reflecting where recent advances in machine learning, particularly deep learning, might play a role. This avenue is burgeoning so rapidly it would be difficult given the scope of this article to discuss representative ideas in any suitable detail. Thus, we will only mention a few interesting trends, for each of the three strands of logic vs learning (discussed immediately below), learning for logic and logic for learning (in subsequent subsections).

To begin with, the learning of hypothesis and representations is clearly one venue where mainstream learners could prove useful, but we defer this to the next section. As far as inference is particularly concerned, there have been initiatives to search for logically consistent artifacts via deep learning. (The precursor of this is perhaps classical clausal search in inductive logic programming, defined by the generality of the clauses, over some structured hypothesis space.) These artifacts could be satisfying assignments, program completions, and more generally, theorems. For example, in [108] the discovery of first-order theorems is enabled by empowering the search of proofs with learning. In [109], the search for solutions for NP-complete problems is explored via graph neural networks [7]. In [110], the completion of programs against a partial specification – i.e., partially-specified program – is augmented using learning. The key trade-off worth noting here is the balance of prediction against explicit symbolic reasoning to achieve maximal logical consistency. Finally, in [111], the approximation of WMC is explored via deep learning.

3.4. Machine Learning for Logic

Expert knowledge is hard to come by, difficult to articulate at different levels of granularity and is often brittle. We need a mechanism for obtaining axiomatic knowledge.

3.4.1. Entailment versus Bayesian Scoring

The learning of logical and symbolic artifacts is an important issue in AI, and computer science more generally [45]. There is a considerable body of work on learning propositional and relational formulas, and in context of probabilistic information, learning weighted formulas [50, 112, 113, 1]. Approaches can be broadly lumped together as follows:

1. *Entailment-based scoring*: Given a logical language L , background knowledge $B \subset L$, examples D (usually a set of L -atoms), find a hypothesis $H \in \bar{H}, H \subset L$ such that $B \cup H$ entail the instances in D . Here, the set $\bar{H}$ places restrictions of the syntax of H so as to control model complexity and generalization. (For example, $H = D$ is a trivial hypothesis that satisfies the entailment stipulation.)
2. *Likelihood-based scoring*: Given L, B and D as defined above, find $H \subset L$ such that $\text{score}(H, D) > \text{score}(H', D)$ for every $H' \neq H$. As discussed before, we might define $\text{score}(H, D) \propto \log \Pr(D | H) - \text{size}(H)$. Here, like $\bar{H}$ above, $\text{size}(H)$ attempts to the control model complexity and generalization.

Generally speaking, the idea is to find hypotheses that do not assume more information than provided, but nonetheless capture all of the examples. In that regard, entailment-based approaches attempt to find the *least general* clause, in the sense that the hypothesis is a clause that entails all and only the examples. Likelihood-based scoring attempt to find a suitable representation that captures the *distribution of the data*, in the sense that sampling from the representation will recover the data.

In the most standard setting, Bayesian network learning algorithms would be of the first type, and inductive logic programming of the second type. But in many recent works, the distinction is not strict. For example, we may use entailment-based inductive synthesis for an initial estimate of the hypothesis, and then resort to Bayesian scoring models to refine that hypothesis [70]. The synthesis step might invoke neural machinery [14]. We might not require that the hypothesis entails every example in D but only the largest consistent subset, which is sensible when we expect the examples to be noisy [113], or if the search for such a comprehensive hypothesis is infeasible or computationally expensive. We might compile B to circuit, and perform Bayesian scoring on that structure [114], and so B could be seen as deterministic domain-specific constraints. Finally, we might stipulate the conditions under which a “correct” hypothesis may be inferred wrt unknown ground truth, only a subset of which is provided in D . This is perhaps best represented by the (probably approximately correct) PAC-semantics that captures the quality possessed by the output of learning algorithm whilst costing for the number of (noisy) examples that need to be observed [115, 116, 117].

Broadly speaking, then, there are *three major flavors of learning*: statistical relational learning, inductive logic programming, and learning under PAC-semantics. Not surprisingly, there are significant differences in terms of the learning regime, the notion of correctness, and the underlying algorithmic machinery, although as suggested above,

there continues to be a cross-pollination of ideas. For example, fragments of inductive logic programming can be shown to be PAC-learnable [118], in the sense of postulating the conditions under which the learning is tractable. However, as mentioned, inductive logic programming treats the input examples as defining a domain for which we aim to construct a hypothesis. In PAC-Semantics, on the one hand, we only seek to bound the probability that queries are true. But on the other, the framework allows for some unknown distribution over models of the underlying theory, from which we can sample. We never assume access to complete knowledge of this theory, or even the distribution. This makes the setting significantly different from inductive logic programming.

It is worth noting that many of these ideas carry over to the learning of symbolic (but not explicitly logical) artifacts, such as programs [45, 110, 119].

3.4.2. Learning in Infinite Spaces

Recall the interplay between SAT and probabilistic inference, on the one hand, and probabilistic inference and learning, on the other. By means of logical and probabilistic reasoning tools for arithmetic fragments such as SMT and WMI, it should be clear that it now becomes possible to extend learning schemes to continuous and mixed discrete-continuous spaces. All three flavours of learning have, in fact, been adapted to the continuous setting in the recent years.

For example, one could learn SMT formulas by guessing a sentence that entails all and only the examples [120]. For an account that also tries to evaluate a SMT formula hypothesis that is correct wrt unknown ground truth via the PAC-semantics, see [121]. In [122], it is further shown how the PAC-semantics approach could be compared against other induction schemes for such arithmetic fragments.

If the overall objective is to obtain a distribution of the data, other possibilities present themselves. For example, extending the work on learning SMT formulas, the learning of weighted SMT formulas is also considered in [120]. That is, just as learning probabilistic propositional and relational sentences corresponds to representations for which we can compute WMC, the learning of weighted SMT sentences essentially corresponds to learning WMI formulations.

Such ideas could be integrated with circuits, where as discussed, probability estimation was positioned to be computable in time polynomial in the size of the structure [74, 86]. In [123, 124], by means of likelihood-based scoring, probabilistic circuits are constructed, where the internal nodes of the structure would represent arithmetic formulas, that might be additionally decorated with probability mass or density functions.

WMI, and closely-related notions, can also be considered with rule learning. In [125], real-valued data points are first lumped together to obtain atomic continuous random variables. From these, relational formulas are constructed so as to yield hybrid probabilistic programs. The rule learning is based on likelihood scoring. In [126], the real-valued data points are first intervalized, and polynomials are learned for those intervals based on likelihood scoring. These weighted atoms are then used for learning clauses by entailment judgements, which is an extension to the discrete setting [113].

What about countably infinite domains? There is a long history in philosophical logic on learning universals [31, 33], as indicated in our discussion on Aristotle. Early in the history of inductive synthesis, numerous proposals were considered in terms of learning in the limit [127, 36, 128]. In most pragmatic instances on learning logical artifacts,

however, the difference between the uncountable and countably infinite setting is this: in the former, we see finitely many real-valued samples as being drawn from an (unknown) interval, and we could inspect these samples to crudely infer a lower and upper bound. In the latter, based on finitely many relational atoms, we would need to infer a universally quantified clause, such as $\forall x, y, z (\text{parent}(x, y) \wedge \text{parent}(y, z) \supset \text{grandparent}(x, z))$. If we are after a hypothesis that is simply guaranteed to be consistent wrt the observed examples, then standard rule induction strategies might suffice [112], and we could naively interpret the rules as quantifying over a countably infinite domain. But this is somewhat unsatisfactory, as there is no distinction between the rules learned in the standard finite setting and its supposed applicability to the infinite setting. What is really needed is an analysis of what rule learning would mean wrt the infinitely many examples that have *not* been observed. This was recently considered via the PAC-semantics in [129], by appealing to ideas on reasoning with open universes discussed earlier [106].

3.4.3. Gradient-based Structure Learning

Because data can be noisy, and there is a need to build representations at scale, neural approaches to rule induction are on the rise. In [14, 130], for example, inductive logic programming is reconsidered in a neural setting, by extracting the program from a latent representation. In [131], a latent representation of a probabilistic relational model is considered. In [67], a fuzzy logical semantics is shown to capture the representation and reasoning of neural network outputs.

Before concluding this section, it is worth noting that although the above discussion is primarily related to the learning of logical artifacts, it can equivalently be seen as a class of machine learning methods that leverage symbolic domain knowledge. Indeed, logic-based probabilistic inference over deterministic constraints, and entailment-based induction augmented with background knowledge are instances of such a class [132, 6]. Analogously, the automated construction of relational and statistical knowledge bases [133, 134] by combining background knowledge with extracted tuples (obtained, for example, by applying natural language processing techniques to large textual data) is another instance of such a class [135, 136]. In the next section, we will consider yet another way in which logical and symbolic artifacts can influence learning: we will see how such logical artifacts are useful to enable tractability, correctness, modularity and compositionality.

3.5. Logic for Machine Learning

There are two obvious ways in which a logical framework can provide insights on machine learning theory. First, consider that computational tractability is of central concern when applying logic in computer science, knowledge representation, database theory and search [137, 138, 139]. Thus, the natural question to wonder is whether these ideas would carry over to probabilistic machine learning. On the one hand, probabilistic extensions to tractable knowledge representation frameworks could be considered [140]. But on the other, as discussed previously, ideas from knowledge compilation, and the use of circuits, in particular, are proving very effective for designing tractable paradigms for machine learning. While there has always been an interest in capturing tractable distri-

butions by means of low tree-width models [141], knowledge compilation has provided a way to also represent high tree-width models and enable exact inference for a range of queries [79, 142, 86, 114]. See [143] for a comprehensive view on the use of knowledge compilation for machine learning.

The other obvious way logic can provide insights on machine learning theory is by offering a formal apparatus to reason about *context*. Machine learning problems are often positioned as atomic tasks, such as a classification task where regions of images need to be labeled as cats or dogs. However, even in that limited setting, we imagine the resulting classification system as being deployed as part of a larger system, which includes various modules that communicate or interface with the classification system. We imagine an implicit accountability to the labelling task in that the detected object is either a cat or a dog, but not both. If there is information available that all the entities surrounding the object of interest have been labelled as lions, we might want to (heuristically) accord a high probability to the object being a cat, possibly a wild cat. There is a very low chance of the object being a dog, then. If this is part of a vision system on a robot, we should ensure that the robot never tramples on the object, regardless of whether it is a type of cat or a dog. To inspect such patterns, and provide meta-theory for machine learning, it can be shown that symbolic, programmatic and logical artifacts are extremely useful. We will specifically consider correctness, modularity and compositionality to explore the claim.

We remark that there is not much to be said about the distinction between finite versus infinite wrt most of these properties. In principle, many of these ideas are likely amenable to extensions to an infinite setting in the ways discussed in the previous sections (e.g., considering constraints of a continuous or a countably infinite nature). So we will not dwell on the size of the domain in quite an explicit manner as before.

3.5.1. Correctness

On the topic of correctness, the classical framework in computer science is *verification*: can we provide a formal specification of what is desired, and can the system be checked against that specification? In machine learning, we might ask whether the system, during or after training, satisfies a specification. The specification here might mean constraints about the physical laws of the domain, or notions of perturbation in the input space while ensuring that the labels do not change, or insisting that the prediction does not label an object as being both a cat and a dog, or otherwise ensuring that outcomes are not subject to, say, gender bias. Although there is a broad body of work on such issues, touching more generally on *trust* [144], we discuss approaches closer to the thrust of this article. For example, [145] show that a trained neural network can be verified by means of an SMT encoding of the network. In recent work, [146] show that the loss function of deep learning systems can be adjusted to logical constraints by insisting that the distribution on the predictions is proportional to the weighted model count of those constraints. In [114], prior (logical) constraints are compiled to a circuit to be used for probability estimation. In [147], circuits are shown to be amenable to training against probabilistic and causal prior constraints, including assertions about fairness, for example. In [148], fairness is formalized in a probabilistic relational language to enable equitable properties in, say, social networks by explicitly relating the (human) entities of the domain.

3.5.2. Modularity

In [149, 15], a somewhat different approach to respecting domain constraints is taken: the low-level prediction is obtained as usual from a machine learning module, which is then interfaced with a probabilistic relational language and its reasoning apparatus. That is, the reasoning is positioned to be tackled directly by the symbolic engine. In a sense, such approaches cut across the three strands: the symbolic engine uses weighted model counting, the formulas in the language could be obtained by (say) entailment-based scoring, and the resulting language supports modularity and compositionality (discussed below).

On the topic of modularity, recall that the general idea is to reduce, simplify or otherwise abstract a (probabilistic) computation as an atomic entity, which is then to be referenced in another, possibly more complex, entity. (In this regard, modularity also aids in assessing the correctness of systems since one can inspect the modules independently too, in addition to the overall behavior.) In standard programming languages, this might mean the compartmentalization and interrelation of computational entities. For machine learning, approaches such as probabilistic programming [150, 151] support probabilistic primitives in the language, with the intention of making learning modules re-usable and modular. It can be shown, for example, that the computational semantics of some of these languages reduce to WMC [152, 153]. Thus, in the infinite case, a corresponding reduction to WMI follows [154, 126, 155].

A second dimension to modularity is the notion of *abstraction*. Here, we seek to model, reason and explain the behavior of systems in a more tractable search space, by omitting irrelevant details. The idea is widely used in natural and social sciences. Think of understanding the political dynamics of elections by studying micro level phenomena (say, voter grievances in counties) versus macro level events (e.g., television advertisements, gerrymandering). In particular, in computer science, it is often understood as the process of mapping one representation onto a simpler representation by suppressing irrelevant information. In fact, integrating low-level behavior with high-level reasoning, exploiting relational representations to reduce the number of inference computations, and many other search space reduction techniques can all loosely be seen as instances of abstraction [156].

While there has been significant work on abstraction in deterministic systems [157], for machine learning, however, a probabilistic variant is clearly needed. In [158], an account of abstraction for loop-free propositional probabilistic programs is provided, where certain parts of the program (possibly involving continuous properties) can be reduced to a Bernoulli random variable. For example, suppose every occurrence of the continuous random variable x , drawn uniformly on the interval $[0,1]$, in a program is either of the form $x \leq 7$ or of the form $x > 7$. Then, we could use a discrete random variable b with a 0.7 probability of being true to capture $x \leq 7$; and analogously, $\neg b$ to capture $x > 7$. The resulting program is likely to be simpler. In [156], an account of abstraction for probabilistic relational models is considered, where the notion of abstraction also extends to deterministic constraints and complex formulas. For example, a single probabilistic variable in the abstracted model could denote a complex logical formula in the original model. Moreover, the logical properties that enable verifying and inducing abstractions are also considered, and it is shown how WMC is sufficient for the computability of these properties (also see [153]).

Incidentally, abstraction brings to light a reduction from infinite to finite: it is shown in [156] that the modeling of piecewise densities as weighted propositions, which is

leveraged in WMI [92, 154], is a simple case of the more general account. Therefore, it is worthwhile to investigate whether this or other accounts of abstraction could emerge as general-purpose tools that allow us to inspect the conditions under which infinitary statements reduce to finite computations.

A broader point here is the role abstraction might play in generating explanations [159, 160]. For example, a user's understanding of the domain is likely to be different from the low-level data that a machine learning system interfaces with [161], and so, abstractions can establish a map these two levels in a formal way.

3.5.3. Compositionality

Finally, we turn to the topic of compositionality, which, of course, is closely related to modularity in that we want to distinct modules to come together to form a complex composition. Not surprisingly, this is of great concern in AI, as it is widely acknowledged that most AI systems will involve heterogeneous components, some of which may involve learning from data, and others reasoning, search and symbol manipulation [9]. In continuation with the above discussion, probabilistic programming is one such endeavor that purports to tackle this challenge by allowing modular components to be composed over programming and/or logical connectives [150, 151, 70, 15, 149, 162, 163, 164, 165, 61]. (See [139, 166, 167] for ideas in deterministic systems.) However, probabilistic programming only composes probabilistic computations, but does not offer an obvious means to capture other types of search-based computations, such as SAT, and integer and convex programming.

Recall that the computational semantics of probabilistic programs reduces to WMC [152, 153]. Following works such as [168, 169], an interesting observation made in [170] is that by appealing to a sum of products computation over different semiring structures, we can realize a large number of tasks such as satisfiability, unweighted model counting, sensitivity analysis, gradient computations, in addition to WMC. It was then shown in [171] that the idea could be generalized further for infinite domains: by defining a measure on first-order models, WMI and convex optimization can also be captured. As the underlying language is a logical one, composition can already be defined using logical connectives. But an additional, more involved, notion of composition is also proposed, where a sum of products over different semirings can be concatenated. To reiterate, the general idea behind these proposals [169, 170, 171] is to arrive at a principled paradigm that allows us to interface learned modules with other types of search and optimization computations for the compositional building of AI systems. See also [172] for analogous discussions, but where a different type of coupling for the underlying computations is suggested. Overall, we observed that a formal apparatus (symbolic, programmatic and logical artifacts) help us define such compositional constructions by providing a meta-theory.

3.5.4. Empowering Neural Artifacts

As argued above, concerns of correctness, modularity and compositionality apply to almost all of machine learning. Given the vast number of parameter and architectural choices in deep learning, such concerns are particularly prominent. We have reviewed corresponding advances in the above sections, but we reiterate the key ideas for the sake of completeness.

On the topic of correctness, ensuring that the training of neural networks respect safety and physical constraints has received considerable attention [145, 146], ranging from the use of SMT encodings to verify the sensitivity of trained models to the use of WMC for capturing domain knowledge whilst training models.

On the topic of modularity and compositionality, a general observation is that it is clearly advantageous to combine probabilistic computations from various modules, neural or otherwise. We previously argued for the use of probabilistic programming and related proposals to systematically capture such interactions. Consequently, probabilistic programming languages that allow neural components are in active development [173]. In [169, 174], for example, neural computations are positioned as one of the many possible types of computations in a semiring framework. This is very much in the spirit of semiring-based generalizations to WMC discussed earlier [170, 171]. Likewise, abstraction can be seen as a simple schema for unifying deep learning and traditional symbolic reasoning. For example, much of the work on neuro-symbolic systems attempt to integrate neural network outputs as abstract (and external) predicates in a logical framework [5, 67, 6, 8, 175].

Naturally, much work remains to be done on how the integration between neural and logical computations can be made deeper, in the sense of making the best possible trade-offs between learning and reasoning. But, from our discussions, it is clear that logical methods provide a powerful and rigorous regime to establish those tradeoffs and develop meta-theory for constructing hybrid models.

3.6. Conclusions

In this article, we surveyed work that provides further evidence for the connections between logic and learning. Our narrative was structured in terms of three strands: logic versus learning, machine learning for logic, and logic for machine learning, but naturally, there was considerable overlap.

We covered a large body of work on what these connections look like, including, for example, pragmatic concerns such as the use of hard, domain-specific constraints and background knowledge, all of which considerably eases the requirement that all of the agent's knowledge should be derived from observations alone. (See discussions in [176, 11] on the limitations of learned behavior, for example.) Where applicable, we placed an emphasis on how extensions to infinite domains are possible. Moreover, logical artifacts can help in constraining, simplifying and/or composing machine learning entities, and in providing a principled way to study the underlying representational and computational issues.

In general, this type of work could help us move beyond the narrow focus of the current learning literature so as to deal with time, space, abstraction, causality, quantified generalizations, relational abstractions, unknown domains, unforeseen examples, among other things, in a principled fashion. In fact, what is being advocated is the tackling of problems that symbolic logic and machine learning might struggle to address individually. One could even think of the need for a recursive combination of the strands: purely reactive components interact with purely cogitative elements, but then those reactive components are learned against domain constraints, and the cogitative elements are induced from data, and so on. More broadly, making progress towards a formal real-

ization of *System 1* versus *System 2* processing might also contribute to our understanding of human intelligence, and if not that, at least capture the human-like capability of balancing reaction versus deliberation in automated systems.

Acknowledgements. This chapter is a significantly extended version of a previous article by the author [177]. The author was supported by a Royal Society University Research Fellowship. The author is grateful to Paulius Dilkas, Kwabena Nuamah, Alan Bundy, and Luc De Raedt for comments.

References

- [1] Raedt LD, Kersting K, Natarajan S, et al. Statistical relational artificial intelligence: Logic, probability, and computation. *Synthesis Lectures on Artificial Intelligence and Machine Learning*. 2016;10(2):1–189.
- [2] Getoor L, Taskar B, editors. *An introduction to statistical relational learning*. MIT Press; 2007.
- [3] Lakemeyer G, Levesque HJ. Cognitive robotics. In: *Handbook of knowledge representation*. Elsevier; 2007. p. 869–886.
- [4] Kaelbling LP, Lozano-Pérez T. Integrated task and motion planning in belief space. *I J Robotic Res*. 2013;32(9-10):1194–1227.
- [5] Garcez Ad, Gori M, Lamb LC, et al. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *arXiv preprint arXiv:190506088*. 2019;.
- [6] De Raedt L, Manhaeve R, Dumancic S, et al. Neuro-symbolic= neural+ logical+ probabilistic. In: *NeSy'19@ IJCAI, the 14th International Workshop on Neural-Symbolic Learning and Reasoning*; 2019. p. 1–4.
- [7] Lamb L, Garcez A, Gori M, et al. Graph neural networks meet neural-symbolic computing: A survey and perspective. *arXiv preprint arXiv:200300330*. 2020;.
- [8] Sarker MK, Zhou L, Eberhart A, et al. Neuro-symbolic artificial intelligence current trends. *arXiv preprint arXiv:210505330*. 2021;.
- [9] Marcus G, Davis E. *Rebooting ai: Building artificial intelligence we can trust*. Pantheon; 2019.
- [10] Zellers R, Bisk Y, Schwartz R, et al. Swag: A large-scale adversarial dataset for grounded commonsense inference. *arXiv preprint arXiv:180805326*. 2018;.
- [11] Kambhampati S. Polanyi's revenge and ai's new romance with tacit knowledge. *Communications of the ACM*. 2021;64(2):31–32.
- [12] Bunel R, Hausknecht M, Devlin J, et al. Leveraging grammar and reinforcement learning for neural program synthesis. *arXiv preprint arXiv:180504276*. 2018;.
- [13] Xu K, Li J, Zhang M, et al. What can neural networks reason about? *arXiv preprint arXiv:190513211*. 2019;.
- [14] Evans R, Grefenstette E. Learning explanatory rules from noisy data. *Journal of Artificial Intelligence Research*. 2018;61:1–64.
- [15] Manhaeve R, Dumancic S, Kimmig A, et al. Deepprolog: Neural probabilistic logic programming. In: *Advances in Neural Information Processing Systems*; 2018. p. 3749–3759.
- [16] Kahneman D. *Thinking, fast and slow*. Macmillan; 2011.
- [17] Feeney A, Heit E. *Inductive reasoning: Experimental, developmental, and computational approaches*. Cambridge University Press; 2007.
- [18] Russell B. *History of western philosophy: Collectors edition*. Routledge; 2013.
- [19] Beaney M. *The oxford handbook of the history of analytic philosophy*. Oxford University Press; 2013.
- [20] Levesque HJ. *Thinking as computation: A first course*. MIT Press; 2012.
- [21] Hanna R. *Rationality and logic*. MIT Press; 2009.
- [22] van Benthem J, Ter Meulen A. *Handbook of logic and language*. Elsevier; 1996.
- [23] Gabbay DM, Woods JH. *Handbook of the history of logic*. Vol. 2009. Elsevier North-Holland; 2004.
- [24] Smullyan R. *First-order logic*. Dover Publications; 1995.
- [25] Hughes GE, Cresswell MJ. *A companion to modal logic*. Routledge; 1984.

- [26] Kowalski RA. Predicate logic as programming language. In: IFIP Congress; 1974. p. 569–574.
- [27] Biere A, Heule M, van Maaren H. Handbook of satisfiability. Vol. 185. IOS press; 2009.
- [28] Muggleton S. Inductive logic programming. New generation computing. 1991;8(4):295–318.
- [29] Lange M. Hume and the problem of induction. In: Handbook of the history of logic. Vol. 10. Elsevier; 2011. p. 43–91.
- [30] Carnap R, Jeffrey RC. Studies in inductive logic and probability. Vol. 2. Univ of California Press; 1980.
- [31] Hintikka J. Carnap and essler versus inductive generalization. Erkenntnis. 1975;9(2):235–244.
- [32] Carnap R. Logical foundations of probability. Routledge and Kegan Paul London; 1951.
- [33] Zabell SL. Carnap and the logic of inductive inference. In: Handbook of the history of logic. Vol. 10. Elsevier; 2011. p. 265–309.
- [34] Plotkin GD. A note on inductive generalization. Machine intelligence. 1970;5(1):153–163.
- [35] Michalski RS. A theory and methodology of inductive learning. In: Machine learning. Elsevier; 1983. p. 83–134.
- [36] Shapiro EY. Inductive inference of theories from facts. Yale University, Department of Computer Science; 1981.
- [37] De Raedt L, Frasconi P, Kersting K, et al., editors. Probabilistic inductive logic programming: theory and applications. Berlin, Heidelberg: Springer-Verlag; 2008.
- [38] Quinlan JR. Learning first-order definitions of functions. Journal of artificial intelligence research. 1996;5:139–161.
- [39] Peirce CS. Collected papers of charles sanders peirce. Vol. 7. Harvard University Press; 1974.
- [40] Carson D. The abduction of sherlock holmes. International Journal of Police Science & Management. 2009;11(2):193–202.
- [41] Tsamoura E, Michael L. Neural-symbolic integration: A compositional perspective. arXiv preprint arXiv:201011926. 2020;.
- [42] Dai WZ, Muggleton SH. Abductive knowledge induction from raw data. arXiv preprint arXiv:201003514. 2020;.
- [43] Poole D. Abducing through negation as failure: Stable models within the independent choice logic. The Journal of Logic Programming. 2000;44(1-3):5–35.
- [44] Poole D. Probabilistic horn abduction and bayesian networks. Artificial Intelligence. 1993;64(1):81–129.
- [45] Gulwani S. Dimensions in program synthesis. In: Proceedings of the 12th international ACM SIGPLAN symposium on Principles and practice of declarative programming; 2010. p. 13–24.
- [46] Genesereth MR, Nilsson NJ. Logical foundations of artificial intelligence. Morgan Kaufmann; 2012.
- [47] Bradley AR, Manna Z. The calculus of computation: decision procedures with applications to verification. Springer Science & Business Media; 2007.
- [48] Kersting K, Natarajan S, Poole D. Statistical relational AI: Logic, probability and computation. 2011;.
- [49] Cropper A, Dumancic S. Inductive logic programming at 30: a new introduction. Journal of Artificial Intelligence Research. 1993;1:1–15.
- [50] Benedikt M, Kersting K, Kolaitis PG, et al. Logic and learning (dagstuhl seminar 19361). Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik; 2020.
- [51] Brewka G, Delgrande JP, Romero J, et al. asprin: Customizing answer set preferences without a headache. In: AAAI; 2015. p. 1467–1474.
- [52] Levesque H, Reiter R, Lespérance Y, et al. Golog: A logic programming language for dynamic domains. Journal of Logic Programming. 1997;31:59–84.
- [53] McCarthy J. Programs with common sense. In: Semantic Information Processing. MIT Press; 1968. p. 403–418.
- [54] Nilsson NJ. Probabilistic logic. Artificial intelligence. 1986;28(1):71–87.
- [55] Halpern J. An analysis of first-order logics of probability. Artificial Intelligence. 1990;46(3):311–350.
- [56] Bacchus F. Representing and reasoning with probabilistic knowledge. MIT Press; 1990.
- [57] Raedt LD, Kimmig A, Toivonen H. Problog: A probabilistic prolog and its application in link discovery. In: Proc. IJCAI; 2007. p. 2462–2467.
- [58] McCarthy J, Hayes PJ. Some philosophical problems from the standpoint of artificial intelligence. In: Machine Intelligence; 1969. p. 463–502.
- [59] Kooi B. Probabilistic dynamic epistemic logic. Journal of Logic, Language and Information. 2003; 12(4):381–408.
- [60] Fagin R, Halpern JY. Reasoning about knowledge and probability. J ACM. 1994;41(2):340–367.

- [61] Belle V. Logic meets probability: Towards explainable AI systems for uncertain worlds. In: IJCAI; 2017.
- [62] Fagin R. Probabilities on finite models. *The Journal of Symbolic Logic*. 1976;41(1):50–58.
- [63] Mitchell DG, Selman B, Levesque HJ. Hard and easy distributions of sat problems. In: *Proc. AAAI*; 1992. p. 459–465.
- [64] Shafer G. Perspectives on the theory and practice of belief functions. *International Journal of Approximate Reasoning*. 1990;4(5–6):323 – 362.
- [65] Giles R. Łukasiewicz logic and fuzzy set theory. *International Journal of Man-Machine Studies*. 1976; 8(3):313–327.
- [66] Bach SH, Broecheler M, Huang B, et al. Hinge-loss markov random fields and probabilistic soft logic. *arXiv preprint arXiv:150504406*. 2015;.
- [67] Serafini L, Garcez Ad. Logic tensor networks: Deep learning and logical reasoning from data and knowledge. *arXiv preprint arXiv:160604422*. 2016;.
- [68] Koller D, Friedman N. Probabilistic graphical models - principles and techniques. MIT Press; 2009.
- [69] Niu F, Ré C, Doan A, et al. Tuffy: Scaling up statistical inference in markov logic networks using an rdbms. *Proceedings of the VLDB Endowment*. 2011;4(6):373–384.
- [70] Richardson M, Domingos P. Markov logic networks. *Machine learning*. 2006;62(1):107–136.
- [71] Poole D. First-order probabilistic inference. In: *Proc. IJCAI*; 2003. p. 985–991.
- [72] Suciu D, Olteanu D, Ré C, et al. Probabilistic databases. *Synthesis Lectures on Data Management*. 2011;3(2):1–180.
- [73] Gomes CP, Sabharwal A, Selman B. Model counting. In: *Handbook of satisfiability*. IOS Press; 2009.
- [74] Chavira M, Darwiche A. On probabilistic inference by weighted model counting. *Artificial Intelligence*. 2008;172(6-7):772–799.
- [75] Renkens J, Shterionov D, Van den Broeck G, et al. ProbLog2: From probabilistic programming to statistical relational learning. In: Roy D, Mansinghka V, Goodman N, editors. *Proceedings of the NIPS Probabilistic Programming Workshop*; Dec.; 2012.
- [76] Dilkas P, Belle V. Weighted model counting without parameter variables. In: *SAT*; 2021.
- [77] Dilkas P, Belle V. Weighted model counting with conditional weights for Bayesian networks. In: *UAI*; 2021.
- [78] Dilkas P, Belle V. Generating random logic programs using constraint programming. In: *CP*; 2020.
- [79] Chavira M, Darwiche A. Compiling Bayesian networks with local structure. In: *IJCAI*; Vol. 19; 2005. p. 1306.
- [80] Muise C, McIlraith S, Beck J, et al. D sharp: Fast d-DNNF compilation with sharpSAT. *Advances in Artificial Intelligence*. 2012;;356–361.
- [81] Sang T, Beame P, Kautz HA. Performing bayesian inference by weighted model counting. In: *AAAI*; 2005. p. 475–482.
- [82] Dudek JM, Phan VH, Vardi MY. DPMC: Weighted model counting by dynamic programming on project-join trees. In: *International Conference on Principles and Practice of Constraint Programming*; Springer; 2020. p. 211–230.
- [83] Wei W, Selman B. A new approach to model counting. In: *Theory and Applications of Satisfiability Testing*; Springer; 2005. p. 96–97.
- [84] Chakraborty S, Fremont DJ, Meel KS, et al. Distribution-aware sampling and weighted model counting for SAT. In: *AAAI*; 2014. p. 1722–1730.
- [85] Darwiche A, Marquis P. A knowledge compilation map. *J Artif Intell Res*. 2002;17:229–264.
- [86] Poon H, Domingos P. Sum-product networks: A new deep architecture. *UAI*. 2011;;337–346.
- [87] Van den Broeck G. *Lifted Inference and Learning in Statistical Relational Models [dissertation]*. KU Leuven; 2013.
- [88] Haaren JV, den Broeck GV, Meert W, et al. Lifted generative learning of markov logic networks. *Machine Learning*. 2016;103(1):27–55.
- [89] Barrett C, Sebastiani R, Seshia SA, et al. Satisfiability modulo theories. In: *Handbook of satisfiability*. Chapter 26. IOS Press; 2009. p. 825–885.
- [90] Shenoy P, West J. Inference in hybrid Bayesian networks using mixtures of polynomials. *International Journal of Approximate Reasoning*. 2011;52(5):641–657.
- [91] Sanner S, Abbasnejad E. Symbolic variable elimination for discrete and continuous graphical models. In: *AAAI*; 2012.
- [92] Belle V, Passerini A, Van den Broeck G. Probabilistic inference in hybrid domains by weighted model

- integration. In: IJCAI; 2015. p. 2770–2776.
- [93] Chistikov D, Dimitrova R, Majumdar R. Approximate counting in SMT and value estimation for probabilistic programs. In: Tacas. Vol. 9035; 2015. p. 320–334.
 - [94] Morettin P, Passerini A, Sebastiani R. Advanced smt techniques for weighted model integration. *Artificial Intelligence*. 2019;275:1–27.
 - [95] Merrell D, Albarghouthi A, D’Antoni L. Weighted model integration with orthogonal transformations. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*; 2017.
 - [96] Zeng Z, Van den Broeck G. Efficient search-based weighted model integration. *arXiv preprint arXiv:190305334*. 2019;.
 - [97] Kolb S, Mladenov M, Sanner S, et al. Efficient symbolic integration for probabilistic inference. In: IJCAI; 2018.
 - [98] Kolb S, Morettin P, Zuidberg Dos Martires P, et al. The pywmi framework and toolbox for probabilistic inference using weighted model integration. *IJCAI*. 2019;.
 - [99] Zuidberg Dos Martires P, Dries A, De Raedt L. Knowledge compilation with continuous random variables and its application in hybrid probabilistic logic programming. *arXiv preprint arXiv:180700614*. 2018;.
 - [100] Feldstein J, Belle V. Lifted reasoning meets weighted model integration. In: *UAI*; 2021.
 - [101] Russell SJ. Unifying logic and probability. *Commun ACM*. 2015;58(7):88–97.
 - [102] Gutmann B, Thon I, Kimmig A, et al. The magic of logical inference in probabilistic programming. *Theory and Practice of Logic Programming*. 2011;11(4-5):663–680.
 - [103] Milch B, Marthi B, Sontag D, et al. Approximate inference for infinite contingent bayesian networks. In: *AISTATS*; 2005. p. 238–245.
 - [104] Singla P, Domingos PM. Markov logic in infinite domains. In: *UAI*; 2007. p. 368–375.
 - [105] Grohe M, Lindner P. Probabilistic databases with an infinite open-world assumption. In: *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*; 2019. p. 17–31.
 - [106] Belle V. Open-universe weighted model counting. In: *AAAI*; 2017. p. 3701–3708.
 - [107] Belle V. Weighted model counting with function symbols. In: *UAI*; 2017.
 - [108] Loos S, Irving G, Szegedy C, et al. Deep network guided proof search. *arXiv preprint arXiv:170106972*. 2017;.
 - [109] Prates M, Avelar PH, Lemos H, et al. Learning to solve np-complete problems: A graph neural network for decision tsp. In: *Proceedings of the AAAI Conference on Artificial Intelligence*; Vol. 33; 2019. p. 4731–4738.
 - [110] Nye M, Hewitt L, Tenenbaum J, et al. Learning to infer program sketches. In: *International Conference on Machine Learning*; PMLR; 2019. p. 4861–4870.
 - [111] Abboud R, Ceylan I, Lukasiewicz T. Learning to reason: Leveraging neural networks for approximate dnf counting. In: *Proceedings of the AAAI Conference on Artificial Intelligence*; Vol. 34; 2020. p. 3097–3104.
 - [112] Muggleton S, De Raedt L. Inductive logic programming: Theory and methods. *The Journal of Logic Programming*. 1994;19:629–679.
 - [113] De Raedt L, Dries A, Thon I, et al. Inducing probabilistic relational rules from probabilistic examples. In: *Twenty-Fourth International Joint Conference on Artificial Intelligence*; 2015.
 - [114] Liang Y, Bekker J, Van den Broeck G. Learning the structure of probabilistic sentential decision diagrams. In: *Proceedings of the 33rd Conference on Uncertainty in Artificial Intelligence (UAI)*; 2017.
 - [115] Valiant LG. Robust logics. *Artificial Intelligence*. 2000;117(2):231–253.
 - [116] Cohen WW. PAC-learning nondeterminate clauses. In: *AAAI*; 1994. p. 676–681.
 - [117] Grohe M, Ritzert M. Learning first-order definable concepts over structures of small degree. In: *2017 32nd annual ACM/IEEE symposium on logic in computer science (LICS)*; IEEE; 2017. p. 1–12.
 - [118] De Raedt L, Džeroski S. First-order jk -clausal theories are PAC-learnable. *Artificial Intelligence*. 1994; 70(1):375–392.
 - [119] Ellis K, Solar-Lezama A, Tenenbaum J. Sampling for bayesian program learning. In: *NIPS*; 2016. p. 1289–1297.
 - [120] Kolb S. *Learn+ solve: Learning and solving constrained hybrid inference problems* [dissertation]. KU Leuven; 2019.
 - [121] Mocanu IG, Belle V, Juba B. Polynomial-time implicit learnability in smt. In: *ECAI*; 2020.
 - [122] Rader AP, Mocanu IG, Belle V, et al. Learning implicitly with noisy data in linear arithmetic. *IJCAI*.

- 2021;.
- [123] Molina A, Vergari A, Di Mauro N, et al. Mixed sum-product networks: A deep architecture for hybrid domains. In: Thirty-second AAAI conference on artificial intelligence; 2018.
 - [124] Bueff A, Speichert S, Belle V. Tractable querying and learning in hybrid domains via sum-product networks. KR Workshop on Hybrid Reasoning. 2018;.
 - [125] Nitti D, Ravkic I, Davis J, et al. Learning the structure of dynamic hybrid relational models. In: Proceedings of the Twenty-second European Conference on Artificial Intelligence; IOS Press; 2016. p. 1283–1290.
 - [126] Speichert S, Belle V. Learning probabilistic logic programs in continuous domains. In: ILP; 2019.
 - [127] Shapiro EY. An algorithm that infers theories from facts. In: IJCAI; Citeseer; 1981. p. 446–451.
 - [128] Gold EM. Language identification in the limit. *Information and control*. 1967;10(5):447–474.
 - [129] Belle V, Juba B. Implicitly learning to reason in first-order logic. In: Advances in Neural Information Processing Systems; 2019. p. 3376–3386.
 - [130] Campero A, Pareja A, Klinger T, et al. Logical rule induction and theory learning using neural theorem proving. arXiv preprint arXiv:180902193. 2018;.
 - [131] Marra G, Kuželka O. Neural markov logic networks. arXiv preprint arXiv:190513462. 2019;.
 - [132] Domingos P. The master algorithm: How the quest for the ultimate learning machine will remake our world. Basic Books; 2015.
 - [133] Niu F, Zhang C, Ré C, et al. Deepdive: Web-scale knowledge-base construction using statistical learning and inference. VLDS. 2012;12:25–28.
 - [134] Carlson A, Betteridge J, Kisiel B, et al. Toward an architecture for never-ending language learning. In: AAAI; 2010. p. 1306–1313.
 - [135] Cohen W, Yang F, Mazaitis KR. Tensorlog: A probabilistic database implemented using deep-learning infrastructure. *Journal of Artificial Intelligence Research*. 2020;67:285–325.
 - [136] Minervini P, Bošnjak M, Rocktäschel T, et al. Differentiable reasoning on large knowledge bases and natural language. In: Proceedings of the AAAI Conference on Artificial Intelligence; Vol. 34; 2020. p. 5182–5190.
 - [137] Liu Y, Levesque H. Tractable reasoning with incomplete first-order knowledge in dynamic systems with context-dependent actions. In: Proc. IJCAI; 2005. p. 522–527.
 - [138] Levesque HJ, Brachman RJ. Expressiveness and tractability in knowledge representation and reasoning. *Computational Intelligence*. 1987;3:78–93.
 - [139] Mitchell DG, Ternovska E. A framework for representing and solving NP search problems. In: AAAI; 2005. p. 430–435.
 - [140] Koller D, Levy A, Pfeffer A. P-classic: a tractable probabilistic description logic. In: Proc. AAAI / IAAI; 1997. p. 390–397.
 - [141] Bach FR, Jordan MI. Thin junction trees. In: Advances in Neural Information Processing Systems; 2002. p. 569–576.
 - [142] Darwiche A. A differential approach to inference in Bayesian networks. *Journal of the ACM (JACM)*. 2003;50(3):280–305.
 - [143] Darwiche A. Three modern roles for logic in ai. In: Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems; 2020. p. 229–243.
 - [144] Rudin C, Ustun B. Optimized scoring systems: Toward trust in machine learning for healthcare and criminal justice. *Interfaces*. 2018;48(5):449–466.
 - [145] Huang X, Kwiatkowska M, Wang S, et al. Safety verification of deep neural networks. In: International Conference on Computer Aided Verification; Springer; 2017. p. 3–29.
 - [146] Xu J, Zhang Z, Friedman T, et al. A semantic loss function for deep learning with symbolic knowledge. In: International Conference on Machine Learning; 2018. p. 5502–5511.
 - [147] Papantonis I, Belle V. Closed-form results for prior constraints in sum-product networks. *Frontiers in Artificial Intelligence*. 2021;.
 - [148] Farnadi G, Babaki B, Getoor L. Fairness in relational domains. In: Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society; 2018. p. 108–114.
 - [149] Dries A, Kimmig A, Davis J, et al. Solving probability problems in natural language. In: IJCAI; 2017.
 - [150] Goodman ND, Mansinghka VK, Roy DM, et al. Church: A language for generative models. In: Proceedings of UAI; 2008. p. 220–229.
 - [151] De Raedt L, Kimmig A. Probabilistic (logic) programming concepts. *Machine Learning*. 2015; 100(1):5–47.

- [152] Fierens D, Van den Broeck G, Thon I, et al. Inference in probabilistic logic programs using weighted CNF's. In: UAI; 2011. p. 211–220.
- [153] Holtzen S, Van den Broeck G, Millstein T. Dice: Compiling discrete probabilistic programs for scalable inference. arXiv preprint arXiv:200509089. 2020;.
- [154] Dos Martires PZ, Dries A, De Raedt L. Exact and approximate weighted model integration with probability density functions using knowledge compilation. In: Proceedings of the AAAI Conference on Artificial Intelligence; Vol. 33; 2019. p. 7825–7833.
- [155] Albarghouthi A, D'Antoni L, Drews S, et al. Quantifying program bias. CoRR. 2017;abs/1702.05437.
- [156] Belle V. Abstracting probabilistic models: Relations, constraints and beyond. Knowledge-Based Systems. 2020;:105976.
- [157] Banihashemi B, De Giacomo G, Lespérance Y. Abstraction in situation calculus action theories. In: AAAI; 2017. p. 1048–1055.
- [158] Holtzen S, Millstein T, Van den Broeck G. Probabilistic program abstractions. In: UAI; 2017.
- [159] Gunning D. Explainable artificial intelligence (xai). DARPA/I20; 2016.
- [160] Belle V, Papantonis I. Principles and practice of explainable machine learning. Frontiers in Big Data. 2021;.
- [161] Sreedharan S, Srivastava S, Kambhampati S. Hierarchical expertise level modeling for user specific contrastive explanations. In: IJCAI; 2018. p. 4829–4836.
- [162] Nitti D, Belle V, De Laet T, et al. Planning in hybrid relational mdps. Machine Learning. 2017; 106(12):1905–1932.
- [163] Bundy A, Nuamah K, Lucas C. Automated reasoning in the age of the internet. In: International Conference on Artificial Intelligence and Symbolic Computation; Springer; 2018. p. 3–18.
- [164] Belle V, Levesque HJ. Allegro: Belief-based programming in stochastic dynamical domains. In: IJCAI; 2015.
- [165] Halpern JY. Reasoning about uncertainty. MIT Press; 2003.
- [166] Ensan A, Ternovska E. Modular systems with preferences. In: IJCAI; 2015. p. 2940–2947.
- [167] Lierler Y, Truszczyński M. An abstract view on modularity in knowledge representation. In: AAAI; 2015. p. 1532–1538.
- [168] Bistarelli S, Montanari U, Rossi F. Semiring-based constraint logic programming: syntax and semantics. TOPLAS. 2001;23(1):1–29.
- [169] Eisner J, Filardo NW. Dyna: Extending Datalog for modern AI. In: Datalog reloaded. (LNCS; Vol. 6702). Springer; 2011. p. 181–220.
- [170] Kimmig A, Van den Broeck G, De Raedt L. Algebraic model counting. J Appl Log. 2017;22:46–62.
- [171] Belle V, De Raedt L. Semiring programming: A declarative framework for generalized sum product problems. AAAI Workshop: Statistical Relational Artificial Intelligence. 2020;.
- [172] Kordjamshidi P, Roth D, Kersting K. Systems ai: A declarative learning based programming perspective. In: IJCAI; 2018. p. 5464–5471.
- [173] Le TA, Baydin AG, Wood F. Inference compilation and universal probabilistic programming. In: Artificial Intelligence and Statistics; PMLR; 2017. p. 1338–1348.
- [174] Obermeyer F, Bingham E, Jankowiak M, et al. Functional tensors for probabilistic programming. arXiv preprint arXiv:191010775. 2019;.
- [175] Riegel R, Gray A, Luus F, et al. Logical neural networks. arXiv preprint arXiv:200613155. 2020;.
- [176] Levesque HJ. Common sense, the turing test, and the quest for real ai. MIT Press; 2017.
- [177] Belle V. Symbolic logic meets machine learning: A brief survey in infinite domains. In: International Conference on Scalable Uncertainty Management; Springer; 2020. p. 3–16.

Chapter 4

Graph Reasoning Networks and Applications

Qingxing Cao, Sun Yat-sen University

Wentao Wan, Sun Yat-sen University

Xiaodan Liang, Sun Yat-sen University

Liang Lin, Sun Yat-sen University

Deep neural networks have achieved significant success in various domains [1, 2, 3]. The dominant paradigm lies in the stack of more complicated layers and training with more data. However, such data-driven networks compromise the feature interpretability, lack the global reasoning capability, and are unable to incorporate external knowledge that is crucial for complicated real-world tasks. As emphasized in [4], reasoning over external knowledge is crucial for human decision-making. Structured knowledge provides rich cues to record human observations and commonsense using symbolic words (e.g. nouns or predicates). It is thus desirable to bridge symbolic semantics with learned local feature representations for better graph reasoning. Some works [5, 6, 7] thus formulated graphical models and structure constraints (e.g. CRF [8, 9]) as recurrent networks to effect on final convolution predictions. Other works utilized the knowledge bases. For example, Deng et al. [10] employed a label relation graph to guide network learning while Ordonez et al. [11] learned the mapping of common concepts to entry-level concepts. Young et al. [12] retrieved and encoded the commonsense knowledge based on the concepts covered in the dialogue. However, they cannot explicitly enhance feature representations, leading to the limited generalization capability.

To address the above problem, recent works [13] exploited graph neural network [14] to incorporate rich commonsense human knowledge into an intermediate feature representation. Generally speaking, these methods firstly construct a domain-specific graph that represents related human knowledge; then, characterize representations of different nodes with intermediate neural network features; next, perform graph reasoning for enhancing visual evidence of these symbolic nodes via graph neural network; Finally, mapping the evolved node feature back into the neural network for further forward propagation. Through integrating knowledge graphs into neural networks, one can collaborate feature learning and graph reasoning with the same supervised loss function and achieve a more effective and interpretable way to introduce structure constraints. In this chapter, we present details of three works that apply the graph neural network on three different tasks - image classification, segmentation, and automatic diagnosis.

4.1. Semantic Relationships Guided Representation Learning for Facial Action Unit Recognition

The facial action units (AUs) refer to the local facial muscle actions and play a paramount role in understanding facial expressions. Due to its potential applications in the field of human-robot interaction and computer vision, AU recognition has become a long-standing research topic in the field of artificial intelligence and computer vision [15, 16, 17]. In recent years, deep convolutional neural networks have been widely used in AU recognition due to their powerful feature representation and end-to-end efficient learning scheme. These works [17, 16, 18, 19] have improved the performance of this field significantly. However, recent efforts based on deep convolutional neural networks focus on designing more complex network structures without explicitly considering the local characteristics of the facial organs and the relationship between the facial muscles.

Since one facial expression is related to multiple action units, facial action units usually depend on each other. As shown in Figure 1, a happiness expression activates both AU6 and AU12, as we generally “raise cheek” and “pull lip corners” when we smile. On the other hand, due to structural limitations of facial anatomy, some action units are generally not likely to be activated simultaneously. For instance, we can not simultaneously stretch our mouths and raise our cheeks.

To fully exploit the structural relations between AUs, we incorporate the end-to-end differentiable graph neural network and propose the AU semantic relationship embedded representation learning (SRERL) framework. Specifically, we organize the facial action units in the form of a structured graph and propose to integrate a Gated Graph Neural Network (GGNN) [14] with a multi-scale convolutional network. The proposed framework can learn the AUs representation and their pair-wise relationships in an end-to-end manner. Thus the learned feature involves both the appearance and the relation characteristics, which makes the reasoning more robust towards challenging problems, e.g., illumination change and partial occlusion. The overall pipeline of our framework is illustrated in Figure 2.

4.1.1. Gated Graph Neural Network

Given a graph with N nodes and the initial representation of each node, GGNN [14] update each graph node through propagating feature among all nodes. Specifically, assume the node set is V and its corresponding adjacency matrix is A represents the structure of the graph. For each node $v \in V$, its hidden state representation at every time step t is h_v^t . When $t = 0$, the hidden state representation is the initial node feature x_v . The basic recurrent process of GGNN is formulated as

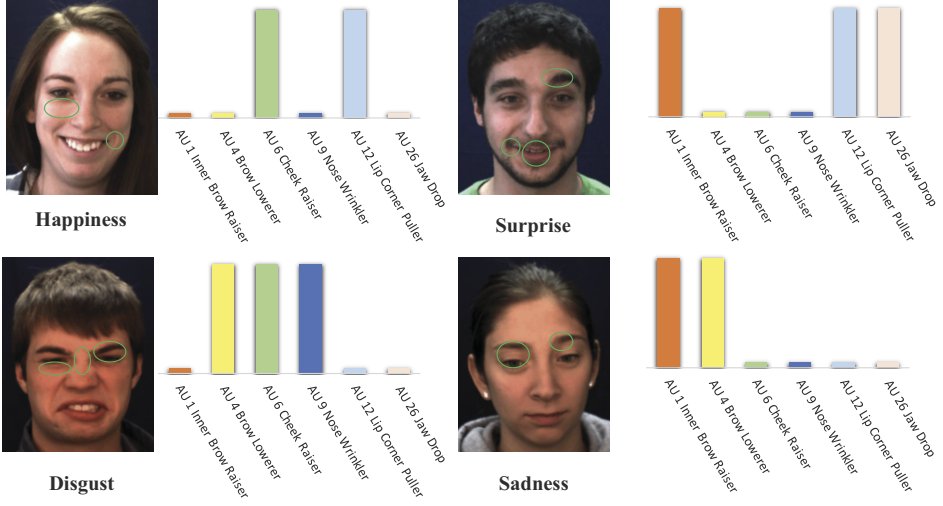


Figure 1. Coupling effect of multiple AUs caused by a variety of facial expressions.

$$h_v^{(1)} = [x_v^T, 0] \quad (1)$$

$$a_v^{(t)} = A_v^T [h_1^{(t-1)} \dots h_{|V|}^{(t-1)}]^T + b \quad (2)$$

$$z_v^t = \sigma(W^z a_v^{(t)} + U^z h_v^{(t-1)}) \quad (3)$$

$$r_v^t = \sigma(W^r a_v^{(t)} + U^r h_v^{(t-1)}) \quad (4)$$

$$\tilde{h}_v^t = \tanh(W a_v^{(t)} + U(r_v^t \odot h_v^{(t-1)})) \quad (5)$$

$$h_v^{(t)} = (1 - z_v^t) \odot h_v^{(t-1)} + z_v^t \odot \tilde{h}_v^t \quad (6)$$

where A_v is a sub-matrix of A and refers to the adjacency matrix related to the node v . W and U are learnable parameters. *sigma* and *tanh* are the logistic sigmoid and hyperbolic tangent functions. $\odot$ is the element-wise multiplication operation. The essence of the iterative update of GGNN is to compute the next hidden state of each node given current hidden state of their adjacent nodes, as shown in Eq (3-6). After T time steps, we get the final hidden states, and the node-level output can be computed as follow:

$$o^v = g(h_v^{(T)}, x_v) \quad (7)$$

where x_v is the annotation of node v , function g is a fully connected network.

4.1.2. AU Relationship Graph Construction

AU relationship graph represents the globally structured relationships between AUs. It is composed of a node-set V and an edge set E . Each node $v \in V$ represents a specific

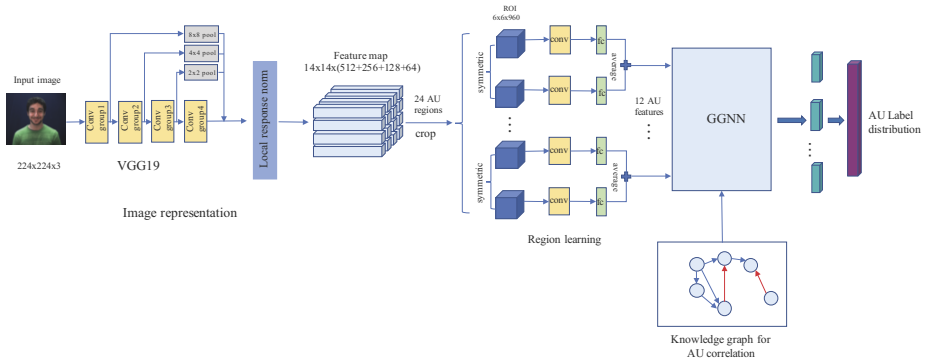


Figure 2. An overall pipeline of our proposed AU semantic relationship embedded representation learning (SRERL) framework. It is composed of a multi-scale feature learning and cropping module followed by a GGNN for node information propagation.

AU and the edges $e \in E$ represent the correlations between AUs. We construct the graph based on the training set statistic and priori connections gathered from common facial expressions and facial anatomy analysis. We detail the construction of V and E as follow:
Node Set V : Given a dataset that contains C AUs, the constructed graph is formed by a node-set V containing C entities.

Edge Set E : According to FACS [20], there exists strong relationship between AUs due to the linkage of each facial expression to multiple action units and the anatomical characteristics of faces. The relationship can further be divided into positive and negative ones, where positive correlation means that two AUs are likely to co-occur while negative correlation refers to that the two rarely appear together. We capture most of the AU relationship by calculating the conditional probability on the training samples, which is formulated as

$$a_pos_{i,j} = [P(y_i = 1 | y_j = 1) - P(y_i = 1) > p_{pos}], \quad (8)$$

$$a_neg_{i,j} = [P(y_i = 1 | y_j = 1) - P(y_i = 1) < p_{neg}], \quad (9)$$

where y_n denote the n -th AU label. p_{pos} and p_{neg} is the threshold for positive and negative relationship, respectively. $a_pos_{i,j} \in A_{pos}$ and $a_neg_{i,j} \in A_{neg}$. Thus, we can generate the adjacency matrix of AU relationship graph with the positive relationship matrix A_{pos} and negative relationship matrix A_{neg} , i.e., $A = [A_{pos} - I(A), -A_{neg}, (A_{pos} - I(A))^T, -A_{neg}^T]$. $I(A)$ denotes the identity matrix of the same dimension as A .

However, due to the limitation of the dataset, a few common relationships are not considered. Following the previous work [21], we add (AU4, AU7), (AU15, AU24) as positive correlations, and (AU2, AU6), (AU12, AU15), (AU12, AU17) as negative correlations for BP4D dataset [22]. These relationships are summarized in Figure 3, where green lines represent positive correlation and red lines denote negative ones. For clarity, we use a line with an arrow to represent a one-way relationship and a line without arrows to represent a two-way relationship.

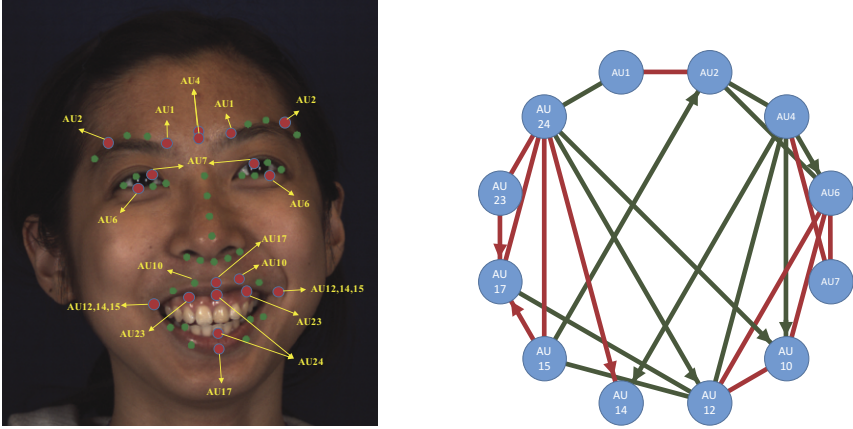


Figure 3. The location of facial AUs and the constructed AU relationship graph.

4.1.3. Relationship-Embedded Representation Learning

As shown in Figure 2, our proposed SRERL framework is composed of a multi-scale CNN for feature cropping and a GGNN for node information propagation. The GGNN uses the cropped facial region feature as input and propagates them on the defined knowledge graph to obtain the relationship-embed AU feature representation. We elaborate on these two components in the following.

Multiscale Feature Learning and Cropping We choose the VGG19 [23] model as our backbone network, which is composed of 5 groups of convolutional layers with down-sampling. For the sake of the trade-off between feature performance and its resolution, we use the first four convolutional layers for initial appearance feature extraction. The input to the feature extractor is a 224×224 RGB facial image with its landmarks information. Since AU regions have different sizes (for example, AU6 “Cheek Raiser” region is larger than AU12 “Lip Corner Puller”), we concatenate the four extracted features and resize them to 14×14 . The concatenated feature maps are further fed into a local response normalization layer which could normalize pixels between channels and help to converge. The formulation is:

$$b_c = \frac{a_c}{(k + \frac{\alpha \times C}{n} \sum_{c'=max(0, c-n/2)}^{min(C-1, c+n/2)} a_{c'}^2)^\beta}, \quad (10)$$

where k, α, β are hyperparameters, C represents the channel number of the feature maps. a_c denotes the pixel in c^{th} channel and n is set to 2 which refers to the number of neighboring channels chosen to normalize the pixel a_c . Then given the location of the facial landmarks, we crop a 6×6 region from the concatenated global feature maps as corresponding AU features. Figure 3 illustrates the correspondence between facial AUs and facial landmarks.

We further the AUs region feature into separate streams to obtain local AU feature representations. Suppose we have C AUs, we obtain $2C$ patch-wise feature maps (each AU corresponds to two patches) and design $2C$ independent regional feature learning branches. Each branch contains a 3×3 convolutional layer and a fully connected layer.

4.1.4. Balanced Loss Function

Data imbalance is a common problem in AU recognition. We design an adaptive loss function for imbalanced data training.

$$loss = -\frac{1}{C \cdot N} \sum_{j=1}^N \sum_{i=1}^C 2\{r_{\text{neg}}^i[l_j^i = 1] \cdot \log(\frac{p_j^i + 0.05}{1.05}) + r_{\text{pos}}^i[l_j^i = 0] \log(\frac{1.05 - p_j^i}{1.05})\} \quad (11)$$

$$r_{\text{pos}}^i = \frac{\sum_{k=1}^M [l_k^i = 1]}{M} \quad (12)$$

$$r_{\text{neg}}^i = 1 - r_{\text{pos}}^i \quad (13)$$

where l is the ground truth and p is the probability predicted by our method. $[x]$ is an indicator function which returns 1 when the statement x is true, and 0 otherwise. Let C denote the number of AUs, and N denote the batch size. r_{pos}^i and r_{neg}^i are constant, which indicate the proportion of positive samples or negative samples in the training set. M denotes the number of samples in the training set.

GGNN for action unit detection As shown in Figure 2, we generate the initial local feature of $2C$ regions using our multiscale feature learning and cropping module. Let f_i denotes the feature of a specific AU region, $i \in [0, 1, 2, \dots, 2C - 1]$. Considering the symmetry of a human face, each AU corresponds to two specific regions. The initial feature of each node in GGNN is thus given as

$$x_v = \frac{f_{2v} \oplus f_{2v+1}}{2} \quad (14)$$

where $\oplus$ denotes element-wise summation. According to Equation 1, the hidden state vector is initialized with the corresponding feature x_v . At each time-step t , the hidden state can be updated according to Eq 2–6. Finally, the output of each node in GGNN can be computed using Eq 7, which is further concatenated for final label prediction.

4.1.5. Experiments

Datasets We evaluated our model on two dataset: BP4D[22] and DISFA[24]. BP4D contains 2D and 3D facial expression data of 41 young adult subjects, including 18 males and 23 females. Each subject participated in 8 tasks, each of which corresponds to a specific expression. There are a total of 146,847 face images with labeled AUs. We follow [17] and split the dataset into three folds. We take turns to use two folds for training and the other for testing, and report the average results of multiple tests. DISFA contains stereo videos of 27 subjects with different ethnicity. There are a total of 130,815 frame images. Each frame is labeled with intensities from 0-5. We choose frames with AU intensities higher or equal to C-level as positive samples and the rest as negative samples. C is chosen as 2 in our experiment. As with BP4D, we also split the dataset into three folds for reliable testing. We use VGG19 trained on BP4D to extract the initial appearance feature, and finetune the rest parameters in DISFA.

Table 1. Comparison of quantitative results on the BP4D dataset. Our proposed SRERL achieves the best performance, which outperforms the second best method by 4.0% and 18.1% in terms of F1-score and AUC respectively.

AU	F1-score								AUC				
	LSVM	JPML	LCN	DRML	ROI	DSIN	Ours		LSVM	JPML	LCN	DRML	Ours
1	23.2	32.6	45.0	36.4	36.2	51.7	[46.9]		20.7	40.7	51.9	[55.7]	67.6
2	22.8	25.6	41.2	[41.8]	31.6	40.4	45.3		17.7	42.1	50.9	[54.5]	70.0
4	23.1	37.4	42.3	43.0	43.4	56.0	[55.6]		22.9	46.2	53.6	[58.8]	73.4
6	27.2	42.3	58.6	55.0	77.1	76.1	77.1		20.3	40.0	53.2	[56.6]	78.4
7	47.1	50.5	52.8	67.0	73.7	73.5	78.4		44.8	50.0	[63.7]	61.0	76.1
10	77.2	72.2	54.0	66.3	85.0	79.9	[83.5]		73.4	[75.2]	62.4	53.6	80.0
12	63.7	74.1	54.7	65.8	[87.0]	85.4	87.6		55.3	60.5	[61.6]	60.8	85.9
14	[64.3]	65.7	59.9	54.1	62.6	62.7	63.9		46.8	53.6	[58.8]	57.0	64.4
15	18.4	38.1	36.1	33.2	[45.7]	37.3	52.2		18.3	50.1	49.9	[56.2]	75.1
17	33.0	40.0	46.6	48.0	58.0	[62.9]	63.9		36.4	42.5	48.4	[50.0]	71.7
23	19.4	30.4	33.2	31.7	38.3	[38.8]	47.1		19.2	51.9	50.3	[53.9]	71.6
24	20.7	[42.3]	35.3	30.0	37.4	41.6	53.3		11.7	53.2	47.7	[53.9]	74.6
Avg	35.3	45.9	46.6	48.3	56.4	[58.9]	62.9		32.2	50.5	54.4	[56.0]	74.1

Table 2. Comparison of quantitative results on the DISFA dataset. Our proposed SRERL achieves the best performance, which outperforms the second best method by 2.3% and 28.4% in terms of F1-score and AUC respectively.

AU	F1-score						AUC			
	LSVM	LCN	DRML	ROI	DSIN	Ours	LSVM	LCN	DRML	Ours
1	10.8	12.8	17.3	41.5	[42.4]	45.7	21.6	44.1	[53.3]	76.2
2	10.0	12.0	17.7	26.4	[39.0]	47.8	15.8	52.4	[53.2]	80.9
4	21.8	29.7	37.4	[66.4]	68.4	59.6	17.2	47.7	[60.0]	79.1
6	15.7	23.1	29.0	50.7	28.6	[47.1]	8.7	39.7	[54.9]	80.4
9	11.5	12.4	10.7	8.5	46.8	[45.6]	15.0	40.2	[51.5]	76.5
12	70.4	26.4	37.7	89.3	70.8	[73.5]	93.8	54.7	54.6	[87.9]
25	12.0	46.2	38.5	[88.9]	90.4	84.3	3.4	[48.6]	45.6	90.9
26	22.1	30.0	20.1	15.6	[42.2]	43.6	20.1	[47.0]	45.3	73.4
Avg	21.8	24.0	26.7	48.5	[53.6]	55.9	27.5	46.8	[52.3]	80.7

Evaluation Criteria We evaluate our method with two metrics, including F1 score and AUC. F1 score considers both the precision p and the recall r , and is widely used in binary classification. F1 score can be computed as $F1 = 2 \frac{p \cdot r}{p + r}$, and AUC refers to the area under the ROC curve.

Comparison with the State of the Art We compare our method to alternative methods, including linear SVM(LSVM) [25], Joint Patch and Multi-label Learning(JPML) [26], ConvNet with locally connected layer(LCN) [27], Deep Region and Multi-label Learning(DRML) [17], Region Adaptation, Multi-label Learning(ROI) [16] and Deep Structure Inference Network(DSIN) [28].

Table 1 and table 2 show the result of 12 AUs on BP4D and 8 AUs on DISFA. We can see that our model outperforms all of these state-of-the-art methods. Compared to LSVM, our method achieved 27.6% and 41.9% higher F1-score and AUC on the BP4D dataset and achieved 34.1% and 53.2% higher F1-score and AUC on the DISFA dataset. The reason for the higher performance is that LSVM only uses the handcrafted feature and independent label inference. Our method outperforms JPML by 17% in terms of F1-score and 23.6% in AUC on BP4D on average. The main reason lies in that JPML use the predefined feature and is not end-to-end trainable. LCN, DRML, ROI, and DSIN are four state-of-the-art methods that use convolutional neural networks for end-to-end

Table 3. The effectiveness of relationship-embedded feature enhancement on DISFA dataset.

AU	F1-score		AUC	
	MS_RL	SRERL	MS_RL	SRERL
1	39.7	45.7	76.4	76.2
2	44.5	47.8	80.8	80.9
4	52.4	59.6	74.9	79.1
6	44.7	47.1	77.8	80.4
9	44.7	45.6	76.8	76.5
12	69.0	73.5	87.7	87.9
25	86.3	84.3	91.9	90.9
26	42.7	43.6	75.3	73.4
Avg	53.0	55.9	80.2	80.7

feature learning and multi-label classification. Compared to LCN, our method reaches higher F1-frame and AUC in all AUs on BP4D and DISFA datasets. DRML proposed to divide the feature map into 8*8 patches and use region learning for each patch. ROI used facial landmarks to crop the feature and maps to 20 patches, which also applied region learning for each patch. Compared with DRML and ROI, our model achieves 14.6% and 6.5% higher performance in F1-score on BP4D while achieves 29.2% and 7.4% higher w.r.t F1-score on DISFA. More impressively, our proposed SRERL greatly outperforms DSIN by 4% and 2.3% in terms of F1-score on BP4D and DISFA respectively. Noted that DSIN also simultaneously modeled the deep feature learning and the structured AU relationship in a unified framework. However, the relationship inference part of DSIN works as a post-processing step at the label level and is being isolated with the feature representation. Compared with DSIN, our performance gain well proves the significance of the joint optimization of feature expression and semantic relationship modeling for face action unit classification, and on the other hand, evidences the effectiveness of graph neural network in exploiting more thorough relationship in facial AU representation.

Effectiveness of Knowledge-Graph based Feature Embedding To verify the effectiveness of AU relationship-embedded feature enhancement, we have compared the performance of our proposed SRERL to those without relationship modeling (MS_RL). As shown in Table 3. The accuracy improvement is significant in the DISFA dataset, which is 2.90% and 0.50% in terms of F1-score and AUC respectively. Also, it can be seen from the analysis of individual AU performance that relationship modeling plays a crucial role in improving the accuracy of recognition. For instance, there exists a strong positive relationship between AU1 and AU2. In the DISFA dataset, AU1 and AU2 increased by 6.0% and 3.3% in F1-score respectively when compared to MS_RL. The experimental results demonstrate that the relationship-embedded representation learning can greatly enhance the regional facial feature by fully exploiting the structural collaboration between AUs.

4.2. Bidirectional Graph Reasoning Network for Panoptic Segmentation

The panoptic segmentation task is to assign each pixel in an image a semantic label and an instance id. Current methods typically address this issue with a unified model using two branches for foreground *things* and background *stuff* separately [29, 30, 31,

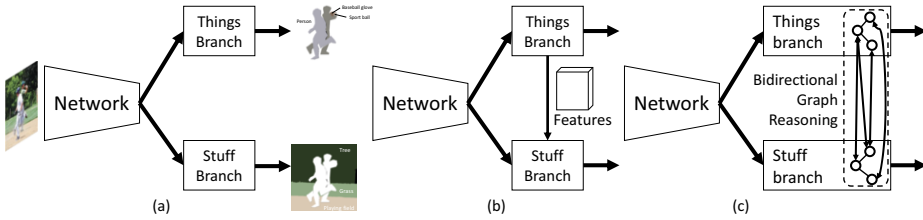


Figure 4. Different architectures for panoptic segmentation. (a) Simple multi-branch structure [30, 34] where two branches have no connection. (b) Unidirectional feature connection structure [32] that propagates information from *things* branch to *stuff* branch. (c) Our Bidirectional Graph Reasoning Network that enables mutual interaction and promotion for *things* and *stuff* based on graph convolution.

32]. As shown in Figure 4(a, b), for an input image, the final panoptic segmentation result was generated by combining results from two branches using fusion strategy [33]. However, these approaches overlooked the explicit semantic and co-occurrence relations between objects and background in a complicated environment, which leads to limited performance gain.

It is obvious and remarkable that *things* and *stuff* can benefit each other by information propagation in one unified network to boost the overall performance of panoptic segmentation. Thus this section introduces the Bidirectional Graph Reasoning Network (named BGRNet) that incorporates graph structure into the conventional panoptic segmentation network to encode the semantic and co-occurrence relations as well as diffuse information between *things* and *stuff*, as shown in Figure 4(c).

Specifically, we first build image-specific graphs in two separate network branches to enable flexible reasoning at the proposal level and class level, as shown in Figure 6. In the instance segmentation branch, a region graph is established to capture the pair-wise relationships among proposals. In the semantic segmentation branch, we build a graph based on the extracted class center that allows efficient global reasoning in a coarse-to-fine paradigm. Then, the proposed Bidirectional Graph Connection Module deduces the implicit semantic relations between *things* and *stuff* in a learnable fashion. After diffusing information across various nodes, intra-modular reasoning is performed to refine the visual features of two branches. In this way, we explicitly model the correlations between *things* and *stuff* class and leverage their complementary relations in a global view, which facilitates panoptic segmentation and has substantial practical merit in our experiments. An overview of our Bidirectional Graph Reasoning Network is shown in Figure 5.

4.2.1. Graph Representation

Formally, we define a graph as $G = (V, A, X)$ where V is the set of nodes, A denotes the adjacency matrix and X is the feature matrix where each row corresponds to a node in V .

Building Thing-Graph. In the classic object detection paradigm, extracted regions are analyzed separately without considering the underlying dependencies between objects, which leads to inconsistent detection results and limited performance in more challenging tasks like panoptic segmentation. To remedy this issue, we introduce a Thing-Graph to reason directly beyond local regions, which can refine visual features of certain regions that suffer from occlusions, class ambiguities, and tiny-size objects. Specifically, we build a Thing-Graph $G_{th} = (V_{th}, A_{th}, X_{th})$ on each input image, where $|V_{th}|$ equals to

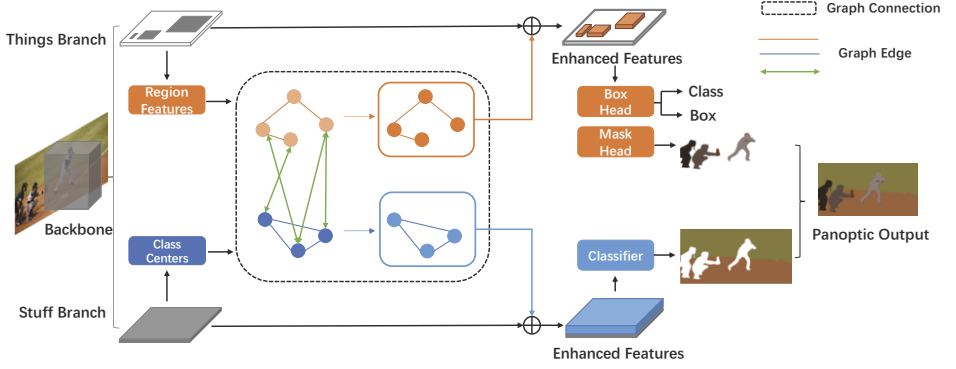


Figure 5. An overview of our BGRNet that can be stacked on any existing two branches panoptic segmentation network. The image features extracted by deep convolutional networks are fed into *things* branch and *stuff* branch. We construct Thing-Graph based on the region features after pooling. And we obtain Stuff-Graph node representations by extracting class centers from local feature. Then Bidirectional Graph Connection Module is used to propagate the high-level semantic graph representations within separate branches and across branches. Finally, we re-project the graph features to enhance the discriminability of visual features and improve the performance of both *things* and *stuff* branch.

the number of detected regions in the image, $X_{th} \in \mathbb{R}^{|V_{th}| \times N}$ are extracted features from backbone of all regions and N is the dimension of the region feature. Considering the diverse relations among regions, we render the edges in G_{th} learnable to allow flexible reasoning among multiple proposals.

Building Stuff-Graph. As for semantic segmentation, a naive idea of building a Stuff-Graph can be considering each pixel as a graph node similar to the non-local network [35]. However, this approach exhibits clear limitations in dense predictions of semantic segmentation since it requires a large amount of computation and vast GPU memory occupation. Thus, to reduce the computation overhead as well as capture the long-range dependencies, we project the entire feature map to the vertices of Stuff-Graph so that every vertex represents a specific *stuff* class. Regarding Stuff-Graph $G_{st} = (V_{st}, A_{st}, X_{st})$, given the coarse score map $S_{coarse} \in \mathbb{R}^{|V_{st}| \times H \times W}$ produced by the original segmentation head in the baseline network, and segmentation feature map $F \in \mathbb{R}^{N \times H \times W}$, where N is the number of feature channels, we first reshape S_{coarse} to $\mathbb{R}^{HW \times |V_{st}|}$ and F to $\mathbb{R}^{N \times HW}$. After performing softmax along the HW channel on score map, we can obtain class nodes feature $X_{st} \in \mathbb{R}^{|V_{st}| \times N}$ by matrix multiplication and transposition:

$$X_{st} = (\bar{F} \bar{S}_{coarse})^T, \quad (15)$$

where $\bar{F}$ and $\bar{S}_{coarse}$ represent F and S_{coarse} after reshaping. The intuition behind Equation 15 is that local features, i.e., the features of pixels, are gathered to obtain class nodes feature based on pixel affinity via soft-mapping. By assigning global class nodes features to X_{st} , we significantly reduce computation overhead in building a Stuff-Graph since $HW \gg |V_{st}|$. Besides, the extracted *stuff* nodes are more representative and can provide global clues to further benefit the final classification process after remapping them to local features. The processes of building Thing-Graph and Stuff-Graph are visualized in Figure 6.

4.2.2. Bidirectional Graph Connection Module

Given the Thing-Graph and Stuff-Graph, we aim to model the mutual relations between *things* and *stuff* and propagate the features across all nodes in both G_{th} and G_{st} . The rationale behind the design of graph nodes features fusion module across branches is quite straightforward and comprehensible since there exists a consistent pattern of the co-occurrence of foreground *things* and background *stuff* in real-world scenarios. For example, when there exist objects like *persons*, *sports balls*, *baseball bats* and *baseball gloves* in an image, it is more reasonable to predict the stuff of *sand* and *playing field*, and vice versa. Therefore, we distill this insight into Graph Connection Module to bridge all semantic information across branches (between foreground *things* and background *stuff*). In this way, the information, relations, or visual correlations of different categories from separate branches can be exploited.

The Graph Connection from Thing-Graph to Stuff-Graph can be formulated as:

$$X_{t-s} = A_{t-s} X_{th} W_{st}, \quad (16)$$

where $A_{t-s} \in \mathbb{R}^{|V_{st}| \times |V_{th}|}$ is a transfer matrix for propagating the information from Thing-Graph to Stuff-Graph, $W_{st} \in \mathbb{R}^{N \times D_0}$ is a trainable projection matrix. X_{t-s} is the mapped node features from Thing-Graph to Stuff-Graph. Similarly, the Graph Connection from Stuff-Graph to Thing-Graph can be obtained utilizing X_{st} and transfer matrix A_{s-t} with a trainable matrix W_{th} . Therefore, we seek for appropriate transfer matrix $A_{t-s} = \{a_{ij}^{t-s}\}$ and $A_{s-t} = \{a_{ij}^{s-t}\} \in \mathbb{R}^{|V_{th}| \times |V_{st}|}$, where a_{ij}^{s-t} denotes the connection weight from the j^{th} node of Stuff-Graph to the i^{th} node of Thing-Graph.

Based on the graph representation and Graph Connection, our graph structure can be naturally decomposed into blocks, given by

$$\hat{\mathbf{A}} = \begin{bmatrix} A_{th} & A_{s-t} \\ A_{t-s} & A_{st} \end{bmatrix}, \hat{\mathbf{X}} = \begin{bmatrix} X_{th} \\ X_{st} \end{bmatrix}, \quad (17)$$

where $A_{th}, A_{st}, A_{t-s}, A_{s-t}$ are normalized adjacency matrices for thing-to-thing pairs, stuff-to-stuff pairs, thing-to-stuff pairs, and stuff-to-thing pairs respectively. To model the distribution of different node features and adaptively handle their pairwise relations, we resort to attention mechanism [36] to obtain sufficient expressive power in our model. Formally, for any two nodes x_i, x_j in $\hat{\mathbf{X}}$, the edge weight α_{ij} is computed by:

$$\alpha_{ij} = \frac{\exp(\delta(W[x_i || x_j]))}{\sum_{k \in \mathcal{N}_i} \exp(\delta(W[x_i || x_k]))}, \quad (18)$$

where $||$ is the concatenation operation, $\mathcal{N}_i$ is the neighborhood of node i , δ is LeakyReLU nonlinear activation function, and W is weight matrix. For simplicity, we build a fully connected graph for $\hat{\mathbf{X}}$, i.e., $\mathcal{N}_i$ contains all nodes in $\hat{\mathbf{X}}$.

Updating node features. Formally, with normalized graph adjacency matrix $\hat{\mathbf{A}}$ and node features $\hat{\mathbf{X}}$, a single graph reasoning layer is given by

$$\tilde{\mathbf{X}} = \begin{bmatrix} \tilde{X}_{th} \\ \tilde{X}_{st} \end{bmatrix} = \hat{\mathbf{X}} \oplus \sigma(\hat{\mathbf{A}} \hat{\mathbf{X}} \otimes \hat{\mathbf{W}}), \quad (19)$$

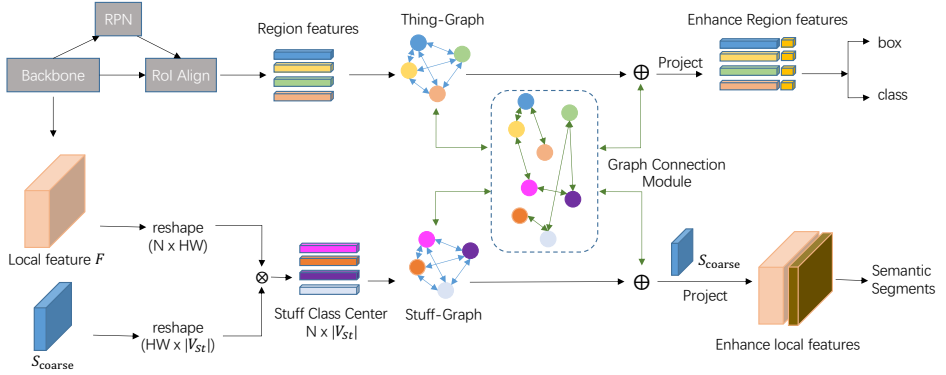


Figure 6. Diagram of our intra-modular graph, i.e., Thing-Graph and Stuff-Graph, and our inter-modular Graph Connection module. For Thing-Graph, we utilize the pooled region features as region graph nodes. For Stuff-Graph, we extract the class center from the local feature via the coarse score map. Then Graph Connection Module diffuses information across various graph nodes and intra-modular graph reasoning is performed to project graph nodes features to visual features at the proposal and pixel level, respectively, to refine the results of instance segmentation and semantic segmentation, which are then heuristically combined in an NMS-like procedure following [33].

where

$$\hat{\mathbf{W}} = \begin{bmatrix} W_{th} \\ W_{st} \end{bmatrix}, \hat{\mathbf{X}} \otimes \hat{\mathbf{W}} = \begin{bmatrix} X_{th} W_{th} \\ X_{st} W_{st} \end{bmatrix}, \quad (20)$$

$W_{th}, W_{st} \in \mathbb{R}^{D_0 \times D_0}$ are trainable weight matrices, $\tilde{X}_{th}, \tilde{X}_{st}$ are node features of new Thing-Graph and Stuff-Graph respectively, $\oplus$ denotes concatenation, and σ is ReLU nonlinear function. Using T Graph Reasoning layers, the model will propagate and update the information among classes to build more discriminating representations.

4.2.3. Project Nodes Features to Visual Features

To refine the results of instance and semantic segmentation, we project graph nodes features to visual features at the proposal and pixel level, respectively. We illustrate this process in Figure 6.

Intra-modular reasoning for detection. When enhancing the features of *things* branch, we only care about the features in proposals. Hence we concatenate the updated Thing-Graph features to each proposal after adjusting their dimension:

$$f_{th} = A_{th} \tilde{X}_{th} W_{th}^{intra}, \quad (21)$$

where $W_{th}^{intra} \in \mathbb{R}^{(N+D_0) \times D_1}$ is the weight matrix for intra-modular reasoning in *things* branch. Then we concatenate enhanced features f_{th} to the visual features of proposals and feed them into the final fully connected layer to obtain the detection results.

Intra-modular reasoning for segmentation. To facilitate the dense prediction in the *stuff* branch, we need to enhance the local feature of each pixel under the guidance of extracted class centers. This can be regarded as the inverse operation of Equation 15. We reshape S_{coarse} to $\mathbb{R}^{HW \times |V_{st}|}$, the enhanced feature of *stuff* branch can be calculated as:

Table 4. Performance comparisons with the state-of-the-art on the COCO val set. † indicates our implementation. Panoptic-FPN-D is the deformable counterpart of Panoptic-FPN [30]. All methods use ResNet50-FPN as the backbone network.

Method	DF Conv.	PQ	PQ Th	PQ St
Panoptic-FPN [30]		39.0	45.9	28.7
Panoptic-FPN-D [†]	✓	39.9	46.9	29.3
AUNet [32]		39.6	49.1	25.2
OANet [34]		39.0	48.3	26.6
UPNet-C [39]	✓	41.5	47.5	32.6
UPNet-CP [39]	✓	41.5	47.3	32.8
UPNet [39]	✓	42.5	48.5	33.4
SpatialFlow [40]		40.9	46.8	31.9
Our BGRNet	✓	43.2	49.8	33.4

$$f_{st} = S_{coarse} \tilde{X}_{st} W_{st}^{intra}, \quad (22)$$

where $W_{st}^{intra} \in \mathbb{R}^{(N+D_0) \times D_2}$ is the weight matrix for intra-modular reasoning in *stuff* branch. Then f_{st} is concatenated with local feature F , which is then fed into the final convolution layer to obtain semantic segmentation results.

4.2.4. Experiments

Datasets and Evaluation Metrics. We evaluate our method on COCO [37] and ADE20K [38]. COCO is one of the most challenging datasets for panoptic segmentation consisting of 115k images for training, 5k images for validation, and 20k images for *test-dev* with 80 *things* and 53 *stuff* classes. ADE20K is a densely annotated dataset for panoptic segmentation containing 20k images for training, 2k images for validation and 3k images for test, with 100 *things* and 50 *stuff* classes. Following [33], we adopt *panoptic quality* (PQ), *semantic quality* (SQ), and *recognition quality* (RQ) for evaluation.

Comparisons with state-of-the-art Comparisons with recent state-of-the-art methods on COCO and ADE20K dataset are listed in Table 4, 5. Some previous methods achieve high performance with over 42.5% PQ, thanks to the specially designed panoptic head [34], multi-scale information [30, 34], and two sources of attention [32]. Unlike previous methods [39, 34, 32], our BGRNet does not rely on complicated feature fusion process, i.e., RoI-Upsample [32], spatial ranking module [34], mask pruning process [39]. Instead, we utilize powerful graph models to capture intra-modular and inter-modular dependencies across separate branches. Thus, we achieve consistent accuracy gain over existed methods and set the new state-of-the-art results in terms of PQ, PQTh, PQSt. The advanced results demonstrate the superiority of our BGRNet that incorporates the reciprocal information and deduces underlying relations between *things* and *stuff* appeared in the image.

The qualitative results on the ADE20K dataset are shown in Figure 7. As can be observed, our approach outputs more semantically meaningful and precise predictions than baseline methods despite the existence of complex object appearances and challenging background contents. For example, the baseline mistakes *field* for *grass* while our BGRNet predicts correctly thanks to the propagated information from the *things* in the image. More visual results on COCO and ADE20K can be found in Supplementary Materials.

Table 5. Performance comparisons on ADE20K val set. Panoptic-FPN-D is the deformable counterpart of Panoptic-FPN [30]. † indicates our implementation.

Methods	PQ	PQ Th	PQ St
Panoptic-FPN [†] [30]	29.3	32.5	22.9
Panoptic-FPN-D [†] [30]	30.1	33.1	24.0
Our BGRNet	31.8	34.1	27.3

Table 6. Ablation studies on ADE20K val set.

Methods	PQ	PQ Th	PQ St
Baseline	30.1	33.3	23.7
w Thing-Graph	30.6	33.7	24.9
w Stuff-Graph	30.7	33.0	26.2
w Thing-Graph/Stuff-Graph	31.1	33.5	26.5
Our BGRNet	31.8	34.1	27.3

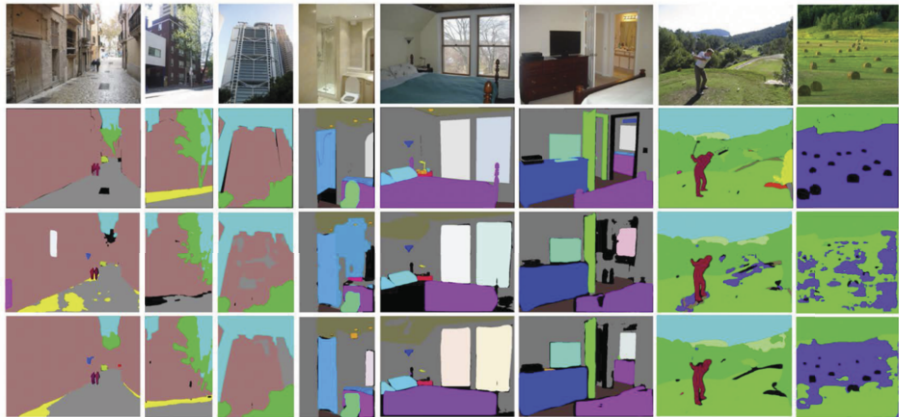


Figure 7. Visualized comparisons of panoptic segmentation outputs on ADE20K dataset. Raw images, Ground-Truth Segmentation, Panoptic-FPN outputs and BGRNet outputs are presented from top to bottom.

Combinations of intra-modular and inter-modular graphs. Table 6 shows the performance of different components of our BGRNet on ADE20K val set. “w Thing(Stuff)-Graph” only has a single graph for foreground or background branch, while “w Thing-Graph/Stuff-Graph” contains graphs in both two branches with no inter-branch interaction, and the graph nodes are re-projected to visual features similar to Section 4.2.3.

We first analyze the effect of a single graph in either *things* branch or *stuff* branch. For single Thing-Graph, both PQ^{Th} and PQ^{St} get improved thanks to the region-wise reasoning that considers the correlations among proposals. For single Stuff-Graph, PQ^{St} got a 2.5% relative improvement, which showcases the great effect of extracting class centers to refine local features in a coarse-to-fine paradigm. Incorporating these two graphs with no connection across branches, the overall PQ is already 1% higher than the baseline, which is a considerable improvement on the challenging ADE20K dataset. Furthermore, we introduce the graph connection module, which greatly improves the segmentation

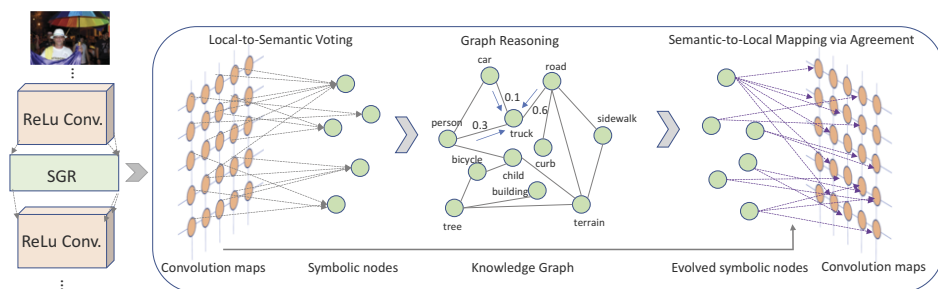


Figure 8. An overview of the proposed SGR layer. Each symbolic node receives votes from all local features via a local-to-semantic voting module (long gray arrows), and its evolved features after graph reasoning are then mapped back to each location via a semantic-to-local mapping module (long purple arrows). For simplicity, we omit more edges and symbolic nodes in the knowledge graph.

quality of *things* and *stuff*, due to the ability to mine the underlying relations between foreground and background. As can be seen from the last row in Table 6, our BGRNet improves PQ^{Th} and PQ^{St} by 0.8% and 3.6% respectively, resulting in 31.8% overall PQ, which outperforms Panoptic-FPN [30] by a large margin.

4.3. Symbolic Graph Reasoning Meets Convolutions

The commonsense human knowledge can be formed as various undirected graphs consisting of rich relationships (e.g. semantic hierarchy, spatial/action interactions and attributes, concurrence) among concepts. For example, “Shetland Sheepdog” and “Husky” share one superclass “dog” due to some common characteristics; people wear a hat and play guitar not vice-versa; orange is yellow color. After associating structured knowledge with the visual domain, all these symbolic entities (e.g. dog) can be connected with visual evidence from images, and humans can thus integrate the visual appearance and commonsense knowledge to help recognize.

In this section, we attempt to mimic this reasoning procedure and integrate it into convolution networks via the proposed Symbolic Graph Reasoning (SGR) layer. The core of the SGR layer consists of three modules, as illustrated in Figure 8. First, personalized visual evidence of each symbolic node can be produced by voting from all local representations, named as a local-to-semantic voting module. The voting weights stand for the semantic agreement confidence of each local feature to a certain node. Second, given a prior knowledge graph, the graph reasoning module is instantiated to propagate information over this graph for evolving visual features of all symbolic nodes. Finally, a dual semantic-to-local module learns appropriate associations between the evolved symbolic nodes and local features to join forces of local and global reasoning. It thus enables the evolved knowledge of a specific symbolic node to only drive the recognition of semantically compatible local features with the help of global reasoning.

4.3.1. General-purposed Graph Construction

The commonsense knowledge graph is used to depict distinct correlations between entities (e.g. classes, attributes and relationships) in general. To support the general purposed

graph reasoning, the knowledge graph can be formulated as $\mathcal{G} = (\mathcal{N}, \mathcal{E})$, where $\mathcal{N}$ and $\mathcal{E}$ denote the symbol set and edge set, respectively. Here we give three examples: a) class hierarchy graph is constructed by a list of entity classes (e.g. person, motorcyclist) and its graph edges shoulder the responsibility of concept belongings (e.g. “is kind of” or “is part of”). The networks equipped with such hierarchy knowledge can encourage the learning of feature hierarchy by passing the shared representations of parent classes into its child nodes; b) class occurrence graph defines the edges as the occurrence of two classes across images, characterizing the rationality of predictions; c) as a higher-level semantic abstraction, a semantic relationship graph can extend symbolic nodes to include more actions (e.g. “ride”, “play”), layouts (e.g. “on top of”) and attributes (e.g. color or shape) while graph edges are statistically collected from language descriptions. Incorporating such high-level commonsense knowledge can facilitate networks to prune spurious explanations after knowing the relationship of each entity pair, resulting in good semantic coherency.

Based on this general formula, the graph reasoning is required to be compatible and general enough for soft graph edges (e.g. occurrence probabilities) and hard edges (e.g. belongings), as well as diverse symbolic nodes. Various structure constraints can thus be modeled as edge connections over symbolic nodes, just like humans use language tools. Our SGR layer is designed to achieve the general graph reasoning that is applicable for encoding a wide range of knowledge graph forms. As illustrated in Figure 8, it consists of a local-to-semantic voting module, a graph reasoning module and a semantic-to-local mapping module, as presented in the following sections.

4.3.2. Local-to-Semantic Voting Module

Given local feature tensors from convolution layers, our target is to leverage global graph reasoning to enhance local features with external structured knowledge. We thus first summarize the global information encoded in local features into representations of symbolic nodes, that is, local features that are correlated to a specific semantic meaning (e.g. cat) are aggregated to depict the characteristics of its corresponding symbolic node. Formally, we use the feature tensor $X^l \in \mathbf{R}^{H^l \times W^l \times D^l}$ after l -th convolution layer as the module inputs, where H^l and W^l are height and weight of feature maps and D^l is the channel number. This module aims to produce visual representations $H^{ps} \in \mathbf{R}^{M \times D^c}$ of all $M = |\mathcal{N}|$ symbolic nodes using X^l , where D^c is the desired feature dimension for each node n , which is formulated as the function ϕ :

$$H^{ps} = \phi(A^{ps}, X^l, W^{ps}), \quad (23)$$

where $W^{ps} \in \mathbf{R}^{D^l \times D^c}$ is the trainable transformation matrix for converting each local feature $x_i \in X^l$ into the dimension D^c , and $A^{ps} \in \mathbf{R}^{H^l \times W^l \times M}$ denotes the voting weights of all local features to each symbolic node. Specifically, visual features $H_n^{ps} \in H^{ps}$ of each node n are computed by summing up all weighted transformed local features via the voting weight $a_{x_i \rightarrow n} \in A^{ps}$ that represents the confidence of assigning local feature x_i to the node n . More specifically, the function ϕ is computed as:

$$H_n^{ps} = \sum_{x_i} a_{x_i \rightarrow n} x_i W^{ps}, \quad a_{x_i \rightarrow n} = \frac{\exp(W_n^{aT} x_i)}{\sum_{n \in \mathcal{N}} \exp(W_n^{aT} x_i)}. \quad (24)$$

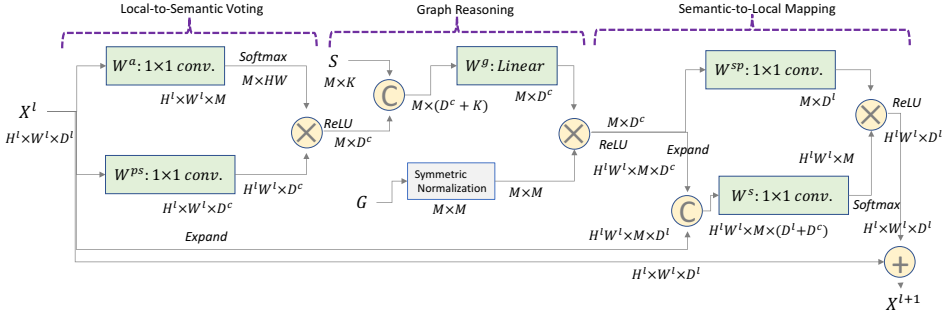


Figure 9. Implementation details of one SGR layer by taking the convolution feature tensors of $H^l \times W^l \times D^l$ as inputs. $\otimes$ denotes matrix multiplication, and $\oplus$ denotes element-wise summation and the circle with C denotes the concatenation. The softmax operation, tensor expansion, ReLU operation are performed when noted. The green boxes denote 1×1 convolution or linear layer.

Here $W^a = \{W_n^a\} \in \mathbf{R}^{D^l \times M}$ is a trainable weight matrix for calculating voting weights. A^{ps} is normalized by using a softmax at each location. In this way, different local features can adaptively vote to representations of distinct symbolic nodes.

4.3.3. Graph Reasoning Module

Based on visual evidence of symbolic nodes, the reasoning guided by structured knowledge is employed to leverage semantic constraints from human commonsense to evolve global representations of symbolic nodes. Here, we incorporate both linguistic embedding of each symbolic node and knowledge connections (i.e. node edges) for performing graph reasoning. Formally, for each symbolic node $n \in \mathcal{N}$, we use the off-the-shelf word vectors [41] as its linguistic embedding, denoted as $\mathcal{S} = \{s_n\}, s_n \in \mathbf{R}^K$. The graph reasoning module performs graph propagation over representations H^{ps} of all symbolic nodes via the matrix multiplication form, resulting in the evolved features H^g :

$$H^g = \sigma(A^g B W^g), \quad (25)$$

where $B = [\sigma(H^{ps}), \mathcal{S}] \in \mathbf{R}^{M \times (D^c + K)}$ concatenates features of transformed H^{ps} via the activation function $\sigma(\cdot)$ and the linguistic embedding $\mathcal{S}$. $W^g \in \mathbf{R}^{(D^c + K) \times (D^c)}$ is a trainable weight matrix. The node adjacency weight $a_{n \rightarrow n'} \in A^g$ is defined according to the edge connections in $(n, n') \in \mathcal{E}$. As discussed in Section 4.3.1, the edge connections can be soft weights (e.g. 0.8) or hard weight (i.e. $\{0, 1\}$) according to different knowledge graph resources. The naive multiplication with A^g will completely change the scale of the feature vectors. Inspired from graph convolutional networks [42], we can normalize A^g such that all rows sum to one to get rid of this problem, i.e. $Q^{-\frac{1}{2}} A^g Q^{-\frac{1}{2}}$, where Q is the diagonal node degree matrix of A^g . This symmetric normalization corresponds to taking the average of neighboring node features. This formulation arrives at the new propagation rule:

$$H^g = \sigma(\hat{Q}^{-\frac{1}{2}} \hat{A}^g \hat{Q}^{-\frac{1}{2}} B W^g), \quad (26)$$

where $\hat{A}^g = A^g + I$ is the adjacency matrix of the graph $\mathcal{G}$ with added self-connections for considering its own representation of each node and I is the identity matrix. $\hat{Q}_{ii} = \sum_j \hat{A}_{ij}^g$.

4.3.4. Semantic-to-Local Mapping Module

Finally, the evolved global representations $H^g \in \mathbf{R}^{M \times D^c}$ of symbolic nodes can be used to further boost the capability of each local feature representation. As the feature distributions of each symbolic node have been changed after graph reasoning, a critical question is how to find the most appropriate mappings from the representation $h^g \in H^g$ of each symbolic node to all x_i . This can be agnostic to learning the compatibility matrix between local features and symbolic nodes. Inspired by message-passing algorithms [43], we compute the mapping weights $a_{h^g \rightarrow x_i} \in A^{sp}$ by evaluating the compatibility of each symbolic node h^g with each local feature x_i :

$$a_{h^g \rightarrow x_i} = \frac{\exp(W^{sT}[h^g, x_i])}{\sum_{x_i} \exp(W^{sT}[h^g, x_i])}, \quad (27)$$

where $W^s \in \mathbf{R}^{D^l + D^c}$ is a trainable weight matrix. The compatibility matrix $A^{sp} \in \mathbf{R}^{H \times W \times M}$ is again row-normalized. The evolved features X^{l+1} by graph reasoning, posed as inputs in the $l + 1$ convolution layer can be updated as:

$$X^{l+1} = \sigma(A^{sp} H^g W^{sp}) + X^l, \quad (28)$$

where $W^{sp} \in \mathbf{R}^{D^c \times D^l}$ is the trainable matrix for transforming the dimension of symbolic node representation back into D^l , and we use residual connection [1] to further enhance local representations with the original local feature tensor X^l . Each local feature is updated by the weighted mappings from each symbolic node that represents different characteristics of semantics.

4.3.5. Symbolic Graph Reasoning Layer

Each symbolic graph reasoning layer is constituted by the stack of a local-to-semantic voting module, a graph reasoning module, and a semantic-to-local mapping module. The SGR layer is instantiated by a specific knowledge graph with different numbers of symbolic nodes and distinct node connections. Combining multiple SGR layers with distinct knowledge graphs into convolutional networks can lead to hybrid graph reasoning behaviors. We implement the modules of each SGR via the combination of 1×1 convolution operations and non-linear functions, detailed as Figure 9. Our SGR is flexible and general enough for injecting it between any local convolutions. Nonetheless, as SGR is designated to incorporate high-level semantic reasoning, using SGR in later convolution layers is more preferable, as demonstrated in our experiments.

4.3.6. Experiments

Dataset. We evaluate our method on two public benchmarks for segmenting over large-scale categories, which pose more realistic challenges than other small segmentation datasets (e.g. PASCAL-VOC) and can better validate the necessity of global symbolic reasoning. Specifically, **Coco-Stuff** [54] contains 10,000 images with dense annotations of 91 thing (e.g. book, clock) and 91 stuff classes (e.g. flower, wood), including 9,000 for training and 1,000 for testing. **PASCAL-Context** [55] includes 4,998 images for training

Table 7. Comparison on Coco-Stuff test set (%)

Method	Class acc.	acc.	mean IoU
FCN [44]	38.5	60.4	27.2
DeepLabv2 (ResNet-101) [2]	45.5	65.1	34.4
DAG RNN + CRF [45]	42.8	63.0	31.2
OHE + DC + FCN [46]	45.8	66.6	34.3
DSSPN (ResNet-101) [47]	47.0	68.5	36.2
SGR (w/o residual)	47.9	68.4	38.1
SGR (scene graph)	49.1	69.6	38.3
SGR (concurrence graph)	48.6	69.5	38.4
SGR (w/o mapping)	47.3	67.9	37.2
SGR (ConvBlock4)	47.6	68.3	37.5
Our SGR (ResNet-101)	49.3	69.9	38.7
Our SGR (ResNet-101 2-layer)	49.4	69.7	38.8
Our SGR (ResNet-101 Hybrid)	49.8	70.5	39.1

Table 8. Comparison on PASCAL-Context test set.

Method	mean IoU (%)
FCN [44]	37.8
CRF-RNN [5]	39.3
ParseNet [48]	40.4
BoxSup [49]	40.5
HO CRF [50]	41.3
Piecewise [51]	43.3
VeryDeep [52]	44.5
DeepLab-v2 (ResNet-101) [2]	45.7
RefineNet (Res152) [53]	47.3
Our SGR (ResNet-101)	50.8
Our SGR (Transfer convs)	51.3
Our SGR (Transfer SGR)	52.5

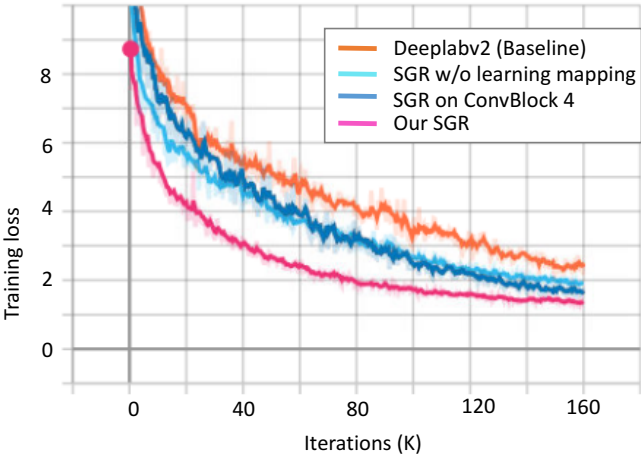


Figure 10. Curves of the training losses on Coco-Stuff for the Deeplabv2 (Baseline) [2] and our three variants. Following [2], the loss is the summations of losses for inputs of three scales (i.e. 1, 0.75, 0.5).

and 5105 for testing, annotated with 59 object categories and one background. We use standard evaluation metrics of pixel accuracy (pixAcc) and mean Intersection of Union (mIoU).

Comparison with the state-of-the-arts Table 7, 8 report the comparisons with recent state-of-the-art methods on Coco-Stuff and Pascal-Context dataset, respectively. Incorporating our SGR layer significantly outperforms existing methods, demonstrating its effectiveness of performing explicit graph reasoning beyond local convolutions for large-scale pixel-level recognition. Figure 11 shows the qualitative comparison with the baseline “Deeplabv2 [2]”. Our SGR obtains better segmentation performance, especially for some rare classes (e.g. umbrella, teddy bear), benefiting from the joint reasoning with frequent concepts over the concept hierarchy graph.

Which ConvBlock to add SGR layer? Table 7 and Table 8 compare the variants of adding a single SGR layer into different stages of ResNet-101. “SGR ConvBlock4” means the SGR layer is added to right before the last residual block of res4 while all

other variants add SGR layer before the last residual block of res5 (final residual block). The performance of “SGR ConvBlock4” is worse than “Our SGR (ResNet-101)” while using SGR layer for both res4 and res5 (“Our SGR (ResNet-101 2-layer)”) can slightly improve the results. Note that in order to use pretrained weights from ResNet-101, “Our SGR (ResNet-101 2-layer)” directly fuses the prediction results from two SGR layers after res4 and res5 via the summation to get the final prediction. One possible explanation for this observation is that the final res5 can encode more semantically abstracted features, which is more suitable for conducting symbolic graph reasoning. Furthermore, we find removing residual connection in Eqn. 28 would decrease the final performance but is still better than other baselines, by comparing “SGR (w/o residual)” with our full SGR. The reason is that the SGR layer induces more smoothing local features enhanced by global reasoning and thus may degrade some discriminative capability in boundaries.

The effect of semantic-to-local mapping. Note that our SGR learns distinct voting weights and mapping weights in the local-to-semantic modules and semantic-to-local module, respectively. The advantages of reevaluating mapping weights can be seen by comparing “Our SGR (ResNet-101)” with “SGR (w/o mapping)” in both testing performance and training convergence in Table 7 and Table 8. This justifies that estimating new semantic-to-local mapping weights can accommodate the reasoning process with the evolved feature distributions after graph reasoning, otherwise the evolved symbolic nodes will be misaligned with local features.

Different prior knowledge graphs. As discussed in Section 4.3.1, our SGR layer is general for any form of knowledge graphs with either soft or hard edge weights. We thus evaluate the results of leveraging distinct knowledge graphs in Table 7. First, class concurrence graph is often used to represent the frequency of any two concepts appearing in one image, which depicts inter-class rationality in a statistical view. We calculate the class concurrence graph from all training images on Coco-Stuff and feed it as the input of the SGR layer, as “SGR (concurrence graph)”. We can see that incorporating a concurrence-driven SGR layer can also boost the segmentation performance, but is slightly inferior to that with concept hierarchy. Second, we also sequentially stack one SGR layer with hierarchy graph and one layer with concurrence graph, leading to a hybrid version as “Our SGR (ResNet-101 Hybrid)”. This variant achieves the best performance among all models, verifying the benefits of boosting semantic reasoning capability with the mixture of knowledge constraints. Finally, we further explore a rich scene graph that includes concepts, attributes and relationships for encoding higher-level semantics, as the “SGR (scene graph)” variant. Following [56], the scene graph is constructed from the Visual Genome [57]. For simplicity, we only select the object categories, attributes, and predicates, which appear at least 30 times and are associated with our targeted 182 concepts in Coco-Stuff. It leads to an undirected graph with 312 object nodes, 160 attribute nodes, and 68 predicate nodes. “SGR (scene graph)” is slightly worse than “Our SGR (ResNet-101)” but better than “SGR (concurrence graph)”. Observed from all these studies, we thus use the concept hierarchy graph for all rest experiments by balancing the efficiency and effectiveness.

Transferring SGR learned from one domain to other domains. Our SGR layer naturally learns to encode explicit semantic meanings for general symbolic nodes after voting from local features, whose weights can be easily transferred from one domain into other domains only if these domains share one prior graph. Due to the usage of a single hierarchy graph for both Coco-Stuff and PASCAL-Context datasets, we can use the

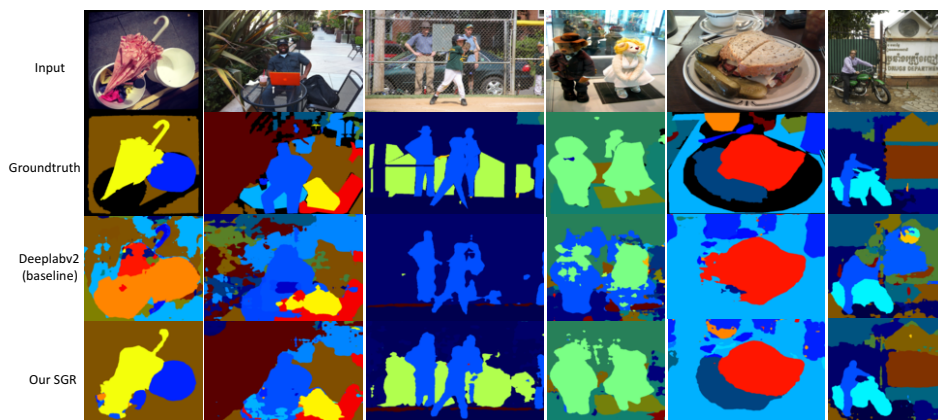


Figure 11. Qualitative comparison results on Coco-stuff dataset.

SGR model pretrained on Coco-Stuff to initialize the training on the PASCAL-Context dataset, as reported in Table 8. “Our SGR (Transfer convs)” denotes only the pretrained weights of residual blocks are used while “Our SGR (Transfer SGR)” is the variant of further using the parameters of the SGR layer. We can see that transferring parameters of the SGR layer can give more improvements than that of solely transferring convolution blocks.

4.4. Conclusion

In this chapter, we introduced three works that incorporated knowledge graphs into neural networks for end-to-end feature representation learning. The first exploited the knowledge of linkage relationships between the facial muscles and constructed the relationship graph between facial action units. The second incorporates graph structure into the conventional panoptic segmentation network to mine the intra-modular and inter-modular relations within and between foreground *things* and background *stuff* classes. The third proposed a Symbolic Graph Reasoning (SGR) layer to endow the local convolution networks with the capability of global graph reasoning. Through modeling the domain knowledge, graph reasoning has shown to have substantial practical merit for many tasks.

References

- [1] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. In: CVPR; 2016. p. 770–778.
- [2] Chen LC, Papandreou G, Kokkinos I, et al. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. arXiv preprint arXiv:160600915. 2016;.
- [3] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: Advances in neural information processing systems; 2017. p. 5998–6008.
- [4] Biederman I, Mezzanotte RJ, Rabinowitz JC. Scene perception: Detecting and judging objects undergoing relational violations. Cognitive psychology. 1982;14(2):143–177.
- [5] Zheng S, Jayasumana S, Romera-Paredes B, et al. Conditional random fields as recurrent neural networks. In: ICCV; 2015. p. 1529–1537.

- [6] Schwing AG, Urtasun R. Fully connected deep structured networks. arXiv preprint arXiv:150302351. 2015;.
- [7] Chandra S, Usunier N, Kokkinos I. Dense and low-rank gaussian crfs using deep embeddings. In: ICCV; 2017.
- [8] Lafferty J, McCallum A, Pereira FC. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. 2001;.
- [9] Krähenbühl P, Koltun V. Efficient inference in fully connected crfs with gaussian edge potentials. In: NIPS; 2011. p. 109–117.
- [10] Deng J, Ding N, Jia Y, et al. Large-scale object classification using label relation graphs. In: ECCV; 2014. p. 48–64.
- [11] Ordonez V, Deng J, Choi Y, et al. From large scale image categorization to entry-level categories. In: ICCV; 2013. p. 2768–2775.
- [12] Young T, Cambria E, Chaturvedi I, et al. Augmenting end-to-end dialog systems with commonsense knowledge. arXiv preprint arXiv:170905453. 2017;.
- [13] Zhou J, Cui G, Hu S, et al. Graph neural networks: A review of methods and applications. AI Open. 2020;1:57–81. Available from: <https://www.sciencedirect.com/science/article/pii/S2666651021000012>.
- [14] Li Y, Tarlow D, Brockschmidt M, et al. Gated graph sequence neural networks. arXiv preprint arXiv:151105493. 2015;.
- [15] Li W, Abtahi F, Zhu Z, et al. Eac-net: A region-based deep enhancing and cropping approach for facial action unit detection. arXiv preprint arXiv:170202925. 2017;.
- [16] Li W, Abtahi F, Zhu Z. Action unit detection with region adaptation, multi-labeling learning and optimal temporal fusing. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; IEEE; 2017. p. 6766–6775.
- [17] Zhao K, Chu WS, Zhang H. Deep region and multi-label learning for facial action unit detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2016. p. 3391–3399.
- [18] Bishay M, Patras I. Fusing multilabel deep networks for facial action unit detection. In: Proceedings of the IEEE International Conference on Automatic Face & Gesture Recognition; 2017. p. 681–688.
- [19] Chu WS, De la Torre F, Cohn JF. Modeling spatial and temporal cues for multi-label facial action unit detection. arXiv preprint arXiv:160800911. 2016;.
- [20] Friesen E, Ekman P. Facial action coding system: a technique for the measurement of facial movement. Palo Alto. 1978;.
- [21] Zhang Y, Dong W, Hu BG, et al. Classifier learning with prior probabilities for facial action unit recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2018. p. 5108–5116.
- [22] Zhang X, Yin L, Cohn JF, et al. A high-resolution spontaneous 3d dynamic facial expression database. In: IEEE International Conference and Workshops on Automatic Face and Gesture Recognition; 2013. p. 1–6.
- [23] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:14091556. 2014;.
- [24] Mavadati SM, Mahoor MH, Bartlett K, et al. Disfa: A spontaneous facial action intensity database. IEEE Transactions on Affective Computing. 2013;4(2):151–160.
- [25] Fan RE, Chang KW, Hsieh CJ, et al. Liblinear: A library for large linear classification. Journal of machine learning research. 2008;9(8):1871–1874.
- [26] Zhao K, Chu WS, De la Torre F, et al. Joint patch and multi-label learning for facial action unit detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2015. p. 2207–2216.
- [27] Taigman Y, Yang M, Ranzato M, et al. Deepface: Closing the gap to human-level performance in face verification. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2014. p. 1701–1708.
- [28] Corneanu CA, Madadi M, Escalera S. Deep structure inference network for facial action unit recognition. arXiv preprint arXiv:180305873. 2018;.
- [29] de Geus D, Meletis P, Dubbelman G. Panoptic segmentation with a joint semantic and instance segmentation network. arXiv preprint arXiv:180902110. 2018;.
- [30] Kirillov A, Girshick R, He K, et al. Panoptic feature pyramid networks. arXiv preprint arXiv:190102446. 2019;.

- [31] Li J, Raventos A, Bhargava A, et al. Learning to fuse things and stuff. arXiv preprint arXiv:181201192. 2018;.
- [32] Li Y, Chen X, Zhu Z, et al. Attention-guided unified network for panoptic segmentation. In: CVPR; June; 2019.
- [33] Kirillov A, He K, Girshick R, et al. Panoptic segmentation. arXiv preprint arXiv:180100868. 2018;.
- [34] Liu H, Peng C, Yu C, et al. An end-to-end network for panoptic segmentation. In: CVPR; June; 2019.
- [35] Wang X, Girshick R, Gupta A, et al. Non-local neural networks. ????.
- [36] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks. arXiv preprint arXiv:171010903. 2017;.
- [37] Lin TY, Maire M, Belongie S, et al. Microsoft coco: Common objects in context. In: ECCV; Springer; 2014. p. 740–755.
- [38] Zhou B, Zhao H, Puig X, et al. Scene parsing through ade20k dataset. In: CVPR; 2017.
- [39] Xiong Y, Liao R, Zhao H, et al. Upsnet: A unified panoptic segmentation network. arXiv preprint arXiv:190103784. 2019;.
- [40] Chen Q, Cheng A, He X, et al. Spatialflow: Bridging all tasks for panoptic segmentation. arXiv preprint arXiv:191008787. 2019;.
- [41] Joulin A, Grave E, Bojanowski P, et al. Fasttext. zip: Compressing text classification models. arXiv preprint arXiv:161203651. 2016;.
- [42] Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. ICLR. 2017;.
- [43] Gilmer J, Schoenholz SS, Riley PF, et al. Neural message passing for quantum chemistry. arXiv preprint arXiv:170401212. 2017;.
- [44] Long J, Shelhamer E, Darrell T. Fully convolutional networks for semantic segmentation. In: CVPR; 2015. p. 3431–3440.
- [45] Shuai B, Zuo Z, Wang B, et al. Scene segmentation with dag-recurrent neural networks. TPAMI. 2017;.
- [46] Hu H, Deng Z, Zhou GT, et al. Labelbank: Revisiting global perspectives for semantic segmentation. arXiv preprint arXiv:170309891. 2017;.
- [47] Liang X, Zhou H, Xing E. Dynamic-structured semantic propagation network. CVPR. 2018;.
- [48] Liu W, Rabinovich A, Berg AC. Parsenet: Looking wider to see better. arXiv preprint arXiv:150604579. 2015;.
- [49] Dai J, He K, Sun J. Boxesup: Exploiting bounding boxes to supervise convolutional networks for semantic segmentation. In: ICCV; 2015. p. 1635–1643.
- [50] Arnab A, Jayasumana S, Zheng S, et al. Higher order conditional random fields in deep neural networks. In: ECCV; 2016. p. 524–540.
- [51] Lin G, Shen C, Van Den Hengel A, et al. Efficient piecewise training of deep structured models for semantic segmentation. In: CVPR; 2016. p. 3194–3203.
- [52] Wu Z, Shen C, Hengel Avd. Bridging category-level and instance-level semantic image segmentation. arXiv preprint arXiv:160506885. 2016;.
- [53] Lin G, Milan A, Shen C, et al. Refinenet: Multi-path refinement networks for high-resolution semantic segmentation. In: CVPR; 2017.
- [54] Caesar H, Uijlings J, Ferrari V. Coco-stuff: Thing and stuff classes in context. arXiv preprint arXiv:161203716. 2016;.
- [55] Mottaghi R, Chen X, Liu X, et al. The role of context for object detection and semantic segmentation in the wild. In: CVPR; 2014.
- [56] Liang X, Lee L, Xing EP. Deep variation-structured reinforcement learning for visual relationship and attribute detection. In: CVPR; 2017.
- [57] Krishna R, Zhu Y, Groth O, et al. Visual genome: Connecting language and vision using crowdsourced dense image annotations. International Journal of Computer Vision. 2017;123(1):32–73.

Chapter 5

Answering Natural-Language Questions with Neuro-Symbolic Knowledge Bases

Haitian Sun, Machine Learning Department, Carnegie Mellon University

Pat Verga, Google AI

William W. Cohen, Google AI

5.1. Introduction

Symbolic reasoning systems based on first-order logics are computationally powerful: for instance, Datalog is EXPTIME-complete [1], and can therefore emulate any exponential-time computation. In contrast neural networks are built by cascading a fixed number of simple numeric operations that can be performed quickly on a parallel computer, so unless $P=NP$, neural networks cannot, in general, emulate symbolic logics. Hence bridging the gap between neural and symbolic methods requires achieving a delicate balance: one needs to incorporate just enough of symbolic reasoning to be useful for a task, but not so much as to cause computational intractability.

In this chapter we first present results that make this claim precise, and then use these formal results to inform the choice of a neuro-symbolic knowledge-based reasoning system, based on a set-based dataflow query language. We then present experimental results with a number of variants of this neuro-symbolic reasoner. One variant allows use of embedded representations of sets, and another variant extends the logic from a traditional fixed vocabulary of predicates to “open” predicates described by text.

Finally we show that this neuro-symbolic reasoner can be closely integrated into modern neural language models, resulting in a new fact-injected language model (FILM). FILM is a very effective knowledge-based question-answering system, and has some properties common to most symbolic systems, but uncommon in neural NLP systems: in particular, FILM can be “told” new facts declaratively, by simply updating the memory that encodes external knowledge.

5.2. The computational dilemma of neural implementations of first-order logic

5.2.1. First order logics are expressive

Devising scalable, elegant neuro-symbolic AI systems is long-standing technical problem. One underlying issue is that AI systems based on probabilistic first-order logics, a popular choice in the symbolic AI literature, are simply too computationally powerful to be scalably implemented by neural networks.

To illustrate this point, consider a simple Datalog rule

$$p(X, Y) \leftarrow q(Y, Z), r(Z, Y)$$

The first step toward implementing this in a neural network would be to convert it to a finite boolean disjunction. If C is the set of objects in the database, this could be written

$$\bigvee_{\exists x, y, z \in C} (p(x, y) \vee \neg q(y, z) \vee \neg r(z, y))$$

Now this Boolean disjunction can be embedded in a neural network, e.g. to initialize an architecture [2] or as a regularizer [3, 4]. For probabilistic first-order languages (e.g., Markov logic networks [5]), this “grounding” process results in a directed graphical model—for a survey, see [6].

The problem with this approach is that groundings can be very large: even the small rule above gives a grounding of size $o(|C|^3)$, which is likely much larger than the size of the KB, and a grounding of size $o(|C|^n)$ would result from a longer rules, such as $p(X_0, X_n) \leftarrow q_1(X_0, X_1), q_2(X_1, X_2), \dots, q_n(X_{n-1}, X_n)$. This exponential growth makes grounding an inefficient basis for neural implementations of logic. In retrospect this is not surprising: most probabilistic logics are #P-complete or worse, and neural networks can be evaluated in time polynomial in their size, so an efficient neural logic would only be possible if P=#P.

5.2.2. A restricted probabilistic first-order logic

In prior work [7] we considered the complexity of restricted probabilistic Datalog programs. Briefly, let a *database*, D , be a set $\{f_1, \dots, f_N\}$ of ground facts, let a *theory*, T , be a set of function-free Horn clauses, and let the set C be the set of all database constants. The *least model* for a pair D, T , written $\text{model}(D, T)$, is the smallest superset of D that is deductively closed with respect to every clause in T . The least model can be computed by iterative bottom-up inference, which takes time polynomial in the size of the database $|D|$, but exponential in the length of the longest clause in T [8]. Alternatively it can be computed by top-down inference, using an algorithm that inputs a query fact f and determines if $f \in \text{model}(D, T)$ by searching for a proof tree. More generally, top-down proof methods usually find all derivable facts that match some pattern Q , e.g., find all values of Y such that $\text{uncle}(\text{joe}, Y)$ holds.¹ If T is not recursive, or if recursion is limited to a fixed depth, then answers to a query Q can be found by systematically searching the proof tree.

¹Formally, given $Q = \text{uncle}(\text{joe}, Y)$, we would like to find all $f \in \text{model}(D, T)$ which are *instances* of Q , where an f is defined to be an *instance* of Q iff there is some way of replacing the variables in Q with constants from C to construct the fact f .

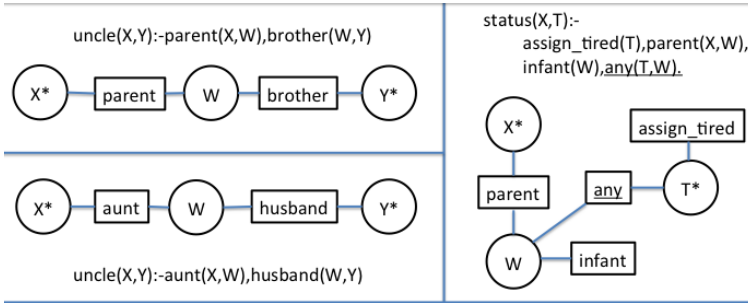


Figure 1. Examples of factor graphs for three sample clauses.

It is well-known that probabilistic Datalog extensions are computationally complex with the most commonly-used probabilistic semantics, including even the simplest “possible worlds” model of tuple independence model [9]. We thus considered an alternative probabilistic extension of top-down proof procedures called *stochastic logic programs* (SLPs) [10]. In SLPs the theory is extended by associating a non-negative scalar weight with each rule. The weights of the rules used in a proof are multiplied to derive a weight for a proof, and the weights for alternative proofs of a fact are added. Finally, if we assume that some answers to Q do exist, SLPs define a conditional probability distribution $P(f|Q)$ over answers f to the query Q by normalizing these weights. SLPs can be thought of as logic-program analogs to probabilistic programming languages like Church [11], and are closely related to stochastic grammars, such as PCFGS, except that stochastic choices are made during theorem-proving, rather than rewriting a string.

Unfortunately, SLPs are also #P hard, even in very restricted settings:

Theorem 1. *Computing $P(f|Q)$ in SLPs is #P hard even if there are only two answers to Q , the theory contains only two non-recursive clauses, all predicates are of arity 2, and the database contains only 13 facts [7].*

However, there is an interesting tractable restriction to SLPs, which is suggested by the illustrations of Figure 1. In particular, any single clause can be converted to a factor graph [12]: for each logical variable W in the body, there is a random variable W ; and for every literal $q(W_i, W_j)$ in the body of the clause, there is a factor with potentials linking variables W_i and W_j . Inference on factor graphs is efficient when the graphs are polytrees (i.e., when there is just one path between any two nodes in the graph), leading to a tractable subset of SLPs. This subset can be extended to theories with many clauses, and to theories with limited-depth recursion.

This subset of logic was implemented in the Tensorlog system [7].

5.3. Neural implementations of set-based dataflow languages

5.3.1. Set operations defined with sparse matrix and vector operations

Given a query Q , TensorLog compiles out the message-passing process used for factor-graph inference on a theory, and converts the numerical operations used in inference into

NQL expression		Vector-matrix specification	Comments
$\mathbf{s.rel}_j()$	$\equiv$	$\mathbf{sM}_j$	
$\mathbf{s.rel}_j(-1)$	$\equiv$	$\mathbf{sM}_j^T$	
$\mathbf{s} \mid \mathbf{t}$	$\equiv$	$\mathbf{s} + \mathbf{t}$	
$\mathbf{s} \& \mathbf{t}$	$\equiv$	$\mathbf{s} \odot \mathbf{t}$	$\odot$ is Hadamard product
$\mathbf{s.if_any}(\mathbf{t})$	$\equiv$	$\mathbf{s} \ \mathbf{t}\ _1$	
$\mathbf{s.follow}(\mathbf{r})$	$\equiv$	$\mathbf{s}(\sum_{i=1}^k \mathbf{r}[i] \mathbf{M}_{\pi_i})$	

Table 1. Matrix-vector implementation for NQL operators. Vectors $\mathbf{s}, \mathbf{t}, \mathbf{r}$ correspond to s, t, r respectively, $\mathbf{r}$ is over the relation group $\pi_1, \dots, \pi_k$, and $\mathbf{M}_\pi$ corresponds to the relation rel .

a neural network, which can then be cross-compiled into a computation graph in Theano or Tensorflow.

Interestingly, the message-passing algorithm for factor graphs which forms the basis of Tensorlog’s inference engine is based on only a few simple operations. We assume without loss of generality that every database constant (aka entity) in C is an integer between 1 and N . Then a set of k entities can be encoded as a k -hot vector in $\mathbb{R}^N$, and more generally any mapping from entities to real numbers can be represented as a vector in $\mathbb{R}^N$. Binary relations between entities can be encoded as sparse matrices in $\mathbb{R}^{N \times N}$. The only operations needed for message passing in Tensorlog are the following, all of which correspond directly to weighted versions of natural logical operations. Below $\mathbf{x}_i \in \mathbb{R}^N$ corresponds to a weighting over entities, $\mathbf{M}_j \in \mathbb{R}^{N \times N}$ corresponds to a binary relation r_j , and the Hadamard product of two vectors $\mathbf{x}_1$ and $\mathbf{x}_2$ is written $\mathbf{x}_1 \odot \mathbf{x}_2$:

- $\mathbf{x}_{out} = \mathbf{x}_{in} \mathbf{M}_j$, which computes a weighted version of $\mathbf{x}_{out} = \{y \mid r_j(x, y) \wedge x \in \mathbf{x}_{in}\}$
- $\mathbf{x}_{out} = \mathbf{x}_{in} \mathbf{M}_j^T$, which computes a weighted version of $\mathbf{x}_{out} = \{y \mid r_j(y, x) \wedge x \in \mathbf{x}_{in}\}$
- $\mathbf{x}_{out} = \mathbf{x}_{in1} + \mathbf{x}_{in2}$, which computes a weighted version of the union of $\mathbf{x}_{in1}$ and $\mathbf{x}_{in2}$
- $\mathbf{x}_{out} = \mathbf{x}_{in1} \odot \mathbf{x}_{in2}$, which computes a weighted version of the intersection of $\mathbf{x}_{in1}$ and $\mathbf{x}_{in2}$
- $\mathbf{x}_{out} = \mathbf{x}_{in1} \odot \|\mathbf{x}_{in2}\|_1$, which is a sort of conditional statement: if the set $\mathbf{x}_{in2}$ is empty, then it returns an empty set, but if the set $\mathbf{x}_{in2}$ has many elements, then it returns a set with the same non-zero elements as $\mathbf{x}_{in1}$.

Based on this observation, we designed a dataflow query language called NQL (for Neural Query Language) which included exactly these operations, plus one additional operation we called “follow”. The operations are summarized in Table 1, and the main impetus for this language redesign was simply to make the power of Tensorlog available to neural-network practitioners who are unfamiliar with logic and declarative programming. The “follow” operation also allows a natural way to support second-order logic constructions—in particular, to reason with sets of predicates as well as with sets of entities.

NQL is embedded in Python. Some NQL expressions are produced using an *NQL context object*, which contains information about the database. For example, suppose `context` is a context object initialized with a database derived from a widely-used example database of genealogy information about European royal families² from which

²The dataset is widely distributed as an example of the GED format, for example under <https://github.com/jdfekete/geneaquilt>

we have derived the 12 familial relations aunt, brother, daughter, father, husband, mother, nephew, niece, sister, son, uncle, and wife. One can create singleton set vectors using a method called “one” and a context object, e.g.

```
henry8 = context.one('Henry_VIII of house of Tudor', 'person.t')
```

(Here `person.t` is a type for the object: NQL supports typed entities, but for simplicity we will not discuss this here.) Then `henry8.wife()` would evaluate to a set of six people, `{'Anne of house of Boleyn':1.0, 'Anne of Cleves':1.0,..., 'Jane of house of Seymour':1.0}`. Relations can also be chained: for example, the set of sons of Henry VIII's daughters could be written as `henry8.daughter().son()`, and the inverse of a relation can be accessed by adding an argument `-1` to a relation-name method: for example, Henry VIII's parents could be found with the expression `henry8.son(-1)`, and intersections, unions, and conditionals are also supported.

NQL also allows one to manipulate sets of *relations*, thus allowing dataflow operations similar to second-order logical constructs. For example, one could create the set

```
child = context.one('daughter', 'rel.t') | context.one('son', 'rel.t')
```

and then `henry8.follow(child)` would evaluate to the set of all daughters and sons of Henry VIII.

Because relation sets and entity sets are simply vectors, their parameters can be learned. This makes NQL very powerful for specifying models that use existing knowledge, as we will see below.

5.3.2. Dataflow on sets is not first-order logic

It should be emphasized that the set-based dataflow operations described above which comprise NQL are *not* equivalent expressively to first-order logic. Since they include the message-passing operations used in TensorLog, they can be used to emulate any expression in the *restricted* subset of logic supported by TensorLog—i.e., clauses that correspond to polytree factor graphs. However they are certainly less expressive than first-order logic, even restricted to binary predicates, and for completeness we include a proof of this hardness result below.

Theorem 2. *There exists a KG G with only 34 binary facts such that every 3SAT problem with m variables and n clauses can be reduced to finding a solution to a conjunction of $O(m+n)$ binary predicates.*

Corollary 1. *Let $\mathcal{L}$ be any language, including NQL, whose expressions can be evaluated in polynomial time. Then there exist arity-two Datalog programs over small knowledge graphs that do not correspond to any polynomial-sized expression in $\mathcal{L}$.*

We include this result because several NQL-like graph query languages have been incorrectly conjectured to be as expressive as arity-two Datalog languages—for example, see [13, Theorem 1] and [14].

Proof. Consider a 3SAT formula ϕ with m variables and n clauses

$$\phi = (v_{1,1} \vee v_{1,2} \vee v_{1,3}) \wedge \dots \wedge (v_{n,1} \vee v_{n,2} \vee v_{n,3})$$

where $v_{i,j}$ is either x_k or $\neg x_k$ for some $k \in \{1, \dots, m\}$. We will start with a simple construction which uses arity-3 predicates and then extend it. Consider a KG that contains the six facts $\text{not}(0,1)$, $\text{not}(1,0)$, $\text{or}(0,0,1)$, $\text{or}(0,1,1)$, $\text{or}(1,0,1)$, $\text{or}(1,1,1)$. It is easy to see that ϕ is satisfiable iff the following clause has a satisfying assignment:

$$\text{not}(P_1, N_1), \dots, \text{not}(P_m, N_m), \text{or}(V_{1,1}, V_{1,2}, V_{1,3}), \dots, \text{or}(V_{n,1}, V_{n,2}, V_{n,3}) \quad (1)$$

where $V_{i,j}$ is replaced with the variable P_k if $v_{i,j} = x_k$ and $V_{i,j}$ is replaced with the variable N_k if $v_{i,j} = \neg x_k$. Thinking of this operationally as a Prolog program, the first m literals pick an consistent assignment to the m boolean variables, encoded in $2m$ logical variables $P_1, \dots, P_m, N_1, \dots, N_m$, so either $P_k = 0$ and $N_k = 1$ or vice-versa. The remaining literals check if this assignment satisfies the clauses of ϕ .

To get rid of the ternary “or” predicate, we can create a new type of object which represents a vector of three bits. To do this we will add to the KG these 24 facts

$$\begin{aligned} &\text{bit1}(000,0) \text{ bit1}(001,0) \dots \text{bit1}(111,1) \\ &\text{bit2}(000,0) \text{ bit2}(001,0) \dots \text{bit2}(111,1) \\ &\text{bit3}(000,0) \text{ bit3}(001,1) \dots \text{bit3}(111,1) \end{aligned}$$

which collectively define predicates like $\text{bit2}(B, X)$, which holds when X is the second bit of bit-vector B . We can then replace the four ternary “or” facts with eight additional facts

$$\text{or}(000,0), \text{or}(001,1), \dots, \text{or}(111,1)$$

and finally replace every literal $\text{or}(V_{i,1}, V_{i,2}, V_{i,3})$ in conjunction of Eq 1 with the conjunction of four literals

$$C_i \equiv \text{bit1}(B_i, V_{i,1}), \text{bit2}(B_i, V_{i,2}), \text{bit3}(B_i, V_{i,3}), \text{or}(B_i, 1)$$

This conjunction is satisfied when B_i correctly encodes the values of the $V_{i,j}$ ’s, and has at least one bit set, so it also checks the truth of the i -th clause of ϕ . Hence the conjunction

$$\text{not}(P_1, N_1), \dots, \text{not}(P_m, N_m), C_1, \dots, C_n$$

is true iff ϕ is satisfiable. $\square$

5.3.3. Experimental results with NQL

To illustrate an application of NQL, we consider the KB question-answering (KBQA) dataset WebQuestionsSP [15], where the model takes natural language questions as inputs and predicts a set of entities as answers. This dataset contains 4737 natural language questions, all of which are answerable using FreeBase [16], a large open-domain KB. Each question q is again labeled with the entities $\mathbf{x}_q$ that appear in it. FreeBase contains two kinds of nodes: real-world *entities*, and *compound value types* (CVTs), which represent non-binary relationships or events (e.g., a movie release event, which includes a movie id, a date, and a place.) Real-world entity nodes can be related to each other or to a CVT node, but CVT nodes are never directly related to each other. In this dataset, all

questions can be answered with 1- or 2-hop chains, and all 2-hop reasoning chains pass through a CVT entity. Below is a simple NQL model that learns to answer WebQuestionsSP questions:

$$\begin{aligned}
 \mathbf{r}_{E \rightarrow E} &= f_{E \rightarrow E}(q); \quad \mathbf{r}_{E \rightarrow CVT} = f_{E \rightarrow CVT}(q); \quad \mathbf{r}_{CVT \rightarrow E} = f_{CVT \rightarrow E}(q) \\
 \hat{\mathbf{a}} &= \mathbf{x}_q.\text{follow}(\mathbf{r}_{E \rightarrow CVT}).\text{follow}(\mathbf{r}_{CVT \rightarrow E}) \mid \mathbf{x}_q.\text{follow}(\mathbf{r}_{E \rightarrow E}) \\
 \text{loss} &= \text{tf.softmax_cross_entropy_with_logits}(\mathbf{y}, \hat{\mathbf{a}})
 \end{aligned} \tag{2}$$

The model contains three neural functions, which map a question to relation vectors; one line of NQL which uses the output of these functions to produce a predicted answer set; and a loss function defined in terms of the predicted answer $\hat{\mathbf{a}}$. In training, any errors in the predicted answer set $\hat{\mathbf{a}}$ are back-propagated through the NQL expressions to the f functions. In previous experiments [17] we used simple linear projections of a sum-pooled word2vec encoding of q , and obtained an accuracy on WebQuestionsSP of 52.7%, well above a baseline key-value memory model (which obtained 46.7%) but below the state-of-the-art at the time (which was 68.1%).

5.3.4. Learning to embed entities to support dataflow operations

NQL is heavily dependent on sparse matrix operations, which are not well supported on all accelerators, the sparse “localist” representations used for sets do not always generalize well. To address these problems, we introduced a variant of NQL called Embedding Query Language (EmQL). Different from NQL, where a set of entities is treated as a k -hot vector in $\mathbb{R}^N$, EmQL represents entities and relations as vectors and performs the relational following operation $Y = X.\text{follow}(r)$ in the embedding space. However, simple geometric operations in embedding space do not have the closure properties required by logic: for example, hyperrectangles are closed under intersection but not union. To address this limitation, EmQL uses a novel dense-sparse combination of embeddings and randomized data structures.

In particular, EmQL represents a set of entities $X = \{x_1, \dots, x_n\}$ as a pair of vectors $(\mathbf{a}_X, \mathbf{b}_X)$. Here $\mathbf{a}_X$ is the dense representation of X that describes, intuitively, the *type* of entities in X , computed as the geometric centroid of entities in the embedding space. Let $\mathbf{x}_i \in \mathbb{R}^d$ the embedding of the entity x_i and $\mathbf{v}_X[i]$ the weight of entity x_i in X .

$$\mathbf{a}_X = \sum_{x_i \in X} \mathbf{v}_X[i] \mathbf{x}_i$$

The second component $\mathbf{b}_X$ of the EmQL representation memorizes a limited-size set of entities of this type. Here, we adopt the count-min sketch to represent $\mathbf{b}_X$, i.e. $\mathbf{b}_X = \mathbf{S}_H(\mathbf{v}_X)$. The count-min sketch, $\mathbf{S}_H$, is a randomized data structure that stores the a set of values efficiently with a limited size of memory, and will return the correct values with high probability if queried a limited number of times. We denote the step of looking up the value of the i 'th element x_i from the count-min sketch $\mathbf{b}_X$ as $CM(i, \mathbf{b}_X)$.

Reconstructing the set X from its encoding $(\mathbf{a}_X, \mathbf{b}_X)$ is performed as a “search and filter” process. EmQL first searches the k nearest neighbours with its dense representation $\mathbf{a}_X$. The nearest neighbour search can be performed efficiently with maximum inner

product search (MIPS) [18]. We denote it as $\text{top}_k(\mathbf{a}_X, \mathbf{E})$ where $\mathbf{E}$ is the entity embedding table. The top- k retrieved elements are then filtered by the count-min sketch to get the weights of elements in X .

$$\tilde{\mathbf{v}}_X[i] = \begin{cases} CM(i, \mathbf{b}_X) \cdot \text{softmax}(\mathbf{a}_X, \mathbf{e}_i), & \text{if } i \in \text{top}_k(\mathbf{a}_X, \mathbf{E}) \\ 0, & \text{otherwise} \end{cases}$$

The EmQL representation $(\mathbf{a}_X, \mathbf{b}_X)$ is closed under intersection and union, letting

$$\begin{aligned} \mathbf{a}_{A \cap B} &= \mathbf{a}_{A \cup B} = \frac{1}{2}(\mathbf{a}_A + \mathbf{a}_B) \\ \mathbf{b}_{A \cap B} &= \mathbf{b}_A \odot \mathbf{b}_B, \quad \mathbf{b}_{A \cup B} = \mathbf{b}_A + \mathbf{b}_B \end{aligned}$$

In order to achieve a high recall from the nearest neighbour search, entity embeddings are pretrained to ensure similar entities are close to each other. Various pretraining targets are described in [19].

Another important operation on KBs is relational following, i.e. $Y = X.\text{follow}(R)$. Here, we consider a general case where R is a set of relations with its representation $(\mathbf{a}_R, \mathbf{b}_R)$. In EmQL, the relational following operation, similar to reconstructing sets, is implemented as a “search and filter” process. EmQL constructs a query vector $\mathbf{q}_{R,X} = [\mathbf{a}_R; \mathbf{a}_X; \vec{0}]$ by concatenating the relation embedding $\mathbf{a}_R$, the set embedding $\mathbf{a}_X$, and a zero vector. The embedding table $\mathbf{K} \in \mathbb{R}^{|K| \times 3d}$ to search from is similarly constructed by concatenating the subject, relation, and object embeddings of a triple $t = r_i(x_j, y_l)$, i.e. $\mathbf{r}_t = [\mathbf{e}_{r_i}; \mathbf{e}_{x_j}; \mathbf{e}_{y_l}] \in \mathbb{R}^{3d}$, for all $t \in K$ in the database.

$$\tilde{\mathbf{v}}_K[t] = \begin{cases} CM(i, \mathbf{b}_R) \cdot CM(j, \mathbf{b}_X) \cdot \text{softmax}(\mathbf{q}_{R,X}, \mathbf{r}_t), & \text{if } t \in \text{top}_k(\mathbf{q}_{R,X}, \mathbf{K}) \\ 0, & \text{otherwise} \end{cases}$$

And we project the objects of the retrieved triples as the set of answers Y . The weights of elements in Y are

$$\tilde{\mathbf{v}}_Y[l] = \sum_{t \in \text{top}_k(\mathbf{q}_{R,X}, \mathbf{K}), t=(\cdot, \cdot, y_l)} \tilde{\mathbf{v}}_K[t]$$

5.3.5. Experimental results with EmQL

EmQL was evaluated on synthetic queries on KBs, and also used as an inference engine for solving multihop KB question answering (KBQA) tasks, analogously to the way we used NQL above.

For the synthetic queries, we followed [20], who defined nine different types of queries (see Table 2) that perform different types combinations of discrete operations such as set intersection and union. (For example, queries of type “ip” compute the intersection of two relational following tasks and then perform another relational following operation on the intersected set.) Training and testing data for Query2Box, the method described in [20], are automatically constructed from three widely used KB completion (KBC) benchmarks. In our first experiments we evaluated to what extent EmQL was

“faithful” to the logic it approximates—i.e., the extent to which this differential neural query language approximates the corresponding expressions in a symbolic language. EmQL achieves an average of 91.4, 98.8 and 94.2 Hits@3 among the nine different types of queries on three different KBs, greatly outperforming the models of [20], which had an average Hits@3 of 43.7, 51.1, and 36.2.

	Query Template
1p	$X.\text{follow}(R)$
2p	$X.\text{follow}(R_1).\text{follow}(R_2)$
3p	$X.\text{follow}(R_1).\text{follow}(R_2).\text{follow}(R_3)$
2i	$X_1.\text{follow}(R_1) \cap X_2.\text{follow}(R_2)$
3i	$X_1.\text{follow}(R_1) \cap X_2.\text{follow}(R_2) \cap X_3.\text{follow}(R_3)$
ip	$(X_1.\text{follow}(R_1) \cap X_2.\text{follow}(R_2)).\text{follow}(R)$
pi	$X_1.\text{follow}(R_1).\text{follow}(R_2) \cap X_2.\text{follow}(R_3)$
2u	$X_1.\text{follow}(R_1) \cup X_2.\text{follow}(R_2)$
up	$(X_1.\text{follow}(R_1) \cup X_2.\text{follow}(R_2)).\text{follow}(R)$

Table 2. Nine query templates used in the Query2Box dataset.

Besides testing EmQL for faithfulness, we also tested the generalizability of EmQL on unseen tuples at test time. (This was the principal way that Query2Box was evaluated). Specifically, we hold out some triples from the KB for validation and test, and take the remaining triples as the training KB. EmQL is tested to predict answers for the nine different types of queries using the full KB. Predictions are only evaluated on the subset of answers that require at least one novel tuples at test time. EmQL achieves 49.5, 46.8, and 35.8 Hits@3 on three KBs, bettering the previous state-of-the-art model Query2Box, which gets 48.4, 30.6, and 26.8.

We finally evaluated EmQL on KBQA datasets, two MetaQA [21] and WebQuestionsSP. We ran EmQL using similar inference rules to NQL, from Eq. 2. EmQL achieved a new state-of-the-art performance on all three datasets: 98.6 and 99.1 on MetaQA 2hop and 3hop questions, and 75.5 on WebQuestionsSP, better than NQL, as well as more complicated graph-based models such as GRAFT-Net [22] and Pullnet [23].

5.3.6. Extending dataflow languages with open predicates

NQL and EmQL allow accurate neuro-symbolic reasoning over symbolic KBs. However, even though substantial effort has been made to increase the coverage of KBs, e.g. automated information extraction from text [24, 25], KBs are inevitably incomplete and restricted by its predefined schema. An alternative source of knowledge is text corpora that contain passages describing factual knowledge in natural language.

We thus also extended our neuro-symbolic query languages to operate over a virtual knowledge base (VKB) extracted from text, to tackle the problem of incompleteness KBs. Here, we focus on representing binary relationships between pairs of entities (e_1, e_2) , and leave the n -ary relations for future work. Different from KBs that have a predefined vocabulary of relations, the relationship between pairs of entities in the corpus are described in text, so there is no fixed vocabulary of relations. Hence the first step to build a virtual KB, similar to EmQL in the embedding space, is to learn to encode as

vectors the relationship between pairs of entities appearing in text. In prior work, such neural encodings of entity-tagged text has been called a “virtual KB” [26], and we follow that terminology here. Since this representation uses an “open” set of relations, rather than a fixed set, we called the query language over this VKB the Open Predicate Query Language (OPQL).

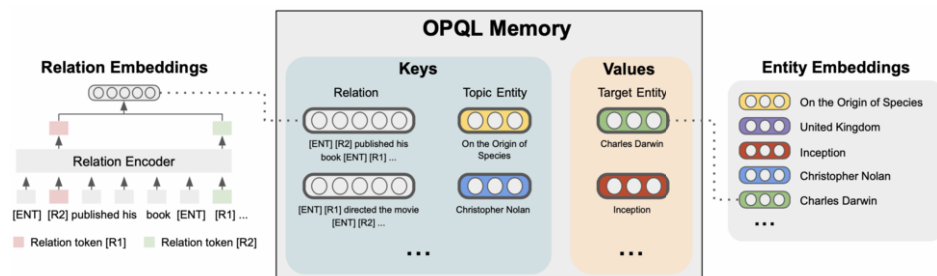


Figure 2. Virtual KB constructed by OPQL. The VKB is a key-value memory. Keys are computed from embeddings of the topic entity, e.g. *On the Origin of Species*, looked up from the entity embedding table, and relation embeddings from the pretrained relation encoder. Values are embeddings of target entities. The memory is constructed from any entity-linked text corpus, e.g. Wikipedia.

The first step to build the virtual KB is to learn a relation encoder that encodes the relationship between a pair of entity (e_1, e_2) . The input to the relation encoder is a sequence of n tokens $C = [c_0, c_1 \dots c_n]$ for an arbitrary text span, e.g. “*Charles Darwin* published his book *On the Origin of Species* in 1859”, that mentions a set of entities, e.g. *Charles Darwin* (Q1035) and *On the Origin of Species* (Q20124) in Wikidata. An entity pair (e_1, e_2) contains a subject entity e_1 and an object entity e_2 . In the example above, *On the Origin of Species* is the subject entity e_1 and *Charles Darwin* is the object entity e_2 . Potentially³ each distinct entity pair in a sentence can be encoded. The relationship between a pair of entities is directional, so the pairs (e_1, e_2) and (e_2, e_1) are encoded differently.

OPQL encodes the relationship between a pair of entities (e_1, e_2) in a natural language sentence C where both e_1 and e_2 are mentioned. Intuitively C describes the relationship between e_1 and e_2 in the context, and we train a relation encoder to represent the relationship as a vector. To indicate the location of the topic and target entities e_1 and e_2 we introduce two special tokens [R1] and [R2] that are inserted directly after the mentions of the topic and target entities in the sentence (e.g., C_r becomes “*Charles Darwin* [R2] published his book *On the Origin of Species* [R1] in 1859”). The contextual encodings of [R1] and [R2] will be used to compute relation embeddings. We also introduce another special token [ENT] to mask the topic and target entity mentions; thus sentence C_r finally becomes “[ENT] [R2] published his book [ENT] [R1] in 1859”. Masking the mentions of entities prevents the relation encoder from memorizing the surface forms of the entities, and helps it generalize to similar relations involving other entities.

Given this preprocessing of a subject-object entity pair (e_1, e_2) , the relation embedding $\mathbf{r}_{e_1, e_2}$ is defined as follows. Let s and t be the location of tokens [R1] and [R2] in the masked sentence C_r . We construct a projection of the concatenation of the contextual embeddings $\mathbf{h}_s$ and $\mathbf{h}_t$ at the locations s and t , as follows:

³Heuristics can be applied to limit the number of entity pairs derived from text.

$$\mathbf{r}_{e_1, e_2} = \mathbf{W}_r^T [\mathbf{h}_s; \mathbf{h}_t] \quad (3)$$

We train the relation encoder such that relation mentions containing the same entity pairs are more similar to each other than relation mentions that contain different entity pairs. Specifically, mini-batches are constructed which contain at least two documents that contain entity pair (e_1, e_2) , as well as negative documents containing different relation pairs. Use $\mathbf{r}_{e_1, e_2}$ to denote the embedding from input C that of (e_1, e_2) , use $\mathbf{r}_{e_i, e_j}$ for embeddings from documents $C'_0, \dots, C'_n$ of other pairs (e_i, e_j) 's, and let $\mathbb{I}_{e_i=e_1, e_j=e_2}$ be an equality indicator for entity pairs in the minibatch. We maximize the inner product between relation embeddings iff the same pair (e_1, e_2) is mentioned in the candidates. The pretraining data is distantly constructed from Wikipedia.

$$L_{\text{rel}} = \text{cross_entropy}(\text{softmax}(\mathbf{r}_{e_1, e_2}^T \mathbf{r}_{e_i, e_j}), \mathbb{I}_{e_i=e_1, e_j=e_2})$$

The virtual KB constructed by OPQL is stored as a key-value memory $(\mathbf{K}, \mathbf{V})$. To build the memory, we iterate through all sentences in the Wikipedia corpus that contain two or more entities. Given an input C that mentions a pair of entities e_1 and e_2 , the key embedding $\mathbf{k}_{e_1, e_2}$ for the pair (e_1, e_2) is constructed compositionally using the embeddings of topic entities $\mathbf{e}_1$ from the entity embedding table $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times d_e}$ and the pretrained relation embedding $\mathbf{r}_{e_1, e_2}$. $\mathbf{W}_k$ is a linear projection matrix that will be learned in the finetuning tasks. The value $\mathbf{v}_{e_1, e_2}$ is the embedding of the target entity $\mathbf{e}_2$, giving us:

$$\mathbf{k}_{e_1, e_2} = \mathbf{W}_k^T [\mathbf{e}_1; \mathbf{r}_{e_1, e_2}] \in \mathbf{K}, \quad \mathbf{v}_{e_1, e_2} = \mathbf{e}_2 \in \mathbf{V}$$

The relational following operation $Y = X.\text{follow}(R)$ is executed with a key-value lookup from the OPQL memory. Specifically, a query vector $\mathbf{q}_{X,Y}$ is composed using the embedding of the topic entity X and the relation embedding $\mathbf{r}_{X,Y}$ of the question:

$$\mathbf{e}_X = \sum_i \alpha_i \cdot \mathbf{e}_{x_i}, \quad x_i \in X, \quad \mathbf{q}_{X,Y} = \mathbf{W}_q^T [\mathbf{e}_X; \mathbf{W}_t^T \mathbf{r}_{X,Y}]$$

where in general $\mathbf{e}_X$ is the weighted average of embeddings of entities $e_{x_i} \in X$ (this general case applies for multi-hop questions), α_i is the weight of x_i in X , and $\mathbf{W}_t$ is a learned projection matrix.

The query $\mathbf{q}_{X,Y}$ is then used to retrieve against the key memory $\mathbf{K}$, returning the top k entries with the largest inner product scores (k_{e_i, e_j}, e_j) .⁴ The T_k set of top- k retrieved values $\{e_j\}$ are the answers Y . The weights β_{e_i, e_j} of entity e_j is the softmax of scores of the top k retrieval result, denoted as $\text{top}_k(\mathbf{q}_{X,Y}, \mathbf{K})$. The retrieval process is optimized with cross entropy with softmax against the oracle set of answers.

$$\beta_{e_i, e_j} = \begin{cases} \text{softmax}(\mathbf{q}_{X,Y}^T \mathbf{k}_{e_i, e_j}) & \text{for } (e_i, e_j) \in T_k \\ 0, & \text{else} \end{cases}$$

$$L_{\text{follow}} = \text{cross_ent}(\beta_{e_i, e_j}, \mathbb{I}_{e_j \in \text{Ans}})$$

⁴The value v_{e_i, e_j} of the entry is always e_j , so we write it as (k_{e_i, e_j}, e_j) for short.

As in NQL and EmQL, relation following operations can be chained to answer multi-hop questions: e.g. two-hop question can be decomposed into

$$Y = X.\text{follow}(R_1).\text{follow}(R_2).$$

In this case, the predicted set of intermediate entities from the previous hop $X^{(t)} = X^{(t-1)}.\text{follow}(R_{t-1})$ becomes the input to the next hop, i.e. $X^{(t+1)} = X^{(t)}.\text{follow}(R_t)$. In our model we use a single relation embedding $\mathbf{r}_{X,Y}$ for the question but learn a different projection matrix $\mathbf{W}_t^{(t+1)}$ for each hop: e.g. we use

$$\mathbf{q}_{X,Y}^{(2)} = \mathbf{W}_q^T [\mathbf{e}_{X^{(1)}}; \mathbf{W}_t^{(2)T} \mathbf{r}_{X,Y}]$$

to form the query $\mathbf{q}_{X,Y}^{(2)}$ for the second hop. In the multi-hop setting, the loss is only computed at the last step. The number of hops is a hyper-parameter and in our experiments it is given for each dataset. We experimented with OPQL on the MetaQA and the Multihop Slot Filling (MSF) [26] datasets. Here, we use the text corpus of MetaQA to construct the virtual KB and run the reasoning task. The MSF dataset is similar to MetaQA that contains semi-structured queries and a preprocessed text corpus, but has 888 relations, much more than MetaQA. Experiments showed that constructing the virtual KB using the pretrained relation encoder leads to 1.6 and 3.7 points improvement on the MSF dataset over the prior state-of-the-art [26] (called DrKIT) and gets very competitive results on MetaQA 2hop and 3hop questions. If the relation encoder is finetuned on the datasets, OPQL outperforms the baselines by 2.5 points on MetaQA 2hop questions, and by 2.3 and 5.3 on MSF 2hop and 3hop questions.

The virtual KB constructed by OPQL can also be generalized to novel relations that only appear at test time. To demonstrate this benefit, we hold out some portions of relations in the pretraining phase of DrKIT but evaluate on queries where at least one of the held-out relations is required to answer the queries. This simulates a scenario where KBs are incomplete and limited in their relation vocabularies. In this experiment, the performance of the baseline models drops significantly when evaluated on queries with novel relations not seen at pretraining time (13.8 points drop on MSF 2hop and 7.4 points on MSF 3hop), while the performance of OPQL is nearly unchanged.

5.4. Embedding dataflow languages into neural language models

Neural language models (LMs) [27, 28, 29] that have been pre-trained by self-supervision on large corpora contain rich knowledge about the syntax and semantics of natural language [30]. Pre-trained LMs also contain large amounts of *factual* knowledge about the world [31, 32, 33]. However, while large LMs can be coerced to answer factual queries, they still lack many of the properties that knowledge bases (KBs) typically have. In particular, it is difficult to distinguish answers produced by memorizing factual statements in the pre-training corpus from lower-precision answers produced by linguistic generalization [34]. It is also difficult to add or remove factual information without retraining the LM.

Rather than relying completely on the latent knowledge encoded in an LM, we can instead extend these models with embedded neuro-symbolic reasoning components. We

do this by augmenting a BERT [28] transformer [35] with additional memory components representing KB triples, producing a new model called a Fact Injected Language Model (FILM) [36]). We also consider injecting world knowledge from text with open predicates, producing a model we called OPQL-LM [37]. Since in both cases, the external memory is constructed as a key-value memory, we will mainly discuss the retrieval and integration process in Fact Injected Language Model (FILM), and then briefly discuss the changes made in OPQL-LM to adapt FILM to the virtual KB.

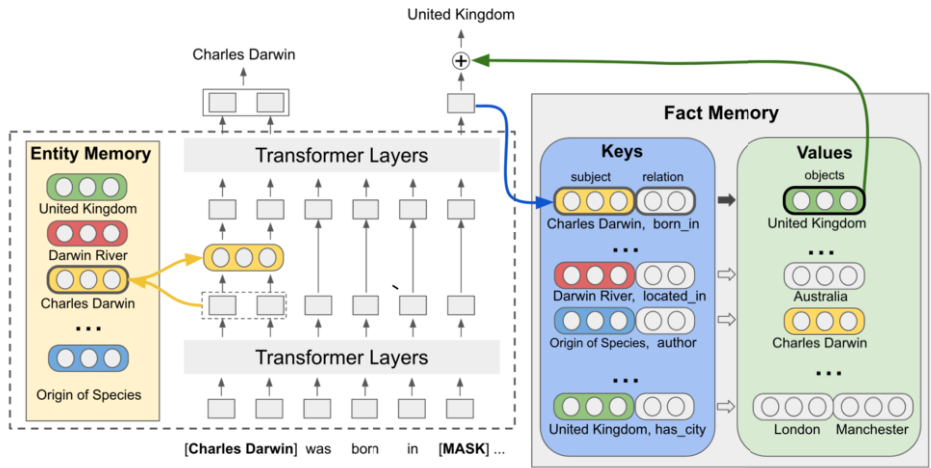


Figure 3. FILM architecture. The model takes a piece of text (a question during fine-tuning or arbitrary text during pre-training) and first contextually encodes it with an entity enriched transformer. FILM uses the contextually encoded MASK token as a query to the fact memory. In this case, the contextual query chooses the fact key (*Charles Darwin, born_in*) which returns the set of values {*United Kingdom*} (The value set can be multiple entity objects such as the case from calling the key [*United Kingdom, has_city*]). The returned object representation is incorporated back into the context in order to make the final prediction.

5.4.1. Representing and Retrieving Entities

The input to our model is a piece of text; either a question during fine tuning or a paragraph in pre-training. Pretraining is formulated as a cloze-type Question Answering (QA) task: given a paragraph $p = \{w_1, \dots, w_{|p|}\}$ with mentions $\{m_1, \dots, m_n\}$, we sample a single mention m_i to act as the cloze answer and replace all tokens of m_i with [MASK] tokens. The entity in $\mathcal{E}$ named by the masked entity is the answer to the cloze question q ('United Kingdom' in the example input of Figure 3). Mentions in the paragraph other than m are referred to below as *context mentions*. We train this model to jointly link context entities and predict answer entities.

Our entity memory $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times d_e}$ is a matrix containing a vector for each entity in $\mathcal{E}$ and trained as an entity-masked LM. The model input is a text span containing unlinked entity mentions with known boundaries⁵. Mentions are masked with some prob-

⁵[38] also showed the model is capable of learning to predict these boundaries. For simplicity, in this work we assume they are given.

ability. Our entity memory follows Entity as Experts (EaE) [38] which interleaves standard Transformer [35] layers with layers that access the entity memory⁶.

Given a piece of text $q = \{w_1, \dots, w_{|q|}\}$ the contextual embedding $\mathbf{h}_i^{(l)}$ is the output at the i 'th token of the l 'th intermediate transformer layer.

These contextual embeddings are used to compute query vectors that interface with the entity memory. For each context mention $m_i = (e_{m_i}, s_{m_i}, t_{m_i})$ in q , we form a query vector to access the Entity memory by concatenating the context embeddings for the mention m_i 's start and end tokens, $\mathbf{h}_{s_{m_i}}^{(l)}$ and $\mathbf{h}_{t_{m_i}}^{(l)}$ and projecting them into the entity embedding space. We use this query to compute attention weights over the full entity vocabulary and produce an attention-weighted sum of entity embeddings $\mathbf{u}_{m_i}^{(l)}$. The result is then projected back to the dimension of the j -indexed contextual token embeddings, and added to what would have been the input to the next layer of the Transformer:

$$\mathbf{h}_{m_i}^{(l)} = \mathbf{W}_e^T [\mathbf{h}_{s_{m_i}}^{(l)}; \mathbf{h}_{t_{m_i}}^{(l)}] \quad (4)$$

$$\mathbf{u}_{m_i}^{(l)} = \text{softmax}(\mathbf{h}_{m_i}^{(l)}, \mathbf{E}) \times \mathbf{E} \quad (5)$$

$$\tilde{\mathbf{h}}_j^{(l+1)} = \mathbf{h}_j^{(l)} + \mathbf{W}_2^T \mathbf{u}_{m_i}^{(l)}, \quad s_{m_i} < j < t_{m_i} \quad (6)$$

After the final transformer layer T , $\mathbf{h}_{m_i}^{(T)}$ is used to predict the context entities $\hat{e}_{m_i}$ and produce a loss with $\mathbb{I}_{e_{m_i}}$, the one-hot label of entity e_{m_i} . Following [38], we supervise the entity access for the intermediate query vector in Eq. 4.

$$\hat{e}_{m_i} = \text{argmax}_{e_i \in \mathcal{E}} (\mathbf{c}_{m_i}^T \mathbf{e}_i)$$

$$\text{loss}_{\text{ctx}} = \text{cross_entropy}(\text{softmax}(\mathbf{c}_{m_i}, \mathbf{E}), \mathbb{I}_{e_{m_i}})$$

$$\text{loss}_{\text{ent}} = \text{cross_entropy}(\text{softmax}(\mathbf{h}_{m_i}^{(l)}, \mathbf{E}), \mathbb{I}_{e_{m_i}})$$

5.4.2. Representing and Retrieving Predicates and Facts

The model also contains a second memory, populated by either triples from the knowledge base $\mathcal{K}$ or predicates from entity pair text mentions. The fact memory shares its entity representations with the entity memory embeddings in $\mathbf{E}$, but each element of the fact memory corresponds to a symbolic substructure, namely a key-value pair $((s, r), \{o_1, \dots, o_n\})$.

The key (s, r) is a (subject entity, relation) pair, and the corresponding value $\{o_1, \dots, o_n\}$ is the list of object entities associated with s and r , i.e. $(s, r, o_i) \in \mathcal{K}$ for $i = \{1, \dots, n\}$. Conceptually, KB triples with the same subject entity and relation are grouped into a single element. We call the subject and relation pair $a_j = (s, r) \in A$ a *head pair* and the list of objects $b_j = \{o_1, \dots, o_n\} \in B$ a *tail set*⁷. In more detail, we encode a head pair $a_j = (s, r) \in A$ by concatenating embeddings for the subject entity and relation, and then projecting them linearly to a new head-pair embedding space. More pre-

⁶We follow the implementation of [38] and have a single entity memory access between the fourth and fifth transformer layers.

⁷The size of the tail set b_j can be large for a popular head pair (s, r) . In such cases, we randomly select a few tails and drop the rest of them. The maximum size of the tail set is 32 in the experiments in this paper.

cisely, let $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times d_e}$ be the entity embeddings described in the previous section, and $\mathbf{R} \in \mathbb{R}^{|\mathcal{R}| \times d_r}$ be embeddings of relations $\mathcal{R}$ in the knowledge base $\mathcal{K}$. We encode a head pair a as:

$$\mathbf{a}_j = \mathbf{W}_a^T [\mathbf{s}; \mathbf{r}] \in \mathbb{R}^{d_a}$$

where $\mathbf{s} \in \mathbf{E}$ and $\mathbf{r} \in \mathbf{R}$ are the embeddings of subject s and relation r , and $\mathbf{W}_a$ is a learned linear transformation matrix. We let $\mathbf{A} \in \mathbb{R}^{|A| \times d_a}$ denote the embedding matrix of all head pairs.

Let the answer for q be denoted e_{ans} , and its masked mention $m_{\text{ans}} = (e_{\text{ans}}, s_{\text{ans}}, t_{\text{ans}})$. For a masked mention m_{ans} , define a query vector to access the fact memory as:

$$\mathbf{v}_{m_{\text{ans}}} = \mathbf{W}_f^T [\mathbf{h}_{s_{\text{ans}}}^{(T)}; \mathbf{h}_{t_{\text{ans}}}^{(T)}] \quad (7)$$

where $\mathbf{h}_{s_{\text{ans}}}^{(T)}$ and $\mathbf{h}_{t_{\text{ans}}}^{(T)}$ are the contextual embeddings for the start and end tokens of the mention m_{ans} , and $\mathbf{W}_f$ is the linear transformation matrix into the embedding space of head pairs $\mathbf{A}$.

Head pairs in A are scored by the query vector $\mathbf{v}_{m_{\text{ans}}}$ and the top k head pairs with the largest inner product are retrieved. This retrieval process on the fact memory is distantly supervised. We define a head pair to be a *distantly supervised positive example* $a_{\text{ds}} = (s, r)$ for a passage if its subject entity s is named by a context mention m_i and the masked entity e_{ans} is an element of the corresponding tail set, i.e. $e_{\text{ans}} \in b_{\text{ds}}$. When no distantly supervised positive example exists for a passage, it is trained to retrieve a special “null” fact comprised of the s_{null} head entity and r_{null} relation: i.e. $a_{\text{ds}} = (s_{\text{null}}, r_{\text{null}})$ and its tail set is empty. This distant supervision is encoded by a loss function:

$$\begin{aligned} \text{TOP}_k(\mathbf{v}_{m_{\text{ans}}}, \mathbf{A}) &= \underset{k, j \in \{1, \dots, |A|\}}{\text{argmax}} \mathbf{a}_j^T \mathbf{v}_{m_{\text{ans}}} \\ \text{loss}_{\text{fact}} &= \text{cross_entropy}(\text{softmax}(\mathbf{v}_{m_{\text{ans}}}, \mathbf{A}), \mathbb{I}_{a_{\text{ds}}}) \end{aligned}$$

The result of this query is that the tail sets associated with the top k scored head pairs, i.e. $\{b_j | j \in \text{TOP}_k(\mathbf{v}, \mathbf{A})\}$, are retrieved from the fact memory.

5.4.3. Integrating Knowledge and Context

Next, tail sets retrieved from the fact memory are aggregated. Recall that a tail set b_j returned from the fact memory is the set of entities $\{o_1, \dots, o_n\}$ s.t. $(s, r, o_i) \in \mathcal{K}$ for $i \in \{1, \dots, n\}$ with the associated $a_j = (s, r)$. Let $\mathbf{o}_i \in \mathbf{E}$ be the embedding of entity o_i . We encode the returned tail set b_j as a weighted centroid of the embeddings of entities in the tail set b_j .

$$\mathbf{b}_j = \sum_{o_i \in b_j} \alpha_i \mathbf{o}_i \in \mathbb{R}^{d_e}$$

where α_i is a context-dependent weight of the object entity o_i . To compute the weights α_i , we use a process similar to Eq. 7: we compute a second query vector $\mathbf{z}_{m_{\text{ans}}}$ to score

the entities inside the tail set b_j , and the weights α_i are the softmax of the inner products between the query vector $\mathbf{z}_{m_{\text{ans}}}$ and the embeddings of entities in the tail set b_j .

$$\mathbf{z}_{m_{\text{ans}}} = \mathbf{W}_b^T [\mathbf{h}_{s_{\text{ans}}}^{(T)}; \mathbf{h}_{t_{\text{ans}}}^{(T)}] \quad (8)$$

$$\alpha_i = \frac{\exp(\mathbf{o}_i^T \mathbf{z}_{m_{\text{ans}}})}{\sum_{o_l \in b_j} \exp(\mathbf{o}_l^T \mathbf{z}_{m_{\text{ans}}})} \quad (9)$$

where $\mathbf{W}_b$ is a transformation matrix distinct from $\mathbf{W}_e$ in Eq. 4 and $\mathbf{W}_f$ in Eq. 7.

The top k tail sets $\mathbf{b}_j$ are further aggregated using weights β_j , which are the softmax of the retrieval (inner product) scores of the top k head pairs a_j . This leads to a single vector $\mathbf{f}_{m_{\text{ans}}}$ that we call the *knowledge embedding* for the masked mention m_{ans} .

$$\mathbf{f}_{m_{\text{ans}}} = \sum_{j \in \text{TOP}_k(\mathbf{v}_{m_{\text{ans}}}, \mathbf{A})} \beta_j \mathbf{b}_j \quad (10)$$

$$\beta_j = \frac{\exp(\mathbf{a}_j^T \mathbf{v}_{m_{\text{ans}}})}{\sum_{t \in \text{TOP}_k(\mathbf{v}_{m_{\text{ans}}}, \mathbf{A})} \exp(\mathbf{a}_t^T \mathbf{v}_{m_{\text{ans}}})} \quad (11)$$

Intuitively $\mathbf{f}_{m_{\text{ans}}}$ is the result of retrieving a set of entities from the fact memory. The last step is to integrate this retrieved set into the Transformer's contextual embeddings. Of course, KBs are often incomplete, and especially during pre-training, it might be necessary for the model to ignore the result of retrieval, if no suitable triple appears in the KB.

To model this, the final step in the integration process is to construct an integrated query $\mathbf{q}_{m_{\text{ans}}}$ with a learnable mixing weight λ .

Algorithmically, λ is computed as the probability of retrieving a special “null” head a_{null} from the fact memory, i.e. whether an oracle head pair exists in the knowledge base. $\mathbf{q}_{m_{\text{ans}}}$ is used to predict the masked entity.

$$\mathbf{q}_{m_{\text{ans}}} = \lambda \cdot \mathbf{c}_{m_{\text{ans}}} + (1 - \lambda) \cdot \mathbf{f}_{m_{\text{ans}}}, \quad \lambda = P(a_{\text{null}})$$

$$\hat{e}_{\text{ans}} = \operatorname{argmax}_{e_i \in \mathcal{E}} (\mathbf{q}_{m_{\text{ans}}}^T \mathbf{e}_i)$$

$$\text{loss}_{\text{ans}} = \text{cross_entropy}(\text{softmax}(\mathbf{q}_{m_{\text{ans}}}, \mathbf{E}), \mathbb{I}_{e_{\text{ans}}})$$

The final loss is the sum of the individual losses.

$$\text{loss}_{\text{final}} = \text{loss}_{\text{ent}} + \text{loss}_{\text{ctx}} + \text{loss}_{\text{fact}} + \text{loss}_{\text{ans}}$$

5.4.4. Adapting FILM to Virtual KB

Now we discuss a few changes we made to adapt FILM to the virtual KB built by OPQL. We call this model OPQL-LM. There are two main changes: (1) query embeddings are constructed compositionally from a mention embedding of the topic entity and the relation embedding computed from text, (2) when integrating the retrieved knowledge with the LM context, the null fact in OPQL-LM's virtual KB also depends on e_1 .

Query Embedding The query $\mathbf{q}_{X,Y}$ is computed by concatenating the contextual mention embedding $\mathbf{m}_{e_1}$ ⁸ of the topic mention m_{e_1} (as described by entity linking in Eq. 4) and the relation embedding $\mathbf{r}_{X,Y}$. The relation embedding $\mathbf{r}_{X,Y}$ comes from the relation encoder in Eq. 3 that operates on special tokens [R1] and [R2].⁹

$$\mathbf{q}_{X,Y} = \mathbf{W}_q^T [\mathbf{m}_{e_1}; \mathbf{W}_t^T \mathbf{r}_{X,Y}]$$

Mixing with LM Similar to FILM, the retrieval result is mixed with the LM by introducing a null entry in the virtual KB. However, instead of directly learning the key embedding of the null entry $\mathbf{k}_{\text{null}}$, OPQL-LM introduces a null relation to the virtual KB, whose embedding $\mathbf{r}_{\text{null}}$ is a learned variable, and constructed a null entry with key embedding $\mathbf{k}_{\text{null}}$ and value embedding $\mathbf{v}_{\text{null}} = \vec{\mathbf{0}}$. The query is encouraged to retrieve the null entry if no relevant pair of (e_1, e_2) exists in the virtual KB. We re-use the memory key projection matrix $\mathbf{W}_k$ to compute the key embedding $\mathbf{k}_{\text{null}}$.

$$\mathbf{k}_{\text{null}} = \mathbf{W}_k^T [\mathbf{m}_{e_1}; \mathbf{r}_{\text{null}}]$$

Multi-hop OPQL-LM Some open-domain questions require multi-hop reasoning to find the answers, e.g. “Where is the author of *On the Origin of Species* educated?”. To answer such questions, the model should first find the answer of the initial-hop of the question, and use the result as the topic entity to answer the second-hop of the question. Models like FILM and OPQL-LM can naturally solve this problem by repeating the retrieval and mixing steps, but to date experiments with multi-hop questions have only been performed with OPQL-LM.

Let $\mathbf{m}'_{e_2}{}^{(t)}$ be the mixed contextual embedding of the t 'th hop. The query vector $\mathbf{q}_{X,Y}^{(t+1)}$ at the $t+1$ hop will be computed from $\mathbf{m}'_{e_2}{}^{(t)}$. Specifically, at the $t+1$ hop, $\mathbf{m}'_{e_2}{}^{(t)}$ becomes the embedding of the topic entities. The relation embedding $\mathbf{r}_{X,Y}$ is kept unchanged, but the model learns another projection matrix $\mathbf{W}_t^{(t+1)}$. The query vector $\mathbf{q}_{X,Y}^{(t+1)}$ will then be used to retrieve relevant entries from the virtual KB and to compute the mixed contextual embedding for the next step.

$$\mathbf{q}_{X,Y}^{(t+1)} = \mathbf{W}_q^T [\mathbf{m}'_{e_2}{}^{(t)}; \mathbf{W}_t^{(t+1)T} \mathbf{r}_{X,Y}]$$

5.4.5. Results for FILM and OPQL+LM

We first evaluate FILM on the LAMA TREx [31] dataset. LAMA TREx contains a set of fact-related cloze questions. Pretrained models are evaluated on LAMA without finetuning to test models' ability to memorize common facts. FILM outperforms the previous state-of-the-art [38] by 5.6 points.

⁸ $\mathbf{m}_{e_1}$ is the same as $\mathbf{h}_{m_i}$ in Eq. 4 in FILM. Here, we use the notation $\mathbf{m}_{e_1}$ for OPQL modules.

⁹Since the mention of entity e_1 is not masked from the input, the relation embedding $\mathbf{r}_{X,Y}$ may contain some leaked information from the topic entity e_1 . One could encode the relation embedding $\mathbf{r}_{X,Y}$ separately from an input where both e_1 and e_2 are masked. We do not take this solution as it doubles the computation cost of the expensive Transformer layers.

We also experiment with FILM on two QA datasets: WebQuestionsSP and TriviaQA [39]. FILM achieves the state-of-the-art in the closed-book setting, outperforming the baseline models by 15.7 and 4.1 points.¹⁰

As pointed out in [40], there is a significant train-test overlap in these datasets. After removing the overlap from training, the performance of a conventional pretrained LM (BART [41]) drops from 36.7 to 8.3 and from 30.6 to 1.0 on the two datasets respectively, while the performance of FILM is more consistent (dropping from 78.1 to 72.2 on WebQuestionsSP and from 37.3 to 28.4 on TriviaQA).

More interestingly, we experimented with FILM in a novel setting, where facts not seen at pretraining and finetuning time are added to the memory only at test time. This tests the model’s ability to, like a symbolic KB, declaratively add to a store of knowledge, by simply adding a new fact, rather than by retraining a model. Practically, updating a system by modifying the external memory without having to retrain the LM saves a significant amount of resources. In the experiment, the performance of the baseline models drops by 17.2 points with the facts required to answer the questions removed from training. The performance of FILM also drops by 17.8 points if the facts are removed, but adding the facts back only at test time can recover most of the performance (9.3 points).

With the virtual KB, OPQL-LM achieves the state-of-the-art performance on the WebQuestionsSP and multi-hop version of WebQuestionsSP called Complex WebQuestions [42]. (The new state-of-the-art results holds only for prior systems which use text, rather than a KB).

5.5. Conclusions

The computational limits of neural networks suggest that neuro-symbolic methods should be based on carefully chosen symbolic frameworks. Here we have presented a series of results that rely on a particular neuro-symbolic KB reasoning system based on set-oriented dataflow query languages. Although these languages are less expressive than even simple first-order reasoning systems, they are efficient, expressive, easily modified to be “soft”, and they are well-matched to the sorts of knowledge-based reasoning used in modern KBQA systems. Variants of these query languages can compactly express a wide collection of sets using a compact data structure consisting of a point in neural embedding space and a sketch, leading to reasoning methods that can faithfully execute logical queries, but which also generalize well—leading to sometimes dramatic improvements in accuracy of reasoning on synthetic queries, and strong improvements in KBQA performance. This approach can also be easily adapted to encode “virtual KBs” derived from entity-linked text, and importantly, this approach can be used to construct a virtual KB from entity-linked text alone, improving over earlier methods like DrKIT that require a partial KB.

Finally we show that this reasoning can be closely integrated into modern neural language models (LMs), by extending these LMs to include neuro-symbolic reasoning steps as well as the usual Transformer-based manipulations. Since pre-trained LMs combined with fine-tuning are the basis of the vast majority of modern NLP systems, this provides a method for injecting symbolic knowledge into a wide range of neural NLP systems.

¹⁰Numbers are reported on a subset of questions whose answers are entities in Wikidata.

Future work in this area might include more efficient ways of incorporating into LMs the large neuro-symbolic memories used to store knowledge, and better ways of mining from text the knowledge most relevant for question-answering tasks.

References

- [1] Gottlob G, Papadimitriou C. On the complexity of single-rule datalog queries. *Information and Computation*. 2003;183(1):104–122.
- [2] Towell GG, Shavlik JW. Knowledge-based artificial neural networks. *Artificial intelligence*. 1994;70(1-2):119–165.
- [3] Hu Z, Ma X, Liu Z, et al. Harnessing deep neural networks with logic rules. *arXiv preprint arXiv:160306318*. 2016;.
- [4] Rocktäschel T, Singh S, Riedel S. Injecting logical background knowledge into embeddings for relation extraction. In: *Proc. of ACL/HLT*; 2015.
- [5] Richardson M, Domingos P. Markov logic networks. *Mach Learn*. 2006;62(1-2):107–136.
- [6] Kimmig A, Mihalkova L, Getoor L. Lifted graphical models: a survey. *Machine Learning*. 2015;99(1):1–45.
- [7] Cohen W, Yang F, Mazaitis KR. Tensorlog: A probabilistic database implemented using deep-learning infrastructure. *Journal of Artificial Intelligence Research*. 2020;67:285–325.
- [8] Ramakrishnan R, Ullman JD. A survey of deductive database systems. *The journal of logic programming*. 1995;23(2):125–149.
- [9] Suciu D, Olteanu D, Ré C, et al. Probabilistic databases. *Synthesis Lectures on Data Management*. 2011; 3(2):1–180.
- [10] Cussens J. Parameter estimation in stochastic logic programs. *Machine Learning*. 2001;44(3):245–271.
- [11] Goodman N, Mansinghka V, Roy DM, et al. Church: a language for generative models. *arXiv preprint arXiv:12063255*. 2012;.
- [12] Kschischang FR, Frey BJ, Loeliger HA. Factor graphs and the sum-product algorithm. *Information Theory, IEEE Transactions on*. 2001;47(2):498–519.
- [13] Hamilton WL, Bajaj P, Zitnik M, et al. Embedding logical queries on knowledge graphs. *arXiv preprint arXiv:180601445*. 2018;.
- [14] Arakelyan E, Daza D, Minervini P, et al. Complex query answering with neural link predictors. *arXiv preprint arXiv:201103459*. 2020;.
- [15] Yih Wt, Richardson M, Meek C, et al. The value of semantic parse labeling for knowledge base question answering. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*; 2016. p. 201–206.
- [16] Bollacker K, Evans C, Paritosh P, et al. Freebase: a collaboratively created graph database for structuring human knowledge. In: *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*; 2008. p. 1247–1250.
- [17] Cohen WW, Sun H, Hofer RA, et al. Scalable neural methods for reasoning with a symbolic knowledge base. In: *International Conference on Learning Representations*; 2020. Available from: <https://openreview.net/forum?id=BJlguT4YPr>.
- [18] Musmann S, Ermon S. Learning and inference via maximum inner product search. In: Balcan MF, Weinberger KQ, editors. *Proceedings of The 33rd International Conference on Machine Learning*; (Proceedings of Machine Learning Research; Vol. 48); 20–22 Jun; New York, New York, USA. PMLR; 2016. p. 2587–2596. Available from: <http://proceedings.mlr.press/v48/musmann16.html>.
- [19] Sun H, Arnold AO, Bedrax-Weiss T, et al. Faithful embeddings for knowledge base queries. *Advances in Neural Information Processing Systems*. 2020;33.
- [20] Ren H, Hu W, Leskovec J. Query2box: Reasoning over knowledge graphs in vector space using box embeddings ; 2020.
- [21] Zhang Y, Dai H, Kozareva Z, et al. Variational reasoning for question answering with knowledge graph. In: *Thirty-Second AAAI Conference on Artificial Intelligence*; 2018.
- [22] Sun H, Dhingra B, Zaheer M, et al. Open domain question answering using early fusion of knowledge bases and text ; 2018.
- [23] Sun H, Bedrax-Weiss T, Cohen WW. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text ; 2019.

- [24] Angeli G, Premkumar MJJ, Manning CD. Leveraging linguistic structure for open domain information extraction. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers); 2015. p. 344–354.
- [25] Mitchell T, Cohen W, Hruschka E, et al. Never-ending learning. In: Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence (AAAI-15); 2015.
- [26] Dhingra B, Zaheer M, Balachandran V, et al. Differentiable reasoning over a virtual knowledge base. In: International Conference on Learning Representations; 2020. Available from: <https://openreview.net/forum?id=SJxstlHFPH>.
- [27] Peters M, Neumann M, Iyyer M, et al. Deep contextualized word representations. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers); 2018. p. 2227–2237.
- [28] Devlin J, Chang MW, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers); 2019. p. 4171–4186.
- [29] Raffel C, Shazeer N, Roberts A, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. arXiv preprint arXiv:1910.10683. 2019;.
- [30] Tenney I, Das D, Pavlick E. Bert rediscovers the classical nlp pipeline. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics; 2019. p. 4593–4601.
- [31] Petroni F, Rocktäschel T, Lewis P, et al. Language models as knowledge bases? arXiv preprint arXiv:1909.01066. 2019;.
- [32] Roberts A, Raffel C, Shazeer N. How much knowledge can you pack into the parameters of a language model? arXiv preprint arXiv:2002.08910. 2020;.
- [33] Brown TB, Mann B, Ryder N, et al. Language models are few-shot learners. arXiv preprint arXiv:2005.14165. 2020;.
- [34] Poerner N, Waltinger U, Schütze H. Bert is not a knowledge base (yet): Factual knowledge vs. name-based reasoning in unsupervised qa. arXiv preprint arXiv:1911.03681. 2019;.
- [35] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: Advances in neural information processing systems; 2017. p. 5998–6008.
- [36] Verga P, Sun H, Soares LB, et al. Adaptable and interpretable neural memory over symbolic knowledge. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies; 2021. p. 3678–3691.
- [37] Sun H, Verga P, Dhingra B, et al. Reasoning over virtual knowledge bases with open predicate relations ; 2021.
- [38] Févry T, Soares LB, FitzGerald N, et al. Entities as experts: Sparse memory access with entity supervision ; 2020.
- [39] Joshi M, Choi E, Weld DS, et al. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics; July; Vancouver, Canada. Association for Computational Linguistics; 2017.
- [40] Lewis P, Stenetorp P, Riedel S. Question and answer test-train overlap in open-domain question answering datasets ; 2020.
- [41] Lewis M, Liu Y, Goyal N, et al. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension ; 2019.
- [42] Talmor A, Berant J. The web as a knowledge-base for answering complex questions. In: North American Association for Computational Linguistics (NAACL); 2018.

Chapter 6

Tractable Boolean and Arithmetic Circuits

Adnan Darwiche, Computer Science Department, UCLA

Tractable Boolean and arithmetic circuits have been studied extensively in AI for over two decades now. These circuits were initially proposed as “compiled objects,” meant to facilitate logical and probabilistic reasoning, as they permit various types of inference to be performed in linear-time and a feed-forward fashion like neural networks. In more recent years, the role of tractable circuits has significantly expanded as they became a computational and semantical backbone for some approaches that aim to integrate knowledge, reasoning and learning. In this chapter, we review the foundations of tractable circuits and some associated milestones, while focusing on their core properties and techniques that make them particularly useful for the broad aims of neuro-symbolic AI.

6.1. Introduction

Tractable circuits, both Boolean and arithmetic, have been receiving an increased attention in AI and computer science more broadly. These circuits represent Boolean and real-valued functions, respectively. They are called *tractable* because they allow one to answer some hard queries about these functions in polytime, typically through a linear-time, feed-forward pass through the circuit structure. The foundations of these circuits have been developed within the area of *knowledge compilation*, which aims to compile knowledge into tractable representations for the goal of facilitating efficient reasoning. This area of research has a long tradition in AI; see, e.g., [1] and [2, 3, 4, 5, 6, 7, 8]. A turning point, however, has been the work of [4] which presented a comprehensive theory of knowledge compilation based on *tractable Boolean circuits*, which are *deep* representations, in contrast to earlier efforts which focused on *flat* representations based on conjunctive and disjunctive normal forms; see, e.g., [2, 3]. Another turning point has been the work of [9, 10] which employed tractable Boolean circuits in probabilistic reasoning, giving birth to an extensive line of work on *tractable arithmetic circuits*. The literature on tractable circuits has enjoyed a number of additional and exciting developments over the years, which have broadened and extended their applications from reasoning, to learning and more recently to neuro-symbolic AI; an area that is concerned with integrating neural and symbolic approaches to AI [11, 12, 13]. These developments have also raised key questions, and in some cases confusions, particularly on tractable arithmetic circuits

and their relationship to tractable Boolean circuits and the now prevalent technique of *weighted model counting* [14]. The goal of this chapter is to discuss the foundations of tractable Boolean and arithmetic circuits, to clarify their relationships, and to highlight some of the key developments that have contributed to the growing interest surrounding tractable circuits today. But first, we find it pertinent to further elaborate on how and why the role of tractable circuits has evolved over the years.

The original motivation behind knowledge compilation and tractable circuits was based on the following observation. In many reasoning tasks, one is typically interested in posing a large number of queries so the (high) cost of *offline compilation* can be amortized over the large number of *online queries*. Today, however, tractable circuits are playing a significantly broader role for a number of reasons. First, these circuits have been providing a systematic methodology for computation, particularly for problems beyond NP which include important tasks in probabilistic reasoning and machine learning; see [8] for a recent survey. Second, reasoning with tractable circuits is not only efficient but can almost always be conducted using linear-time algorithms that traverse these circuits in a feed-forward fashion like neural networks. Moreover, tractable circuits are *differentiable*. In fact, backpropagation has been performed on these circuits, both Boolean [15] and arithmetic [10], as early as two decades ago when the derivatives were first employed in reasoning tasks. These properties of tractable circuits made them very suitable for integration with modern pipelines for machine learning and neuro-symbolic AI; see, for example, [16, 17, 18, 19, 20, 21, 22, 23, 24, 25] where tractable circuits have been recently employed in and/or integrated with neural networks, deep reinforcement learning, Bayesian network classifiers and (deep) probabilistic logic programs.¹ Even the traditional offline/online divide that originally motivated knowledge compilation for reasoning is now being exploited in modern settings as it is aligned with the training/inference divide that governs modern AI systems; see, e.g., [16]. As such, a recent trend has emerged in which tools and techniques that were initially envisioned for reasoning tasks are now being employed to facilitate learning and its integration with knowledge and reasoning. A few additional milestones have contributed to broadening the applications of tractable circuits and their expanded role today. First is the *learning* of tractable arithmetic circuits from data, starting with [40]; see also [41]. Second is *handcrafting* the structure of these circuits, which started with [42]. These developments have significantly expanded the utility of tractable circuits as they provided other modes of usage, beyond compilation from models. They also triggered modern treatments of the theory of tractable arithmetic circuits that are independent of compilation, starting with [43] that we shall discuss later. Another milestone is the integration of tractable Boolean and arithmetic circuits, starting with [44], which provided a profound, new formalism for integrating symbolic knowledge into circuits that perform probabilistic reasoning.

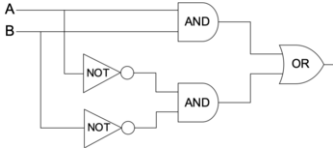
This chapter is organized as follows. We first treat tractable Boolean circuits in Section 6.2, followed by tractable arithmetic circuits in Section 6.3. We then discuss algo-

¹Tractable circuits have also attracted interest in areas of computer science such as database theory; see, e.g., [26, 27], and benefited from areas of computer theory such as communication complexity; see, e.g., [28]. They also provided tractable approaches for constrained sampling [29], explainable AI [30, 31, 32, 33, 34, 35, 36, 37, 8] and attracted interest in design tasks such as Boolean functional synthesis [38, 39]. We also remark that (non-tractable) Boolean and arithmetic circuits have their own branches of computational complexity theory: circuit complexity and algebraic complexity, respectively. The theory of tractable circuits has a different focus though compared to what is commonly investigated in these areas.

rithms that compile knowledge into tractable circuits in Section 6.4 and close with some remarks in Section 6.5. We do not treat the rich subjects of handcrafting and learning (the structure of) tractable arithmetic circuits as this is beyond the scope of this chapter.²

6.2. Tractable Boolean Circuits

The theory of tractable Boolean circuits is based on *Negation Normal Form (NNF)* circuits. These are Boolean circuits that have three types of gates: and-gates, or-gates and inverters, except that inverters can only feed from the circuit variables; see figure on the right. NNF circuits are not tractable.



However, by imposing certain properties on them we can attain different degrees of tractability. A comprehensive, but now incomplete, treatment of tractable NNF circuits was given in [45], in which these circuits were studied across the two dimensions of *tractability* and *succinctness*. As we increase the strength of properties imposed on NNF circuits, their tractability increases by allowing more queries to be performed on them in polytime. This typically comes at the expense of succinctness as the size of circuits gets larger. We will next review some classes of tractable Boolean circuits, with increasingly stronger properties, which will allow us to efficiently solve decision problems (and their functional variants) that are complete for the complexity classes $NP \subseteq PP \subseteq NP^{PP} \subseteq PP^{PP}$; see also [8]. The algorithms for these increasingly complex problems all take time linear in the circuit size,³ and operate by traversing the circuit in a feed-forward fashion like neural networks. In the following discussion, we will omit inverters from NNF circuits and use $\neg X$ instead to represent an inverted variable X .

Decomposability One of the simplest properties that make NNF circuits tractable is *decomposability* [46]. According to this property, circuit fragments that feed into an and-gate cannot share variables. Figure 1 illustrates this property by highlighting two fragments (shaded) that feed into an and-gate. The fragment on the left feeds from variables K and L . The one on the right feeds from variables A and P . NNF circuits that satisfy the decomposability property are known as *DNNF circuits*. SAT, an NP-complete problem [47], can be decided in linear time on DNNF circuits [46]. In this context, SAT is the problem of deciding whether the circuit has a satisfying input (generates output 1).

Determinism The next property we consider is *determinism* [15], which applies to or-gates in an NNF circuit. According to this property, at most one input for an or-gate can be high under any circuit input. Figure 1 illustrates this property when all circuit variables A, K, L, P are high. Examining the or-gates in this circuit, under this circuit input, one sees that each or-gate has either one high input or no high inputs. This property corresponds to mutual exclusiveness when an or-gate is viewed as a disjunction of its inputs. NNF circuits that are decomposable and deterministic are known as *d-DNNF circuits* [15] and

²Two video tutorials complement the treatment in this chapter: “Beyond NP with Tractable Circuits,” <https://www.youtube.com/watch?v=kdMzmgyLfQs> and “Three Modern Roles for Logic in AI,” <https://www.youtube.com/watch?v=3PrYYLppjXA>.

³The size of a circuit is defined as the number of its edges.

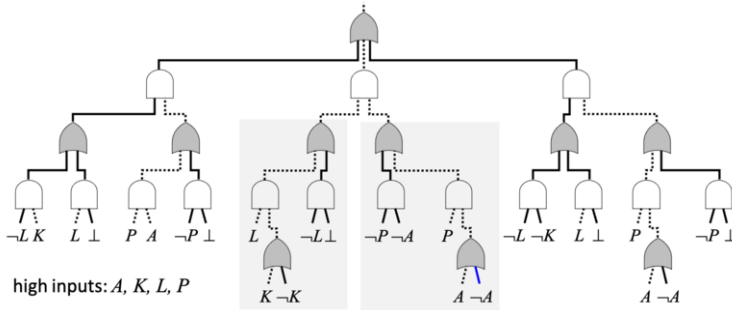
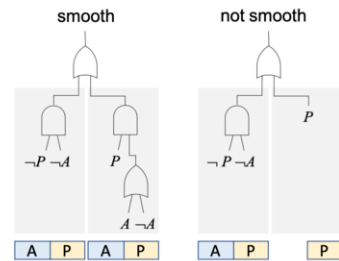


Figure 1. Illustrating the decomposability and determinism properties of NNF circuits. Dotted lines are high wires. Solid lines are low wires. The illustration does not tie shared inputs of the circuit for clarity of exposition.

they are exponentially less succinct than DNNF circuits [28].⁴ The PP-complete problem MAJSAT [48] can be decided in polytime on d-DNNF circuits. In this context, MAJSAT is the problem of deciding whether the majority of circuit inputs satisfy the circuit. If d-DNNF circuits are also *smooth* [15], a property that can be enforced in quadratic time, these circuits allow one to perform #SAT [49] in linear time; that is, counting the number of satisfying circuit inputs, also known as *model counting*. If one assigns a weight to each variable value, one can define a weight for each circuit input as the product of weights assigned to its variable values. One can then sum the weights of satisfying circuit inputs also in linear time, a problem that is known as *weighted model counting* (WMC) [14].

Smoothness This property requires all circuit fragments feeding into an or-gate to mention the same variables; see illustration on the right. Enforcing smoothness can introduce trivial gates into the circuit, such as the bottom or-gate in the illustration. One can enforce smoothness in quadratic time [15] and sometimes more efficiently [50].⁵ An example of model counting using a d-DNNF circuit is depicted in Figure 2 (left). Every literal in the circuit, whether a positive literal (A) or a negative literal ($\neg A$), is assigned the value 1. Constants $\top$ and $\perp$ are assigned the values 1 and 0, respectively. We then propagate these numbers upwards, multiplying numbers assigned to the inputs of an and-gate and summing numbers assigned to the inputs of an or-gate. The number we obtain for the circuit output is the model count. In this example, the circuit has 9 satisfying inputs out of 16 possible ones. We can also obtain model counts under variable settings in linear time. For example, if we wish to count the number of satisfying circuit inputs in which $A=1$ and $K=0$, we simply assign 0 to literals $\neg A$ and K instead of 1 and then propagate counts as just discussed. This is illustrated in Figure 2 (right), which shows that 2 out of the 9 satisfying circuit inputs have $A=1$ and



⁴That is, there are Boolean functions that can be represented using DNNF circuits of polynomial size but their d-DNNF circuits must have exponentially size.

⁵The following time-stamped video link provides an intuitive explanation of the role of smoothness in model counting: <https://youtu.be/kdMzmgyLfqs?t=1586>

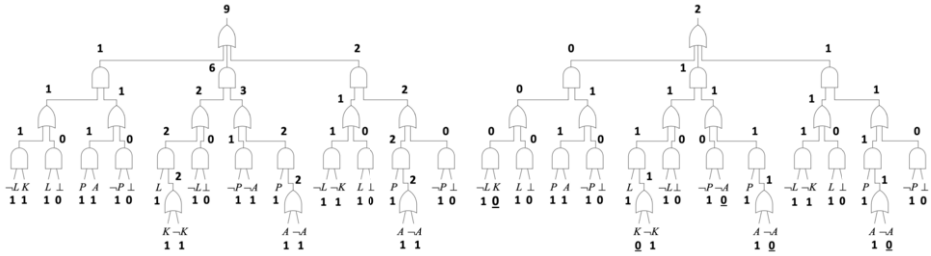
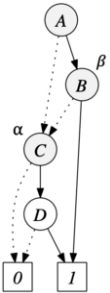


Figure 2. Model counting through linear-time circuit traversal. Left: counting the number of satisfying circuit inputs. Right: counting the number of satisfying circuit inputs with $A=1$ and $K=0$. Unsatisfiable subcircuits always generate a count of 0 even if not smooth, so we can exclude them when checking smoothness.

$K=0$. If our goal is to find the model counts under each setting of a single variable, then all such counts can be obtained using a second pass on the smooth d-DNNF circuit. This pass performs backpropagation to compute partial derivatives which can be used to obtain these counts [15]. To perform weighted model counting, we simply assign a weight to a literal instead of the value 1 and propagate the counts as usual. Model counting is a special case of weighted model counting when the weight of each literal is 1.

Decision There are stronger versions of decomposability and determinism which give rise to additional, tractable NNF circuits. A stronger version of determinism is known as the *decision property*. It requires each or-gate g to have the form $g = (X \wedge \alpha) \vee (\neg X \wedge \beta)$, where X is a circuit variable known as the *decision variable* of gate g (this or-gate will then satisfy the determinism property). NNF circuits that satisfy decomposability and decision are known as *Decision-DNNF* circuits [51] and they are exponentially less succinct than d-DNNF circuits [15, 52]. Suppose we split the circuit variables into $\mathbf{X}$ and $\mathbf{Y}$. We will say that the Decision-DNNF is $\mathbf{X}$ -constrained iff no or-gate with a decision variable in $\mathbf{X}$ can appear below an or-gate with a decision variable in $\mathbf{Y}$ [53]. These circuits allow us to solve E-MAJSAT and its functional variant in time linear in the circuit size. E-MAJSAT is a decision problem that is NP^{PP} -complete [54]. It asks: is there an instantiation $\mathbf{x}$ under which the majority of instantiations $\mathbf{y}$ yield a satisfying circuit input $\mathbf{xy}$? The functional version of E-MAJSAT includes the computation of most likely partial instantiations in probabilistic reasoning [55, 53]. The corresponding feed-forward algorithm performs summation at or-gates with decision variables in $\mathbf{Y}$, maximization at or-gates with decision variables in $\mathbf{X}$, and multiplication at and-gates [53].

Ordered Binary Decision Diagrams (OBDDs) [56] are perhaps one of the most studied, tractable representations of Boolean functions. OBDDs are a special case of Decision-DNNF circuits even though they are notated differently as shown on the right. Each internal node in an OBDD is labeled with a variable and has two outgoing edges: a low edge (usually dotted) and a high edge (usually solid). The leaf nodes of an OBDD are labeled with 0 ($\perp$) or 1 ($\top$). Variables must follow the same order on any path from the root to a leaf in an OBDD. Consider the root node in the OBDD on the right, which is labeled with variable A . This node has a low child α and a high child β . If we replace this node with the or-gate $(\neg A \wedge \alpha) \vee (A \wedge \beta)$ and repeat the same process recursively for OBDDs α and β , we obtain a Decision-DNNF circuit. That is, we obtain an NNF circuit



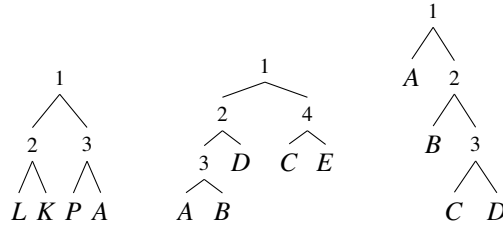


Figure 3. Three different types of vtrees: balanced, constrained for $CE|ABD$, and right-linear. A *constrained vtree* for $\mathbf{Y}|\mathbf{X}$ is a vtree over variables $\mathbf{X} \cup \mathbf{Y}$ and contains a unique node u with following properties: (1) u can be reached from the vtree root by following right children only and (2) the variables outside u are precisely $\mathbf{X}$. A *right-linear vtree* is one in which the left child of every internal node is a leaf. A right-linear vtree corresponds to a total variable order (A, B, C, D, E in this case).

that satisfies the decomposability and decision properties [4]. *Free Binary Decision Diagrams (FBDDs)* [57] generalize OBDDs by relaxing the variable ordering requirement while keeping a weaker property known as *test-once*: each variable must appear at most once on any path from the root to a leaf. FBDDs are also a special case of Decision-DNNF circuits once we adjust for notation and there is a quasipolynomial simulation of Decision-DNNF circuits by equivalent FBDDs [58]. Hence, while FBDDs are less compact than Decision-DNNF circuits, they are not exponentially less succinct. OBDDs are exponentially less succinct than FBDDs though [57] so they are also exponentially less succinct than Decision-DNNF circuits.

Structured Decomposability This is a stronger version of decomposability which is stated with respect to a full binary tree whose leaves are in one-to-one correspondence with the circuit variables [59]. Three such trees are depicted in Figure 3, which are known as *vtrees*. Structured decomposability requires each and-gate to have exactly two inputs i_1 and i_2 , and to have a corresponding node v in the vtree such that the variables of subcircuits feeding into i_1 and i_2 are in the left and right subtrees of node v . The DNNF circuit in Figure 1 is structured according to the vtree on the left of Figure 3. For example, all and-gates below the root or-gate conform to vtree node $v=1$ in Figure 3 (left).

Partitioned Determinism Structured decomposability and a stronger version of determinism, known as *partitioned determinism*, yield a class of NNF circuits known as Sentential Decision Diagrams (SDDs) [60]. These circuits allow one to solve MAJMAJSAT and its functional variant in linear time. MAJMAJSAT is a decision problem that is PP^{PP} -complete and is also based on splitting the circuit variables into $\mathbf{X}$ and $\mathbf{Y}$. It asks: is there a majority of instantiations $\mathbf{x}$ under which the majority of instantiations $\mathbf{y}$ yield a satisfying circuit input $\mathbf{xy}$? The functional variant of MAJMAJSAT includes the computation of expectations such as [61]. To illustrate partitioned determinism, consider Figure 4 and the highlighted circuit fragment. This fragment corresponds to the Boolean expression $(p_1 \wedge s_1) \vee (p_2 \wedge s_2) \vee (p_3 \wedge s_3)$, where each p_i is called a *prime* and each s_i is called a *sub* (primes and subs correspond to subcircuits). Partitioned determinism requires that under any circuit input, precisely one prime will be high (i.e., the primes form a partition). In Figure 4, under the given circuit input, prime p_2 is high while primes p_1 and p_3 are low. This means that this circuit fragment, which acts as a multiplexer, is actually passing the value of sub s_2 while suppressing the values of subs s_1 and s_3 . As a result, the or-gate in this circuit fragment is guaranteed to be deterministic: at most one input of the or-gate

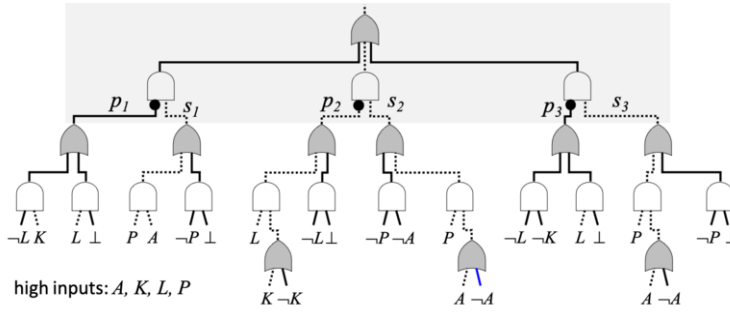


Figure 4. Illustrating the partitioned determinism property of NNF circuits. Dotted lines are high wires. Solid lines are low wires. Precisely one of p_1 , p_2 and p_3 will be high for any circuit input.

will be high under any circuit input. SDD circuits result from recursively applying this multiplexer construct to a given vtree (the SDD circuit in Figure 4 is structured with respect to the vtree on the left of Figure 3).⁶ Recall that MAJMAJSAT, the prototypical problem for the complexity class PP^{PP} [54], is stated with respect to a split of circuit variables into $\mathbf{X}$ and $\mathbf{Y}$. If the vtree is *constrained for $\mathbf{Y}|\mathbf{X}$* , then this problem and its functional variant can be solved in linear time on the corresponding SDD [62]. Figure 3 illustrates the concept of a constrained vtree. When an SDD is structured with respect to a *right-linear vtree*, the result corresponds to an OBDD after adjusting for notation as discussed earlier. Figure 3 illustrates the concept of a right-linear vtree. In this case, every circuit fragment will have the form $(X \wedge \alpha) \vee (\neg X \wedge \beta)$ where literals X and $\neg X$ are primes, and where α and β are subs. OBDDs are therefore a subset of both FBDDs and SDDs and they are exponentially less succinct than SDDs [63]. SDDs and FBDDs are not comparable though in terms of succinctness [64, 65].⁷

We have covered in this section only a subset of the tractable Boolean circuits known today, but ones that provide the basis for many of the further refinements and additions; see, e.g. [66] and [39] for some recent additions and [7] for a relatively recent tutorial that covers more circuit types and discusses further the relative succinctness of these circuits. The circuit properties we covered also form a basis for the most influential types of tractable arithmetic circuits. We shall cover these circuit types in the next section.

6.3. Tractable Arithmetic Circuits

We saw in the previous section how counting the models of a tractable Boolean circuit is done through the application of arithmetic operations at Boolean gates: additions at or-gates and multiplications at and-gates. This counting task induces an arithmetic circuit that shares the structure of underlying Boolean circuit as shown in Figure 5, therefore inheriting its properties such as decomposability, determinism and smoothness. Decom-

⁶The vtree of an SDD is ordered: the distinction between left and right children matters.

⁷SDDs can be exponentially less succinct than FBDDs for some Boolean functions [64] and FBDDs can be exponentially less succinct than SDDs for some other functions [65] so these circuit types are not comparable. Hence, SDDs and Decision-DNNFs are not comparable either since FBDDs are a subset of Decision-DNNFs and can simulate them quasipolynomially [58]. However, both SDDs and Decision-DNNFs are exponentially less succinct than d-DNNF circuits since FBDDs are exponentially less succinct than d-DNNF circuits [15].

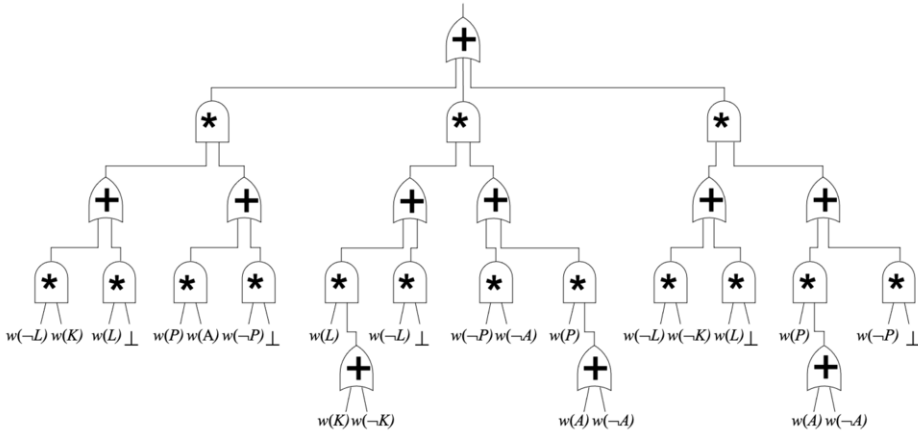


Figure 5. Computing the weighted model count of a tractable Boolean circuit, where $w(\ell)$ represents the weight of literal ℓ . This computation induces an arithmetic circuit that is superimposed on the Boolean circuit.

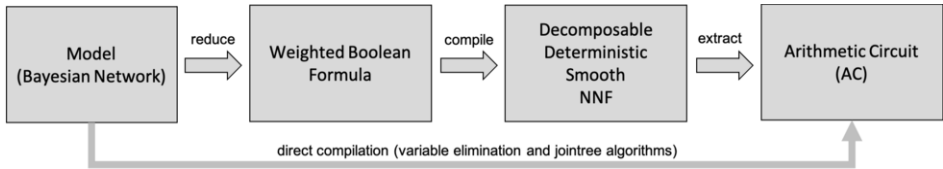


Figure 6. Constructing arithmetic circuits through a compilation process.

possibility and smoothness, being structural properties, maintain their exact definitions when moving from the Boolean to the arithmetic side. However, determinism ends up being phrased slightly differently as we shall see later.

The above connection between tractable Boolean and arithmetic circuits originated from [9, 10], which compiled Bayesian networks into tractable arithmetic circuits to enable linear-time probabilistic reasoning on the compiled circuits; see Figure 6. According to this proposal, the Bayesian network is first encoded into a Boolean formula with literal weights, allowing one to reduce probabilistic reasoning into weighted model counting. The Boolean formula is then compiled into a tractable Boolean circuit (deterministic, decomposable and smooth), from which a tractable arithmetic circuit known as an AC is finally extracted. This approach is implemented by the ACE system,⁸ which was recently evaluated in [67] and shown to exhibit state-of-the-art performance; see also [68].

A more direct and modern account of tractable arithmetic circuits is given in [43], which reconstructed the proposal in [10] so it is independent of Bayesian networks, model compilation and weighted model counting. This modern treatment was motivated by two influential lines of developments. The first line, which started with [40], utilized arithmetic circuits in the context of learning, in contrast to reasoning. The second line of developments, initiated by [42], handcrafted the structure of arithmetic circuits instead of compiling them from models and dropped the property of determinism while keeping the circuits tractable for certain computations. These developments raised key questions

⁸<http://reasoning.cs.ucla.edu/ace/>

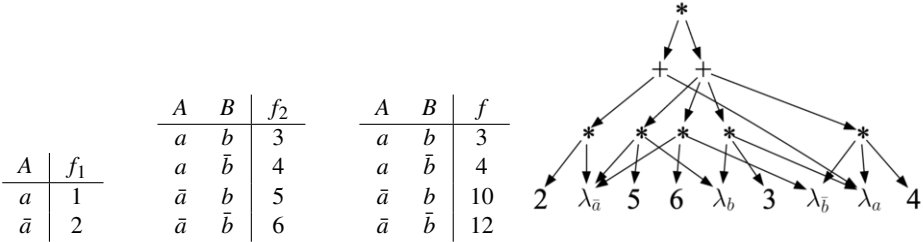


Figure 7. Factors $f_1(A)$, $f_2(A, B)$, their product factor $f(A, B) = f_1 f_2$, and an arithmetic circuit $\mathcal{A}_1$ that computes factor $f(A, B)$. The circuit indicators $(\lambda_a, \lambda_{\bar{a}}, \lambda_b, \lambda_{\bar{b}})$ correspond to the values of variables A and B . The variables of factors and circuits can be discrete in general (they are binary in this example).

but also broadened the applications of tractable arithmetic circuits significantly. We will next present the modern treatment of [43] which will allow us to also explain more recent classes of tractable arithmetic circuits, such as Sum-Product Networks (SPNs) [42] and Probabilistic Sentential Decision Diagrams (PSDDs) [44]. As we shall see, the treatment in [43] is based on a key distinction between arithmetic circuits, which can lookup values, and tractable arithmetic circuits, which can also reason. As in the Boolean case, the degree to which an arithmetic circuit is tractable and, hence, its ability to conduct various forms of reasoning, depends on the specific properties that the circuit satisfies.

The Reference Point of an Arithmetic Circuit

There is a key difference between arithmetic circuits that are compiled from models and those that are handcrafted or learned from data. The former have a clear reference point, the model, which defines the quantities that these circuits are supposed to compute. The latter circuits lack such a reference point which can cause issues when stating and evaluating claims. The theory in [43] starts by defining the reference point of an arithmetic circuit, regardless of where the circuit originates from. This reference point is the notion of a *factor*: a generalization of a Boolean function that maps *complete* variable instantiations into non-negative numbers; see Figure 7 (a distribution is one type of a factor).

An arithmetic circuit is based on a set of *discrete variables*, which define a key ingredient of the circuit: the *indicators*. For each value x of a variable X , we have an indicator λ_x . The arithmetic circuit will then have constants and indicators as its leaf nodes (inputs) with adders and multipliers as its internal nodes; see Figure 7. The factor of an arithmetic circuit (the reference point) is obtained by evaluating the circuit at complete variable instantiations. To evaluate the circuit at an instantiation $\mathbf{e}$, we replace each indicator λ_x with 1 if the value x is compatible with instantiation $\mathbf{e}$ and with 0 otherwise [10]. We then evaluate the circuit bottom-up in the standard way. The factor $f(A, B)$ in Figure 7 has four rows which correspond to the four instantiations of variables A and B . Evaluating the circuit in this figure at each of these complete instantiations yields a value for each instantiation and therefore defines the reference factor. We say in this case that the arithmetic circuit *computes* this factor. We also say that this circuit can *lookup* the values of this factor. For example, in Figure 8 (left), the circuit evaluates to 12 under the complete variable instantiation $A = \bar{a}, B = \bar{b}$ by setting the indicators to $\lambda_a = 0, \lambda_{\bar{a}} = 1, \lambda_b = 0, \lambda_{\bar{b}} = 1$. An arithmetic circuit can be evaluated at a *partial* variable instantiation using the same procedure, but the value returned may not be meaningful

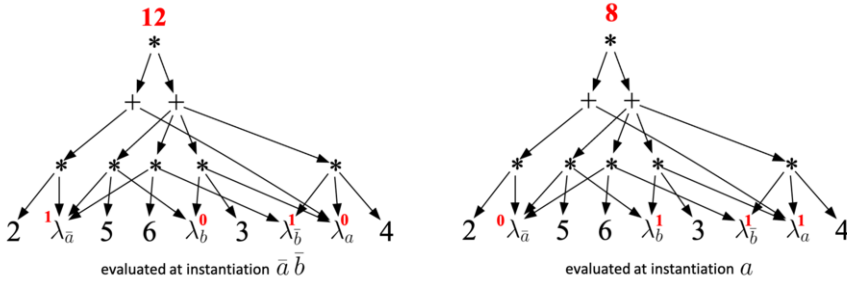


Figure 8. An arithmetic circuit $\mathcal{A}_1$ evaluated at complete variable instantiation $A=\bar{a}, B=\bar{b}$ and partial variable instantiation $A=a$. This circuit computes factor $f(A, B)$ in Figure 7: it can lookup the values of factor f . This circuit is **not decomposable** but is **smooth** and **deterministic**.

unless the circuit is tractable (i.e., unless the circuit satisfies certain properties). For example, Figure 8 (right) evaluates the circuit at partial instantiation $A=a$ by setting the indicators to $\lambda_a=1, \lambda_{\bar{a}}=0, \lambda_b=1, \lambda_{\bar{b}}=1$, leading to a value of 8. However, this value is not meaningful since the circuit is not tractable so it cannot reason about the factor. This is a subject that we shall discuss in depth next.

Circuits that Lookup Values Versus Circuits that Reason

The central question posed and treated in [43] is the following. Suppose we have an arithmetic circuit that *computes* a factor (i.e., looks up its values). Can this circuit *reason* about the factor and why? Constructing an arithmetic circuit that computes a factor can be done efficiently even when the factor is defined implicitly as a product of other factors (e.g., when the factor is defined by a probabilistic graphical model). Consider the factors and arithmetic circuit in Figure 7. Factor $f(A, B)$ is the product of factors $f_1(A)$ and $f_2(A, B)$. The arithmetic circuit computes factor $f(A, B)$ and is obtained by multiplying circuits $\lambda_a + 2\lambda_{\bar{a}}$ and $3\lambda_a\lambda_b + 4\lambda_a\lambda_{\bar{b}} + 5\lambda_{\bar{a}}\lambda_b + 6\lambda_{\bar{a}}\lambda_{\bar{b}}$, which compute factors $f_1(A)$ and $f_2(A, B)$, respectively. This can always be done and the size of resulting circuit is linear in the size of multiplied factors, not the size of their product which can be exponential.⁹

The interest, however, is in *tractable* arithmetic circuits that can reason about a factor, not ones that can only look up its values. One fundamental reasoning task is that of computing the value of a partial variable instantiation, known as the *marginals (MAR)* problem [69]. Another fundamental reasoning task is that of identifying and computing the value of a *most likely, complete variable instantiation*, known as the MPE problem [69]. For factors that are induced by models such as Bayesian networks, the decision

⁹To see this, define the depth-two arithmetic circuit of factor $f_i(\mathbf{X})$ as follows: $\sum_{\mathbf{x}} f_i(\mathbf{x}) \prod_{x \sim \mathbf{x}} \lambda_x$, where $x \sim \mathbf{x}$ means that value x of variable $X \in \mathbf{X}$ is compatible with instantiation $\mathbf{x}$ of variables $\mathbf{X}$. This arithmetic circuit has one layer of multipliers followed by a layer with one adder, and computes factor $f_i(\mathbf{X})$. If a factor f is defined as the product of factors $f_1, \dots, f_n$, then multiplying the depth-two circuits of factors f_i yields a circuit that computes their product factor $f = f_1 \dots f_n$; see [43] for details.

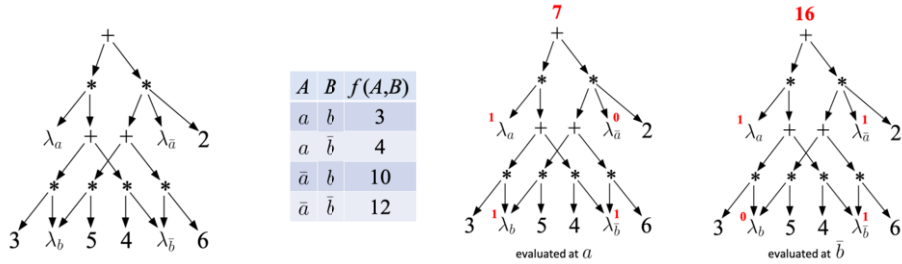


Figure 9. An arithmetic circuit $\mathcal{C}_2$, the factor it computes, and evaluations of the circuit under the partial variable instantiations $A=a$ and $B=\bar{b}$. This arithmetic circuit computes the factor marginals.

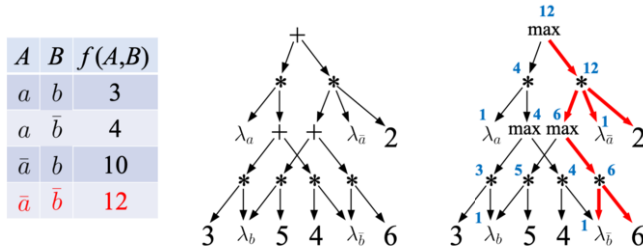


Figure 10. A factor, an arithmetic circuit $\mathcal{C}_2$ that computes this factor, and the corresponding maximizer circuit $\mathcal{M}_2$. This maximizer circuit computes the factor MPEs.

variant of MPE is NP-complete [70], the decision variant of MAR is PP-complete and its functional variant is #P-complete [71], making these hard reasoning tasks.¹⁰

Consider again the factor $f(A,B)$ in Figure 7 and the partial instantiation $A=a$. The marginal for this instantiation, $f(a)$, is the sum of values assigned to rows that are compatible with instantiation $A=a$: $f(a) = f(a,b) + f(a,\bar{b}) = 3 + 4 = 7$. This computation is quite fundamental as it corresponds to computing marginal probabilities when the factor represents a distribution. Interestingly enough, while the arithmetic circuit $\mathcal{C}_1$ of Figure 7 does compute the factor, it does not compute its marginals. That is, it can lookup values of rows but cannot sum them up (i.e., cannot reason). A counterexample is shown on the right of Figure 8, where the circuit evaluates to 8 instead of 7 at instantiation $A=a$.

Figure 9 depicts another arithmetic circuit, $\mathcal{C}_2$, which computes the same factor computed by $\mathcal{C}_1$ of Figure 7. Unlike $\mathcal{C}_1$ though, $\mathcal{C}_2$ does compute the factor marginals. Figure 9 depicts example evaluations of $\mathcal{C}_2$ to be contrasted with the evaluations of $\mathcal{C}_1$ in Figure 8. The question now is: Why did $\mathcal{C}_2$ compute marginals, and hence reason about the factor, while $\mathcal{C}_1$ could not? Before we answer this question, let us first examine another example of reasoning about a factor: computing the value and identity of a most likely, complete variable instantiation (MPE).

Consider Figure 10 which depicts $\mathcal{C}_2$ again and the factor it computes. The figure shows another circuit, $\mathcal{M}_2$, obtained by replacing the adders of $\mathcal{C}_2$ with maximizers.

¹⁰MPE stands for *Most Likely Explanation*. A related problem is that of identifying and computing the value of a most likely instantiation of a partial set of variables, which is known as the MAP problem [69]. The decision variant of this problem is NP^{PP}-complete [55]; see also [72]. MAP stands for *Maximum a Posteriori Hypothesis*. Sometimes, MAP and partial MAP are used instead of MPE and MAP to denote these problems.

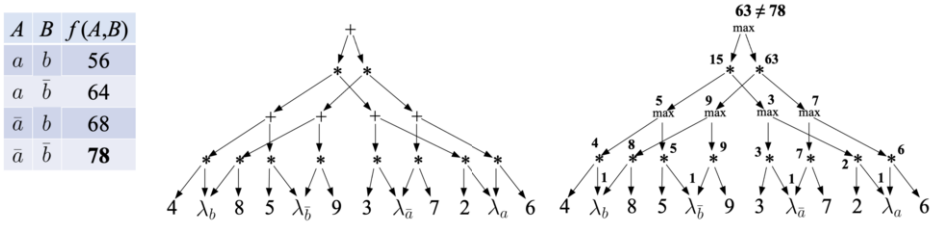


Figure 11. A factor, an arithmetic circuit $\mathcal{AC}_3$ that computes the factor marginals, and a maximizer circuit $\mathcal{MC}_3$ that does not compute the factor MPEs.

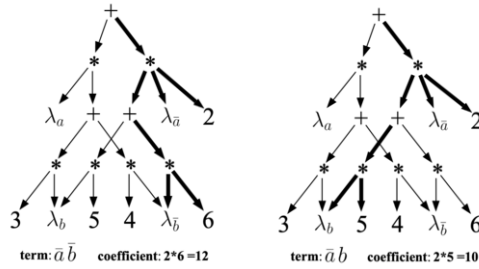


Figure 12. Two complete subcircuits (highlighted edges) with their terms and coefficients.

This is called a *maximizer arithmetic circuit*, introduced in [73]. If we evaluate this circuit at the empty variable instantiation by setting all indicators to 1, we obtain a value of 12 which is the maximal value attained by any row of the factor; see the right of Figure 10. If we further evaluate this maximizer circuit at instantiation $B=b$, we obtain a value of 10 which is the maximal value attained by any row of the factor that is compatible with instantiation $B=b$. One can verify that this maximizer circuit will compute the factor MPEs correctly for any variable instantiation, whether complete or partial.

Consider now Figure 11 which depicts a third arithmetic circuit, $\mathcal{AC}_3$, and the factor it computes. This circuit computes the factor marginals but does not compute the factor MPEs. The figure shows a counterexample where the maximizer circuit fails to correctly compute the MPE under the empty variable instantiation (all indicators set to 1). Why did $\mathcal{MC}_3$ not compute the factor MPEs while $\mathcal{MC}_2$ did, even though both $\mathcal{AC}_2$ and $\mathcal{AC}_3$ compute the marginals of their factors? We are almost ready to address this question, but we first need to introduce another fundamental notion to provide a profound answer.

Complete Subcircuits

The fundamental notion of a *complete subcircuit* was first introduced and utilized in [73]. We obtain a complete subcircuit by traversing a circuit top-down. When visiting an adder (or maximizer) node, we choose a single child of the node. When visiting a multiplier node, we choose all its children. Figure 12 depicts two examples. A complete subcircuit has a *term* and a *coefficient*. The term is the subscripts of indicators appearing in the subcircuit. The coefficient is the product of numbers appearing in the subcircuit. In Figure 12, the left subcircuit has term $\bar{a}, \bar{b}$ and coefficient 12. The right subcircuit has term $\bar{a}, b$ and coefficient 10. If a maximizer circuit computes the factor MPEs, we can identify

a most likely, complete variable instantiation by constructing a complete subcircuit as proposed by [73]. The procedure is simple and illustrated on the right of Figure 10. We traverse the circuit top-down. When visiting a maximizer node, we choose a single child that has the same value as the node. When visiting a multiplier node, we choose all its children. The complete subcircuit selected in Figure 10 has term $\bar{a}, \bar{b}$ and coefficient 12. This means that $\bar{a}, \bar{b}$ is a most likely, complete variable instantiation and 12 is its value.

From Lookup to Reasoning: The Source of Tractability

Suppose we have an arithmetic circuit that can lookup factor values (i.e., computes the factor). As discussed earlier, it is generally efficient to construct such circuits (e.g., for the factors specified by probabilistic graphical models). The fundamental question addressed by [43] is the following. Under what conditions, and *why*, will this circuit attain the ability to reason about the factor (e.g., compute its marginals or compute its MPEs)?

The answer rests in the properties satisfied by the arithmetic circuit: decomposability, determinism and smoothness. We already defined these properties for Boolean circuits. Decomposability and smoothness have identical definitions on arithmetic circuits when viewing adders/multipliers as or/and gates and indicators λ_x as literals $X=x$. Determinism is defined as having at most one non-zero input for each adder node, when the circuit is evaluated under any complete variable instantiation. We already knew from about two decades ago that if the arithmetic circuit is deterministic, decomposable and smooth, it will compute marginals [10]. We also knew later that such a circuit will compute MPEs [73]. It was further observed about a decade ago that determinism is not needed for computing marginals, leading to a class of arithmetic circuits, known as *Sum-Product-Networks (SPNs)* [42], which are only decomposable and smooth ([42] referred to smoothness as *completeness*).¹¹ We also came to know recently that relaxing determinism can lead to an exponential reduction in the size of an arithmetic circuit [43].

While determinism is not needed for computing factor marginals, it is needed for the correctness of the linear-time MPE algorithm of [73] that we discussed earlier. This was missed in some earlier works [42], which used this algorithm on non-deterministic arithmetic circuits (i.e., SPNs) without realizing that it is no longer correct. This oversight was noticed in later works [74, 75], which also showed the hardness of computing MPEs without determinism [74]. The property of determinism was later called *selectivity* in the works on SPNs, initially in [76], leading to what has been called *Selective SPNs*. Since these are arithmetic circuits that satisfy determinism, decomposability and smoothness, they do compute both marginals and MPEs as was already known earlier [10, 73]. As we shall see later, determinism plays another important role even when computing MPE is not of interest as this property allows the compilation of arithmetic circuits from models such as Bayesian networks without the need to search for circuit parameters (constants).

Some of the key insights provided in [43] related to why the properties of decomposability, determinism, and smoothness make arithmetic circuits tractable, particularly their ability to perform reasoning through linear-time circuit traversal. The fundamental notion here is that of a complete subcircuit which we discussed earlier. Consider Figure 13 which depicts an arithmetic circuit, $\mathcal{AC}_3$, the factor $f(A, B)$ it computes, and two complete subcircuits of $\mathcal{AC}_3$. As mentioned earlier, each complete subcircuit has

¹¹ SPNs placed further structure on the location of circuit parameters, attaching them to the edges/inputs of adder nodes such that these inputs are multiplied by the corresponding parameters before being added.

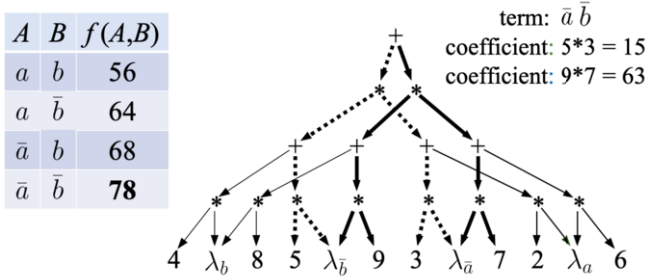


Figure 13. A factor and an arithmetic circuit $\mathcal{A}_3$ that computes it. The circuit is **decomposable** and **smooth** but **not deterministic**. The complete subcircuits with term $\bar{a}, \bar{b}$ are highlighted. The coefficients of these subcircuits are 15 and 63, which add up to the value 78 of instantiation $\bar{a}, \bar{b}$.

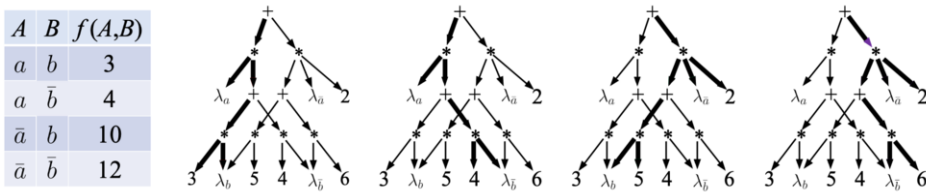


Figure 14. A factor and an arithmetic circuit $\mathcal{A}_2$ that computes it. This circuit is **decomposable, deterministic** and **smooth**. It has four complete subcircuits, in one-to-one correspondence with the factor rows.

a term $\mathbf{x}$ and a coefficient c and will be called an $\mathbf{x}$ -subcircuit. Both of the highlighted subcircuits in Figure 13 have $\bar{a}, \bar{b}$ as their term and these are the only $(\bar{a}, \bar{b})$ -subcircuits. Their coefficients are 15 and 63, which add up to 78. This is the value assigned by factor f to the variable instantiation $\bar{a}, \bar{b}$, $f(\bar{a}, \bar{b}) = 78$, which is not a coincidence. As shown in [43], decomposability ensures that the term of a complete subcircuit is consistent: it does not have conflicting variable values. Moreover, smoothness ensures that every variable is instantiated in the term of a complete subcircuit. Hence, decomposability and smoothness ensure that the term $\mathbf{x}$ of a complete subcircuit is a complete variable instantiation. Furthermore, for a complete variable instantiation $\mathbf{x}$, adding up the coefficients of $\mathbf{x}$ -subcircuits leads to the value $f(\mathbf{x})$ assigned by factor f to instantiation $\mathbf{x}$. Finally, when evaluating an arithmetic circuit at a partial variable instantiation $\mathbf{e}$, the circuit is simply adding up the coefficients of all $\mathbf{x}$ -subcircuits where $\mathbf{x}$ is compatible with $\mathbf{e}$, which yields the correct marginal for that instantiation, $f(\mathbf{e})$ [43]. These results provided the first formal and semantical explanation of why these properties enable an arithmetic circuit that computes a factor to also reason about that factor, therefore making the circuit tractable.

We now get to the property of determinism. As shown in [43], determinism (with decomposability and smoothness) ensures a one-to-one correspondence between complete subcircuits and complete variable instantiations.¹² Moreover, the coefficient c of an $\mathbf{x}$ -subcircuit corresponds to the value assigned by the factor to complete instantiation $\mathbf{x}$: $f(\mathbf{x}) = c$. Figure 14 depicts an arithmetic circuit that is deterministic, decomposable and smooth. This circuit has four complete subcircuits, highlighted in the figure, which are

¹²Assuming no zeros in the circuit or factor; otherwise, the statement of this result is more refined.

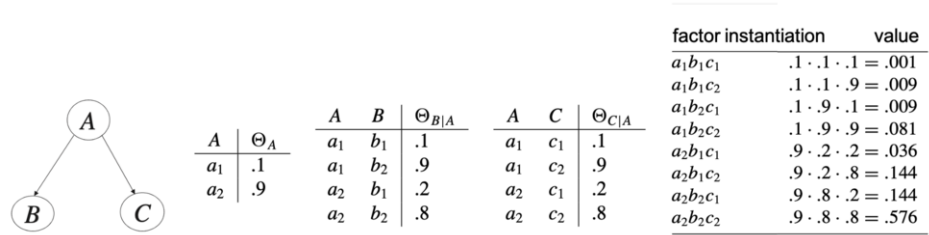


Figure 15. A Bayesian network structure, its conditional probability tables (factors Θ_A , $\Theta_{B|A}$, $\Theta_{C|A}$) and the network distribution (factor f). The distribution is a product of conditional probability tables, $f = \Theta_A \Theta_{B|A} \Theta_{C|A}$.

in one-to-one correspondence with the instantiations of variables A and B . In the order shown in Figure 14, the subcircuits have terms $a, b; a, \bar{b}; \bar{a}, b; \bar{a}, \bar{b}$ and coefficients 3; 4; 10; 12 which match the rows of the factor computed by the circuit.

What determinism does is ensure that each row of the factor (a complete variable instantiation and its value) is represented by a single, complete subcircuit. This is essential for the linear-time MPE algorithm of [73] to work properly. Consider Figure 11 where this algorithm failed to compute the MPE correctly. The arithmetic circuit in this figure is decomposable and smooth but not deterministic. The MPE (under no evidence) is the complete instantiation $\bar{a}, \bar{b}$ with value 78. However, this instantiation and its value is not represented by a single, complete subcircuit. Instead, it is represented by two complete subcircuits with coefficients 15 and 63 that add up to 78 as shown in Figure 13. The MPE algorithm will then return the coefficient 63 as shown in Figure 11, which is not correct.

More on the Need for Determinism

While determinism is not needed for computing marginals, it can be critical for compiling arithmetic circuits from models as we illustrate next. An arithmetic circuit has both a structure and parameters (the constants appearing as circuit inputs). When compiling an arithmetic circuit from a model such as a Bayesian network, one is guaranteed to find a circuit that computes marginals and whose parameters are restricted to the network parameters, if the circuit is deterministic in addition to being decomposable and smooth. However, this guarantee is lost if the circuit is only decomposable and smooth (an SPN) so one must also search for circuit parameters in this case [43]. To elaborate further on this result and its significance, consider the Bayesian network in Figure 15 which has four distinct parameters $\Theta = \{.1, .2, .8, .9\}$. Consider now the distribution (factor f) specified by this network. Each value in this factor can be expressed as a product of some parameters that come from Θ (this is guaranteed by the semantics of Bayesian networks and probabilistic graphical models more generally). We say in this case that the set of parameters Θ is *complete* for the factor f (i.e., we can express the values of f using products of numbers from Θ). If a set of parameters Θ is complete for a factor f , there is always an arithmetic circuit that computes factor f , that is deterministic, decomposable and smooth, and whose parameters come from the set Θ . Therefore, when constructing (compiling) such a circuit, one does not need to search for circuit parameters beyond the set Θ . This no longer holds without determinism so the compilation process must also search for circuit parameters in addition to its structure—we do not know of any

such compilation algorithm today.¹³ Consider the Bayesian network in Figure 15 and the following arithmetic circuit which does not require searching for parameters:

$$.001 * \lambda_{a_1} * \lambda_{b_1} * \lambda_{c_1} + .009 * \lambda_{a_1} * \lambda_{b_1} * \lambda_{c_2} + \dots + .576 * \lambda_{a_2} * \lambda_{b_2} * \lambda_{c_2}.$$

This has been called the *network polynomial* in [10] and the *factor polynomial* in [43]. Each monomial in this polynomial corresponds to a factor row: its indicators capture the row's instantiation and its coefficient captures the row's value (which is a product of Bayesian network parameters). For example, the last monomial $.576 * \lambda_{a_2} * \lambda_{b_2} * \lambda_{c_2}$ captures instantiation a_2, b_2, c_2 and value $.576 = .9 * .8 * .8$. Viewed as a depth-two arithmetic circuit, this polynomial is deterministic, decomposable and smooth, and all its parameters come from the Bayesian network parameters. It is also exponentially sized so it is mostly of theoretical interest such as illustrating the result we just discussed.¹⁴ Existing compilation algorithms construct more compact arithmetic circuits without searching for parameters, assuming the model parameters are known. Even when the model parameters are not known, these algorithms compile arithmetic circuits with symbolic parameters that can be easily replaced with numeric parameters that may be learned from data. This is a topic that we shall discuss in Section 6.4.

It is worth noting that the original treatment of arithmetic circuits in [10] started where our current treatment has ended in the previous paragraph. That is, the starting point of [10] was the notion of a network polynomial which fully captures a distribution. The notion of an arithmetic circuit was then introduced as a tool that can be used to compactly represent the exponentially-sized network polynomial. This formulation was facilitated by the fact that [10] also started by assuming the existence of a model (a Bayesian network), which fully defines the network polynomial. This can no longer be assumed when handcrafting circuits or learning them from data, which prompted the modern treatment in [43]. We finally note that arithmetic circuits which compute distributions and their marginals can be used to reason about uncertain evidence, also called *soft evidence*, in addition to evidence about continuous variables with local, univariate densities. This can be done by setting the circuit indicators to appropriate real values as discussed in [77] and [72, Section 3.7], respectively.¹⁵

¹³This result was shown in Theorem 6 of [43] for circuits that compute Boolean factors $f(\mathbf{X})$. The parameters $\Theta = \{0, 1\}$ are complete for these factors since each row has a value in $\{0, 1\}$. If the circuit is decomposable and smooth but not deterministic, then some row $\mathbf{x}$ with $f(\mathbf{x}) = 1$ must be split over at least two $\mathbf{x}$ -subcircuits with non-zero coefficients. Since the coefficients of these $\mathbf{x}$ -subcircuits must add up to 1, there must exist an $\mathbf{x}$ -subcircuit whose coefficient is in the open interval $(0, 1)$ so the circuit must have a parameter not in $\{0, 1\}$.

¹⁴For another theoretical result with substantial practical implications, we note that the polynomial partial derivatives with respect to indicators and parameters correspond to useful probabilistic quantities which can be computed by backpropagation on any equivalent arithmetic circuit. These quantities correspond to marginals of additional variable instantiations. They allow one to avoid further evaluations of the circuit which can lead to significantly more efficient reasoning; see [10] for details.

¹⁵Consider a variable X with values $x_1, \dots, x_n$. Uncertain evidence on variable X can be modeled as certain (hard) evidence on some noisy sensor η that is connected to variable X , where the noise is specified by the likelihoods of x_i given η , $Pr(\eta|x_i)$. We can assert this uncertain evidence on variable X by setting each indicator λ_{x_i} to its corresponding likelihood $Pr(\eta|x_i)$. Let a be the circuit value under this indicator setting and let b be its value when we further set indicators according to additional evidence $\mathbf{e}$. Then $Pr(\mathbf{e}|\eta) = b/a$. Moreover, this conditional probability is invariant to the specific values of indicators as long as their relative ratios match the corresponding ratios of likelihoods; see Theorem 2 in [77] and [72, Section 3.6] for details. This technique is known as the method of *virtual evidence* in the context of Bayesian networks [69]. The treatment of continuous

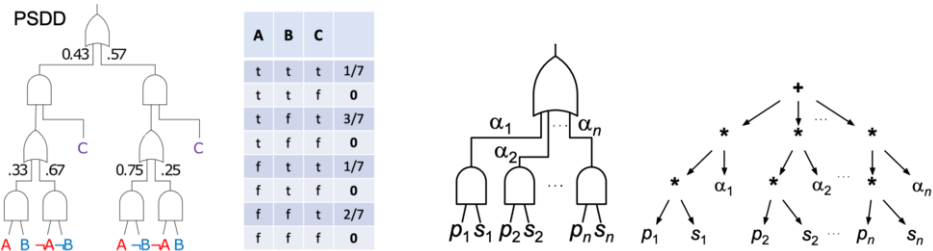


Figure 16. Left: A PSDD and the constrained factor it computes. The factor computed by this PSDD will always assign zero values to rows in which $C=f$, regardless of how the PSDD is parameterized. Right: A PSDD fragment and its corresponding arithmetic circuit fragment. Boolean variables and their negations in the PSDD (e.g., A and $\neg A$) are converted to indicators in the arithmetic circuit (λ_A and $\lambda_{\neg A}$).

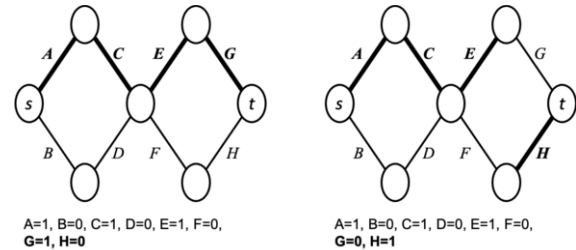


Figure 17. Modeling routes between source s and destination t as variable instantiations that satisfy some conditions. Each edge between two locations is modeled using a Boolean variable which is set to 1 if the edge is on a route and set to 0 otherwise. The variable instantiation on the left corresponds to a valid route. The variable instantiation on the right corresponds to an invalid (disconnected) route.

Circuits that Reason about Constrained Factors

We now turn to a fundamental class of arithmetic circuits, known as *PSDDs*,¹⁶ which can reason about *constrained factors* [44]. These are factors in which some rows have fixed zero values so they define mappings from a *subset* of variable instantiations into non-negative numbers; see Figure 16. Constrained factors, particularly ones representing distributions, have many applications. For example, when learning arithmetic circuits that represent distributions from data, one may have domain knowledge that rules out certain states of the world so we need to ensure that any learned circuit will assign a zero probability to such infeasible states. One may also want to induce or learn distributions over combinatorial (or structured) objects such as total and partial rankings [79], graphs [80], game plays [81], routes on a map [19, 81, 82, 83, 84] and subsets of objects [83]. Distributions over these kind of objects have been captured using constrained factors and reasoned about using PSDDs as done in the works mentioned in the previous sentence.

For an example of how constrained factors can be used to model combinatorial and structured objects, consider Figure 17 where the goal is to define a distribution over routes from source s to destination t . Each edge on the map is modeled by a binary vari-

variables is similar except that the values of indicators are determined by the observed value of the continuous variable and its density functions. This is discussed at length in [72, Section 3.7].

¹⁶PSDD stands for *Probabilistic Sentential Decision Diagram*. A PSDD represents a probability distribution but can also be used to represent and reason about general factors [78].

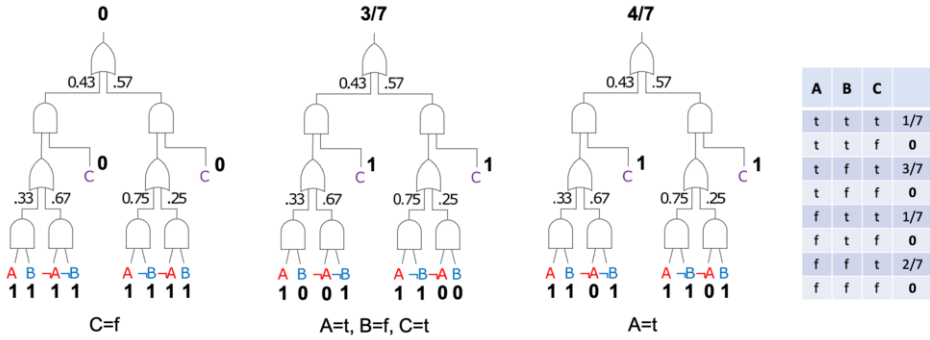


Figure 18. Evaluating a PSDD at different variable instantiations (complete and partial). The PSDD will evaluate to 0 at any variable instantiations that does not satisfy the underlying SDD circuit. This is guaranteed regardless of how the PSDD is parameterized.

able, $A, \dots, H$. A route can be modeled using a variable instantiation that sets the variables of its edges to 1 and all other variables to 0. Clearly, some variable instantiations do not correspond to valid routes so these get assigned probability 0; see the right of Figure 17. The corresponding constrained factor is then guaranteed to induce a distribution over only valid routes from source s to destination t . Similar techniques can be used to represent other combinatorial or structured objects as long as one can define the conditions that specify variable assignments which correspond to the objects of interest.

A constrained factor is defined by specifying feasible variable instantiations and their values. The main insight behind PSDDs is to specify feasible instantiations using an SDD, which is a tractable Boolean circuit that we discussed in Section 6.2; see Figure 16 (left). The SDD should evaluate feasible instantiations to 1 and infeasible instantiations to 0 and is normally obtained through a compilation process of domain constraints. A distribution over the feasible instantiations can then be defined by assigning (local) distributions to the inputs of or-gates in the SDD, leading to a PSDD as shown in Figure 16.

The PSDD is a highly structured arithmetic circuit that comes with strong guarantees. We can convert a PSDD into an arithmetic circuit as shown in Figure 16 (right) by removing reference to the underlying SDD circuit. The resulting arithmetic circuit is guaranteed to not only satisfy determinism, decomposability and smoothness but to also satisfy the stronger properties of structured decomposability and partitioned determinism which are forced by the underlying SDD circuit which satisfies these properties. As a result, the induced arithmetic circuit enjoys many desirable properties. First, the circuit is guaranteed to produce a zero value for any infeasible variable instantiation regardless of how we set the PSDD parameters. Figure 18 depicts example evaluations of a PSDD circuit that can be used to gain insights into this guarantee. Moreover, the PSDD can compute marginals and MPEs by virtue of being deterministic, decomposable and smooth [44]. With an appropriate vtree for the SDD circuit, it can also compute expectations [62]. One can also learn the maximum-likelihood parameters of a PSDD in closed form when the data is complete [44]. An EM algorithm has also been developed for learning PSDD parameters from incomplete data, including highly structured data in which examples can be specified using Boolean constraints, not just variable instantiations [79]. PSDDs also play a key role in probabilistic reasoning as shown in [78].

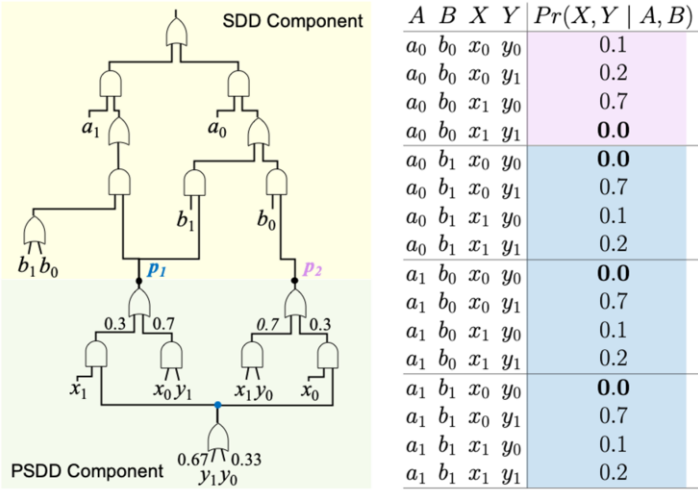


Figure 19. A conditional PSDD, which is composed of an SDD circuit and set of PSDDs that share structure. The SDD defines two constrained factors over variables X and Y depending on the state of variables A and B . These constrained factors are specified by PSDDs that share structure.

Another key development has been the introduction of *conditional PSDDs*, which can compute and reason about factors that are *conditionally constrained* [85]. These are factors in which variables are partitioned into two sets, X and Y , where the infeasible instantiations of Y -variables are conditioned on the state of X -variables. Conditional PSDDs allow one to integrate *conditional constraints* into tractable arithmetic circuits. For example, which routes are feasible in a neighborhood may be conditional on how we enter and exit that neighborhood [84]. Figure 19 depicts a conditional PSDD, which is composed of an SDD circuit and a set of PSDDs that share structure.

Further Extensions

The theory of tractable arithmetic circuits has been developed and explored in various additional ways. For example, additional classes of arithmetic circuits have been studied using further combinations of tractable circuit properties, such as the combination of structured decomposability and smoothness [86]. Other reasoning tasks have also been considered which are enabled by the different properties of tractable circuits; see, e.g., [87]. Some of the new extensions and works have referred to arithmetic circuits as *probabilistic circuits*, while reserving the term “arithmetic circuits” to deterministic, decomposable and smooth circuits. This is in contrast to earlier treatments such as [78, 43] which did not tie this term to specific properties or exclusively to probability distributions. A fundamental new extension has been the class of *Testing Arithmetic Circuits (TACs)* which select their parameters dynamically, based on circuit inputs through the inclusion of *tests* in the arithmetic circuit [88, 89]. Tractable arithmetic circuits represent multilinear functions (of indicators) [10]. Testing arithmetic circuits represent *piecewise* multilinear functions so they are universal function approximators like neural networks [89]. This new class of arithmetic circuits can be used to recover from some modeling errors that lead to compiled circuits that may not be expressive enough to fit the data generated by a mechanism

that has been modeled incorrectly [90]. Recent theoretical results have shown that full recovery from some modeling errors is possible under certain conditions [91]. We finally note that some of the key properties of tractable circuits, including decomposability and determinism, have been exploited for tractable reasoning and learning in a more general, semiring setting; see, e.g., [92, 93, 94, 95].

6.4. Compiling Knowledge and Models into Tractable Circuits

We will next provide an overview of methods for constructing tractable circuits through a process of compilation. We will provide a brief summary of methods for tractable Boolean circuits and elaborate more on methods for tractable arithmetic circuits.

Tractable Boolean circuits are normally compiled from Boolean formulas that represent logical knowledge or constraints. This is done using systems known as *knowledge compilers*, such as D4 [96], C2D [97], CUDD, MINI-C2D [98, 99], DSHARP [100] and the SDD library [101].¹⁷ Knowledge compilers can be categorized as *top-down* or *bottom-up*. Top-down compilers are based on keeping a *trace* of the exhaustive DPLL algorithm [51] and they generally incorporate advanced SAT techniques [102, 99]. These compilers normally operate on Boolean formulas in conjunctive normal form, tend to have better space complexity, and include D4, C2D and DSHARP which yield Decision-DNNFs, and MINI-C2D which yields SDDs. Bottom-up compilers incrementally compile a formula, by first compiling its components and then combining these compilations. They tend to operate on more general classes of Boolean formulas, demand more space, and include CUDD and the SDD library, which yield OBDDs and SDDs, respectively.

Turning next to the compilation of tractable arithmetic circuits, we first note that PSDDs, which represent distributions, are somewhat special as these arithmetic circuits are based on SDDs, which are Boolean circuits. Hence, the structure of a PSDD is obtained by first compiling a Boolean formula into an SDD, which is then parameterized to yield a PSDD. The PSDD parameters are typically obtained through a learning process based on complete or incomplete data [44, 79]. The Boolean formula that triggers this compilation process usually defines feasible states of the world, or characterizes structured (combinatorial) objects such as routes, graphs, and rankings.¹⁸

Arithmetic circuits that satisfy determinism, decomposability and smoothness, also known as ACs, are typically compiled from probabilistic models such as Bayesian networks and probabilistic logic programs. We will only discuss the former but see [21, 23, 25] for some examples of the latter. Recall again that the structure of arithmetic circuits that satisfy only decomposability and smoothness (i.e., SPNs) is normally handcrafted or learned from data since relaxing determinism complicates the compilation of these structures from models as discussed in the previous section.

The compilation of Bayesian networks (and probabilistic graphical models more generally) is done either directly, or indirectly as shown in Figure 6. *Indirect methods*

¹⁷D4: <http://www.cril.univ-artois.fr/kc/d4/>; C2D: <http://reasoning.cs.ucla.edu/c2d/>; CUDD: <https://davidkebo.com/cudd/>; MINI-C2D: <http://reasoning.cs.ucla.edu/minic2d/>; DSHARP: <https://bitbucket.org/haz/dsharp/>; SDD library: <http://reasoning.cs.ucla.edu/sdd>. A Python wrapper of the SDD library is available at <https://github.com/wannesm/PySDD> and an NNF-to-SDD circuit compiler, based on PySDD, is available at <https://github.com/art-ai/nnf2sdd>.

¹⁸See <https://github.com/art-ai/pypsdd>; <http://reasoning.cs.ucla.edu/psdd/>; and https://github.com/hahaXD/hierarchical_map_compiler for some related tools.

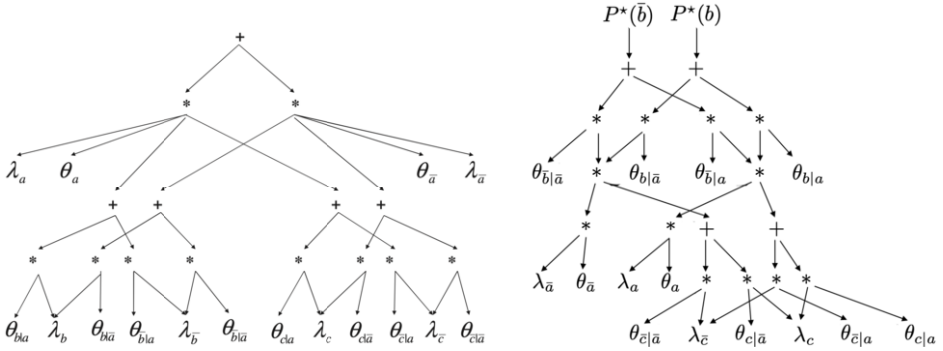


Figure 20. Compiling the Bayesian network $B \leftarrow A \rightarrow C$ into tractable arithmetic circuits. The circuit on the left computes the probability of (partial) evidence on variables A , B and C . The circuit on the right computes the distribution of variable B given (partial) evidence on variables A and C (after normalization); this circuit targets a particular query, $Pr(B|\cdot)$. In both cases, the circuit parameters correspond to Bayesian network parameters so they can be fetched from the Bayesian network, if known; otherwise, they can be learned from data.

are based on reductions to weighted model counting [14]. The model is first *encoded* into a Boolean formula with weights on literals, as initially proposed in [9, 10]. The Boolean formula is then compiled into a smooth d-DNNF circuit (a tractable Boolean circuit), from which a tractable arithmetic circuit is extracted. The size of the final arithmetic circuit depends on the size of compiled Boolean circuit, which is determined by the quality of used encoding scheme and knowledge compiler. These encoding schemes capture both the *global structure* of the model (its topology) and its *local structure* (parameters). Their efficacy relies particularly on how well they encode local structure; that is, the extent to which they capture the properties of model parameters. The first encoding scheme [9] was somewhat basic in nature, but it was later followed by more refined and potent encodings in [103, 104, 105]. A comparative discussion of various encoding schemes, including some more recent ones, is given in [68]. The above approaches have traditionally used top-down compilers of Boolean formulas into tractable Boolean circuits; for example, the ACE system [14] used the top-down C2D knowledge compiler. More recently, bottom-up compilation approaches have also been proposed, which are based on compiling the factors of a probabilistic graphical model into tractable circuits and then combining these compiled circuits in a bottom-up fashion [106, 78].

Direct methods for compiling models into tractable arithmetic circuits are simpler but they tend to be less effective, with some exceptions. These methods include the extraction of a tractable arithmetic circuit from the structure of a jointree for the model [10]. They also include methods based on the variable elimination algorithm [107, 108]; see also [72, Chapter 12]. The most recent of these methods [108] is based on variable elimination and stands out for a number of reasons. First, the method compiles circuits that target a specific class of queries, specified by evidence variables (input) and a query variable (output), which is meant to facilities supervised learning from labelled data; see Figure 20. Second, this recent method compiles arithmetic circuits in the form of computation graphs in which nodes represent tensor operations instead of arithmetic operations, allowing parallelization during parameter learning and inference.¹⁹ Third, this method computationally exploits a new type of local structure: *functional dependencies* whose

¹⁹See also [109, 110] for related representations of circuits with handcrafted or learned structures.

identities are *unknown*. We have a functional dependency between a node and its parents in the Bayesian network when the state of the node is a function of the states of its parents (i.e., there is no uncertainty). A functional dependency is unknown when we do not know the identity of this function. This is significant in a learning context where the goal is to learn such functions from data. This latest algorithm [108] was motivated by the use of tractable arithmetic circuits for supervised learning, as in neural networks, but where the structure of the circuit is compiled from a model instead of being handcrafted as is normally the case with neural networks. Recent results [20] have shown the promise of this approach to supervised learning as it provides a principled method for embedding background knowledge into the supervised learning process (independence constraints, logical constraints, known parameters, and unknown functional dependencies).

We close this section by the following remark on the interplay between the compilation of circuit structure and the learning of circuit parameters. Initially, the goal of compilation algorithms was to facilitate reasoning since various forms of inference can be performed in linear time on the compiled circuits. In this context, the assumption was that one already knows the model parameters before the compilation process starts. Later, however, these compilation algorithms were used to compile only the circuit structure and then coupled with additional algorithms that learn the circuit parameters from data. The key observation which permits this expanded role is that the compilation process can be conducted even if we do not know the model parameters. In particular, the substitution of model parameters into the compiled circuit can be postponed to the extraction phase of Figure 6. Hence, if a parameter is unknown, it can be kept in symbolic form during the extraction phase and then learned from labeled or unlabeled data after the compilation process is concluded; e.g., as in [108, 20] and [44, 79], respectively. This is possible since determinism (with decomposability and smoothness) allows one to compile circuits with parameters that correspond to model parameters, as discussed in the previous section; see also the circuits in Figure 20. Interesting enough, determinism also facilitates the learning of circuit parameters, allowing one to learn parameters in closed form under complete data, in addition to allowing a linear-time evaluation of the EM update equations under incomplete data.²⁰ This further highlights the importance of determinism, which also holds for tractable arithmetic circuits that have handcrafted or learned structures.²¹

6.5. Concluding Remarks

Tractable Boolean and arithmetic circuits have been evolving into a computational and semantical backbone for modern approaches that aim to combine knowledge, reasoning, and learning. The different modes of constructing these circuits—through compilation, handcrafting and learning—have further contributed to their versatility and positioned them as valuable tools for serving the objectives of neuro-symbolic AI. We provided an overview of tractable circuits in this chapter, while focusing on their foundations,

²⁰For complete data, this follows directly from the same result for Bayesian networks. For incomplete data, a second pass (backpropagation) on the circuit provides marginals over families (variables and their parents) which is all that one needs to evaluate the EM update equations [10].

²¹For arithmetic circuits with handcrafted or learned structures, determinism can also facilitate parameter learning if these parameters are properly located as in [76] and [44]. For further perspective, see [111] for a treatment of parameter learning when the arithmetic circuit does not satisfy determinism.

their salient properties, and some of the key developments and milestones that have contributed to their current status in the field. There is much more to say about tractable circuits beyond what has been covered in this treatment. This includes the various hand-crafted architectures, the learning of circuits structures (and parameters) from data, and the compilation of these circuits from higher-level models and other forms of knowledge.

Acknowledgements

This work has been partially supported by grants from NSF IIS-1910317, ONR N00014-18-1-2561, and DARPA N66001-17-2-4032. I wish to thank Yizuo Chen, Haiying Huang, Jason Shen and the anonymous reviewers for providing valuable feedback.

References

- [1] Cadoli M, Donini FM. A survey on knowledge compilation. *AI Commun.* 1997;10(3-4):137–150.
- [2] Marquis P. Knowledge compilation using theory prime implicates. In: *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence (IJCAI)*; 1995. p. 837–845.
- [3] Selman B, Kautz HA. Knowledge compilation and theory approximation. *J ACM.* 1996;43(2):193–224.
- [4] Darwiche A, Marquis P. A knowledge compilation map. *J Artif Intell Res.* 2002;17:229–264.
- [5] Darwiche A. Tractable knowledge representation formalisms. In: *Tractability: Practical approaches to hard problems*. Cambridge University Press; 2014. p. 141–172.
- [6] Darwiche A, Marquis P, Suciu D, et al. Recent trends in knowledge compilation (dagstuhl seminar 17381). *Dagstuhl Reports.* 2017;7(9):62–85.
- [7] Slivovsky F. An introduction to knowledge compilation. In: *KOCOON workshop on knowledge compilation*; 2019. <http://kocoon.gforge.inria.fr/slides/slivovsky.pdf>.
- [8] Darwiche A. Three modern roles for logic in AI. In: *PODS. ACM*; 2020. p. 229–243.
- [9] Darwiche A. A logical approach to factoring belief networks. In: Fensel D, Giunchiglia F, McGuinness DL, et al., editors. *Proceedings of the Eight International Conference on Principles and Knowledge Representation and Reasoning (KR-02)*, Toulouse, France, April 22-25, 2002. Morgan Kaufmann; 2002. p. 409–420.
- [10] Darwiche A. A differential approach to inference in Bayesian networks. *J ACM.* 2003;50(3):280–305.
- [11] Garcez A, Lamb L. Neurosymbolic ai: The 3rd wave. *ArXiv.* 2020;abs/2012.05876.
- [12] Raedt LD, Dumancic S, Manhaeve R, et al. From statistical relational to neuro-symbolic artificial intelligence. In: *IJCAI. ijcai.org*; 2020. p. 4943–4950.
- [13] Besold TR, d’Avila Garcez A, Bader S, et al. Neural-symbolic learning and reasoning: A survey and interpretation ; 2017.
- [14] Chavira M, Darwiche A. On probabilistic inference by weighted model counting. *Artif Intell.* 2008; 172(6-7):772–799. Available from: <https://doi.org/10.1016/j.artint.2007.11.002>.
- [15] Darwiche A. On the tractable counting of theory models and its application to truth maintenance and belief revision. *J Appl Non Class Logics.* 2001;11(1-2):11–34.
- [16] Di Liello L, Ardino P, Gobbi J, et al. Efficient generation of structured objects with constrained adversarial networks. In: Larochelle H, Ranzato M, Hadsell R, et al., editors. *Advances in Neural Information Processing Systems*; Vol. 33. Curran Associates, Inc.; 2020. p. 14663–14674.
- [17] Xu J, Zhang Z, Friedman T, et al. A semantic loss function for deep learning with symbolic knowledge. In: *ICML; (Proceedings of Machine Learning Research*; Vol. 80). PMLR; 2018. p. 5498–5507.
- [18] Xie Y, Xu Z, Meel KS, et al. Embedding symbolic knowledge into deep networks. In: *NeurIPS*; 2019. p. 4235–4245.
- [19] Ling J, Chandak K, Kumar A. Integrating knowledge compilation with reinforcement learning for routes. In: *ICAPS. AAAI Press*; 2021. p. 542–550.
- [20] Chen Y, Choi A, Darwiche A. Supervised learning with background knowledge. In: *PGM; (Proceedings of Machine Learning Research*; Vol. 138). PMLR; 2020. p. 89–100.

- [21] Manhaeve R, Dumancic S, Kimmig A, et al. DeepProbLog: Neural probabilistic logic programming. In: BNAIC/BENELEARN; (CEUR Workshop Proceedings; Vol. 2491). CEUR-WS.org; 2019.
- [22] Saad FA, Rinard MC, Mansinghka VK. SPPL: probabilistic programming with fast exact symbolic inference. In: PLDI. ACM; 2021. p. 804–819.
- [23] Fierens D, den Broeck GV, Renkens J, et al. Inference and learning in probabilistic logic programs using weighted Boolean formulas. *Theory Pract Log Program*. 2015;15(3):358–401.
- [24] Holtzen S, den Broeck GV, Millstein TD. Scaling exact inference for discrete probabilistic programs. *Proc ACM Program Lang*. 2020;4(OOPSLA):140:1–140:31.
- [25] Dries A, Kimmig A, Meert W, et al. Problog2: Probabilistic logic programming. In: ECML/PKDD (3); (Lecture Notes in Computer Science; Vol. 9286). Springer; 2015. p. 312–315.
- [26] Jha AK, Suciu D. Knowledge compilation meets database theory: Compiling queries to decision diagrams. *Theory Comput Syst*. 2013;52(3):403–440.
- [27] Amarilli A. Provenance in databases and links to knowledge compilation. In: KOCOON workshop on knowledge compilation; 2019. <http://kocoon.gforge.inria.fr/slides/amarilli.pdf>.
- [28] Bova S, Capelli F, Mengel S, et al. Knowledge compilation meets communication complexity. In: IJCAI. IJCAI/AAAI Press; 2016. p. 1008–1014.
- [29] Sharma S, Gupta R, Roy S, et al. Knowledge compilation meets uniform sampling. In: LPAR; (EPIc Series in Computing; Vol. 57). EasyChair; 2018. p. 620–636.
- [30] Huang X, Izza Y, Ignatiev A, et al. Efficient explanations for knowledge compilation languages ; 2021.
- [31] Darwiche A, Marquis P. On quantifying literals in Boolean logic and its applications to explainable AI. *CoRR*. 2021;abs/2108.09876.
- [32] Audemard G, Koriche F, Marquis P. On tractable XAI queries based on compiled representations. In: KR; 2020. p. 838–849.
- [33] Arenas M, Bertossi PBL, Monet M. The tractability of shap-score-based explanations over deterministic and decomposable Boolean circuits ; 2021.
- [34] den Broeck GV, Lykov A, Schleich M, et al. On the tractability of shap explanations ; 2021.
- [35] Shi W, Shih A, Darwiche A, et al. On tractable representations of binary neural networks. In: KR; 2020. p. 882–892.
- [36] Shih A, Choi A, Darwiche A. A symbolic approach to explaining Bayesian network classifiers. In: IJCAI. ijcai.org; 2018. p. 5103–5111.
- [37] Darwiche A, Hirth A. On the reasons behind decisions. In: ECAI; (Frontiers in Artificial Intelligence and Applications; Vol. 325). IOS Press; 2020. p. 712–720.
- [38] Akshay S, Arora J, Chakraborty S, et al. Knowledge compilation for Boolean functional synthesis. In: FMCAD. IEEE; 2019. p. 161–169.
- [39] Shah P, Bansal A, Akshay S, et al. A normal form characterization for efficient Boolean skolem function synthesis. In: LICS. IEEE; 2021. p. 1–13.
- [40] Lowd D, Domingos PM. Learning arithmetic circuits. In: Proceedings of the 24th Conference in Uncertainty in Artificial Intelligence (UAI); 2008. p. 383–392.
- [41] Roth D, Samdani R. Learning multi-linear representations of distributions for efficient inference. *Mach Learn*. 2009;76(2-3):195–209.
- [42] Poon H, Domingos PM. Sum-product networks: A new deep architecture. In: UAI. AUAI Press; 2011. p. 337–346.
- [43] Choi A, Darwiche A. On relaxing determinism in arithmetic circuits. In: Proceedings of the Thirty-Fourth International Conference on Machine Learning (ICML); 2017. p. 825–833.
- [44] Kisa D, den Broeck GV, Choi A, et al. Probabilistic sentential decision diagrams. In: KR. AAAI Press; 2014.
- [45] Darwiche A, Marquis P. A knowledge compilation map. *JAIR*. 2002;17:229–264.
- [46] Darwiche A. Decomposable negation normal form. *J ACM*. 2001;48(4):608–647.
- [47] Cook SA. The complexity of theorem-proving procedures. In: STOC. ACM; 1971. p. 151–158.
- [48] III JTG. Computational complexity of probabilistic turing machines. In: STOC. ACM; 1974. p. 91–95.
- [49] Valiant LG. The complexity of enumeration and reliability problems. *SIAM J Comput*. 1979;8(3):410–421.
- [50] Shih A, den Broeck GV, Beame P, et al. Smoothing structured decomposable circuits. In: NeurIPS; 2019. p. 11412–11422.
- [51] Huang J, Darwiche A. The language of search. *J Artif Intell Res*. 2007;29:191–219.
- [52] Beame P, Li J, Roy S, et al. Exact model counting of query expressions: Limitations of propositional

- methods. *ACM Trans Database Syst.* 2017;42(1):1:1–1:46.
- [53] Pipatsrisawat K, Darwiche A. A New d-DNNF-Based Bound Computation Algorithm for Functional EMAJSAT. In: *IJCAI*; July; 2009. p. 590–595.
 - [54] Wagner KW. The complexity of combinatorial problems with succinct input representation. *Acta Informatica.* 1986;23(3):325–356.
 - [55] Park JD, Darwiche A. Complexity results and approximation strategies for MAP explanations. *J Artif Intell Res.* 2004;21:101–133.
 - [56] Bryant RE. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers.* 1986;C-35:677–691.
 - [57] Gergov J, Meinel C. Efficient boolean manipulation with obdd's can be extended to fbdd's. *IEEE Trans Computers.* 1994;43(10):1197–1209.
 - [58] Beame P, Li J, Roy S, et al. Lower bounds for exact model counting and applications in probabilistic databases. In: *UAI. AUAU Press*; 2013.
 - [59] Pipatsrisawat K, Darwiche A. New compilation languages based on structured decomposability. In: *AAAI. AAAI Press*; 2008. p. 517–522.
 - [60] Darwiche A. SDD: A new canonical representation of propositional knowledge bases. In: *IJCAI. IJCAI/AAAI*; 2011. p. 819–826.
 - [61] Choi A, Xue Y, Darwiche A. Same-decision probability: A confidence measure for threshold-based decisions. *Int J Approx Reason.* 2012;53(9):1415–1428.
 - [62] Oztok U, Choi A, Darwiche A. Solving pp^{PP}-complete problems using knowledge compilation. In: Baral C, Delgrande JP, Wolter F, editors. *Principles of Knowledge Representation and Reasoning: Proceedings of the Fifteenth International Conference, KR 2016, Cape Town, South Africa, April 25–29, 2016. AAAI Press*; 2016. p. 94–103.
 - [63] Bova S. Sdds are exponentially more succinct than obdds. In: *AAAI. AAAI Press*; 2016. p. 929–935.
 - [64] Beame P, Liew V. New limits for knowledge compilation and applications to exact model counting. In: *UAI. AUAU Press*; 2015. p. 131–140.
 - [65] Bollig B, Buttkus M. On the relative succinctness of sentential decision diagrams. *Theory Comput Syst.* 2019;63(6):1250–1277.
 - [66] Lai Y, Meel KS, Yap RHC. The power of literal equivalence in model counting. In: *AAAI. AAAI Press*; 2021. p. 3851–3859.
 - [67] Agrawal D, Pote Y, Meel KS. Partition function estimation: A quantitative study. *CoRR.* 2021; abs/2105.11132. Will appear in *IJCAI-21*.
 - [68] Dilkas P, Belle V. Weighted model counting with conditional weights for Bayesian networks. In: *UAI*; 2021.
 - [69] Pearl J. Probabilistic reasoning in intelligent systems: Networks of plausible inference. Morgan Kaufmann; 1988.
 - [70] Shimony SE. Finding MAPs for belief networks is NP-hard. *Artif Intell.* 1994;68(2):399–410.
 - [71] Roth D. On the hardness of approximate reasoning. *Artif Intell.* 1996;82(1-2):273–302.
 - [72] Darwiche A. Modeling and reasoning with Bayesian networks. Cambridge University Press; 2009.
 - [73] Chan H, Darwiche A. On the robustness of most probable explanations. In: *Proceedings of the 22nd Conference in Uncertainty in Artificial Intelligence (UAI)*; 2006.
 - [74] Peharz R, Gens R, Pernkopf F, et al. On the latent variable interpretation in sum-product networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI).* 2016;.
 - [75] Mauá DD, de Campos CP. Approximation complexity of maximum a posteriori inference in sum-product networks. *CoRR.* 2017;abs/1703.06045.
 - [76] Peharz R, Gens R, Domingos PM. Learning selective sum-product networks. In: *LTPM workshop*; 2014.
 - [77] Chan H, Darwiche A. On the revision of probabilistic beliefs using uncertain evidence. *Artif Intell.* 2005;163(1):67–90.
 - [78] Shen Y, Choi A, Darwiche A. Tractable operations for arithmetic circuits of probabilistic models. In: *NIPS*; 2016. p. 3936–3944.
 - [79] Choi A, Van den Broeck G, Darwiche A. Tractable learning for structured probability spaces: A case study in learning preference distributions. In: *IJCAI*; 2015.
 - [80] Nishino M, Yasuda N, Minato Si, et al. Compiling graph substructures into sentential decision diagrams. In: *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence. AAAI Press*; 2017. p. 1213–1221; AAAI'17.

- [81] Choi A, Tavabi N, Darwiche A. Structured features in naive Bayes classification. In: AAAI; 2016.
- [82] Choi A, Shen Y, Darwiche A. Tractability in structured probability spaces. In: Advances in Neural Information Processing Systems 30 (NIPS); 2017.
- [83] Shen Y, Choi A, Darwiche A. A tractable probabilistic model for subset selection. In: Proceedings of the 33rd Conference on Uncertainty in Artificial Intelligence (UAI); 2017.
- [84] Shen Y, Goyanka A, Darwiche A, et al. Structured Bayesian Networks: From inference to learning with routes. In: AAAI. AAAI Press; 2019. p. 7957–7965.
- [85] Shen Y, Choi A, Darwiche A. Conditional PSDDs: Modeling and learning with modular knowledge. In: AAAI. AAAI Press; 2018. p. 6433–6442.
- [86] Dang M, Vergari A, den Broeck GV, Strudel. Learning structured-decomposable probabilistic circuits. In: PGM; (Proceedings of Machine Learning Research; Vol. 138). PMLR; 2020. p. 137–148.
- [87] Vergari A, Choi Y, Liu A, et al. A compositional atlas of tractable circuit operations: From simple transformations to complex information-theoretic queries. CoRR. 2021;abs/2102.06137.
- [88] Choi A, Darwiche A. On the relative expressiveness of Bayesian and neural networks. In: PGM; (Proceedings of Machine Learning Research; Vol. 72). PMLR; 2018. p. 157–168.
- [89] Choi A, Wang R, Darwiche A. On the relative expressiveness of Bayesian and neural networks. Int J Approx Reason. 2019;113:303–323.
- [90] Shen Y, Huang H, Choi A, et al. Conditional independence in Testing Bayesian Networks. In: ICML; (Proceedings of Machine Learning Research; Vol. 97). PMLR; 2019. p. 5701–5709.
- [91] Huang H, Darwiche A. On recovering from modeling errors using testing Bayesian networks. In: ICML; (Proceedings of Machine Learning Research; Vol. 139). PMLR; 2021. p. 4402–4411.
- [92] Kimmig A, Van den Broeck G, De Raedt L. Algebraic model counting. CoRR. 2012;abs/1211.4475. Available from: <http://arxiv.org/abs/1211.4475>.
- [93] Kimmig A, Van den Broeck G, De Raedt L. Algebraic model counting. International Journal of Applied Logic. 2016 Nov;.
- [94] Friesen AL, Domingos PM. The sum-product theorem: A foundation for learning tractable models. In: Proceedings of the 33rd International Conference on Machine Learning (ICML); 2016. p. 1909–1918.
- [95] Belle V, Raedt LD. Semiring programming: A semantic framework for generalized sum product problems. Int J Approx Reason. 2020;126:181–201.
- [96] Lagniez J, Marquis P. An improved decision-dnnf compiler. In: IJCAI. ijcai.org; 2017. p. 667–673.
- [97] Darwiche A. New Advances in Compiling CNF into Decomposable Negation Normal Form. In: ECAI; 2004. p. 328–332.
- [98] Oztok U, Darwiche A. On compiling CNF into decision-DNNF. In: Proceedings of the 20th International Conference on Principles and Practice of Constraint Programming (CP); 2014. p. 42–57.
- [99] Oztok U, Darwiche A. An exhaustive DPLL algorithm for model counting. J Artif Intell Res. 2018; 62:1–32.
- [100] Muise CJ, McIlraith SA, Beck JC, et al. Dsharp: Fast d-dnnf compilation with sharpsat. In: Canadian Conference on AI; (Lecture Notes in Computer Science; Vol. 7310). Springer; 2012. p. 356–361.
- [101] Choi A, Darwiche A. Dynamic minimization of sentential decision diagrams. In: Proceedings of the 27th Conference on Artificial Intelligence (AAAI); 2013.
- [102] Sang T, Bacchus F, Beame P, et al. Combining component caching and clause learning for effective model counting. In: SAT; 2004.
- [103] Sang T, Beame P, Kautz HA. Performing Bayesian inference by weighted model counting. In: AAAI. AAAI Press / The MIT Press; 2005. p. 475–482.
- [104] Chavira M, Darwiche A. Compiling Bayesian networks with local structure. In: Kaelbling LP, Safiotti A, editors. IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence. Professional Book Center; 2005. p. 1306–1312.
- [105] Chavira M, Darwiche A. Encoding cnfs to empower component analysis. In: SAT; (Lecture Notes in Computer Science; Vol. 4121). Springer; 2006. p. 61–74.
- [106] Choi A, Kisa D, Darwiche A. Compiling probabilistic graphical models using sentential decision diagrams. In: ECSQARU; (Lecture Notes in Computer Science; Vol. 7958). Springer; 2013. p. 121–132.
- [107] Chavira M, Darwiche A. Compiling Bayesian networks using variable elimination. In: Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI); 2007. p. 2443–2449.
- [108] Darwiche A. An advance on variable elimination with applications to tensor-based computation. In: ECAI; (Frontiers in Artificial Intelligence and Applications; Vol. 325). IOS Press; 2020. p. 2559–2568.
- [109] Peharz R, Lang S, Vergari A, et al. Einsum networks: Fast and scalable learning of tractable proba-

- bilistic circuits. In: ICML; (Proceedings of Machine Learning Research; Vol. 119). PMLR; 2020. p. 7563–7574.
- [110] Molina A, Vergari A, Stelzner K, et al. SPFlow: An easy and extensible library for deep probabilistic learning using sum-product networks ; 2019.
- [111] Zhao H, Poupart P, Gordon GJ. A unified approach for learning the parameters of sum-product networks. In: NIPS; 2016. p. 433–441.

Chapter 7

Neuro-Symbolic AI = Neural + Logical + Probabilistic AI

Robin Manhaeve, KU Leuven, Dept. of Computer Science; Leuven.AI

Giuseppe Marra, KU Leuven, Dept. of Computer Science; Leuven.AI

Thomas Demeester, Ghent University - imec, Dept. of Information Technology

Sebastijan Dumančić, KU Leuven, Dept. of Computer Science; Leuven.AI

Angelika Kimmig, KU Leuven, Dept. of Computer Science; Leuven.AI

Luc De Raedt, KU Leuven, Dept. of Computer Science; Leuven.AI; Örebro University

There is a broad consensus that both learning and reasoning are essential to achieve true (artificial) intelligence [1]. This explains why the quest for neuro-symbolic artificial intelligence (NeSy) [2, 3, 4, 5], which combines high-level reasoning with low-level perception, is high on the research agenda.

The two most prominent frameworks for reasoning are logic and probability. While in the past, they were studied by separate communities in artificial intelligence, a significant number of researchers has been working towards their integration, and aiming at combining probability with logic and statistical learning; cf. the areas of statistical relational artificial intelligence (StarAI) [6, 7] and probabilistic logic programming [8]. The reasoning abilities of statistical relational artificial intelligence approaches are complementary to the strong pattern-recognition abilities of deep learning.

Generally, neuro-symbolic systems integrate logic with neural networks. Probabilistic theory has already been integrated with logic (cf. statistical relational AI) and neural networks. It therefore makes sense to consider the integration of logic, neural networks and probabilities. This effectively leads to an integration of probabilistic logics with neural networks and opens up new abilities. Furthermore, although at first sight, including probability may appear as a complication, it actually greatly simplifies the integration of neural networks with logic. The reason for this is that probabilistic frameworks provide clear optimisation criteria, such as the likelihood of the training examples. Real-valued probabilistic quantities are also well-suited for gradient-based training procedures, as opposed to their discrete logic counterparts.

In this chapter, we first look at the three base paradigms (i.e. neural, logical and probabilistic methods) separately. Then, we look at the well established integrations, NeSy and StarAI. Next, we consider the integration of all three paradigms as Neural Probabilistic Logic Programming, and exemplify it with the DeepProbLog framework. Finally, we discuss the limitations of the state of the art, and consider future directions based on the parallels between StarAI and NeSy.

7.1. Base paradigms

We first consider the base elements of the integrations: neural, logical and probabilistic methods.

7.1.1. Logical methods

We now introduce the necessary background on the logical methods for this chapter. For more details, we refer to [9]. In this chapter, we focus on the subset of logic based on definite clauses. More formally, a *definite clause* is an expression of the form $h \leftarrow b_1 \wedge \dots \wedge b_n$ where h and the b_i are logical atoms of the form $p(t_1, \dots, t_m)$, with p a predicate of arity m and the t_i terms. Terms then are either constants, logical variables or structured terms of the form $f(t_1, \dots, t_k)$ with f a functor and the t_j terms. Definite clauses for which $n = 0$ are called *facts*. By using the standard Prolog notation (i.e. commas for conjunctions and $:$ – for the implication), the following definite clause states that the alarm goes off if there is an earthquake:

```
alarm :- earthquake.
```

A logic program consists of a set of definite clauses. We show an encoding of the alarm example from [10] in Figure 1. In this example, your alarm can be triggered by either a burglar or an earthquake. If either of your neighbours, John and Mary, are home, they will call you if they hear the alarm. This program is *propositional* since every atom in the head or the body of the clauses has arity zero.

By introducing relations, we move from a propositional logic program to a first-order logic program. First-order programs are much more compact since definite clauses with variables behave as templates for many instantiations of the same clause. In Figure 2, we show a first order variant of the alarm program from Figure 1, where we compactly describe the behavior of multiple individuals (i.e. *mary* or *john*) using relations (i.e. *at_home*) and variables.

A substitution θ is an expression of the form $\{V_1 = t_1, \dots, V_n = t_n\}$ where the V_i are different variables and the t_i terms. Applying a substitution θ to an expression e (term or clause) yields the instantiated expression $e\theta$ where all variables V_i in e have been replaced by their corresponding terms t_i in e . For example, applying the substitution $\theta = \{X = \text{mary}\}$ to the term $g = \text{calls}(X)$ result in the substitution $g\theta = \text{calls}(\text{mary})$.

The Herbrand base of a set of clauses is the set of all ground atoms that can be constructed using the predicates, functors and constants occurring in the clauses. Subsets of the Herbrand base are called Herbrand interpretations. A Herbrand interpretation is a model of a definite clause $h \leftarrow b_1 \wedge \dots \wedge b_n$ if for every substitution θ such that $b_1\theta \wedge$

```

# Facts
burglary.
earthquake.
at_home_mary.
at_home_john.

# Clauses
alarm :- earthquake.
alarm :- burglary.
calls_mary :- alarm, at_home_mary.
calls_john :- alarm, at_home_john.

```

Figure 1. A propositional logic program for the alarm example.

```

burglary.
earthquake.
at_home(mary).
at_home(john).

alarm :- earthquake.
alarm :- burglary.
calls(X) :- alarm, at_home(X).

```

Figure 2. A first-order program for the alarm example.

$\dots \wedge b_n \theta$ is in the interpretation, $h\theta$ is in the interpretation as well. An interpretation is a model of a definite clause program if it is a model of all its definite clauses. Finally, the semantics of a definite clause program is given by its unique Least Herbrand Model, i.e. the smallest model with respect to the set inclusion. This means that the Least Herbrand Model contains only the set of all ground facts that are logically entailed by the program.

Example 7.1.1 (Least Herbrand Model). The Least Herbrand Model of the program in Figure 2 is:

$$\{\text{burglary}, \text{earthquake}, \text{at_home}(\text{mary}), \text{at_home}(\text{john}), \\ \text{alarm}, \text{calls}(\text{mary}), \text{calls}(\text{john})\}$$

This definition of the semantics only holds for programs without negation. The negation of an atom in logic programming (denoted as $\neg p(t_1, \dots, t_m)$) is defined as negation as failure, meaning that the negation of an atom is true if the atom cannot be derived from the program. Programs with negation are not guaranteed to have a unique minimal Herbrand model. Several ways to define a canonical model have been studied (e.g. the well-founded semantics [11]).

7.1.2. Probabilistic methods

Probabilistic graphical models [12] are graphical models that compactly represent a (joint) probability distribution $P(X_1, \dots, X_n)$ over n discrete or continuous random variables $X_1, \dots, X_n$. The key idea is that the joint probability distribution factorizes over some factors f^i specified over subsets X^i of the variables $\{X_1, \dots, X_n\}$.

$$P(X_1, \dots, X_n) = \frac{1}{Z} f_1(X^1) \times \dots \times f_k(X^k) \quad (1)$$

where Z is a normalization constant for P to represent a probability distribution.

The random variables correspond to the nodes in the graphical structure, and the factorization is determined by the edges in the graph. There are two main classes of probabilistic graphical models: Bayesian networks, where the underlying graph structure is a directed acyclic graph, and Markov random fields, where the graph is undirected. In this chapter, we will focus on Bayesian networks as we will establish several parallels with definite clause logic programs.

In a Bayesian network, we define a factor $f^i(X_i | \text{parents}(X_i))$ for each of the nodes X_i , where $\text{parents}(X_i)$ denotes the set of random variables that are a parent of X_i in the graph. The factors correspond to the conditional probability distributions $P(X_i | \text{parents}(X_i))$. In Figure 3 the *burglary alarm* Bayesian network [10] is shown. Here, for example, the random variable `alarm` is associated to the factor $P(\text{alarm} | \text{burglary}, \text{earthquake})$. All the random variables are Boolean, i.e. they can take one of the values $\{\text{true}, \text{false}\}$ or $\{1, 0\}$. For Bayesian networks, because one works with conditional probability distributions as factors, the joint distribution in Equation 1 is normalized and $Z = 1$. A Bayesian network represents a set of conditional independence assumptions: a variable is independent of the other variables that are not its descendants in the graph given its parents.

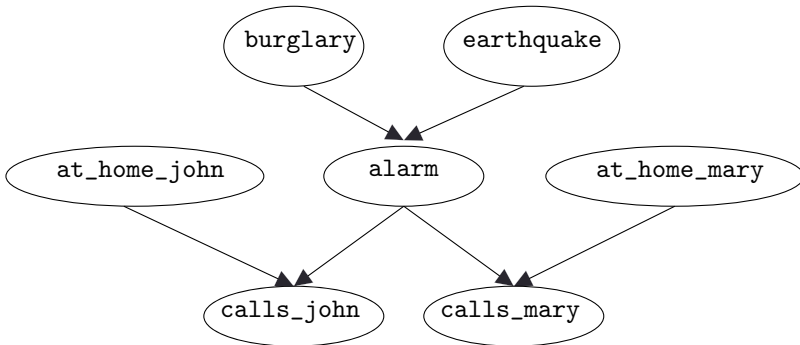


Figure 3. The alarm Bayesian network.

Probabilistic graphical models are an expressive formalism for learning and reasoning under uncertainty. However, inference in such models (e.g. computing specific marginal probabilities), is in general intractable as it is #P-complete. Nevertheless, there has been a lot of progress in devising various types of approximate and exact inference algorithms [13, 14, 15].

Table 1. Conditional probability distribution $P(\text{alarm}|\text{burglary}, \text{earthquake})$ as a Conditional Probability Table, cf. Figure 3.

burglary	earthquake	$P(\text{alarm} \text{burglary}, \text{earthquake})$
true	true	(0.95,0.05)
true	false	(0.95,0.05)
false	true	(0.29,0.71)
false	false	(0.001,0.999)

7.1.3. Neural methods

Neural networks [16] excel in machine learning tasks where the input data is high-dimensional and feature engineering is hard, for instance for analysing images, video or audio, or natural language. The key to their success lies in the combination of two principles. First, they are exceptionally good in automatically extracting features from raw inputs (e.g., pixel images, or textual data). To this end, neural networks learn multiple layers of nonlinear functions that project the input data onto relatively low-dimensional latent spaces. The latent representation of an input pattern is usually called an *embedding*. Second, the use of advanced gradient based optimization techniques [17] allows these nonlinear models to be efficiently trained.

In classification tasks, neural networks typically learn a conditional probability distribution $P(Y|X)$, where Y is a random variable representing the class and X is the input pattern. As an example, a neural network can model the probability $P(Y = 1|X = \text{7})$ that the MNIST image represents the digit 1. It is worth observing that such neural network models are quite similar to a conditional probability table, such as the one in Table 1, as they both represent conditional probability distributions over a set of random variables. However, it would be impractical to represent conditional distributions over complex inputs (e.g. images) using a table, as there would be a different row for any possible raw input, e.g., for every possible 28 x 28 gray scale image. Neural networks can thus be seen as compact approximations of such tables. The approximation is possible since it is assumed that, in high dimensional spaces, similar inputs have similar output distributions. This analogy allows for an intuitive understanding of the integration of neural networks with probabilistic reasoning models by replacing conditional probability tables by neural network building blocks (cf. Section 7.3.1).

Although deep learning has been successfully applied to a wide variety of domains, there is a growing awareness of the limitations of deep learning [18]. Deep learning requires large amounts of (the right kind of) data to train the network, it provides neither justifications nor explanations, and the models are black-boxes that can neither be understood nor modified by domain experts. Although there have been attempts to demonstrate reasoning-like behaviour with deep learning [19, 20, 21], their current reasoning abilities are nowhere close to what is possible with typical high-level reasoning approaches. Even when neural networks are used to perform reasoning, they can fail badly when they have to generalize to unseen reasoning patterns [22]. This failure is for example observed when more reasoning steps are necessary during testing than during training. These types of generalizations are evaluated in datasets such as the CLUTRR dataset [23].

7.2. Integration

Let us now address the question as to how these different paradigms can be integrated in order to support both high-level reasoning and low-level perception.

We start by discussing how the field of Statistical Relational Artificial Intelligence (StarAI) [6] integrates the logic and probabilistic paradigms. This is then exemplified using the ProbLog [24] framework. Next, we will introduce how neuro-symbolic AI integrates sub-symbolic and symbolic systems.

7.2.1. Statistical Relational AI

First-order logic (programming), cf. Section 7.1.1, extends propositional logic by introducing relations. Bayesian networks and probability theory, cf. Section 7.1.2 can be seen as extending propositional logic with uncertainty. In fact, one can reason on the probability of a certain proposition given another one. Statistical Relational Artificial Intelligence (StarAI) brings the best of both worlds by extending first-order logic (programming) with uncertainty, or, equivalently, by extending probability theory with relations. One key idea, due to Poole [25] and Sato [26] is to unify the notion of a logical atom and a random variable.

StarAI models allow defining templated probabilistic models thanks to the use of first order clauses with variables. The parameters of these models can be trained on a specific set of individuals (e.g. *mary* and *john*) and then they can be used to make predictions on new individuals (e.g. *bob*). Indeed, one of the main properties of StarAI models is that they can be defined before knowing which individuals exist, by only reasoning about their potential relationships. Thus the same model can be applied to different populations and the learned regularities about the individuals in the training set can be transferred to new individuals. We will exemplify the domain of StarAI with ProbLog, a probabilistic extension of Prolog.

ProbLog ProbLog [24] lifts Prolog to a probabilistic model through the introduction of one concept: the probabilistic fact. Whereas a fact in Prolog is deterministically true, a probabilistic fact is of the form

$$p :: f$$

where f is a logical atom and p a probability. This allows us to encode the burglary alarm Bayesian network as a ProbLog program:

```
0.1::burglary.                alarm:-earthquake.
0.5::at_home(mary).           alarm:-burglary.
0.2::earthquake.              calls(X):-alarm,at_home(X).
0.4::at_home(john).
```

A ProbLog program consists of a set of probabilistic facts $\mathcal{F}$ and a set of definite clauses $\mathcal{R}$. Each ground instance $f\theta$ of a probabilistic fact f corresponds to an *independent Boolean random variable* that is true with probability p and false with probability

$1 - p$. Let us denote the set of all ground instances of probabilistic facts in $\mathcal{F}$ as $\mathcal{F}\Theta$. Every subset $F \subseteq \mathcal{F}\Theta$ defines a possible world $w_F = F \cup \{h\theta \mid \mathcal{R} \cup F \models h\theta \text{ and } h\theta \text{ is ground}\}$, that is, the world w_F is the least Herbrand model of the logic program obtained by adding F to the set of clauses $\mathcal{R}$, i.e., the set of definite clauses. For instance

$$w_{\{\text{burglary, at_home}(\text{mary})\}} = \{\text{burglary, at_home}(\text{mary})\} \cup \{\text{alarm, calls}(\text{mary})\}$$

To keep the presentation simple, we focus on the case of finitely many ground probabilistic facts, but note that the semantics is also well-defined for the countably infinite case. The probability $P(w_F)$ of such a possible world w_F is given by the product of the probabilities of the truth values of the probabilistic facts:

$$P(w_F) = \prod_{f_i \in F} p_i \prod_{f_i \in \mathcal{F}\Theta \setminus F} (1 - p_i) \quad (2)$$

For instance,

$$P(w_{\{\text{burglary, at_home}(\text{mary})\}}) = 0.1 \times 0.5 \times (1 - 0.2) \times (1 - 0.4) = 0.024$$

The probability of a ground atom q , also called *success probability* of q , is then defined as the sum of the probabilities of all worlds containing q , i.e.,

$$P(q) = \sum_{F \subseteq \mathcal{F}\Theta: q \in w_F} P(w_F) \quad (3)$$

For ease of modeling, ProbLog supports non-ground probabilistic facts as a shortcut for introducing a set of ground probabilistic facts, as well as annotated disjunctions (ADs), which are expressions of the form

$$p_1 :: h_1 ; \dots ; p_n :: h_n :- b_1, \dots, b_m.$$

where the p_i are probabilities that sum to at most one, the h_i are atoms, and the b_j are literals. The meaning of an AD is that whenever all b_i hold, the AD causes one of the heads h_j to be true, or none of them with probability $1 - \sum p_i$. Note that several of the h_i may be true at the same time if they also appear as heads of other definite clauses or ADs. This is convenient to model choices between different categorical variables, e.g. different severities of the earthquake:

$$0.4 :: \text{no_earthquake} ; 0.4 :: \text{mild_earthquake} ; 0.2 :: \text{severe_earthquake}.$$

or without explicitly representing the event of no earthquake:

$$0.4 :: \text{mild_earthquake} ; 0.2 :: \text{severe_earthquake}.$$

in which neither `mild_earthquake` nor `severe_earthquake` will be true with probability 0.4. Annotated disjunctions do not change the expressivity of ProbLog, as they can alternatively be modeled through independent facts and definite clauses; cf. [8] for technical details.

To obtain some intuitions about the probabilistic logic program representation, it is instructive to see how they can represent Bayesian networks. Consider Bayesian net-

works involving only Boolean random variables. Each node without a parent then corresponds to a probabilistic fact. Observe that both nodes without any parents in a Bayesian network and probabilistic facts in ProbLog are marginally independent. Furthermore, each entry in a conditional probability table can be mapped onto an annotated disjunction. Assume the parents of the node n are x and y . Then there would be four annotated disjunctions of the form $p_i :: n :- x_v, y_v$, where x_v and y_v are the positive or negative literals corresponding to x , resp. y . This shows that annotated disjunctions can be used to specify conditional probability tables. For example, consider the node `alarm` in Figure 3 and its parents `burglary` and `earthquake`. The CPT in Table 1 can be encoded with the following ProbLog program:

```
0.95 :: alarm :- burglary, earthquake.
0.95 :: alarm :- burglary, \+ earthquake.
0.29 :: alarm :- \+ burglary, earthquake.
0.001 :: alarm :- \+ burglary, \+ earthquake.
```

7.2.2. Neuro-symbolic AI

We now discuss how neuro-symbolic AI (NeSy) systems differ from each other along two important aspects.

7.2.2.1. Two types of neuro-symbolic AI

Neuro-symbolic AI integrates neural networks with symbolic representations, often using symbolic logic. There are at least two ways of approaching such an integration. First, there are the neural networks approaches that use logic as a regularizer. Second, there are the logical approaches that are extended with neural constructs.

Neural networks with logical aspects. These methods extend neural network models with logical aspects. One way to do this is to define logical constraints on their output. These constraints are not hard constraints (i.e. they are not guaranteed to hold). Rather, they are only used during the training, where the degree in which these constraints are not satisfied serves as an additional loss term. Thus the logic acts as a kind of regularization function during the optimization. The effect is that the logic behaves as a kind of soft constraint on the outputs, which enforces the neural methods to make predictions that adhere better to the logic. This formalism effectively encodes the logic into the parameters of the network, so that even when the logic is not explicitly present, the model should still satisfy the logical constraints more than models that were trained without these constraints. Examples of such systems include Semantic Based Regularization [27] and the Semantic Loss function [28].

Another way to extend neural networks with logical aspects is to encode the logic into the structure. This can be done through a carefully designed architecture that can perform reasoning using common differentiable operations [20, 21, 22]. One can also use the logic to define the neural network structure in a templating approach [29, 30, 31].

Logical methods with neural constructs. Whereas the other line of work extends neural network with logic, this line of work tackles the integration from the other side. These methods extend logic-based frameworks with neural constructs. The neural constructs create an interface between the logic-based framework and the neural network. These constructs generally allow the logic to evaluate neural networks in a differentiable manner such that their parameters can be optimized together with any possible parameters in the logic. This type of extension similar to how Prolog was extended into the StarAI system ProbLog through the introduction of probabilistic facts. Example of systems in this line of work include DeepProbLog [32] and NeurASP [33].

7.2.2.2. Numbers vs Booleans

Neuro-symbolic AI systems need to connect the output of the neural networks to the Boolean truth values in logic. When we consider neural networks as classifiers, they output a confidence score for each class. If the neural network are sufficiently confident, it might suffice to only select the top prediction. However, in general, and especially when the neural network still needs to be trained, this is not feasible. There are several strategies for dealing with this.

One strategy is to choose one of the outputs of the neural network, based on the confidence score. When the choices turn out to be logically inconsistent, different outputs are chosen until they are consistent. These choices can then be used as pseudo-labels to train the neural network to make this output more likely. Systems that use this approach include ABL [34] and NGS [35].

Another strategy is to use the confidence scores directly in the logic. To do this, the logical operators are turned into real-valued functions, and hence, relax the Boolean truth values to the continuous $[0, 1]$ interval. This introduces the semantics of *fuzzy logic* (or soft logic), which is mathematically grounded in the t-norm theory. For example, in the product t-norm, the truth degree of the conjunction, i.e. $t(a \wedge b)$, is equal to the product of the truth degrees of its arguments, i.e. $t(a \wedge b) = t(a)t(b)$. This approach is used in systems such as LRNN [30] and DiffLog [36].

Finally, it is also possible to interpret the confidence scores as a probability distribution, and to use a probabilistic logic to deal with the uncertainty. We will argue in the next section that this integration is very natural and has benefits over the use of a fuzzy logic. A probabilistic approach is used in systems such as DeepProbLog [32], NeurASP [33] and Semantic Loss [28].

The difference in how the probabilistic and the fuzzy methods relax the discrete logical semantics allows, in general, a computationally cheaper inference for the fuzzy approaches. However, this comes at the cost of a misalignment with the original Boolean theory, whose semantics is instead preserved by probabilistic approaches. In particular, in probabilistic methods, one defines a probability distribution over possible worlds or models. The probability of an atom or formula is computed as a sum over the probabilities of the worlds in which they are true. Their probability is as such defined only semantically and independent of the syntax. On the contrary, by relaxing the operators to continuous functions, fuzzy logics represent an alternative semantics and the truth degree of atoms and formulas may become a function of the syntax.

Example 7.2.1. Consider for example the following annotated program.

```

0.3::b.
a1 :- b.
a2 :- b,b.

```

Here, 0.3 is the label of b without any particular semantics yet. Given the idempotency of the Boolean conjunction, we would expect that the scores of both $a1$ and $a2$ to be identical since both b and $b \wedge b$ are true when b is true. In a probabilistic approach, the score is interpreted as a probability, i.e. $p(b) = 0.3$. The probability of any atom is the sum of the probabilities of all the worlds where that atom is true. It is straightforward to see that, for both $a1$ and $a2$, they are the worlds where b is true, thus $p(a1) = p(a2) = p(b) = 0.3$. On the other side, in the fuzzy settings, the score of b is interpreted as its truth degree, i.e. $t(b) = 0.3$. Let us consider the product t-norm, where $t(x \wedge y) = t(x)t(y)$, then $t(a1) = t(b) = 0.3$ while $t(a2) = t(b)t(b) = 0.09$. While this issue could be solved by choosing a different t-norm (e.g. the minimum t-norm), similar issues arise due to the relaxed logical semantics.

7.3. Neural Probabilistic Logic Programming

Based on our experience in upgrading machine learning systems towards the use of (probabilistic) logical and relational representations [37, 38], we argue that *a first desirable property of frameworks that integrate two other frameworks A and B, is to have the original frameworks A and B as a special case of the integrated one*. If A or B cannot be fully reconstructed, one clearly loses certain abilities, which is not only undesirable but which also implies that there is no true unification. Applying this property to neuro-symbolic computation implies that the existing frameworks should have both the neural and the symbolic representations as special case. When this is the case, one retains both the learning abilities of the neural component as well as the reasoning and learning abilities and the semantics of the symbolic representations. Unfortunately, this property is not satisfied by the vast majority of neuro-symbolic approaches in that they either push the symbolic representation inside the neural network (from which the logic cannot be recovered), or vice versa, apply neural learning principles to symbolic representations (and risk losing both the pure neural component and the logical semantics), as we shall show in the next section.

We also advocate a second desirable property for neuro-symbolic computation: models that learn from observed samples should be able to deal with uncertainty. Therefore, *one should not only integrate logic with neural networks in neuro-symbolic computation, but also probability*.

Although at first sight this may appear as a complication, it actually can greatly simplify the integration of neural networks with logic. The reason for this is that the probabilistic framework provides a clear optimisation criterion, namely the probability of the training examples. Real-valued probabilistic quantities are also well-suited for gradient-based training procedures, as opposed to discrete logic quantities.

We will now show how DeepProbLog achieves both of the desired properties by introducing the neural predicate as the interface between neural networks. Although the examples below are focused on reasoning with image classification, neural probabilistic

logic programming can be applied to a wide variety of domains, ranging from nature language reasoning to robotics.

7.3.1. DeepProbLog

DeepProbLog [32, 39] is a neuro-symbolic extension of ProbLog. In ProbLog, the probabilities of all random choices are explicitly specified as part of probabilistic facts or annotated disjunctions. DeepProbLog extends ProbLog to basic random choices whose probabilities are parameterized by neural networks. This is realized through the neural predicate, the interface between the probabilistic logic and the neural networks.

In [39], two types of neural predicates are introduced. We will only introduce the most general definition here, namely the neural annotated disjunction.

Definition 1 (Neural annotated disjunction). *A neural annotated disjunction (nAD) is an expression of the form*

$$nn(m_r, [X_1, \dots, X_k], O, [y_1, \dots, y_n]) :: r(X_1, \dots, X_k, O)$$

where nn is a reserved functor, m_r uniquely identifies a neural network model (i.e., its architecture as well as its trainable parameters) that defines a probability distribution $p_{m_r}(O|X = x)$ over the domain $O \in \{y_1, \dots, y_n\}$ given the input $x = [x_1, \dots, x_k]$. The $X_1, \dots, X_k$ are variables representing the inputs to the neural network, O is the output variable, the ground terms $y_1, \dots, y_n$ define the domain of the output distribution, and r is a predicate symbol. Note that the arguments of predicate r can be in an arbitrary order.

Formally, such a neural AD represents a set of ground neural ADs of the following form, one for every sequence of ground terms $x_1, \dots, x_k$ representing inputs to the neural network:

$$\begin{aligned} p_{m_r}(O = y_1 | X_1 = x_1, \dots, X_k = x_k) &:: r(x_1, \dots, x_k, y_1) ; \\ \dots ; \\ p_{m_r}(O = y_n | X_1 = x_1, \dots, X_k = x_k) &:: r(x_1, \dots, x_k, y_n) \end{aligned}$$

The neural network thus represents a discriminative classifier, which naturally maps onto an annotated disjunction for each input. For instance, in the MNIST addition example, we would specify the nAD

$$nn(m_digit, [X], Y, [0, \dots, 9]) :: digit(X, Y).$$

where m_digit is a network that classifies MNIST digits. For input image **3**, the ground nAD is

$$p_{m_digit}(Y = 0 | X = \mathbf{3}) :: digit(\mathbf{3}, 0) ; \dots ; p_{m_digit}(Y = 9 | X = \mathbf{3}) :: digit(\mathbf{3}, 9).$$

The neural network could take any shape, e.g., a convolutional network for image encoding, a recurrent network for sequence encoding, etc. However, its output layer, which feeds the corresponding neural predicate, needs to be normalized.

In Section 7.1.3, we discussed how neural network classifiers naturally encode conditional probability distributions and that these could represent efficient approximations of conditional probability tables over high-dimensional inputs (like images). Moreover, in Section 7.2.1, we described a one-to-one mapping between ProbLog annotated disjunctions and entries of conditional probability tables in Bayesian networks. Neural annotated disjunctions in DeepProbLog exploit these two analogies to introduce neural network capabilities (i.e. image recognition, natural language processing) into (probabilistic) logic programming.

The semantics of a DeepProbLog program with respect to a fixed set of possible inputs to every neural network used in the neural facts and neural ADs is the semantics of the ProbLog program that replaces each neural fact and neural AD by the corresponding sets of ground neural facts and ground neural ADs for these inputs.

Inference in DeepProbLog We now briefly describe how inference happens in DeepProbLog. It is largely identical to inference in ProbLog and only needs to be extended with the evaluation of neural networks. ProbLog inference proceeds in four steps. The first step is the grounding step, in which the logic program is grounded with respect to the query. This step uses backward reasoning to determine which ground clauses are relevant to derive the truth value of the query, and may perform additional logical simplifications that do not affect the query's probability.

The second step rewrites the ground logic program into a formula in propositional logic that defines the truth value of the query in terms of the truth values of probabilistic facts. We can calculate the query success probability by performing *weighted model counting* (WMC) on this logic formula (cf. [40]). However, performing WMC on this logical formula directly is not efficient.

The third step is knowledge compilation [41]. During this step, the logic formula is transformed into a compiled structure that allows for efficient weighted model counting.

The fourth and final step transforms this compiled structure into an arithmetic circuit (AC). This is done by putting the probabilities of the probabilistic facts or their negations on the leaves, replacing the OR nodes with addition and the AND nodes by multiplication. The WMC is then calculated with an evaluation of the AC.

The only change required for DeepProbLog inference is that we need to instantiate nADs and neural facts to regular ADs and probabilistic facts. This is done in two steps. During grounding, we obtain ground nADs and ground neural facts with a symbolic representation of the probabilities. In a separate step after grounding, the concrete parameters are determined by making a forward pass on the relevant neural network with the ground input.

We exemplify inference with the following program that defines the addition of two MNIST digits.

```
nn(m_digit, [X], Y, [0...9]) :: digit(X,Y).
addition(X,Y,Z) :- digit(X,N1), digit(Y,N2), Z is N1+N2.
```

We now pose DeepProbLog the following query: `addition([0],[7],1)`. First, the program is grounded on this query, which results in the following relevant ground program.

```
nn(m_digit, [0], 0) :: digit(0,0); nn(m_digit, [7], 1) :: digit(7,1).
```

```
nn(m_digit, [7], 0)::digit([7], 0); nn(m_digit, [7], 1)::digit([7], 1).
addition([0], [7], 1) :- digit([0], 0), digit([7], 1).
addition([0], [7], 1) :- digit([0], 1), digit([7], 0).
```

This ground program gets rewritten to the following formula : $(\text{digit}([0], 0) \wedge \text{digit}([7], 1)) \vee (\text{digit}([0], 1) \wedge \text{digit}([7], 0))$. This formally is then compiled and transformed into an arithmetic circuit, shown in Figure 4. The bottom of the diagram shows the neural network evaluation on the two images $[0]$ and $[7]$. The output of these probabilities are then combined through a bottom-up evaluation of the AC to derive the probability of the query.

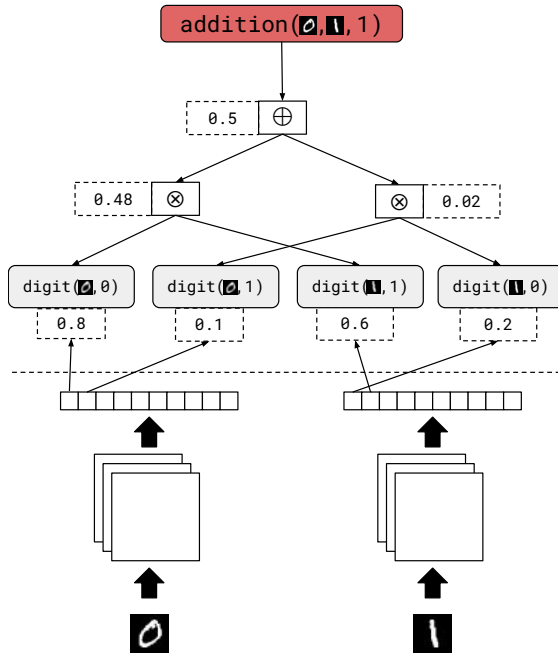


Figure 4. The arithmetic circuit for the MNIST addition example on the query $\text{addition}([0], [7], 1)$. Figure adapted from [39].

Learning in DeepProbLog In contrast to the earlier approach for ProbLog parameter learning in this setting by [42], DeepProbLog uses gradient based learning instead of expectation maximization. This allows for seamless integration with the training of neural network parameters. The key insight here is that the arithmetic circuit used for inference can be used for gradient computations as well. This AC is a differentiable structure, as it is composed of addition and multiplication operations.

DeepProbLog relies on the automatic differentiation capabilities of ProbLog to derive these gradients. More specifically, to compute the gradient with respect to the probabilistic logic program part, DeepProbLog uses aProbLog [43], a generalization of the ProbLog language and inference to arbitrary commutative semirings, including the gradient semiring [44]. Whereas ProbLog is confined to only calculating probabilities, the use of this gradient semiring in aProbLog allows the system to calculate the gradient

alongside the probabilities. This done by replacing the normal operations on probabilities in the arithmetic circuit:

$$p_1 \oplus p_2 = p_1 + p_2$$

$$p_1 \otimes p_2 = p_1 p_2$$

by simultaneous operations on probabilities and gradients, effectively calculating the gradients alongside the query probability.

$$(p_1, \nabla p_1) \oplus (p_2, \nabla p_2) = (p_1 + p_2, \nabla p_1 + \nabla p_2)$$

$$(p_1, \nabla p_1) \otimes (p_2, \nabla p_2) = (p_1 p_2, p_2 \nabla p_1 + p_1 \nabla p_2)$$

Preserving the base paradigms. We now show how the different base paradigms are preserved. ProbLog itself retains its base paradigms (probabilistic graphical models and logic). If no probabilistic facts are included in ProbLog, it essentially becomes Prolog and supports purely logical reasoning. As discussed in Section 7.2.1, probabilistic graphical models such as Bayesian networks can easily be represented in ProbLog. As DeepProbLog is an extension of ProbLog that adds the neural predicate, yet doesn't remove anything from this base framework, it becomes plain ProbLog when no neural predicates are present in the program. As such, it retains the same base paradigms as ProbLog.

The neural base paradigm is also preserved in DeepProbLog. For example, a classifier can be represented in DeepProbLog as a program with only a single neural predicate. Other neural methods such as embedding based methods (e.g. TransE, DistMult, ...) can also be represented as shown below in Example 3.

7.3.2. DeepProbLog examples

We will now use the DeepProbLog framework to showcase several desirable properties of neuro-symbolic integrations.

Example 1: Logical reasoning with DeepProbLog To show that DeepProbLog supports both logical reasoning and deep learning, we extend the classic learning task on the MNIST dataset [45] to a more complex problem that requires reasoning. Instead of using labeled single digits, we train on pairs of images, labeled with the sum of the individual labels. The DeepProbLog program consists of the clause

$$\text{addition}(X, Y, Z) :- \text{digit}(X, X2), \text{digit}(Y, Y2), Z \text{ is } X2 + Y2$$

and a neural AD for the `digit/2` predicate, which classifies an MNIST image. We compare to a CNN baseline classifying the two images into the 19 possible sums. Results shown in Table 2 indicate that performing reasoning and learning at the same time outperforms methods that can only learn, namely a neural baseline.

Example 2: Probabilistic reasoning with DeepProbLog To show the benefit of integrating the probabilistic paradigm, we consider an extension to the previous experiment. The extension is that a fraction of the labels is replaced by a number uniformly selected between 0 and 18. If we model this label noise, DeepProbLog can reason probabilistically about each experiment whether its label is correct or not, and train the neural network

Table 2. The accuracy of a neural baseline and DeepProbLog on the addition task for different number of training examples.

Model	Number of training examples		
	30 000	3 000	300
Neural Baseline	93.46 ± 0.49	78.32 ± 2.14	23.64 ± 1.75
DeepProbLog	97.20 ± 0.45	92.18 ± 1.57	67.19 ± 25.05

Table 3. The accuracy on the noiseless test set for MNIST addition experiment trained with label noise.

	Fraction of noisy labels					
	0.0	0.2	0.4	0.6	0.8	1.0
Baseline	93.46	87.85	82.49	52.67	8.79	5.87
DeepProbLog	97.20	95.78	94.50	92.90	46.42	0.88
DeepProbLog w/ explicit noise	96.64	95.96	95.58	94.12	73.22	2.92
$p(\text{noisy})$	0.000	0.212	0.415	0.618	0.803	0.985

accordingly. In the code below, we have a probabilistic fact `noisy`. This fact is used to split the `addition/3` predicate into two cases. If `noisy` is false, then the addition happens as in Example 1. If it's true, then we assume that `Z` is uniformly sampled between 0 and 19. The probability of the `noisy` fact is a learnable parameter, which allows the model to estimate the fraction of label noise in the training data.

```
nn(classifier, [X], Y, [0 ..9]) :: digit(X,Y).
t(0.2) :: noisy.

1/19 :: uniform(X,Y,0) ; ... ; 1/19 :: uniform(X,Y,18).

addition(X,Y,Z) :- noisy, uniform(X,Y,Z).
addition(X,Y,Z) :- \noisy, digit(X,N1), digit(Y,N2), Z is N1+N2.
```

We compare three models: a CNN baseline, the DeepProbLog model from Example 1, and the DeepProbLog model shown above. Table 3 shows the final accuracy on the noiseless test set. For the DeepProbLog model that explicitly models the noise, we also report the value of this parameter, which indicates how much noise the DeepProbLog model believes there is. As we can see, probabilistic reasoning makes DeepProbLog robust and also gives it the ability to detect the amount of label noise in the dataset.

Example 3: DeepProbLog as a neuro-symbolic programming language In this experiment, we showcase that DeepProbLog is a neuro-symbolic programming language by manipulating embeddings in a declarative, yet differentiable way. A straightforward way to obtain image embeddings, is by optimizing a clustering objective. That is, we minimize the distance between the embeddings of images of the same number, while maximizing the distance between the embedding of images of different numbers. As a similarity metric, the radial basis function $\varphi(x,y)$ (RBF) can be used (cf. the neural theorem prover [31]). This can be included in DeepProbLog as a predicate `rbf/2` that succeeds with probability $p = \varphi(x,y) = e^{-||x-y||_2}$ where x and y are embeddings that are the ar-

guments of the predicate. This can be used to implement a predicate that determines the similarity between two images:

```
similar(I1,I2) :-
    %Encode images I1 and I2 into E1 and E2
    cnn_encode(I1,E1), cnn_encode(I2,E2),
    rbf(E1,E2).
```

Here, `cnn_encode/2` unifies the second argument with an embedding of the first argument calculated by a convolutional neural network. The `rbf/2` acts as a constraint that leads to clustering in embedding space.

As a proof of concept of the wider possibilities of implicitly training embeddings through DeepProbLog, we extend the clustering objective towards inducing an order relation in embedding space. The successor relationship between two MNIST images is defined as `successor(I1,I2,R)` where `I1` and `I2` are MNIST images, and `R` is the difference of the digits represented by the two images (e.g. `successor(5,3,-2)`). We model the successor relationship as a translation (cf. TransE [46]). We learn an embedding r_s of the successor relationship in this embedding space such that we minimize the distance between $e_x + nr_s$ and e_y , where e_x and e_y are the embeddings of the two images and n is the difference of the image labels.

```
successor(I1,I2,N) :-
    %Encode images I1 and I2 into E1 and E2
    cnn_encode(I1,E1), cnn_encode(I2,E2),
    %Embed the successor relation into embedding S
    embed(successor, S),
    %The relation should not be trivial (i.e. 0)
    \+rbf(S,0).
    %E = E1 + N*S should be similar to E2
    mul(S,N,S2), add(E1,S2,E),
    rbf(E,E2).
```

If $n = 0$, then the `successor/3` relationship becomes identical to the `similarity/2` relationship. We also specify that r_s should not be zero to avoid collapsing on a trivial solution. Note that this constraint on the size of the embedding can be seen as a form of regularization defined in the logic. We define a predicate `embed/2` that unifies the second argument with a learnable embedding for the first argument. `mul/3` is the product between a scalar and a vector and `add/3` an element-wise addition. These create the mathematical definition of the successor relationship. The number 0 is used as a zero vector of the same dimension as the embeddings.

Figure 5 shows the embeddings of the MNIST test set (crosses) and the mean of each of these images, grouped by label (solid dots). It shows that the model has learned to embed images into clusters corresponding to the labels of these images, which are positioned sequentially along the embedding dimension. As can be seen, the cluster centers can be (approximately) linearly mapped to the actual image labels.

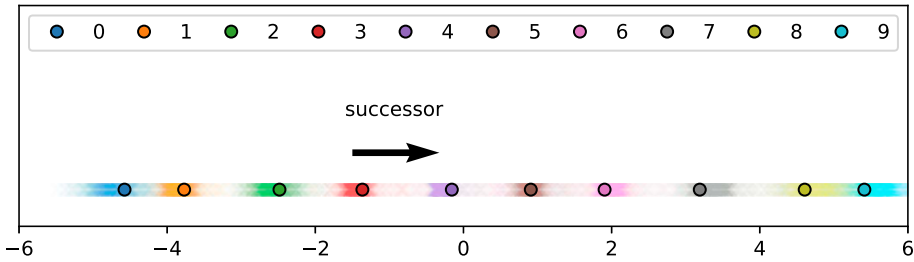


Figure 5. A visualization of the embedding space learned in Example 3. Individual MNIST image embedding are shown in crosses, with the mean for each label as a solid dot. The direction of the successor relation is marked by the arrow.

7.4. Conclusions and open challenges

In this chapter, we discussed the base paradigms that make up the vast majority of neuro-symbolic systems. We then discussed the similarities and differences of two paradigms that integrate reasoning and learning, namely statistical relational AI and neuro-symbolic AI. Next, we discussed the desirable properties of neuro-symbolic systems. Firstly, a framework that is the integration of two frameworks should retain either framework as a special case. Otherwise, certain abilities have been lost in the integration. Secondly, neuro-symbolic systems should integrate a logical, neural and a probabilistic base paradigm. We have showcased this for the neural probabilistic logic programming paradigm with the DeepProbLog framework.

However, there are still open challenges that need to be solved by the NeSy community. One challenge to overcome is the scalability of neuro-symbolic systems. Especially the inference of systems that have a separate logical inference step can be prohibitively expensive. Including probabilistic inference can make this problem even worse. Although some progress has been made in this direction by investigating approximate inference, there is not yet a solution that can be applied to arbitrary settings. Furthermore, a lot of effort has gone into making the training of (large) neural networks as efficient as possible, maximally utilizing the parallelization capabilities of hardware. The same considerations will have to be made for neuro-symbolic systems if they are to be made universally applicable.

Another open challenge is performing structure learning in neuro-symbolic systems. Although some progress has been made in this direction, the possibilities of these systems are still not on the level of structure learning seen in pure logic programming / StarAI. Introducing neural networks into the structure learning problem further complicates this setting as these neural networks might also have to be trained themselves.

A final challenge directly related to the previous challenges is that the field is not yet mature enough to be applied to a showcase application. This is partly due to the limited scalability of certain neuro-symbolic methods, but also due to the fact that many systems assume that the logical structure is given by the user. However, this is not viable for all areas of application, and a robust system that can discover (part of) the logical structure would be essential.

References

- [1] Kahneman D. Thinking, fast and slow. Farrar, Straus and Giroux New York; 2011.
- [2] Garcez AsD, Broda KB, Gabbay DM. Neural-symbolic learning systems: foundations and applications. Springer Science & Business Media; 2012.
- [3] Garcez Ad, Besold TR, De Raedt L, et al. Neural-symbolic learning and reasoning: contributions and challenges. In: 2015 AAAI Spring Symposium Series; 2015.
- [4] Hammer B, Hitzler P. Perspectives of neural-symbolic integration. Vol. 8. Springer Heidelberg;; 2007.
- [5] De Raedt L, Dumancic S, Manhaeve R, et al. From statistical relational to neuro-symbolic artificial intelligence. In: IJCAI; 2020.
- [6] De Raedt L, Kersting K, Natarajan S, et al. Statistical relational artificial intelligence: Logic, probability, and computation. Synthesis Lectures on Artificial Intelligence and Machine Learning. 2016;10(2):1–189.
- [7] Getoor L, Taskar B. Introduction to statistical relational learning. MIT press; 2007.
- [8] De Raedt L, Kimmig A. Probabilistic (logic) programming concepts. Machine Learning. 2015;100(1):5–47.
- [9] Flach P. Simply logical: intelligent reasoning by example. John Wiley & Sons, Inc.; 1994.
- [10] Pearl J. Probabilistic reasoning in intelligent systems: networks of plausible inference. Morgan Kaufmann Publishers Inc.; 1988.
- [11] Van Gelder A, Ross KA, Schlipf JS. The well-founded semantics for general logic programs. Journal of the ACM. 1991;38(3):620–650.
- [12] Koller D, Friedman N. Probabilistic graphical models: principles and techniques. MIT press; 2009.
- [13] Robert CP, Casella G. Monte carlo statistical methods. Springer; 2004. Springer Texts in Statistics.
- [14] Wainwright MJ, Jordan MI. Graphical models, exponential families, and variational inference. Now Publishers Inc; 2008.
- [15] Darwiche A. Modeling and reasoning with bayesian networks. Cambridge university press; 2009.
- [16] Goodfellow I, Bengio Y, Courville A. Deep learning. MIT Press; 2016. <http://www.deeplearningbook.org>.
- [17] Kingma DP, Ba J. Adam: A method for stochastic optimization. In: International Conference on Learning Representations (ICLR); 2015. p. 1–13.
- [18] Marcus G. Deep learning: A critical appraisal. arXiv preprint arXiv:180100631. 2018;.
- [19] Santoro A, Raposo D, Barrett DG, et al. A simple neural network module for relational reasoning. In: NIPS; 2017.
- [20] Makni B, Hendler JA. Deep learning for noise-tolerant RDFS reasoning. Semantic Web. 2019; 10(5):823–862.
- [21] Ebrahimi M, Sarker MK, Bianchi F, et al. Neuro-symbolic deductive reasoning for cross-knowledge graph entailment. In: Martin A, Hinkelmann K, Fill H, et al., editors. Proceedings of the AAAI 2021 Spring Symposium on Combining Machine Learning and Knowledge Engineering (AAAI-MAKE 2021), Stanford University, Palo Alto, California, USA, March 22-24, 2021; (CEUR Workshop Proceedings; Vol. 2846). CEUR-WS.org; 2021.
- [22] Ebrahimi M, Eberhart A, Hitzler P. On the capabilities of pointer networks for deep deductive reasoning. CoRR. 2021;abs/2106.09225. Available from: <https://arxiv.org/abs/2106.09225>.
- [23] Sinha K, Sodhani S, Dong J, et al. Clutrr: A diagnostic benchmark for inductive reasoning from text. Empirical Methods of Natural Language Processing (EMNLP). 2019;.
- [24] De Raedt L, Kimmig A, Toivonen H. ProbLog: A probabilistic Prolog and its application in link discovery. In: IJCAI; 2007. p. 2462–2467.
- [25] Poole D. Probabilistic horn abduction and bayesian networks. Artif Intell. 1993;64(1):81–129. Available from: [https://doi.org/10.1016/0004-3702\(93\)90061-F](https://doi.org/10.1016/0004-3702(93)90061-F).
- [26] Sato T. A statistical learning method for logic programs with distribution semantics. In: Sterling L, editor. Logic Programming, Proceedings of the Twelfth International Conference on Logic Programming, Tokyo, Japan, June 13-16, 1995. MIT Press; 1995. p. 715–729.
- [27] Diligenti M, Gori M, Sacca C. Semantic-based regularization for learning and inference. Artificial Intelligence. 2017;244:143–165.
- [28] Xu J, Zhang Z, Friedman T, et al. A semantic loss function for deep learning with symbolic knowledge. In: Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmssmässan, Stockholm, Sweden, July 10-15, 2018; 2018. p. 5498–5507.

- [29] Andreas J, Rohrbach M, Darrell T, et al. Neural module networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2016. p. 39–48.
- [30] Sourek G, Aschenbrenner V, Zelezny F, et al. Lifted relational neural networks: Efficient learning of latent relational structures. *Journal of Artificial Intelligence Research*. 2018;62:69–100.
- [31] Rocktäschel T, Riedel S. End-to-end differentiable proving. In: *Advances in Neural Information Processing Systems*; Vol. 30; 2017. p. 3788–3800.
- [32] Manhaeve R, Dumancic S, Kimmig A, et al. Deepproblog: Neural probabilistic logic programming. In: *NeurIPS*; 2018.
- [33] Yang Z, Ishay A, Lee J. Neurasp: Embracing neural networks into answer set programming. In: Bessiere C, editor. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*. ijcai.org; 2020. p. 1755–1762. Available from: <https://doi.org/10.24963/ijcai.2020/243>.
- [34] Dai WZ, Xu Q, Yu Y, et al. Bridging machine learning and logical reasoning by abductive learning. In: Wallach H, Larochelle H, Beygelzimer A, et al., editors. *Advances in Neural Information Processing Systems*; Vol. 32. Curran Associates, Inc.; 2019. Available from: <https://proceedings.neurips.cc/paper/2019/file/9c19a2aa1d84e04b0bd4bc888792bd1e-Paper.pdf>.
- [35] Li Q, Huang S, Hong Y, et al. Closed loop neural-symbolic learning via integrating neural perception, grammar parsing, and symbolic reasoning. In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event; (Proceedings of Machine Learning Research; Vol. 119)*. PMLR; 2020. p. 5884–5894. Available from: <http://proceedings.mlr.press/v119/li20f.html>.
- [36] Si X, Raghothaman M, Heo K, et al. Synthesizing datalog programs using numerical relaxation. In: *IJCAI*; 2019. Available from: <https://doi.org/10.24963/ijcai.2019/847>.
- [37] De Raedt L. *Logical and relational learning*. Springer; 2008.
- [38] Muggleton S, De Raedt L, Poole D, et al. Ilp turns 20. *Machine learning*. 2012;86(1):3–23.
- [39] Manhaeve R, Dumančić S, Kimmig A, et al. Neural Probabilistic Logic Programming in Deep-ProbLog. *Artificial Intelligence*. 2021;298:103504. Available from: <https://www.sciencedirect.com/science/article/pii/S0004370221000552>.
- [40] Fierens D, Van den Broeck G, Renkens J, et al. Inference and learning in probabilistic logic programs using weighted Boolean formulas. *Theory and Practice of Logic Programming*. 2015;15(3):358–401.
- [41] Darwiche A, Marquis P. A knowledge compilation map. *Journal of Artificial Intelligence Research*. 2002;17:229–264.
- [42] Gutmann B, Kimmig A, Kersting K, et al. Parameter learning in probabilistic databases: A least squares approach. In: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*; Springer; 2008. p. 473–488.
- [43] Kimmig A, Van den Broeck G, De Raedt L. An algebraic Prolog for reasoning about possible worlds. In: *AAAI*; 2011.
- [44] Eisner J. Parameter estimation for probabilistic finite-state transducers. In: *Proceedings of the 40th annual meeting on Association for Computational Linguistics*; Association for Computational Linguistics; 2002. p. 1–8.
- [45] Lecun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition. In: *Proceedings of the IEEE*; 1998. p. 2278–2324.
- [46] Bordes A, Usunier N, Garcia-Duran A, et al. Translating embeddings for modeling multi-relational data. In: *Advances in neural information processing systems*; 2013. p. 2787–2795.

Chapter 8

A Constraint-Based Approach to Learning and Reasoning

Michelangelo Diligenti, DIISM, University of Siena, Italy

Francesco Giannini, DIISM, University of Siena, Italy

Marco Gori, DIISM, University of Siena, Italy

Marco Maggini, DIISM, University of Siena, Italy

Giuseppe Marra, KU Leuven, Belgium

8.1. Introduction

Neural-symbolic approaches aim at bridging the gap between sub-symbolic machine learning, which can efficiently learn complex classification functions starting from supervised data, and symbolic approaches, which can perform logic reasoning and consider high-level relational dependencies in a certain domain of discourse.

A limitation of sub-symbolic approaches like artificial neural networks [1, 2] is their dependency on the amount and quality of the training data together with their inability to explicitly represent relevant input or output structures, which have been shown to bring significant benefits in many challenging applications like image segmentation tasks [3]. On the other hand, symbolic approaches based on probabilistic logic reasoners [4, 5, 6] can directly express relational structures and can perform exact or approximate inference in presence of uncertainty. However, these methods have so far shown limited scalability and they generally lack an integrated and effective procedure to process feature-based representations.

This chapter presents a general approach to integrate reasoning and learning that is based on a translation of the available prior knowledge into an undirected graphical model, whose potentials can accommodate dependencies among random variables by means of a set of trainable functions, like the ones computed by neural networks. This provides a general neural-symbolic platform that can bridge the advantages of both the symbolic and sub-symbolic worlds. Whereas exact inference is intractable within this model, this chapter shows how different tractable models can be derived from this common ancestor depending on different assumptions and approximations. In particular, this chapter shows how to derive Semantic-based Regularization (SBR) [7], Deep Logic

Models (DLM) [8] and Relational Neural Machines (RNM) [9]. SBR is an extremely scalable neural-symbolic model with the disadvantage that the way the knowledge is employed is decided before-hand and not co-trained during learning. DLM preserves the scalability of SBR, whereas providing a flexible employment of the logic knowledge by co-training the parameters of the reasoner during the learning procedure, with the disadvantage that scalability is based on strong statistical assumptions. Finally, unlike SBR and DLM, RNMs are able to perfectly replicate the effectiveness of training from supervised data of standard deep architectures, while preserving the same generality and expressivity of Markov Logic Networks (MLN) [10], when considering pure reasoning on symbolic data. The bonding is very general as any (deep) learner can be integrated and any output or input structure can be expressed. Since, exact inference within an RNM is generally intractable, different factorizations can be used to increase the scalability of the approach.

The outline of the chapter is the following: Section 8.2 reviews some prior work on the field and Section 8.3 introduces the general setting and the notation used throughout this chapter, together with the explicit problem statement. Section 8.4 presents the main characteristics of the considered basic model. Sections 8.5, 8.6, 8.7 derive the three different implementations corresponding to SBR, DLM and RNM models, respectively. Finally Section 8.8 draws some conclusions and provides an analysis of the open problems within this field.

8.2. Related Work

Probabilistic reasoners define a more flexible inference process than their counterparts based on standard logic. For example, Markov Logic Networks (MLN) [10] translate a First-Order Logic (FOL) knowledge base, grounded over a finite set of constants, into an undirected graphical model, used to perform parameter learning and inference. Probabilistic Soft Logic (PSL) [11] overtakes some of the scalability limitations of MLNs by defining a tractable fraction of FOL and relaxing inference using fuzzy logic. The model proposed in this chapter can be considered an extension of MLNs, where the potential functions may also depend on sub-symbolic representations of data, by using neural architectures. In a similar way, if the potentials are relaxed to continuous values by using fuzzy logic operators, the proposed model also extends PSL to a neuro-symbolic setting, where the input representations are considered as well.

Neural-symbolic methods [12] bridge the reasoning capabilities of logic reasoners with the ability of sub-symbolic systems to deal with the feature-based representations that typically represent sensory inputs like video, images or sounds. Distillation methods like teacher-student setups [13] and Logic Tensor Networks [14] inject logic knowledge into the network weights by enforcing the knowledge on the predictor outputs. In spite of their simplicity, these methods are powerful when the logic knowledge should be applied to the entire output space in a flat fashion, unlike what needed to obtain a full probabilistic reasoning process. However, this class of approaches considers the reasoning layer as frozen, without allowing to jointly train its parameters. This is a big limitation, as these methods work better only with hard constraints, while they are less suitable in presence of reasoning under uncertainty.

DeepProbLog [15] is a neural-symbolic approach extending the popular ProbLog [5] probabilistic programming framework with the integration of deep learners. Deep-

ProbLog requires the output from the neural networks to be probabilities and an independence assumption among atoms in the logic is required to make inference tractable. This is a strong restriction, since the sub-symbolic layer often consists of several neural layers sharing weights. Semantic Loss [16] instead translates the desired output structure representing logic knowledge via the definition of a loss, which can be employed by any differentiable learner. This solution is very flexible but their application has been limited to small tasks where exact inference is possible. Lifted Relational Neural Networks [17] and Neural Theorem Provers (NTP) [18, 19] realize a soft forward or backward chaining via a gradient based scheme. In particular, NTPs construct an end-to-end differentiable architecture capable of proving queries using sub-symbolic vector representations within logic programs. NTPs allow soft-unification among symbols using their tensorial representation to allow a more flexible matching. However, this introduces scalability issues as reasoning paths can not be easily pruned without relying on heuristics. Neural LP [20] learns rules using differentiable operators defined in TensorLog [21], that is a framework to get advantage of deep learning computational power to perform probabilistic logic reasoning.

The integration of deep learning with Conditional Random Fields (CRFs) [22] is also an alternative approach to enforce some structure on the network output. This approach has been proved to be quite successful on sequence labeling for natural language processing tasks. This methodology can be seen as a special case of the more general methodology presented in this chapter, when the potential functions are used to represent the correlations among consecutive outputs of a recurrent deep network. Deep Structured Models [3, 23] use a similar graphical model to bridge the sensory and semantic levels. However, they have mainly focused on imposing correlations on the output layer, without any focus on logic reasoning. Furthermore, the proposed methodology transforms the training process into an iterative constrained optimization problem, which is very different from the approximation of the partition function used in Deep Structured Models.

8.3. The Neural–Symbolic Setting

Let us consider a First-Order Logic (FOL) language where, $\mathcal{C}$ and $\mathcal{R}$ denote the set of constants and predicates (or relations), respectively. We use the notation r/k to refer to a relation r of arity k . For simplicity, throughout the chapter we assume that the set of constants $\mathcal{C}$ will always coincide with the domain of discourse.

Traditionally, in symbolic methods, each constant $e \in \mathcal{C}$ is represented by an identifier, e.g. the string *Simba*. Here we consider the case in which constants are also associated with a sub-symbolic representation, e.g. a vector of features $\mathbf{x}_{Simba}$, and the set of the sub-symbolic representations of all the constants in the language is expressed as $\mathbf{x} = \{\mathbf{x}_e : e \in \mathcal{C}\}$. A tuple of constants $g = (e_1, \dots, e_n)$ has a sub-symbolic representation as the concatenation of the vectors of each of the constants of the tuple, i.e. $\mathbf{x}_g = \mathbf{x}_{(e_1, \dots, e_n)} = \text{concat}(\mathbf{x}_{e_1}, \dots, \mathbf{x}_{e_n})$. For example, let us consider the set of constants indicating the animals $\mathcal{C} = \{Simba, Mufasa, Alex\}$. Each constant can be accompanied with an image of the corresponding animal. Then, we have: $\mathbf{x}_{Simba} =$ , $\mathbf{x}_{(Simba, Mufasa)} =$ , $\mathbf{x} = \{\mathbf{x}_{Simba}, \mathbf{x}_{Mufasa}, \mathbf{x}_{Alex}\}$.

The Herbrand Base (HB) is the set of all the atoms (or facts) that can be built with the constants and predicates. An assignment $\mathbf{y}$ of truth values to the Herbrand Base cor-

responds to an *interpretation* or a possible world. Given an atom $a = p(e_1, \dots, e_k)$, where $e_1, \dots, e_k \in \mathcal{C}$ and $p \in \mathcal{R}$, we refer to y_a as the assignment in $\mathbf{y}$ corresponding to atom a . Moreover, we define $\mathbf{y}_s$ a subset of $\mathbf{y}$ corresponding to the set $s \subseteq HB$.

Assuming to have available the following predicates: $\mathcal{R} = \{\text{savanna}/1, \text{sameLoc}/2\}$, the Herbrand Base w.r.t. the animal set, is the set of facts:

$$\begin{aligned} HB = \{ & \text{savanna}(\text{Simba}), \text{savanna}(\text{Mufasa}), \text{savanna}(\text{Alex}), \\ & \text{sameLoc}(\text{Simba}, \text{Simba}), \text{sameLoc}(\text{Simba}, \text{Mufasa}), \text{sameLoc}(\text{Simba}, \text{Alex}), \\ & \text{sameLoc}(\text{Mufasa}, \text{Simba}), \text{sameLoc}(\text{Mufasa}, \text{Mufasa}), \text{sameLoc}(\text{Mufasa}, \text{Alex}), \\ & \text{sameLoc}(\text{Alex}, \text{Simba}), \text{sameLoc}(\text{Alex}, \text{Mufasa}), \text{sameLoc}(\text{Alex}, \text{Alex}) \}. \end{aligned}$$

A possible world is given by the following assignment:

$$\begin{aligned} \mathbf{y} = \{ & \text{savanna}(\text{Simba}) = 1, \text{savanna}(\text{Mufasa}) = 1, \text{savanna}(\text{Alex}) = 1, \\ & \text{sameLoc}(\text{Simba}, \text{Simba}) = 1, \text{sameLoc}(\text{Simba}, \text{Mufasa}) = 1, \\ & \text{sameLoc}(\text{Simba}, \text{Alex}) = 0, \text{sameLoc}(\text{Mufasa}, \text{Simba}) = 1, \\ & \text{sameLoc}(\text{Mufasa}, \text{Mufasa}) = 1, \text{sameLoc}(\text{Mufasa}, \text{Alex}) = 0, \\ & \text{sameLoc}(\text{Alex}, \text{Simba}) = 0, \text{sameLoc}(\text{Alex}, \text{Mufasa}) = 0, \\ & \text{sameLoc}(\text{Alex}, \text{Alex}) = 1 \}. \end{aligned}$$

Finally, a knowledge base KB is composed of a set of FOL formulas. Let $c(v_1, \dots, v_m)$ be a logic formula (or *constraint*) with m quantified variables $v_1, \dots, v_m$. We define the *grounding* $\mathbf{y}_{c,g}$ as the subset of the interpretation $\mathbf{y}$ containing only the assignments to atoms in c with substitution g to $(v_1, \dots, v_m)$, where g is an m -tuple of constants. Then, $G_c(\mathbf{y})$ is the set of groundings of c for all possible substitutions g .

If the following KB is available:

$$\begin{aligned} KB = \{ & c_1(x) = \forall x \text{ lion}(x) \rightarrow \text{savanna}(x) \vee \text{zoo}(x), \\ & c_2(x, y) = \forall x \forall y \text{ sameLoc}(x, y) \rightarrow (\text{savanna}(x) \leftrightarrow \text{savanna}(y)) \} \end{aligned}$$

Then, $g = (\text{Simba}, \text{Mufasa})$ is a valid substitution for c_2 : $c_2(\text{Simba}, \text{Mufasa}) = \text{sameLoc}(\text{Simba}, \text{Mufasa}) \rightarrow (\text{savanna}(\text{Simba}) \leftrightarrow \text{savanna}(\text{Mufasa}))$.

Finally, given the previous interpretation $\mathbf{y}$:

$$\mathbf{y}_{c_2,g} = \{ \text{sameLoc}(\text{Simba}, \text{Mufasa}) = 1, \text{savanna}(\text{Simba}) = 1, \text{savanna}(\text{Mufasa}) = 1 \}$$

Problem Statement 1 specifies the class of learning tasks that can be expressed by the proposed framework. This statement is general enough to cover multiple cases of interest of both the deep learning and the statistical relational AI communities:

- **Classification.** The set of constants is the set of input patterns (e.g. images) and their correspondent features $\mathbf{x}$, while the set of relations is the set of classes, i.e. unary predicates. Finding the interpretation $\mathbf{y}_q$ corresponds to find a class assign-

PROBLEM STATEMENT 1

Given:

- a set of constants $\mathcal{C}$ and a set of relations $\mathcal{R}$, defining atom assignments $\mathbf{y}$ for each element of the Herbrand Base;
- a knowledge base KB ;
- the set $\mathbf{x}$ of sub-symbolic representations for some constants in $\mathcal{C}$;
- a subset $e \subset HB$ (i.e. the *evidence*), whose interpretation $\mathbf{y}_e$ is known;
- a subset $q \subseteq HB$ (i.e. the *query*), whose interpretation $\mathbf{y}_q$ is unknown

Find:

- the interpretation $\mathbf{y}_q$ of q

ment for the input patterns of interest. For example, in the *Animal* domain, we can define a classification task where, $\mathcal{C} = \{Simba, Mufasa, Alex\}$, $\mathbf{x} = \{\text{img1}, \text{img2}, \text{img3}\}$ and $\mathcal{R} = \{lion/1, zebra/1, cat/1, \dots\}$. The set of queries q is the set of class assignments, e.g. $q = \{lion(Simba), zebra(Simba), cat(Simba), \dots\}$ to determine.

- **Link Prediction.** In knowledge graphs [24], the constants correspond to the entities, each represented by a node in the graph, while the relations form the multi-relational edges among entities. The evidence $\mathbf{y}_e$ is the set of known existing edges, and finding the interpretation of the query variables $\mathbf{y}_q$ corresponds to discovering the existence of unknown relations (i.e. links) between constants. In the *Animal* example, $\mathcal{C} = \{Simba, Mufasa, Alex, Pumba\}$, $\mathcal{R} = \{sameLoc/2\}$, $\mathbf{y}_e = \{sameLoc(Mufasa, Pumba), sameLoc(Simba, Pumba)\}$. The target query is the link $q = \{sameLoc(Simba, Mufasa)\}$.
- **Automated Reasoning.** In this case, an input logic language, a set of evidences $\mathbf{y}_e$ and a knowledge base KB are provided. Finding the interpretation $\mathbf{y}_q$ corresponds to discovering the truth values of the query facts that satisfy the set of logic formulas and the evidence $\mathbf{y}_e$. For example, extending the link prediction task, we can add a knowledge base like $KB = \{\forall x \forall y \forall z : sameLoc(x, y) \wedge sameLoc(y, z) \rightarrow sameLoc(x, z)\}$.
- **Neural-symbolic reasoning.** All the elements defined by the statement 1 are employed, so that finding interpretations $\mathbf{y}_q$ in this setting means answering queries by integrating both feature based classification and (relational) reasoning. We show a complete neural-symbolic task in Example 1.

Example 1 (Neural-symbolic task). Let us consider a multi-object detection task in the *Animal* domain. The set $\mathcal{C}$ contains the identifiers of the constants (i.e. $\mathcal{C} = \{Simba, Mufasa, Alex, \dots\}$ and each of the constants $e \in \mathcal{C}$ is accompanied with the corresponding pixel vector $\mathbf{x}_e$. The predicates $\mathcal{R}$ represent properties of the images and are used to state if a specific object or a property r is present in the input image. For instance, $lion/1$ detects if a *lion* is in the image; $wall/1$ is true if the image contains a *wall*, whereas $savanna/1$ and $zoo/1$ state if the image context is that

Correlations between images	$\forall x_1 \forall x_2 \text{ sameLoc}(x_1, x_2) \Rightarrow (O(x_1) \wedge O(x_2)) \vee (\neg O(x_1) \wedge \neg O(x_2)) \text{ (with } O \in \mathcal{O})$
Knowledge Graphs	$\forall x O(x) \Rightarrow \text{mammal}(x) \text{ (with } O \in \{\text{lion}, \text{cat}, \text{dog}, \text{sheep}, \dots\})$ $\forall x \text{ mammal}(x) \Rightarrow \text{animal}(x)$ $\forall x \text{ mammal}(x) \Rightarrow \text{legs}(x) \wedge \text{body}(x) \wedge \text{tail}(x)$
General Knowledge	$\forall x \text{ lion}(x) \rightarrow \text{savanna}(x) \vee \text{zoo}(x)$ $\forall x \text{ wall}(x) \rightarrow \neg \text{savanna}(x)$
Correlations between categories	$\forall x \text{ polarbear}(x) \wedge \text{lion}(x)$ $\forall x \text{ antelope}(x) \wedge \text{lion}(x)$

Table 1. The knowledge base KB for the object detection task. $\mathcal{O} \subseteq \mathcal{R}$ here denotes the set of all the possible objects (each corresponding to unary predicates to be learned). Rules are grouped into 4 subsets that correspond to different types of knowledge.

of the *savanna* or a *zoo*. Images may be correlated by the locations where they have been shot, as provided by the binary predicate *sameLoc*/2 (this predicate is known and has not to be learnt, being based on the meta-information attached to the images).

The task requires to find a truth assignment $\mathbf{y}_q$ to a set of query facts q , e.g. $q = \{\text{savanna}(\text{Simba}), \text{zoo}(\text{Simba}), \text{zoo}(\text{Alex}), \dots\}$. In doing so, we may be provided with a set of evidence facts whose interpretation is known, e.g. $\mathbf{y}_e = \{\text{savanna}(\text{Mufasa}) = 1, \text{zoo}(\text{Mufasa}) = 0, \text{sameLoc}(\text{Simba}, \text{Mufasa}) = 1, \dots\}$. Moreover, we are provided with a KB composed of many rules belonging to different categories as shown in Table 1. For instance, a knowledge graph may be available to describe hierarchical dependencies among the object classes, or object compositions.

Solving the tasks requires integrating standard supervised learning with reasoning on the knowledge base. It requires generalization on the perceptual space for constants of which we may not have any evidence (e.g. *Alex*). In facts, neural networks tend to provide similar embedded representations to related constants. Finally, assuming to work in an uncertain environment, the relevance of each rule should be estimated. For example, the formula $\forall x \text{ antelope}(x) \wedge \text{lion}(x)$ is likely to be associated to a higher relevance than $\forall x \text{ polarbear}(x) \wedge \text{lion}(x)$, that is less likely to be true in the data.

8.4. The Model

Following a successful practice in Statistical Relational Artificial Intelligence (StarAI), the proposed model establishes a probability distribution p over the HB interpretations or possible worlds $\mathbf{y}$. This is often referred to as the distribution semantics of a First-Order Logic language, and it creates a bridge between logic and probabilistic reasoning. According to this semantics, single atom assignments $y \in \mathbf{y}$ are treated as Boolean random variables. This allows for standard logic reasoning mechanisms to be enhanced with uncertainty, which is a fundamental requirement in real world decision making.

Differently from StarAI, our proposed model is also provided with the sub-symbolic patterns $\mathbf{x}$ as input. This brings us in a more neural-symbolic setting. We assume that the

sub-symbolic patterns can be processed by a set of deep architectures, having weights $\mathbf{w}$. The role of these networks is to compute a confidence score for (a subset of) the atoms in the HB, based on the corresponding sub-symbolic pattern representations. We indicate as $\mathbf{f}^w$ the outputs of these networks over the considered world. In particular, given a considered atom $a = p(e_1, \dots, e_n)$, the neural architecture outputs the score $f_a^w(\mathbf{x}_g)$ by processing the symbolic patterns concatenation $\mathbf{x}_g$ as input, with $g = (e_1, \dots, e_n)$. Since for each atom a there is a unique corresponding concatenation $\mathbf{x}_g$ of patterns, we simply use the simplified notation f_a^w as there is no room for ambiguity. For example, if $a = \text{savanna}(\text{Simba})$, $f_a^w = f_a^w(\text{Simba})$. Notice that, at this stage, we do not make any particular assumption about the space of the confidence scores nor about the nature of the deep learning architectures.

The conditional distribution for the variables $\mathbf{y}$ is computed by a decomposition of the probability density function (pdf) based on a set of $C + 1$ potentials Φ_c , that model a set of properties that depend on both the variables $\mathbf{y}$ and $\mathbf{f}^w$. Formally, we refer to a pdf from an exponential family defined as:

$$p(\mathbf{y}|\mathbf{f}^w, \lambda) = \frac{1}{Z} \exp \left(\sum_c \lambda_c \Phi_c(\mathbf{f}^w, \mathbf{y}) \right) \quad (1)$$

where Z is the partition function and the parameters $\lambda = \{\lambda_1, \dots, \lambda_C\}$ represent the weights of the corresponding potentials Φ_c , while it is assumed $\lambda_0 = 1$ for simplicity.

In particular, the potentials are usually split into the potential Φ_0 expressing the consistency between $\mathbf{y}$ and $\mathbf{f}^w$, and the other potentials $\Phi_1, \dots, \Phi_C$ accounting only for the variables $\mathbf{y}$.

In particular, Φ_0 is typically used to enforce the consistency between the neural confidence scores and the final model predictions. Given a set A of atoms on which we want to enforce this consistency, Φ_0 is assumed to factorize over the single atoms:

$$\Phi_0(\mathbf{f}^w, \mathbf{y}) = \sum_{a \in A} \phi_0(f_a^w, y_a) , \quad (2)$$

As an example, a possible choice for ϕ_0 could be a negative quadratic: $\phi_0(f_a^w, y_a) = -\frac{1}{2}(y_a - f_a^w)^2$, but Section 8.7 will present a better theoretically justified choice for this potential.

Using an approach similar to MLNs, we can associate a potential Φ_c to each formula in KB , which is associated to a weight λ_c , that correlates the variables in $\mathbf{y}$. Each grounded atom corresponds to a node in the graphical structure associated to the grounding of the rule. All the nodes appearing in the same rule are connected by undirected edges in the graph. Hence, there is one clique for each grounding g_c of a formula c defined on $\mathbf{y}$. The potential of a rule factorizes over the cliques corresponding to the rule groundings $G_c(\mathbf{y})$:

$$\Phi_c(\mathbf{y}) = \sum_{\mathbf{y}_{c,g} \in G_c(\mathbf{y})} \phi_c(\mathbf{y}_{c,g}) ,$$

with $\mathbf{y}_{c,g}$ the assignment to the variables in $\mathbf{y}$ used in the grounding g of the rule c .

The potential $\phi_c(\mathbf{y}_{c,g})$ is designed to have a value equal to 1 if the grounded formula holds true and 0 if false. The resulting expression for the distribution is the following,

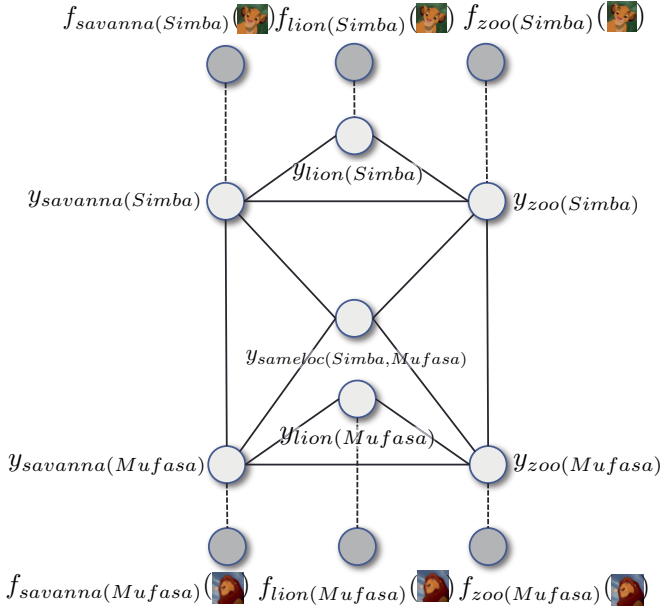


Figure 1. The graphical model representing the proposed model, where the variables $\mathbf{y}$ depend on the output $\mathbf{f}^w$ of the functions, processing the images $x_{Simba} = \text{image of Simba}$, $x_{Mufasa} = \text{image of Mufasa}$. The graph represents the rules $\forall x \text{ lion}(x) \Rightarrow \text{savanna}(x) \vee \text{zoo}(x)$, $\forall x \forall x' \text{ sameLoc}(x, x') \wedge \text{savanna}(x) \Rightarrow \text{savanna}(x')$, and $\forall x \forall x' \text{ sameLoc}(x, x') \wedge \text{zoo}(x) \Rightarrow \text{zoo}(x')$. The dashed connections come from the Φ_0 potentials, while the solid ones from the Φ_c potentials.

$$p(\mathbf{y} | \mathbf{f}^w, \lambda) = \frac{1}{Z} \exp \left(\sum_{a \in A} \phi_0(f_a^w, y_a) + \sum_c \lambda_c \sum_{\mathbf{y}_{c,g} \in G_c(\mathbf{y})} \phi_c(\mathbf{y}_{c,g}) \right) \quad (3)$$

The training process optimizes both the weights $\mathbf{w}$ of the network computing the values and the rule weights λ_c .

The learning task requires to train the low-level networks by integrating a set of supervisions with a knowledge base. Different categories of rules may be available as shown in Table 1.

Example 2. Still referring to Example 1, Figure 1 shows the graphical model correlating the variables $\mathbf{y}$ and the computed values $\mathbf{f}^w$ for two constants *Simba* and *Mufasa*, that are derived from the three rules $\forall x \text{ lion}(x) \Rightarrow \text{savanna}(x) \vee \text{zoo}(x)$, $\forall x \forall x' \text{ sameLoc}(x, x') \wedge \text{savanna}(x) \Rightarrow \text{savanna}(x')$, and $\forall x \forall x' \text{ sameLoc}(x, x') \wedge \text{zoo}(x) \Rightarrow \text{zoo}(x')$.

8.4.1. Learning

The model parameters are the weights associated to the potentials collected in λ and the weights $\mathbf{w}$ of the neural models. The objective of the learning process is to maximize the probability of the assignments specified by the training data $\hat{\mathbf{y}}_q \subset \mathbf{y}$, given the set of examples in $\mathbf{x}$ input to the networks and the evidence $\mathbf{y}_e \subset \mathbf{y}$. To keep the notation

simple, we assume that the world used for training is fully observed, so that it holds that $\mathbf{y} = \hat{\mathbf{y}}_q \cup \mathbf{y}_e$. Extending the methodology to partially observed worlds can be done with standard techniques [25]. Therefore, the learning problem can be stated as:

$$\underset{\lambda, \mathbf{w}}{\operatorname{argmax}} p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)$$

Please note that the computation of the likelihood is in general intractable since it requires to compute the partition function Z by a sum over all the configurations of the variables in $\hat{\mathbf{y}}_q$. For instance, if the variables are binary, the complexity for the computation of Z is $O(2^{|\hat{\mathbf{y}}_q|})$.

A gradient-based optimization procedure requires to compute the derivatives of the pdf with respect to the model parameters. The derivatives with respect to the weights of the first stage neural networks can be computed by considering the chain rule for function composition, taking into account that these networks compute the components of the vector $\mathbf{f}^w$. By considering the log-likelihoods, the derivatives with respect to the constraint weights λ_c are:

$$\frac{\partial \log p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial \lambda_c} = \Phi_c([\hat{\mathbf{y}}_q, \mathbf{y}_e]) - E_p[\Phi_c], \quad (4)$$

where $E_p[\cdot]$ is the expected value of the specified variable with respect to the distribution. Please note that, the parameter λ_c admits also a negative value, when the c -th constraint turns out to be too satisfied by the actual distribution with respect to the satisfaction degree on the training data. The derivatives with respect to a weight $w \in \mathbf{w}$ of the function f is:

$$\frac{\partial \log p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial w} = \left[\frac{\partial \Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e])}{\partial f} - E_p \left[\frac{\partial \Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e])}{\partial f} \right] \right] \cdot \frac{\partial f}{\partial w}. \quad (5)$$

A gradient-based optimization of the log-likelihood would require a direct computation of the expected values defined by equation 4 and 5. However, this is generally intractable as it requires to iterate over all possible variable assignments. The next sections will define various schema to approximate and apply the general framework described in this section in real settings.

8.4.2. Inference

Inference consists in finding the MAP assignment $\mathbf{y}_q^*$ for the of query atoms $q \subseteq HB$ given the evidence assignments $\mathbf{y}_e$ and fixed values of the parameters $\{\mathbf{w}, \lambda\}$. Formally, MAP inference corresponds to solving the maximization problem:

$$\mathbf{y}_q^* = \underset{\mathbf{y}_q}{\operatorname{argmax}} \log p(\mathbf{y}_q | \mathbf{y}_e, \mathbf{f}^w)$$

The maximization problem does not require to compute the partition function, which acts as a normalization constant common to all possible solutions: $\log p(\mathbf{y}_q | \mathbf{y}_e, \mathbf{f}^w) \propto \Phi_0(\mathbf{f}^w, [\mathbf{y}_q, \mathbf{y}_e]) + \sum_c \lambda_c \Phi_c([\mathbf{y}_q, \mathbf{y}_e])$. Therefore,

t-norm operation	Product	Gödel	Łukasiewicz
$x \wedge y$	$x \cdot y$	$\min(x, y)$	$\max(0, x + y - 1)$
$x \vee y$	$x + y - x \cdot y$	$\max(x, y)$	$\min(1, x + y)$
$\sim x$	$x = 0?1 : 0$	$x = 0?1 : 0$	$1 - x$
$\neg x$	$1 - x$	$1 - x$	$1 - x$
$x \Rightarrow y$	$x \leq y?1 : \frac{y}{x}$	$x \leq y?1 : y$	$\min(1, 1 - x + y)$

Table 2. The algebraic operations corresponding to some logic connectives of the fundamental t-norm fuzzy logics.

$$\mathbf{y}_q^* = \arg \max_{\mathbf{y}_q} \left[\Phi_0(\mathbf{f}^{wv}, [\mathbf{y}_q, \mathbf{y}_e]) + \sum_c \lambda_c \Phi_c([\mathbf{y}_q, \mathbf{y}_e]) \right] \quad (6)$$

The output query variables are Boolean and, hence, we need to solve a discrete MAP optimization problem, for which gradient-based techniques can not be directly applied. The following sections will define a continuous approximation to find the MAP solution of equation 6.

8.4.2.1. Potentials and T-Norm Fuzzy Logic

Boolean logic assumes that formulas in a certain KB , so as logic potentials Φ_c associated to them, are evaluated with a crisp truth-value. A possible way to consider a continuous relaxation of Boolean formulas is by exploiting a so called *fuzzy logic*, that extends the set of truth-values 0 (*false*) and 1 (*true*) to truth-degrees in the unit interval $[0, 1]$. Fuzzy logic connectives are suitable to deal with continuous values, while recovering the Boolean connectives behaviour if restricted on crisp values, and they are generally semantically interpreted by continuous and (almost everywhere) differentiable operations. In particular the notion of *t-norm* (triangular-norm) [26], a binary operation extending the Boolean conjunction to $[0, 1]$ -values, plays a crucial role in defining a specific *t-norm fuzzy logic* [27]. Indeed, by choosing a certain t-norm ($\wedge$), we may consequently derive a set of logic connectives defining a whole fuzzy logic. For instance, given two variables $a, b \in [0, 1]$, the truth-function of the *implication* ($\Rightarrow$) can be defined as the residuum of a given t-norm by $a \Rightarrow b := \sup\{c : a \wedge c \leq b\}$, while the *residual-negation* ($\sim$) by $\sim a := a \Rightarrow 0$. A special interest is devoted to the so called *strong negation* ($\neg$), that is evaluated as $\neg a := 1 - a$, and allows the definition of additional logic connectives. For instance, by means of a t-norm and the strong negation, we may get a so called *t-conorm* ($\vee$), extending the Boolean disjunction, according to the DeMorgan law $a \vee b := \neg(\neg a \wedge \neg b)$.

Despite several t-norm fuzzy logics have been proposed in the literature, it has been shown that any continuous t-norm can be obtained starting from three fundamental ones. Table 2 reports the operations computed by different logic connectives for these three fundamental continuous t-norms, i.e. Product, Gödel and Łukasiewicz t-norm fuzzy logics. In addition, in order to assure that logic formulas are translated into differentiable convex functions for constrained optimization, recently a fragment of the Łukasiewicz logic [28] has been proposed.

First-Order Logic formulas also allow the aggregation over sets of variables in specific domains of discourse by means of the *universal quantifier* ($\forall$) and the *existential quantifier* ($\exists$), expressing that a certain formula is satisfied over all or at least one ground-

ing, respectively. In this regard, a universal quantifier can be thought of as a conjunction, while the existential quantifier as a disjunction, among all the possible groundings, hence they are generally converted into real-valued functions as a t-norm and as a t-conorm over the groundings, respectively. However, other aggregation functions, such as average operators, can be exploited according to the problem under investigation. The conversion of quantifiers into real-valued functions can be recursively performed from the outer to the inner variables in case multiple quantified variables are present.

As an example, let us consider the FOL language with constants $\mathcal{C} = \{Simba, Mufasa\}$ and predicates $\mathcal{R} = \{lion/1, savanna/1, zoo/1\}$ and the following FOL rule from Table 1:

$$\forall x lion(x) \rightarrow savanna(x) \vee zoo(x) .$$

In case of Łukasiewicz logic connectives and universal quantifier converted with the arithmetic mean, the continuous surrogate for the FOL rule grounded over the whole set of constants is given by:

$$\Phi^s(\mathbf{y}) = \frac{1}{2} \left(\min(1, 1 - y_{lion(Simba)} + \min(1, y_{savanna(Simba)} + y_{zoo(Simba)})) + \min(1, 1 - y_{lion(Mufasa)} + \min(1, y_{savanna(Mufasa)} + y_{zoo(Mufasa)}) \right).$$

8.4.2.2. Inference using Fuzzy Logic approximations

We assume to relax the $\mathbf{y}$ variables to take values in the $[0, 1]$ interval by replacing each Boolean-based potential $\Phi_c(\mathbf{f}^w, \mathbf{y})$ with a continuous surrogate $\Phi_c^s(\mathbf{f}^w, \mathbf{y})$, using the fuzzy logic potentials defined in the previous section. Inference can now be performed by maximizing the relaxed version of Equation 6 using gradient descent with respect to the continuously relaxed version of the $\mathbf{y}_q$:

$$\mathbf{y}_q^* = \arg \max_{\mathbf{y}_q} \left[\Phi_0(\mathbf{f}^w, [\mathbf{y}_q, \mathbf{y}_e]) + \sum_c \lambda_c \Phi_c^s([\mathbf{y}_q, \mathbf{y}_e]) \right],$$

8.5. Semantic Based Regularization

Semantic-Based Regularization (SBR) [7] defines a learning and reasoning framework, which trains neural networks under the constraints imposed by the prior logic knowledge. The declarative language Lyrics [29] is available to provide a flexible and easy to use front-end for the SBR framework. This section shows how Semantic Based Regularization can be derived as a tractable approximation of the framework presented in the previous section. In particular, the basic assumption of SBR is that the λ parameters are frozen and known a-priori and the potentials are expressed via a differentiable form based on a continuous relaxation of the variables $\mathbf{y}$.

8.5.1. Learning

Instead of performing likelihood optimization of the variables $\mathbf{y}$, SBR directly defines the constraints over the $\mathbf{f}^w$ variables and then optimizes the $\mathbf{f}^w$ given the training targets $\hat{\mathbf{y}}_q$. In particular, using the Bayes rule:

$$\begin{aligned} p(\mathbf{f}^w | [\hat{\mathbf{y}}_q, \mathbf{y}_e]) &\propto p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w) \cdot p(\mathbf{f}^w) = \\ &= \underbrace{\frac{1}{Z_1} \exp(\Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e]))}_{p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w)} \cdot \underbrace{\frac{1}{Z_2} \exp\left(\sum_c \lambda_c \Phi_c^s(\mathbf{f}^w)\right)}_{p(\mathbf{f}^w)} \end{aligned} \quad (7)$$

where Z_1 and Z_2 are partition functions, Φ_c^s is the continuous relaxation of the c -th logic rule, $p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w)$ is used to express the fitting of the supervised data and the prior $p(\mathbf{f}^w)$ gives preference to the functions respecting the logic constraints. The parameters $\mathbf{w}$ of the $\mathbf{f}^w$ (typically implemented using a neural network or kernel machine) of an SBR can be optimized via gradient ascent by maximizing the likelihood of the training data. In the SBR literature, the $\Phi_c^s(\mathbf{f}^w)$ potentials are commonly derived as a continuous approximation of a logic rule using fuzzy logic as expressed in the previous subsection, so that Equation 7 can be directly optimized using gradient-based techniques. Since the training phase focuses only on the learnable functions $\mathbf{f}^w$, SBR directly optimizes the numerators without having to consider the partition function. This schema is extremely scalable and its complexity only depends on the cost of computing the constraints Φ_c^s .

Discussion. SBR provides a scalable platform for neural-symbolic tasks. A major disadvantage, over the other learning schema proposed in the next sections, is that SBR can not learn the weights used for reasoning, which are required to be known a-priori. This is very unlikely to happen in most of the real world scenarios, where the strength of each rule must be co-trained with the learning system. SBR, on the other hand, requires to cross-validate these values, that maybe impractical when a large number of logic rules is provided.

8.6. Deep Logic Models

Deep Logic Models (DLM) [8] improve over SBR by allowing to co-train the learner and reasoner parameters, providing a tighter integration for learning and reasoning. Indeed, learning the logic rule weights allows DLM to control the constraint enforcement and to decide if and how much a certain rule should be satisfied, where a higher value corresponds to a stronger enforcement of the corresponding constraint. Deep Logic Models, as originally proposed, are as a special case of Equation 3, where $\Phi_0(\mathbf{f}^w, \mathbf{y}) = -\frac{1}{2} \sum_{a \in A} (y_a - f_a^w)^2$.

8.6.1. Learning

As shown before, a direct computation of the derivatives of the log-likelihood with respect to the parameters is not feasible. A possible approximation [1, 30] relies on replacing the expected values in equations 4 and 5 with the corresponding value of the MAP

Input: Input data $\mathbf{x}$, training data $\hat{\mathbf{y}}_q$, evidence $\mathbf{y}_e$, functions defining $\mathbf{f}^w$

Output: Trained model parameters $\{\lambda, \mathbf{w}\}$

Initialize $\lambda = \mathbf{0}$, random $\mathbf{w}$;

while not converged do

Expectation:

 Compute the function outputs $\mathbf{f}^w$ on $\mathbf{x}$ using the weights $\mathbf{w}$;

 Compute the MAP solution $\mathbf{y}_q^* = \arg \max_{\mathbf{y}_q} \log p(\mathbf{y}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)$;

Maximization:

 Compute gradient $\nabla C_{\lambda, \mathbf{w}}([\hat{\mathbf{y}}_q, \mathbf{y}_q^*, \mathbf{y}_e])$ defined by Equation 8;

 Update the weights $\lambda, \mathbf{w}$ via gradient descent;

end

Algorithm 1: Training algorithm for Deep Logic Models.

solution $\mathbf{y}_q^*$, assuming that most of the probability mass of the distribution is centered around it. Under this assumption the expected values computed by equations 4 and 5 can be approximated as:

$$\frac{\partial \log p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial \lambda_c} \approx \Phi_c([\hat{\mathbf{y}}_q, \mathbf{y}_e]) - \Phi_c([\mathbf{y}_q^*, \mathbf{y}_e]),$$

$$\frac{\partial \log p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial \mathbf{w}} \approx \left[\frac{\partial \Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e])}{\partial f} - \frac{\partial \Phi_0(\mathbf{f}^w, [\mathbf{y}_q^*, \mathbf{y}_e])}{\partial f} \right] \cdot \frac{\partial f}{\partial \mathbf{w}}.$$

From the above approximation, it emerges that the likelihood tends to be maximized when the MAP solution is close to the training data, namely if $\Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e]) \simeq \Phi_0(\mathbf{f}^w, [\mathbf{y}_q^*, \mathbf{y}_e])$ and $\Phi_c([\hat{\mathbf{y}}_q, \mathbf{y}_e]) \simeq \Phi_c([\mathbf{y}_q^*, \mathbf{y}_e]) \forall c$. Furthermore, the probability distribution is more centered around the MAP solution when $\Phi_0(\mathbf{f}^w, [\mathbf{y}_q^*, \mathbf{y}_e])$ is close to its maximum value. Since Φ_0 is negative and has zero as upper bound: $\Phi_0(\mathbf{f}^w, \mathbf{y}) \leq 0 \forall \mathbf{y}, \mathbf{f}^w$, the constraint $\Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e]) \simeq \Phi_0(\mathbf{f}^w, [\mathbf{y}_q^*, \mathbf{y}_e])$ is transformed into the two separate constraints $\Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e]) \simeq 0$ and $\Phi_0(\mathbf{f}^w, [\mathbf{y}_q^*, \mathbf{y}_e]) \simeq 0$.

This means that, given the current MAP solution, DLMs increase the log likelihood by computing the gradient and weight updates using the following surrogate for the log-likelihood: $\Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e]) + \Phi_0(\mathbf{f}^w, [\mathbf{y}_q^*, \mathbf{y}_e]) + \sum_c \lambda_c [\Phi_c([\hat{\mathbf{y}}_q, \mathbf{y}_e]) - \Phi_c([\mathbf{y}_q^*, \mathbf{y}_e])]$. Finally, replacing the selected forms for the potentials and changing the sign to transform a maximization problem into a minimization one, yields the following cost function, given the current MAP solution:

$$C_{\lambda, \mathbf{w}}(\hat{\mathbf{y}}_q, \mathbf{y}_q^*, \mathbf{y}_e) = \frac{1}{2} \sum_{a \in A} (y_a' - f_a^w)^2 + \frac{1}{2} \sum_{a \in A} (y_a^* - f_a^w)^2 + \sum_c \lambda_c [\Phi_c([\hat{\mathbf{y}}_q, \mathbf{y}_e]) - \Phi_c([\mathbf{y}_q^*, \mathbf{y}_e])]. \quad (8)$$

Minimizing this cost function is just a local approximation of the full likelihood maximization for the current MAP solution. Therefore, the training process alternates the computation of the MAP solution, the computation of the gradient for $C_{\lambda, \mathbf{w}}(\hat{\mathbf{y}}_q, \mathbf{y}_q^*, \mathbf{y}_e)$ and one weight update step.

Algorithm 1 summarizes this iterative training algorithm.

Discussion. DLMs preserve the scalability of SBR, while having the fundamental advantage of co-training the reasoner's parameters in an integrated procedure. The main limitation of the DLM model is in its strong statistical assumptions. In particular, during training most of the probability mass of the distribution is assumed to be centered around the MAP solution, to avoid carrying out an exhaustive search on the assignment space.

8.7. Relational Neural Machines

Relational Neural Machines (RNM) [9] aim at defining a general model which collapses to a classical supervised learning model, when only the fitting of the supervised data is considered, and a fully wedged probabilistic reasoning model like Markov Logic Networks when only symbolic knowledge is provided. When mixing these two capabilities, RNMs provide a powerful, expressive and general neural-symbolic methodology.

Firstly, let us focus on the classical *supervised learning* setup, in which the input examples are target values for a set of atoms A (the examples). The examples are typically assumed to be i.i.d. and the variables $\mathbf{y}, \mathbf{f}^w$ can therefore be split into disjoint sets collecting the variables of each example, corresponding to different cliques. In particular, we assume that the set of considered atoms A can be split into disjoint sets of atoms sharing the same grounded constants: $A = \bigcup_g A(g) = \bigcup_g [r_1(g), \dots, r_n(g)]$, where the $r_a \in \mathcal{R}$ are the predicates for which we want to define the supervisions. Considering the setting in Example 1, we may consider the following a set of atoms of interest:

$$\begin{aligned} A &= \{zoo(Simba), savanna(Simba), zoo(Mufasa), savanna(Mufasa)\} \\ &= A_{Simba} \cup A_{Mufasa} \end{aligned}$$

Therefore, when considering only the fitting of the supervised data, the resulting pdf is:

$$p_0(\mathbf{y}|\mathbf{f}^w) = \frac{1}{Z} \exp \left(\sum_g \sum_{a \in A(g)} \phi_0(f_a^w, y_a) \right), \quad (9)$$

Given this formulation, the two cases of single class or multi-class assignment are detailed in the following.

Single class. In this setting, the atoms corresponding to the same grounded constants are mutually exclusive $\forall g \nexists i, j \in A(g) : y_i = y_j = 1, i \neq j$ and we indicate as $a(g)$ the unique atom such that $y_{a(g)} = 1$ for a given g . Therefore, the output configurations, for which a given input is assigned to more than one class, must have null probability. Hence, to meet this requirement we can associate the following potential to each grounded constants:

$$\phi_0(f_a^w, y_a) = \begin{cases} -\infty & \text{if } \exists g \exists i, j \in A(g) : y_i = y_j = 1, i \neq j \\ f_a^w \cdot y_a & \text{otherwise} \end{cases}$$

Given this potential associated to each example, it is guaranteed that $p_0(\mathbf{y}|\mathbf{f}^w, \lambda) = 0$ for the output configurations $\mathbf{y}$ in which more than one component per grounding is not null. Instead, for the cases in which there exists only an element $y_a, a \in A(g)$ for which $y_a = 1$ for each g , the distribution can be rearranged as:

$$\begin{aligned}
p_0(\mathbf{y}|\mathbf{f}^w) &= \frac{\exp\left(\sum_g \sum_{a \in A(g)} \phi_0(f_a^w, y_a)\right)}{\sum_{\mathbf{y}'} \exp\left(\sum_g \sum_{a \in A(g)} \phi_0(f_a^w, \mathbf{y}'_a)\right)} = \frac{\prod_g \prod_{a \in A(g)} \exp(\phi_0(f_a^w, y_a))}{\sum_{\mathbf{y}'} \prod_g \prod_{a \in A(g)} \exp(\phi_0(f_a^w, \mathbf{y}'_a))} = \\
&= \frac{\prod_g \exp(f_{a(g)}^w \cdot y_{a(g)})}{\prod_g \sum_{a \in A(g)} \exp(f_a^w \cdot y_a)} = \prod_g \frac{\exp(f_{a(g)}^w \cdot y_{a(g)})}{\sum_{a \in A(g)} \exp(f_a^w \cdot y_a)} = \\
&= \prod_g \text{softmax}(\mathbf{f}_g^w, \mathbf{y}_g, a(g))
\end{aligned}$$

where, in the simplification in the third step, we exploited the fact that only one $y_{a(g)}$ is not null in each configuration. This result supports the use of the softmax output for networks used in single-class classification tasks. Basically, the potential favors the assignment to the class corresponding to the component of $\mathbf{f}^w$ having the highest value for each example.

Multi-class. In this case each example can be associated to one or more labels, hence the potential is expressed as the cosine correlation between the configurations of y_a and f_a^w for an atom a :

$$\phi_0(f_a^w, y_a) = f_a^w \cdot y_a.$$

By inserting this definition of the potential into Equation 9 and following the same first steps as in the single-class case, we obtain the following intermediate expression

$$p_0(\mathbf{y}|\mathbf{f}^w) = \frac{\prod_{a \in A} \exp(f_a^w \cdot y_a)}{\sum_{\mathbf{y}'} \prod_{a \in A} \exp(f_a^w \cdot \mathbf{y}'_a)}$$

In the denominator the sum ranges over all the $2^{|\mathbf{y}|}$ binary configurations of $\{0, 1\}^{|\mathbf{y}|}$ and it holds that

$$\sum_{\mathbf{y}'} \prod_{a \in A} \exp(f_a^w \cdot \mathbf{y}'_a) = \prod_{a \in A} [1 + \exp(f_a^w)]$$

such that

$$\begin{aligned}
p_0(\mathbf{y}|\mathbf{f}^w) &= \frac{\prod_{a \in A} \exp(f_a^w \cdot y_a)}{\sum_{\mathbf{y}'} \prod_{a \in A} \exp(f_a^w \cdot \mathbf{y}'_a)} = \prod_{a \in A} \frac{\exp(y_a \cdot f_a^w)}{[1 + \exp(f_a^w)]} = \\
&= \prod_{a: y_a=1} \sigma(f_a^w) \cdot \prod_{a: y_a=0} (1 - \sigma(f_a^w)),
\end{aligned}$$

where σ is the sigmoid function. This result gives a theoretical justification to the use of the sigmoidal output layer for multi-label classification tasks. As it can be easily seen from the terms in the factorization of the pdf, the assignment of y_a that maximizes the probability depends on how much $\sigma(f_a)$ is close to zero or one.

8.7.1. Learning

The training complexity can be reduced by assuming that the partition function can be factorized by grouping the variables in $\mathbf{y}$ in independent subsets $\mathbf{y}_i \subseteq \mathbf{y}$, such that $Z \approx \prod_i Z_i$, where each Z_i depends only on the variables in $\mathbf{y}_i$. In fact, in this case the computation can be done for each Z_i by considering only the configurations of $\mathbf{y}_i$.

An interesting case is that of piecewise likelihood [31], for which the partition function is assumed to be factorized by the potentials as

$$Z \approx Z_0 \cdot \prod_c Z_c = \sum_{\mathbf{y}'} \exp(\Phi_0(\mathbf{f}^w, \mathbf{y}')) \cdot \prod_c \left[\sum_{\mathbf{y}'_{c,g}} \exp(\lambda_c \Phi_c(\mathbf{y}'_{c,g})) \right]$$

where $\mathbf{y}'_{c,g}$ iterates over all possible assignments of the variables in $G_c(\mathbf{y})$, e.g. the subset of variables in $\mathbf{y}$ that are used to compute Φ_c . It should be noticed that, in general, the sets $G_c(\mathbf{y})$ are not disjoint since the same variables are usually involved in more than one potential. In this case the overall probability distribution may be decomposed by defining the *piecewise-local* probability distribution over the subset of assignments $\mathbf{y}_c$ used within the c -th potential as

$$p_c^{PL}(\mathbf{y}_c | \lambda_c) = \frac{\exp(\lambda_c \Phi_c(\mathbf{y}_c))}{Z_c} = \frac{\exp(\lambda_c \Phi_c(\mathbf{y}_c))}{\sum_{\mathbf{y}'_{c,g}} \exp(\lambda_c \Phi_c(\mathbf{y}'_{c,g}))} \quad (10)$$

In fact, given the previous approximation for the partition function, the pdf is actually approximated as

$$p(\mathbf{y} | \mathbf{f}^w, \lambda) \approx p_0(\mathbf{y} | \mathbf{f}^w) \cdot \prod_c p_c^{PL}(\mathbf{y}_c | \lambda_c)$$

that creates an independence assumption between the groups of variables involved in each potential (i.e. each constraint). The main advantage of this local decomposition is that the complexity is reduced to $O(|\mathbf{y}_{\bar{c},g}| \cdot 2^{n_{\bar{c}}})$, where $\bar{c}$ is the index of the formula corresponding to the potential with the largest number $n_{\bar{c}}$ of variables to ground.

However, this decomposition may yield factors depending on potentials that still involve too many variables. In this case, to reduce the complexity, we can apply a pseudo-likelihood decomposition, where we additionally consider the assignments from a constraints as independent on the others, as

$$p_c^{PL}(\mathbf{y}_c | \lambda_c) \approx \prod_{y_a \in \mathbf{y}_c} p_c^{PPL}(y_a | \mathbf{y}_c \setminus y_a, \lambda_c) = \prod_{y_a \in \mathbf{y}_c} \frac{\exp(\lambda_c \Phi_c(\mathbf{y}_c))}{\sum_{b=\{0,1\}} \exp(\lambda_c \Phi_c([y_a = b, \mathbf{y}_c \setminus y_a]))}.$$

In this case the computational cost is proportional to the number of variables n , but the value of the partition function depends on the argument of the pdf.

Given the structure of the constraints and their associated potentials, we may organize them into two groups $\{C_1, C_2\}$ on the basis of the complexity for computing the partition function. The first group will contain those potentials approximated by the local

piecewise partitioning PL (higher cost but better approximation); the second one will be based on the pseudo-likelihood PPL (lower cost but dependence on the argument and worse approximation). With this structure, the pdf can be written as:

$$p(\mathbf{y}|\mathbf{f}^w, \lambda) \approx p_0(\mathbf{y}|\mathbf{f}^w) \cdot \prod_{c \in C_1} p_c^{PL}(\mathbf{y}_c|\lambda_c) \cdot \prod_{c \in C_2} \prod_{y_a \in \mathbf{y}_c} p_c^{PPL}(y_a|\mathbf{y}_c \setminus y_a, \lambda_c) \quad (11)$$

The training procedure aims at maximizing the likelihood of the output distribution on an assigned learning set, that implies to tune the model parameters (i.e. the first stage network weights and the potential weights) in order to improve the fulfillment of the constraints encoded by the potentials (i.e. supervision and rules from the KB). The optimization procedure requires to compute the derivatives of the pdf with respect to the model parameters. The derivatives with respect to the weights of the first stage neural networks can be computed by considering the chain rule for function composition, taking into account that these networks compute the components of the vector $\mathbf{f}^w$.

For the sake of simplicity, let us assume a fully observed world during training, so that the set of query and evidence atoms $e \subset HB, q \subseteq HB : q \cup e = HB$. By considering the log-likelihood of the training data, the derivatives with respect to the constraint weights λ_c of the potentials factorized using the PL partitioning are:

$$\frac{\partial \log p(\hat{\mathbf{y}}_q|\mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial \lambda_c} \approx \Phi_c([\hat{\mathbf{y}}_q, \mathbf{y}_e]) - E_{p_c^{PL}}[\Phi_c], \quad (12)$$

where $E_{p_c^{PL}}[\cdot]$ is the expected value of the specified variable with respect to the distribution p_c^{PL} . The derivatives with respect to the function weight $w \in \mathbf{w}$ of function f are the following:

$$\frac{\partial \log p(\hat{\mathbf{y}}_q|\mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial w} \approx \left[\frac{\partial \Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e])}{\partial f} - E_{p_0} \left[\frac{\partial \Phi_0(\mathbf{f}^w, [\hat{\mathbf{y}}_q, \mathbf{y}_e])}{\partial f} \right] \right] \cdot \frac{\partial f}{\partial w}, \quad (13)$$

Similar results can be obtained for the terms approximated with the PPL factorization. In fact, by considering the log-likelihood, the probability factorization of Equation 11 is the sum of the terms for the groups C_1 and C_2 of constraints. Each term in the group C_2 is again a sum of terms depending on the components y_a . The derivatives with respect to λ_c and w of $p_c^{PPL}(y_a|\mathbf{y}_c \setminus y_a, \mathbf{f}^w, \lambda_c)$ are similar to those in equations 12 and 13, with the only difference that the expected value is computed with respect to the PPL probability distribution.

Potentials expressing logic knowledge. Under the assumption that each grounding of the formula is independent on all the others, the partition function Z_c can be approximated as:

$$Z_c \approx Z_c^{|G_c(\mathbf{y})|} = \left(\sum_{\mathbf{y}'_{c,g}} \exp(\lambda_c \phi_c(\mathbf{y}'_{c,g})) \right)^{|G_c(\mathbf{y})|}$$

where $|G_c(\mathbf{y})|$ is the total number of groundings of the c -th potential. The local partition function Z_c^l is shared among all groundings and it can be efficiently computed from the

Input: Input data $\mathbf{x}$, training data $\hat{\mathbf{y}}_q$, evidence $\mathbf{y}_e$, functions defining $\mathbf{f}^w$

Output: Trained model parameters $\{\lambda, \mathbf{w}\}$

Initialize $\lambda = \mathbf{0}$, random $\mathbf{w}$;

while *not converged* **do**

Expectation:

 Compute the function outputs $\mathbf{f}^w$ on $\mathbf{x}$ using the weights $\mathbf{w}$;

$\forall c$ compute $\text{Avg}(\Phi_c, [\hat{\mathbf{y}}_q, \mathbf{y}_e])$;

Maximization:

$\forall c$ compute λ_c with Equation 15 on $[\hat{\mathbf{y}}_q, \mathbf{y}_e]$;

 Backpropagation for $\mathbf{w}$ with the derivatives of Equation 5;

 Update the weights $\mathbf{w}$;

end

Algorithm 2: Training algorithm for Relational Neural Machines.

numbers n_c^+, n_c^- of possible different grounding assignments satisfying or not satisfying the c -th formula, respectively. If n_c is the number of atoms in the formula, there are 2^{n_c} possible assignments and $n_c^- + n_c^+ = 2^{n_c}$, yielding:

$$Z_c^l = \sum_{\mathbf{y}'_{c,g}} \exp(\lambda_c \phi_c(\mathbf{y}'_{c,g})) = \underbrace{n_c^-}_{\text{each False evaluates to } e^0=1} + \underbrace{n_c^+ e^{\lambda_c}}_{\text{each True evaluates to } e^{\lambda_c}} = n_c^- + n_c^+ e^{\lambda_c}$$

In this case the potential depends only on the parameter λ_c and there is not a direct dependence on the variables $\mathbf{f}^w$ computed by the first stage. Hence, only the derivative of the log-likelihood with respect to λ_c is needed in the training algorithm. This derivative can be computed as

$$\begin{aligned} \frac{\partial \log p(\hat{\mathbf{y}}_q | \mathbf{y}_e, \mathbf{f}^w, \lambda)}{\partial \lambda_c} &\approx \sum_{\mathbf{y}_{c,g} \in G_c(\hat{\mathbf{y}}_q)} \phi_c([\mathbf{y}_{c,g}, \mathbf{y}_e]) - |G_c(\hat{\mathbf{y}}_q)| \cdot \sum_{\mathbf{y}'_{c,g}} p_c^{PL}([\mathbf{y}'_{c,g}, \mathbf{y}_e] | \lambda_c) \phi_c([\mathbf{y}'_{c,g}, \mathbf{y}_e]) = \\ &= \sum_{\mathbf{y}_{c,g} \in G_c(\hat{\mathbf{y}}_q)} \phi_c([\mathbf{y}_{c,g}, \mathbf{y}_e]) - |G_c(\hat{\mathbf{y}}_q)| \cdot \mathbb{E}_{p_c^{PL}}[\phi_c] \end{aligned}$$

From the previous equation we may observe that the stationary points with respect to the λ_c parameters (i.e. the points for which the derivative is null) must satisfy

$$\frac{1}{|G_c(\hat{\mathbf{y}}_q)|} \sum_{\mathbf{y}_{c,g} \in G_c(\hat{\mathbf{y}}_q)} \phi_c([\mathbf{y}_{c,g}, \mathbf{y}_e]) = \mathbb{E}_{p_c^{PL}}[\phi_c] \quad (14)$$

Let us indicate as $\text{Avg}(\Phi_c, [\hat{\mathbf{y}}_q, \mathbf{y}_e]) = \frac{1}{|G_c(\hat{\mathbf{y}}_q)|} \sum_{\mathbf{y}_{c,g} \in G_c(\hat{\mathbf{y}}_q)} \phi_c([\mathbf{y}_{c,g}, \mathbf{y}_e])$ the average satisfaction of the c -th formula on the data training data.

The expected value of the satisfaction of the formula for a grounding, $\mathbb{E}_{p_c^{PL}}[\phi_c]$ can be efficiently computed for a $\{0, 1\}$ -valued ϕ_c as:

$$\begin{aligned}
\mathbb{E}_{p_c^{PL}}[\phi_c] &= \sum_{\mathbf{y}'_{c,g}} p_c^{PL}(\mathbf{y}'_{c,g} | \mathbf{y}_e, \lambda_c) \phi_c([\mathbf{y}'_{c,g}, \mathbf{y}_e]) = \\
&= \sum_{\mathbf{y}'_{c,g}} \frac{1}{Z_c^l} \exp(\lambda_c \phi_c([\mathbf{y}'_{c,g}, \mathbf{y}_e])) \phi_c([\mathbf{y}'_{c,g}, \mathbf{y}_e]) = \\
&= \frac{n_c^+ e^{\lambda_c}}{Z_c^l} = \frac{n_c^+ e^{\lambda_c}}{n_c^- + n_c^+ e^{\lambda_c}}.
\end{aligned}$$

yielding the following optimal assignment to the λ_c for a given assignment $\mathbf{y}$:

$$\lambda_c = \log \frac{\text{Avg}(\Phi_c, [\hat{\mathbf{y}}_q, \mathbf{y}_e])}{1 - \text{Avg}(\Phi_c, [\hat{\mathbf{y}}_q, \mathbf{y}_e])} - \log \frac{n_c^+}{n_c^-} \quad (15)$$

Hence, the log-likelihood is maximized when λ_c is equal to the difference between the log odds of the constraint satisfaction of the data and the log odds of the prior satisfaction of the constraint if all assignments are equally probable. The training procedure is sketched in Algorithm 2. When the world is not fully observed during training, an iterative EM schema can be used to marginalize over the unobserved data in the expectation step using the inference procedure described before. Then, the average constraint satisfaction can be recomputed, and then the $\lambda, \mathbf{w}$ parameters can be updated in the maximization step.

Discussion. RNMs fully co-train the function and reasoning parameters, while relaxing the strong statistical assumptions of Deep Logic Models. The factorization of the partition function is an approximation which has been proved to work well in many applications, but it is still a limiting factor when learning and inference need to carry out a complex multi-hop reasoning process. The next section will discuss future directions to improve the current state-of-the-art in the context of the discussed models.

8.8. Conclusions and open challenges

The framework presented in this chapter provides a general platform for neural-symbolic integration, encoding in a homogeneous way several standard learning problems both from the deep learning and the statistical relational learning communities. This formulation naturally allows the transition to hybrid neural-symbolic settings, where both neural and logic based features are used to shape an exponential distribution over mixed symbolic and sub-symbolic data. The generality and expressivity of the approach unfortunately correspond to a high computational complexity for inference and learning. This chapter presented some schema devised to perform approximate inference in settings where exact inference is unpractical.

The future improvements can be categorized into two main directions: approximate logic inference and approximate probabilistic inference. We argue that these two directions are not unique to the proposed framework but are actually shared by many other neural-symbolic frameworks.

Approximate logic inference. The proposed framework inherits the knowledge base model construction already used by several statistical relational learning methods, where the logic rules are used as templates for the factors of a probabilistic graphical model. Such templates are then instantiated (or unrolled) over the current domain. We called this procedure the *grounding* process, and the corresponding graphical model as the *grounded* model. The computational complexity of the grounding is known to be polynomial. However, in large scale problems, the size of the full grounded model is unfeasible and thus the following question arise: *how can we approximate the grounded model retaining its representation power?* One possible solution is using a query-oriented approach [32], where only the portion of the grounded model that is considered as *relevant* to the query is instantiated. Logic programming based systems [15, 17] can be considered an instance of such approach. However, the relevance metric is not always trivial to define in a probabilistic setting and the eventual pruning is often not sufficient. Dynamical selection strategies [19] are a promising direction which has not been explored yet.

Probabilistic inference. The proposed framework requires to perform probabilistic inference in a possibly large graphical model. There has been a lot of work in how to approximate probabilistic inference both in the deep learning [1] and statistical relational learning community [33, 34]. From a neural-symbolic perspective, it would be extremely interesting to investigate how to cross-fertilize those ideas. An interesting direction is to use neural models to guide structured sampling-based methods [35] or to extend (deep) variational approaches to a relational setting [36, 37].

Acknowledgements

This work has been partially supported by AI4EU and TAILOR, projects funded by EU Horizon 2020 research and innovation program under grant agreement numbers 825619, 952215, respectively. GM is funded by the Research Foundation Flanders, FWO, (1239422N).

References

- [1] Goodfellow I, Bengio Y, Courville A, et al. Deep learning. Vol. 1. MIT press Cambridge; 2016.
- [2] LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*. 2015;521(7553):436.
- [3] Chen LC, Schwing A, Yuille A, et al. Learning deep structured models. In: *International Conference on Machine Learning*; 2015. p. 1785–1794.
- [4] Muggleton S, De Raedt L. Inductive logic programming: Theory and methods. *The Journal of Logic Programming*. 1994;19:629–679.
- [5] De Raedt L, Kimmig A, Toivonen H. Problog: A probabilistic prolog and its application in link discovery. In: *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*. Morgan Kaufmann Publishers Inc.; 2007. p. 2468–2473.
- [6] Kimmig A, Bach S, Broecheler M, et al. A short introduction to probabilistic soft logic. In: *Proceedings of the NIPS Workshop on Probabilistic Programming: Foundations and Applications*; 2012. p. 1–4.
- [7] Diligenti M, Gori M, Sacca C. Semantic-based regularization for learning and inference. *Artificial Intelligence*. 2017;244:143–165.
- [8] Marra G, Giannini F, Diligenti M, et al. Integrating learning and reasoning with deep logic models. In: *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML/KDD)*; Springer; 2019. p. 517–532.

- [9] Marra G, Diligenti M, Giannini F, et al. Relational neural machines. In: *Proceeding of the 24th European Conference on Artificial Intelligence ECAI 2020*; (Frontiers in Artificial Intelligence and Applications; Vol. 325). IOS Press; 2020. p. 1340–1347. Available from: <https://doi.org/10.3233/FAIA200237>.
- [10] Richardson M, Domingos P. Markov logic networks. *Machine learning*. 2006;62(1):107–136.
- [11] Bach SH, Broecheler M, Huang B, et al. Hinge-loss markov random fields and probabilistic soft logic. *Journal of Machine Learning Research*. 2017;18:1–67.
- [12] Raedt Ld, Dumančić S, Manhaeve R, et al. From statistical relational to neuro-symbolic artificial intelligence. In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*; 7; 2020. p. 4943–4950. Survey track; Available from: <https://doi.org/10.24963/ijcai.2020/688>.
- [13] Hu Z, Ma X, Liu Z, et al. Harnessing deep neural networks with logic rules. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL) (Volume 1: Long Papers)*; Aug.; Berlin, Germany. Association for Computational Linguistics; 2016. p. 2410–2420. Available from: <https://www.aclweb.org/anthology/P16-1228>.
- [14] Donadello I, Serafini L, d’Avila Garcez. Logic tensor networks for semantic image interpretation. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI*; 2017. p. 1596–1602. Available from: <https://doi.org/10.24963/ijcai.2017/221>.
- [15] Manhaeve R, Dumancic S, Kimmig A, et al. Deepproblog: Neural probabilistic logic programming. In: *Advances in Neural Information Processing Systems (NeurIPS)*; Vol. 31. Curran Associates, Inc.; 2018. Available from: <https://proceedings.neurips.cc/paper/2018/file/dc5d637ed5e62c36ecb73b654b05ba2a-Paper.pdf>.
- [16] Xu J, Zhang Z, Friedman T, et al. A semantic loss function for deep learning with symbolic knowledge. In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*; (Proceedings of Machine Learning Research; Vol. 80). PMLR; 2018. p. 5502–5511. Available from: <http://proceedings.mlr.press/v80/xu18h.html>.
- [17] Sourek G, Aschenbrenner V, Zelezny F, et al. Lifted relational neural networks: Efficient learning of latent relational structures. *Journal of Artificial Intelligence Research*. 2018;62:69–100.
- [18] Rocktäschel T, Riedel S. End-to-end differentiable proving. In: *Advances in Neural Information Processing Systems (NeurIPS)*; Vol. 30; 2017. Available from: <https://proceedings.neurips.cc/paper/2017/file/b2ab001909a8a6f04b51920306046ce5-Paper.pdf>.
- [19] Minervini P, Riedel S, Stenetorp P, et al. Learning reasoning strategies in end-to-end differentiable proving. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*; Vol. 119; 13–18 Jul. PMLR; 2020. p. 6938–6949. Available from: <http://proceedings.mlr.press/v119/minervini20a.html>.
- [20] Yang F, Yang Z, Cohen WW. Differentiable learning of logical rules for knowledge base reasoning. In: *Advances in Neural Information Processing Systems (NeurIPS)*; 2017. p. 2319–2328. Available from: <https://proceedings.neurips.cc/paper/2017/hash/0e55666a4ad822e0e34299df3591d979-Abstract.html>.
- [21] Cohen W, Yang F, Mazaitis K. Tensorlog: A probabilistic database implemented using deep-learning infrastructure. *Journal of Artificial Intelligence Research*. 2020;67:285–325.
- [22] Ma X, Hovy E. End-to-end sequence labeling via bi-directional lstm-cnns-crf. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; 2016. p. 1064–1074.
- [23] Lin G, Shen C, Van Den Hengel A, et al. Efficient piecewise training of deep structured models for semantic segmentation. In: *IEEE Conference on Computer Vision and Pattern Recognition*; 2016. p. 3194–3203.
- [24] Bordes A, Gabrilovich E. Constructing and mining web-scale knowledge graphs: Kdd 2014 tutorial. In: *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*; 2014. p. 1967–1967.
- [25] Ruud PA. Extensions of estimation methods using the em algorithm. *Journal of Econometrics*. 1991; 49(3):305–341.
- [26] Klement E, Mesiar R, Pap E. *Triangular norms*. Kluwer Academic Publisher; 2000.
- [27] Hájek P. *Metamathematics of fuzzy logic*. Vol. 4. Springer Science & Business Media; 2013.
- [28] Giannini F, Diligenti M, Gori M, et al. On a convex logic fragment for learning and reasoning. *IEEE Transactions on Fuzzy Systems*. 2018;27(7):1407–1416.

- [29] Marra G, Giannini F, Diligenti M, et al. LYRICS: A general interface layer to integrate ai and deep learning. In: Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases (ECML/PKDD) 2019; (Lecture Notes in Computer Science; Vol. 11907). Springer; 2019. p. 283–298. Available from: https://doi.org/10.1007/978-3-030-46147-8_17.
- [30] Haykin S. Neural networks: A comprehensive foundation. 1st ed. Upper Saddle River, NJ, USA: Prentice Hall PTR; 1994.
- [31] Sutton C, McCallum A. Piecewise pseudolikelihood for efficient training of conditional random fields. In: International conference on Machine learning (ICML); ACM; 2007. p. 863–870.
- [32] Frasconi P, Costa F, De Raedt L, et al. klog: A language for logical and relational learning with kernels. Artificial Intelligence. 2014;217:117–143.
- [33] Kimmig A, Costa VS, Rocha R, et al. On the efficient execution of problog programs. In: International Conference on Logic Programming; Springer; 2008. p. 175–189.
- [34] Poon H, Domingos P. Sound and efficient inference with probabilistic and deterministic dependencies. In: AAAI; Vol. 6; 2006. p. 458–463.
- [35] Marra G, Kuželka O. Neural markov logic networks. In: UAI; 2021.
- [36] Zhang Y, Chen X, Yang Y, et al. Efficient probabilistic logic reasoning with graph neural networks. In: ICLR; 2020.
- [37] Kipf TN, Welling M. Variational graph auto-encoders. arXiv preprint arXiv:161107308. 2016;.

Chapter 9

Spike-Based Symbolic Computations on Bit Strings and Numbers

Ceca Krajšniković, Institute of Theoretical Computer Science, Graz University of Technology, Inffeldgasse 16b, Graz, Austria

Wolfgang Maass*, Institute of Theoretical Computer Science, Graz University of Technology, Inffeldgasse 16b, Graz, Austria

Robert Legenstein*, Institute of Theoretical Computer Science, Graz University of Technology, Inffeldgasse 16b, Graz, Austria

*, Joint last authors

9.1. Introduction

Spiking neural networks (SNNs) constitute the third generation of neural network models [1]. The most salient difference to standard artificial neural networks (ANNs) that are usually employed in deep learning is the means of communication between neurons in the network. While ANNs communicate real numbers in each computational step, spiking neurons in SNNs communicate events in an asynchronous manner. The output of a spiking neuron is zero most of the time, only occasionally turning to “1”, which is referred to as spiking or sending a spike. This communication scheme is much closer to the way neurons communicate in the brain and it has some advantages that we will discuss below. In this chapter, we discuss symbolic computations with SNNs. In particular, since we concentrate on tasks that involve temporally sequential input which necessitates recurrent connections in order to combine temporally dispersed information, we consider here networks of recurrently connected spiking neurons (RSNNs). This also reflects the fact that neuronal networks in the brain are highly recurrent. We use the term RSNNs whenever we want to emphasize the recurrent architecture of the considered network. We review the current state of this research direction and in addition present new results which show that surprisingly small SNNs can perform symbolic computations on bit sequences and numbers.

Why should we bother about RSNNs when thinking about neuro-symbolic computation? There are at least two answers to this question. First, the brain is very likely a computing system that merges sub-symbolic connectionist computation with symbolic com-

ponents. It has been argued many times that a pure subsymbolic system cannot achieve the generalization abilities of symbolic systems where variables can be used in various contexts [2, 3, 4]. Hence, the superb generalization capabilities of the brain likely result from symbolic computations. On the other hand, judged purely from the hardware, it is undisputed that the brain is a neural system. However, if we want to learn from the brain, we should take its basic computational units seriously. The event-based nature of spiking communication in the brain is in many aspects different from communication in ANNs with real-valued neurons. One can in principle quite easily emulate every ANN with an SNN since each real-valued output can be encoded by a spike rate, and several such and more advanced conversion methods exist [5, 6]. On the other hand, due to the asynchronous and spike-timing based communication in SNNs, the emulation of an SNN with an ANN is much harder [1]. Hence, it might not be straightforward to map the solution that evolution has found for symbolic computation to ANNs. Second, the brain is a network of a gigantic number of neurons, and the experience with deep learning has shown that bigger is usually better. Hence, we can expect that neuro-symbolic systems that approach at least some aspects of human intelligence will be huge. To make them practically useful, one should consider energy-efficient hardware solutions. One reason why the brain is extremely energy-efficient is its spike-based communication, where spikes are transmitted only when necessary. Energy-efficient neuromorphic hardware is based on this basic principle utilizing RSNNs [7, 8, 9].

Another lesson we learned from recent artificial intelligence (AI) research is that learning-based approaches are the most promising path to AI systems. Hence, we should not try to construct symbolic systems but rather equip neural networks with inductive biases and then train them to perform symbolic reasoning tasks. Since SNNs can exploit precise spike timing, some functions can be implemented with less resources by SNNs as compared to ANNs [1]. In practice however, training of SNNs was problematic as the standard training algorithms for ANNs — Backprop for feed-forward networks and Backprop-through-time (BPTT) for recurrent networks — were not applicable due to the non-differentiability of the spiking output of the neurons. Recently however, several solutions to this problem have been proposed [10, 11] and it turned out that SNNs can also compete with ANNs in practice. Therefore, there is no longer a reason to resort to ANNs when studying neuro-symbolic computation. Here, we will consider a training algorithm for RSNNs called e-prop [11]. This algorithm does not only produce competitive results for RSNNs, it is also biologically plausible and can — as opposed to BPTT — easily be implemented in neuromorphic hardware.

In the following section, we will briefly discuss the basic principles of spike-based computations in the brain and introduce spiking neural network models. We will then touch on the main theories of symbolic computation in the brain. In the main part of this chapter, we will give an overview of the state-of-the-art of neuro-symbolic computation with RSNNs. We will show in particular, through a few examples, that biologically realistic RSNNs can perform symbolic tasks on bit strings and numbers. These results show that tasks in which symbols are manipulated can be learned by models of spiking neural networks and even trained using the biologically plausible learning rule e-prop [11]. These results are not only of interest from a biological perspective to understand the mechanisms used by our brain, but also as a computing paradigm for energy-efficient neuromorphic hardware. In that light, our results can be seen as creating a bridge from

symbolic artificial intelligence to energy-efficient implementation in spike-based neuro-morphic hardware.

9.2. Neural networks of the brain and spiking neural networks

Neurons are the basic computational units of the brain. They are cells specialized for information processing and information transmission via brief (approximately 1 ms) stereotyped electrical pulses, so-called action potentials or spikes. A spike — a short, sudden increase in voltage — has an amplitude of about 100 mV and lasts 1 – 2 ms [12]. Since the spikes of a neuron are all alike, a spike itself (the form of the pulse) does not convey much information. Instead, information can be carried by the number of spikes in a certain time window, the precise timing of a spike in relation to the spikes of other neurons, and the temporal spike pattern. As the shape of the action potential is considered irrelevant, one can abstract over it and define the output of a neuron as a sequence of time points $t^{(1)}, t^{(2)}, t^{(3)}, \dots$ where $t^{(i)}$ is the i -th spike of the neuron. Such a sequence of spike times from one neuron is also called a spike train.

The brain consist of large networks of such neurons, connected via synapses. Incoming spikes to a neuron initiate changes in the membrane potential of the cell body (soma), and occasionally, if a certain voltage threshold is exceeded, the neuron generates a spike that is transmitted via synapses to other neurons.

This process is formalized mathematically in spiking neuron models. These models come at various granularities and abstraction levels, with the common characteristic that their outputs are spike trains.

9.2.1. The leaky integrate and fire (LIF) neuron model

One of the simplest spiking neuron models is the leaky integrate and fire (LIF) neuron model, see Figure 1 (left).

It captures the essential temporal dynamics of biological neurons, while abstracting over many — potentially important — aspects of its biological counterparts. The dynamics of the membrane potential V_j of a LIF neuron j is described by a first order linear differential equation:

$$\tau_m \dot{V}_j(t) = -V_j(t) + R_m I_j(t), \quad (1)$$

where τ_m is the membrane time constant (on the order of 10 to 30 ms), R_m is the resistance of the cell membrane, and I_j the input current. This equation describes a leaky integrator. The current I_j is integrated over time, however the integration is leaky, i.e., if there is no input current, the membrane potential decays to zero with time constant τ_m . The neuron outputs a spike at the time when its membrane potential V rises above the threshold v_{th} . After a spike is generated, the neuron dynamics is reset: the threshold value v_{th} is subtracted from the neuron's membrane potential, and the neuron enters a refractory period. During the refractory period, the neuron cannot spike.

We consider networks of such neurons where network neurons $1, \dots, N$ are recurrently connected. We term these networks recurrent spiking neural networks (RSNNs). RSNNs reflect two important biological facts: first, that neurons in the brain are heavily recurrently connected (in contrast to the often used feed-forward architectures in arti-

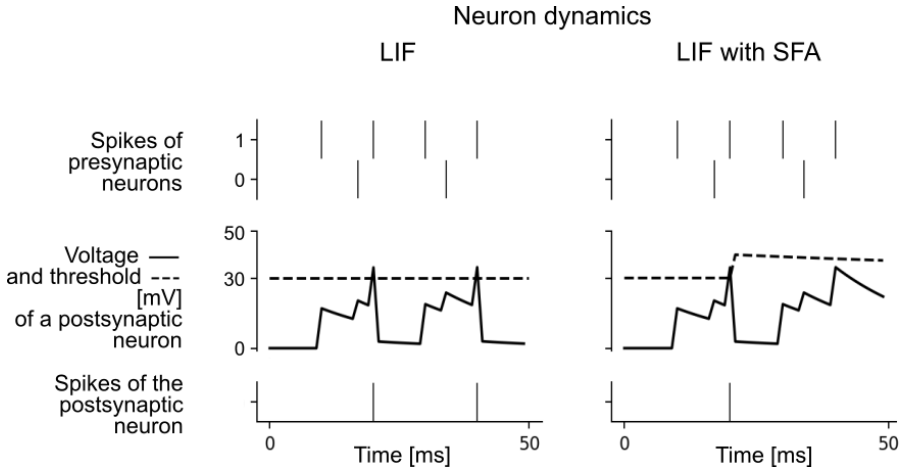


Figure 1. Neuron dynamics for LIF neuron without (left) and with SFA mechanism (right). Top to bottom: The same spike trains of two presynaptic neurons that were connected to a postsynaptic neuron, voltage traces (solid line) and threshold (dashed line) for the postsynaptic neuron without (left), and with spike frequency adaptation (SFA) mechanism (right), output spikes of the postsynaptic neuron. Due to the SFA, the postsynaptic neuron in the right panel outputs only one spike. Neuron parameters used: $\tau_m = 20$ ms, threshold $v_{th} = 30$ mV, $\beta = 1.7$ mV, $\tau_a = 100$ ms.

ficial neural networks), and second, that these neurons communicate with spikes. The input current $I_j(t)$ to neuron j in the network is defined as the weighted sum of spikes from external inputs x and other neurons in the network z :

$$I_j(t) = \sum_i W_{ji}^{\text{in}} x_i(t - d_{ji}^{\text{in}}) + \sum_i W_{ji}^{\text{rec}} z_i(t - d_{ji}^{\text{rec}}), \quad (2)$$

where W_{ji}^{in} and W_{ji}^{rec} denote the input and the recurrent synaptic weights respectively, and d_{ji}^{in} and d_{ji}^{rec} the corresponding synaptic delays. Here, the weights model the efficacies of biological synapses and are similar to the connection weights in standard artificial neural networks. The delay d_{ji} models the time that passes between an action potential of the presynaptic neuron i and the voltage response at the soma of the postsynaptic neuron j .

Neurons are usually simulated in discrete time. Writing (1) in discrete time with time-step δt and incorporating the reset after a spike, we obtain

$$V_j(t + \delta t) = \alpha V_j(t) + (1 - \alpha) R_m I_j(t) - v_{th} z_j(t) \delta t, \quad (3)$$

where $\alpha = \exp(-\frac{\delta t}{\tau_m})$ and $z_j(t) \in \{0, \frac{1}{\delta t}\}$ represents the spike output of neuron j . The term $-v_{th} z_j(t) \delta t$ implements the voltage reset after each output spike. The output z_j of the neuron is obtained by comparison with a voltage threshold v_{th}

$$z_j(t) = H(V_j(t) - v_{th}) \frac{1}{\delta t}, \quad (4)$$

with $H(x) = 0$ if $x < 0$ and 1 otherwise being the Heaviside step function.

9.2.2. Enhancing temporal computing capabilities of RSNNs with Spike Frequency Adaptation (SFA)

Upon inspection of the membrane potential dynamics of a LIF neuron, (1), we observe that spiking neurons have some internal memory about previous inputs on the time scale of the membrane time constant τ_m , which is on the order of 10's of milliseconds. This is a difference from standard artificial neurons in deep learning where the output is an instantaneous function of the momentary input. A notable exception are long short-term memory (LSTM) units [13], which turned out to be very powerful units in artificial recurrent neural networks. There, so-called memory cells can store their internal state arbitrarily long, which turned out to boost the memory capabilities of recurrent ANNs. The membrane time constant of spiking neurons is however too short to account for short-term memory at behaviorally relevant time scales of seconds in biological organisms. Instead, it has been shown that biological mechanisms at longer time scales can be utilized to boost temporal computing capabilities of RSNNs [10, 14]. One such mechanism is spike frequency adaptation (SFA), which can easily be incorporated in the standard LIF neuron model.

In contrast to LIF neurons, the threshold of a LIF neuron with SFA is dynamic, see Figure 1 (right). After each spike that the neuron produces, its firing threshold is increased by a fixed amount, and then it decays exponentially back to zero. The effective threshold $A_j(t)$ of a LIF neuron j with SFA is given by the equations

$$A_j(t) = v_{th} + \beta a_j(t), \quad (5)$$

$$a_j(t+1) = \rho_j a_j(t) + (1 - \rho_j) z_j(t), \quad (6)$$

where v_{th} is the constant baseline of the firing threshold and $\beta > 0$ scales the amplitude of the activity-dependent component $a(t)$. The parameter $\rho_j = \exp\left(-\frac{\delta t}{\tau_{a,j}}\right)$ controls the speed by which $a_j(t)$ decays back to 0, with $\tau_{a,j}$ as the adaptation time constant of neuron j . Experimental data show that many neurons in the brain exhibit SFA with time constants $\tau_{a,j}$ from hundreds of milliseconds up to tens of seconds [15, 16].

9.2.3. Training RSNNs with e-prop

Recurrently connected neural networks are trained on temporal sequences of input-output pairs via Backprop-through-time (BPTT) – an algorithm that unfolds the recurrent network in time and operates in two phases. In the first phase, a forward pass is performed, that is, the network outputs are computed, and a cost function that measures the distance between the network's outputs and desired values is evaluated. In the second phase, by applying the chain rule of calculus, the derivatives of the cost function with respect to the parameters, i.e., synaptic weights, are evaluated for each time step, and the synaptic weights are updated. This approach requires the full history of the network activity to be stored, hence it is quite unlikely that the brain performs such an algorithm for learning. An alternative to the BPTT algorithm that is more biologically realistic is the e-prop learning rule [11]. Unlike BPTT, for updating the synaptic weights of the network this learning rule requires only information locally available to each synapse and neuron – an eligibility trace and a learning signal broadcasted across the network. Eligibility

traces model preceding activity in neurons and synapses at the molecular level, whereas learning signals model top-down signals (e.g., dopamine, acetylcholine, and neural firing related to the error-related negativity) that target different populations of neurons and govern the synaptic plasticity, i.e., the process of adjusting the strengths of synaptic weights.

More precisely, consider a cost function E that should be minimized in the learning process. The gradient $\frac{dE}{dw_{ji}}$ of the cost function with respect to the synaptic weight w_{ji} , needed for the update of this weight can be calculated as follows:

$$\frac{dE}{dw_{ji}} = \sum_t L_j(t) e_{ji}(t), \quad (7)$$

with $L_j(t)$ representing the learning signal for neuron j , and $e_{ji}(t)$ the eligibility trace for the synapse from neuron i to neuron j . In e-prop, one uses an approximate learning signal

$$L_j(t) = \sum_k B_{jk}(y_k(t) - y_k^*(t)) \quad (8)$$

which captures errors occurring at the output neurons k at the current time step t , that is, deviations of the outputs $y_k(t)$ from the targets $y_k^*(t)$ are calculated, and fed back from output neuron k to every network neuron j via randomly chosen weights B_{jk} . The eligibility trace $e_{ji}(t)$ represents a transient memory for the synapse connecting the presynaptic neuron i and postsynaptic neuron j , that is, it integrates the history of the synapse up to time t based on locally available information. For a LIF neuron, it is based on a low-pass filtered version of the presynaptic spike train and the postsynaptic membrane potential.

The learning signal $L_j(t)$ only takes into account the current time step t , however, the eligibility trace $e_{ji}(t)$ accounts for the past of the neuron. Since they are readily available in every time step t of the forward phase, as soon as the forward phase is completed, the updates of synaptic weights can be performed, without performing the backward phase as required in BPTT. There is no need for unrolling the recurrent network in time for calculating the errors from previous time steps as in BPTT, and as such, e-prop is an online learning method for training models of recurrently connected neurons. For an illustrative, short summary of the e-prop learning rule, see [17].

9.3. Symbolic computation and arithmetics in the brain

9.3.1. Representation of numbers in the brain

There exists experimental evidence for at least two types of neural codes for numbers in the cortex. It has been shown that individual neurons in the lateral intraparietal area (LIP) exhibit a Gaussian tuning for cardinality when macaque monkeys perform a number matching task [18]. A population of such neurons, each tuned to a different represented value, can collectively encode a number. To make this more precise, we describe here a simple mathematical model for this coding scheme which will also be used in

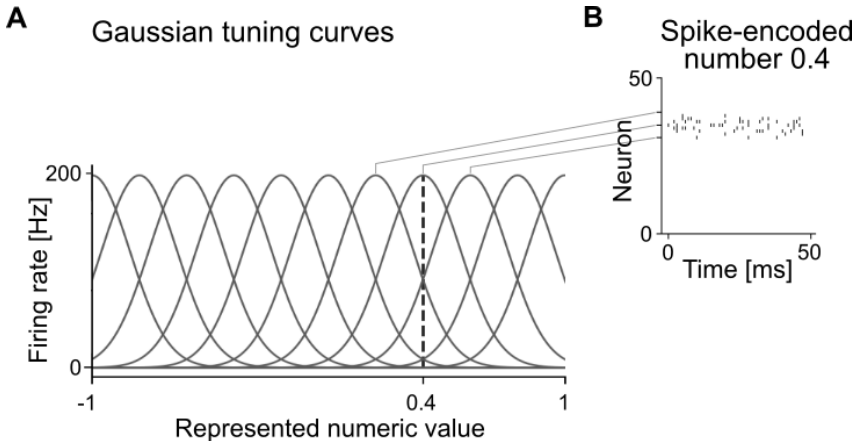


Figure 2. Population coding of numbers. This encoding was used in the middle panel of Figure 4C to encode numbers. (A) The numeric value from $[-1, 1]$ that is represented by a population of 50 neurons is indicated on the x-axis. The y-axis indicates the firing rate of 11 neurons out of this population in response to the corresponding number. Each neuron responds with its maximal firing rate to its preferred value (at the peak of its Gaussian tuning). When the represented value is off this preferred value, the firing rate decreases according to the indicated Gaussian profile. The dashed vertical line indicates one example value of 0.4. The neuron with its tuning curve centered on this value fires at 200 Hz. The two neurons with the shown neighboring tuning curves fire at approximately 80 Hz. (B) Spiking activity of 50 spiking neurons of this population when the number 0.4 was encoded, as indicated in panel A. Each row corresponds to one neuron of the population, where neurons are sorted by their preferred value from -1 to 1 . Each tick indicates one spike. Gray lines connect the centers of tuning curves in A with the corresponding neurons in panel B.

later sections in our simulations. For a represented number z , each neuron i in such a population spikes with a firing rate of

$$f_i = f_{\max} \exp\left(-\frac{(m_i - z)^2}{2\sigma^2}\right), \quad (9)$$

where $f_{\max}$ is the maximum firing rate of neurons in the population, m_i is the value to which the neuron is tuned to, and σ is the standard deviation of the Gaussian tuning curve which is chosen here to be equal for all neurons in the population for simplicity. The preferred values m_i for these neurons are typically chosen equally spaced in the range of numbers that can be represented. Hence, each neuron responds with its maximal firing rate $f_{\max}$ if its preferred value m_i is represented. When the represented value is off this preferred value, the firing rate decreases according to a Gaussian profile. Such population coding of numeric values between -1 and 1 is illustrated in Fig. 2 for a population of 50 neurons. This coding scheme abstracts over input stimulus properties, as it is preserved over input modalities such as visual or auditory input [19].

In other tasks, some neurons in LIP fire more strongly for larger numbers and others show the inverse behavior, hence exhibiting a spike frequency code [20]. It has been hypothesized that these two types of codes may separate the actual entity represented (i.e., the number represented with a Gaussian tuning) from the structure of the number space (a linear firing rate code which captures similarity between numbers) [21, 22]. The authors also state that structure-disentangled codes are not unique to numbers and magnitude, rather this might be a general coding principle for structured information

(e.g., space). There exists evidence for similar coding principles in humans. EEG activity varies smoothly with numerical distance when Arabic numbers are presented to humans [23, 24].

9.3.2. Circuits for arithmetic in the brain

Little is known about the specific circuit mechanisms that implement arithmetic operations in the brain. Several cortical regions are activated during arithmetic tasks. Here, the strength of activation of specific areas often depends on the type of task (e.g. approximate versus exact), task difficulty (e.g. $2 + 3$ versus 27×32), and expert level of the subject [25]. In general, frontal areas are more strongly involved in more complex calculations [26]. Interestingly, several lines of research indicate that growing expertise leads to the increased involvement of specific parietal areas whereas frontal areas become less strongly involved [25]. This supports the theory that arithmetic operations can be performed by more general-purpose processing, but practice may train special purpose circuits which can perform the trained tasks with less effort. Our novel results below show that training of complex arithmetic operations can be performed on surprisingly small SNNs.

9.3.3. Symbolic computation

A symbol has meaning and significance; it refers to ideas, objects, relationships and concepts [27]. There are a few important aspects to consider when studying the emergence of meaning from symbols, for example, the establishment of links between symbols and objects/concepts that they represent, learning symbolic meaning from context, or generalization over a range of instances of symbols. Also, different brain regions are involved in general meaning processing, but semantic integration mechanisms draw on higher cortical areas of the neocortex, such as prefrontal, posterior parietal, anterior, inferior, and posterior temporal cortex [28].

Unlike computations performed directly on values such as the activities of sensory receptors, symbolic computation addresses the question of the manipulation of symbols using rules. A symbolic expression can be expressed using variables that are instantiated by concrete values. In algebra, abstract variables x or y can be substituted by numbers; in language, terms *subject* or *verb* can be replaced by words. The abstract terms act as placeholders, and they are a higher-order representation of knowledge. Through the concept of variable binding — how the placeholders are instantiated by concrete values — one can study how computations performed by neurons give rise to cognitive processes, and eventually elucidate the underlying process in the brain which contribute to language and abstract thought [4].

Symbols can represent categories (e.g., *cat*, *dog*), variables (e.g., x , y), computational operations (e.g., ‘+’, ‘−’, *concatenate*, *compare*), and individuals (e.g., *Felix*) [2]. The mind is a system that represents variables, operations over variables, and structured representations, capable of distinguishing between categories and individuals. A possible physical implementation of such a system would be a set of buckets, with the bucket’s content representing an instantiation of a category represented by the bucket. Digital computers already use binary registers that correspond to buckets being either empty or full (bits 0 or 1), and the operations over the registers are performed in parallel. Next,

each variable should correspond to an anatomically defined region or register. For example, the symbol “black” in “black cat” or “black sedan” should apply equally, irrespective of the categories animals and cars that may be represented in widely separated regions of the cortex [4].

Several neural network models have been proposed for the implementation of symbolic computation via variables and variable-binding mechanisms that bind specific content to a given variable. Pointer-based models (e.g., [29]) assume that pointers are implemented by single neurons or co-active populations of neurons which are synaptically linked to content. Another class of models is based on the idea of indirect addressing. These models assume that a neural space for a variable encodes an address to another brain region where the corresponding content is represented [30, 31]. In anatomical binding models, different brain areas are regarded as distinct placeholders [32, 33]. In Dynamically Partitionable Autoassociative Neural Networks (DPANN) [32], a given symbol corresponds to a global attractor state of the dynamics of a large-scale network that links many regions of the brain. Information flow between particular sets of registers is achieved through turning on and off subsets of synapses, i.e., gating. In neural blackboard architectures, one assumes that besides assemblies for content, there exist also assemblies for structural entities such as phrases, semantic roles, et cetera. Specialized circuits (so-called gating circuits and memory circuits) are then used to bind content to structure [34]. This way, complex syntactic structures can be built. Vector-symbolic architectures [35] use high-dimensional vectors to represent the symbolic concepts encoded by activity patterns of a group of neurons and a defined set of mathematical operations. Both the variables and potential role filters, such as subject, object or verb, are represented as vectors. Compositional operations typical for symbol manipulation are then implemented through a mathematical transformation of these vectors. Activity patterns for binding, for example, a subject and an instance, are achieved through multiplication of two vectors [36]. Most of these models rely on specifically constructed circuitry that performs the binding operations. In contrast, a rather generic SNN model was proposed in [37]. In this assembly pointer model, binding emerges through local Hebbian plasticity processes from the activation of assemblies in areas that contain variables to areas that contain content.

9.4. Prior work on symbolic computations on bit strings and numbers with SNNs

Some of the above mentioned neural network models for variable binding have been implemented in SNNs (e.g., [36, 37]). In this review, we concentrate more specifically on symbolic computations on numbers, and in contrast to the above mentioned models, we propose a more generic approach where SNNs are trained to perform specific tasks.

In a recent work [14], a spiking neural network was trained on two tasks based on a rule that was given at the beginning of each trial. For the same input sequence, the task of the network was either to reproduce the sequence in the same order, if the rule was *duplicate*, or the sequence in reversed order if the rule was *reverse*. The SNN had to learn to observe data stimuli (symbol sequences) given as input, and two different rules on how to manipulate them, also given as an input, independent of each other, then to use both data stimuli and the given rule to produce an output. After training, the SNN was able to produce sequences with all correct symbols in 95.88% of test trials. Through

an analysis of spiking activity on the test trials, groups of neurons that had specialized in encoding structural information of the task, such as symbol identity, position, and task identity, were identified. The analysis revealed that many neurons specialized in encoding non-linear combinations of conditions, indicating a non-linear mixed selectivity of neurons. In other words, while some neurons showed some preference in their tuning to specific aspects necessary to solve the tasks, there was no clear-cut encoding of symbols by individual neurons in this trained circuit — different from what one would usually do to construct a neural network to perform these operations. Such mixed selectivity of neurons has also been observed in higher cortical areas [38].

In another experiment of [14], it was shown that SNNs can learn to follow dynamic rules, i.e., make decisions, for example, choose the “Left” or “Right” button, depending on the currently active rule and specific subsequences of symbols relevant for that particular rule. In a long sequence of symbols, the currently active rule, relevant and distractor symbols appeared sequentially. Even for humans, the task is quite demanding, since it requires not only the maintenance and an occasional update of the active rule, but also maintaining relevant symbols and ignoring irrelevant ones until the target subsequence is complete. Such a task requires a hierarchical working memory – higher-level memory for maintenance of the active rule, and lower-level memory for maintenance of the symbols relevant in the context of the active rule, especially because the distractor symbols occur rather frequently. A generic SNN was able to solve this task almost perfectly without requiring a special-purpose architecture. In 97.79% of trials consisting of sequences of 90 symbols, all choices of the button “Left” or “Right” were correct. This result shows that cognitively demanding sequence processing tasks can be solved by generic SNNs.

9.5. New results on spike-based symbolic computation on bit sequences and numbers

In this section, we present new results on how SNNs can perform symbolic computations on bit sequences and numbers. Human infants can perform mental arithmetic using non-symbolic, approximate number representations. Even before children learn algorithms for manipulating numerical symbols and formal mathematics, they are able to perform approximate addition and subtraction of large numbers. In such experiments, children performed well above chance – they were able to do so by comparing quantities instead of providing an exact solution [39]. Certainly, we use non-symbolic representations before we master symbolic arithmetic, which takes years [40]. Some authors refer to this capability as “the number sense” [41].

There are many open questions: How do humans perform arithmetic? How are the numbers encoded? What are the mental representations behind the comparison of two numbers or quantities? For the case of symbolic arithmetic, we face even more general questions – those of symbol manipulation and sequence processing.

Mastering symbolic representations and symbol processing, in general, helps us perform cognitively demanding tasks. Certainly, the human brain is capable of processing and manipulating symbols, or even sequences of symbols, thereby following the abstract relationships between mental concepts (symbols). These abstract relationships are known as rules, and the brain can process sequences of symbols even when rules dynamically change or when the stimulus is novel [2].

Given a sequence of items, humans are able to infer the rules that generated the sequence and apply the same rule even on sequences of items that they had never encountered before [42]. If we revisit the question of numbers, for example, Arabic numbers and their representation, we notice that single-digit numbers are represented by symbols 0 to 9. Multi-digit numbers are represented by sequences of symbols, where the position in the sequence is of particular relevance. Moreover, we learned the meaning of the most significant and least significant digit, and that the length of the sequence representing the number carries another crucial piece of information. To compare two numbers given in a symbolic form, we can first rely on the rule of comparing the lengths of sequences. In case the lengths are the same, we immediately proceed by applying the second rule - comparing the corresponding significant digits, iteratively if necessary, starting from the most significant digits towards the least significant ones, but we stop when we can unambiguously conclude which number is greater, or after we compared the least significant digit. In fact, we can perform such sophisticated sequence processing during the comparison of two symbolic numbers with little effort. We have mastered this processing during years of practice through schooling, and we are applying the learned rules rather automatically. This subjective observation is backed up by experiments which suggest that specialized cortical circuits learn complex arithmetic operations during training [25], see Section 9.3.2.

We performed two experiments – both of them show that rather small, generic SNNs can be trained to perform demanding sequence processing tasks. In the first experiment, we considered the number comparison task described above, a task that has often been used to study number processing in cognitive science [43]. For simplicity, we used binary numbers instead of previously described Arabic numbers, since the same sequence processing rules for comparison also apply when comparing numbers in any base. Note that, the rules describing how to compare the numbers were not explicitly given to the network, rather, the network learned to perform the task through training.

In the second experiment, an SNN was trained to perform arithmetic operations (addition, subtraction, and multiplication) on real numbers encoded into spiking activity using population coding with Gaussian tuning, a number encoding that has been observed in monkeys, see Section 9.3.1. Arithmetic expressions — the instructions indicating the rule to be applied on the data stream, i.e., real numbers — were given to the network in a separate stream, and sequentially. The challenge of this task was to incorporate pieces of information processed earlier, that is, to reuse the result of the computation from the previous step while following new instructions. Without explicitly being trained to do so, the network had also to solve the binding problem – between variables that appeared in the expression and the neurons encoding the number. The network learned successfully to process a sequence of arithmetic expressions, and similar to humans, to perform at a level of expertise.

9.5.1. Spike-based symbolic computation on bit strings

The comparison of two numbers is a task often considered in cognitive science (see e.g., [43] for a recent example). We wondered if an SNN could learn to perform the comparison task on expressions with numbers encoded as bit strings. We refer now to the numbers represented in a binary base as bit strings rather than sequences, since we use the term sequence to denote the (temporal) processing dimension and that symbols appear sequentially, one after the other.

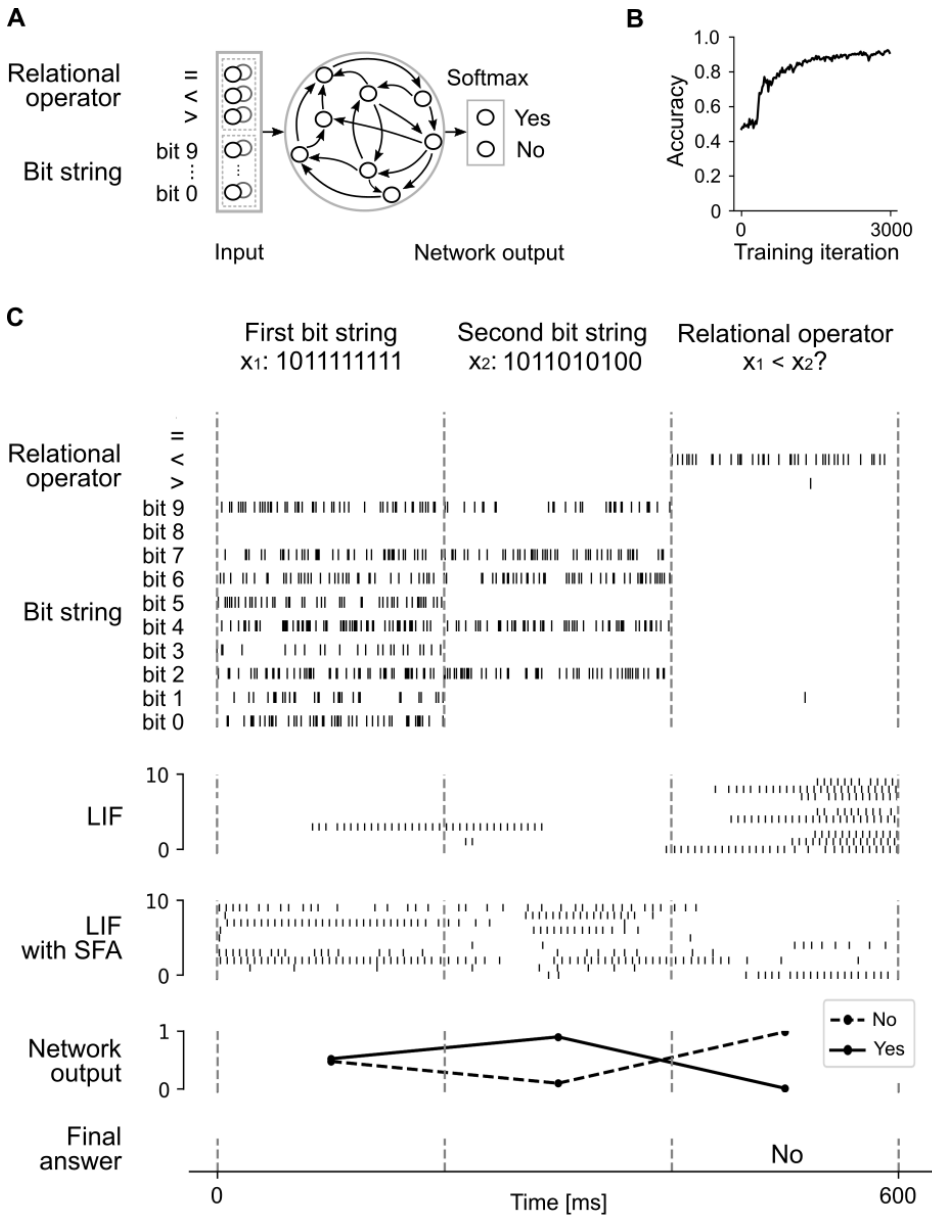


Figure 3. Comparison of two bit strings. (A) The schematic network architecture: input spiking neurons encoding relational operators and bit strings of width 10 (bit 9 the most significant, and bit 0 the least significant bit), recurrently connected LIF neurons, and two output neurons. The network output is the softmax of the output neurons producing “Yes/No” answers. (B) Training curve: Accuracy over training iteration. Test accuracy for novel trials was 0.8723. (C) Spiking activity of the network for a trial where the network compared if the first bit string $x_1 = 1011111111$ was smaller than the second bit string $x_2 = 1011010100$. Top to bottom: Spiking activity of neurons encoding the relational operator and bit strings (one neuron shown out of 2 for each symbol), 10 sample out of 100 LIF neurons, 10 sample out of 100 LIF neurons with SFA, network output – the softmax applied to readout neurons dedicated to producing the answer, the final (maximum value of those for “Yes” and “No” neurons) and correct answer “No” to the question “ $x_1 < x_2?$ ”.

The schematic network architecture is shown in Fig. 3A. The input to the network were binary strings of width 10 bits representing numbers in the binary base. Another input stream represented the relational operator to be applied, where we used *equals* ‘=’, *less than* ‘<’, and *greater than* ‘>’. Variables x_1 , x_2 , and the relational operator were presented to the network sequentially as Poisson spike trains. A Poisson spike train is a random sequence of spikes with a given rate (i.e., expected number of spikes per second), similar to spike trains observed in the brain. Mathematically, such a spike train is generated by a Poisson point process. Each presentation of a variable or relational operator lasted for 200 ms, with high firing activity coding for 1 and low firing activity coding for 0, see Fig. 3C, top. Only after the full sequence indicating the first number x_1 , the second number x_2 , and the relational operator *rel_op* has been shown, does the network have all the pieces of information required to determine whether the provided relation $x_1 \text{ rel_op } x_2$ was correct. As the answer, the network was required to output “Yes” or “No”.

We trained a recurrently connected spiking neural network consisting of 200 LIF neurons (100 LIF neurons with SFA, 100 without) on this task using the biologically plausible learning algorithm e-prop [11], for which the accuracy over training iteration is shown in Fig. 3B. The curve was smoothed using bins of 20 values. Fig. 3C illustrates a trial where the network had to answer whether the binary number $x_1 = 1011111111$ was smaller than binary number $x_2 = 1011010100$, for which the final answer is “No”. Tested on previously unseen instances $x_1 \text{ rel_op } x_2$, that is, combinations of bit strings and relational operators that were not used during training, the proportion of correct answers (accuracy) achieved was 0.8723 ± 0.0235 (mean $\pm$ standard deviation), averaged over 5 different network initializations.

This result demonstrates that rather small SNNs can learn to perform arithmetic tasks using a biologically plausible learning algorithm. In this task, numbers were represented in a symbolic manner (i.e., using symbol strings as in Arabic numbers) rather than as magnitudes, and the network had no prior knowledge on the structure of this number encoding or on the structure of the task. Nevertheless, the network was able to generalize to novel task instances, indicating that explicitly designed symbolic computations are not necessary for generalization for this task.

9.5.2. Evaluation of nested arithmetic expressions

One of the most basic arithmetic tasks is the evaluation of an operation applied to two numbers such as $12 + 4$. One can replace numbers in such expressions by variables to obtain an expression such as $x_1 + x_2$. The solution to the expression can be obtained when the values of the variables are defined. For example, defining $x_1 = 12$ and $x_2 = 4$, the result for the above expression is 16. This calculation demands a basic type of computation where values need to be assigned to variables which then have to be combined by some arithmetic operation. We wondered whether such basic symbolic computations on numbers and variables could be learned by SNNs. However, we took a further step by considering long nested expressions built based on this basic template with three basic arithmetic operators addition ‘+’, subtraction ‘−’, and multiplication ‘*’. An example of such a complex expression is

$$\begin{array}{c}
 (((((((((x_2 * x_4) * x_5) * x_3) - x_3) + x_5) * x_4) - x_5) * x_4) * x_1) - x_4) \\
 \underbrace{\hspace{1.5cm}}_{= r_1} \\
 \underbrace{\hspace{1.5cm}}_{= r_2} \\
 \vdots
 \end{array}$$

The expressions we considered have a nested structure, where the result r_1 of the evaluation of the expression in the innermost nesting level, here, $(x_2 * x_4)$ is used to evaluate the result r_2 for the next nesting level, and so on, until the final result is obtained. We considered nested expressions of 10 operators combining 5 variables, leading to 9 intermediate results $r_1, \dots, r_9$ and the final result r_{10} of the evaluation.

More precisely, we define the task as the sequential evaluation of expressions in 10 steps $i = 1, \dots, 10$ as follows:

$$\begin{aligned}
 i = 1: & \quad r_1 = (\text{Operand}_1 \text{ operator}_1 \text{ Operand}_2), \\
 i = 2: & \quad r_2 = (r_1 \text{ operator}_2 \text{ Operand}_3), \\
 i = 3: & \quad r_3 = (r_2 \text{ operator}_3 \text{ Operand}_4), \\
 & \quad \vdots \\
 i = 10: & \quad r_{10} = (r_9 \text{ operator}_{10} \text{ Operand}_{11}),
 \end{aligned}$$

where in the first step ($i = 1$), Operand_1 and Operand_2 are drawn uniformly over $\{x_1, \dots, x_5\}$ without replacement, and operator_1 was drawn uniformly over $\{+, *\}$ (the operator ‘ $-$ ’ was omitted in this step to avoid the need of encoding the order of Operand_1 and Operand_2). For the following steps, each of $\text{operator}_2, \dots, \text{operator}_{10}$ was drawn uniformly from $\{+, -, *\}$, and each of $\text{Operand}_3, \dots, \text{Operand}_{11}$ uniformly over $\{x_1, \dots, x_5\}$. This describes the generation procedure for the symbolic part of the task, but numeric values are also required in order to evaluate such nested expressions. We allowed that the values of variables can change after each step i . This was done by drawing in each step i for each of the variables $x_1, \dots, x_5$ a random value from the uniform distribution over $[-1, 1]$. An example value assignment and target results for the expression given above are shown in Table 1.

We show here that an SNN can be trained to evaluate such complex arithmetic expressions. The nested expression was given sequentially, starting from the expression in the innermost nesting level ($x_2 * x_4$ in the above example), together with the numeric assignments to all operands $x_1, \dots, x_5$. These inputs were encoded by spiking input populations (see below for the encoding) and presented to the network for 50 ms, see Fig. 4A for a schematic of the input populations. The network was demanded to produce at its readout the intermediate result r_1 . Then, the expression for the next nesting level ($*x_5$ in the above example) was provided for 50 ms and the result r_2 was demanded as the output, and so on, see Fig. 4C for the temporal sequence of inputs to the network.

Consequently, the challenges for the network were the processing of symbols with associated numeric values, i.e., it had to learn to bind a concrete value to a variable, and at the same time, carry out the demanded mathematical operations in an online fashion. An example of computations performed by a trained SNN on the previously mentioned nested expression is illustrated in Table 1.

The schematic architecture of a spiking network that we trained is illustrated in Fig. 4A. Symbolic inputs to the network were encoded by subpopulations of neurons

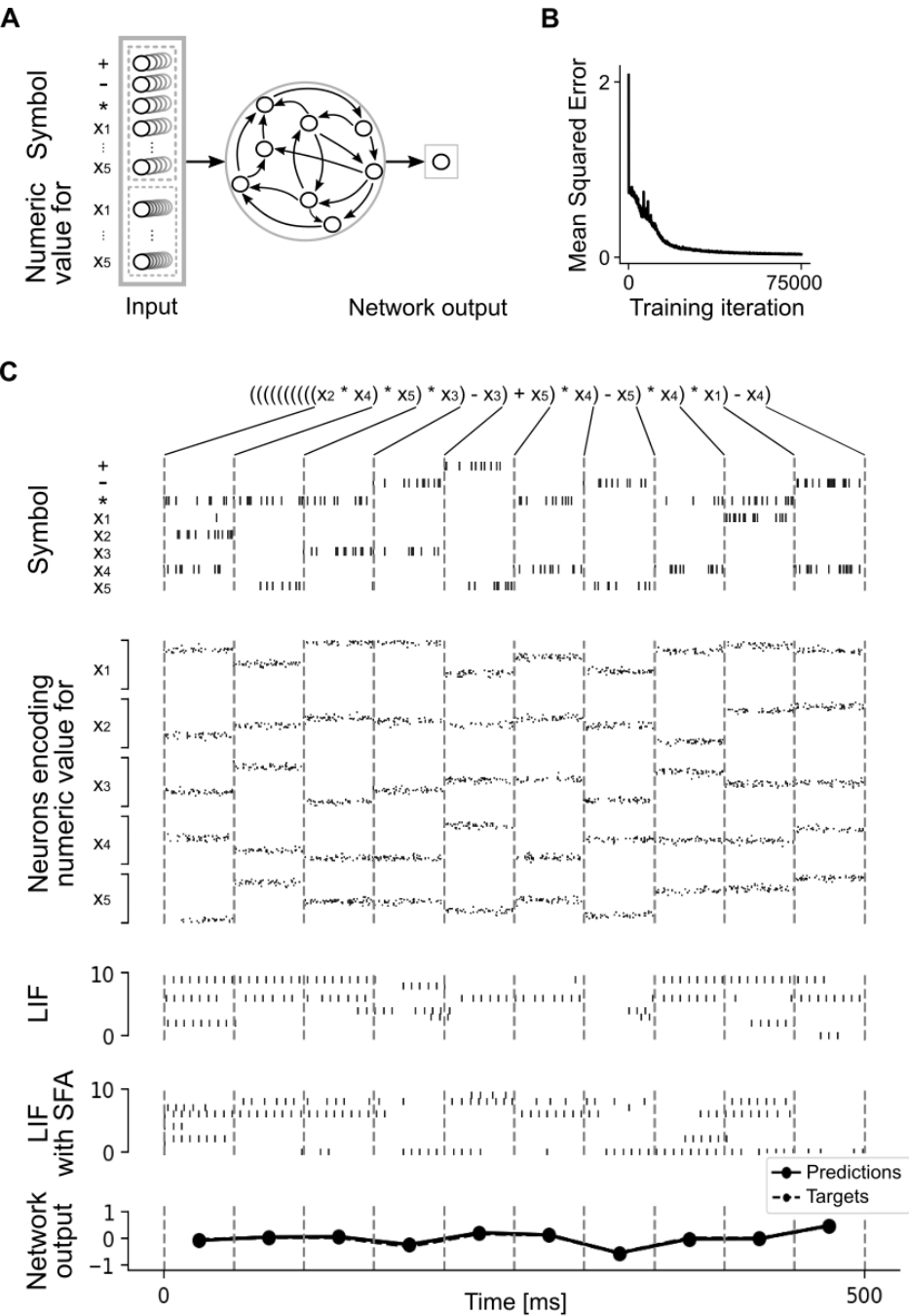


Figure 4. (Caption on the next page.)

Step i	Arith. expression	Numeric expression	Output r_i	Target	Absolute error
1.	$x_2 * x_4$	$0.4966 * (-0.1113)$	-0.0794	-0.0553	0.0241
2.	$r_1 * x_5$	$(-0.0794) * (-0.7100)$	0.0510	0.0564	0.0054
3.	$r_2 * x_3$	$0.0510 * 0.8209$	0.0629	0.0419	0.0210
4.	$r_3 - x_3$	$0.0629 - 0.3314$	-0.2308	-0.2685	0.0377
5.	$r_4 + x_5$	$(-0.2308) + 0.4717$	0.2096	0.2409	0.0313
6.	$r_5 * x_4$	$0.2096 * 0.7014$	0.1304	0.1470	0.0166
7.	$r_6 - x_5$	$0.1304 - 0.6650$	-0.5417	-0.5346	0.0071
8.	$r_7 * x_4$	$(-0.5417) * (-0.0165)$	-0.0222	0.0089	0.0311
9.	$r_8 * x_1$	$(-0.0222) * (-0.8182)$	-0.0128	0.0182	0.0310
10.	$r_9 - x_4$	$(-0.0128) - (-0.4727)$	0.4739	0.4599	0.0140

Table 1. Evaluation of nested arithmetic expressions with an SNN. One example input sequence with network outputs, targets, and errors. In each input step i — each step lasting for 50 ms — an arithmetic expression and numeric values were given to the network, for which the network produced the output r_i . Output values were very close to target values, that is, the absolute error was close to zero.

Figure 4. Spike-based arithmetic on numbers and variables. (A) The schematic network architecture consisting of input spiking neurons that encoded symbols used in arithmetic expressions (operators and variables) and numeric values for variables, recurrently connected LIF neurons, and a linear readout neuron that produced the result for each arithmetic expression. (B) Training curve (Mean Squared Error over training iteration) for the network trained to evaluate nested mathematical expressions. The curve was smoothed using bins of 50 values. (C) Spiking activity of the network for the nested arithmetic expression shown at the top. Solid lines from the expression to dashed lines of the spike raster indicate the time when this subexpression was presented to the network. Spike rasters from top to bottom: Spiking activity of neurons encoding the symbols appearing in the arithmetic expression (one neuron shown out of 5 for each symbol), encoding of numeric values that instantiate variables, 10 sample out of 150 LIF neurons (without SFA), 10 sample out of 150 LIF neurons with SFA, real-valued network outputs (predictions) that almost perfectly match targets (and consequently, the dashed line representing targets is difficult to see).

producing Poisson spike trains, dedicated to each symbol $\{+, -, *, x_1, \dots, x_5\}$ (5 neurons per symbol) and indicating the presence (absence) of a symbol through a high (low) firing rate. Numeric values for each of variables $x_1, \dots, x_5$ were encoded by another set of 5 subpopulations of neurons, each consisting of 50 neurons using a population coding scheme. See Fig. 2 for an illustration of population coding of a single number with a population of 50 neurons. Each neuron of this population was assigned a preferred value for which it exhibited its maximal firing rate of 200 Hz. When the represented number was off this preferred value, its rate decreased according to a Gaussian profile. The preferred values of the neurons in this population uniformly tiled the whole space of values between -1 and 1 (Fig. 2A). Hence, for a specific represented value, neurons with preferred tuning close to the value were active, while others remained mostly silent, see Fig. 2B.

As illustrated in Fig. 4A, the spiking activity of the input populations was projected to an RSNN. The output of the network was a single linear non-spiking neuron (a readout neuron), where the numeric value of its output (a real number) was interpreted as the result of the computation. We trained an RSNN consisting of 300 LIF neurons (150 with SFA and 150 without SFA) with all-to-all connectivity using e-prop for 75000 iterations on this task. The objective minimized was the Mean Squared Error (MSE) between the

network outputs (predictions) and targets for all intermediate and the final result of the arithmetic expression. The evolution of the network error (MSE) over the training iterations is shown in Fig. 4B. The network achieved an MSE of 0.0341 and a mean absolute error (MAE) of 0.1307 in the mean over all intermediate results, i.e., all computational steps in all of 5000 test trials.

The column “Absolute error” of the example illustrated in Table 1 confirms that the network learned the task very well. The spiking input (encoding of symbols, and of numeric values for variables) and network spiking activity of LIF neurons for the same trial is shown in Fig. 4C, where the bottom panel shows an almost perfect match of outputs (predictions) and targets.

We note that we made sure that the nested symbolic expressions used in the testing trials were different from any nested expression used during training. Also, the used numeric values for the operands were drawn randomly, which, altogether, indicates good generalization capabilities of the trained SNN. This result shows that nested arithmetic expressions can be learned by rather small SNNs to high precision with a biologically plausible learning algorithm. Again, no explicit mechanisms for variable binding were necessary.

9.6. Conclusions

Most studies of computing capabilities of RSNNs have focused on sensory processing tasks. But the brain uses RSNNs also for higher cognitive functions such as symbolic computations, in particular, mathematical computations. We have reviewed recent results, and added two new ones, which show that RSNNs are very good at these tasks. Moreover, they are able to acquire these capabilities through a biologically plausible learning method – e-prop, and they are able to apply learned computing procedures to new instances of the task that never occurred during training. This is somewhat surprising, since RSNNs do not compute in a clocked mode, like digital computers or ANNs. Rather, they compute in an asynchronous mode where also the timing of spikes carries information. Also, the shown examples for such RSNNs operate in a rather low firing rate regime, where they could not simply emulate ANNs by encoding continuous values through firing rates. Thus we arrive here at a new paradigm for symbolic computation in neural networks that provides concrete hypotheses about the organization of symbolic computations in the brain. Furthermore, since RSNNs can be implemented more energy-efficiently in neuromorphic hardware than ANNs, the results that we have discussed also suggest new methods for improving the energy efficiency of computing hardware for symbolic computations.

9.7. Technical details to new results

9.7.1. Network architecture and neuron model

We trained a recurrent spiking neural network of LIF neurons with all-to-all connections. Since in both experiments that we demonstrated temporal integration of pieces of information was crucial for a good performance, a fraction of LIF neurons (one half) was

equipped with spike frequency adaptation (SFA). LIF neuron model was described in Section 9.2.1, and LIF neuron model with SFA in Section 9.2.2.

The firing threshold of LIF neurons was $v_{th} = 30$ mV. The same value of 30 mV was used as a baseline firing threshold v_{th} for LIF neurons with SFA, with an adaptation strength of $\beta = 1.7$ mV (see (5)). All LIF neurons had a synaptic delay of 1 ms, a refractory period of 5 ms, and a membrane time constant of $\tau_m = 20$ ms. The resistance of the cell membrane was $R_m = 1$ G Ω . A population of input neurons encoded inputs to the recurrently connected neurons in the form of spike trains. The network output was produced by linear readout neurons, from the average activity of recurrent neurons in the network during the time window representing one step of a trial.

9.7.2. Training method e-prop

In all tasks, input, recurrent, and readout weights were trained simultaneously. To train the weights of the network by the learning rule random e-prop [11], we used the Adam optimizer with a learning rate of 0.001. The input and recurrent weights were initialized with values from a standard normal distribution, and then scaled with a factor $\frac{1}{\sqrt{n_{in}}}$, and $\frac{1}{\sqrt{n_{rec}}}$, respectively, with n_{in} being the total number of input neurons, and n_{rec} the number of recurrent neurons in the network. The readout weights (weights from recurrent to output neurons) were initialized with values from a normal distribution with mean zero, and standard deviation 1.05. The unit for all input and recurrent weights was *pAs*.

9.7.3. Tasks

Comparison of bit strings. An SNN consisting of 200 recurrently connected neurons was trained for 3000 iterations, with a batch size of 100, to answer the “Yes/No” questions of the form “Is x_1 Equal to / Smaller than / Greater than x_2 ?”. Inputs to the network were two binary strings x_1 and x_2 of width 10 bits, and in fact, represented binary encoded numbers. Binary strings x_1 and x_2 were given successively in the first and the second step of each trial, then followed by a relational operator $=, <$ or $>$, given in the third step. In the third, final step, the network had to produce a “Yes/No” answer to the question that got complete, after all three important pieces of information were shown to the network. Each step lasted 200 ms, hence the total duration of the trial was 600 ms. 10-bit binary strings x_1 and x_2 were encoded into spike trains by a population of 20 spiking neurons – two neurons were dedicated to encoding each bit. Relational operators $\{=, <, >\}$ were encoded using 6 input spiking neurons (2 neurons per operator), in each trial exactly one operator being given in the third step. If a bit represented a value of 1, or if the operator was used in the expression, dedicated neurons fired with a high rate of 200 Hz. To encode a bit value of 0 or that the operator was not used in the expression, neurons fired with a low rate of 2 Hz. Such spike trains encoding the inputs were received by a population of recurrently connected LIF neurons – 100 LIF neurons with SFA and 100 without. Neurons with SFA had an adaptation time constant uniformly distributed from [1, 600]. The linear readout consisted of 2 neurons on which we applied softmax – one neuron was dedicated to “Yes” and one for “No” output. Final answer was determined from the maximum value after the softmax – the higher value indicated either “Yes” or “No”.

The loss function minimized was cross-entropy function with an additional regularization term. The regularization term represented the squared difference between the average firing rate of the recurrent neurons and a target firing rate of 20 Hz, scaled by a factor of 10. On average, in 87.23% of, in total, 10000 testing trials, the network produced the correct answer. The average was calculated over 5 different initializations of networks with the same architecture and trained in the same way (the weights and adaptation time constants of LIF neurons with SFA differed in each run). The inputs in each test trial were novel, i.e., not used during training.

Evaluation of nested arithmetic expressions. An SNN consisting of 300 recurrent neurons was trained for 75000 iterations, with a batch size of 50 to perform symbolic computation on nested arithmetic expressions involving numbers and variables. Symbolic input to the network was an arithmetic expression formed by the variables $x_1, \dots, x_5$ and arithmetic operators $\{+, -, *\}$. In the first step of each trial of length 10, two different variables were drawn randomly to form a valid expression with two operands, and the possible operator was limited to either addition (+) or multiplication (*), to avoid the problem of encoding the order of operands appearing in the expression, which a non-commutative operator subtraction (-) would require. In all other steps of a trial, a variable and an operator were drawn randomly over $\{x_1, \dots, x_5\}$ and $\{+, -, *\}$ respectively – one variable was necessary, since the result of the previous step was considered as the value that always instantiated the first operand in the subexpression. If a variable $x_1, \dots, x_5$ or operator $\{+, -, *\}$ used in an arithmetic expression appeared in a step of the trial, they were encoded by the “high” spiking activity of dedicated 5 neurons (200 Hz), otherwise by their “low” activity (1 Hz). The numeric inputs to the network were analog values (numbers from $[-1, 1] \in \mathbb{R}$) and encoded by a subpopulation of 250 input spiking neurons. In each step, different numeric values that instantiated the variables $x_1, \dots, x_5$ were given, but the network had to learn the binding (“variable - concrete number”), and also to ignore all unnecessary values if the variable did not appear in the current expression. No special modules that would help the binding of variables with numeric values were included. Each analog value used in arithmetic expressions was encoded using a population coding scheme with a population of 50 input neurons. Neurons in this population were tuned to numbers using Gaussian tuning curves where neuron i responded maximally with a firing rate of $f_{\max} = 200$ Hz if the represented number was equal to its preferred value m_i . The preferred values m_i for these 50 neurons were chosen equally spaced in the input domain $[-1, 1]$. The firing rate f_i of neuron i was given by

$$f_i = f_{\max} \exp\left(-\frac{(m_i - z)^2}{2\sigma^2}\right), \quad (10)$$

where z is the represented value and the standard deviation of $\sigma = 0.08$ was equal for all input neurons. Each step lasted 50 ms, and consequently, a duration of a trial was 500 ms. From 300 recurrently connected LIF neurons, a half (150 neurons) had an SFA mechanism. Adaptation time constant for each of the neurons with SFA was chosen randomly from a uniform distribution $[1, 500]$. The output was a linear readout – in each step, output weights were multiplied with the average firing activity of the recurrently connected neurons. This output per step was a single real value.

The network was trained by minimizing the Mean-Squared-Error (MSE) between the network outputs and targets for each step. The final test MSE of 0.0341, and mean

absolute error (MAE) of 0.1307 were achieved, calculated for all computational steps in all of, in total, 5000 trials. All test trials had novel sequences of computational steps, that is, test trials were different from those used for training.

Acknowledgements

General: We would like to thank Anand Subramoney for his help and numerous discussions about the experiments done during the master's thesis of Ceca Krašniković. These experiments were partly the motivation for the new experiments presented here.

Funding: This research was partially supported by the Human Brain Project (Grant Agreement number 785907 and 945539), the SYNCH project (Grant Agreement number 824162) of the European Union, and under partial support by the Austrian Science Fund (FWF) within the ERA-NET CHIST-ERA programme (project SMALL, project number I 4670-N).

References

- [1] Maass W. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*. 1997;10(9):1659 – 1671. Available from: <http://www.sciencedirect.com/science/article/pii/S0893608097000117>.
- [2] Marcus GF. *The Algebraic Mind - Integrating connectionism and cognitive science*. MIT Press; 2001.
- [3] Marcus G. Deep learning: A critical appraisal. *CoRR*. 2018;abs/1801.00631. Available from: <https://arxiv.org/pdf/1801.00631.pdf>.
- [4] Marcus G, Marblestone A, Dean T. The atoms of neural computation. *Science*. 2014;346(6209):551–552. Available from: <https://science.sciencemag.org/content/346/6209/551>.
- [5] Rueckauer B, Liu SC. Conversion of analog to spiking neural networks using sparse temporal coding. In: 2018 IEEE International Symposium on Circuits and Systems (ISCAS); IEEE; 2018. p. 1–5.
- [6] Kim R, Li Y, Sejnowski TJ. Simple framework for constructing functional spiking recurrent neural networks. *Proceedings of the national academy of sciences*. 2019;116(45):22811–22820.
- [7] Maass W. Energy-efficient neural network chips approach human recognition capabilities. *Proceedings of the National Academy of Sciences*. 2016; Available from: <http://www.pnas.org/content/early/2016/10/03/1614109113>.
- [8] Davies M, Wild A, Orchard G, et al. Advancing neuromorphic computing with loihi: A survey of results and outlook. *Proceedings of the IEEE*. 2021;109(5):911–934.
- [9] Furber S. Large-scale neuromorphic computing systems. *Journal of neural engineering*. 2016;13(5):051001.
- [10] Bellec G, Salaj D, Subramoney A, et al. Long short-term memory and learning-to-learn in networks of spiking neurons. 32nd Conference on Neural Information Processing Systems (NIPS 2018), Montreal, Canada. 2018;.
- [11] Bellec G, Scherr F, Subramoney A, et al. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature communications*. 2020;11(1):1–15.
- [12] Gerstner W, Kistler WM. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press; 2002.
- [13] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural computation*. 1997;9(8):1735–1780.
- [14] Salaj D, Subramoney A, Krašniković C, et al. Spike frequency adaptation supports network computations on temporally dispersed information. *eLife*. 2021;.
- [15] Pozzorini C, Naud R, Mensi S, et al. Temporal whitening by power-law adaptation in neocortical neurons. *Nature neuroscience*. 2013;16(7):942–948.
- [16] Pozzorini C, Mensi S, Hagens O, et al. Automated high-throughput characterization of single neurons by means of simplified spiking models. *PLoS Comput Biol*. 2015;11(6):e1004275.
- [17] Manneschi L, Vasilaki E. An alternative to backpropagation through time. *Nature Machine Intelligence*. 2020;2(3):155–156.

- [18] Nieder A, Miller EK. Coding of cognitive magnitude: Compressed scaling of numerical information in the primate prefrontal cortex. *Neuron*. 2003;37(1):149–157.
- [19] Nieder A. Supramodal numerosity selectivity of neurons in primate prefrontal and posterior parietal cortices. *Proceedings of the National Academy of Sciences*. 2012;109(29):11860–11865.
- [20] Roitman JD, Brannon EM, Platt ML. Monotonic coding of numerosity in macaque lateral intraparietal area. *PLoS Biol*. 2007;5(8):e208.
- [21] Summerfield C, Luyckx F, Sheahan H. Structure learning and the posterior parietal cortex. *Progress in neurobiology*. 2020;184:101717.
- [22] Behrens TE, Muller TH, Whittington JC, et al. What is a cognitive map? organizing knowledge for flexible behavior. *Neuron*. 2018;100(2):490–509.
- [23] Spitzer B, Waschke L, Summerfield C. Selective overweighting of larger magnitudes during noisy numerical comparison. *Nature Human Behaviour*. 2017;1(8):1–8.
- [24] Teichmann L, Grootswagers T, Carlson T, et al. Decoding digits and dice with magnetoencephalography: evidence for a shared representation of magnitude. *Journal of cognitive neuroscience*. 2018;30(7):999–1010.
- [25] Zamarian L, Ischebeck A, Delazer M. Neuroscience of learning arithmetic—evidence from brain imaging studies. *Neuroscience & Biobehavioral Reviews*. 2009;33(6):909–925.
- [26] Semenza C, Miceli L, Girelli L. A deficit for arithmetical procedures: Lack of knowledge or lack of monitoring? *Cortex*. 1997;33(3):483–498.
- [27] Gazzaniga MS, Ivry RB, Mangun G. *Cognitive neuroscience: The biology of the mind* ; 2009.
- [28] Pulvermüller F. How neurons make meaning: brain mechanisms for embodied and abstract-symbolic semantics. *Trends in Cognitive Sciences*. 2013;17(9):458–470.
- [29] Zylberberg AD, Paz L, Roelfsema PR, et al. A neuronal device for the control of multi-step computations. *Papers in Physics*. 2013;5:050006.
- [30] Kriete T, Noelle DC, Cohen JD, et al. Indirection and symbol-like processing in the prefrontal cortex and basal ganglia. *Proceedings of the National Academy of Sciences*. 2013;110(41):16390–16395.
- [31] O'Reilly RC. Biologically based computational models of high-level cognition. *Science*. 2006;314(5796):91–94.
- [32] Hayworth KJ. Dynamically partitionable autoassociative networks as a solution to the neural binding problem. *Frontiers in computational neuroscience*. 2012;6:73.
- [33] Hayworth KJ, Marblestone AH. How thalamic relays might orchestrate supervised deep training and symbolic computation in the brain. *bioRxiv*. 2018;:304980.
- [34] van der Velde F, de Kamps M. Neural blackboard architectures of combinatorial structures in cognition. *Behavioral and Brain Sciences*. 2006;29(01):37–70.
- [35] Plate TA. Holographic reduced representations. *IEEE Transactions on Neural Networks*. 1995;6(3):623–641.
- [36] Eliasmith C, Stewart TC, Choo X, et al. A large-scale model of the functioning brain. *Science*. 2012;338(6111):1202–1205.
- [37] Müller MG, Papadimitriou CH, Maass W, et al. A model for structured information representation in neural networks of the brain. *Eneuro*. 2020;7(3).
- [38] Rigotti M, Barak O, Warden MR, et al. The importance of mixed selectivity in complex cognitive tasks. *Nature*. 2013;497(7451):585–590.
- [39] Gilmore C, McCarthy SE, Spelke ES. Symbolic arithmetic knowledge without instruction ; 2007. Available from: <https://hdl.handle.net/2134/8760>.
- [40] Gilmore CK, McCarthy SE, Spelke ES. Non-symbolic arithmetic abilities and mathematics achievement in the first year of formal schooling. *Cognition*. 2010 Jun;115(3):394–406. Available from: <https://linkinghub.elsevier.com/retrieve/pii/S0010027710000363>.
- [41] Dehaene S. *The number sense: How the mind creates mathematics*. OUP USA; 2011.
- [42] Liu Y, Dolan RJ, Kurth-Nelson Z, et al. Human replay spontaneously reorganizes experience. *Cell*. 2019;178(3):640–652.
- [43] Sheahan H, Luyckx F, Nelli S, et al. Neural state space alignment for magnitude generalization in humans and recurrent networks. *Neuron*. 2021;109(7):1214–1226.

Chapter 10

Explainable Neuro-Symbolic Hierarchical Reinforcement Learning

Daoming Lyu, Auburn University

Fangkai Yang, NVIDIA

Hugh Kwon, Auburn University

Bo Liu¹, Auburn University

Wen Dong, SUNY Buffalo

Levent Yilmaz, Auburn University

Abstract

Human-robot interactive decision-making is increasingly becoming ubiquitous, and explainability is an influential factor in determining the reliance on autonomy. However, it is not reasonable to trust systems beyond our comprehension, and typical machine learning and data-driven decision-making are black-box paradigms that impede explainability. Therefore, it is critical to establish computational efficient decision-making mechanisms enhanced by explainability-aware strategies. To this end, we propose the Trustworthy Decision-Making (TDM²), which is an explainable neuro-symbolic approach by integrating symbolic planning into hierarchical reinforcement learning. The framework of TDM enables the subtask-level explainability from the causal relational and understandable subtasks. Besides, TDM also demonstrates the advantage of the integration between symbolic planning and reinforcement learning, reaping the benefits of both worlds. Experimental results validate the effectiveness of proposed method while improving the explainability in the process of decision-making.

Key Words: Symbolic Planning, Sequential Decision-making, Interactive Machine Learning, Explainable Machine Learning.

¹Corresponding author: Bo Liu <boliu@auburn.edu>.

²This chapter is primarily adapted from [1].

10.1. Introduction

From self-driving cars to voice assistants on the phone, artificial intelligence (AI) is progressing rapidly. Though AI systems are undeniably powerful and continue to expand their role in our daily lives, emerging issues, such as non-transparency and potential risk in decision-making, give rise to a vital concern: ***why should the end-users trust the decision support system?*** Reliance on technology and autonomy is strongly influenced by trust [2]. However, a recent study reveals that a mere belief in interacting with autonomous teammates led to diminished performance and passive behavior among human participants even though they were collaborating with a remote human partner [3], indicating that we do not trust the capabilities of AI systems as much as we trust humans.

Building explainable AI systems [4, 5, 6, 7], therefore, becomes an important issue for humans to reap the full spectrum of societal and industrial benefits from AI, as well as for minimizing the undesired consequences in the use of the system [8]. Indeed, in modern AI systems, there are many interactive relations, primarily in the form of *interactions* between humans, between humans and the smart agents (human-computer interaction systems), between agents and the environment (interactive machine learning), and among smart agents (multi-agent systems). Besides, AI systems, in general, need to comply with the norms of the social and cultural context if they are expected to operate side by side with humans. Therefore, trust is particularly important for normative agents that are expected to exhibit behavior consistent with the norms of a society or group. From the humans' perspective, we trust things that behave as we expect them to, indicating that trust derives from the process of minimizing the perceived risk [9]. Perceived risk is often defined in terms of uncertainty about the possibility of failure or the likelihood of exhibiting improper behavior.

To open up the AI black-box and facilitate trust, it is critical to enable the AI system to be reliable, transparent, and explainable so that the system can achieve reproducible results, justify the decisions it makes, and understand what is important in the decision-making process. This combination of features can be called "explainability". Deep learning algorithms have been successfully applied to sequential decision-making problems involving high-dimensional sensory inputs such as Atari games [10]. However, deep learning algorithms have limited explainability and transparency, which gives rise to doubt by the human end-users due to a lack of trust and confidence in the AI systems. Prior research efforts [11] on explainability often focus on explaining the results of learning when the underlying problem and architecture are complex. Symbolic planning [12, 13, 14, 15, 16] has been used to break down the complex task into simpler explainable subtasks. However, existing studies [17, 18] focus on the performance gain of symbolic planning when it is applied to general sequential decision-making rather than explainability.

Motivated by understanding the task-level behavior of the algorithm and elevating the trust between human end-users and the smart agents, we propose an explainable neuro-symbolic hierarchical reinforcement learning approach, which we call it Trustworthy Decision-Making (TDM). Symbolic planning is therefore utilized to perform causal reasoning and planning on explicitly represented knowledge, allowing humans to predict the future and to understand the causes of events. This ultimately results in task-level explainability. The key insight is that the success is better measured when the tasks are explainable. Achieving a reliable success rate of the tasks is equivalent to minimizing perceived risks of the agent's behavior, thereby enhancing its trustworthiness.

The rest of the paper is structured as follows. Section 10.2 introduces the background on symbolic planning along with an overview of the interactive sequential decision problem. Section 10.3 provides a detailed description of the TDM framework. We introduce a learning algorithm facilitated by our formal framework in Section 10.4. The optimality analysis for the converged plan generated by the algorithm is presented to substantiate the utility of our strategy. The experimental results of Section 10.5 validate the explainability of the high-level behavior of the system while maintaining improved data efficiency. Section 10.6 concludes by summarizing our results and delineating potential avenues of future research.

10.2. Preliminaries

In this section, we introduce the sequential decision making, hierarchical reinforcement learning, and symbolic planning.

Interactive Sequential Decision-Making. In sequential decision-making problems, an agent takes a sequence of actions to determine its utility. However, the utility is often difficult to model in practical settings when the environment is complex or changes dynamically. In such cases, the utility is determined through feedback from interaction. This can be modeled as a Markov Decision Process (MDP)³, which is defined by a tuple $(\mathcal{S}, \mathcal{A}, P_{ss'}^a, r, \gamma)$. Specifically, $\mathcal{S}$ is the set of states and $\mathcal{A}$ is the set of actions. Then $P_{ss'}^a$ is the transition kernel that, given a state $s \in \mathcal{S}$ and an action $a \in \mathcal{A}$, defines the probability the next state will be $s' \in \mathcal{S}$. Moreover, $r(s, a) : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ is a reward function bounded by $r_{\max}$, and $0 \leq \gamma < 1$ is a discount factor. A solution to an MDP is a policy $\pi : \mathcal{S} \mapsto \mathcal{A}$ that maps a state to an action.

To evaluate a policy π , there are two types of performance measures: the expected discounted sum of reward for infinite-horizon problems and the expected un-discounted sum of rewards for finite horizon problems. In this paper we adopt the latter metric defined

as $J_{\text{avg}}^\pi(s) = \mathbb{E}[\sum_{t=0}^T r_t | s_0 = s]$. We define the *gain reward* $\rho^\pi(s)$ reaped by policy π from

s as $\rho^\pi(s) = \lim_{T \rightarrow \infty} \frac{J_{\text{avg}}^\pi(s)}{T} = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}[\sum_{t=0}^T r_t]$. Gain reward $\rho^\pi(s)$ is often used to measure the lower bound of the expected cumulative rewards of a task w.r.t a given policy. For example, $\rho^\pi(s)$ is instrumental in deciding whether a specific task is rewarding enough to make the “stay-or-leave” decision [20]. More mathematical details of gain reward can be referred to [21, 22, 23, 24].

Hierarchical Reinforcement Learning and Options. Hierarchical reinforcement learning (HRL) [25] overcomes the “curse of dimensionality” via hierarchical decomposition within the policy space of the problem. Options [26] are a well-known HRL framework which is the temporally extended course of actions. An *option* consists of three components: a policy $\pi : \mathcal{S} \times \mathcal{A} \mapsto [0, 1]$, a termination condition $\beta : \mathcal{S} \mapsto [0, 1]$, and an initiation set $\mathcal{I} \subseteq \mathcal{S}$. An option (I, π, β) is available in state s_t iff $s_t \in I$. After the option is taken, a course of actions is selected according to π until the option is terminated stochastically according to the termination condition β . With the introduction of options, the decision-making has a hierarchical structure with two levels, where the upper level is

³We follow the style of notation in the 1st edition of reinforcement learning book by Sutton and Barto [19].

the option level (also termed as task level) and the lower level is the (primitive) action level. Markovian property exists among different options at the option level.

Action Language $\mathcal{BC}$. An *action description* D in the language $\mathcal{BC}$ [27] includes two kinds of symbols on signature σ , *fluent constants* that represent the properties of the world, and *action constants*, representing actions that influences the world. A *fluent atom* associates a fluent constant f to a value v and is expressed as $f = v$. In particular, we consider a boolean domain for f . *Causal laws* can now be defined on the fluent atoms and action constants, describing the relationship among fluent atoms and the effects of actions on the value of fluent atoms. A *static law* may state that a fluent atom A is true at a given time step when $A_1, \dots, A_m$ are true (A **if** $A_1, \dots, A_m$) or the value of f equals v by default (**default** $f = v$). On the other hand, a *dynamic law* describes an action a (**nonexecutable** a **if** $A_1, \dots, A_m$) or the effect of a on the fluent atom A (a **causes** A **if** $A_1, \dots, A_m$). An inertia is a special case of dynamic law that states the value of fluent constant f does not change with time (**inertial** f). An action description is a finite set of causal laws, capturing the domain dynamics as transitions.

Symbolic Planning. Given the action description D , symbolic planning (SP) is performed with action language $\mathcal{BC}$. With a slight abuse of notations, let $\langle s, a, s' \rangle$ denote a transition from a symbolic state s to a symbolic state s' by a set of action a . Then a planning problem can be formulated as a tuple (I, G, D) . The planning problem has a plan of length $l - 1$ if and only if there exists a transition path of length l such that $I = s_1$ and $G = s_l$. In the rest of the paper, Π denotes both the plan and the transition path by following that plan. Given the above semantics represented in $\mathcal{BC}$, automated planning can be achieved by an answer set solver such as CLINGO [28], and provide the solution to the planning problem.

SP has been used in tasks that must be highly explainable to human users, such as interacting and cooperating with mobile robots [12, 13, 14, 15, 16]. By abstracting the planning problem into a symbolic representation, including the knowledge of objects, properties, and the effects of executing actions in a dynamic system, SP can solve the problem based on logical reasoning. Such symbolic representation is typically implemented via a formal, logic-based language as the Planning Domain Definition Language (PDDL) [29] or an action language [30] that relates to logic programming under answer set semantics (answer set programming) [31].

SP algorithms are white-box algorithms. With the symbolic knowledge designed to be human-readable, the behavior of an SP agent that plans and reasons based on causal relations entailed by actions and events, which is naturally predictable and explainable. Prior works combine symbolic planning with other sequential-decision making algorithms to bring stability to plan-based learning [17, 18]. However, these works only highlight the performance improvement from the prior domain knowledge represented by SP.

10.3. The TDM Framework

This section will introduce the main framework of Trustworthy Decision-Making (TDM), which is a Neuro-Symbolic HRL approach. The underlying sequential decision-making problem is first defined as an MDP tuple $(\widetilde{\mathcal{S}}, \widetilde{\mathcal{A}}, \widetilde{P}_{ss'}^a, r, \widetilde{\gamma})$, where $\widetilde{\mathcal{S}}$ refers to a set of states in the problem (e.g., pixel images in Atari games), $\widetilde{\mathcal{A}}$ is the set of primitive actions, $\widetilde{P}_{ss'}^a$ is the transition kernel, r is the reward function, and $\widetilde{\gamma}$ is a discounting factor. To make the notations clear, in the following paper, $\widetilde{\mathcal{S}}, \widetilde{\mathcal{A}}$ are used to denote the MDP

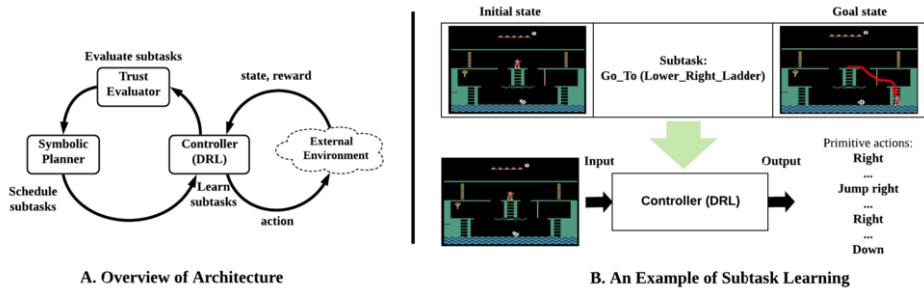


Figure 1. The left subfigure (A) shows the architecture overview of TDM, while the right subfigure (B) gives an example of subtask learning on Montezuma’s Revenge. The subtask of “Go_To (Lower_Right_Ladder)” means the avatar in the middle platform needs to arrive at the position of lower right ladder. In this case, the controller accepts the pixel images as an input, and learns a sub-policy for this subtask.

state space and action space, while $\mathcal{S}, \mathcal{A}$ represent the symbolic state space and action space. The goal of TDM is to seek a reasonable sequence of subtasks and learn the corresponding sub-policies such that performing the learned subtasks in order can lead to the maximal cumulative reward. It should be noted that our framework is not restricted to high-dimensional sensory inputs. However, we choose to use them as an example since learning from such inputs is more challenging and relevant to real-world applications.

A symbolic representation of the problem provided by human experts is assumed to be available, including a set of causal rules that captures objects, fluents and how their values are changed by executing subtasks. Although this may need some effort, it is possible to develop general-purpose action modules [32, 33, 34] as it has been shown that dynamic domains share many actions in common. Therefore, the symbolic formulation for one problem can be adapted to another with a little changes by instantiating a different set of objects or adding a few more rules. To make TDM general, the domain formulation in our framework is always in a coarse granular and high-level abstraction. This also enables the decision-making process to be robust and flexible when facing uncertainty and domain changes.

The overview of TDM framework is shown in Fig. 1(A). With a symbolic representation, the symbolic planner performs planning by a sequence of symbolic actions based on the evaluation feedback from the trust evaluator. These symbolic actions are actually the subtasks that can be sequenced as a high-level plan. In addition, a mapping function is implemented to perform symbol grounding in our experiments, associating each MDP state (e.g., pixel images in Atari games) with a symbolic state. In different domains, such function is assumed to be available. As a result, the subtasks defined on the MDP space can be induced based on symbolic states. After symbol grounding, the subtasks in the plan will be sent to the controller to be executed so that each sub-policy for the corresponding subtask is learned by the controller. Since it is possible that the controller cannot successfully learn the sub-policies, a metric is introduced to evaluate the competence of learned sub-policies, such as the success ratio over several episodes. After all subtasks in the plan have been executed, the trust evaluator computes the plan quality score by utilizing the “degree of trustworthiness” for subtasks. The evaluation for subtasks is returned to the symbolic planner and is used to generate new plans by either exploring new subtasks or sequencing learned subtasks that supposedly can achieve a higher plan quality score in the next iteration.

10.3.1. A Computational Framework for Plan Quality Evaluation

We formulate the plan evaluation with a success ratio of task execution as the plan quality score. This is used to quantitatively measure both the performance level of a plan, and the reliability of learning for each subtask in this plan. Therefore, the plan with a low plan quality score will have less ability to be considered for the final solution. Besides, the insight here is not only just explain how and why the agent's behavior can produce outcomes but also maintain high performance level on achieving the task. So the plan evaluation can be tied to the trustworthiness, including both the explainability for the plan and the reliability for each subtask learning. In essence, we try to solve an optimization problem to maximize the explicit plan quality score subject to the constraint of implicit explainability:

$$\max_{\Pi} \frac{1}{|\Pi|} \sum_{o \in \Pi} \varepsilon(o),$$

where ε is the success ratio of a subtask o , Π is a symbolic plan.

The success ratio with respect to explainability is the key to our plan evaluation mechanism. But explainability is a qualitative measure, and it is difficult to formulate a traditional optimization approach to satisfy this constraint. Instead, we use symbolic representation to naturally induce explainability in our problem. We discuss the details of our approach in the subsequent sections.

10.3.2. Explainability Enhancement via Symbolic Representation

Let us consider a planning problem (I, G, D) . Although a symbolic formulation is possible with various planning or action languages, we will use $\mathcal{BC}$ to represent D as a demonstration. Specifically, we add the following causal laws to D to formulate gain rewards of executing actions and their effects on cumulative plan quality:

- For any symbolic state that contains atoms $\{A_1, \dots, A_n\}$, D contains static laws of the form:

$$s \text{ if } A_1, \dots, A_n, \text{ for state } s \in \mathcal{S}.$$

- We introduce new fluent symbols of the form $\rho(s, a)$ to denote the gain reward at state s following action a . D contains a static law stating that the gain reward is initialized optimistically by default to promote exploration and is denoted as *INF*:

$$\text{default } \rho(s, a) = \text{INF}, \text{ for } s \in \mathcal{S}, a \in \sigma_A(D).$$

- We use fluent symbol *quality* to denote the total “degree of trustworthiness” of a plan, termed as *plan quality* or *plan quality score*. D contains dynamic laws of the form

$$a \text{ causes } \text{quality} = C + Z \text{ if } s, \rho(s, a) = Z, \text{quality} = C.$$

- D contains a set P of facts of the form $\rho(s, a) = z$.

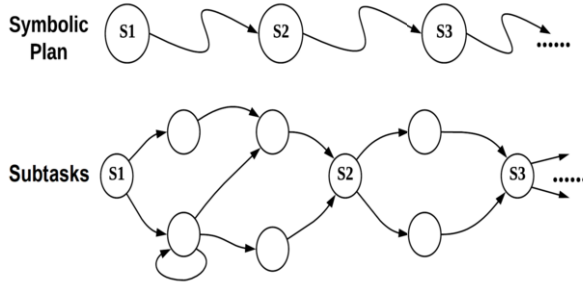


Figure 2. The mapping from a symbolic transition path to subtasks

In our case, I is still the initial symbolic planning state, but the goal state G is also a linear constraint of the form

$$quality > quality(\Pi), \quad (1)$$

for a symbolic plan Π measured by the internal utility function $quality$ defined as

$$quality(\Pi) = \sum_{\langle s_i, a_i, s_{i+1} \rangle \in \Pi} \rho(s_i, a_i). \quad (2)$$

It should be noted that (1) in TDM doesn't have the logical constraint part and enables "model-based exploration by planning", which is more suitable for the problems where the plan quality score drives the agent's behavior.

From the causal point of view, symbolic planning centers on reasoning about changes and causal relations entailed by actions and events. This enables to predict the outcomes of the agent's behavior with causality, and further help humans to understand and justify the outcomes.

10.3.3. Implicit Explainability Constraint Satisfaction: From Symbolic Transitions to Subtasks

Our discussion of plan evaluation and explainability enhancement has been restricted to general machine learning. Symbolic planning can be used in many problem domains, e.g., image classification, and success ratio may be replaced with a suitable metric, such as accuracy, in the respective domain. Here, we narrow our focus to a sequential decision-making problem, which will allow us to devise a specific algorithm based on the TDM framework.

We assume an oracle exists to determine whether the symbolic properties specified as fluent atoms of the form $f = v$ in s are true in $\tilde{s}$. This oracle is actually a mapping function as $\mathbb{F} : \mathcal{S} \times \tilde{\mathcal{S}} \mapsto \{\text{True}, \text{False}\}$. If a symbolic state $s \in \mathcal{S}$ corresponds to an MDP state $\tilde{s} \in \tilde{\mathcal{S}}$, then $\mathbb{F} = \text{True}$. $\mathbb{F} = \text{False}$ if otherwise. For example, this oracle can be a pre-trained perception module for object recognition in images when it is used for Atari games.

Given $\mathbb{F}$ and a pair of symbolic states $s, s' \in \mathcal{S}$, we can induce a subtask as a triple (I, π, β) where the initiation set $I = \{\tilde{s} \in \tilde{\mathcal{S}} : \mathbb{F}(s, \tilde{s}) = \text{True}\}$, $\pi : \tilde{\mathcal{S}} \mapsto \tilde{\mathcal{A}}$ is the intra-subtask policy, and β is the termination condition such that

$$\beta(\tilde{s}') = \begin{cases} 1, & \mathbb{F}(s', \tilde{s}') = \text{True}, \text{ for } \tilde{s}' \in \widetilde{\mathcal{S}} \\ 0, & \text{otherwise} \end{cases}$$

This definition maps symbolic transition to a subtask (as illustrated in Fig. 2), which is similar to an option and incorporates the hierarchical structure into the decision-making. At the high-level, the symbolic planner generates the symbolic plan that includes a sequence of subtasks, while each subtask will be accomplished with the primitive actions at the low-level. It also implies that the execution of the subtask will be feasible if the termination condition is 1, while it will be infeasible if the termination condition is 0. Since the symbolic knowledge is provided by the human experts, subtasks are pre-defined by the human experts as well.

10.3.4. Explicit Success Ratio Maximization

To motivate the subtask learning from the low-level control, $t_e(\tilde{s}')$ can be assigned in a binary form. It is defined as:

$$t_e(\tilde{s}') = \begin{cases} +1, & \beta(\tilde{s}') = 1 \\ -1, & \text{otherwise} \end{cases} \quad (3)$$

which means that the $t_e(\tilde{s}')$ will be +1 if the subtask terminated at $\tilde{s}'$ can be achieved, while the $t_e(\tilde{s}')$ will be -1 if subtask fails to be accomplished. Other more sophisticated distribution (e.g., Bernoulli distribution) can also be modeled other than this 0-1 distribution model in practice.

To evaluate the high-level planning, we further define $r_e(s, o)$ as $r_e(s, o) = f(\epsilon)$, where f is a function of ϵ , and ϵ is the success ratio that denotes the average rate of completing the subtask successfully over the previous 100 episodes. This metric is used to measure whether the subtask o at symbolic state s is learnable or not. This is different from the definition of $t_e(\tilde{s}')$ since $r_e(s, o)$ requires reliably achieving the subtask after the training by episodes and keeping the success ratio above a certain threshold. Therefore, f is defined as

$$f(\epsilon) = \begin{cases} -\psi, & \epsilon < E \\ r(s, o), & \epsilon \geq E \end{cases} \quad (4)$$

ψ is a large positive numerical value, $r(s, o)$ is the reward obtained from the MDP environment by following the subtask o hyper-parameter E is a predefined threshold of success ratio. In our experiments, we set $E = 0.9$ according to empirical performance observations. Intuitively, it means if the sub-policy can reliably achieve the subtask after the training by episodes. The “degree of trustworthiness” of $r_e(s, o)$ at s' reflects true cumulative reward from the MDP environment by following the subtask; otherwise, the “degree of trustworthiness” will be very low (as a large negative number), indicating that the sub-policy performs badly and is probably not learnable. A plan Π of (I, G, D) is considered to be *optimal* iff $\sum_{\langle s, a, s' \rangle} r_e(s, o)$ is maximal among all plans.

10.4. Algorithm Design

We present a learning algorithm for the TDM framework in Algorithm 1 and show TDM's planning and learning process. Each episode starts with the symbolic planner generating a symbolic plan Π_t for the problem (I, G, D) (Line 4). The symbolic transitions of Π_t are mapped to subtasks (Line 10) to be learned by a controller (Line 11). The controller performs deep Q-learning with t_e using experience replay (Lines 12–15) to estimate the Q value $Q(\tilde{s}, \tilde{a}; o) \approx Q(\tilde{s}, \tilde{a}; \theta, o)$, where θ is the parameter of the non-linear function approximator. The observed transition $(\tilde{s}_t, \tilde{a}_t, r_e(\tilde{s}_{t+1}, g), \tilde{s}_{t+1})$ is stored as an experience in $\mathcal{D}_o$. The loss at t^{th} iteration is calculated as an expectation over the collected experience and is given by

$$L(\theta; o) = \mathbb{E}_{(\tilde{s}, \tilde{a}, g, t_e, \tilde{s}') \sim \mathcal{D}_o} [r_e + \gamma \max_{\tilde{a}'} Q(\tilde{s}, \tilde{a}'; \theta_{i-1}, o) - Q(\tilde{s}, \tilde{a}; \theta_i, o)]^2. \quad (5)$$

When the inner loop terminates, a symbolic transition $\langle s_t, a_t, s_{t+1} \rangle$ for the subtask o_t is completed, and we are ready to update the plan quality score of the subtask (Line 18). Although various learning methods, such as Q-learning, may be used here, we use R-learning [35] to evaluate the plan quality score as an average reward case rather than a discounted reward case. With R-learning, the update rule becomes

$$\begin{aligned} R_{t+1}(s_t, o_t) &\stackrel{\alpha}{\leftarrow} r_e - \rho_t^{o_t}(s_t) + \max_o R(s_t, o) \\ \rho_{t+1}^{o_t}(s_t) &\stackrel{\beta}{\leftarrow} r_e + \max_o R_t(s_{t+1}, o) - \max_o R_t(s_t, o) \end{aligned} \quad (6)$$

The *quality* of Π_t is measured by (2) (Line 20) and the plan quality score of the plan is updated according to $\text{quality}(\Pi_t)$ (Line 21). The symbolic formulation is then updated with the learned ρ values (Line 22). With the symbolic formulation changed, an improved plan may be generated in the next episode. The loop terminates when no such improvement can be made. The algorithm guarantees symbolic level optimality conditioned on R-learning convergence.

Theorem 1 (Termination). *If the plan evaluator's R-learning converges, Algorithm 1 terminates iff an optimal symbolic plan exists.*

Proof. When R-learning converges, for any transition $\langle s, a, t \rangle$, the increment terms in (6) diminish to 0, which implies

$$\begin{aligned} R(s, o) &= \max_{o'} R(s, o'), \\ \rho^a(s) &= r_e(s, o) - \max_{o'} R(s, o') + \max_{a'} R(t, o') \end{aligned} \quad (7)$$

Algorithm 1 terminates iff there exists an upper bound of plan quality iff there does not exist a plan with a loop L such that $\sum_{\langle s, a, t \rangle \in L} \rho^a(s) > 0$. By (7), it is equivalent to $\sum_{\langle s, o, t \rangle \in L} (r_e(s, o) - R(s, o) + R(t, o)) \leq 0$ iff $\sum_{\langle s, o, t \rangle \in L} r_e(s, o) - R(s_{|L|}, o) + R(s_0, o) \leq 0$. Since L is a loop, $s_{|L|} = s_0$, so $\sum_{\langle s, a, t \rangle \in L} r_e(s, o) \leq 0$ iff any plan Π does not have a positive loop of cumulative reward iff optimal plan exists, which completes the proof.

Theorem 2 (Optimality). *If the plan evaluator's R-learning converges, when Algorithm 1 terminates, Π^* is an optimal symbolic plan.*

Algorithm 1 TDM Planning and Learning Loop**Input:** $(I, G, D, \mathbb{F})$ where $G = (quality > 0)$, and an exploration probability ϵ **Output:** An optimal symbolic plan Π^* $P_0 \Leftarrow \emptyset, \Pi \Leftarrow \emptyset$ **while** True **do** $\Pi^* \Leftarrow \Pi$ Take ϵ probability to solve planning problem and obtain a plan $\Pi \Leftarrow \text{CLINGO.solve}(I, G, D \cup P_t)$ **if** $\Pi = \emptyset$ **then****return** Π^* **end if****for** symbolic transition $\langle s, a, s' \rangle \in \Pi$ **do**Obtain current state $\tilde{s}$ Correspond to subtask o by using $\mathbb{F}$ to obtain initiation set and terminate condition**while** $\beta(\tilde{s}) \neq 1$ and maximal step is not reached **do**Pick up an action $\tilde{a}$ and obtain transition $(\tilde{s}, \tilde{a}, \tilde{s}', t_e(\tilde{s}'))$ Store transition in experience replay buffer $\mathcal{D}_o$ Estimate $Q(\tilde{s}, \tilde{a}; \theta, o)$ by minimizing loss function (5) when there are sufficient samples in $\mathcal{D}_o$ Update current state $\tilde{s} \Leftarrow \tilde{s}'$ **end while**Calculate $r_e(s, o)$ with “degree of trustworthiness” s Update $R(s, o)$ and $\rho^o(s)$ using (6).**end for**Calculate quality of Π by (2).Update planning goal $G \Leftarrow (quality > quality_t(\Pi))$.Update facts $P_t \Leftarrow \{\rho(s, a) = z : \langle s, a, s' \rangle \in \Pi, \rho_t^a(s) = z\}$ **end while****Proof.** By Lee et al. [27, Theorem 2], Π^* is a plan for planning problem (I, G, D) . For Π_o returned by Algorithm 1 when it terminates, $quality(\Pi) \leq quality(\Pi^*)$ for any Π iff

$$\sum_{\langle s, a, t \rangle \in \Pi} \rho^a(s) \leq \sum_{\langle s, a, t \rangle \in \Pi_o} \rho^a(s).$$

By (7), the inequality is equivalent to

$$\sum_{\langle s, a, t \rangle \in \Pi} r_e(s, o) + R(s_{|\Pi|}, o) \leq \sum_{\langle s, a, t \rangle \in \Pi_o} r_e(s, o) + R(s_{|\Pi^*|}, o).$$

Since $s_{|\Pi|}$ and $s_{|\Pi^*|}$ are terminal states of each symbolic plan with no subtask policies available, we have $\sum_{\langle s, a, t \rangle \in \Pi} r_e(s, o) \leq \sum_{\langle s, a, t \rangle \in \Pi_o} r_e(s, o)$. This completes the proof.

10.5. Experiment

We use the Taxi domain [25] to demonstrate the behavior of learning and planning, Grid World [17] to show the ability to learn unmodeled domain knowledge, and on

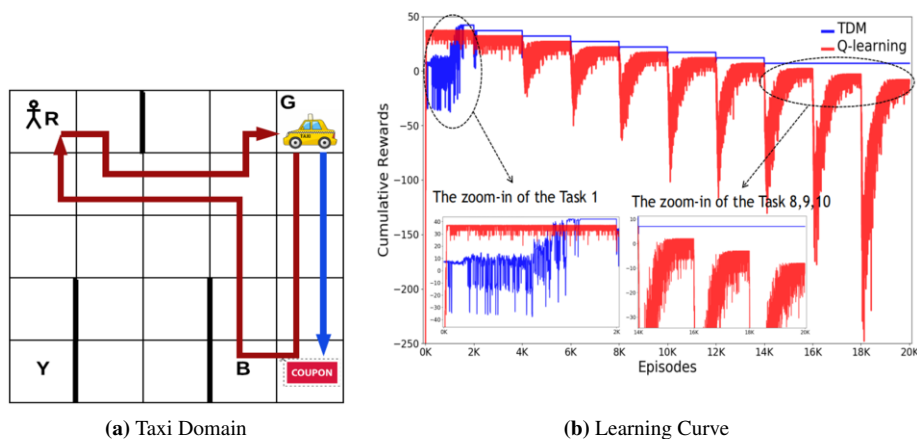


Figure 3. Experimental Results on Taxi Domain

Montezuma's Revenge [10] for explainability and data efficiency. We use 1M to denote 1 million and 1K to denote 1000.

10.5.1. Taxi Domain

The objective of the Taxi domain is to maximize reward by successfully picking-up and dropping-off a passenger at a specified destination on a grid map (Fig. 3a). A taxi agent moves from a cell to an adjacent cell with -1 reward collected at each time step. The agent receives a reward of 50 for dropping the passenger off at the destination while receiving a reward of -10 when attempting to pick-up or drop-off at an incorrect location. Additionally, there is a one-time reward coupon the taxi can collect at (4,4). With a reward of 10, we will observe the behavior of learning agents when the environment changes.

Setup. For each episode, the taxi starts at (0,4), and we will call every 2000 episodes a task. The experiment consists of 10 tasks where the reward for the successful drop-off decreases in each task by 5, i.e., 50 in Task 1, 45 in Task 2, and so forth. Other reward settings stay constant throughout the tasks. We compare TDM with Q-learning.

Experimental Results. First, let us consider the optimal policies as shown in Fig. 3a. The dark red colored route that collects the coupon, picking up, and then dropping off the passenger is the optimal policy from Task 1 to Task 7. From Task 8 and onward, the reward for the passenger drop-off (≤ 15) is no longer worth the effort, and hence, the optimal policy changes to simply collecting the coupon as indicated by the blue line. As shown in the learning curve in Fig. 3b, averaged over 10 runs, TDM successfully learns the optimal policy for all Tasks. As noted, the optimal policy changes at Task 8, and TDM quickly abandons the sub-tasks that lead to a sub-optimal policy and learns the new optimal policy. All these are in contrast to Q-learning. As shown in the zoom-in of Task 1 in Fig. 3b, Q-learning converges faster than TDM since the model-free exploration of Q-learning is more aggressive than the TDM's exploration guided by the planning. However, the policy learned by Q-learning is worse than that of TDM over all 10 Tasks, which results in a gap of cumulative rewards between them. This gap becomes larger when the optimal policy has changed, especially from Task 8 to Task 10. On the contrary,

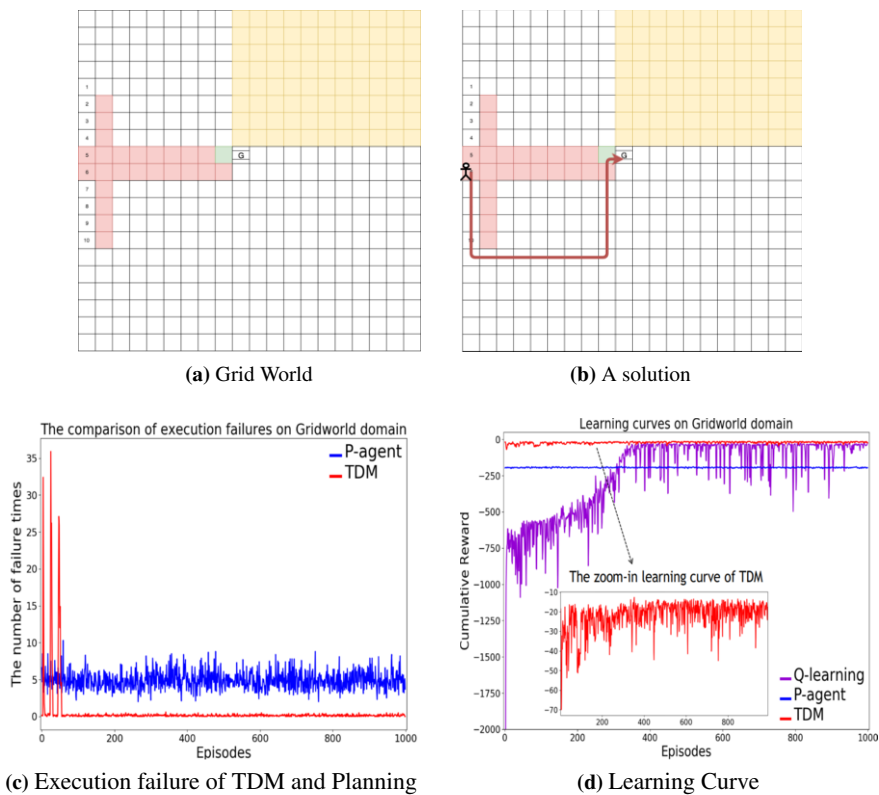


Figure 4. Grid World

the plan evaluation mechanism in TDM enables to adapt to the domain changes very well and select the plan with higher plan quality score. In practice, the reward is often collected from the interaction and may change over time in a dynamic environment. This experiment shows the advantage of TDM in such situations.

10.5.2. Grid World

TDM can learn domain details that are not modeled into symbolic knowledge. We demonstrate this behavior with the Grid World adapted from Leonetti et al. [17], shown in Fig. 4a. An agent must navigate to (9,10), which can only be entered through a door at (9, 9). The agent must grab a doorknob at the door, turn it, and then push the door to reach the goal, all of which may fail and incur a -10 penalty. As in the Taxi domain, every movement has a reward of -1.

Setup. Horizontal and vertical bumpers represent the domain details that the agent needs to learn. The agent receives an additional penalty for a movement into a bumper. Penalties are -30 and -15 for red and yellow bumpers, respectively. The agent starts at random on one of the numbered grids in the first column. We use Q-learning and a standard planning agent (P-agent) as baselines for this domain. P-agent generates plans using CLINGO and executes the plans without any learning capability.

No.	Layer	Details
1	Convolutional	32 filters, kernel size=8, stride=4, activation='relu'
2	Convolutional	64 filters, kernel size=4, stride=2, activation='relu'
3	Convolutional	64 filters, kernel size=3, stride=1, activation='relu'
4	Fully Connected	512 nodes, activation='relu'
5	Output	activation='linear'

Table 1. Neural Network Architecture for Montezuma’s Revenge

Experimental Results. Learning curves of cumulative reward are shown in Fig. 4d where the performance of TDM surpasses both that of Q-learning and P-agent. Specifically, TDM can learn domain details that are not included in the symbolic knowledge, such as bumpers in this case. With plan quality scores, TDM can abandon the unsuccessful plans, avoid the bumpers and learn to reliably open and push through the door (e.g., Fig. 4b). This also achieves the optimal behavior. As a comparison, P-agent prefers shortest plans and goes over the bumpers directly, which are costly in this case. Besides, P-agent relies on re-planning upon encountering failures without learning. As such, the number of execution failures to enter the door remains high throughout episodes, as shown in Fig. 4c.

10.5.3. Montezuma’s Revenge

In “Montezuma’s Revenge,” the player controls a game character through a labyrinth avoiding dangerous enemies and collecting items helpful to the player. We focus on the first room of the labyrinth. Here, the player has to collect a key to unlock the door to the next room. In a typical setting for DRL, an agent receives +100 reward for collecting the key and +300 for opening the door with the key. This is a challenging domain for DRL as it involves a long sequence of high-level tasks and many primitive actions to achieve those tasks. This is a challenging domain where the vanilla DQN often fails to learn [10].

Setup. Our experiment setup follows the DQN controller architecture [36] with double-Q learning [37] and prioritized experience replay [38]. The architecture of the deep neural networks is shown in Table 1. The experiment is conducted using Arcade Learning Environment (ALE) [39]. We have implemented the symbolic mapping function $\mathbb{F}$ based on ALE API. The binary “degree of trustworthiness” follows (3) for when a subtask successfully completes or the agent loses its life. The subtask plan quality score follows (4) where $\psi = 100$ and define $r(s,o) = -10$ for $\varepsilon > 0.9$ to encourage shorter plan. We use hierarchical DQN (hDQN) [36] as the baseline.

Symbolic Representation. We represent the transition dynamics and domain knowledge in action language $\mathcal{B}\mathcal{L}$. There are 6 pre-defined locations or objects: middle platform (mp), right door (rd), left of rotating skull (ls), lower left ladder (lll), lower right ladder (lrl), and key (key). Note that the number of predefined locations or objects depends on the users and their domain knowledge. We then formulate 13 subtasks based on the symbolic locations. The corresponding symbolic transitions from the mapping function $\mathbb{F}$ are shown in Table 2. The difference between our and hDQN’s subtask definitions should be noted. A subtask is only associated with an object in hDQN; However, in our work, it is defined as a symbolic transition with an initiation set and termination condition based on the states satisfying symbolic properties. With this informative symbolic representation, our approach is more general, descriptive, and explainable. It also makes sub-policy for each subtask to be more easily learned and subtasks more easily sequenced.

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

No.	Subtask	Policy learned	In the optimal plan
1	MP to LRL, no key	✓	✓
2	LRL to LLL, no key	✓	✓
3	LLL to key, no key	✓	✓
4	key to LLL, with key	✓	✓
5	LLL to LRL, with or without key	✓	✓
6	LRL to MP, with or without key	✓	✓
7	MP to RD, with key	✓	✓
8	LRL to LS, with or without key	✓	
9	LS to key, with or without key	✓	
10	MP to RD, no key	✓	
11	LRL to key, with or without key		
12	key to LRL, with key		
13	LRL to RD, with key		

Table 2. Subtasks for Montezuma’s Revenge

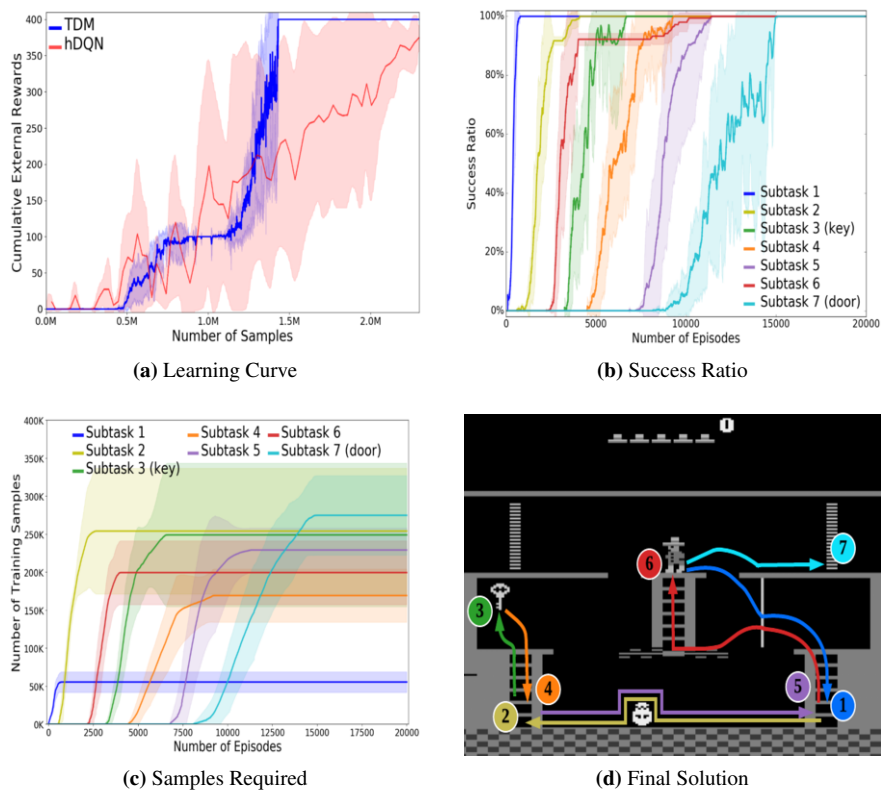


Figure 5. Experimental Results on Montezuma’s Revenge

Experimental Results. It is easy to see that TDM is more data-efficient than the baseline hDQN from the learning curve (Fig. 5a). Explainability requires a qualitative analysis. We

will emphasize how plan quality score guides TDM on planning, learning, and sequencing of the subtasks, and therefore learns the optimal behavior, all of which are visualized in our figures.

The results have been averaged over 10 runs and we shown mean-variance plots in the Fig. 5a, 5b, 5c. The description of subtasks can refer to both Fig. 5d and Table 2. The environment rewards are only given for completing Subtask 3 (picking up the key, +100) and Subtask 7 (opening the right door, +300). Since other subtasks do not receive any reward from the environment, we can infer from Fig. 5a that TDM first learns the plan to sequence Subtasks 1–3. The agent learns other subtasks through exploration as the plan quality score-based planning encourages executing untried subtasks and try different locations until it reaches the door. At that point, we can see in the learning curve that the agent quickly converges to the optimal plan that sequences through Subtasks 1–7 (Fig. 5d), resulting in the maximal reward of 400. In contrast, hDQN does not reliably achieve the maximal reward even after TDM stably converges. Also, it's difficult to see how hDQN sequenced through its subtasks due to high variance. TDM achieves a smaller variance than hDQN partially because our definition of the subtask is easier to learn than the one defined in hDQN, leading to more robust and stable learning.

The symbolic planning of TDM allows flexibility in reusing the learned subtasks in various plans. Fig. 5b shows that TDM learns Subtask 6 before Subtask 3, but in the optimal plan, Subtask 3 must be sequenced before Subtask 6. This means Subtask 6 has been learned as a part of a different plan but re-selected in the optimal plan, which is possible because subgoals of the subtasks are associated by their starting states and only activated by the planner when the agent satisfies the starting states.

During the experiment, Subtasks 1–10 are successfully learned by controllers (or DQNs), with 7 of them being selected in the final solution with achieving a success ratio of 100%, shown in Fig. 5b. TDM prunes other Subtasks 8–13 based on the low plan quality score achieved during training. Subtask 8, from the lower right ladder to the left of the rotating skull, reaches a success ratio of 0.9 but later quickly drops back to 0, due to the instability of DQN. Subtasks 9 and 10 reach the required success ratio but are discarded as they do not contribute to the optimal plan, whereas the success ratios on subtasks 11–13 are poor, suggesting they are difficult to learn.

Explainability and Trust The evaluation criteria are designed through human involvement in our proposed framework, based on the human expert's interpretation of the agent's execution. The symbolic representation can then be refined to improve the agent's performance. Therefore, the agent's decision-making trustworthiness is rooted in the trust we place in the underlying symbolic representation. Although many existing explainable frameworks focus on simplifying the underlying model to generate human-understandable explanations [6, 40, 41], we show that the trade-off between the explainability and performance of the model is not always necessary. In our case, a performance-based plan performance metric is used to guide the learning agent to maximize its performance and justify the subtasks it learns. This is, in essence, a separation of descriptive and persuasive explanation tasks [42], where we handle the descriptive task through symbolic representation and perform persuasive explanation through the use of the plan quality score..

We now relate these concepts to the comparison of our framework to the baseline. In both hDQN and TDM, the bounding box defined by a human expert is utilized, considering the locations of objects only. As a result, the bounding box can be meaningful and

Explainability	TDM		hDQN
	Explainable	Yes	Yes
	Descriptive	Strong	Weak
	Persuasive	Strong	Weak

Table 3. Comparison of TDM and hDQN

explainable with the human expert’s domain knowledge. The subtasks of hDQN are only associated with the bounding box. However, TDM utilizes symbolic representation to derive subtasks except for the bounding box and can provide a relational perspective, such as objects/entities and their relations. This makes the subtasks of TDM more descriptive since a symbolic state may contain more rich information than hDQN. According to the plan evaluation on subtasks, plan quality scores would motivate the TDM agent to maximize its performance by executing the learnable subtasks while discarding the unlearnable ones. The experiment results show TDM’s persuasive explanation is more convincing than hDQN. Also, the variance about the curves (Fig. 5a) demonstrates the degree of robustness when the agent faces uncertainties. The summarization is shown in Table 3.

10.6. Conclusions

Since explainability of the high-level behavior is critical to enhancing applicability in a complex sequential decision-making problem, we proposed the TDM framework in this paper by integrating symbolic planning with hierarchical reinforcement learning. Specifically, deep RL is used to learn low-level control policies for subtasks managed by high-level symbolic planning based on explicit symbolic knowledge. Both theoretical analysis and empirical studies on benchmark problems validate that our plan quality score-based algorithmic framework brings *task-level explainability* to deep reinforcement learning and *improved data efficiency* induced by the symbolic-planning-learning framework of the agent.

There are several promising future work potentials in this research direction. For example, it would be intriguing to apply TDM in the human-computer interaction systems, allowing layperson to understand the system behavior and provide meaningful evaluation feedback to the machine. The second interesting direction is to apply the framework of TDM in multi-agent systems. Trust is important in interactions among different agents where the integrity of some agents is not always guaranteed. Explainability may be used to help improve the behavior of the agents and lead to a better equilibrium.

References

[1] Lyu D, Yang F, Kwon H, et al. TDM: Trustworthy decision-making via interpretability enhancement. IEEE Transactions on Emerging Topics in Computational Intelligence. 2021;.

[2] Lee JD, See KA. Trust in automation: Designing for appropriate reliance. Human factors. 2004;46(1):50–80.

[3] Demir M, McNeese NJ, Cooke NJ. The impact of perceived autonomous agents on dynamic team behaviors. IEEE Transactions on Emerging Topics in Computational Intelligence. 2018;2(4):258–267.

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

- [4] Marsh S. Trust in distributed artificial intelligence. In: European Workshop on Modelling Autonomous Agents in a Multi-Agent World; Springer; 1992. p. 94–112.
- [5] O'Donovan J, Smyth B. Trust in recommender systems. In: Proceedings of the 10th international conference on Intelligent user interfaces; ACM; 2005. p. 167–174.
- [6] Ribeiro MT, Singh S, Guestrin C. Why should I trust you?: Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining; ACM; 2016. p. 1135–1144.
- [7] Ong YS, Gupta A. Air 5: Five pillars of artificial intelligence research. *IEEE Transactions on Emerging Topics in Computational Intelligence*. 2019;3(5):411–415.
- [8] Hoff KA, Bashir M. Trust in automation: Integrating empirical evidence on factors that influence trust. *Human Factors*. 2015;57(3):407–434.
- [9] Rousseau DM, Sitkin SB, Burt RS, et al. Not so different after all: A cross-discipline view of trust. *Academy of management review*. 1998;23(3):393–404.
- [10] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nature*. 2015;518(7540):529–533.
- [11] Biran O, McKeown KR. Human-centric justification of machine learning predictions. In: *IJCAI*; 2017. p. 1461–1467.
- [12] Hanheide M, Göbelbecker M, Horn GS, et al. Robot task planning and explanation in open and uncertain worlds. *Artificial Intelligence*. 2015;.
- [13] Chen K, Yang F, Chen X. Planning with task-oriented knowledge acquisition for a service robot. In: *International Joint Conference on Artificial Intelligence*; 2016. p. 812–818.
- [14] Khandelwal P, Zhang S, Sinapov J, et al. Bwibots: A platform for bridging the gap between ai and human–robot interaction research. *The International Journal of Robotics Research*. 2017;36(5-7):635–659.
- [15] Jeong IB, Ko WR, Park GM, et al. Task intelligence of robots: Neural model-based mechanism of thought and online motion planning. *IEEE Transactions on Emerging Topics in Computational Intelligence*. 2016; 1(1):41–50.
- [16] Lyu D, Yang F, Liu B, et al. SDRL: interpretable and data-efficient deep reinforcement learning leveraging symbolic planning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*; Vol. 33; 2019. p. 2970–2977.
- [17] Leonetti M, Iocchi L, Stone P. A synthesis of automated planning and reinforcement learning for efficient, robust decision-making. *Artificial Intelligence*. 2016;241:103–130.
- [18] Yang F, Lyu D, Liu B, et al. PEORL: Integrating symbolic planning and hierarchical reinforcement learning for robust decision-making. In: *International Joint Conference on Artificial Intelligence*; 2018.
- [19] Sutton RS, Barto AG. *Reinforcement learning: An introduction*. MIT press Cambridge; 1998.
- [20] Shuvaev S, Starosta S, Kvitsiani D, et al. R-learning in actor-critic model offers a biologically relevant mechanism for sequential decision-making. *NeurIPS*. 2020;33.
- [21] Puterman ML. *Markov Decision Processes*. New York, USA: Wiley Interscience; 1994.
- [22] Mahadevan S. Average reward reinforcement learning: Foundations, algorithms, and empirical results. *Machine learning*. 1996;22(1):159–195.
- [23] Tadepalli P, Ok D, et al. H-learning: A reinforcement learning method to optimize undiscounted average reward. *Oregon State University*; 1994. 94–30–1.
- [24] Mahadevan S, Liu B. Basis construction from power series expansions of value functions. *Advances in Neural Information Processing Systems (NIPS)*. 2010;23:1540–1548.
- [25] Barto A, Mahadevan S. Recent advances in hierarchical reinforcement learning. *Discrete Event Systems Journal*. 2003;13:41–77.
- [26] Sutton RS, Precup D, Singh S. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*. 1999;112(1-2):181–211.
- [27] Lee J, Lifschitz V, Yang F. Action Language $\mathcal{B}\mathcal{L}$: A Preliminary Report. In: *International Joint Conference on Artificial Intelligence (IJCAI)*; 2013.
- [28] Gebser M, Kaufmann B, Schaub T. Conflict-driven answer set solving: From theory to practice. *Artificial Intelligence*. 2012;187-188:52–89.
- [29] McDermott D, Ghallab M, Howe A, et al. PDDL-the planning domain definition language; 1998.
- [30] Gelfond M, Lifschitz V. Action languages. *ETAI*. 1998;.
- [31] Lifschitz V. What is answer set programming? In: *AAAI*. MIT Press; 2008. p. 1594–1597.
- [32] Erdoğan ST, Lifschitz V. Actions as special cases. In: *KR*; 2006. p. 377–387.
- [33] Erdoğan ST. A library of general-purpose action descriptions [dissertation]. University of Texas at Austin;

- 2008.
- [34] Inclezan D, Gelfond M. Modular action language ALM. *Theory and Practice of Logic Programming*. 2016;16(2):189–235.
 - [35] Mahadevan S. Average reward reinforcement learning: Foundations, algorithms, and empirical results. *Machine Learning*. 1996;22:159–195.
 - [36] Kulkarni TD, Narasimhan K, Saeedi A, et al. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In: *Advancement on Neural Information Processing Systems*; 2016. p. 3675–3683.
 - [37] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double q-learning. In: *Association for the Advancement of Artificial Intelligence*; Vol. 16; 2016. p. 2094–2100.
 - [38] Schaul T, Quan J, Antonoglou I, et al. Prioritized experience replay. *arXiv preprint arXiv:151105952*. 2015;.
 - [39] Bellemare MG, Naddaf Y, Veness J, et al. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*. 2013;47:253–279.
 - [40] Lipton ZC. The mythos of model interpretability. *arXiv preprint arXiv:160603490*. 2016;.
 - [41] Wood DA. A transparent open-box learning network provides insight to complex systems and a performance benchmark for more-opaque machine learning algorithms. *Advances in Geo-Energy Research*. 2018;2(2):148–162.
 - [42] Herman B. The promise and peril of human evaluation for model interpretability. *arXiv preprint arXiv:171107414*. 2017;.

Chapter 11

Neuro-Symbolic Semantic Reasoning

Bassem Makni, Arizona State University

Monireh Ebrahimi, IBM Watson

Dagmar Gromann, University of Vienna

Aaron Eberhart, Kansas State University

11.1. Introduction

Humans have astounding reasoning capabilities. They can learn from very few examples while providing explanations for their decision-making process. In contrast, deep learning techniques—even though robust to noise and very effective in generalizing across several fields including machine vision, natural language understanding, speech recognition, etc. —require large amounts of data and are mostly unable to provide explanations for their decisions. Attaining human-level robust reasoning requires combining sound symbolic reasoning with robust connectionist learning as outlined by Valiant in “Knowledge Infusion: In Pursuit of Robustness in Artificial Intelligence” “We argue that to face this challenge one first needs a framework in which inductive learning and logical reasoning can be both expressed and their different natures reconciled.” ([1]) However, connectionist learning uses low-level representations—such as embeddings—rather than “symbolic representations used in knowledge representation” ([2]). This challenge constitutes what is referred to as the *Neuro-Symbolic gap*. A field of study to bridge this gap between the two paradigms has been called neuro-symbolic integration [3] or neuro-symbolic computing [4]. This chapter aims to present approaches that contribute towards bridging the *Neuro-Symbolic gap* specifically in the Semantic Web field, RDF Schema (RDFS) and $\mathcal{EL}^+$ reasoning and to discuss the benefits and shortcomings of neuro-symbolic reasoning.

A significant strength of symbolic approaches and rule-based reasoners is their soundness and completeness. However, they suffer from two main limitations:

1. **Dealing with noisy data.** By design, rule-based reasoners are affected by noisy data, and the veracity of the inferred triples is dependent on the veracity of the input triples ([5] presented a taxonomy of noise types in Semantic Web (SW) data with respect to its effect on the inference).

2. **Lack of support for approximate reasoning.** Rule-based reasoners aim at sound and complete inference. However, in many scenarios—especially in interactive applications—it is preferable to provide the user with an incomplete set of inferred triples in a timely manner rather than a “delayed” complete inference.

Since neural approaches are generally robust, joining them with symbolic approaches holds the promise of noise-tolerant reasoning [5]. Added value can also be expected in terms of algorithm runtime and ability to transfer learned knowledge with a joint neuro-symbolic reasoning approach [3]. Additionally, *approximate reasoning* [6] trades off soundness and completeness to improve efficiency and performance.

A large variety of techniques to generate Resource Description Framework (RDF) or Knowledge Graph (KG) embedding techniques has been proposed (see Section 11.3 for a brief overview). Even though RDFS closure generation can be seen as a KG completion problem, the problem setting is different—making KG embedding techniques that were designed for link prediction not suitable for RDFS or $\mathcal{EL}^+$ reasoning. This motivated the introduction of neuro-symbolic approaches that are tailored for semantic reasoning.

This chapter provides an overview of recent neuro-symbolic reasoning approaches focusing on RDFS reasoning (in section 11.4.1) and $\mathcal{EL}^+$ reasoning (in section 11.4.3). One of the main limitations of the described deep RDFS reasoner is that it needs to be re-trained for each ontology. This limitation is addressed in section 11.4.2 that proposes an approach allowing transfer learning between ontologies. Together with related work and some preliminary details, these three approaches present a solid overview of the current state of neuro-symbolic reasoning. Finally, to further advance research in this field, this chapter will conclude with a discussion on the benefits and shortcomings of neuro-symbolic reasoning.

11.2. Preliminaries

11.2.1. RDFS Reasoning

The famous saying in the SW community, “A Little Semantics Goes a Long Way” [7] advocates for using light semantics such as RDFS [8] to achieve the SW vision—where humans and machines interact to process, produce and reason about the Web of data.

RDF Schema is “a data-modelling vocabulary for RDF data” ([9]) that extends the RDF semantics with class hierarchy, property hierarchy as well as domain and range restrictions for the properties. Given an RDF graph describing a set of resources (A-Box) and an RDFS ontology describing the classes and relations (T-Box), RDFS entailment rules [8] generate new facts that extend our knowledge about the A-Box. Table 1 lists the RDFS rules from [8], for example RDFS5 is the *rdfs:subPropertyOf* transitivity rule, RDFS11 is the *rdfs:subClassOf* transitivity rule while RDFS2 and RDFS3 are type inference rules based on domain and range restrictions respectively.

Type inference consists of inferring the corresponding classes from the TBox for resources in the ABox. It can be considered as a main step towards full RDFS reasoning as almost half of the rules in RDFS- namely *RDFS1*, *RDFS2*, *RDFS3*, *RDFS4a*, *RDFS4b* and *RDFS9* [8]- generate type inference.

RDFS closure can be seen as a Knowledge Graph Completion (KGC) problem—multi-relational link prediction problem in particular—where each RDFS rule generates different types of links:

Table 1. RDFS rules premises and conclusions (from [8]) extended with pattern types column

Rule	Premise	Pattern type	Conclusion
RDFS1	any IRI aaa in D	-	aaa rdf:type rdfs:Datatype .
RDFS2	aaa rdfs:domain xxx . yyy aaa zzz .	TBox ABox	yyy rdf:type xxx .
RDFS3	aaa rdfs:range xxx . yyy aaa zzz .	TBox ABox	zzz rdf:type xxx .
RDFS4a	xxx aaa yyy .	-	xxx rdf:type rdfs:Resource .
RDFS4b	xxx aaa yyy.	-	yyy rdf:type rdfs:Resource .
RDFS5	xxx rdfs:subPropertyOf yyy . yyy rdfs:subPropertyOf zzz .	TBox TBox	xxx rdfs:subPropertyOf zzz .
RDFS6	xxx rdf:type rdf:Property .	TBox	xxx rdfs:subPropertyOf xxx .
RDFS7	aaa rdfs:subPropertyOf bbb . xxx aaa yyy .	TBox ABox	xxx bbb yyy .
RDFS8	xxx rdf:type rdfs:Class .	TBox	xxx rdfs:subClassOf rdfs:Resource .
RDFS9	xxx rdfs:subClassOf yyy . zzz rdf:type xxx .	TBox ABox	zzz rdf:type yyy .
RDFS10	xxx rdf:type rdfs:Class .	TBox	xxx rdfs:subClassOf xxx .
RDFS11	xxx rdfs:subClassOf yyy . yyy rdfs:subClassOf zzz .	TBox TBox	xxx rdfs:subClassOf zzz .
RDFS12	xxx rdf:type $\leftarrow \perp$ rdfs:ContainerMembershipProperty .	TBox	xxx rdfs:subPropertyOf rdfs:member .
RDFS13	xxx rdf:type rdfs:Datatype .	TBox	xxx rdfs:subClassOf rdfs:Literal .

- (a) links between TBox concepts (*RDFS10*, *RDFS11*)
- (b) links between TBox properties (*RDFS5*, *RDFS6*)
- (c) links between ABox entities and TBox concepts (*RDFS2*, *RDFS3*, *RDFS9*)
- (d) links between ABox entities (*RDFS7*)

We refer to the RDFS closure computation as *schema-guided KGC* as the links are generated according to the ontology (TBox), unlike *data-driven KGC* where the links are predicted based on the analysis of the existing links in the KG. Data-driven KGC models “heavily rely on the connectivity of the existing KG and are best able to predict relationships between existing, well-connected entities” ([1]). The predicted links from data-driven KGC might be seen as links between ABox entities and thus similar to the *dcase* in the schema-guided KGC. However, there is a crucial difference: all the relations generated by the data-driven KGC are necessarily seen before in the initial KG. In contrast, the generated relation (or link label) by the *RDFS7* rule is a super-property that might not be seen in the initial KG as it is defined only in the TBox as a node (not even as a link type).

11.2.2. $\mathcal{EL}^+$ Reasoning

$\mathcal{EL}^+$ is a lightweight and highly tractable description logic. A typical reasoning task in $\mathcal{EL}^+$ is a sequential process with a fixed endpoint, similar to RDFS, making it a

Table 2. $\mathcal{EL}^+$ Semantics

Description	Expression	Semantics
Individual	a	$a \in \Delta^{\mathcal{I}}$
Top	$\top$	$\Delta^{\mathcal{I}}$
Concept	C	$C^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$
Role	R	$R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$
Conjunction	$C \sqcap D$	$C^{\mathcal{I}} \cap D^{\mathcal{I}}$
Existential Restriction	$\exists R.C$	$\{ a \mid \text{there is } b \in \Delta^{\mathcal{I}} \text{ such that } (a, b) \in R^{\mathcal{I}} \text{ and } b \in C^{\mathcal{I}} \}$
Concept Subsumption	$C \sqsubseteq D$	$C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$
Role Subsumption	$R \sqsubseteq S$	$R^{\mathcal{I}} \subseteq S^{\mathcal{I}}$
Role Chain	$R_1 \circ \dots \circ R_n \sqsubseteq R$	$R_1^{\mathcal{I}} \circ \dots \circ R_n^{\mathcal{I}} \subseteq R^{\mathcal{I}}$

with $\circ$ signifying standard binary composition

Table 3. $\mathcal{EL}^+$ Completion Rules

(1)	$A \sqsubseteq C$	$C \sqsubseteq D$	$\models A \sqsubseteq D$
(2)	$A \sqsubseteq C_1$	$A \sqsubseteq C_2$	$C_1 \sqcap C_2 \sqsubseteq D \models A \sqsubseteq D$
(3)	$A \sqsubseteq C$	$C \sqsubseteq \exists R.D$	$\models A \sqsubseteq \exists R.D$
(4)	$A \sqsubseteq \exists R.B$	$B \sqsubseteq C$	$\exists R.C \sqsubseteq D \models A \sqsubseteq D$
(5)	$A \sqsubseteq \exists S.D$	$S \sqsubseteq R$	$\models A \sqsubseteq \exists R.D$
(6)	$A \sqsubseteq \exists R_1.C$	$C \sqsubseteq \exists R_2.D$	$R_1 \circ R_2 \sqsubseteq R \models A \sqsubseteq \exists R.D$

perfect candidate for sequence learning. Unlike RDFS, which reasons over instance data in a triple format, $\mathcal{EL}^+$ reasoning occurs on the predicate level and does not consider instances. Thus, we will have to train a neural system to actually learn the reasoning patterns and logical structure of $\mathcal{EL}^+$ directly from encoded knowledge bases.

11.2.2.1. Syntax

The signature Σ for $\mathcal{EL}^+$ is defined as $\Sigma = \langle N_I, N_C, N_R \rangle$ with N_I, N_C, N_R pairwise disjoint. N_I is a set of individual names, N_C is a set of Concept names that includes $\top$, and N_R is a set of Role names. Expressions in $\mathcal{EL}^+$ use the following grammar:

$$\begin{aligned} \mathbf{R} &::= N_R \\ \mathbf{C} &::= N_C \mid \mathbf{C} \sqcap \mathbf{C} \mid \exists \mathbf{R}. \mathbf{C} \end{aligned}$$

11.2.2.2. Semantics

An interpretation I where $I = (\Delta^I, \cdot^I)$ maps N_I, N_C, N_R to elements, sets, and relations in Δ^I with function $\cdot^I$. For an interpretation I and $C(i), R(i) \in \Sigma$, the function $\cdot^I$ is defined in Table 2.

11.2.2.3. Reasoning

It is a well-established result that any $\mathcal{EL}^+$ knowledge base has a least fixed point that can be determined by repeatedly applying a finite set of rules that produce all entailments of a desired type [10, 11]. One such set of rules, the set we have used in our experiment,

is illustrated in Table 3. The reasoning reaches *completion* when there are no new conclusions to be made. A standard reasoning task for $\mathcal{EL}^+$ is to derive all normal-form concept inclusions and restrictions, since this is sufficient to classify the atomic concepts in a knowledge base, and those are the types of statements we choose to include in our reasoning.

11.2.3. Noise Handling

The Web is inherently noisy, and as such, its extension is noisy as well. This noise results from inevitable human error when creating the content, designing the tools that facilitate the data exchange, conceptualizing the ontologies that allow machines to understand the data content, mapping concepts from different ontologies, etc. For instance, the noise can be a consequence of building Linked Open Data (LOD) from semi-structured or non-structured data. When LOD is built from non-structured data, such as text using Named Entity Linking (NEL) tools—whose accuracy is not perfect—they generate erroneous triples. Thus, the integrity of the inference becomes questionable.

Many research efforts concentrate on noise detection and data cleansing in the Web of data. Knowing that there will always be other instances or types of noise that will be overlooked, other research efforts focus on noise tolerance instead. The strategies of handling noise in Semantic Web data can be classified into two categories:

Active noise handling consists of detecting noise and cleansing the data before performing any tasks that might be affected by the presence of noise

Adaptive noise handling approaches focus rather on building techniques that are noise-tolerant, given the unrealistic expectation of cleansing every type of noise in Semantic Web data.

11.3. Related Work

Knowledge graphs and graph-based representations, including graph embedding techniques, are omnipresent nowadays. Their broad application scenarios range from social network analysis and recommendation systems to protein interaction networks and knowledge and Semantic Web graphs in particular. This can be considered as the primary motive for graph analytics research. Graph analytics tasks include centrality analysis, nodes classification [12], link prediction [13], etc. The last one is closest to the scope of this chapter because the inferred RDF graph can be seen as the link prediction applied to the input graph. Thus, we will focus on RDF and Web Ontology Language (OWL) graph embeddings and refer the interested reader to these surveys on knowledge graph embedding techniques and their applications [14, 15] for a more comprehensive introduction to the topic. An introduction to machine learning techniques for approximate semantic reasoning will follow.

11.3.1. Knowledge Graph Embeddings

Graph embedding approaches can be classified using several criteria. One particular criterion from [16] is the “problem setting”. The problem setting uses the type of graph input as well as the embedding output to classify the embedding approach. For the input part of the problem setting, the graph can be either:

- **Homogeneous.** Where all the nodes have same type and all the edges have same type as well.
- **Heterogeneous.** Where there are multiple types of nodes and/or multiple types of edges. This is the case for RDF graphs.

The majority of graph embedding approaches yield node representation in a low-dimensional space. This is why graph embedding and node embedding are often used interchangeably. However, there are other types of graph embedding outputs such as:

- **Edge embedding.** The output, in this case, is a low dimensional representation of the edges. This is particularly useful for knowledge graphs [17] where the type of edges between nodes is crucial to determine their similarities.
- **Whole graph embedding.** The output is a vector representation of the whole graph—not only node or edge vectors. The embedding vectors of similar graphs should be neighbors in the embedding space. The embedding of RDF graphs—to learn their inference—falls under this category.

Embedding of RDF Graphs. RDF embedding techniques can be classified into:

Graph kernels for RDF. One of the earliest works in this class was [18] where the authors apply general graph kernel methods on RDF graphs and propose two kernels specific to RDF, namely intersection graph kernels and intersection tree kernels. In [19], the authors consider state-of-the-art graph kernels which are Weisfeiler-Lehman graph kernels [20] and adapt them to RDF graphs. [21] proposes an h-hop neighborhood-based graph kernel for LOD and they apply it in a linked data recommender system.

2vec RDF embedding. These approaches use the following generic method: generate sequences of entities from the RDF graph using graph walks or other graph kernels and then apply a technique similar to word2vec [22]—where each entity in the sequence is treated as a word in a sentence. In RDF2Vec [23], the sequences are generated using graph walks and using the Weisfeiler-Lehman adaptation to RDF graphs [19] mentioned previously. [24] improves RDF2Vec by using biased graph walks to generate the entities' sequences. [25] substitute word2vec with a technique similar to GloVe (Global Vectors) [26] to explore the global patterns of the RDF graph instead of the local patterns as in RDF2Vec. The authors report similar embedding quality as RDF2Vec but with the ability to incorporate larger portions of the graph.

2vec OWL approaches. OWL2Vec captures the semantics of OWL 2 ontologies by exploring the neighborhood classes using random walks [27]. Since this preliminary approach did not fully capture all logical and lexical information available in a graph, OWL2Vec* [28] was introduced. It also walks over the graph forms but generates three documents as output: the graph structure and logical constructors, the lexical information of a KG, and a combination of the previous two. To this end, the authors adopt the skip-gram architecture of word2vec [22]. The resulting embeddings have been tested on class membership prediction and class subsumption prediction, and the authors report better results than RDF2Vec and other KG embedding methods.

For learning RDFS reasoning, the whole-graph embedding is required because the input of the learning algorithm is the input graph, and the output is the inference graph. However, existing embedding techniques for KGs were not designed for full RDFS reasoning, and they raise two main challenges if they were to be used for this task.

1. The first challenge is the need to check the validity of every possible triple (using the scoring function) to generate the full materialization.
2. The second challenge is embedding the relations seen only in the inference as the embeddings should be learned only from the input graph and be used in generating the inference graphs. For instance, when the LUBM's property *mastersDegreeFrom* appears in the input graph, its super-property *degreeFrom* appears in the inference graph by applying the RDFS7 rule [8]. If *degreeFrom* was not seen in the input graph, then its embedding was not learned.

11.3.2. Machine Learning and Approximate Semantic Reasoning

A second major class of related approaches are approximate semantic reasoning methods relying on machine learning techniques. In terms of type inference, *SDType* algorithm [29, 30] used a statistical distribution of types to predict the type of object and subject in a triple given that they are connected with a certain property. Their statistical approach makes this type inference mechanism robust to noise in RDF data. [31] targeted inferring the missing types in DBpedia resources through an inductive and an abductive approach. In the inductive approach, the k-Nearest Neighbors algorithm is used to determine the closest concepts from the DBpedia ontology to which the resource should be linked. In the abductive approach, the Encyclopedia Knowledge Paths [32] are used in a similarity metric.

In terms of consistency checking, [33] aimed to detect systematic errors in DBpedia by aligning the DBpedia ontology, and the upper-level ontology DOLCE-Zero [34]. By clustering the reasoning results, they found that 40 clusters cover 96% of the inconsistencies. This observation was among the motivations that approximate semantic reasoning could cover most of the use cases where ontological reasoning is required. To speedup the process of ABox consistency checking, [35] used an approximate semantic reasoning approach based on machine learning. The authors formalized the problem as a binary classification problem where each classifier C is trained for a specific TBox to decide if any ABox is consistent or not with respect to the TBox. In order to transform the RDF graphs into feature vectors, graph walks [36] were used. The decision tree model achieves 95% accuracy within 2% of the time required by a semantic reasoner.

In [37], the authors extend the clash queries [38] for *DL-Lite* [39] and caching to reduce the required calls to a semantic reasoner in order to check ABox inconsistency. This approach had better running time and empirical accuracy than [35].

The idea of neuro-symbolic integration dates back to 1943 when McCulloch and Pitts [40] showed how propositional logic formulas can be represented in simple neural network models. A more recent survey on neuro-symbolic reasoning has been proposed by [10]. One of the closest research efforts to the scope of this chapter is [41] which proposes Relational Tensor Networks (RTN) as an adaptation of Recursive Neural Tensor Networks (RNTN) [42] for relational learning. RNTNs were originally designed by Socher to support learning from tree-structured data such as sentences' parse trees, and they were used successfully to improve sentiment analysis results. In [41], the authors

start by building a Directed Acyclic Graph (DAG) representation of the RDF input. Every resource in the graph is initially represented as an incidence vector that indicates the set of *rdf:type(s)* of the resource. Then, the embeddings of the resources are computed using the RTN model that takes into consideration the type or the relation that each resource has. Two types of targets are considered: a unary target for type prediction and a binary target for predicate classification. The input for the binary targets are the embeddings of two resources—to which the predicates are being classified.

This initial idea was extended to a Recursive Reasoning Network (RRN) that seeks to encode input and inference information in the embedding space by performing local reasoning during embedding training [43]. In this work the authors focus on OWL RL reasoning [44, 45]. An attempt to embed $\mathcal{EL}^{++}$ building on the translation-based approach for training KG embeddings TransE [46] is also a promising direction, even though the RBox and certain $\mathcal{EL}^{++}$ axioms have not been considered [47]. To address neuro-symbolic stream reasoning, [48] approximate reasoning with Continuous SPARQL (C-SPARQL) [49] by building on RNNs and CNNs to process queries over various RDF streams simultaneously, and update knowledge bases with new information obtained. This approach holds the promise of processing higher quantities of data.

Data-driven KGC literature includes [50, 51, 52, 53, 54, 55]. Logic Tensor Networks [54] allow the definition of logical constraints to improve the KGC. [55] aims not only at predicting missing relations in a KG but also at inducing the logical rules from it. In this direction and building on Neural LP, [56] proposed Neural-Num-LP to explicitly address numerical operations, such as comparison, aggregation, and negation, and [57] propose Neural Logic Inductive Learning (NLIL) to express complex rules based on a stack of three hierarchical Transformer networks.

Most of the above works focus on intact data, not considering noise-tolerance, and training and evaluating on the same knowledge graph, making the systems inapt for transfer learning.

11.3.3. Handling Noise in Semantic Web Data

Active Noise Handling Cleansing potentially noisy data before feeding it to a rule-based reasoner can be achieved through a probabilistic approach, a supervised or unsupervised approach. An example of a probabilistic approach is *SDValidate* [30] where the authors aim to identify wrong triples when there is a large deviation between the resource types by assigning a *relative predicate frequency*—describing the frequency of predicate/object combinations. Probability distributions are then used to decide if a statement with low *relative predicate frequency* should be considered erroneous. Detecting noisy type assertions can also be framed as a supervised multi-class classification problem. In [58], the authors define a conflict $\langle i, A, B \rangle$ which can fall into one of these four categories: full noisy, fake noisy, 1st half noisy or 2nd half noisy depending on the combinations of $A(i)$ and $B(i)$ being noisy. Then they train a multi-class classifier to detect the category of the conflict. Finally, unsupervised learning can be used to detect the outliers of numerical values for each property. [59] uses a two-phase detection approach: in the first phase, outliers of numerical values are detected for every property, and in the second phase, the *owl:sameAs* property is used to confirm or reject the outliers. [60] uses a few unsupervised learning techniques including Kernel Density Estimation (KDE) [61] combined with semantic grouping to identify the outliers.

Adaptive noise handling Rather than cleansing specific types of noise in the input graphs before feeding them to rule-based reasoners, adaptive noise handling aims at generating the inference graph from noisy data. In the *SDType* algorithm [29, 30], the *rdf:type* inference uses information from the ABox rather than ontological descriptions from the TBox. For instance, instead of using the *rdfs:domain* and *rdfs:range* of the properties to infer the resources' types, which will propagate noise, a weighted voting heuristic is used instead to determine the types of the resources. The weights are generated from the statistical distribution between predicates and types. For example, given that the property *dbo:location* is mostly connected to objects of type *dbo:Place*, then this property will have high weight to infer the type *dbo:Place*. [62] propose TrustE, a KG embedding approach to train entity type embeddings with a focus on learning the tuple trustworthiness to handle noisy KGs. To the best of our knowledge, most of the previous work in the literature about reasoning with noisy Semantic Web data focuses on type inference. This research is the first to aim at full RDFS reasoning with noise-tolerance capability.

11.4. Approaches to Neuro-Symbolic Reasoning

11.4.1. RDFS Reasoning

The first step towards bridging the Neuro-Symbolic gap for RDFS reasoning is to represent RDF graphs in a format that can be fed to neural networks. The most intuitive representation to use is graph representation. However, the different graph models for RDF in the literature were neither designed for RDFS reasoning requirements nor are they suitable for neural network input. The proposed graph model for RDF consists of layering RDF graphs and encoding them in the form of 3D adjacency matrices. Each layer layout in the 3D adjacency matrices forms what we termed a *graph word*. Every input graph and its corresponding inference are then represented as sequences of graph words. The RDFS inference becomes equivalent to the translation of graph words that is achieved through neural network translation.

11.4.1.1. Layered Graph Model for RDF

Despite its effectiveness as a standardized “framework for representing information in the Web” ([63]) and as an essential building block for the SW, the graph representation for the RDF model remains an open question in the SW research community. Even though the RDF conceptual model is designed as a graph, it differs from the graph theory definition of graphs in a number of ways. RDF graphs are heterogeneous multi-graphs. Moreover, an edge in the ABox can be a node in the TBox (describing the properties hierarchy for example). Current research efforts to represent the RDF model as graphs—based on a: bipartite graph model [64], hypergraph model [65, 66, 67] or meta-graph models [68]—target different goals ranging from storing and querying RDF graphs to reducing space and time complexity to solving the reification and provenance problem. Unfortunately, these goals do not coincide with RDFS reasoning and were not formalized to be suitable for neural network input.

The following describes a layered graph model that uses simple directed graphs to achieve the goal of representing RDF graphs and their inference graphs according to

the RDFS rules. It is important to note that the mapping between RDF to the proposed layered model is irreversible—meaning that the reconstruction of the original RDF graph is not guaranteed. Thus, the layered graph model is not suitable for storing and querying RDF data.

Notations and Definitions In Table 1, the premises of RDFS rules were classified into ABox patterns and TBox patterns.

Definition 1. A TBox rule is a rule where its premises are all of type TBox pattern.

The Tbox rules in RDFS are: *RDFS5* (the *subPropertyOf* transitivity rule), *RDFS6* (the *subPropertyOf* reflexivity rule), *RDFS11* (the *subClassOf* transitivity rule) and *RDFS10* (the *subClassOf* reflexivity rule). As these rules' patterns are present in the ontological level and there is only one ontology per training set, there are not enough samples to learn these rules. Thus, it is assumed that there is a materialized version of the ontology where the TBox rules are already applied. This materialized version is inferred only once and is part of the training input. Let:

O : be the materialized ontology.

P : be the set of properties in O .

$P^+ = P \cup \{rdf:type\}$

np : be the size of the set P^+

$(p_1, p_2, \dots, p_{np})$: be a tuple of the elements of P^+ (It is crucial to maintain the same order of elements in this tuple throughout the training process)

$SubjObj(T)$: be the set of subject and object resources of the RDF graph T .

Definition 2. A Layered directed graph is a graph that has multiple sets of directed edges where each layer has its own set of edges.

An n -layered directed graph is a layered directed graph of n layers. More formally, an n -layered directed graph is defined as:

$G(V, (E_1, E_2, \dots, E_n))$ where the edges part is a tuple containing n sets of directed edges.

Definition 3. Layered directed graph for RDF:

An RDF graph T is represented by a layered directed graph:

$G(SubjObj(T), (E_1, E_2, \dots, E_{np}))$ where:

$$(e_i, e_j) \in E_l \iff \begin{cases} (e_i, p_l, e_j) \in T \\ e_i \in SubjObj(T) \\ e_j \in SubjObj(T) \end{cases}$$

It is important to note that the transformation of an RDF graph into its layered directed graph representation is not bijective—given that two non-isomorphic RDF graphs can have the same layered directed graph representation. However, this transformation guarantees that if two RDF graphs have the same layered directed graph representation, then their RDFS inference graphs—according to the ontology O —are isomorphic.

11.4.1.2. RDFS Reasoning as Neural Machine Translation of Graph Words

The way the generic methodology of supervised machine learning is applied in this work is depicted in Figure 1, where the pair (input, target) is the input graph and its corresponding inference. In a nutshell, tensors representing the input graph g and its corresponding inference i are created. The tensors of these graphs are then used in the training phase. The algorithm outputs the tensor of the graph and its encoding dictionary that will be used in the decoding phase to regenerate the original graph.

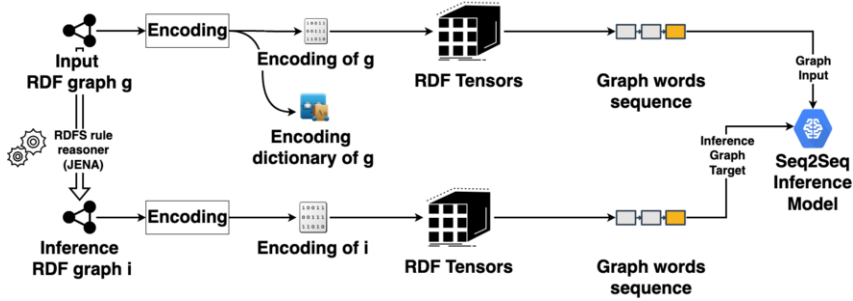


Figure 1. Approach overview

In addition to preparing the RDF graph for input into a neural network, the main goal of this phase is to capture the pattern similarities between graphs in such a way that “similar” graphs will have similar tensors. An example of “similar” graphs is: two graphs containing RDF descriptions of two resources of the same type (such as two *Publications*’ descriptions in the LUBM1 dataset).

1. **Layering the RDF graphs.** The layered graph version for each input and inference graph in the training set is generated according to the layered graph model for RDF. The order of the properties corresponding to each layer can be chosen arbitrarily, but it is crucial to maintain the same order across the dataset.
2. **RDF tensors creation.** Using the layered version of the RDF graphs, an ID must be assigned to each resource in the graph to allow it to be represented as a 3D adjacency matrix. Two dictionaries were created: one for the subject and object IDs—which is split into a global and a local dictionary—and one for the property IDs. The global resources dictionary contains the subject and object resources that are used throughout the G set (which are basically the RDFS classes in the ontology). While the local resources’ dictionaries contain the IDs for the resources specific to each graph.
3. **Graph words generation.** At this stage, every RDF graph is represented as a 3D adjacency matrix of size:
 $(np, \text{max_number_of_resources}, \text{max_number_of_resources})$
 where np is the size of the set P^+ mentioned in 11.4.1.1 i.e the number of layers in each layered RDF graph.
 In theory the maximum number of possible layer layouts in a dataset of size dataset_size is:

$$\min(2^{\text{max_number_of_resources}^2}, \text{dataset_size} * np)$$

However, in practice, the number of layouts is much smaller than this theoretical bound. This observation is a good indication that the encoding algorithm has achieved one of its primary goals of having similar encodings for “similar” graphs. By assigning an ID for each layer’s layout, the 3D adjacency matrix can be represented as a sequence of layouts’ IDs as shown in Figure 2. The layouts are termed “*graph words*”, as the sequence (or phrase) of graph words represents a 3D adjacency matrix and thus an RDF graph. Representing an RDF graph as a sequence of graph words has two main advantages:

- (a) Reducing the size of the encoded dataset: only the ID of the layer’s layout along with a catalog of layouts is saved.
- (b) Exploitation of the research results in neural machine translation.

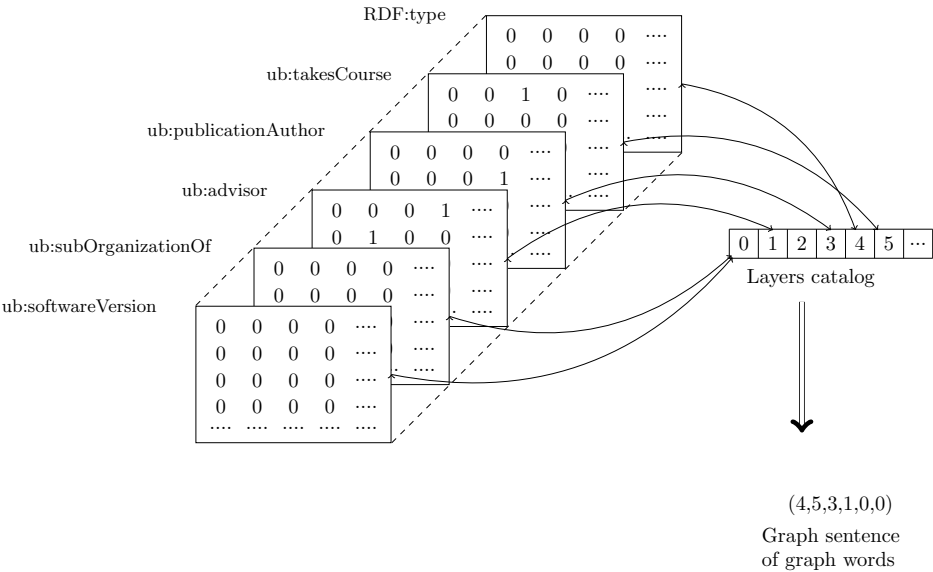


Figure 2. From a 3D adjacency matrix to a sentence of graph words

11.4.1.3. Results

After approximately 12 minutes of training, 98.8% training accuracy was achieved. When testing the trained model on the intact LUBM1 test set, an overall per-graph accuracy of 97.7% was obtained. The trained model was then tested on the noisy datasets as described in [69]. Two metrics were designed:

- **Macroscopic metric: Per-graph accuracy** : Inferences in this metric are scored correct when *dr* and *i* are isomorphic—in other words, when the deep reasoner inference from the corrupted graph is isomorphic to the Jena inference from the intact graph.
- **Microscopic metric: Per-triple precision/recall.** The previous metric overlooks the fact that some triples, generated by the deep reasoner and not by Jena, were in fact valid.

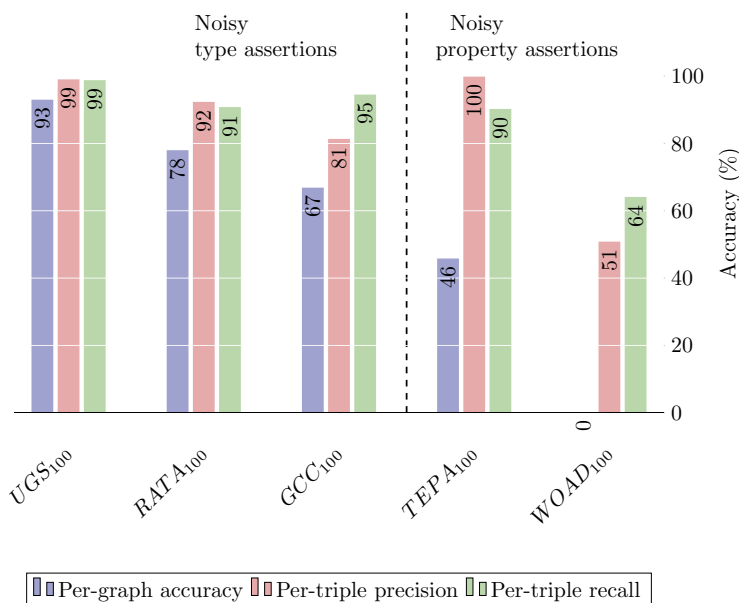


Figure 3. Macro and micro evaluation on noisy LUBM1 datasets

The macro and micro evaluation on the 5 noisy datasets (Figure 3) shows exceptional noise-tolerance compared to rule-based reasoners: 99% for the deep reasoner *vs* 0% (by design) for Jena in the UGS_{100} dataset.

11.4.1.4. Discussion

The main contribution of the graph words translation model is the empirical evidence that deep learning can be used to learn semantic reasoning—RDFS rules specifically. The goal was not to reinvent the wheel and design a Yet another Semantic Reasoner (YaSR) using new technology; it was rather to fill a gap that existing rule-based semantic reasoners could not satisfy, which is noise-tolerance. However, one of the main deficiencies of this neuro-symbolic approach compared to rule-based reasoners is the lack of derivations for the inferred triples (i.e. explainability in AI terms). In many applications such as Question Answering (QA) [70, 71, 72], not only the inferred fact is required but also the derivation of the triple. The derivation can be useful to justify the answer provided by the QA system, which allows the Human-in-the-loop (HITL) to validate or reject the answer.

Another limitation of this approach is that the training is done on a dataset that uses only one ontology for the inference. After training the graph-to-graph model on one dataset, the model hyperparameters need to be adapted for the new datasets according to the size of the graph words sequences, and the training is started from scratch. The following section explores transfer learning to mitigate this limitation.

11.4.2. Transfer Learning for RDFS Reasoning

Current neuro-symbolic systems learn fact relations and entailment patterns specific to a particular domain/vocabulary and hence do not truly learn to reason, and must be re-trained for each new domain they are challenged with entailing. This inherent shortcoming is due to both the representation learning functions used and the learning process. The major concern originates from the mere reliance of these models on the representation of entities learned during the training or in the pre-training phase stored in an embedding lookup table. An even bigger limitation is that reasoning in most of the sub-symbolic methods hinges more on the notion of similarity and geometric-based proximity of real-valued vectors (induction) as opposed to performing transitive reasoning (deduction) over them. To address these limitations, here, we introduce a differentiable end-to-end deep memory network-based model that learns over abstract, generic symbols to discover entailment patterns common to our RDFS reasoning task. An important part of our system is a simple but highly effective normalization process for learning the representation of knowledge graph entities within memory networks. Memory networks [73] are a class of learning models capable of conducting multiple computational steps over an explicit memory component before returning an answer. We have opted for such networks since we believe that they are a primary choice to conduct well for deductive logical reasoning. Their sequential multi-hop nature corresponds, conceptually, to the sequential process underlying some deductive entailment algorithms. The attention modeling helps to pull only relevant statements (logical axioms) necessary for the next reasoning step. And their success in natural language entailment is also promising: although natural language entailment does not follow a formal logical semantics, logical deductive entailment is, however, similar to natural language reasoning in some aspects. Additionally, since attention can be traced over the run of a memory network, we will furthermore get insights into the "reasoning" underlying the network output, as we will be able to see which pieces of the input knowledge graph are taken into consideration at each step leading to interpretable results.

As such, the goal is to train a neural model that will learn to reason over one set of theories, and can then transfer that learning to new theories over the same logic. One of the main barriers encountered in this task is figuring out how to represent training and test data. To leverage standard neural approaches, symbols will have to be mapped over the real coordinate space R as vectors, matrices, or tensors. Many embeddings for knowledge graphs have been suggested, but we are not aware of an existing representation suitable for the deductive reasoning scenario. Indeed, the salient use case explored for knowledge graph embeddings is not deductive in nature. Rather, it concerns the problem of the link prediction, i.e., discovery or suggestion of additional links or edges in the graph, together with their appropriate labels. In this link prediction setting, the actual names for entities or relations in the graph, and as such their commonsense meanings, are likely important, and most existing embeddings reflect this. However, for deductive reasoning, the labels of entities and relations are insubstantial and should not be captured by an embedding.

We hence need representations that are agnostic to the terms used as primitives in the knowledge graph. To construct such a representation, we use what we call syntactic normalization: a renaming of primitives from the logical symbols to a set of predefined entity names that are used across different normalized theories. By randomly assigning the mapping for the renaming, the network's learning will be based on the structural

information within the theories and not on the actual names of the primitives. Note that this normalization not only plays the role of “forgetting” irrelevant label names but also makes it possible to transfer learning from one knowledge graph to the other. Indeed, the network can be trained over many knowledge graphs and then subsequently tested on completely new ones. Note that our deductive transfer learning work falls under the umbrella of transfer learning as it uses the knowledge gained while solving a source problem and applying it to a different but related target problem. However, our solution is different from the common solutions in transductive and inductive transfer learning tasks in the natural language processing domain as it does not rely on the pre-stored knowledge in the form of pre-trained embeddings.

11.4.2.1. Approach

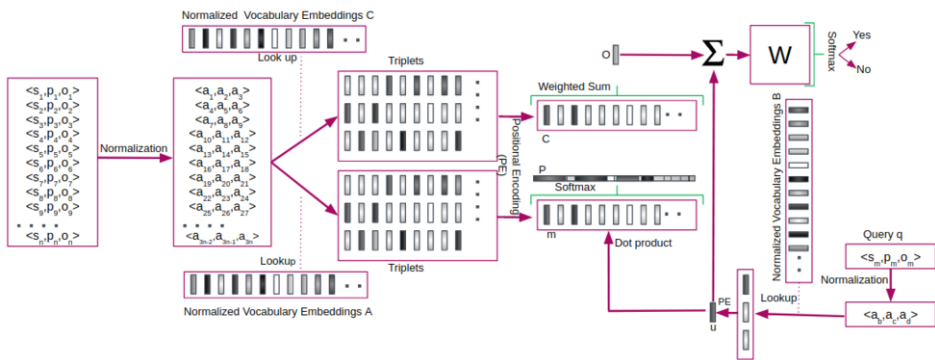


Figure 4. Diagram of the proposed model, for $K=1$

We leverage a model architecture that modifies the MemN2N with fundamental alterations necessary for abstract reasoning. A high-level view of our model is demonstrated in Figure 4. It takes a discrete set G of normalized RDFS statements (called *triples*) $t_1, \dots, t_n$ that are stored in memory, a query q , and outputs a “yes” or “no” to determine if q is entailed by G . Each of the normalized t_i and q contains symbols coming from a general dictionary with V normalized words shared among all of the normalized RDFS theories in both training and test sets. The model stores all triples in the memory and then obtains a continuous representation for G and q . Through multi-hop attention over those embeddings, the model then classifies the query. The model is trained by back-propagation of error from output to the input through multiple memory accesses. More specifically, the model is augmented with an external memory that stores the embeddings of the normalized triples in our knowledge graph. This memory is defined as an $n \times d$ tensor where n denotes the number of triples in the knowledge graph, and d is the dimensionality of the embeddings. The knowledge graph is stored in the memory vectors with two continuous representations of m_i and c_i obtained from two input and output embedding matrices of A and C with size $d \times V$ where V is the size of the vocabulary. Likewise, the query q is embedded via a matrix B to calculate an internal state u . In each reasoning step, those memory slots useful for finding the correct answers should have their contents retrieved. To do this, we use an attention mechanism for q over memory input representations by taking an internal product followed by a softmax:

$$p_i = \text{Softmax}(u^T(m_i))$$

It calculates a probability vector p over the memory inputs, the output vector o is then computed as the weighted sum of the transformed memory contents c_i with respect to their corresponding probabilities p_i by $o = \sum_i p_i c_i$. This describes the computation within a single computational hop. The internal state of the query vector updates for the next hop as $u^{k+1} = u^k + o^k$. The process repeats K times where K is the number of computational hops. The output of the K^{th} hop is used to predict the label $\hat{a}$ by passing o^K and u^K through a weight matrix of size $V \times d$ and a softmax:

$$\hat{a} = \text{Softmax}(W(u^{K+1})) = \text{Softmax}(W(u^k + o^k)).$$

Figure 4 shows the model for $K = 1$. The parameters to be learned by backpropagation are A, B, C , and W matrices.

Memory Content An RDFS knowledge graph is a set of statements stored as triples $(e1, r, e2)$ where $e1$ and $e2$ are called *subject* and *object*, respectively, while r is a relation connecting $e1$ and $e2$. Every entity in an RDFS knowledge graph is represented by a unique Universal Resource Identifier (URI). We normalize these triples by systematically renaming all URIs which are not in the RDF or RDFS namespaces. Each such URI is mapped to a set of arbitrary strings in a predefined set $\mathcal{A} = \{a_1, \dots, a_n\}$, where n is taken as a training hyper-parameter giving an upper bound for the largest number of entities in a knowledge graph the system will be able to manage. Note that URIs in the RDF/RDFS namespaces are not renamed since they are important for deductive reasoning according to the RDFS model-theoretic semantics. Thus, each normalized RDFS knowledge graph will be a collection of facts stored as triples $\{(a_i, a_j, a_k)\}$.

It is worth mentioning that each symbol is mapped into an element of $\mathcal{A}$ regardless of its position in the triple. Yet the position of an element within a triple is an important feature to take into account. Thus we employ a positional encoding (PE) [74] to encode the position of each element within the triple. Each memory slot thus represents the positional-weighted summation of each triple. The PE ensures that the order of the elements now affects the encoding of each memory slot m_i .

11.4.2.2. Evaluations

Dataset We have collected RDFS datasets from the Linked Data Cloud¹ and the Data Hub². Our training set (“OWL-centric”) is comprised of a set of RDFS knowledge graphs each of size 1,000 triples, sampled from populating around 20 OWL ontologies with different data. To test our model’s ability to generalize to completely different datasets, we have collected another dataset called the OWL-Centric Test Set. To assure our evaluation represents real-world RDFS data completely independent of the training data, we have used almost all RDFS knowledge graphs listed in [75]; we call this the Linked Data test set. Furthermore, to test the limitations of our model on artificially difficult data, we have created a small synthetic dataset that requires long reasoning chains if done with a sym-

¹<https://lod-cloud.net/>

²<https://datahub.io/>

Training Dataset	Test Dataset	Accuracy	
		Our model	Baseline
OWL-Centric	Linked Data	96	43
OWL-Centric(90%)	OWL-Centric (10%)	90	82
OWL-Centric	OWL-Centric Test Set *	69	61
OWL-Centric	Synthetic Data	52	48

* Completely Different Domain.

Table 4. Experimental results of the proposed model

bolic reasoner. For each knowledge graph, we have generated the finite set of inferred triples using the Apache Jena³ API. These inferred triples comprise our positive class instances. We obtained non-inferred triples by random permutation of triple entities and removing those triples which were entailed.

Results Training was done over 10 epochs using the Adam optimizer with a learning rate of $\eta = 0.005$, a learning rate decay of $\eta/2$, and a batch size of 100 over triples. All embeddings are vectors of size 20. We have used $K = 10$. Adjacent weight sharing was used where the output embedding of one layer is the input embedding of the next one. All the weights are initialized by a Gaussian distribution with $\mu = 0$ and $\sigma = 0.1$. Here we report the average accuracy over all the knowledge graphs in the test set, calculated for both valid and invalid sets of triples. We have considered the non-normalized embedding version of our memory network as a baseline. Our technique shows a considerable advantage over the baseline, as shown in Table 4. A further even more important benefit of using our normalization model is its training time. Indeed, this considerable time complexity difference is the result of the remarkable size difference of embedding matrices in the original and normalized cases. For instance, the size of embedding matrices to be learned by our algorithm for the normalized OWL-Centric dataset is $3,033 \times 20$ as opposed to $811,261 \times 20$ for the non-normalized one (and $1,974,062 \times 20$ for Linked Data which is prohibitively big). That has caused a remarkably high decrease in training time and space complexity. In the case of the OWL-Centric dataset, for instance, the space required for saving the normalized model is 80 times less than the intact model. Similarly, the normalized model is almost 40 times faster to train than the non-normalized one for this dataset. Thus, the importance of using normalization cannot be emphasized enough.

General Embeddings Visualization We have plotted a Principal Component Analysis (PCA) two-dimensional vector visualization of embeddings obtained for the RDF(S) terms and all normalized words in the knowledge graphs, in Figure 5. The embeddings were fetched from the matrix B (embedding query lookup table) in the hop 1 of our model trained over the OWL-Centric dataset. Words are located in the plot based on the proximity of their embedding vectors. As anticipated, all the normalized words tend to form a single cluster as opposed to multiple ones. The PCA projection demonstrates the ability of our model to automatically organize RDF(S) concepts and implicitly learn the relationships between them. For instance, `rdfs:domain` and `rdfs:range` have been positioned very close together and far from normalized entities. `rdf:subject`, `rdf:predicate` and

³<https://jena.apache.org/>

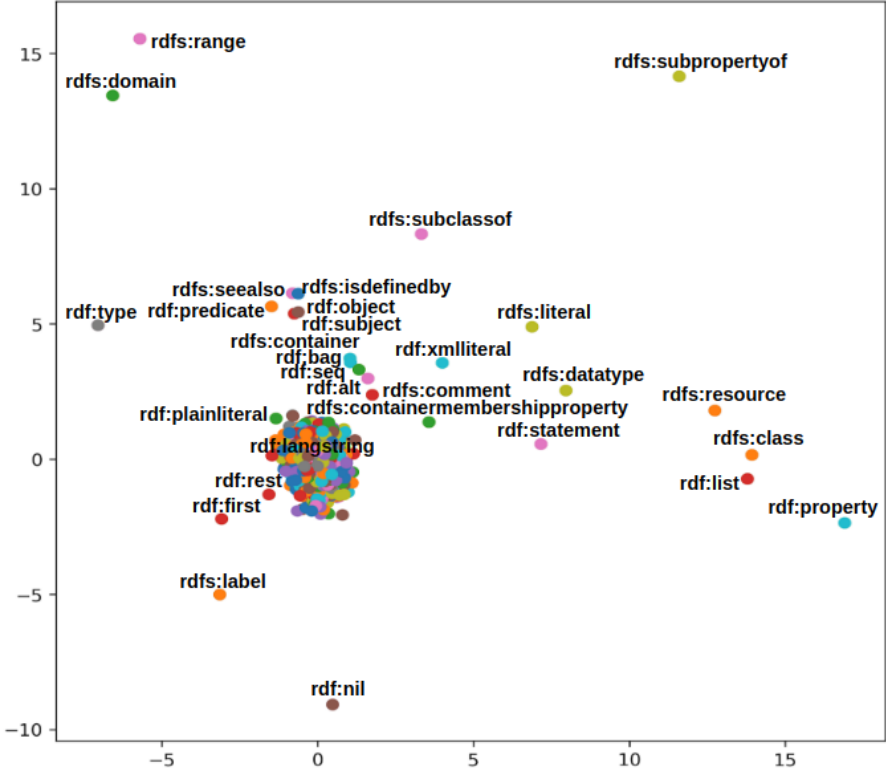


Figure 5. PCA projection of embeddings for the whole vocabulary

rdf:object vectors are very similar, and the same for rdf:seesAlso and rdf:isDefinedBy. Likewise, rdfs:container, rdf:bag, rdf:seq, and rdf:alt are in the vicinity of each other.

11.4.2.3. Discussion

We have shown how emulating symbolic reasoning through sub-symbolic reasoning can lead to a model capable of transferring its reasoning ability from one domain to another without any re/pre-training or fine-tuning over the new domain. To attain this goal, we have introduced a normalization technique to the representation learning in memory networks. We empirically showed our proposed model comfortably outperforms its unnormalized counterparts. Our future goal is to test the scalability of our work and explore other logics and models.

11.4.3. $\mathcal{EL}^+$ Completion Reasoning Emulation

We present a new approach to $\mathcal{EL}^+$ reasoning that seeks to emulate reasoning structure, which can be inspected part-way through, rather than simply learning reasoner answers, which is typical in many black-box systems currently in use. We demonstrate that this idea is feasible by training a long short-term memory (LSTM) artificial neural network to learn $\mathcal{EL}^+$ reasoning patterns with two different data sets. LSTMs are a type of recurrent neural network that work well with sequential data that has long-term dependencies.[76]

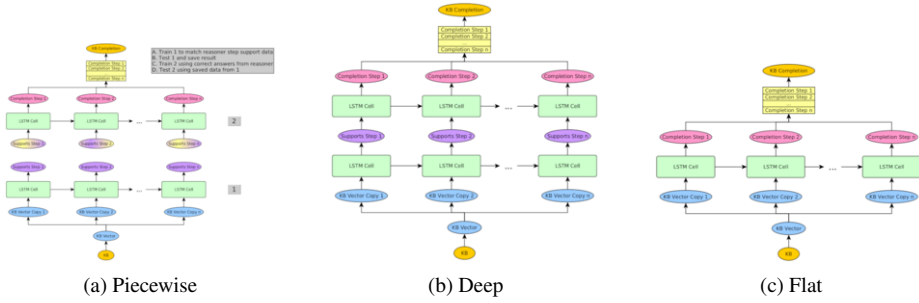


Figure 6. Architectures

In our experiment, logic data is fed into three different LSTM architectures with identical input and output dimensionalities. There is one hidden layer representing the supports for each reasoner answer which are defined in $\mathcal{EL}^+$ [77]. One architecture, which we call “Deep”, does not train with support data but has a hidden layer the same size as the supports we have defined. Another architecture, called “Piecewise”, trains two separate half-systems, one with knowledge bases and one with supports from the reasoner. The last system, called “Flat”, simply learns to map inputs directly to outputs for each reasoning step.

For the piecewise system, depicted in Figure 6 (a), we use two separate flat single LSTMs that have the number of neurons matching first the support shape and later the output shape. The deep system shown in Figure 6 (b) is constructed to match this shape so that the two will be comparable, it simply stacks the LSTMs together so that it can train without supports. We also train a flat system, shown in Figure 6 (c), that has the same input and output shape but lacks internal supports. We have done this three ways because we can compare the behavior of the systems with support against the flat system as a baseline and then see whether the piecewise training is helping. All designs treat the reasoning steps as the sequence to learn, so the number of LSTM cells is defined by the maximum reasoning sequence length. We train our LSTMs using regression on mean squared error to minimize the distances between predicted answers and correct answers.

For input data, we use two sources. The first is a synthetic data set obtained from a random $\mathcal{EL}^+$ normalized axiom generator. We also import data from the SNOMED 2012 ontology to ensure that our method is applicable to non-synthetic data. SNOMED is a widely-used, publicly available ontology of medical terms and relationships [78]. More information about these sources can be found in the original publication [77].

11.4.3.1. Results

We have attempted to create a system that emulates a reasoning task rather than reasoning output. Some examples are reproduced here to demonstrate the effect; a more in-depth look at the results can be found in the paper [77]. If we examine the example output from the synthetic data inputs in Table 5, it is clear that it is getting very close to many correct answers. When it misses, it still appears to be learning the shape, which makes us optimistic about its future potential. The SNOMED predictions are much more dense and do not fit well into a table, but we have included a few good examples in Table 6 with the original data labels translated into English sentences.

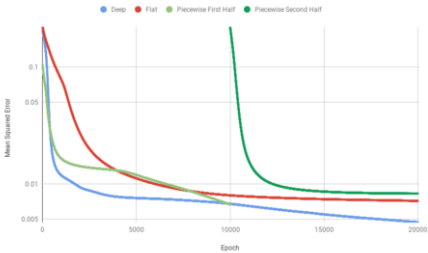
Table 5. Example Synthetic Output

	Correct Answer	Predicted Answer
Step 0	$C9 \sqsubseteq C11$	$C8 \sqsubseteq C9$
	$C2 \sqsubseteq C10$	$C1 \sqsubseteq C9$
	$C9 \sqsubseteq C12$	$C8 \sqsubseteq C9$
	$C7 \sqsubseteq C6$	
	$C9 \sqsubseteq \exists R4.C11$	$C8 \sqsubseteq \exists R4.C9$
	$C2 \sqsubseteq \exists R4.C9$	$C1 \sqsubseteq \exists R5.C9$
	$C9 \sqsubseteq \exists R5.C9$	$C8 \sqsubseteq \exists R4.C9$
	$C2 \sqsubseteq \exists R5.C11$	
	$C9 \sqsubseteq \exists R7.C12$	$C9 \sqsubseteq \exists R5.C9$
	$C2 \sqsubseteq \exists R6.C12$	
Step 1	$C9 \sqsubseteq C13$	$C8 \sqsubseteq C12$
	$C2 \sqsubseteq C11$	$C2 \sqsubseteq C10$
	$C2 \sqsubseteq C12$	$C1 \sqsubseteq C11$
	$C2 \sqsubseteq \exists R4.C11$	$C1 \sqsubseteq \exists R3.C12$
	$C2 \sqsubseteq \exists R5.C9$	$C1 \sqsubseteq \exists R4.C8$
	$C2 \sqsubseteq \exists R7.C12$	
Step 2	$C2 \sqsubseteq C13$	$C1 \sqsubseteq C12$

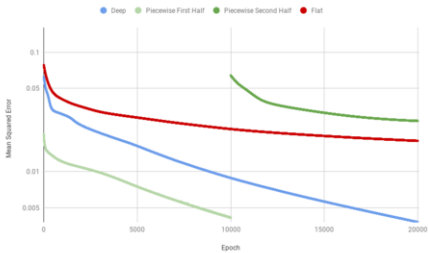
Table 6. Example SNOMED Outputs

Correct Answer	$C6 \sqsubseteq \exists R4.C1$
Meaning	if something is a zone of superficial fascia, then there is a subcutaneous tissue that it is PartOf
Prediction	$C6 \sqsubseteq \exists R4.C3$
Meaning	if something is a zone of superficial fascia, then there is a subcutaneous tissue of palmar area of little finger that it is PartOf

Correct Answer	$C8 \sqsubseteq \exists R3.C2$
Meaning	if something is an infrapubic region of pelvis, then there is a perineum that it is PartOf
Prediction	$C9 \sqsubseteq \exists R3.C2$
Meaning	if something is a zone of integument, then there is a perineum that it is PartOf



(a) Synthetic



(b) SNOMED

Figure 7. Dataset Training Comparison

Due to the piecewise network stopping half-way, we can see the higher training difficulty for reaching the completion versus reaching the supports in the SNOMED data in Figure 7 (a) and (b). The synthetic data is more uniform, so it is easier to train in both cases, as the figures indicate. This capability to inspect the middle is a clear indicator that at least limited explainability is possible when the structure of a process, rather than the best answers, are used to define the precise network shape.

11.4.3.2. Discussion

This experiment is a proof-of-concept that training a neural network to learn structured behavior is feasible and has advantages. To test this, the network was specifically not optimized in any way that might cause it to seem to work without actually learning what was intended. As a result, the precision and accuracy of the system are not impressive, they were not intended to be. However, it is important to note that by training to learn a structure, accuracy can also improve since it will more correctly emulate the deductive task. An improved system may be able to do both.

A beneficial side-effect of emphasizing structure is the ability of networks like the piecewise architecture to effectively provide a midpoint that could be used for debugging. And indeed this is intuitive for a neuro-symbolic system, since the deductive process that is emulated has precise answers and values at every stage of its execution. Why would we want to trust a neural network to find the appropriate weights and shapes for these intermediate steps when exact values for them can be determined? This intermediate step is not only beneficial from the standpoint of designing networks with debugging capabilities, but also may provide an important avenue for designing neuro-symbolic networks that are more explainable.

11.5. Benefits and Shortcomings of Neuro-Symbolic Reasoning

Symbolic knowledge representation and reasoning, and neural networks are fundamentally different techniques of Artificial Intelligence with complementary power. The former are interpretable and data-efficient, but they are sensitive to noise and cannot be applied to non-symbolic problems where the data is ambiguous. The latter can learn complex tasks from examples, are robust to noise, but are not interpretable, need huge amounts of—usually obtained with difficulty—data for training, are slow to learn, and susceptible to adversarial examples. Either paradigm excels at certain types of problems where the other paradigm performs poorly. Hence, to achieve more powerful AI systems, integrated neuro-symbolic systems that combine artificial neural networks and symbolic reasoning are getting traction.

One major asset of this neuro-symbolic integration is to benefit from the noise resistance for **noise-tolerant** reasoning, which could centrally contribute to **safety and trust**. For instance, whenever the reasoner is obliged to enter the noise-tolerant mode, provenance information on the noisy data could be inspected to identify the noise culprit and assign trust levels to resources and potentially even originators. On the downside, to train noise-tolerant neuro-symbolic reasoners, a proportion of the noisy data has to be tagged in order to be able to compare inferences from intact data against inferences from noisy data.

Neural approaches can potentially lend their zero-shot learning capabilities—the ability to transfer learned knowledge on domains, applications, languages, etc.—to reasoning in a neuro-symbolic transfer learning approach, as discussed by means of memory networks in this chapter. Thereby, learned reasoning on a set of theories can be transferred to another set of theories over the same logic. This integration can potentially provide insights into domain adaptation in the reasoning domain. A highly interesting direction to further explore **zero-shot reasoning** would be to allow to transfer learned knowledge from one logic to another, similar to Neural Machine Translation (NMT) models trained on specific language pairs but then capable of translating to any language in the set.

While these benefits are promising, there is still a need to achieve more transparency in neuro-symbolic integration. A first step in this direction is to aim at an intermediate inspection of the learned structured behavior, instead of only evaluating the reasoner answers. The approach presented in this chapter marks an important stepping stone towards heightened **explainability** of neural and neuro-symbolic approaches (see e.g. [79]), especially since it highlights the need to propose alternative approaches to utilizing pre-trained embeddings. With such embeddings, estimating the source of reasoning behavior becomes inevitably less traceable.

In general, neuro-symbolic reasoning approaches conventionally operate on comparatively small datasets. However, to truly consider neuro-symbolic applications on a Web-scale, such approaches would need to be scalable. To ensure **scalability** of approaches presented in this chapter and the related work, experimental setups would need to go beyond a proof of concept and include extensive testing at this point.

11.6. Conclusion

Since the current state-of-the-art neuro-symbolic reasoning and transfer learning, some of which are presented in detail in this chapter, is promising, it might be time to reach higher. Most recent approaches on neuro-symbolic integration have focused on benefiting from neural techniques for Semantic Web tasks and challenges. However, integration from symbolic to neural is equally important and holds the promise of contributing centrally to AI explainability (e.g. [80]). To this end, neuro-symbolic integration as a field would largely benefit from more and deep(er) theoretical contributions to a better understanding of the differences between symbolic and neural approaches (some approaches for explainable reasoning [79, 81] have been proposed). Regarding scalability, the discussion in neuro-symbolic reasoning goes beyond processing large amounts of data and addresses the ability to learn and interpret rules and transfer learned reasoning to other theories, domains, and potentially types of logic. With the overview of existing approaches as well as their benefits and shortcomings in this chapter, we hope to spark and intensify research in this field towards explainable, transferrable, and scalable neuro-symbolic reasoning.

References

- [1] Valiant LG. Knowledge Infusion: In Pursuit of Robustness in Artificial Intelligence. In: Hariharan R, Mukund M, Vinay V, editors. IARCS Annu. Conf. Foundations Software Technol. Theoretical Computer

- Science; Vol. 2; Dagstuhl, Germany. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik; 2008. p. 415–422. Available from: <http://drops.dagstuhl.de/opus/volltexte/2008/1770>.
- [2] Garcez Ad, Besold TR, De Raedt L, et al. Neural-symbolic learning and reasoning: Contributions and challenges. In: 2015 AAAI Spring Symposia, Stanford University, Palo Alto, California, USA, March 22-25, 2015. AAAI Press; 2015. Available from: <http://www.aaai.org/ocs/index.php/SSS/SSS15/paper/view/10281>.
 - [3] Hitzler P, Bianchi F, Ebrahimi M, et al. Neural-symbolic integration and the semantic web. *Semantic Web*. 2020;11(1):3–11.
 - [4] Garcez Ad, Gori M, Lamb LC, et al. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *CoRR*. 2019;abs/1905.06088. Available from: <http://arxiv.org/abs/1905.06088>.
 - [5] Makni B, Hendler J. Deep learning for noise-tolerant RDFS reasoning. *Semantic Web*. 2019;10(5):823–862. Available from: <https://doi.org/10.3233/SW-190363>.
 - [6] Hitzler P, Van Harmelen F. A reasonable Semantic Web. *Semantic Web*. 2010;1(1-2):39–44. Available from: <https://doi.org/10.3233/SW-2010-0010>.
 - [7] Hendler J. A little semantics goes a long way ; 2019. Available from: <https://www.cs.rpi.edu/~hendler/LittleSemanticsWeb.html>.
 - [8] Hayes PJ, Patel-Schneider PF. RDF 1.1 semantics. W3C; 2014. W3C recommendation 25 february 2014. Available from: <https://www.w3.org/TR/rdf11-mt/>.
 - [9] Brickley D, Guha R. RDF Schema 1.1. W3C; 2014. W3C recommendation 25 february 2014. Available from: <https://www.w3.org/TR/rdf-schema/>.
 - [10] Besold TR, Garcez Ad, Bader S, et al. Neural-symbolic learning and reasoning: A survey and interpretation. *CoRR*. 2017;abs/1711.03902. Available from: <http://arxiv.org/abs/1711.03902>.
 - [11] Kazakov Y, Krötzsch M, Simančík F. ELK: a reasoner for OWL EL ontologies. *System Description*. 2012;.
 - [12] Bhagat S, Cormode G, Muthukrishnan S. Node classification in social networks ; 2011. Available from: <http://arxiv.org/abs/1101.3291>.
 - [13] Liben-Nowell D, Kleinberg J. The link-prediction problem for social networks. *J Assoc Inform Sci Technology*. 2007 5;58(7):1019–1031. Available from: <http://dx.doi.org/10.1002/asi.v58:7>.
 - [14] Wang Q, Mao Z, Wang B, et al. Knowledge graph embedding: A survey of approaches and applications. *IEEE Trans Knowl Data Eng*. 2017;29(12):2724–2743. Available from: <https://doi.org/10.1109/TKDE.2017.2754499>.
 - [15] Ji S, Pan S, Cambria E, et al. A survey on knowledge graphs: Representation, acquisition and applications. *CoRR*. 2020;abs/2002.00388. Available from: <https://arxiv.org/abs/2002.00388>.
 - [16] Cai H, Zheng VW, Chang KC. A comprehensive survey of graph embedding: Problems, techniques and applications ; 2017. Available from: <http://arxiv.org/abs/1709.07604>.
 - [17] Bordes A, Weston J, Collobert R, et al. Learning structured embeddings of knowledge bases. In: Burgard W, Roth D, editors. *Proc. Twenty-Fifth AAAI Conf. Artificial Intelligence*. AAAI Press; 2011. p. 301–306. Available from: <http://www.aaai.org/ocs/index.php/AAAI/AAAI11/paper/view/3659>.
 - [18] Lösch U, Bloehdorn S, Rettinger A. Graph kernels for RDF data. In: Simperl I, Cimiano P, Polleres A, et al., editors. *Semantic Web: Res. Applications*; Berlin, Germany. Springer Berlin Heidelberg; 2012. p. 134–148. Available from: https://doi.org/10.1007/978-3-642-30284-8_16.
 - [19] de Vries GKD. A fast approximation of the Weisfeiler-Lehman graph kernel for RDF data. In: Blockeel H, Kersting K, Nijssen S, et al., editors. *Mach. Learning Knowledge Discovery in Databases*; Berlin, Germany. Springer Berlin Heidelberg; 2013. p. 606–621. Available from: https://doi.org/10.1007/978-3-642-40988-2_39.
 - [20] Shervashidze N, Schweitzer P, Van Leeuwen EJ, et al. Weisfeiler-Lehman graph kernels. *J Mach Learning Research*. 2011 11;12:2539–2561. Available from: <http://dl.acm.org/citation.cfm?id=1953048.2078187>.
 - [21] Ostuni VC, Di Noia T, Mirizzi R, et al. A linked data recommender system using a neighborhood-based graph kernel. In: Hepp M, Hoffner Y, editors. *E-Commerce Web Technologies*; New York, NY, USA. Springer International Publishing; 2014. p. 89–100. Available from: https://doi.org/10.1007/978-3-319-10491-1_10.
 - [22] Mikolov T, Sutskever I, Chen K, et al. Distributed Representations of Words and Phrases and Their Compositionality. In: *Proc. 26th Int. Conf. Neural Inform. Process. Syst. - Volume 2*; Red Hook, NY,

- USA. Curran Associates Inc.; 2013. p. 3111–3119. Available from: <http://dl.acm.org/citation.cfm?id=2999792.2999959>.
- [23] Ristoski P, Paulheim H. RDF2Vec: RDF Graph Embeddings for Data Mining. In: Groth PT, Simperl E, Gray AJG, et al., editors. Semantic Web - ISWC 2016 - 15th Int. Semantic Web Conference, Kobe, Japan, October 17-21, 2016, Proceedings, Part I; Vol. 9981; 2016. p. 498–514. Available from: https://doi.org/10.1007/978-3-319-46523-4_30.
- [24] Cochez M, Ristoski P, Ponzetto SP, et al. Biased graph walks for RDF graph embeddings. In: Akerkar R, Cuzzocrea A, Cao J, et al., editors. Proc. 7th Int. Conf. Web Intelligence, Mining Semantics; New York, NY, USA. ACM; 2017. p. 21:1–21:12. Available from: <http://doi.acm.org/10.1145/3102254.3102279>.
- [25] Cochez M, Ristoski P, Ponzetto SP, et al. Global RDF Vector Space Embeddings. In: d'Amato C, Fernández M, Tamma VAM, et al., editors. Semantic Web - ISWC 2017 - 16th Int. Semantic Web Conference, Part I; Vol. 10587; New York, NY, USA. Springer; 2017. p. 190–207. Available from: https://doi.org/10.1007/978-3-319-68288-4_12.
- [26] Pennington J, Socher R, Manning CD. Glove: Global Vectors for Word Representation. In: Moschitti A, Pang B, Daelemans W, editors. Proc. 2014 Conf. Empirical Methods in Natural Language Process. (EMNLP); 10; Doha, Qatar. Association for Computational Linguistics; 2014. p. 1532–1543. Available from: <http://www.aclweb.org/anthology/D14-1162>.
- [27] Holter OM, Myklebust EB, Chen J, et al. Embedding OWL ontologies with OWL2Vec. In: CEUR Workshop Proceedings; Vol. 2456; Technical University of Aachen; 2019. p. 33–36.
- [28] Chen J, Hu P, Jiménez-Ruiz E, et al. Owl2vec*: Embedding of OWL ontologies. CoRR. 2020; abs/2009.14654. Available from: <https://arxiv.org/abs/2009.14654>.
- [29] Paulheim H, Bizer C. Type Inference on Noisy RDF Data. In: Alani H, Kagal L, Fokoue A, et al., editors. Semantic Web – ISWC 2013; Berlin, Germany. Springer Berlin Heidelberg; 2013. p. 510–525.
- [30] Paulheim H, Bizer C. Improving the quality of linked data using statistical distributions. Int J Semantic Web Inform Syst. 2014;10(2):63–86. Available from: <https://doi.org/10.4018/ijswis.2014040104>.
- [31] Nuzzolese AG, Gangemi A, Presutti V, et al. Type inference through the analysis of wikipedia links. In: Bizer C, Heath T, Berners-Lee T, et al., editors. WWW2012 Workshop Linked Data Web; Vol. 937; Lyon, France. CEUR-WS.org; 2012. p. 1–9. Available from: <http://ceur-ws.org/Vol-937/ldow2012-paper-13.pdf>.
- [32] Nuzzolese AG, Gangemi A, Presutti V, et al. Encyclopedic knowledge patterns from wikipedia links. In: Aroyo L, Welty C, Alani H, et al., editors. Semantic Web – ISWC 2011; Vol. 7031; Berlin, Germany. Springer Berlin Heidelberg; 2011. p. 520–536. Available from: https://doi.org/10.1007/978-3-642-25073-6_33.
- [33] Paulheim H, Gangemi A. Serving dbpedia with DOLCE - more than just adding a cherry on top. In: Arenas M, Corcho Ó, Simperl E, et al., editors. The Semantic Web - ISWC 2015 - 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part I; (Lecture Notes in Computer Science; Vol. 9366). Springer; 2015. p. 180–196. Available from: https://doi.org/10.1007/978-3-319-25007-6_11.
- [34] Gangemi A, Guarino N, Masolo C, et al. Sweetening ontologies with DOLCE. In: Gómez-Pérez A, Benjamins VR, editors. Knowledge Engineering and Knowledge Management. Ontologies and the Semantic Web, 13th International Conference, EKAW 2002, Sigüenza, Spain, October 1-4, 2002, Proceedings; (Lecture Notes in Computer Science; Vol. 2473). Springer; 2002. p. 166–181. Available from: https://doi.org/10.1007/3-540-45810-7_18.
- [35] Paulheim H, Stuckenschmidt H. Fast Approximate A-Box Consistency Checking Using Machine Learning. In: Sack H, Blomqvist E, d'Aquin M, et al., editors. Semantic Web. Latest Advances New Domains; New York, NY, USA. Springer International Publishing; 2016. p. 135–150.
- [36] Lösch U, Bloehdorn S, Rettinger A. Graph kernels for RDF data. In: Simperl E, Cimiano P, Polleres A, et al., editors. The Semantic Web: Research and Applications - 9th Extended Semantic Web Conference, ESWC 2012, Heraklion, Crete, Greece, May 27-31, 2012. Proceedings; (Lecture Notes in Computer Science; Vol. 7295). Springer; 2012. p. 134–148. Available from: https://doi.org/10.1007/978-3-642-30284-8_16.
- [37] Meilicke C, Ruffinelli D, Nolle A, et al. Fast ABox Consistency Checking Using Incomplete Reasoning and Caching. In: Costantini S, Franconi E, Woensel WV, et al., editors. Rules and Reasoning - International Joint Conference, RuleML+RR 2017, London, UK, July 12-15, 2017, Proceed-

- ings; (Lecture Notes in Computer Science; Vol. 10364). Springer; 2017. p. 168–183. Available from: https://doi.org/10.1007/978-3-319-61252-2_12.
- [38] Lembo D, Lenzerini M, Rosati R, et al. Query Rewriting for Inconsistent DL-Lite Ontologies. In: Rudolph S, Gutiérrez C, editors. Web Reasoning and Rule Systems - 5th International Conference, RR 2011, Galway, Ireland, August 29-30, 2011. Proceedings; (Lecture Notes in Computer Science; Vol. 6902). Springer; 2011. p. 155–169. Available from: https://doi.org/10.1007/978-3-642-23580-1_12.
- [39] Calvanese D, De Giacomo G, Lembo D, et al. Tractable reasoning and efficient query answering in description logics: The *DL-Lite* family. *J Autom Reasoning*. 2007;39(3):385–429. Available from: <https://doi.org/10.1007/s10817-007-9078-x>.
- [40] McCulloch WS, Pitts W. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*. 1943;5(4):115–133.
- [41] Hohenecker P, Lukasiewicz T. Deep learning for ontology reasoning; 2017. Available from: <http://arxiv.org/abs/1705.10342>.
- [42] Socher R, Perelygin A, Wu J, et al. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In: Proc. 2013 Conf. Empirical Methods in Natural Language Processing; 10; Seattle, WA, USA. Association for Computational Linguistics; 2013. p. 1631–1642. Available from: <http://www.aclweb.org/anthology/D13-1170>.
- [43] Hohenecker P, Lukasiewicz T. Ontology reasoning with deep neural networks. *J Artif Intell Res*. 2020; 68:503–540. Available from: <https://doi.org/10.1613/jair.1.11661>.
- [44] Hitzler P, Krötzsch M, Rudolph S. Foundations of Semantic Web Technologies. Chapman and Hall/CRC Press; 2010. Available from: <http://www.semantic-web-book.org/>.
- [45] Motik B, Grau BC, Horrocks I, et al. OWL 2 Web ontology language profiles. W3C; 2009. W3C recommendation 11 december 2012. Available from: <https://www.w3.org/TR/2009/REC-owl2-profiles-20091027/>.
- [46] Bordes A, Usunier N, Garcia-Duran A, et al. Translating embeddings for modeling multi-relational data. In: Advances in neural information processing systems; 2013. p. 2787–2795.
- [47] Kulmanov M, Liu-Wei W, Yan Y, et al. EL Embeddings: Geometric construction of models for the Description Logic EL++. *arXiv preprint arXiv:1902.10499*. 2019;.
- [48] Ferreira R, Lopes C, Gonçalves R, et al. Deep neural networks for approximating stream reasoning with C-SPARQL. *CoRR*. 2021;abs/2106.08452. Available from: <https://arxiv.org/abs/2106.08452>.
- [49] Barbieri DF, Braga D, Ceri S, et al. C-SPARQL: a continuous query language for RDF data streams. *International Journal of Semantic Computing*. 2010;4(01):3–25.
- [50] Socher R, Chen D, Manning CD, et al. Reasoning with neural tensor networks for knowledge base completion. In: Burges CJC, Bottou L, Ghahramani Z, et al., editors. Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States.; 2013. p. 926–934. Available from: <http://papers.nips.cc/paper/5028-reasoning-with-neural-tensor-networks-for-knowledge-base-completion>.
- [51] Lin Y, Liu Z, Sun M, et al. Learning entity and relation embeddings for knowledge graph completion. In: Bonet B, Koenig S, editors. Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA. AAAI Press; 2015. p. 2181–2187. Available from: <http://www.aaai.org/ocs/index.php/AAAI/AAAI15/paper/view/9571>.
- [52] Trouillon T, Dance CR, Gaussier É, et al. Knowledge graph completion via complex tensor factorization. *Journal of Machine Learning Research (JMLR)*. 2017;18(130):1–38.
- [53] Ji G, Liu K, He S, et al. Knowledge graph completion with adaptive sparse transfer matrix. In: Schuurmans D, Wellman MP, editors. Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA. AAAI Press; 2016. p. 985–991. Available from: <http://www.aaai.org/ocs/index.php/AAAI/AAAI16/paper/view/11982>.
- [54] Serafini L, Garcez Ad. Logic tensor networks: Deep learning and logical reasoning from data and knowledge. In: Besold TR, Lamb LC, Serafini L, et al., editors. Proceedings of the 11th International Workshop on Neural-Symbolic Learning and Reasoning (NeSy'16) co-located with the Joint Multi-Conference on Human-Level Artificial Intelligence (HLAI 2016), New York City, NY, USA, July 16-17, 2016.; (CEUR Workshop Proceedings; Vol. 1768). CEUR-WS.org; 2016. Available from: http://ceur-ws.org/Vol-1768/NESY16_paper3.pdf.
- [55] Rocktäschel T, Riedel S. End-to-end differentiable proving. In: Guyon I, von Luxburg U, Bengio S, et al.,

- editors. *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, 4-9 December 2017, Long Beach, CA, USA; 2017. p. 3791–3803. Available from: <http://papers.nips.cc/paper/6969-end-to-end-differentiable-proving>.
- [56] Wang PW, Stepanova D, Domokos C, et al. Differentiable learning of numerical rules in knowledge graphs. In: *International Conference on Learning Representations*; 2020.
- [57] Yang Y, Song L. Learn to explain efficiently via neural logic inductive learning. *CoRR*. 2019; abs/1910.02481. Available from: <http://arxiv.org/abs/1910.02481>.
- [58] Zhu M, Gao Z, Quan Z. Noisy type assertion detection in semantic datasets. In: Mika P, Tudorache T, Bernstein A, et al., editors. *The Semantic Web - ISWC 2014 - 13th International Semantic Web Conference*, Riva del Garda, Italy, October 19-23, 2014. *Proceedings, Part I*; (Lecture Notes in Computer Science; Vol. 8796). Springer; 2014. p. 373–388. Available from: https://doi.org/10.1007/978-3-319-11964-9_24.
- [59] Fleischhacker D, Paulheim H, Bryl V, et al. Detecting errors in numerical linked data using cross-checked outlier detection. In: Mika P, Tudorache T, Bernstein A, et al., editors. *Semantic Web – ISWC 2014*; New York, NY, USA. Springer International Publishing; 2014. p. 357–372. Available from: https://doi.org/10.1007/978-3-319-11964-9_23.
- [60] Wienand D, Paulheim H. Detecting Incorrect Numerical Data in DBpedia. In: Presutti V, d'Amato C, Gandon F, et al., editors. *Semantic Web: Trends Challenges*; New York, NY, USA. Springer International Publishing; 2014. p. 504–518. Available from: https://doi.org/10.1007/978-3-319-07443-6_34.
- [61] Parzen E. On estimation of a probability density function and mode. *Ann Math Statistics*. 1962; 33(3):1065–1076. Available from: <http://www.jstor.org/stable/2237880>.
- [62] Zhao Y, Li Z, Deng W, et al. Learning entity type structured embeddings with trustworthiness on noisy knowledge graphs. *Knowledge-Based Systems*. 2021;215:106630.
- [63] Cyganiak R, Wood D, Lanthaler M. RDF 1.1 concepts and abstract syntax. *W3C*; 2014. W3C. Available from: <https://www.w3.org/TR/rdf11-concepts/>.
- [64] Hayes J, Gutiérrez C. Bipartite graphs as intermediate model for RDF. In: McIlraith SA, Plexousakis D, van Harmelen F, editors. *Semantic Web - ISWC 2004: Third Int. Semantic Web Conference*; Vol. 3298; Berlin, Germany. Springer; 2004. p. 47–61. Available from: https://doi.org/10.1007/978-3-540-30475-3_5.
- [65] Morales AAM, Serodio MEV. A directed hypergraph model for RDF. In: Simperl EPB, Diederich J, Schreiber G, editors. *Proc. KWEPSY 2007 Knowledge Web PhD Symp. 2007*; Vol. 275; 6; Innsbruck, Austria. CEUR-WS.org; 2007. p. 1–2. Available from: <http://ceur-ws.org/Vol-275/paper24.pdf>.
- [66] Liu H, Dou D, Jin R, et al. Mining biomedical ontologies and data using RDF hypergraphs. In: *12th Int. Conf. Mach. Learning Applications, ICMLA 2013*; 12; Miami, FL, USA. IEEE; 2013. p. 141–146. Available from: <https://doi.org/10.1109/ICMLA.2013.31>.
- [67] Wu G, Li J, Hu J, et al. System π : A native RDF repository based on the hypergraph representation for RDF data model. *J Computer Sci Technology*. 2009;24(4):652–664. Available from: <https://doi.org/10.1007/s11390-009-9265-9>.
- [68] Chernenkiy V, Gapanyuk Y, Nardid A, et al. Using the metagraph approach for addressing RDF knowledge representation limitations. In: *2017 Internet Technologies Appl. (ITA)*; 9; 2017. p. 47–52.
- [69] Makni B, Hendler J. Deep learning for noise-tolerant RDFS reasoning. *Semantic Web*. 2019;10(5):823–862. Available from: <https://doi.org/10.3233/SW-190363>.
- [70] Unger C, Bühmann L, Lehmann J, et al. Template-based question answering over RDF data. In: *WWW. ACM*; 2012. p. 639–648.
- [71] Zou L, Huang R, Wang H, et al. Natural language question answering over RDF: a graph data driven approach. In: *SIGMOD Conference. ACM*; 2014. p. 313–324.
- [72] Unger C, Forascu C, López V, et al. Question answering over linked data (QALD-4). In: *CLEF (Working Notes)*; (CEUR Workshop Proceedings; Vol. 1180). CEUR-WS.org; 2014. p. 1172–1180.
- [73] Weston J, Chopra S, Bordes A. Memory networks. In: Bengio Y, LeCun Y, editors. *3rd International Conference on Learning Representations, ICLR 2015*, San Diego, CA, USA, May 7-9, 2015, *Conference Track Proceedings*; 2015. Available from: <http://arxiv.org/abs/1410.3916>.
- [74] Sukhbaatar S, Weston J, Fergus R, et al. End-to-end memory networks. In: *Advances in neural information processing systems*; 2015. p. 2440–2448.
- [75] Sam S, Hitzler P, Janowicz K. On the quality of vocabularies for linked dataset papers published in the

- Semantic Web journal. Semantic Web. 2018;9(2):207–220.
- [76] Hochreiter S, Schmidhuber J. LSTM can solve hard long time lag problems. In: Advances in neural information processing systems; 1997. p. 473–479.
 - [77] Eberhart A, Ebrahimi M, Zhou L, et al. Completion reasoning emulation for the description logic EL+. In: Martin A, Hinkelmann K, Fill H, et al., editors. Proceedings of the AAAI 2020 Spring Symposium on Combining Machine Learning and Knowledge Engineering in Practice, AAAI-MAKE 2020, Palo Alto, CA, USA, March 23–25, 2020, Volume I; (CEUR Workshop Proceedings; Vol. 2600). CEUR-WS.org; 2020. Available from: <http://ceur-ws.org/Vol-2600/paper5.pdf>.
 - [78] De Silva TS, MacDonald D, Paterson G, et al. Systematized nomenclature of medicine clinical terms (snomed ct) to represent computed tomography procedures. Comput Methods Prog Biomed. 2011 Mar; 101(3):324–329. Available from: <http://dx.doi.org/10.1016/j.cmpb.2011.01.002>.
 - [79] Makni B, Abdelaziz I, Hendler J. Explainable deep RDFS reasoner. In: Ninth International Workshop on Statistical Relational AI; 2020.
 - [80] Sarker MK, Xie N, Doran D, et al. Explaining Trained Neural Networks with Semantic Web Technologies: First Steps. In: Besold TR, d’Avila Garcez AS, Noble I, editors. Proc. Twelfth Int. Workshop Neural-Symbolic Learning Reasoning; Vol. 2003; London, UK. CEUR-WS.org; 2017. p. 1–10. Available from: http://ceur-ws.org/Vol-2003/NeSy17_paper4.pdf.
 - [81] Bhowmik R, de Melo G. A joint framework for inductive representation learning and explainable reasoning in knowledge graphs. CoRR. 2020;abs/2005.00637. Available from: <https://arxiv.org/abs/2005.00637>.

Chapter 12

Learning Reasoning Strategies in End-to-End Differentiable Proving

Pasquale Minervini, University College London

Sebastian Riedel, University College London, Facebook AI Research

Pontus Stenetorp, University College London

Edward Grefenstette, University College London, Facebook AI Research

Tim Rocktäschel, University College London, Facebook AI Research

Abstract Attempts to render deep learning models interpretable, data-efficient, and robust have seen some success through hybridisation with rule-based systems, for example, in Neural Theorem Provers (NTPs). These neuro-symbolic models can induce interpretable rules and learn representations from data via back-propagation, while providing logical explanations for their predictions. However, they are restricted by their computational complexity, as they need to consider all possible proof paths for explaining a goal, thus rendering them unfit for large-scale applications. We present Conditional Theorem Provers (CTPs), an extension to NTPs that learns an optimal rule selection strategy via gradient-based optimisation. We show that CTPs are scalable and yield state-of-the-art results on the CLUTRR dataset, which tests *systematic generalisation* of neural models by learning to reason over smaller graphs and evaluating on larger ones. Finally, CTPs show better link prediction results on standard benchmarks in comparison with other neural-symbolic models, while being explainable. All source code and datasets are available online.¹

12.1. Introduction

Neural Natural Language Understanding (NLU) systems—wherein a deep neural network is used as a function approximator [1, 2]—have been extremely successful at various natural language tasks, such as Question Answering (QA) and Natural Language Inference (NLI) [3], achieving strong generalisation results on datasets available for these tasks [4,

¹At <https://github.com/uclnlp/ctp>

5, 6, 7]. Even strong performance on NLU problems have been recently achieved with advent of large models pre-trained via self-supervision, such as BERT [8], XLNet [9], and RoBERTa [10].

Generalisation in Neural Models However, there are growing concerns about the ability of NLU systems, and neural networks more generally, to generalise in a systematic and robust way [11, 12, 13, 14]. For instance, [15] highlight the brittleness of NLU systems to adversarial examples, while [16, 17] show that neural NLU models tend to exploit annotation artefacts and spurious correlations in the data. Furthermore, analysing and supervising the inner workings of such models is not trivial, due to their inherent black-box nature [18, 19].

More generally, [20] emphasise several limitations of neural models, in terms of *i*) data inefficiency and high sample complexity—the need of high volumes of training data in order to be effective, *ii*) poor generalisation—modern neural models may not produce the correct predictions when exposed to data outside the training distribution, and *iii*) lack of interpretability—such models are *black boxes* where internal representations and computations are hardly interpretable by humans.

In this vein, [14] measured and compared the systematic generalisation abilities of several neural models (including very strong baselines such as BERT [8] and Graph Attention Networks (GATs) [21]) on the task of answering questions about family relationship graphs, by evaluating on held-out combinations of reasoning patterns and by adding curated distracting noisy facts. Interestingly, they found that performance degrades monotonically for every model in their pool as they increase the complexity of the relational graph, highlighting the challenge of systematic generalisation [12, 22].

Neuro-Symbolic Reasoning A promising direction for overcoming these issues consists in combining *neural models* and *symbolic reasoning* given their complementary strengths and weaknesses [23, 24, 20]. We focus on NTPs [25], a family of neuro-symbolic reasoning models: NTPs are continuous relaxations of the backward-chaining reasoning algorithm that replace discrete symbols with their continuous embedding representations.

NTPs have interesting properties: they can jointly learn representations and interpretable rules from data via backpropagation, and can potentially combine such rules in ways that may have not been observed during training. However, a major limitation in NTPs is that, during training, they need to consider *all rules* for explaining a given goal or sub-goal. This quickly renders them ineffective in settings requiring a large number of rules or reasoning steps.

Conditional Theorem Provers For addressing limitations of NTPs, we propose CTPs, an extension that is able to learn an adaptive strategy for selecting subsets of rules to consider at each step of the reasoning process. This is achieved by a `select` module that, given a goal, produce the rules needed for proving it. Predicates and constants in the produced rules lie in a continuous embedding space. Hence, the `select` module is end-to-end differentiable, and can be trained jointly with the other modules via gradient-based optimisation.

12.2. End-to-End Differentiable Proving

NTPs [25] are a continuous relaxation of the *backward chaining algorithm* [26]: this algorithm works backward from the goal, chaining through rules to find known facts supporting the proof.

Algorithm 1 Overview of the neural backward chaining algorithm proposed by [25] – intuitively, it recursively proves each goal with all rules in the KB (OR module) and, for each rule, it proves its premise (AND module), up to d recursion steps.

<pre> 1: function or(G, d, S) 2: for $H :- \mathbb{B} \in \mathcal{H}$ do 3: for $S \in \text{and}(\mathbb{B}, d, \text{unify}(H, G, S))$ do 4: yield S 1: function and($\mathbb{B}, d, S$) 2: if $\mathbb{B} = []$ or $d = 0$ then yield S else 3: for $S' \in \text{or}(\text{sub}(\mathbb{B}_0, S_\psi), d-1, S)$ do 4: for $S'' \in \text{and}(\mathbb{B}_1, d, S')$ do 5: yield S'' </pre>	<pre> 1: function unify($H, G, S = (S_\psi, S_\rho)$) 2: $S'_\psi = S_\psi \cup_i T_i$ with $T_i = \begin{cases} \{H_i/G_i\} & \text{if } H_i \in \mathcal{V} \\ \{G_i/H_i\} & \text{if } G_i \in \mathcal{V}, H_i \notin \mathcal{V} \\ \emptyset & \text{otherwise} \end{cases}$ 3: $S'_\rho = \min \{S_\rho\} \cup_{H_i, G_i \notin \mathcal{V}} \{K(\theta_{H_i}, \theta_{G_i})\}$ 4: return (S'_ψ, S'_ρ) </pre>
--	--

Given a *query* (or *goal*) G , backward chaining first attempts to unify it with the facts available in a given Knowledge Base (KB). If no matching fact is available, it considers all rules $H :- B$, where H denotes the *head* (or *consequence*) and B the *body* (or *premise*), and H can be unified with the query G resulting in a substitution for the variables contained in H . Then, the backward chaining algorithm applies the substitution to the body B , and recursively attempts to prove the atoms contained therein.

Backward chaining can be seen as a type of *and/or* search: *or* because the goal can be proven by any rule in the KB, and *and* because all the conjuncts in the premise of a rule must be proven.

Example 1 (Backward Chaining). Consider a KB with facts $p(\text{RICK}, \text{BETH})$ and $p(\text{BETH}, \text{MORTY})$, and rule $g(X, Y) :- p(X, Z), p(Z, Y)$, where p and g denote the relationships *parent* and *grandparent*, respectively. The goal $G = g(\text{RICK}, \text{MORTY})$ can be proven by unifying G with the head of the rule $g(X, Y)$, with the substitution $\{X/\text{RICK}, Y/\text{MORTY}\}$, and then by recursively proving the subgoals

$$p(\text{RICK}, Z), p(Z, \text{MORTY}),$$

which hold true for the substitution $\{Z/\text{BETH}\}$. $\triangle$

NTPs make this reasoning process more flexible and end-to-end differentiable by replacing the comparison between symbols with a soft matching of their respective trainable dense vector representations. NTPs recursively build a neural network enumerating all possible proof paths for proving a query given KB, and aggregate all proof scores via max pooling. They do so by relying on three modules: a *unification module*, which compares sub-symbolic representations of logic atoms, and mutually recursive *or* and *and* modules, which jointly enumerate all possible proof paths, before the final aggregation selects the highest scoring one. The whole process is outlined in Algorithm 1.

Example 2 (Neural Theorem Provers). Consider a variant of Example 1, where each predicate and constant lives in a continuous embedding space, i.e. $\theta_p, \theta_q, \theta_{\text{RICK}}, \theta_{\text{BETH}}, \theta_{\text{MORTY}} \in \mathbb{R}^k$. [25] propose replacing comparisons between symbols with a differentiable similarity measure $K : \mathbb{R}^k \times \mathbb{R}^k \rightarrow [0, 1]$, such as a Gaussian kernel, between their embeddings. Their model enumerates all possible proofs for a goal G , and generates a *proof score* for each of them, given by the *minimum* of all embedding similarities. For

Algorithm 2 In Conditional Theorem Provers, the set of rules is conditioned on the goal G .

```

1: function or( $G, d, S$ )
2:   for  $H := \mathbb{B} \in \text{select}_{\theta}(G)$  do
3:     for  $S \in \text{and}(\mathbb{B}, d, \text{unify}(H, G, S))$  do
4:       yield  $S$ 

```

instance, if $G = \text{grandPa}(\text{RICK}, \text{MORTY}) = [\text{grandPa}, \text{RICK}, \text{MORTY}]$, one candidate proof consists in using the facts

$$F = p(\text{RICK}, \text{BETH}) \text{ and } F' = p(\text{BETH}, \text{MORTY})$$

and the rule

$$g(X, Y) :- p(X, Z), p(Z, Y)$$

from the KB, yielding the score $K(\theta_{\text{grandPa}}, \theta_g)$. It is important to mention that NTPs allow unifying symbols like grandPa and g (which, in this case, share the same semantics), even though they are lexically different. The score for G is given by the *maximum* of all proof scores. $\triangle$

12.3. Conditional Proving Strategies

The NTPs proposed by [25] use a fixed set of rules, either specified by the user or learned from data via provided rule templates. In this model, given a goal, there is no *hard* decision mechanism for deciding which rules can be used for reformulating a given goal into subgoals: all rules in the KB need to be considered when proving each goal. For this reason, NTPs were shown not to scale to large datasets [25].

Differentiable Goal Reformulation In order to explicitly *learn* which rule to consider at each step, we propose the following solution. Rather than relying on a fixed, potentially very large set of rules, we propose to dynamically *generate* a minimal set of rules via a neural network architecture conditioned on the goal to prove.

Example 3 (Conditional Theorem Proving). Assume that the goal to prove is $G = g(\text{RICK}, \text{MORTY})$, we want the model to be able to only consider the best rules for proving G , such as $g(X, Y) :- p(X, Z), p(Z, Y)$, rather than all rules in the KB. Remember that, in NTPs, relations and constants in the KB are represented by their embedding vectors, and the aforementioned rule selection can be implemented via a mapping from θ_g to $[\theta_p, \theta_p]$. $\triangle$

Consider the `or` module in Algorithm 1. Selecting which rule to use during the proving process for a given goal G can be implemented by rewriting the `or` module as in Algorithm 2, where the set of clauses is produced by a module that, given the goal, generates a set of rules for proving G .

The core difference between NTPs and CTPs is that, when proving a goal G , rather than iterating through a possibly very large set of clauses in the KB $\mathcal{K}$ (see Line 2 in the

or module definition in Algorithm 1), the conditional or module in Algorithm 2 only iterates through a small set of generated clauses, whose generation is conditioned on G (see Line 2 in Algorithm 2). Given a goal G , the select module with parameters θ in Algorithm 2 produces a set of clauses, each specifying which sub-goals to prove in order to produce a proof for G .

Note that the select module can be implemented by an end-to-end differentiable parametrised function $f(\cdot)$ that, given a goal G , produces a finite sequence of corresponding subgoals:

$$\text{select}_{\theta}(G) : \mathcal{A} \rightarrow [\mathcal{A} :- \mathcal{A}^*], \quad (1)$$

where $\mathcal{V}$ is a set of variables, and $\mathcal{A} \triangleq \mathbb{R}^k \times (\mathbb{R}^k \cup \mathcal{V}) \times (\mathbb{R}^k \cup \mathcal{V})$ denotes the embedding representation of a goal, such as $g(\text{RICK}, \text{MORTY})$.

For instance, the select module in Eq. 1 can be implemented by a neural network that, given a goal such as $G = [\theta_{\mathcal{G}}, \theta_{\text{RICK}}, \theta_{\text{MORTY}}]$, generates $H :- B$ with $H = [\theta_{\mathcal{G}}, X, Y]$ and $B = [[\theta_p, X, Z], [\theta_p, Z, Y]]$, corresponding to the symbolic rule $g(X, Y) :- p(X, Z), p(Z, Y)$. If the positions of the variables in the rule are fixed, the whole module is end-to-end differentiable with respect to its parameters θ .

Neural Goal Reformulation Here, we define select as a linear function of the goal predicate:

$$\text{select}_{\theta}(G) \triangleq [F_H(G) :- F_{B_1}(G), F_{B_2}(G)], \quad (2)$$

where the head and body of the resulting rule are given by $F_H(G) = [f_H(\theta_{G_1}), X, Y]$, $F_{B_1}(G) = [f_{B_1}(\theta_{G_1}), X, Z]$, and $F_{B_2}(G) = [f_{B_2}(\theta_{G_1}), Z, Y]$. Every $f_i : \mathbb{R}^k \rightarrow \mathbb{R}^k$ is a differentiable function, such as the linear projection $f_i(\mathbf{x}) = \mathbf{W}_i \mathbf{x} + \mathbf{b}$, with $\mathbf{W}_i \in \mathbb{R}^{k \times k}$ and $\mathbf{b} \in \mathbb{R}^k$. Thus, instead of iterating through a possibly very large set of rules in the KB $\mathcal{K}$, we can generate a significantly smaller set of rules, whose generation is conditioned on the goal G and can be trained end-to-end on downstream reasoning tasks.

Attentive Goal Reformulation We can incorporate a useful prior in the select module architecture—namely that predicate symbols in the rule already exist in the KB, among the available relations $\mathcal{R}$. A method for incorporating this prior consists in using the given goal G for generating a distribution over the set of relations $\mathcal{R}$:

$$f_i(\mathbf{x}) = \alpha \mathbf{E}_{\mathcal{R}} \quad \text{where} \quad \alpha = \text{softmax}(\mathbf{W}_i \mathbf{x}) \in \Delta^{|\mathcal{R}|-1}, \quad (3)$$

where $\mathbf{E}_{\mathcal{R}} \in \mathbb{R}^{|\mathcal{R}| \times k}$ denotes the predicate embedding matrix, $\mathbf{W}_i \in \mathbb{R}^{k \times |\mathcal{R}|}$, and α is an attention distribution $\alpha \in \Delta^{|\mathcal{R}|-1}$ over the predicates in $\mathcal{R}$, where Δ^n denotes the standard n -simplex.² This is especially helpful if the embedding size is larger than the number of relationships, i.e. $k \gg |\mathcal{R}|$.

Memory-Based Goal Reformulation A problem with using black-box neural networks for reformulating goals into sub-goals is that it can be difficult to inspect the rules by analysing the model parameters, as in [25]. For this reason, we propose an alternative goal reformulation module, where rules are stored in a differentiable memory.

More precisely, n rules are stored as memory matrices $[\mathbf{M}_1, \dots, \mathbf{M}_m]$, where each $\mathbf{M}_i \in \mathbb{R}^{n \times k}$ denotes the k -dimensional embeddings of the i -th predicates in the n rules.

²The standard n -simplex Δ^n is defined as $\Delta^n = \{(\alpha_0, \dots, \alpha_n) \in \mathbb{R}^{n+1} \mid \sum_{i=0}^n \alpha_i = 1 \wedge \forall i : \alpha_i \geq 0\}$

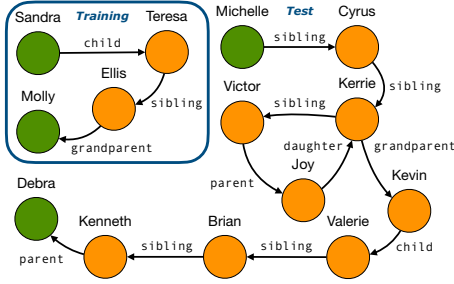


Figure 1. Example of a train and test instance in CLUTRR – the training instance (upper left) is composed by a graph with three edges, while the test instance is composed by ten edges; the task consists in identifying the relationships between the green nodes.

CLUTRR – Sample of Learned Rules	
$\text{child}(X, Y) \Leftarrow \text{child}(X, Z), \text{sibling}(Z, Y)$	
$\text{child}(X, Y) \Leftarrow \text{SO}(X, Z), \text{child}(Z, Y)$	
$\text{grand}(X, Y) \Leftarrow \text{child}(X, Z), \text{child}(Z, Y)$	
$\text{grand}(X, Y) \Leftarrow \text{grand}(X, Z), \text{sibling}(Z, Y)$	
$\text{grand}(X, Y) \Leftarrow \text{SO}(X, Z), \text{grand}(Z, Y)$	
$\text{in-law}(X, Y) \Leftarrow \text{child}(X, Z), \text{SO}(Z, Y)$	
$\text{in-law}(X, Y) \Leftarrow \text{sibling-in-law}(X, Z), \text{child}(Z, Y)$	
$\text{sibling}(X, Y) \Leftarrow \text{sibling}(X, Z), \text{sibling}(Z, Y)$	
$\text{sibling}(X, Y) \Leftarrow \text{child}(X, Z), \text{uncle}(Y, Z)$	
$\text{sibling}(X, Y) \Leftarrow \text{child}(X, Z), \text{child}(Y, Z)$	

Figure 2. Rules learned on Compositional Language Understanding and Text-based Relational Reasoning (CLUTRR) by CTPs – symbols were obtained by decoding the goal reformulations with the nearest predicate in embedding space.

Then, the goal G is used to compute an attention distribution over rules $\alpha \in \Delta^{n-1}$, where each α_i denotes the attention weight on the i -th rule. The attention distribution can be formalised as follows:

$$f_i(\mathbf{x}) = \alpha \mathbf{M}_i \quad \text{where} \quad \alpha = \text{softmax}(\mathbf{W}\mathbf{x}) \in \Delta^{n-1}, \quad (4)$$

where each $f_i: \mathbb{R}^k \rightarrow \mathbb{R}^k$ is a differentiable function that, given the goal, produces an attention distribution $\alpha \in \Delta^{n-1}$ over the rules and for indexing a memory $\mathbf{M}_i$, analogous to a key-value memory network [27].

12.4. Related Work

Memory-Augmented Networks Memory-augmented neural architectures aim at improving the generalisation and reasoning abilities in neural networks by disentangling representations from computations. By enriching neural networks with a differentiable *external memory*, these models were able to multi-hop reason over texts [28], induce algorithmic behaviours [29, 30, 31, 32], and rapidly assimilate new data [33].

Neuro-Symbolic Models *Differentiable interpreters* enable translating declarative or procedural knowledge into neural networks exhibiting strong inductive biases of that knowledge [34, 25, 24]. [34] propose $\partial 4$, a differentiable abstract machine for the Forth programming language.

[25] propose a differentiable implementation for the backward chaining algorithm, effectively implementing a differentiable Datalog interpreter. [24] propose a differentiable forward-chaining reasoning process, while [35] propose a continuous generalisation of the semantics of first-order logic. [36] and [37] propose an approach for learning function-free Datalog clauses from KBs by means of a differentiable graph traversal operator, while [38] propose learning policies for navigating a KB via reinforcement learning.

A major problem with these approaches is their computational complexity, which renders them unusable for larger-scale learning problems. In order to address this issue, [39] propose Greedy NTPs (GNTPs), an extension to NTPs where, for each goal, only the top- k facts and rules are considered during the differentiable reasoning process.

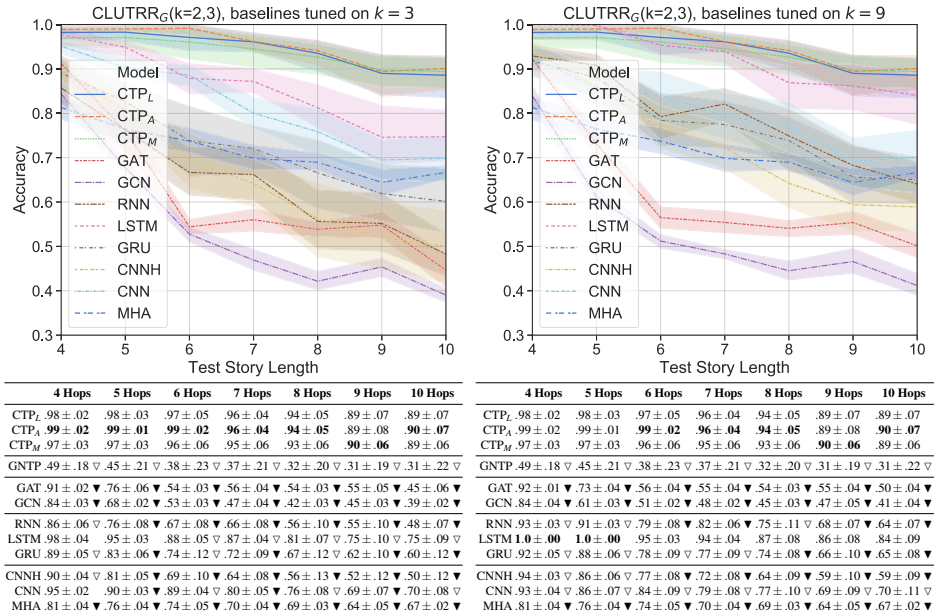


Figure 3. Results on the CLUTRR dataset after training on stories of lengths $\{2, 3\}$ and evaluating on stories of length $\{4, 5, \dots, 10\}$ – hyperparameters were fine-tuned on either short stories (left) and long stories (right). Significance testing was assessed via a unequal variances t -test in comparison with CTP_L; ▼ (resp. ▽) represents a p -value lower than 10^{-4} (resp. 10^{-2}).

Neural Module Networks [40] introduce Neural Module Networks (NMNs), an end-to-end differentiable composition of jointly trained neural modules. Analogously, NTPs can be seen as a recursive differentiable composition of *or* and *and* modules, jointly trained on downstream reasoning tasks. NMNs allow defining and training end-to-end differentiable composable models, and interpret and execute their compositions as simple programs. This is especially useful when dealing with reasoning tasks from visual and natural language inputs, such as reasoning over text with arithmetic modules [41] or visual question answering [40]. Interestingly, in this work the structure of the composition is statically drawn from the data, while [42] propose a way of learning the model composition via a coordination model [42].

Incorporating Knowledge via Regularisation Another branch of works uses symbolic background knowledge to learn better representations for entities and relationships in a KB. An early work in this space is [43], which regularise a relation extraction model by penalising inconsistency with respect to a set of logical constraints and sampled entities. [44] regularise relation representations to incorporate equivalence and inversion axioms for a set of neural link prediction models, while [45] focus on simple implication axioms.

[46] propose adversarially regularising neural models by identifying, during training, inputs that violate a given set of constraints, and regularising the model to decrease the degree of such violations. [47] propose a similar idea, by using a *semantic loss* measuring to which extent a model matches a set of given constraints.

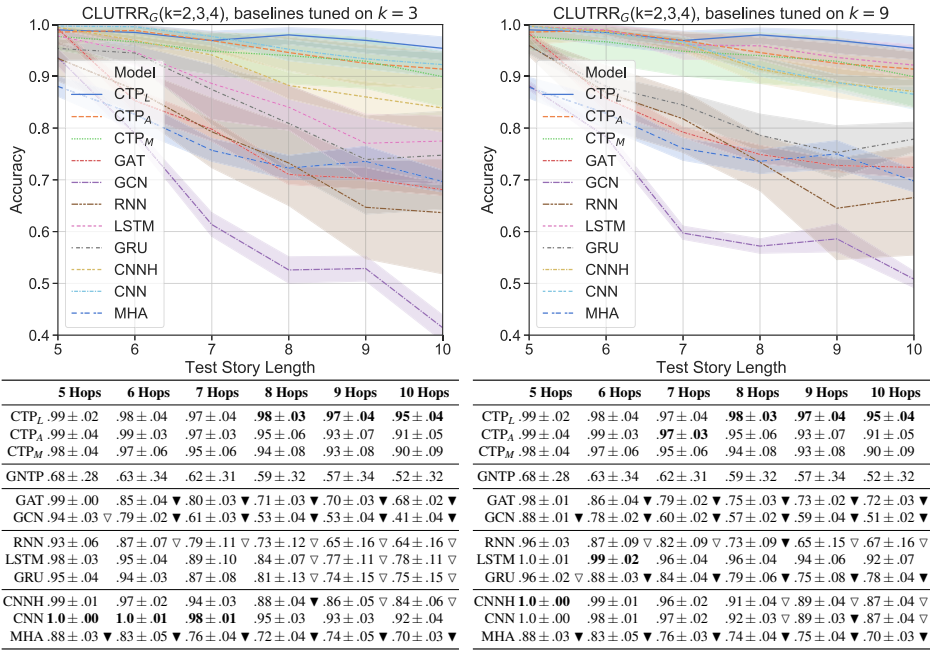


Figure 4. Results on the CLUTRR dataset after training on stories of lengths $\{2, 3, 4\}$ and evaluating on stories of length $\{5, \dots, 10\}$ – hyperparameters were fine-tuned on either short stories (left) and long stories (right). Significance testing was assessed via a unequal variances t -test in comparison with CTP_L: ▼ (resp. ▽) represents a p -value lower than 10^{-4} (resp. 10^{-2}).

12.5. Experiments

We evaluate CTPs on two datasets: systematic generalisation on the CLUTRR dataset, and link prediction in Knowledge Graphs (KGs). Datasets are introduced in Section 12.5.1, while baselines are described in Section 12.5.2.

12.5.1. Datasets and Tasks

Systematic Generalisation CLUTRR—Compositional Language Understanding and Text-based Relational Reasoning [14]—contains a large set of graphs modelling hypothetical family relationships. Given a set of family relations, encoded as a graph with a variable number of nodes and edges, the goal is to infer the relationship between two family members, whose relationship is not explicitly mentioned. To solve this task, a learning agent should be able to induce the logical rules governing the kinship relationships, such as the *parent* of a *parent* is a *grandparent*, and use these rules to infer the relationship between a given pair of entities.

CLUTRR allows testing a learning agent’s ability for *systematic generalisation*, by testing on graphs containing combinations of logical rules that were not seen during training. Each edge in the graph is labelled with one out of nine family relation type from $\mathcal{R} = \{ \text{child, grand, in-law, inv-child, inv-grand, inv-in-law, inv-un, sibling, un} \}$, and the task consists in inferring the relationship between two of the nodes in the graph. During training, a model is trained to infer such relationship by traversing a limited number of

edges (such as two, three, and four edges), and during evaluation the model has to traverse up to ten edges.

Fig. 1 shows an example of a training instance and a test instance in CLUTRR: the training instances consists in a graph modelling a set of family relations of only three edges, while the test instance is composed by a graph with ten edges. In both cases, the task consists in inferring the relationships between two of the nodes in the graph.

Link Prediction Furthermore, we evaluate CTPs on neural link prediction tasks, following the same evaluation protocols as [25] on the Countries [48], Nations, UMLS, and Kinship [49] datasets.

The Countries dataset contains countries, regions, and sub-regions as entities, and it is carefully designed to test the logical reasoning and learning capabilities of neural link prediction models: queries have the form `locatedIn(C, ·)`, where the answers are regions. This dataset comes with three tasks ($S1$, $S2$, and $S3$) each requiring reasoning skills of increasing complexity; we refer to [25] for more details about this dataset.

The Unified Medical Language System (UMLS) dataset is from bio-medicine: entities are biomedical concepts, and relations include treatments and diagnoses. The Kinship dataset contains kinship relationships among members of the Alyawarra tribe from Central Australia.

12.5.2. Models and Baselines

Models We consider the following three CTP model variants: *i*) CTP_L , where the mappings f_i from goal predicates to rule predicates are implemented via a linear projection, *ii*) CTP_A , where it is implemented via *attentive goal reformulation*, as described in Section 12.3, and *iii*) CTP_M , where it is implemented via *memory-based goal reformulation*, also described in Section 12.3.

Baselines We consider two classes of baselines: *graph-based* and *sequence-based*. Graph-based baselines consist in neural architectures specifically designed for graph data and. We consider GNTPs, a neuro-symbolic reasoning model, and two Graph Neural Network (GNN) architectures, namely GATs [21] and Graph Convolutional Networks (GCNs) [50].

Sequence-based baselines are neural architectures originally proposed for encoding sequences: by linearising the relational graphs into sequences of subject-predicate-object triples, we can use such models for encoding graphs. We consider several sequence encoding models, namely Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs) [51], Gated Recurrent Units (GRUs) [52], Convolutional Neural Networks (CNNs) [53], CNN with Highway Encoders (CNNHs) [54], Multi-Headed Attention Networks (MHAs) [55].

Encoding For both graph-based and sequence-based baselines, we considered two approaches: *i*) encoding the the KB and goal independently, as in [14], and *ii*) conditioning the KB encoding on the goal. Let enc_{θ_e} denote an encoder that, given a set of ground facts (such as a KB or a goal), produces a continuous k -dimensional representation, and $\hat{y}$ denote a conditional distribution over the candidate relationship types. The encoder in item (i), where the goal G and the KB $\mathcal{K}$ are encoded independently, can be summarised as $\hat{y} = \text{softmax}(\mathbf{W} [\text{enc}_{\theta_e}(\mathcal{K}); \text{enc}_{\theta_e}(G)])$. The encoder in item (ii), where G and $\mathcal{K}$ are encoded jointly, can be summarised as $\hat{y} = \text{softmax}(\mathbf{W} \text{enc}_{\theta_e}([G; \mathcal{K}]))$.

For model selection, we generate a CLUTRR-like dataset using the code published by [14] composed of training set graphs with $\{2, 3\}$ edges, and two validation sets, one

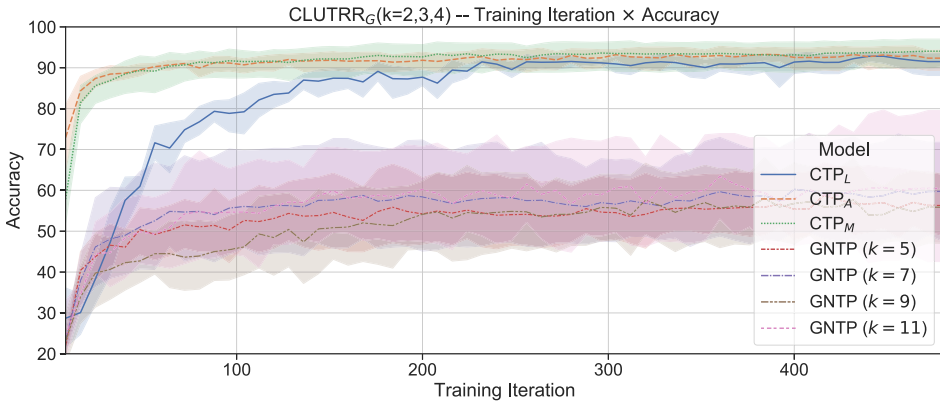


Figure 5. Training dynamics of CTPs and GNTPs.

with graphs with three edges, and another with graphs with nine edges. We then select two sets of hyperparameters for each of the baselines: one that maximises the validation accuracy on graphs with three edges, and another that maximises the test accuracy on graphs with nine edges.

To assess the statistical significance of our results, we ran each of the experiments 10 times, each time with a different seed, and compared the resulting accuracy values using an *unequal variances t-test*, or Welch's *t-test*.³

12.5.3. Results

CLUTRR We evaluated three CTP variants and all considered baselines on two datasets published by [14] under the identifiers 089907f8 and db9b8f04—we refer to these datasets as $\text{CLUTRR}_G(k=2,3)$ and $\text{CLUTRR}_G(k=2,3,4)$, where k denotes the number of edges in the training graphs. Results for $\text{CLUTRR}_G(k=2,3)$ are summarised in Fig. 3, while results for $\text{CLUTRR}_G(k=2,3,4)$ are summarised in Fig. 4.

In Fig. 3, we observe that, after training on graphs with two and three edges, baseline models tend to be able to generalise correctly to slightly longer stories (such as graphs with four and five edges), but that predictive accuracy quickly decreases with increasing graph sizes and this phenomenon happens when tuning hyper-parameters either on graphs with three edges or on graph with nine edges.

In our experiments, LSTMs had a strikingly different behaviour in comparison with other baselines: for graphs with nine edges, the accuracy decrease caused by using the LSTMs baseline is only significant with $p \leq 10^{-2}$ (for all other baselines this change is significant with $p \leq 10^{-4}$), with a drop in significance for smaller graphs.

The phenomenon that LSTMs yield surprisingly accurate results on the CLUTRR dataset can be seen across every experiment in our empirical evaluation, while other recurrent models such as RNNs and GRUs do not show this.

Model Analysis A great feature of CTPs is that we can analyse the goal reformulation process to understand the reasoning process underlying a given prediction, and extract

³We assume accuracy values to be Gaussian-distributed, as they approach a normal distribution for large numbers of re-runs, due to the Central Limit Theorem.

Table 1. Comparison of CTPs, with GNTPs [39], NeuralLP [36] and MINERVA [38] (from [39]) on benchmark datasets: hyperparameters were selected based on the validation MRR, and we report the mean and standard deviation over 10 random seeds.

Datasets	Metrics	CTP	Models				Learned Rules
			GNTP		NeuralLP	MINERVA	
			Standard	Attention			
Countries	S1	100.0 ± 0.00	99.98 ± 0.05	100.0 ± 0.00	100.0 ± 0.0	100.0 ± 0.0	locIn(X,Y) :- locIn(X,Z), locIn(Z,Y)
	S2 AUC-PR	91.81 ± 1.07	90.82 ± 0.88	93.48 ± 3.29	75.1 ± 0.3	92.36 ± 2.41	neighOf(X,Y) :- neighOf(X,Z), locIn(Z,Y)
	S3	94.78 ± 0.00	87.70 ± 4.79	91.27 ± 4.02	92.20 ± 0.2	95.10 ± 1.20	neighOf(X,Y) :- neighOf(Y,X)
Kinship	MRR	0.764 ± 0.00	0.719	0.759	0.619	0.720	term0(X, Y) :- term22(Y, X)
	Hits@1	0.646 ± 0.01	0.586	0.642	0.475	0.605	term4(X, Y) :- term4(Y, X)
	Hits@3	0.859 ± 0.01	0.815	0.850	0.707	0.812	term20(X,Y) :- term24(X, Z), term6(Z, Y)
	Hits@10	0.958 ± 0.00	0.958	0.959	0.912	0.924	term2(X,Y) :- term4(X, Z), term7(Z, Y)
Nations	MRR	0.709 ± 0.03	0.658	0.645	—	—	tourism3(X, Y) :- eemigrants(Y, X)
	Hits@1	0.562 ± 0.05	0.493	0.490	—	—	independence(X,Y) :- commonbloc0(Y,X)
	Hits@3	0.813 ± 0.03	0.781	0.736	—	—	relngo(X,Y) :- timesinceally(Y,X)
	Hits@10	0.995 ± 0.00	0.985	0.975	—	—	relstudents(X, Y) :- relexportbooks(X, Y)
UMLS	MRR	0.852 ± 0.01	0.841	0.857	0.778	0.825	isa(X,Y) :- isa(X,Z), isa(Z,Y)
	Hits@1	0.752 ± 0.01	0.732	0.761	0.643	0.728	resultOf(X,Y) :- resultOf(X,Z), resultOf(Z,Y)
	Hits@3	0.947 ± 0.01	0.941	0.947	0.869	0.900	treats(X, Y) :- prevents(X, Z), resultOf(Z, Y)
	Hits@10	0.984 ± 0.00	0.986	0.983	0.962	0.968	uses(X,Y) :- produces(X,Y)

explainable rules. In Fig. 2, we show a sample of the rules and common-sense reasoning patterns learned by CTPs on the CLUTRR dataset.

We can see that, for example, CTPs successfully identify that *e.g. the child of a child is a grandchild, the child of one's significant other is also one's child, and the parent of a significant other is an in-law*.

Training Dynamics We analyse the training dynamics of CTP_L, CTP_A, CTP_M, and GNTPs [39] on CLUTRR_G(k=2,3,4).

The CTP variants consisted of 5 goal reformulators, each implemented by an independent `select` module, while GNTP has a KB of 32 rules and $k \in \{5, 7, 9, 11\}$. For all models, the embedding size for entities and relations was set to 50.

Figure 5 demonstrates how the training accuracy of such models evolves during training. We can see that, while the three CTP variants get to a nearly-perfect training set accuracy in less than 300 iterations, GNTPs is unable to match this result, even after careful hyperparameter tuning. A possible explanation is that, in order to scale to large rule sets, GNTPs only considers the top- k rules, based on the similarity between the goal and the head of the rules. This is equivalent to a hard attention mask, which is known to be problematic to train via gradient-based optimisation [56].

Link Prediction In Table 1, we show link prediction results in comparison with three other neuro-symbolic reasoning methods, namely GNTPs [39], NeuralLP [36] and MINERVA [38]. GNTPs are an extension of NTPs where rules are heuristically selected by search for the rules where the head predicate is closest to the sub-goal predicate in embedding space.

Our experiments show that CTPs produce significantly more accurate or very competitive link prediction results, while controlling the complexity of the reasoning process via the goal-conditioned rule selection. For instance, in the Nations dataset, only two rules were generated by CTPs for each goal, while in [25] NTPs were required to iterate over sixty rules.

Furthermore, in this case, CTPs were able to produce explanations for each of their predictions. For instance, in the Nations dataset, CTPs successfully extracted logical patterns such as

$$\begin{aligned} \text{commonbloc1}(X, Y) &:- \text{relngo}(Y, X), \\ \text{timesincewar}(X, Y) &:- \text{independence}(X, Y), \\ \text{unweightedunvote}(X, Y) &:- \text{relngo}(X, Y), \text{ and} \\ \text{ngo}(X, Y) &:- \text{independence}(Y, X). \end{aligned}$$

12.6. Conclusions

We introduced CTPs, an extension to NTPs for learning the optimal rule selection strategy via gradient-based optimisation. For each sub-goal, a `select` module produces a smaller set of rules, which is then used during the proving mechanism. Furthermore, we proposed three variants of the rule selection mechanism, where the sub-goal reformulations are obtained by linear projections of the sub-goal predicate, attention distributions over predicate embeddings, and a key-value memory lookup over a set of rules.

We showed that CTPs are scalable and yield state-of-the-art results on the CLUTRR dataset, which explicitly tests the systematic generalisation of neural models, in comparison with a wide set of neural baselines. Finally, we demonstrated that CTPs yield competitive results in standard link prediction benchmark in comparison with other neuro-symbolic approaches.

Future Work An open problem is how CTPs can be able to process CLUTRR instances where family relationships are not directly provided as a labelled graph, but rather as free-form text. A possible solution, proposed by [39], consists in having an end-to-end differentiable encoder for producing the fact embeddings conditioned on the text, and we are currently analysing several options in this space.

Acknowledgements This work was supported by the EU Horizon 2020 Research and Innovation Programme under the grant 875160. We thank Yihong Chen, Joe Stacey, and all the amazing folks in the UCL NLP group for the enlightening discussions and support. Finally, we thank NVIDIA for GPU donations.

References

- [1] LeCun Y, Bengio Y, Hinton GE. Deep learning. *Nature*. 2015;521(7553):436–444.
- [2] Goodfellow IJ, Bengio Y, Courville AC. Deep learning. MIT Press; 2016. Adaptive computation and machine learning.
- [3] Goldberg Y. Neural network methods for natural language processing. Morgan & Claypool Publishers; 2017. Synthesis Lectures on Human Language Technologies.
- [4] Seo MJ, Kembhavi A, Farhadi A, et al. Bidirectional attention flow for machine comprehension. In: ICLR (Poster). OpenReview.net; 2017.
- [5] Hu M, Peng Y, Huang Z, et al. Reinforced mnemonic reader for machine reading comprehension. In: IJCAI. ijcai.org; 2018. p. 4099–4106.
- [6] Shen Y, Huang P, Gao J, et al. Reasonet: Learning to stop reading in machine comprehension. CoRR. 2016;abs/1609.05284.

- [7] Huang H, Zhu C, Shen Y, et al. Fusionnet: Fusing via fully-aware attention with application to machine comprehension. In: ICLR (Poster). OpenReview.net; 2018.
- [8] Devlin J, Chang M, Lee K, et al. BERT: pre-training of deep bidirectional transformers for language understanding. In: NAACL-HLT (1). Association for Computational Linguistics; 2019. p. 4171–4186.
- [9] Yang Z, Dai Z, Yang Y, et al. Xlnet: Generalized autoregressive pretraining for language understanding. CoRR. 2019;abs/1906.08237.
- [10] Liu Y, Ott M, Goyal N, et al. Roberta: A robustly optimized BERT pretraining approach. CoRR. 2019;abs/1907.11692.
- [11] Bahdanau D, Murty S, Noukhovitch M, et al. Systematic generalization: What is required and can it be learned? In: ICLR (Poster). OpenReview.net; 2019.
- [12] Lake BM, Baroni M. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In: ICML; (Proceedings of Machine Learning Research; Vol. 80). PMLR; 2018. p. 2879–2888.
- [13] Johnson J, Hariharan B, van der Maaten L, et al. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. In: CVPR. IEEE Computer Society; 2017. p. 1988–1997.
- [14] Sinha K, Sodhani S, Dong J, et al. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In: EMNLP/IJCNLP (1). Association for Computational Linguistics; 2019. p. 4505–4514.
- [15] Jia R, Liang P. Adversarial examples for evaluating reading comprehension systems. In: EMNLP. Association for Computational Linguistics; 2017. p. 2021–2031.
- [16] Gururangan S, Swayamdipta S, Levy O, et al. Annotation artifacts in natural language inference data. In: NAACL-HLT (2). Association for Computational Linguistics; 2018. p. 107–112.
- [17] Kaushik D, Lipton ZC. How much reading does reading comprehension require? A critical investigation of popular benchmarks. In: EMNLP. Association for Computational Linguistics; 2018. p. 5010–5015.
- [18] Doshi-Velez F, Kim B. Towards a rigorous science of interpretable machine learning. CoRR. 2017;abs/1702.08608.
- [19] Lipton ZC. The mythos of model interpretability. Commun ACM. 2018;61(10):36–43.
- [20] Garnelo M, Shanahan M. Reconciling deep learning with symbolic artificial intelligence: representing objects and relations. Current Opinion in Behavioral Sciences. 2019;29:17 – 23. SI: 29: Artificial Intelligence (2019).
- [21] Velickovic P, Cucurull G, Casanova A, et al. Graph attention networks. In: ICLR (Poster). OpenReview.net; 2018.
- [22] Sodhani S, Chandar S, Bengio Y. On training recurrent neural networks for lifelong learning. CoRR. 2018;abs/1811.07017.
- [23] d’Avila Garcez AS, Besold TR, Raedt LD, et al. Neural-symbolic learning and reasoning: Contributions and challenges. In: AAAI Spring Symposia. AAAI Press; 2015.
- [24] Evans R, Grefenstette E. Learning Explanatory Rules from Noisy Data. JAIR. 2018;61:1–64.
- [25] Rocktäschel T, Riedel S. End-to-end differentiable proving. In: NIPS; 2017. p. 3788–3800.
- [26] Russell SJ, Norvig P. Artificial intelligence - A modern approach, third international edition. Pearson Education; 2010.
- [27] Miller AH, Fisch A, Dodge J, et al. Key-value memory networks for directly reading documents. In: EMNLP. The Association for Computational Linguistics; 2016. p. 1400–1409.
- [28] Sukhbaatar S, Szlam A, Weston J, et al. End-To-End Memory Networks. In: Cortes C, et al., editors. NIPS; 2015. p. 2440–2448.
- [29] Graves A, Wayne G, Danihelka I. Neural Turing Machines. CoRR. 2014;abs/1410.5401.
- [30] Joulin A, Mikolov T. Inferring Algorithmic Patterns with Stack-Augmented Recurrent Nets. In: NIPS; 2015. p. 190–198.
- [31] Grefenstette E, Hermann KM, Suleyman M, et al. Learning to Transduce with Unbounded Memory. In: NIPS; 2015. p. 1828–1836.
- [32] Kaiser Ł, Sutskever I. Neural GPUs Learn Algorithms. In: ICLR; 2016.
- [33] Santoro A, Bartunov S, Botvinick M, et al. Meta-learning with memory-augmented neural networks. In: International conference on machine learning; 2016. p. 1842–1850.
- [34] Bošnjak M, Rocktäschel T, Naradowsky J, et al. Programming with a Differentiable Forth Interpreter. In: ICML; Vol. 70. PMLR; 2017. p. 547–556.
- [35] Donadello I, Serafini L, d’Avila Garcez AS. Logic tensor networks for semantic image interpretation. In: IJCAI. ijcai.org; 2017. p. 1596–1602.
- [36] Yang F, Yang Z, Cohen WW. Differentiable Learning of Logical Rules for Knowledge Base Reasoning.

- In: Guyon I, et al., editors. NIPS; 2017. p. 2316–2325.
- [37] Sadeghian A, Armandpour M, Ding P, et al. DRUM: end-to-end differentiable rule mining on knowledge graphs. In: NeurIPS; 2019. p. 15321–15331.
- [38] Das R, Dhuliawala S, Zaheer M, et al. Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In: ICLR (Poster). OpenReview.net; 2018.
- [39] Minervini P, Bosnjak M, Rocktäschel T, et al. Differentiable reasoning on large knowledge bases and natural language. In: AAAI. AAAI Press; 2020. p. 5182–5190.
- [40] Andreas J, Rohrbach M, Darrell T, et al. Neural module networks. In: CVPR. IEEE Computer Society; 2016. p. 39–48.
- [41] Gupta N, Lin K, Roth D, et al. Neural module networks for reasoning over text. CoRR. 2019; abs/1912.04971.
- [42] Jiang Y, Bansal M. Self-assembling modular networks for interpretable multi-hop reasoning. In: EMNLP/IJCNLP (1). Association for Computational Linguistics; 2019. p. 4473–4483.
- [43] Rocktäschel T, Singh S, Riedel S. Injecting logical background knowledge into embeddings for relation extraction. In: HLT-NAACL. The Association for Computational Linguistics; 2015. p. 1119–1129.
- [44] Minervini P, Costabello L, Muñoz E, et al. Regularizing knowledge graph embeddings via equivalence and inversion axioms. In: ECML/PKDD (1); (Lecture Notes in Computer Science; Vol. 10534). Springer; 2017. p. 668–683.
- [45] Demeester T, Rocktäschel T, Riedel S. Lifted rule injection for relation embeddings. In: EMNLP. The Association for Computational Linguistics; 2016. p. 1389–1399.
- [46] Minervini P, Demeester T, Rocktäschel T, et al. Adversarial sets for regularising neural link predictors. In: UAI. AUAI Press; 2017.
- [47] Xu J, Zhang Z, Friedman T, et al. A semantic loss function for deep learning with symbolic knowledge. In: ICML; (Proceedings of Machine Learning Research; Vol. 80). PMLR; 2018. p. 5498–5507.
- [48] Bouchard G, Singh S, Trouillon T. On approximate reasoning capabilities of low-rank vector spaces. In: AAAI Spring Symposia. AAAI Press; 2015.
- [49] Kemp C, Tenenbaum JB, Griffiths TL, et al. Learning Systems of Concepts with an Infinite Relational Model. In: AAAI; 2006. p. 381–388.
- [50] Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. In: ICLR (Poster). OpenReview.net; 2017.
- [51] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Computation*. 1997;9(8):1735–1780.
- [52] Cho K, van Merriënboer B, Gülçehre Ç, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: EMNLP. ACL; 2014. p. 1724–1734.
- [53] Kim Y. Convolutional neural networks for sentence classification. In: EMNLP. ACL; 2014. p. 1746–1751.
- [54] Kim Y, Jernite Y, Sontag DA, et al. Character-aware neural language models. In: AAAI. AAAI Press; 2016. p. 2741–2749.
- [55] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: NIPS; 2017. p. 5998–6008.
- [56] Luong T, Pham H, Manning CD. Effective approaches to attention-based neural machine translation. In: EMNLP. The Association for Computational Linguistics; 2015. p. 1412–1421.

Chapter 13

Generalizable Neuro-Symbolic Systems for Commonsense Question Answering

Alessandro Oltramari, Bosch Research (Pittsburgh, PA USA)

Jonathan Francis, Bosch Research (Pittsburgh, PA USA); School of Computer Science, Carnegie Mellon University (Pittsburgh, PA USA)

Filip Ilievski, Information Sciences Institute, Viterbi School of Engineering, University of Southern California (Marina del Rey, CA USA)

Kaixin Ma, School of Computer Science, Carnegie Mellon University (Pittsburgh, PA USA)

Roshanak Mirzaee, Department of Computer Science and Engineering, Michigan State University (East Lansing, MI USA)

13.1. Introduction

In this chapter, we provide a retrospective analysis of our recent work, wherein we develop methods for integrating neural language models and knowledge graphs. We characterize the situations in which this combination is most appropriate, and we offer quantitative and qualitative evaluation of these neuro-symbolic modeling strategies on a variety of commonsense question answering benchmark datasets. Our overarching goal is to illustrate how suitable combinations of learning mechanisms and knowledge can enable domain generalizability and robustness in downstream tasks.

The chapter is structured as follows: section 13.2 describes some limitations observed when state-of-the-art language models are used in commonsense question answering tasks; in section 13.3 we discuss the main findings of our empirical investigations in discriminative tasks, focused on knowledge-injection and pre-training/fine-tuning methods (13.3.3-13.3.4), and we present novel ideas about improving reasoning capabilities of neuro-symbolic systems, laying down the path towards generalizability (13.3.5).

13.2. Beneath the Oceans Of Text

Neural Language Modeling (NLM) has yielded impressive results, with near-human level performance, in various Natural Language Processing (NLP) and Robotics tasks [1, 2, 3, 4, 5, 6, 7, 8, 9], such as: text classification, text generation, summarization, question answering, named-entity recognition, machine translation, and instruction-following in robot navigation. In particular, systems such as OpenAI's GPT-3 model, which was trained with 45TB of text data, promise perhaps the first instance of large-scale model generalizability across tasks. However, by examining the failure modes of models like GPT-3 [10], we can observe that, despite of the indisputable successes, basic reasoning capabilities are still largely missing from Language Models (LMs). In metaphorical terms, language models display great skill in surfing on vast oceans of text, but are not apt for penetrating the surface of meaning and exploring the depth of context.

Let's consider some examples from discriminative question answering, where a model is tasked with selecting the best response out of a pool of candidates, given a question and/or context. In ProtoQA [11], GPT-3 fails to select options like 'pumpkin', 'cauliflower', 'cabbage' as top candidates, for the question *one vegetable that is about as big as your head is?*; instead, 'broccoli', 'cucumber', 'beet', 'carrot' are predicted. In this case, the different models learn some essential properties of vegetables from the training data, but do not seem to acquire the capability of comparing their size to that of other types of objects, revealing a substantial lack of analogical reasoning [12]. Along these lines, recent work [13] has shown that complex inferences, role-based event prediction, and understanding the conceptual impacts of negation, are some of the missing capabilities diagnosed when BERT [14], one of the most popular and versatile open-source language models available today, is applied to benchmark datasets. ProtoQA, again, provides good examples of these deficiencies: in general, neural models struggle to correctly interpret the scope of modifiers like 'not', 'often', and 'seldom'—where the first requires reasoning under negation, while the last two consider temporal reasoning. Regarding the latter, in task 14 of bAbI [15], a comprehensive benchmark challenge designed by Facebook Research, models exhibit variable accuracy in grasping temporal ordering, implied by prepositions like 'before' and 'after'. Similarly, in bAbI task-17, which concerns spatial reasoning, language models fail to infer basic positional information that require interpreting the semantics of 'to the left/right of', 'above/below', etc. Here, neural models are inaccurate in understanding some common characteristics of the physical world, and their performance does not improve when dealing with sentiment: for instance, in SocialIQA [16], given a context like 'in the school play, Robin played a hero in the struggle to death with the angry villain', models are generally unable to consistently select 'hopeful that Robin will succeed' over 'sorry for the villain' when required to pick the correct answer to *how would others feel afterwards?*. It is not surprising that *reasoning about emotional reactions* represents a difficult task for pure learning systems, when we consider that such form of inference is deeply rooted in the sphere of human subjective experiences and social life, which involves a "layered" understanding of mental attitudes, intentions, motivations, and empathy.

The anecdotal mistakes presented above are, in fact, indications of a widespread phenomenon: a pure NLM approach is currently not able to engage in robust machine commonsense reasoning. Accordingly, we claim here that two orthogonal extensions are required: first, augmenting NLM with symbolic commonsense knowledge, which

is available in large amounts thanks to open source computational resources; and, secondly, *teaching* these neuro-symbolic language models how to dynamically leverage that commonsense knowledge to perform reasoning at different levels, including analogies, spatio-temporal properties, affect, and mental attitudes. We refer to the former direction as **horizontal augmentation**, as it focuses on expanding the coverage of the knowledge available to NLM methods, and to the latter as **vertical augmentation**, as it explores in depth how some specific forms of reasoning can be enabled.

13.3. Investigations on Commonsense Q/A

In this section we discuss the main resources, methods, and results presented in our previous work on integrating commonsense knowledge graphs and neural language models: sections 13.3.1 and 13.3.2 present, respectively, the knowledge bases and datasets we used in our experiments; section 13.3.3 and 13.3.4 are focused on discriminative Q/A tasks [1, 17]. Finally, section 13.3.5 provides some initial considerations on vertical augmentation.

13.3.1. Knowledge Bases

We select knowledge bases according to two main criteria, namely scale and usability, where the former indicates that the KB covers a variety of commonsense types or dimensions, and the latter denotes a resource that is freely available, well-documented (e.g., APIs) and actively maintained.

- WordNet [18] is a dictionary that was designed as a semantic network, where *synsets* (sets of synonym terms) represent common world knowledge. WordNet's interrelated structure is consistent with the evidence for the way speakers organize their mental lexicons. In this regard, WordNet can be considered as a 'grandfather' to most of today's computational lexicons and structured knowledge resources (it's been actively developed since 1985).
- ConceptNet [19] contains over 21 million edges and 8 million nodes, from which one may generate triples of the form $(C1, r, C2)$, wherein the natural-language concepts $C1$ and $C2$ are associated by commonsense relation r , e.g., (*dinner*, *AtLocation*, *restaurant*). Thanks to its coverage, ConceptNet is one of the most popular semantic networks for commonsense.
- ATOMIC [20] is a knowledge-base that focuses on procedural knowledge: in particular, it expresses pre- and post-states for events and their participants with nine relations. Its head nodes are events, whereas the tail nodes are either events or attributes.
- Visual Genome [21] is a multi-modal commonsense resource that combines visual and textual information. In particular, it consists of 108K images annotated by crowd-workers, and automatically categorized into WordNet [18] senses. The annotations focus on the conceptual relations holding between the entities depicted in the images, and on relevant attributes of the individual entities.

- CSKG [22] is a ‘super’ knowledge graph for common sense, as it federates different resources, leveraging statistical computations and syntactic and semantic mappings. Integrating different resources of common sense is, first and foremost, a problem of consolidating the underlying semantic models, each typically built on a unique set of features. The authors of CSKG approach this challenge by adopting clear methodological principles: *node diversity*, i.e. opting for heterogeneity of node representation across KBs over imposing strong formal constraints to resolve structural semantic differences; *edge reusability*, i.e. the principle according to which the domain and range of an edge defined within one KB can be extended to other KBs if the intended semantics is preserved; *bridging through external mappings*, where if two unconnected resources have existing links to a third, well-established resource, e.g. WordNet, these can be used to bridge the gap between them; *link prediction*, to improve connectedness using probabilistic algorithms; *enabled label access*, to foster human disambiguation of node semantics. In our experiments, we used a specific partition of CSKG, CWWV, which includes ConceptNet, WordNet, Wikidata [23]¹, and VisualGenome.

13.3.2. Datasets

We select commonsense tasks based on two criteria. Firstly, we strive to cover a diverse set of tasks, both in terms of their format (question answering, pronoun resolution, natural language inference), as well as their type of knowledge (e.g., social or physical knowledge). Secondly, we prefer larger task datasets that are manually constructed. The five datasets presented below were preferred, in our experiments, to artificially constructed datasets like COPA [25], or HellaSwag [26].

- Abductive NLI (aNLI) [27] is posed as a natural language inference task. Given the beginning and the ending of a story, the task is to choose the more plausible hypothesis out of two options. The dataset consists of nearly 170k entries.
- CommonsenseQA (CSQA) [28] evaluates a broad range of common sense notions. Each entry contains a question and 5 answer candidates. The questions are crowd-sourced based on a subgraph from ConceptNet. The answer candidates combine ConceptNet nodes with additional crowd-sourced distractors.
- PhysicalQA (PIQA) [29] is a two-choice question answering dataset which focuses on physical reasoning. Given a question, the system (or human) is asked to pick the more plausible out of two possible continuations.
- SocialQA (SIQA) [16] requires reasoning about social interactions. Each entry contains a context, a question, and 3 answer candidates. The context is derived from the ATOMIC knowledge graph, the questions are generated based on nine templates (corresponding to the relations in ATOMIC), and the answers are crowd-sourced.
- WinoGrande (WG) [30] contains 44 thousand pronoun resolution problems. Each entry consists of a context description with an emphasized pronoun, and two options are offered as its possible references.

¹CSKG only includes a commonsense subset of Wikidata extracted using heuristics based on [24].

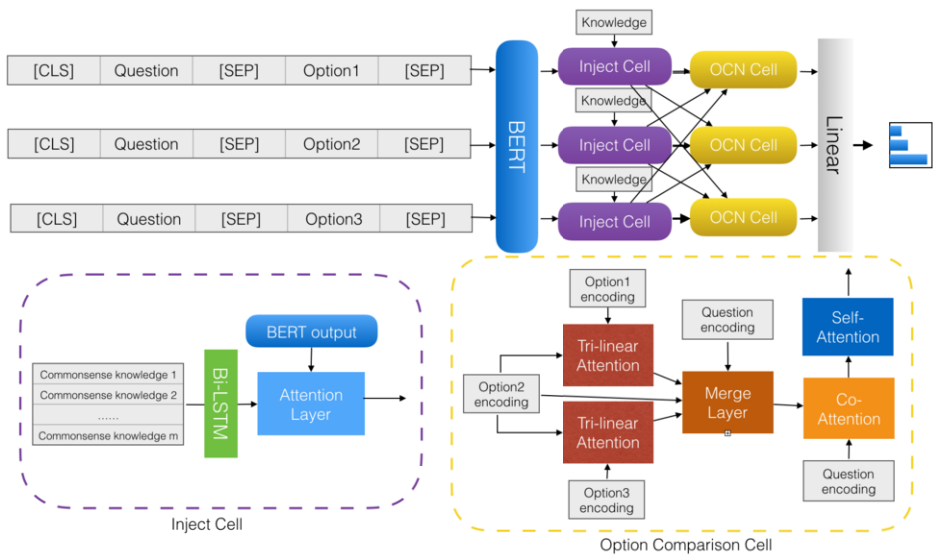


Figure 1. Option Comparison Network (OCN) with Knowledge Injection

13.3.3. Horizontal Augmentation through Attention-based Knowledge Injection

Different ways of injecting knowledge into models have been introduced, such as attention-based gating mechanisms [31], key-value memory mechanisms [32, 33], extrinsic scoring functions [34], and graph convolution networks [35, 36]. Our approach is to combine suitable pre-trained language models with structured knowledge. we incorporate ConceptNet and ATOMIC using attention knowledge injection on top of OCN (Option Comparison Network) model mechanism [37]. We evaluate the obtained neuro-symbolic models on the CommonsenseQA dataset (see example question in Table 1).

Question: A revolving door is convenient for two direction travel, but it also serves as a security measure at a what?
Answer choices: A. Bank*; B. Library; C. Department Store; D. Mall; E. New York;

Table 1. An example from the CommonsenseQA dataset; the asterisk (*) denotes the correct answer.

13.3.3.1. Model architecture

The model class we select is that of the *Bidirectional Encoder Representations with Transformer* (BERT) model [14], as it has been applied to numerous QA tasks and has achieved very promising performance, particularly on the CommonsenseQA dataset. When utilizing BERT on multiple-choice QA tasks, the standard approach is to concatenate the question with each answer-option, in order to generate a list of tokens which is then fed into BERT encoder; a linear layer is added on top, in order to predict the answer. One aspect of this strategy is that each answer-option is encoded independently, which limits the model’s ability to find correlations between answer-options and with respect

to the original question context. To address this issue, the *Option Comparison Network* (OCN) [37] was introduced to explicitly model the pairwise answer-option interactions, making OCN better-suited for multiple-choice QA task structures. The OCN model uses BERT as its base encoder: the question/option encoding is produced by BERT and further processed in a Option Comparison Cell, before being fed into the linear layer. The Option Comparison Cell is illustrated in the bottom right of figure 1. We re-implemented OCN while keeping BERT as its upstream encoder (we refer an interested reader to [37, 1] for more details).

13.3.3.2. Knowledge elicitation

We identify ConceptNet relations that connect questions to the answer-options. The intuition is that these relation paths would provide explicit evidence that would help the model find the answer. Formally, given a question Q and an answer-option O , we find all ConceptNet relations $(C1, r, C2)$ (where $C1$ and $C2$ are, respectively, the source and the target node and r the relation/edge connecting them), such that $C1 \in Q$ and $C2 \in O$, or vice versa. This rule works well for single-word concepts, but requires some adaptation for the large number of phrases that constitute ConceptNet, in particular relaxing exact-match constraint (for details, see our paper [1]).

We also observe that many questions in the CommonsenseQA task ask about which event is likely to occur, given a pre-condition. Superficially, this particular question type seems well-suited for the procedural nature of ATOMIC triples. However, one challenge of extracting knowledge from this resource is that heads and tails of knowledge triples in ATOMIC are short sentences or verb phrases, while rare words and person-references are reduced to blanks and PersonX/PersonY, respectively.

13.3.3.3. Knowledge injection

The next step is to integrate the extracted triple with the OCN component of our model. Inspired by [31], we propose to use attention-based injection. For ConceptNet knowledge triples, we first converted concept-relation entities into tokens from our lexicon, in order to generate a pseudo-sentence. For example, “(book, AtLocation, library)” would be converted to “book at location library.” Next, we used the knowledge injection cell to fuse the commonsense knowledge into BERT’s output, before feeding the fused output into the OCN cell. Specifically, in a knowledge-injection cell, a Bi-LSTM layer is used to encode these pseudo-sentences, before computing the attention with respect to BERT output, as illustrated in bottom left of figure 1.

13.3.3.4. Knowledge pre-training

Our goal is to assess, using CommonsenseQA as benchmark, whether pre-training on ConceptNet and ATOMIC can help the model acquire relevant commonsense to improve performance in specific questions of the task. Our approach follows from a well-known phenomenon studied in NLM: i.e., pre-training large-capacity models (e.g., BERT, GPT [38], XLNet [39]) on large corpora, then fine-tuning on more domain-specific information, leads to increase of accuracy across various tasks. For the ConceptNet pre-training procedure, pre-training BERT on pseudo-sentences formulated from ConceptNet knowledge triples does not provide much gain on performance. Instead, we trained BERT on the *Open Mind Common Sense* (OMCS) corpus [40], the

originating corpus that was used to create ConceptNet. We extracted about 930K English sentences from OMCS and randomly masked out 15% of the tokens; we then fine-tuned BERT, using a masked language model objective, where the model’s objective is to predict the masked tokens, as a probability distribution over the entire lexicon. Finally, we load this fine-tuned model into OCN framework proceed with the downstream CommonsenseQA task. As for pre-training on ATOMIC, we follow previous work’s pre-processing steps to convert ATOMIC knowledge triples into sentences [41]; we created special tokens for 9 types of relations as well as blanks. Next, we randomly masked out 15% of the tokens, only masking out tail-tokens; we used the same OMCS pre-training procedure.

Table 2. Results on CommonsenseQA; the asterisk (*) denotes results taken from leaderboard.

Models	Dev Acc
BERT + OMCS pre-train(*)	68.8
RoBERTa + CSPT(*)	76.2
OCN	64.1
OCN + CN injection	67.3
OCN + OMCS pre-train	65.2
OCN + ATOMIC pre-train	61.2
OCN + OMCS pre-train + CN inject	69.0

13.3.3.5. Results and Discussion - Part 1

For all of our experiments, we run 3 trials with different random seeds and we report average scores tables 2 and 3. Evaluated on CommonsenseQA, ConceptNet knowledge-injection provides a significant performance boost (+2.8%), compared to the OCN baseline, suggesting that explicit links from question to answer-options help the model find the correct answer. Pre-training on OMCS also provides a small performance boost to the OCN baseline. Since both ConceptNet knowledge-injection and OMCS pre-training are helpful, we combine both approaches with OCN and we are able to achieve further improvement (+4.9%). Finally, to our surprise, OCN pre-trained on ATOMIC yields a significantly lower performance.

Table 3. Accuracies for each CommonsenseQA question type: **AtLoc.** means *AtLocation*, **Cau.** means *Causes*, **Cap.** means *CapableOf*, **Ant.** means *Antonym*, **H.Pre.** means *HasPrerequisite*, **H.Sub** means *HasSubevent*, **C.Des.** means *CausesDesire*, and **Des.** means *Desires*. Numbers beside types denote the number of questions of that type.

Models	AtLoc.(596)	Cau.(194)	Cap.(109)	Ant.(92)	H.Pre.(46)	H.Sub.(39)	C.Des.(28)	Des.(27)
OCN	64.9	66.5	65.1	55.4	69.6	64.1	57.1	66.7
+CN inj.	67.4(+2.5)	70.6(+4.1)	66.1(+1.0)	60.9(+5.5)	73.9(+4.3)	66.7(+2.6)	64.3(+7.2)	77.8(+11.1)
+OMCS	68.8(+3.9)	63.9(-2.6)	62.4(-2.7)	60.9(+5.5)	71.7(+2.1)	59.0(-5.1)	64.3(+7.2)	74.1(+7.4)
+ATOMIC	62.8(-2.1)	66.0(-0.5)	60.6(-4.5)	52.2(-3.2)	63.0(-6.6)	56.4(-7.7)	60.7(+3.6)	74.1(+7.4)
+OMCS+CN	71.6(+6.7)	71.6(+5.1)	64.2(+0.9)	59.8(+4.4)	69.6(+0.0)	69.2(+5.1)	75.0(+17.9)	70.4(+3.7)

To understand when a model performs better or worse with knowledge-injection, we analyzed predictions by question type. Since all questions in CommonsenseQA require

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

commonsense reasoning, we classify questions based on the ConceptNet relation between the question concept and correct answer concept, *de facto* investigating commonsense *knowledge dimensions* (whose subtle differences have been discussed at length by philosophers, linguists, and cognitive psychologists [42]²). The intuition is that the model needs to capture a relevant relation between concepts in order to answer a commonsense question. The accuracy measures for each question type are shown in Table 3. Note that the number of samples by question type is very imbalanced. Thus, due to the limited space, we omitted the long tail of the distribution (about 7% of all samples). We can see that with ConceptNet relation-injection, all question types got performance boosts—for both the OCN model and OCN model that was pre-trained on OMCS — suggesting that external knowledge is indeed helpful for the task. In the case of OCN pre-trained on ATOMIC, although the overall performance is much lower than the OCN baseline, it is interesting to see that performance for the “Causes” type is not significantly affected. Moreover, performance for “CausesDesire” and “Desires” types actually got much better. As noted by [20], the “Causes” relation in ConceptNet is similar to “Effects” and “Reactions” in ATOMIC; and “CausesDesire” in ConceptNet is similar to “Wants” in ATOMIC. This result suggests that models with knowledge pre-training perform better on questions that fit the knowledge domain, but perform worse on others. In this case, pre-training on ATOMIC helps the model do better on questions that are similar to ATOMIC relations, even though overall performance is inferior. Finally, we noticed that questions of type “Antonym” appear to be the hardest ones: questions that fall into this category typically contain negations, which indicates that horizontal augmentation is not sufficient to endow the model with the ability to reason over negative sentences, a feature that — we hypothesize — requires vertical augmentation.

In regards to knowledge-injection methods, attention-based injection seems to be the better choice for pre-trained language models such as BERT: even when alignment between knowledge-base and dataset is sub-optimal, the performance would not degrade. On the other hand, pre-training on knowledge-bases would shift the language model’s weight distribution toward its own domain, greatly. If the task domain does not fit knowledge-base well, model performance is likely to drop. When the domain of the knowledge-base aligns with that of the dataset perfectly, both knowledge-injection methods bring performance boosts and a combination of them could bring further gain.

13.3.4. Horizontal Augmentation through Knowledge-driven Data Construction for Zero-shot Evaluation

Although what we presented so far provides significant results in support of using symbolic knowledge to augment NLM approaches in Q/A tasks, there is increasing concern that language models overfit to specific datasets, without learning to utilize commonsense knowledge across tasks. Zero-shot evaluations have shown promise as a more ro-

²Our pragmatic approach, grounded on considering relations/edges in commonsense knowledge graphs (e.g., *part_of*, *located_at*, *cause_of*, etc.) as key knowledge dimensions, led to further work where we specifically looked at the distribution of triples across different resources, studying the impact of each relevant dimension in selected downstream tasks. We demonstrate that identifying the most appropriate commonsense knowledge type(s) required for specific task(s), is instrumental to achieve robust performance. Once the knowledge type is assessed, the most appropriate knowledge resources can be selected, alongside with the corresponding knowledge-extraction methods, and mechanisms to endow language models with relevant triple-based knowledge structures. For details see [43]).

bust measure of a model's generalizability. While KBs have been shown to help in a zero-shot transfer setting recently [44], no comprehensive study exists on the relation between various knowledge, its usage method, and neural models for zero-shot transfer across commonsense tasks. Accordingly, here we show how Q/A data synthetically generated from five knowledge bases (see section 13.3.1) can help to boost performance of language models across the five commonsense question-answering tasks (outlined in 13.3.2). We found that, while an individual knowledge graph is better suited for specific tasks, a global knowledge graph brings consistent gains across different tasks. We refer the reader to our original work for details on the question generation methods, and on the adversarial filtering techniques [2].

13.3.4.1. Language Models

We consider 2 types of language models: auto-regressive language models and masked language models (MLM). Specifically, we use GPT-2 and RoBERTa to select the best answer candidate. Given a context C , a question Q , and a list of answer options $(A_1, A_2 \dots)$, we concatenate C and Q with each answer option to build input sequences $(T_1, T_2 \dots)$. We also use templates to convert a sequence T into a natural language sentence following [45]. For example, we transform the sequence: $[C]$ *What will X want to do next?* $[A_i]$ into: $[C]$, *as a result, X want to* $[A_i]$. The score S for the resulting sequence using an auto-regressive LM is computed as follows:

$$S_{LM}(T) = -\frac{1}{n} \sum_{i=1}^n \log P(t_i | t_1 \dots t_{i-1}) \quad (1)$$

where n is the number of tokens in the sequence and P is the conditional probability provided by the LM. To evaluate MLMs, we mask out one token at a time and compute its loss [46]. We repeat this process for every token in the sequence. The final MLM score is:

$$S_{MLM}(T) = -\frac{1}{n} \sum_{i=1}^n \log P(t_i | \dots t_{i-1}, t_{i+1} \dots) \quad (2)$$

The predicted option is the one with the lowest score.

In the typical model architecture for fine-tuning LM for multiple-choice tasks, a linear layer is added on top of the LM encoder to predict the answer. The model inputs are separated by a model-specific delimiter. However, as this architecture introduces randomly initialized parameters, it may not be able to fully utilize the pre-trained weights [47]. Instead, we re-use the GPT-2 and RoBERTa with LM head for fine-tuning. By keeping the model intact, we can reuse the same converting templates and scoring functions. To train the model, given the scores computed for each answer candidate $S_1, S_2, \dots, S_m$, we use the marginal ranking (MR) loss defined as:

$$\mathcal{L} = \frac{1}{m} \sum_{\substack{i=1 \\ i \neq y}}^m \max(0, \eta - S_y + S_i) \quad (3)$$

Here, η represents the margin and y is the index of the correct answer. For a MLM model, the computation cost for the scoring function scales quadratically with the input length.

Table 4. Zero-shot evaluation results with different combinations of models and knowledge sources, across five commonsense tasks. CSKG represent the combination of ATOMIC and CWWV. We run our experiments three times with different seeds and report average accuracy with 95% confidence interval. SMLM (*) used OMCS for CSQA, ROCStories [48] for aNLI and ATOMIC for SIQA as knowledge resources.

Model	KG	aNLI	CSQA	PIQA	SIQA	WG
Majority	-	50.8	20.9	50.5	33.6	50.4
GPT2-L	-	56.5	41.4	68.9	44.6	53.2
RoBERTa-L	-	65.5	45.0	67.6	47.3	57.5
Self-talk	[45]	-	32.4	70.2	46.2	54.7
COMET-DynaGen	[49] ATOMIC	-	-	-	50.1	-
SMLM	[50] *	65.3	38.8	-	48.5	-
GPT2-L (MR)	ATOMIC	59.2(±0.3)	48.0(±0.9)	67.5(±0.7)	53.5(±0.4)	54.7(±0.6)
GPT2-L (MR)	CWWV	58.3(±0.4)	46.2(±1.0)	68.6(±0.7)	48.0(±0.7)	52.8(±0.9)
GPT2-L (MR)	CSKG	59.0(±0.5)	48.6(±1.0)	68.6(±0.9)	53.3(±0.5)	54.1(±0.5)
RoBERTa-L (MR)	ATOMIC	70.8(±1.2)	64.2(±0.7)	72.1(±0.5)	63.1(±1.5)	59.6(±0.3)
RoBERTa-L (MR)	CWWV	70.0(±0.3)	67.9(±0.8)	72.0(±0.7)	54.8(±1.2)	59.4(±0.5)
RoBERTa-L (MR)	CSKG	70.5(±0.2)	67.4(±0.8)	72.4(±0.4)	63.2(±0.7)	60.9(±0.8)
RoBERTa-L (supervised)	-	85.6	78.5	79.2	76.6	79.3
Human	-	91.4	88.9	94.9	86.9	94.1

To make the training more efficient, we only mask out non-stop tokens in the head and tail nodes.

In order to disentangle the contribution of the KGs from the structure of the QA pairs, we consider different training methods for augmentation of language models with KGs. Specifically, we compare marginal ranking (MR) training with masked language modeling (MLM) training. For MLM, we directly concatenate the question and the correct answer in our synthetic QA set and then train RoBERTa on the these sentences using the MLM objective.

13.3.4.2. Experimental Setup

We compare our results with the following baselines. *Majority* answers each question with the most frequent option in the entire dataset. ‘*Vanilla*’ versions of the language models are used in order to understand the impact of further tuning. Here we directly use the LMs to score the QA pairs without any fine-tuning. We also show the results of other unsupervised systems that leverage KGs: *Self-talk*, *COMET-DynaGen*, and *SMLM*. To indicate the upper bound of this work, we include results of a supervised fine-tuned RoBERTa system and of human evaluation.

13.3.4.3. Results and Discussion - Part 2

Table 4 shows that GPT-2 and RoBERTa outperform the majority baseline by a large margin on all tasks, indicating that the LMs have already learned relevant knowledge during pre-training. Despite being a smaller model, RoBERTa outperforms GPT-2 on 4 out of 5 tasks without pre-training, and on all tasks when pre-training over different

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

synthetic QA sets. This shows the advantage of leveraging bi-directional context. Training RoBERTa on our ATOMIC or CWWV synthetic sets brings notable performance gain on all 5 tasks. We observe that models trained on ATOMIC sets have a large advantage on SIQA compare to models trained on CWWV, while CWWV brings advantage on the CSQA task. This is not surprising as these two tasks are derived from ConceptNet and ATOMIC, respectively. The difference between ATOMIC and CWWV on the remaining three tasks is relatively small. This supports the main findings presented in section 13.3.3: knowledge alignment is crucial for obtaining better performance. Training on the combined question set (CSKG) is mostly able to retain the best of its both partitions. Training on CSKG leads to best performance on three out of five tasks, showing that a global commonsense resource is able to bring consistent gain across different tasks. This shows that adding more diverse knowledge is beneficial for language models. The training regime had a role too: marginal ranking is superior to vanilla language modeling, as it preserved better the structure of the task. Future work should also investigate the impact of this approach on knowledge injection systems, as for instance our attention-based model presented in section 13.3.3.

13.3.5. Vertical Augmentation: Towards Exploring the Depth of Common Sense Reasoning

In past work [51, 52], we observed that it's a signature characteristic of computational lexical resources to have high *concept density*, namely large number of interconnected classes, but low *constraint density*, that is a limited number of axioms used to formalize the semantic relations holding between nodes. Although this distinction was made before the advent of Big Data and the consequent explosion of machine learning-inspired AI, the underlying observation remains valid today, if not even more relevant, when applied to the current landscape of lexical/semantic resources: in particular, when we consider commonsense knowledge graphs, the attention is mostly on size and not on the level of formalization. The explanation is manifold: 1) since the vast majority of resources is based on triple-like structures, the constraint options are bound to the expressivity of the RDF language [53]; 2) crowd-sourcing played a key role in building many seminal resources like ConceptNet, which implies that complying to strict formalization rules was not a design requirement; 3) deep learning represents the mainstream approach to solve a plethora of downstream tasks, as the popularity of language models in NLP testifies, and only very large-scale knowledge resources can be leveraged by deep neural networks³. The body of work presented in the previous sections, focused on horizontal augmentation of language models, can be conceived as an example of what was outlined in point 3) above: we exploited the massive scale of general facts available in state-the-art knowledge graphs, in orchestration with most advanced NLM approaches. However, as claimed in section 13.2, improving models performance in Q/A tasks also depends on enabling specific forms of knowledge-based reasoning (**vertical augmentation**): thus, introducing fine-grained constraints in knowledge graphs becomes an essential step towards robustness and generalizability. But how can this enhancement be implemented if, as stated in point 1), we are dealing with semantic frameworks defined in terms of

³One of the main reasons why research on machine common sense reasoning is experiencing a *renaissance* period in AI is thanks to initiatives like the homonymous DARPA program [54], which promotes the integration between symbolic knowledge and sub-symbolic (neural) algorithms.

subject/predicate/object primitives?

13.3.5.1. Enhancing the coverage of spatial-commonsense knowledge

In a recent publication [43], we identified 13 dimensions shared by the most relevant common sense knowledge graphs in the state of the art (see section 13.3.1). In particular, our study shows that the temporal and intentional dimensions are beneficial for reasoning on Q/A downstream tasks, while lexical information have little impact.⁴ In some cases, such as the spatial dimension, the relations used by knowledge graphs are too generic to help in the downstream tasks. For instance, ConceptNet realizes the spatial dimension through the *AtLocation* edge: when we consider spatially-relevant questions from the dataset presented in section 13.3.2, knowledge about the generic location of entities is clearly insufficient to answer questions that require reasoning over position, orientation, motion. As an example, in PIQA, no language model can answer the question ‘how do you wear a shawl’, with candidates ‘place it *on* your shoulders’ and ‘place it *under* your shoulders’, just by acquiring a triple ‘shawl *AtLocation* shoulders’ from ConceptNet. It is irrelevant whether we use an attention-based injection method (13.3.3), a model pre-trained on synthetically-generated questions and answers (13.3.4), or an alternative approach: models would need to be provided with a triple that includes the specific spatial relation *on*, connecting the concept of ‘shawl’ with the concept of ‘shoulders’, or some related inference.⁵ Visual Genome, one of the federated resources included in CSKG (see section 13.3.1), can help to combat this problem, as it supports a rich set of specific spatial relations. In particular, we propose the following procedure to perform **vertical augmentation** of language models in spatial commonsense Q/A tasks using Visual Genome:

- **Step 1** Extract triples that include spatial relations from Visual Genome. We only consider triples that appear more than three times in the whole dataset as common knowledge (following the same method used in [55]).
- **Step 2** Generate synthetic questions and answers for each triple, and pre-train RoBERTa-Large and GPT-2, following the method mentioned in section 13.3.4. Each sentence must have all the components of the triple in the correct order (i.e., [HEAD]—[RELATION]—[TAIL]), to be considered as a valid linguistic representation of the triple. In general, for generating questions, we convert each sentence to a question by masking the relevant spatial relation (Ex. *Bike parked near the trees* is converted to *Bike parked [MASK] the trees.*)
- **Step 3** To pre-train the language models in an adequate manner, we need to have fair and challenging answer options, i.e. candidates that exemplify which spatial relations can exist between two entities and which cannot. To this end, we categorized all spatial relations in Visual Genome into spatial classes (for instance, *above*, *over*, *up*, *top*, *overhead*, *north*, *upside* are classified using the category *ABOVE*). These classes help us to consolidate triples on the basis of their most salient spatial information. For instance, for the question “The pedestrian [MASK]

⁴For more details we refer the reader to the original article.

⁵For instance, a model can be provided with information that jackets are typically worn on shoulders, and learn to generalize that garments which share similar characteristics, like shawls, should also be worn on the shoulders, and not under them (like, for instance, corsets).

the street”, [Pedestrian, On, Street], [Pedestrian, Above, Street] are unified using ABOVE as spatial category.

- **Step 4** Further pre-train RoBERTa-Large and GPT-2 (already pre-trained on CSKG) on the spatial Q/A data synthetically generated as described in steps 1-3.
- **Step 5** Select a subset of spatially-relevant Q/A pairs from the commonsense benchmarks, using a combination of extraction techniques, including a constituency parser and OIE (Open Information Extraction) models [56]. Table 5 provides the number of extracted examples from the Dev set of each benchmark.
- **Step 6** Evaluate performance both globally, i.e., for the whole dataset under consideration, and locally, i.e., for the partitions that contain only spatially relevant questions.

Table 5. Number of examples with spatial information from different benchmarks

Dataset	Dev. size	Extracted spatial examples
CommonsenseQA [28]	1221	800
PhysicalIQA [29]	1838	1209
SocialIQA [16]	1954	1142
WinoGrande [30]	1267	431
aNLI [27]	1532	1077

Regarding the evaluation of the approach (**Step 6**), two observations can be made at the time of this publication: preliminary results show that 1) pre-training language models on specific spatial knowledge does not alter the general commonsense knowledge gained from pre-training on CSKG; 2) a slight improvement (+2%) is observed only for the SIQA dataset, which indicates that the knowledge extracted from Visual Genome is substantially misaligned with the spatial domain partition of the other benchmarks (a phenomenon also reported in [17, 57]). Due to the early stage of this research work on vertical augmentation, a thorough error analysis cannot be reported here. However, limited random sampling seem to corroborate the anticipated benefits of our approach. For instance, spatial knowledge from Visual Genome helped the model to correctly answer the PIQA question *How do you make a plain toast?*, with [Option 1]: “Gather a slice of bread and a toaster, place the bread *on the top of the toaster* and push the button down”; and [Option 2]: “Gather a slice of bread and a toaster, place the bread *into the toaster* and push the button down”.

The procedure described in this section exploits pre-existing information in the Visual Genome resource and does not prescribe any structural modification to the spatial triples of CSKG. However, the spatial classes mentioned in **Step 3** could be used to upgrade the spatial relations currently implemented in CSKG. Furthermore, we hypothesize that, by modifying Visual Genome through additional annotations based on formally-characterized spatial relations, reasoning capabilities can be advanced further. In particular, as demonstrated by [57], it is possible to obtain high quality textual annotations of topological relations like *connected*, *disconnected*, *overlap*, *in* and *touch*⁶, and boost the performance of models in datasets like bAbI [15] (see section 13.2).

⁶See [58, 59] for an overview of qualitative spatial reasoning, and [60] for an introduction to methods of extraction of spatial relations from text.

13.4. Conclusions

As much as the ‘capability to generalize’ has been considered, for long time, one of the requisites that an artificial system should satisfy in order to exhibit ‘intelligence’ [61], building generalizable systems represents all but a solved problem in today’s AI research. In this chapter, we demonstrated that, within the scope of commonsense question answering, neural language models can achieve robustness within and across downstream tasks, when integrated with large-scale knowledge graphs, according to mechanisms like attention-based injection and pre-training with synthetic questions and answers generated from knowledge graphs. We showed that, through **horizontal augmentation**, language models can access a vast trove of knowledge in the form of triples. Finally, we made the case for enhancing reasoning capabilities in neuro-symbolic systems by introducing more fine-grained semantic relations in the knowledge graphs they are combined with (**vertical augmentation**), outlining a procedure to test this hypothesis. Supported by our empirical investigations, and inspired by recent literature in integrating qualitative reasoning with neural models, we have provided the blueprints of a solution towards constructing generalizable neuro-symbolic systems for commonsense question answering.

13.5. Acknowledgements

The authors would like to thank you all the students and faculty who contributed, with their hard work and precious advice, to make this chapter possible: Monireh Ebrahimi, Sarah Masud Preum, Bin Zhang, Eric Nyberg, Pedro Szekely, Deborah L. McGuinness, and Yonatan Bisk.

References

- [1] Ma K, Francis J, Lu Q, et al. Towards generalizable neuro-symbolic systems for commonsense question answering. In: Proc. of the First Workshop on Commonsense Inference in Natural Language Processing; Nov.; 2019. p. 22–32.
- [2] Ma K, Ilievski F, Francis J, et al. Knowledge-driven data construction for zero-shot evaluation in commonsense question answering. In: Proceedings of the AAAI Conference on Artificial Intelligence; Vol. 35; 2021. p. 13507–13515.
- [3] Ma K, Ilievski F, Francis J, et al. Exploring strategies for generalizable commonsense reasoning with pre-trained models. arXiv preprint arXiv:210902837. 2021;.
- [4] Oltramari A, Francis J, Henson C, et al. Neuro-symbolic architectures for context understanding. arXiv preprint arXiv:200304707. 2020;.
- [5] Bauer L, Bansal M. Identify, align, and integrate: Matching knowledge graphs to commonsense reasoning tasks. In: Proc. of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume; Apr.; Online. Association for Computational Linguistics; 2021. p. 2259–2272. Available from: <https://www.aclweb.org/anthology/2021.eacl-main.192>.
- [6] Shwartz V, West P, Le Bras R, et al. Unsupervised commonsense question answering with self-talk. In: Proc. of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP); Nov.; Online. Association for Computational Linguistics; 2020. p. 4615–4629. Available from: <https://www.aclweb.org/anthology/2020.emnlp-main.373>.
- [7] Francis J, Kitamura N, Labelle F, et al. Core challenges in embodied vision-language planning. arXiv preprint arXiv:210613948. 2021;.
- [8] Lu J, Batra D, Parikh D, et al. Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. In: Wallach HM, Larochelle H, Beygelzimer A, et al., ed-

- itors. *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*; 2019. p. 13–23. Available from: <https://proceedings.neurips.cc/paper/2019/hash/c74d97b01eae257e44aa9d5bade97baf-Abstract.html>.
- [9] Majumdar A, Shrivastava A, Lee S, et al. Improving vision-and-language navigation with image-text pairs from the web. *CoRR*. 2020;abs/2004.14973. Available from: <https://arxiv.org/abs/2004.14973>.
 - [10] Floridi L, Chiriatti M. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*. 2020; 30(4):681–694.
 - [11] Boratko M, Li XL, Das R, et al. Protoqa: A question answering dataset for prototypical common-sense reasoning. *arXiv preprint arXiv:200500771*. 2020;.
 - [12] Ushio A, Espinosa-Anke L, Schockaert S, et al. Bert is to nlp what alexnet is to cv: Can pre-trained language models identify analogies? *arXiv preprint arXiv:210504949*. 2021;.
 - [13] Ettinger A. What bert is not: Lessons from a new suite of psycholinguistic diagnostics for language models. *Transactions of the Association for Computational Linguistics*. 2020;8:34–48.
 - [14] Devlin J, Chang MW, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proc. of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*; Jun.; Minneapolis, Minnesota. Association for Computational Linguistics; 2019. p. 4171–4186. Available from: <https://www.aclweb.org/anthology/N19-1423>.
 - [15] Weston J, Bordes A, Chopra S, et al. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:150205698*. 2015;.
 - [16] Sap M, Rashkin H, Chen D, et al. Social IQa: Commonsense reasoning about social interactions. In: *Proc. of EMNLP-IJCNLP*; Nov.; 2019. p. 4463–4473.
 - [17] Ma K, Ilievski F, Francis J, et al. Knowledge-driven data construction for zero-shot evaluation in commonsense question answering. *Proc of the 35th AAAI Conference on Artificial Intelligence (AAAI-21)*. 2021;.
 - [18] Miller GA. *Wordnet: An electronic lexical database*. MIT press; 1998.
 - [19] Liu H, Singh P. Conceptnet—a practical commonsense reasoning tool-kit. *BT technology journal*. 2004; 22(4):211–226.
 - [20] Sap M, Le Bras R, Allaway E, et al. Atomic: An atlas of machine commonsense for if-then reasoning. In: *Proc. of the AAAI Conference on Artificial Intelligence*; Vol. 33; 2019. p. 3027–3035.
 - [21] Krishna R, Zhu Y, Groth O, et al. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *arXiv preprint arXiv:160207332*. 2016;.
 - [22] Ilievski F, Szekely P, Zhang B. Cskg: The commonsense knowledge graph. In: *European Semantic Web Conference*; Springer; 2021. p. 680–696.
 - [23] Vrandečić D. Wikidata: A new platform for collaborative data collection. In: *Proc. of the 21st international conference on world wide web*; 2012. p. 1063–1064.
 - [24] Ilievski F, Szekely P, Schwabe D. Commonsense knowledge in wikidata. *ISWC Wikidata workshop*. 2020;.
 - [25] Gordon A, Kozareva Z, Roemmele M. Semeval-2012 task 7: Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In: ** SEM 2012: The First Joint Conference on Lexical and Computational Semantics—Volume 1: Proceedings of the main conference and the shared task, and Volume 2: Proceedings of the Sixth International Workshop on Semantic Evaluation (SemEval 2012)*; 2012. p. 394–398.
 - [26] Zellers R, Holtzman A, Bisk Y, et al. HellaSwag: Can a machine really finish your sentence? In: *Proc. of the 57th Annual Meeting of the Association for Computational Linguistics*; Jul.; Florence, Italy. Association for Computational Linguistics; 2019. p. 4791–4800. Available from: <https://www.aclweb.org/anthology/P19-1472>.
 - [27] Bhagavatula C, Bras RL, Malaviya C, et al. Abductive commonsense reasoning. *arXiv preprint arXiv:190805739*. 2019;.
 - [28] Talmor A, Herzig J, Lourie N, et al. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In: *Proc. of NAACL*; Jun.; 2019. p. 4149–4158.
 - [29] Bisk Y, Zellers R, Bras RL, et al. PIQA: Reasoning about Physical Commonsense in Natural Language. In: *Proc. of Thirty-Fourth AAAI Conference on Artificial Intelligence*; 2020. p. 7432–7439.
 - [30] Sakaguchi K, Bras RL, Bhagavatula C, et al. Winogrande: An adversarial winograd schema challenge

- at scale. arXiv preprint arXiv:190710641. 2019;.
- [31] Bauer L, Wang Y, Bansal M. Commonsense for generative multi-hop question answering tasks. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing; Oct.-Nov.; Brussels, Belgium. Association for Computational Linguistics; 2018. p. 4220–4230. Available from: <https://www.aclweb.org/anthology/D18-1454>.
 - [32] Miller A, Fisch A, Dodge J, et al. Key-value memory networks for directly reading documents. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing; Nov.; Austin, Texas. Association for Computational Linguistics; 2016. p. 1400–1409. Available from: <https://www.aclweb.org/anthology/D16-1147>.
 - [33] Mihaylov T, Frank A. Knowledgeable reader: Enhancing cloze-style reading comprehension with external commonsense knowledge. In: Proc. of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers); Jul.; Melbourne, Australia. Association for Computational Linguistics; 2018. p. 821–832. Available from: <https://www.aclweb.org/anthology/P18-1076>.
 - [34] Zhong W, Tang D, Duan N, et al. Improving question answering by commonsense-based pre-training. CoRR. 2018;abs/1809.03568. Available from: <http://arxiv.org/abs/1809.03568>.
 - [35] Kipf TN, Welling M. Semi-supervised classification with graph convolutional networks. CoRR. 2016;abs/1609.02907. Available from: <http://arxiv.org/abs/1609.02907>.
 - [36] Lin BY, Chen X, Chen J, et al. Kagnet: Knowledge-aware graph networks for commonsense reasoning. ArXiv. 2019;abs/1909.02151.
 - [37] Ran Q, Li P, Hu W, et al. Option comparison network for multiple-choice reading comprehension. CoRR. 2019;abs/1903.03033. Available from: <http://arxiv.org/abs/1903.03033>.
 - [38] Radford A, Wu J, Child R, et al. Language models are unsupervised multitask learners. 2019;.
 - [39] Yang Z, Dai Z, Yang Y, et al. Xlnet: Generalized autoregressive pretraining for language understanding ; 2019. Cite arxiv:1906.08237Comment: Pretrained models and code are available at <https://github.com/zihangdai/xlnet>; Available from: <http://arxiv.org/abs/1906.08237>.
 - [40] Singh P, Lin T, Mueller ET, et al. Open mind common sense: Knowledge acquisition from the general public. In: On the Move to Meaningful Internet Systems, 2002 - DOA/CoopIS/ODBASE 2002 Confederated International Conferences DOA, CoopIS and ODBASE 2002; Berlin, Heidelberg. Springer-Verlag; 2002. p. 1223–1237. Available from: <http://dl.acm.org/citation.cfm?id=646748.701499>.
 - [41] Bosselut A, Rashkin H, Sap M, et al. COMET: Commonsense transformers for automatic knowledge graph construction. In: Proc. of the 57th Annual Meeting of the Association for Computational Linguistics; Florence, Italy. Association for Computational Linguistics; 2019. p. 4762–4779. Available from: <https://www.aclweb.org/anthology/P19-1470>.
 - [42] Davis E. Representations of commonsense knowledge. Morgan Kaufmann; 2014.
 - [43] Ilievski F, Oltramari A, Ma K, et al. Dimensions of commonsense knowledge. arXiv preprint arXiv:210104640. 2021;.
 - [44] Banerjee P, Baral C. Self-supervised knowledge triplet learning for zero-shot question answering. arXiv preprint arXiv:200500316. 2020;.
 - [45] Shwartz V, West P, Bras RL, et al. Unsupervised commonsense question answering with self-talk. ArXiv. 2020;abs/2004.05483.
 - [46] Zhou X, Zhang Y, Cui L, et al. Evaluating commonsense in pre-trained language learners models. In: Proc. of the AAAI Conference on Artificial Intelligence; Vol. 34; 2020. p. 9733–9740.
 - [47] Tamborrino A, Pellicano N, Pannier B, et al. Pre-training is (almost) all you need: An application to commonsense reasoning. arXiv preprint arXiv:200414074. 2020;.
 - [48] Mostafazadeh N, Chambers N, He X, et al. A corpus and cloze evaluation for deeper understanding of commonsense stories. In: Proc. of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies; Jun.; San Diego, California. Association for Computational Linguistics; 2016. p. 839–849. Available from: <https://www.aclweb.org/anthology/N16-1098>.
 - [49] Bosselut A, Choi Y. Dynamic knowledge graph construction for zero-shot commonsense question answering. ArXiv. 2019;abs/1911.03876.
 - [50] Banerjee P, Baral C. Self-supervised knowledge triplet learning for zero-shot question answering. ArXiv. 2020;abs/2005.00316.
 - [51] Prévot L, Borgo S, Oltramari A. Interfacing ontologies and lexical resources. In: Proc. of OntoLex 2005- Ontologies and Lexical Resources; 2005.
 - [52] Huang CR, Calzolari N, Gangemi A, et al. Ontology and the lexicon: A natural language processing

- perspective. Cambridge University Press; 2010.
- [53] Pan JZ. Resource description framework. In: Handbook on ontologies. Springer; 2009. p. 71–90.
 - [54] Gunning D. Machine common sense concept paper. arXiv preprint arXiv:181007528. 2018;.
 - [55] Yang W, Wang X, Farhadi A, et al. Visual semantic navigation using scene priors. arXiv preprint arXiv:181006543. 2018;.
 - [56] Angeli G, Premkumar MJJ, Manning CD. Leveraging linguistic structure for open domain information extraction. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers); 2015. p. 344–354.
 - [57] Mirzaee R, Rajaby Faghihi H, Ning Q, et al. SPARTQA: A textual question answering benchmark for spatial reasoning. In: Proc. of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies; Jun.; Online. Association for Computational Linguistics; 2021. p. 4582–4598. Available from: <https://aclanthology.org/2021.naacl-main.364>.
 - [58] Randell DA, Cui Z, Cohn AG. A spatial logic based on regions and connection. KR. 1992;92:165–176.
 - [59] Renz J, Nebel B. Qualitative spatial reasoning using constraint calculi. In: Handbook of spatial logics. Springer; 2007. p. 161–215.
 - [60] Kordjamshidi P, Moens MF, van Otterlo M. Spatial role labeling: Task definition and annotation scheme. In: Proc. of the Seventh conference on International Language Resources and Evaluation (LREC'10); European Language Resources Association (ELRA); 2010. p. 413–420.
 - [61] Simon HA. Bounded rationality. In: Utility and probability. Springer; 1990. p. 15–18.

Chapter 14

Combining Probabilistic Logic and Deep Learning for Self-Supervised Learning

Hoifung Poon, Microsoft Research, Redmond WA, USA

Hai Wang, JD Finance America Corporation, Mountain View CA, USA

Hunter Lang, MIT CSAIL, Cambridge MA, USA

Deep learning has proven effective for various application tasks, but its applicability is limited by the reliance on annotated examples. Self-supervised learning has emerged as a promising direction to alleviate the supervision bottleneck, but existing work focuses on leveraging co-occurrences in unlabeled data for task-agnostic representation learning, as exemplified by masked language model pretraining. In this chapter, we explore *task-specific self-supervision*, which leverages domain knowledge to automatically annotate noisy training examples for end applications, either by introducing labeling functions for annotating individual instances, or by imposing constraints over interdependent label decisions. We first present *deep probabilistic logic* (DPL), which offers a unifying framework for task-specific self-supervision by composing probabilistic logic with deep learning. DPL represents unknown labels as latent variables and incorporates diverse self-supervision using probabilistic logic to train a deep neural network end-to-end using variational EM. Next, we present *self-supervised self-supervision* (S4), which adds to DPL the capability to learn new self-supervision automatically. Starting from an initial seed self-supervision, S4 iteratively uses the deep neural network to propose new self supervision. These are either added directly (a form of *structured self-training*) or verified by a human expert (as in *feature-based active learning*). Experiments on real-world applications such as biomedical machine reading and various text classification tasks show that task-specific self-supervision can effectively leverage domain expertise and often match the accuracy of supervised methods with a tiny fraction of human effort.

14.1. Introduction

Machine learning has made great strides in enhancing model sophistication and learning efficacy, as exemplified by recent advances in deep learning [1]. However, contemporary supervised learning techniques require a large amount of labeled data, which is expensive and time-consuming to produce. This problem is particularly acute in specialized domains like biomedicine, where crowdsourcing is difficult to apply. Self-supervised learning has emerged as a promising paradigm to overcome the annotation bottleneck by automatically generating noisy training examples from unlabeled data. However, existing work predominantly focuses on *task-agnostic representation learning* by modeling co-occurrences

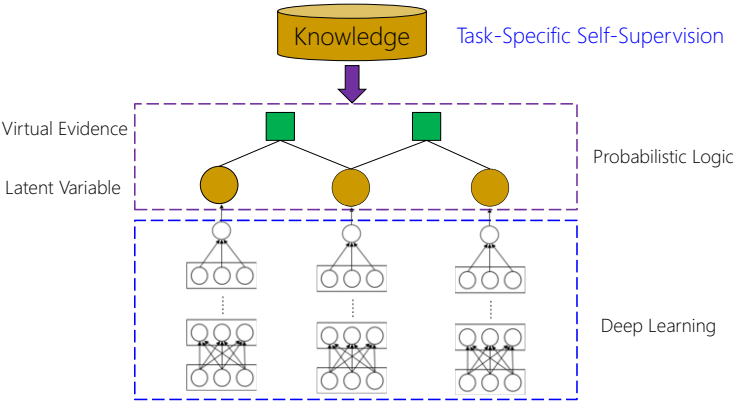


Figure 1. Deep Probabilistic Logic: A general framework for combining task-specific self-supervision strategies by composing probabilistic logic with deep learning. Learning amounts to maximizing conditional likelihood of virtual evidence given input by summing up latent label decisions.

in unlabeled data, as exemplified in masked language model pretraining [2]. While successful, it still requires manual annotation of task-specific labeled examples to fine-tune pretrained models for end applications.

In this chapter, we explore *task-specific self-supervised learning*, which can directly generate training examples for end applications. The key insight is to *introduce a general framework that can best leverage human expert bandwidth by accepting and leveraging alternative forms of supervision*. For example, with relatively small effort, domain experts can produce labeling functions [3, 4] and joint inference constraints [5, 6, 7, 8]. Additionally, there are ample available resources such as ontologies and knowledge bases that can be leveraged to automatically annotate training examples in unlabeled data [9, 10]. Using such self-specified supervision templates, we can automatically produce training examples at scale from unlabeled data.

A central challenge in leveraging such diverse forms of self-supervision is that they are inherently noisy and may be contradictory with each other. We present deep probabilistic logic (DPL), which provides a unifying framework for task-specific self-supervision by composing probabilistic logic with deep learning [11]. DPL models label decisions as latent variables, represents prior knowledge on their relations using Markov logic (weighted first-order logical formulas) [12], and alternates between learning a deep neural network for the end task and refining uncertain formula weights for self-supervision, using variational EM. Probabilistic logic offers a principled way to combine noisy and inconsistent self-supervision. Feedback from deep learning helps resolve noise and inconsistency in the initial self-supervision. Experiments on biomedical machine reading show that distant supervision, data programming, and joint inference can be seamlessly combined in DPL to substantially improve machine reading accuracy, without requiring any manually labeled examples.

DPL still requires human experts to manually specify self-supervision. We next present *Self-Supervised Self-Supervision (S4)*, which is a general framework for learning to add new self-supervision, by extending DPL with structure learning and active learning capabilities (see Figure 2) [13]. S4 first runs DPL using the pre-specified seed self-supervision (represented as a Markov logic network), then iteratively proposes new self-supervision (weighted logical formula) using the trained deep neural network, and

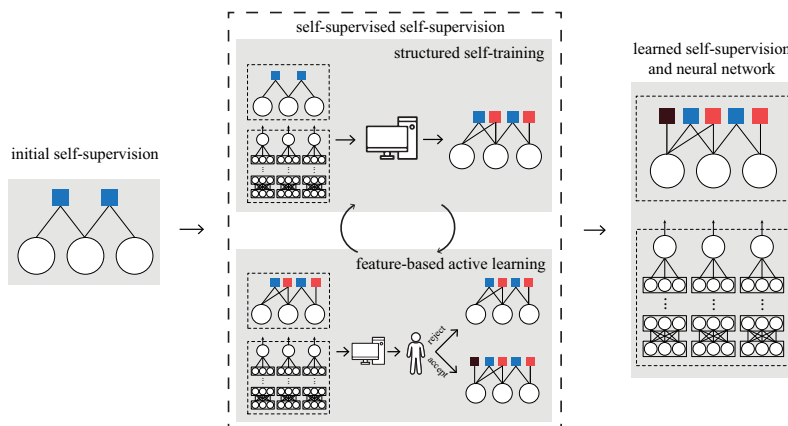


Figure 2. Self-Supervised Self-Supervision (S4): S4 builds on deep probabilistic logic and uses probabilistic logic to represent self-supervision for learning a deep neural network for the end prediction task. Starting from pre-specified self-supervision, S4 interleaves structure learning and active learning steps to introduce new self-supervision for training the neural network and refining the graphical model parameters. Self-supervision factors from initialization, structure learning, and active learning are shown in blue, red, and black, respectively.

determines whether to add it directly to the self-supervision Markov logic network or ask a human expert to vet it. The former can be viewed as *structured self-training*, which generalizes self-training [14] by adding not only individual labels but also arbitrary probabilistic factors over them. The latter subsumes *feature-based active learning* [15] with arbitrary features expressible using probabilistic logic. By combining the two in a unified framework, S4 can leverage both paradigms for generating new self-supervision and subsume many related approaches as special cases. Using transformer-based models [16] for the deep neural network in DPL, we explore various self-supervision proposal mechanisms based on neural attention and label entropy. Our method can learn to propose both unary potential factors over individual labels and joint-inference factors over multiple labels. Experiments on various text classification tasks show that S4 can substantially improve over the seed self-supervision by proposing new self-supervision, often matching the accuracy of fully supervised systems with a fraction of human effort.

We first review related work in self-supervised learning and combining probabilistic logic with deep learning. We then present deep probabilistic logic and self-supervised self-supervision in details. We conclude by discussing several exciting future directions. We focus on natural language processing (NLP) applications in this chapter, but our methods are potentially applicable to other domains and tasks.

14.2. Task-Specific Self-Supervision

Techniques to compensate for the lack of direct supervision come in many names and forms [10, 3, 17, 18, 11]. Self-supervision has emerged as an encompassing paradigm that views these as instances of using self-specified templates to generate noisy labeled examples on unlabeled data. The name *self-supervision* is closely related to *self-training* [14], which bootstraps from a supervised classifier, uses it to annotate unlabeled instances, and iteratively adds confident labels to retrain the classifier. *Task-agnostic self-supervision* generalizes word embedding and language modeling by learning to predict self-specified

masked tokens, as exemplified by recent pretraining methods such as BERT [2]. In this chapter, we focus on *task-specific self-supervision* and use pretrained models as a building block for task-specific learning. Below, we review three popular forms of task-specific self-supervision: distant supervision, data programming, and joint inference.

Distant supervision This paradigm was first introduced for binary relation extraction [9, 10]. In its simplest form, distant supervision generates a positive example if an entity pair with a known relation co-occurs in a sentence, and samples negative examples from co-occurring entity pairs not known to have the given relation. It has recently been extended to cross-sentence relation extraction [19, 20]. In principle, one simply looks beyond single sentences for co-occurring entity pairs. However, this can introduce many false positives and prior work used a small sliding window and filtering (minimal-span) to mitigate training noise. Even so, accuracy is relatively low. Both [19] and [20] used ontology-based string matching for entity linking, which also incurs many false positives, as entity mentions can be highly ambiguous (e.g., PDF and AAAS are gene names). Distant supervision for entity linking is relatively underexplored, and prior work generally focuses on Freebase entities, where links to the corresponding Wikipedia articles are available for learning [21].

Data Programming Instead of annotated examples, domain experts are asked to produce labeling functions, each of which assigns a label to an instance if the input satisfies certain conditions, often specified by simple rules [22]. This paradigm is useful for semantic tasks, as high-precision text-based rules are often easy to come by. However, there is no guarantee on broad coverage, and labeling functions are still noisy and may contradict with each other. The common denoising strategy assumes that labeling functions make random mistakes, and focuses on estimating their accuracy and correlation [3, 17, 23]. A more sophisticated strategy also models instance-level labels and uses instance embedding to estimate instance-level weight for each labeling function [24].

Joint Inference Distant supervision and data programming focus on infusing task-specific self-supervision on individual labels. Additionally, there is rich linguistic and domain knowledge that does not specify values for individual labels, but imposes constraints on their joint distribution. For example, if two mentions are coreferent, they should agree on entity properties [6]. There is a rich literature on joint inference for NLP applications. Notable methodologies include constraint-driven learning [5], general expectation [7], posterior regularization [8], and probabilistic logic [6]. Constraints can be imposed on relational instances or on model expectations. Learning and inference are often tailored for each approach, including beam search, primal-dual optimization, weighted satisfiability solvers, etc. Recently, joint inference has also been used in denoising distant supervision. Instead of labeling all co-occurrences of an entity pair with a known relation as positive examples, one only assumes that at least one instance is positive [25, 26].

Existing self-supervision paradigms are typically special cases of deep probabilistic logic (DPL). E.g., data programming admit only self-supervision for individual instances (labeling functions or their correlations). *Anchor learning* [27] is an earlier form of data programming that, while more restricted, allows for stronger theoretical learning guarantees. *Prototype learning* is an even earlier special case with labeling functions provided by “prototypes” [28, 29]. Using Markov logic to model self-supervision, DPL can incorporate arbitrary prior beliefs on both individual labels and their interdependencies, thereby unleashing the full power of *joint inference* [30, 31, 32, 33] to amplify and propagate self-supervision signals.

Self-supervised self-supervision (S4) further extends DPL with structure learning capability. Most structure learning techniques are developed for the supervised setting, where structure search is guided by labeled examples [34, 35]. Moreover, traditional relational learning induces deterministic rules and is susceptible to noise and uncertainty. *Bootstrapping* learning is one of the earliest and simplest self-supervision methods with some rule-learning capability, by alternating between inducing characteristic contextual patterns and classifying instances [36, 37]. The pattern classes are limited and only applicable to special problems (e.g., “A such as B” to find ISA relations). Most importantly, they lack a coherent probabilistic formulation and may suffer catastrophic *semantic drift* due to ambiguous patterns (e.g., “cookie” as food or compute use). [38] and [39] designed a more sophisticated rule induction approach, but their method uses deterministic rules and may be sensitive to noise and ambiguity. Recently, Snuba [40] extends the data programming framework by automatically adding new labeling functions, but like prior data programming methods, their self-supervision framework is limited to modeling prior beliefs on individual instances. Their method also requires access to a small number of labeled examples to score new labeling functions.

Another significant advance in S4 is by extending DPL with the capability to conduct *structured active learning*, where human experts are asked to verify arbitrary virtual evidences, rather than a label decision. Note that by admitting joint inference factors, this is more general than prior use of *feature-based active learning*, which focuses on per-instance features [15]. As our experiments show, interleaving structured self-training learning and structured active learning results in substantial gains, and provides the best use of precious human bandwidth. [41] previously considered active structure learning in the context of Bayesian networks. Anchor learning [27] can also suggest new self-supervision for human review. Darwin [42] incorporates active learning for verifying proposed rules, but it doesn’t conduct structure learning, and like Snuba and other data programming methods, it only models individual instances.

14.3. Combining Probabilistic Logic with Deep Learning

Probabilistic logic combines logic’s expressive power with the capability of graphical models to handle uncertainty. A representative example is Markov logic [12], which defines a probability distribution using weighted first-order logical formulas as templates for a Markov model. Given random variables X for a problem domain, Markov logic uses a set of weighted first-order logical formulas $(w_i, f_i) : i$ to define a joint probability distribution $P(X) \propto \exp \sum_i w_i \cdot f_i(X)$. Intuitively, the weight w_i correlates with how likely formula f_i holds true. A logical constraint is a special case when w_i is set to ∞ (or a sufficiently large number). Probabilistic logic has been applied to incorporating joint inference and other task-specific self-supervision for various NLP tasks [43, 6, 44], but its expressive power comes at a price: learning and inference are generally intractable, and end-to-end modeling often requires heavy approximation [45].

Recently, there has been increasing interest in combining probabilistic logic with deep learning, thereby harnessing probabilistic logic’s capability in representing and reasoning with knowledge, as well as leveraging deep learning’s strength in distilling complex patterns from high-dimension data. Most existing work focuses on incorporating probabilistic logic in inference, typically by replacing discrete logical predicates with neural representation and jointly learning global probabilistic logic parametrization and

local neural networks, as exemplified by differentiable proving [46] and DeepProbLog [47]. By contrast, we leverage probabilistic logic to provide self-supervision for deep learning. These two approaches are complementary and can be combined, i.e., by using probabilistic logic to infuse prior knowledge for both learning and inference, as explored in recent work on knowledge graph embedding [48].

Deep generative models also combine deep learning with probabilistic models, but focus on uncovering latent factors to support generative modeling and semi-supervised learning [49, 50]. Knowledge infusion is limited to introducing structures among latent variables (e.g., Markov chain) [51]. Deep probabilistic programming provides a flexible interface for exploring such composition [52]. In deep probabilistic logic and self-supervised self-supervision, we instead combine a discriminative neural network predictor with a generative self-supervision model based on Markov logic, and can fully leverage their respective capabilities to advance co-learning [53, 54]. Deep neural networks also provide a powerful feature-induction engine to support structure learning and active learning.

14.4. Deep Probabilistic Logic: A Unifying Framework for Self-Supervision

In this section, we introduce deep probabilistic logic (DPL) as a unifying framework for task-specific self-supervision. Label decisions are modeled as latent variables. Self-supervision is represented as generalized virtual evidence, and learning maximizes the conditional likelihood of virtual evidence given input. The use of probabilistic logic is limited to modeling self-supervision, leaving end-to-end modeling to the deep neural network for end prediction. This alleviates the computational challenges in using probabilistic logic end-to-end, while leveraging the strength of deep learning in distilling complex patterns from high-dimension data.

Infusing knowledge in neural network training is a long-standing challenge in deep learning [55]. [56, 57] first used logical rules to help train a convolutional neural network for sentiment analysis. DPL draws inspiration from their approach, but is more general and theoretically well-founded. [56, 57] focused on supervised learning and the logical rules were introduced to augment labeled examples via posterior regularization [8]. DPL can incorporate both direct and indirect supervision, including posterior regularization and other forms of self-supervision. Like DPL, [57] also refined uncertain weights of logical rules, but they did it in a heuristic way by appealing to symmetry with standard posterior regularization. We provide a novel problem formulation using generalized virtual evidence, which shows that their heuristic is a special case of variational EM and opens up opportunities for other optimization strategies.

We first review the idea of virtual evidence and show how it can be generalized to represent any form of self-supervision. We then formulate the learning objective and show how it can be optimized using variational EM.

14.4.1. Deep Probabilistic Logic

Given a prediction task, let $\mathcal{X}$ denote the set of possible inputs and $\mathcal{Y}$ the set of possible outputs. The goal is to train a prediction module $\Psi(x, y)$ that scores output y given input x . We assume that $\Psi(x, y)$ defines the conditional probability $P(y|x)$ using a deep neural network with a softmax layer at the top. Let $X = (X_1, \dots, X_N)$ denote a sequence of inputs and $Y = (Y_1, \dots, Y_N)$ the corresponding outputs. We consider the setting where Y are unobserved, and $\Psi(x, y)$ is learned using task-specific self-supervision.

Virtual evidence Pearl [58] first introduced the notion of virtual evidence, which has been used to incorporate label preference in semi-supervised learning [59, 60, 61] and grounded learning [62]. Suppose we have a prior belief on the value of y . It can be represented by introducing a binary variable v as a dependent of y such that $P(v = 1|y = l)$ is proportional to the prior belief of $y = l$. $v = 1$ is thus an observed evidence that imposes soft constraints over y . Direct supervision (i.e., observing the true label) for y is a special case when the belief is concentrated on a specific value $y = l^*$ (i.e., $P(v = 1|y = l) = 0$ for any $l \neq l^*$). The virtual evidence v can be viewed as a reified variable for a potential function $\Phi(y) \propto P(v = 1|y)$. This enables us to generalize virtual evidence to arbitrary potential functions $\Phi(X, Y)$. In the rest of the paper, we will simply refer to the potential functions as virtual evidences, without introducing the reified variables explicitly.

DPL Let $K = (\Phi_1, \dots, \Phi_V)$ be a set of virtual evidence derived from prior knowledge. DPL comprises of a supervision module over K and a prediction module over all input-output pairs (Figure 1), and defines a probability distribution:

$$P(K, Y|X) \propto \prod_v \Phi_v(X, Y) \cdot \prod_i \Psi(X_i, Y_i)$$

Without loss of generality, we assume that virtual evidences are log-linear factors, which can be compactly represented by weighted first-order logical formulas [12]. Namely, $\Phi_v(X, Y) = \exp(w_v \cdot f_v(X, Y))$, where $f_v(X, Y)$ is a binary feature represented by a first-order logical formula. A hard constraint is the special case when $w_v = \infty$ (in practice, it suffices to set it to a large number, e.g., 10). In prior use of virtual evidence, w_v 's are generally pre-determined from prior knowledge. However, this may be suboptimal. Therefore, we consider a general Bayesian learning setting where each w_v is drawn from a pre-specified prior distribution $w_v \sim P(w_v|\alpha_v)$. Fixed w_v amounts to the special case when the prior is concentrated on the preset value. For uncertain w_v 's, we can compute their maximum a posteriori (MAP) estimates and/or quantify the uncertainty.

Distant supervision Virtual evidence for distant supervision is similar to that for direct supervision. For example, for relation extraction, distant supervision from a knowledge base of known relations will set $f_{KB}(X_i, Y_i) = \mathbb{I}[\text{In-KB}(X_i, r) \wedge Y_i = r]$, where $\text{In-KB}(X_i, r)$ is true iff the entity tuple in X_i is known to have relation r in the KB.

Data programming Virtual evidence for data programming is similar to that for distant supervision: $f_L(X_i, Y_i) = \mathbb{I}[L(X_i) = Y_i]$, where $L(X_i)$ is a labeling function provided by domain experts. Labeling functions are usually high-precision rules, but errors are still common, and different functions may assign conflicting labels to an instance. Existing denoising strategies either assume that each function makes random errors independently, and resolves the conflicts by weighted votes [22], or attempts to learn and account for the dependencies between labeling functions [63]. In DPL, the former can be done by simply treating error probabilities as uncertain parameters and inferring them during learning, and the latter can be done with structure learning over the virtual evidences.

Joint inference Constraints on instances or model expectations can be imposed by introducing the corresponding virtual evidence [8] (Proposition 2.1). The weights can be set heuristically [5, 64, 6] or iteratively via primal-dual methods [8]. In addition to instance-level constraints, DPL can incorporate arbitrary high-order soft and hard constraints that capture the interdependencies among multiple instances. For example, identical mentions in proximity probably refer to the same entity, which is useful for resolving ambiguous mentions by leveraging their unambiguous coreferences (e.g., an

Algorithm 1 DPL Learning

Input: Virtual evidences $K = \Phi_{1:V}$, deep neural network Ψ , inputs $X = (X_1, \dots, X_N)$, unobserved outputs $Y = (Y_1, \dots, Y_N)$.

Output: Learned prediction module Ψ^*

Initialize: $\Phi^0 \sim \text{priors}$, $\Psi^0 \sim \text{uniform}$.

for $t = 1 : T$ **do**

$q^t(Y) \leftarrow P(Y | \Phi^{t-1}, \Psi^{t-1}, X)$ (approximately)

$\Phi^t \leftarrow \arg \min_{\Phi} D_{KL}(q^t(Y) \parallel \prod_v \Phi_{v(X,Y)})$

$\Psi^t \leftarrow \arg \min_{\Psi} D_{KL}(q^t(Y) \parallel \prod_i \Psi(X_i, Y_i))$

end

return $\Psi^* = \Psi^T$.

acronym in apposition of the full name). This can be represented by the virtual evidence $f_{\text{Joint}}(X_i, Y_i, X_j, Y_j) = \mathbb{I}[\text{Coref}(X_i, X_j) \wedge Y_i = Y_j]$, where $\text{Coref}(X_i, X_j)$ is true iff X_i and X_j are coreferences. Similarly, the common denoising strategy for distant supervision replaces the mention-level constraints with type-level constraints [25]. Suppose that $X_E \subset X$ contains all X_i 's with co-occurring entity tuple E . The new constraints simply impose that, for each E with known relation $r \in KB$, $Y_i = r$ for at least one $X_i \in X_E$. This can be represented by a high-order factor on $(X_i, Y_i : X_i \in X_E)$.

Parameter learning Learning in DPL maximizes the conditional likelihood of virtual evidences $P(K|X)$. We could directly optimize this objective by summing out the latent variable Y to compute the gradient and run backpropagation. Instead, in this work we opted for a modular approach using (variational) EM, since both the E- and M-steps reduce to standard inference and learning problems, and because summing out Y is often computationally intractable when using joint inference. See Algorithm 1.

In the E-step, we compute an approximation $q(Y)$ for $P(Y|K, X)$. Exact inference is generally intractable, but there is a plethora of approximate inference methods that can efficiently produce an estimate for $P(Y|K, X)$. In this work we chose to use loopy belief propagation (BP) [65]. For each virtual evidence factor f , BP computes a “pseudo-marginal” $q_f(Y)$, which is an estimate of $\mathbb{E}_{P(Y|K, X)}[f(X, Y)]$. In particular, BP also computes estimates $q_{BP}(Y_i) \approx \mathbb{E}_{P(Y|K, X)}[Y_i]$, the posterior marginal for each instance's label variable. We set $q(Y) = \prod_i q_{BP}(Y_i)$, the product of approximate marginals returned by BP. While this is not theoretically principled (e.g., mean-field iteration may compute a better factored approximation $q(Y)$), BP provides a good balance of speed and flexibility in dealing with complex, higher-order factors.

Note that inference with *high-order factors* of large size (i.e., factors involving many label variables Y_i) can still be challenging with BP. However, there is a rich body of literature for handling such structured factors in a principled way. For example, in our biomedical machine reading application, we alter the BP message passing schedule so that each at-least-one factor (for denoising distant supervision; see next subsection) will compute messages to its variables jointly by renormalizing their current marginal probabilities with noisy-or [66], which is a soft version of dual decomposition [67].

In the M-step, we treat the approximation $q(Y)$ as giving probabilistic label assignments to Y , and use those assignments to optimize Φ and Ψ via standard supervised learning. The M-step naturally decouples into two separate optimization

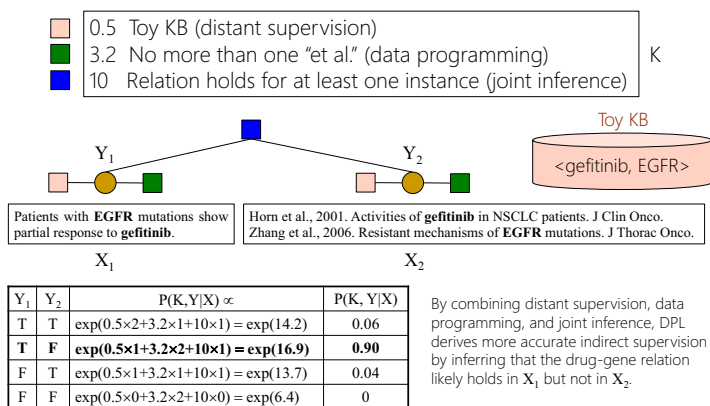


Figure 3. Example of DPL combining various task-specific self-supervision using probabilistic logic.

tasks, one for the supervision module Φ and one for the prediction module Ψ . For the prediction module (represented by a deep neural network), the optimization task $\arg \min_{\Psi} D_{KL}(q^t(Y) \parallel \prod_i \Psi(X_i, Y_i))$ reduces to standard deep learning with cross-entropy loss, where the label distribution over Y is given by our current estimate $q^t(Y_i)$ for the marginal $\mathbb{E}_{P(Y|K,X)}[Y_i]$. That is, we train the deep network Ψ to minimize the cross-entropy $-\frac{1}{N} \sum_i \sum_y q^t(Y_i = y) \log \Psi(X_i, y)$.

For the supervision module Φ , the optimization task

$$\arg \min_{\Phi} D_{KL}(q^t(Y) \parallel \prod_v \Phi_v(X, Y))$$

similarly reduces to standard parameter learning for log-linear models (i.e., learning all w_v 's that are not fixed). Given the estimate for $P(Y|K, X)$ produced in the E-step, this is a convex optimization problem with a unique (under certain conditions) global optimum. Here, we simply use gradient descent, with the partial derivative for w_v being $\mathbb{E}_{\Phi(Y,X)} [f_v(X, Y)] - \mathbb{E}_{q(Y)} [f_v(X, Y)]$. For a tied weight, the partial derivative will sum over all features that originate from the same template. The second expectation is easy to compute given $q(Y)$ estimated in the E-step. The first expectation, on the other hand, requires probabilistic inference in the graphical model *excluding* the prediction module Ψ . But this can again be computed using belief propagation, similar to the E-step, except that the messages are limited to factors within the supervision module (i.e., messages from Ψ are no longer included). Convergence is usually fast, upon which BP returns an approximation to $\mathbb{E}_{\Phi(Y,X)} [f_v(X, Y)]$.

As in the E-step, there are some nuances in the M-step when the BP approximation is not exact. [68] contains a more formal discussion of parameter learning with approximate inference methods. For a more theoretically principled approach, one could use tree-reweighted belief propagation in the M-step and mean-field iteration in the E-step. However, we find that our approach is easy to implement and work reasonably well in practice. As aforementioned, the parameter learning problem for the supervision module is much simpler than end-to-end learning with probabilistic logic, as it focuses on refining uncertain weights for task-specific self-supervision, rather than learning complex input patterns for label prediction (handled in deep learning).

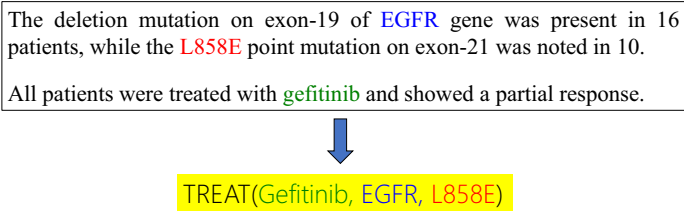


Figure 4. Example of cross-sentence relation extraction for precision cancer treatment.

Example Figure 3 shows a toy example on how DPL combines various task-specific self-supervision for predicting drug-gene interaction (e.g., gefitinib can be used to treat tumors with EGFR mutations). Self-supervision is modeled by probabilistic logic, which defines a joint probability distribution over latent labeling decisions for drug-gene mention pairs in unlabeled text. Here, distant supervision prefers classifying mention pairs of known relations, whereas the data programming formula opposes classifying instances resembling citations, and the joint inference formula ensures that at least one mention pair of a known relation is classified as positive. Formula weight signifies the confidence in the self-supervision, and can be refined iteratively along with the prediction module.

Handling label imbalance One challenge for distant supervision is that negative examples are often much more numerous. A common strategy is to subsample negative examples to attain a balanced dataset. In preliminary experiments, we found that this was often suboptimal, as many informative negative examples were excluded from training. Instead, we restored the balance by up-weighting positive examples. In DPL, an additional challenge is that the labels are probabilistic and change over iterations. In this paper, we simply used hard EM, with binary labels set using 0.5 as the probability threshold, and the up-weighting coefficient recalculated after each E-step.

14.4.2. Biomedical Machine Reading

There is a long-standing interest in biomedical machine reading (e.g., [69, 70]), but prior studies focused on supervised approaches. The advent of big biomedical data creates additional urgency for developing scalable approaches that can generalize to new reading tasks. For example, genome sequencing cost has been dropping faster than Moore’s Law, yet oncologists can only evaluate tumor sequences for a tiny fraction of patients, due to the bottleneck in assimilating relevant knowledge from publications. Recently, [20] formulated precision oncology machine reading as cross-sentence relation extraction (Figure 4) and developed the state-of-the-art system using distant supervision. While promising, their results still leave much room to improve. Moreover, they used heuristics to heavily filter entity candidates, with significant recall loss.

In this section, we use cross-sentence relation extraction as a case study for combining task-specific self-supervision using deep probabilistic logic (DPL). First, we show that DPL can substantially improve machine reading accuracy in a head-to-head comparison with [20], using the same entity linking method. Next, we apply DPL to entity linking itself and attain similar improvement. Finally, we consider further improving the recall by removing the entity filter. By applying DPL to joint entity linking and relation extraction, we more than doubled the recall in relation extraction while attaining comparable precision as [20] with heavy entity filtering.

Distant Supervision: GDKD, CIVIC
Data Programming (Entity)
Mention matches entity name exactly.
Mention not a stop word.
Mention not following figure designation.
Mention's POS tags indicate it is a noun.
Data Programming (Relation)
Less than 30% of words are numbers in each sentence.
No more than three consecutive numbers.
No more than two "et al".
No more than three tokens start with uppercase.
No more than three special characters.
No more than three keywords indicative of table or figure.
Entity mentions do not overlap.
Joint Inference: Relation holds in at least one instance.

(a) Relation Extraction

Distant Supervision: HGNC
Data Programming
No verbs in POS tags.
Mention not a common word.
Mention contains more than two characters or one word.
More than 30% of characters are upper case.
Mention contains both upper and lower case characters.
Mention contains both character and digit.
Mention contains more than six characters.
Dependency label from mention to parent indicative of direct object.
Joint Inference
Identical mentions nearby probably refer to the same entity.
Appositive mentions probably refer to the same entity.
Nearby mentions that match synonyms of same entity probably refer to the given entity.

(b) Entity Linking

Table 1. DPL combines three types of task-specific self-supervision for relation extraction and entity linking

Evaluation Comparing task-specific self-supervision methods is challenging as there is often no annotated test set for evaluating precision and recall. In such cases, we resort to the standard strategy used in prior work by reporting *sample precision* (estimated proportion of correct system extractions) and *absolute recall* (estimated number of correct system extractions). Absolute recall is proportional to recall and can be used to compare different systems (modulo estimation errors).

Datasets We used the same unlabeled text as [20], which consists of about one million full text articles in PubMed Central (PMC)¹. Tokenization, part-of-speech tagging, and syntactic parsing were conducted using SPLAT [71], and Stanford dependencies [72] were obtained using Stanford CoreNLP [73]. For entity ontologies, we used DrugBank² and Human Gene Ontology (HUGO)³. DrugBank contains 8257 drugs; we used the subset of 599 cancer drugs. HUGO contains 37661 genes. For knowledge bases, we used the Gene Drug Knowledge Database (GDKD) [74] and the Clinical Interpretations of Variants In Cancer (CIVIC)⁴. Together, they contain 231 drug-gene-mutation triples, with 76 drugs, 35 genes and 123 mutations.

14.4.3. Cross-sentence relation extraction

Let $e_1, \dots, e_m$ be entity mentions in text T . Relation extraction can be formulated as classifying whether a relation R holds for $e_1, \dots, e_m$ in T . To enable a head-to-head comparison, we used the same cross-sentence setting as [20], where T spans up to three consecutive sentences and R represents the ternary interaction over drugs, genes, and mutations (whether the drug is relevant for treating tumors with the given gene mutation).

Entity linking In this subsection, we used the entity linker from Literome [75] to identify drug, gene, and mutation mentions, as in [20]. This entity linker first identifies candidate mentions by matching entity names or synonyms in domain ontologies, then applies heuristics to filter candidates. The heuristics are designed to enhance precision, at the

¹www.ncbi.nlm.nih.gov/pmc

²www.drugbank.ca

³www.genenames.org

⁴civic.genome.wustl.edu

System	Prec.	Abs. Rec.	Unique
Peng 2017	0.64	6768	2738
DPL + EMB	0.74	8478	4821
DPL	0.73	7666	4144
– DS	0.29	7555	4912
– DP	0.67	4826	2629
– DP (ENTITY)	0.70	7638	4074
– JI	0.72	7418	4011

(a)

Pred. Mod.	Prec.	Abs. Rec.	Unique
BiLSTM	0.60	6243	3427
Graph LSTM	0.73	7666	4144

(b)

Table 2. Comparison of sample precision and absolute recall in test extraction on PMC. (a) Ablation study on self-supervision (DS: distant supervision, DP: data programming, JI: joint inference). DPL + EMB is our system using PubMed-trained word embedding, whereas DPL uses original Wikipedia-trained word embedding in [20]. (b) Comparison of prediction model, with same set of self-supervision and Wikipedia-trained word embedding.

expense of recall. For example, one heuristics would filter candidates of length less than four, which eliminates key cancer genes such as ER or AKT.

Prediction module We used the same graph LSTM as in [20] to enable head-to-head comparison on self-supervision. Briefly, a graph LSTM generalizes a linear-chain LSTM by incorporating arbitrary long-ranged dependencies, such as syntactic dependencies, discourse relations, coreference, and connections between roots of adjacent sentences. A word might have precedents other than the prior word, and its LSTM unit is expanded to include a forget gate for each precedent. See [20] for details.

Supervision module We used DPL to combine three types of task-specific self-supervision for cross-sentence relation extraction (Table 1 (a)). For distant supervision, we used GDKD and CIVIC as in [20]. For data programming, we introduced labeling functions that aim to correct entity and relation errors. Finally, we incorporated joint inference among all co-occurring instances of an entity tuple with the known relation by imposing the at-least-one constraint (i.e., the relation holds for at least one of the instances). For development, we sampled 250 positive extractions from DPL using only distant supervision [20] and excluded them from future training and evaluation.

Experiment results We compared DPL with the state-of-the-art system of [20]. We also conducted ablation study to evaluate the impact of self-supervision strategies. For a fair comparison, we used the same probability threshold in all cases (an instance is classified as positive if normalized probability score is at least 0.5). For each system, sample precision was estimated by sampling 100 positive extractions and manually determining the proportion of correct extractions by an author knowledgeable about this domain. Absolute recall is estimated by multiplying sample precision with the number of positive extractions. Table 2 (a) shows the results. DPL substantially outperformed [20], improving sample precision by ten absolute points and raising absolute recall by 25%. Combining self-supervision is key to this performance gain, as evident from the ablation results. While distant supervision remained the most potent source of task-specific self-supervision, data programming and joint inference each contributed significantly. Replacing out-of-domain (Wikipedia) word embedding with in-domain (PubMed) word embedding [76] also led to a small gain. [20] only compared graph LSTM and linear-chain LSTM in automatic evaluation, where distant-supervision labels were treated as ground truth. They found significant but relatively small gains by graph LSTM. We conducted additional manual evaluation comparing the two in DPL. Surprisingly, we found large performance difference, with graph LSTM outperforming linear-chain LSTM by 13 absolute points in

System	Acc.	F1	Prec.	Rec.
String Match	0.18	0.31	0.18	1.00
DS	0.64	0.71	0.62	0.83
DS + DP	0.66	0.71	0.62	0.83
DS + DP + JI	0.70	0.76	0.68	0.86

(a)

	F1	Precision	Recall
GNormPlus	0.78	0.74	0.81
DPL	0.74	0.68	0.80

(b)

Table 3. (a) Comparison of gene entity linking results on a balanced test set. The string-matching baseline has low precision. By combining various types of task-specific self-supervision, DPL substantially improved precision while retaining reasonably high recall. (b) Comparison of gene entity linking results on BioCreative II test set. GNormPlus is the state-of-the-art system trained on thousands of labeled examples. DPL used only task-specific self-supervision.

precision and raising absolute recall by over 20% (Table 2 (b)). This suggests that [20] might have underestimated performance gain using automatic evaluation.

14.4.4. Entity linking

Let m be a mention in text and e be an entity in an ontology. The goal of entity linking is to predict $\text{Link}(m, e)$, which is true iff m refers to e , for every candidate mention-entity pair m, e . We focus on genes in this paper, as they are particularly noisy.

Prediction module We used BiLSTM with attention over the ten-word windows before and after a mention. The embedding layer is initialized by word2vec embedding trained on PubMed text [76]. The word embedding dimension was 200. We used 5 epochs for training, with Adam as the optimizer. We set learning rate to 0.001, and batch size to 64.

Supervision module As in relation extraction, we combined three types of self-supervision using DPL (Table 1 (b)). For distant supervision, we obtained all mention-gene candidates by matching PMC text against the HUGO lexicon. We then sampled a subset of 200,000 candidate instances as positive examples. We sampled a similar number of noun phrases as negative examples. For data programming, we introduced labeling functions that used mention characteristics (longer names are less ambiguous) or syntactic context (genes are more likely to be direct objects and nouns). For joint inference, we leverage linguistic phenomena related to coreference (identical, appositive, or synonymous mentions nearby are likely coreferent).

Experiment results For evaluation, we annotated a larger set of sample gene-mention candidates and then subsampled a balanced test set of 550 instances (half are true gene mentions, half not). These instances were excluded from training and development. Table 3 (a) compares system performance on this test set. The string-matching baseline has a very low precision, as gene mentions are highly ambiguous, which explains why [20] resorted to heavy filtering. By combining various task-specific self-supervision, DPL improved precision by over 50 absolute points, while retaining a reasonably high recall (86%). All self-supervision contributed significantly, as the ablation tests show. We also evaluated DPL on BioCreative II, a shared task on gene entity linking [69]. We compared DPL with GNormPlus [77], the state-of-the-art supervised system trained on thousands of labeled examples in BioCreative II training set. Despite using zero manually labeled examples, DPL attained comparable F1 and recall (Table 3 (b)). The difference is mainly in precision, which indicates opportunities for more task-specific self-supervision.

System	Prec	Abs. Rec.	Unique
Peng 2017	0.31	11481	5447
DPL (RE)	0.52	17891	8534
+ EL (TRN)	0.55	21881	11047
+ EL (TRN/TST)	0.61	20378	10291

Gene	Drug	Mut.	Gene-Mut.	Relation
27%	4%	20%	45%	24%

(a)

(b)

Table 4. (a) Comparison of sample precision and absolute recall (all instances and unique entity tuples) when all gene mention candidates are considered. [20] used distant supervision only. RE: DPL relation extraction. EL: using DPL entity linking in RE training (TRN) and/or test (TST). (b) Error analysis for DPL relation extraction.

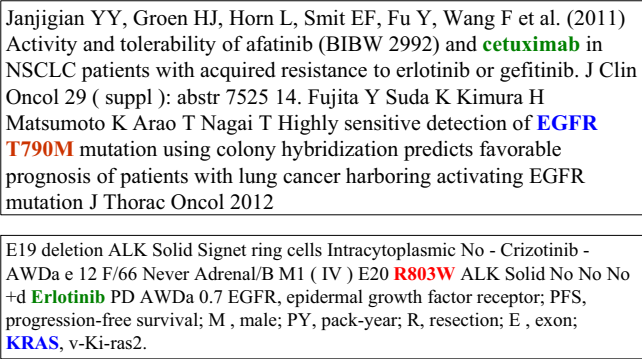


Figure 5. Example of relation-extraction errors corrected by DPL with additional task-specific self-supervision.

14.4.5. Joint entity and relation extraction

An important use case for machine reading is to improve knowledge curation efficiency by offering extraction results as candidates for curators to vet. The key to practical adoption is attaining high recall with reasonable precision [20]. The entity filter used in [20] is not ideal in this aspect, as it substantially reduced recall. In this subsection, we consider replacing the entity filter by the DPL entity linker. See Table 4 (a). Specifically, we added one labeling function to check if the entity linker returns a normalized probability score above p_{TRN} for gene mentions, and filtered test instances if the gene mention score is lower than p_{TST} . We set $p_{\text{TRN}} = 0.6$ and $p_{\text{TST}} = 0.3$ from preliminary experiments. The labeling function discouraged learning from noisy mentions, and the test-time filter skips an instance if the gene is likely wrong. Not surprisingly, without entity filtering, [20] suffered large precision loss. All DPL versions substantially improved accuracy, with significantly more gains using the DPL entity linker.

14.4.6. Discussion

Scalability DPL is efficient to train, taking around 3.5 hours for relation extraction and 2.5 hours for entity linking in our PubMed-scale experiments, with 25 CPU cores (for probabilistic logic) and one GPU (for LSTM). For relation extraction, the graphical model of probabilistic logic contains around 7,000 variables and 70,000 factors. At test time, it is just an LSTM, which predicted each instance in less than a second. In general, DPL learning scales linearly in the number of training instances. For distant supervision and data programming, DPL scales linearly in the number of known facts and labeling functions. As discussed before, joint inference with high-order factors is more challenging, but can be efficiently approximated. For inference in probabilistic logic, we found that loopy belief propagation worked reasonably well, converging after 2-4 iterations. Overall,

Algorithm 2 Self-Supervised Self-Supervision (S4)

Input: Seed virtual evidences I , deep neural network Ψ , inputs $X = (X_1, \dots, X_N)$, unobserved outputs $Y = (Y_1, \dots, Y_N)$, human query budget T .

Output: Learned prediction module Ψ and virtual evidences $K = \{(f_v(X, Y), w_v) : v\}$.

Initialize: $K = I$; $Q = \emptyset$; $i = 0$. **for** $i = 1 \dots M$ **do**

while *Structured Self-Training not converged* **do**

$\Psi, K \leftarrow \text{DPL-Learn}(K, \Psi, X, Y)$

$v = \text{PropSST}(K, \Psi, X, Y)$;

$K \leftarrow K \cup v$;

end

if $|Q| < T$ **then**

$v = \text{PropFAL}(K, \Psi, X, Y, Q)$; $Q \leftarrow Q \cup v$;

if *Human-Accept*(v) **then** $K \leftarrow K \cup v$

end

end

we ran variational EM for three iterations, using ten epochs of deep learning in each M-step. We found these worked well in preliminary experiments and used the same setting in all final experiments.

Accuracy To understand more about DPL’s performance gain over distant supervision, we manually inspected some relation-extraction errors fixed by DPL after training with additional self-supervision. Figure 5 shows two such examples. While some data programming functions were introduced to prevent errors stemming from citations or flattened tables, none were directly applicable to these examples. This shows that DPL can generalize beyond the original self-supervision.

While the results are promising, there is still much to improve. Table 4 (b) shows estimated precision errors for relation extraction by DPL. (Some instances have multiple errors.) Entity linking can incorporate more task-specific self-supervision. Joint entity linking and relation extraction can be improved by feeding back extraction results to linking. Improvement is also sorely needed in classifying mutations and gene-mutation associations. The prediction module can also be improved, e.g., by adding attention to graph LSTM. DPL offers a flexible framework for exploring all these directions.

14.5. Self-Supervised Self-Supervision

In this section, we present the *Self-Supervised Self-Supervision (S4)* framework, which extends deep probabilistic logic (DPL) with the capability to learn new self-supervision. Let $\mathcal{V} = \{(f_v, w_v, \alpha_v) : v\}$ be the set of all candidate virtual evidences, where $f_v(X, Y)$ is a first-order logical formula, w_v is the weight, and α_v is the weight prior (for non-fixed w_v). Let K be the set of virtual evidences maintained by the algorithm, initialized by the seed I . The key idea of S4 is to interleave structure learning and active learning to iteratively propose new virtual evidence $v \in \mathcal{V}$ to augment K (Figure 2).

Specifically, S4 can be viewed as conducting *structure learning* in the factor graph that specifies the virtual evidence. Structure learning has been studied intensively in the graphical model literature [34]. It is also known as feature selection or feature induction in general machine learning literature [78]. Here, we are introducing structured factors for self-supervision, rather than as feature templates to be used during training. Another key difference from standard structure learning is the deep neural network, which provides an alternative view from the virtual evidence space and enables multi-view learning in DPL.

The neural network can also help identify candidate virtual evidences, e.g., via neural attention. Self-training is a special case where candidate virtual evidences are individual label assignments (i.e., $f_v = I[y_v = l_v]$). S4 can thus be viewed as conducting *structured self-training (SST)* by admitting arbitrary Markov logic formulas as virtual evidence.

In data programming and other prior self-supervision methods, human experts need to pre-specify all self-supervision upfront. While it is easy to generate a small seed by identifying the most salient self-supervision, this effort can quickly become tedious and more challenging as the experts are required to enumerate the less salient templates. On the other hand, given a candidate, it's generally much easier for experts to validate it. *This suggests that for the best utilization of human bandwidth, we should focus on leveraging them to produce the initial self-supervision and verify candidate self-supervision.* Consequently, in addition to structured self-training (SST), S4 incorporates *feature-based active learning (FAL)* (i.e., active learning of self-supervision). When SST converges, S4 will switch to the active learning mode by proposing a candidate virtual evidence for human verification (i.e., labeling a feature rather than an instance in standard active learning). Intuitively, in FAL we are proposing virtual evidences for which the labels of the corresponding instances are still uncertain. If the human expert can provide definitive supervision on the label, the information gain will be large. By contrast, in SST, we favor virtual evidences with skewed posterior label assignments for their corresponding instances, as they can potentially amplify the signal.

Algorithm 2 describes the S4 algorithm. S4 first conducts DPL using the initial self-supervision I , then interleaves structured self-training (SST) with feature-based active learning (FAL). SST steps are repeated until there is little change in the probabilistic labels ($< 1\%$ in our experiments). DPL learning updates the deep neural network and the graphical model parameters with warm start (i.e., the parameters are initialized with the previous parameters). All proposed queries are stored and won't be proposed again. The total amount of human effort consists of generating the seed I and validating T queries.

S4 is a general algorithmic framework that can combine various strategies for designing $\mathcal{V}$, PropSST, and PropFAL. In standard structure learning, PropSST would attempt to maximize the learning objective (e.g., conditional likelihood of seed virtual evidences) by iteratively conducting greedy structure changes. However, this is very expensive to compute, since it requires a full DPL run just to score each candidate. Instead, we take inspiration from the feature-induction and relational learning literature and use heuristic approximations that are much faster to evaluate. In the most general setting, $\mathcal{V}$ contains all possible potential functions. In practice, we can restrict it to obtain a good trade-off between expressiveness and computation for the problem domain. Interestingly, as we will see in the experiment section, even with relatively simple classes of self-supervision, S4 can dramatically improve over DPL through structure learning and active learning. We use text classification from natural language processing (NLP) as a running example to illustrate how to apply S4 in practice. Here, the input $X_i = (t_1, \dots, t_{s_i})$ is a sequence of tokens and the output Y_i is the classification label (e.g., pos or neg in sentiment analysis).

14.5.1. Candidate Self-Supervision

For $\mathcal{V}$, the simplest choice is to use tokens. Namely, $f_{i,l}(X_i, Y_i) = \mathbb{I}[t \in X_i \wedge Y_i = l]$. For simplicity, we can use a fixed weight and prior for all initial virtual evidence, i.e., $\mathcal{V} = \{(f_{i,l}, w, \alpha) : t, l\}$. Take sentiment analysis as an example. X_i may represent a movie review and $Y_i \in \{0, 1\}$ the sentiment. A virtual evidence for self-supervision may stipulate

that if the review contains the word “good”, the sentiment is more likely to be positive. This can be represented by the formula $f_{\text{good},1}(X_i, Y_i) = \mathbb{I}[\text{“good”} \in X_i \wedge Y_i = 1]$ with a positive weight. A more advanced choice for $\mathcal{V}$ may include high-order joint-inference factors, such as $f_{ij}(Y_i, Y_j) = \mathbb{I}[Y_i = Y_j]$. If we add this factor for similar pairs X_i, X_j , it stipulates that instances with similar input are likely to share the same label. Here we define similar pairs with a function $\text{Sim}(X_i, X_j)$ between X_i and X_j , such as the cosine similarity between the sentence (or document) embeddings of X_i and X_j , based on the current deep neural network. Note that this is different from graph-based semi-supervised learning or other kernel-based methods in that the similarity metric is not pre-specified and fixed, but rather evolving along with the neural network for the end task.

14.5.2. Structured Self-Training (PropSST)

From DPL learning, we obtain the current marginal estimate of the latent label variables $q_i(Y_i)$, which we would treat as probabilistic labels in assessing candidate virtual evidence. There are many sensible strategies for proposing candidates in structured self-training (i.e., PropSST). For token-based self-supervision, a common technique from the feature-selection literature is to choose a token highly correlated with a label. For example, we can choose the token t that occurs much more frequently in instances for a given label l than others using our noisy label estimates. We find that this often leads to very noisy proposals and semantic drift. A simple refinement is to restrict our scoring to instances containing some initial self-supervised tokens. However, this still has the drawback that a word may occur more often in instances of a class for reasons other than contributing to the label classification. We therefore consider a more sophisticated strategy based on neural attention. Namely, we will credit occurrences using the normalized attention weight for the given token in each instance.

Formally, let $A_\Psi(X_i, j)$ represent the normalized attention weight the neural network Ψ assigns to the j -th token in X_i for the final classification. We define *average weighted attention* for token t and label l as $\text{Attn}(t, l) = \frac{1}{C_t} \sum_{i,j: X_{i,j}=t} q_i(Y_i = l) \cdot A_\Psi(X_i, j)$, where C_t is the number of occurrences of t in X . Then PropSST would simply score token-based self-supervision $f_{t,l}$ using relative average weighted attention: $S_{\text{token}}(t, l) = \text{Attn}(t, l) - \sum_{l' \neq l} \text{Attn}(t, l')$. In each iteration, PropSST picks the top scoring $f_{t,l}$ that has not been proposed yet as the new virtual evidence to add to K .

We also consider an entropy-based score function that works for arbitrary input-based features. It treats the prediction module Ψ as a black box, and only uses the posterior label assignments $q_i(Y_i)$. Consider candidate virtual evidence $f_{b,l}(X_i, Y_i) = \mathbb{I}[b(X_i) \wedge Y_i = l]$, where b is a binary function over input X_i . This clearly generalizes token-based virtual evidence. Define $\text{Ent}(b) = H\left(\frac{1}{C_b} \sum_{i: b(X_i)=1} q_i(Y_i)\right)$, where H is the Shannon entropy and C_b is the number of instances for which the feature b holds true. This function represents the entropy of the average posterior among all instances with $b(X_i) = 1$. PropSST will then use $S_{\text{entropy}}(b) = 1/\text{Ent}(b)$ to choose the b^* with the lowest average entropy and then pick label l^* with the highest average posterior probability for b^* . In our experiments, this performs similarly to attention-based scores.

For joint-inference self-supervision, we consider the similarity-based factors defined earlier, and leave the exploration of more complex factors to future work. To distinguish task-specific similarity from pretrained similarity, we use the difference between the similarity computed using the current fine-tuned BERT model and that using the pretrained one. Formally, let $\text{Sim}_{\text{pretrained}}(X_i, X_j)$ be the cosine similarity between the embeddings

of X_i and X_j generated by the pretrained BERT model, and $\text{Sim}_\Psi(X_i, X_j)$ be that between the embeddings using the current learned network Ψ . PropSST would score the joint-inference factor using the relative similarity and choose the top scoring pairs to add to self-supervision: $S_{\text{joint}}(X_i, X_j) = \text{Sim}_\Psi(X_i, X_j) - \text{Sim}_{\text{pretrained}}(X_i, X_j)$.

14.5.3. Feature-Based Active Learning (PropFAL)

For active learning, a common strategy is to pick the instance with highest entropy in the label distribution based on the current marginal estimate. In feature-based active learning, we can similarly pick the feature b with the highest average entropy $\text{Ent}(b)$. Note that this is opposite to how we use the entropy-based score function in PropSST, where we choose the feature with the lowest average entropy. In PropFAL, we will identify $b^* = \arg \max(\text{Ent}(b))$, present $f_{b^*, l}(X, Y) = \mathbb{I}[b^*(X) \wedge Y = l]$ for all possible labels l , and ask the human expert to choose a label l^* to accept or reject them all.

14.5.4. Experiments

We use the natural language processing (NLP) task of text classification to explore the potential for S4 to improve over DPL using structure learning and active learning. We used three standard text classification datasets: IMDB [79], Stanford Sentiment Treebank [80], and Yahoo! Answers [81]. IMDB contains movie reviews with polarity labels (positive/negative). There are 25,000 training instances with equal numbers of positive and negative labels, and the same numbers for test. Stanford Sentiment Treebank (StanSent) also contains movie reviews, but was annotated with five labels ranging from very negative to very positive. We used the binarized version of StanSent, which collapses the polarized categories and discards the neutral sentences. It contains 6,920 training instances and 1,821 test instances, with roughly equal split. Overall, the StanSent reviews are shorter than IMDB's, and they often exhibit more challenging linguistic phenomena (e.g., nested negations or sarcasm). The Yahoo dataset contains 1.4 million training questions and 60,000 test questions from Yahoo! Answers; these are equally split into 10 classes.

In all our experiments with S4, we withheld gold labels from the system, used the training instances as unlabeled data, and evaluated on the test set. We reported test accuracy, as all of the datasets are class-balanced. For our neural network prediction module $\Psi(X_i, Y_i)$, we used the standard BERT-base model pretrained using Wikipedia [2], along with a global-context attention layer as in [82], which we also used for attention-based scoring. We truncated the input text to 512 tokens, the maximum allowed by the standard BERT model. All of our baselines (except supervised bag-of-words) use the same BERT model. For all virtual evidences, we used initial weight $w = 2.2$ (the log-odds of 90% probability) and used an α corresponding to an L2 penalty of 5×10^{-8} on w . Our results are not sensitive to these values. In all experiments, we use the Adam optimizer with an initial learning rate tuned over $[0.1, 0.01, 0.001]$. The optimizer's history is reset after each EM iteration to remove old gradient information. We always performed 3 EM iterations and trained Ψ for 5 epochs per iteration. For the global-context attention layer, we used a context dimension of 5. The model is warm-started across EM iterations (in DPL), but *not* across the outer iterations in S4 (the for loop). In all experiments, we used the Adam optimizer with an initial learning rate tuned over $[0.1, 0.01, 0.001]$. The optimizer's history is reset after each EM iteration to remove old gradient information.

For virtual evidence, we focus on token-based unary factors and similarity-based joint factors, as discussed in the previous section, and leave the exploration of more complex

Algorithm	Sup. size $ I $	Test acc (%)
BoW	25k	87.1
DNN	25k	91.0
Self-training	100	69.9
	1k	88.5
Snorkel	6	76.6
DPL	6	80.7
S4-SST	6	85.5
S4 ($T = 20$)	6	85.6
Snorkel	20	82.4
DPL	20	78.9
S4-SST	20	86.4
S4 ($T = 20$)	20	86.9
Snorkel	40	84.6
DPL	40	85.2
S4-SST	40	86.6
S4 ($T = 20$)	40	86.8

(a) IMDB

Algorithm	Sup. size $ I $	Test acc (%)
BoW	6.9k	78.9
DNN	6.9k	90.9
Self-training	50	78.0
	100	81.8
Snorkel	6	63.5
DPL	6	57.2
S4-SST	6	73.0
S4-SST + J	6	76.2
S4 + J ($T = 20$)	6	81.4
Snorkel	20	73.0
DPL	20	72.4
S4-SST	20	83.3
S4-SST + J	20	85.1
S4 + J ($T = 20$)	20	84.4
Snorkel	40	73.6
DPL	40	77.0
S4-SST	40	84.9
S4-SST + J	40	86.3
S4 + J ($T = 20$)	40	85.4

(b) Stanford

Table 5. System comparison on IMDB and Stanford.

factors to future work. Even with these factors, our self-supervised Ψ models often nearly match the accuracy of the best supervised models. We also compare with Snorkel, a popular self-supervision system [3]. We use the latest Snorkel version [63], which models correlations among same-instance factors. Snorkel cannot incorporate joint-inference factors across different instances. In all of our Snorkel baselines, we separately tuned the initial learning rate over the same set, and trained the deep neural network for the same number of *total* epochs that DPL uses to ensure a fair comparison.

To simulate human supervision for unary factors, we trained a unigram model using the training data with L1 regularization and selected the 100 tokens with the highest weights for each class as the oracle self-supervision. By default, we used the top tokens for each class in the initial self-supervision I . We also experimented with using random tokens from the oracle in I to simulate lower-quality initial supervision and to quantify the variance of S4. For the set of oracle joint factors, we fine-tuned the standard BERT model on the training set, used the CLS embedding BERT produces to compute input similarity, and picked the 100 input pairs whose similarity changed the most between the fine-tuned model and the initial model.

We first investigate whether structure learning can help in S4 by running without feature-based active learning. We set the query budget $T = 0$ in Algorithm 2. Because we only take structured self-training steps when $T = 0$, we denote this version of S4 as S4-SST. Table 5 (a) shows the results on IMDB. With just six self-supervised tokens (three per class), S4-SST already attained 86% test accuracy, which outperforms self-training with 100 labeled examples by 16 absolute points, and is only slightly worse than self-training with 1000 labeled examples or supervised training with 25,000 labeled examples.

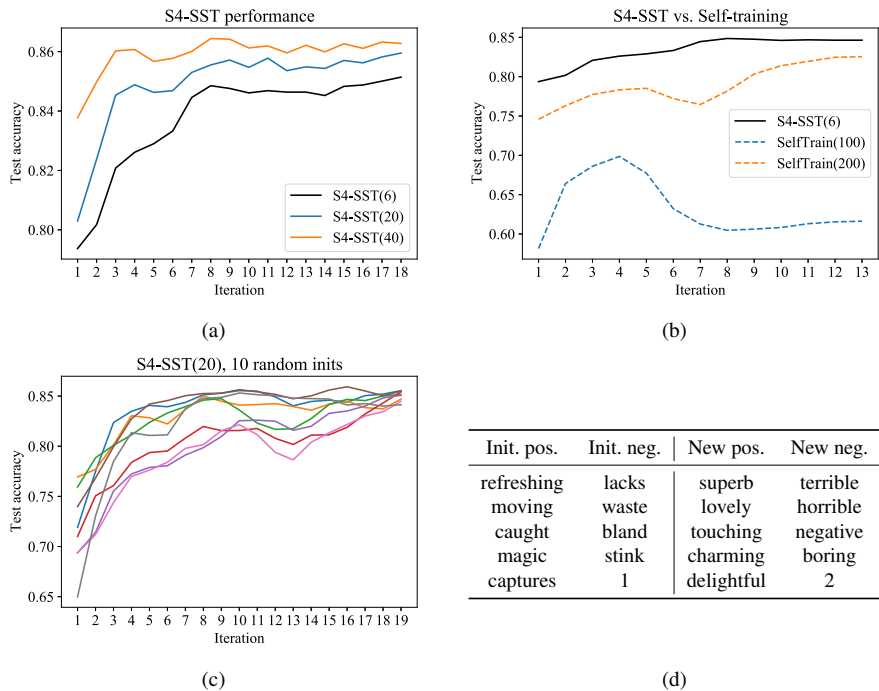


Figure 6. S4-SST learning in IMDB. (a) S4-SST learning curves with various numbers of initial self-supervised tokens. (b) comparison between the learning curves of S4-SST with just six self-supervised tokens and self-training with an order of magnitude more direct supervision (100/200 labeled examples). (c) multiple runs of S4-SST with 20 *random* oracle self-supervised tokens (rather than the top oracle tokens). (d) pre-specified self-supervised tokens and proposed tokens in the first few iterations.

By conducting structure learning, S4-SST substantially outperformed DPL, gaining about 5 absolute points in accuracy (a 25% relative reduction in error), and also outperformed the Snorkel baseline by 8.9 points. Interestingly, with more self-supervision at 20 tokens, DPL’s performance drops slightly, which might stem from more noise in the initial self-supervision. By contrast, S4-SST capitalized on the larger seed self-supervision and attained steady improvement. Even with substantially more self-supervision at 40 tokens, S4-SST still attained similar accuracy gain, demonstrating the power in structure learning. On average across different initial amounts of supervision $|I|$, S4-SST outperforms DPL by 5.6 points and Snorkel by 5.3 points. Next, we consider the full S4 algorithm with a budget of up to $T = 20$ human queries (S4 ($T = 20$)). Overall, by automatically generating self-supervision from structure learning, S4-SST already attained very high accuracy on this dataset. However, active learning can still produce some additional gain. The only randomness in Table 5 is the initialization of the deep network Ψ , with negligible effect.

Figure 6 (a) shows how S4-SST iterations improve the test accuracy of the learned neural network with different amounts of initial virtual evidence. Not surprisingly, with more initial self-supervision, the gain is less pronounced, but still significant. Figure 6 (b) compares the learning curves of S4-SST with those of self-training. Remarkably, with just *six* self-supervised tokens, S4-SST not only attained substantial gain over the iterations, but also easily outperformed self-training despite the latter using an order of magnitude

	$ I = 6$	$ I = 10$	$ I = 20$	$ I = 40$		$ I = 6$	$ I = 10$	$ I = 20$	$ I = 40$
$T = 5$	79.0	83.8	84.0	85.9	$T = 5$	71.4	77.5	82.5	83.9
$T = 10$	79.5	83.7	85.0	86.0	$T = 10$	72.3	77.4	82.5	83.7
$T = 20$	82.7	84.5	85.8	86.4	$T = 20$	77.1	80.8	83.4	84.0

(a) IMDb

(b) Stanford

Table 6. S4-FAL test results (no SST steps)

more label information (up to 200 labeled examples). This shows that S4 is much more effective in leveraging bounded human effort for supervision.

Figure 6 (c) shows 10 runs of S4-SST when it was initialized with 20 *random* oracle tokens (10 per class), rather than the *top* 20 tokens from the oracle. As expected, DPL's initial performance is worse than with the top oracle tokens. However, over the iterations, S4-SST was able to recover even from particularly poor initial state, gaining up to 20 absolute accuracy points over DPL in the process. The final accuracy of S4-SST is 85.2 ± 0.9 , compared to 71.5 ± 6.5 for DPL and 72.9 ± 6.7 for Snorkel, a mean improvement of more than 12 absolute accuracy points over both baselines. S4-SST's gains over DPL and Snorkel are statistically significant using a paired *t*-test (samples are paired when the algorithms have the same initial factors I) with $p = 0.01$. This indicates that S4-SST is robust to noise in the initial self-supervision. Figure 6 (d) shows the initial self-supervised tokens and the ones learned by S4-SST in the first few iterations. We can see that S4-SST is able to learn highly relevant self-supervision tokens.

S4 has similar gains over DPL, Snorkel, and self-training on the Stanford dataset. See Table 5 (b). The Stanford dataset is much more challenging and for a long while, it was hard to exceed 80% test accuracy [80]. Interestingly, S4-SST was able to surpass this milestone using just 20 initial self-supervision tokens. As in IMDb, the only randomness is in the initialization of Ψ , which is negligible.

For IMDb, as can be seen above, even with simple token-based self-supervision, S4-SST already performed extremely well. So we focused our investigation of joint-inference factors on the more challenging Stanford dataset (S4-SST + J). While S4-SST already performed very well with token-based self-supervision, by incorporating joint-inference factors, it could still gain up to 3 absolute accuracy points. An example learned joint-inference factor is between the sentence pair: *This is no "Waterboy!"* and *"It manages to accomplish what few sequels can—it equals the original and in some ways even better"*. Note that Waterboy is widely considered a bad movie, hence the first sentence expresses a positive sentiment, just like the second. It is remarkable that S4 can automatically induce such factors with complex and subtle semantics from small initial self-supervision.

The Stanford results also demonstrated that active learning could play a bigger role in more challenging scenarios. With limited initial self-supervision ($|I| = 6$), the full S4 system (S4+J ($T=20$)) gained 8 absolute points over S4-SST and 5 absolute points over S4-SST+J. With sufficient initial self-supervision and joint-inference, however, active learning was actually slightly detrimental ($|I| = 20, 40$).

Finally, we evaluate S4-FAL, which conducts active learning but not structure learning. See Table 6. As expected, performance improved with larger initial self-supervision (I) and human query budget (T). Active learning helps the most when initial self-supervision is limited. Compared to S4 with structure learning, however, active learning alone is less effective. For example, without requiring any human queries, S4-SST outperformed

S4-FAL on both IMDB and Stanford even when the latter was allowed up to $T = 20$ human queries.

$ I $	Entropy-based	Attention-based	$ I $	Entropy-based	Attention-based
6	82.1	85.5	6	77.2	73.0
20	84.9	86.4	20	85.1	83.3
40	85.5	86.6	40	85.7	84.9

(a) IMDB

(b) Stanford

Table 7. Entropy-based scoring (S_{entropy}) perform comparably as attention-based scoring (S_{token}).

Algorithm	Sup. size	Test acc (%)
BoW	140k	71.2
DNN	140k	79.8
Self-training	50	38.4
Snorkel	50	37.2
DPL	50	41.8
S4-SST	50	49.1
Self-training	100	38.2
Snorkel	100	36.5
DPL	100	41.7
S4-SST	100	52.3

Table 8. Comparison of test accuracy on Yahoo! Answers.

Table 7 compares S4-SST test accuracy using entropy-based scoring (S_{entropy}) and attention-based scoring (S_{token}). Entropy-based scoring slightly outperforms attention-based scoring on Stanford Sentiment and slightly trails on IMDB. Overall, the two perform comparably but entropy-based scoring is more generally applicable.

S4-SST obtains similarly substantial gains on Yahoo using token-based factors and attention scoring. See Table 8. We focused our experiments on a fixed 10% of the training set due to its very large size (1.4 million examples). There are 10 classes, so initial self-supervision size of 50 (100) represents 5 (10) initial tokens per class (for S4, DPL, and Snorkel), or 5 (10) labeled examples per class (for self-training). Compared to binary sentiment analysis in IMDB and Stanford, Yahoo represents a much more challenging dataset, with ten classes and larger input text for each instance. The linguistic phenomena are much more diverse, and therefore neither Snorkel or DPL performed much better than self-training, as token-based self-supervision confers not much more information than a labeled example. However, S4-SST is still able to attain substantial improvement over the initial self-supervision. E.g., with initial supervision size of 100, S4-SST gained about 11 absolute accuracy points over DPL, and 16 absolute points over Snorkel. Additionally, S4-SST is able to better utilize the new factors than Snorkel. If we run Snorkel using the same initial factors as S4-SST and also add the new factors proposed by S4-SST in each iteration, the accuracy improved from 36.5 to 44.2, but still trailed S4-SST (52.3) by 8

absolute points. Interestingly, both DPL and Snorkel perform better on Yahoo with *fewer* initial factors, at 5 per class, suggesting they are sensitive to noise in the less reliable initial self-supervision. By contrast, S4-SST is more noise-tolerant and benefits from additional initial supervision.

14.6. Conclusions

In this chapter, we present deep probabilistic logic (DPL) as a unifying framework for task-specific self-supervised learning, by composing probabilistic logic with deep learning. We further present self-supervised self-supervision (S4), which extends DPL with structure learning and active learning capabilities. Experiments on biomedical machine reading and text classification tasks show that DPL enables effective combination of disparate task-specific self-supervision strategies, while S4 can further improve application performance by proposing additional self-supervision. Overall, this enables the most effective use of human expert bandwidth by focusing on identifying the most salient self-supervision for initialization and verifying proposed self-supervision. While we focus on NLP tasks in this chapter, our methods are general and can potentially be applied to other domains. Future directions include: combining DPL with deep generative models and probabilistic programming methods; further investigation in combining structure learning and active learning in S4; exploring alternative optimization strategies and more sophisticated self-supervision classes and proposal algorithms; applications to other domains.

References

- [1] LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature*. 2015;521(7553):436–444.
- [2] Devlin J, Chang MW, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*. 2018;.
- [3] Ratner AJ, De Sa CM, Wu S, et al. Data programming: Creating large training sets, quickly. In: *Advances in neural information processing systems*; 2016. p. 3567–3575.
- [4] Bach SH, He B, Ratner A, et al. Learning the structure of generative models without labeled data. In: *International Conference on Machine Learning*; 2017. p. 273–282.
- [5] Chang MW, Ratinov L, Roth D. Guiding semi-supervision with constraint-driven learning. In: *Proceedings of the 45th annual meeting of the association of computational linguistics*; 2007. p. 280–287.
- [6] Poon H, Domingos P. Joint unsupervised coreference resolution with markov logic. In: *Proceedings of the conference on empirical methods in natural language processing*; Association for Computational Linguistics; 2008. p. 650–659.
- [7] Druck G, Mann G, McCallum A. Learning from labeled features using generalized expectation criteria. In: *Proceedings of the 31st annual international ACM SIGIR conference on Research and development in information retrieval*; ACM; 2008. p. 595–602.
- [8] Ganchev K, Gillenwater J, Taskar B, et al. Posterior regularization for structured latent variable models. *Journal of Machine Learning Research*. 2010;11(Jul):2001–2049.
- [9] Craven M, Kumlien J. Constructing biological knowledge bases by extracting information from text sources. In: *Proceedings of the Seventh International Conference on Intelligent Systems for Molecular Biology*; 1999.
- [10] Mintz M, Bills S, Snow R, et al. Distant supervision for relation extraction without labeled data. In: *Proceedings of the Joint Conference of the Forty-Seventh Annual Meeting of the Association for Computational Linguistics and the Fourth International Joint Conference on Natural Language Processing*; 2009.
- [11] Wang H, Poon H. Deep probabilistic logic: A unifying framework for indirect supervision. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*; 2018. p. 1891–1902.

- [12] Richardson M, Domingos P. Markov logic networks. *Machine learning*. 2006;62(1-2):107–136.
- [13] Lang H, Poon H. Self-supervised self-supervision by combining deep learning and probabilistic logic. In: *Proceedings of the Thirty Fifth Annual Meeting of the Association for the Advancement of Artificial Intelligence*; 2021.
- [14] McClosky D, Charniak E, Johnson M. Effective self-training for parsing. In: *Proceedings of the main conference on human language technology conference of the North American Chapter of the Association of Computational Linguistics*; Association for Computational Linguistics; 2006. p. 152–159.
- [15] Druck G, Settles B, McCallum A. Active learning by labeling features. In: *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing: Volume 1-Volume 1*; Association for Computational Linguistics; 2009. p. 81–90.
- [16] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: *Advances in neural information processing systems*; 2017. p. 5998–6008.
- [17] Bach SH, He B, Ratner A, et al. Learning the structure of generative models without labeled data. In: *Proceedings of the 34th International Conference on Machine Learning-Volume 70*; 2017. p. 273–282.
- [18] Roth D. Incidental supervision: Moving beyond supervised learning. In: *Thirty-First AAAI Conference on Artificial Intelligence*; 2017.
- [19] Quirk C, Poon H. Distant supervision for relation extraction beyond the sentence boundary. In: *Proceedings of the Fifteenth Conference on European chapter of the Association for Computational Linguistics*; 2017.
- [20] Peng N, Poon H, Quirk C, et al. Cross-sentence n-ary relation extraction with graph lstms. *Transactions of the Association for Computational Linguistics*. 2017;5:101–115.
- [21] Huang H, Heck L, Ji H. Leveraging deep neural networks and knowledge graphs for entity disambiguation. *arXiv preprint arXiv:150407678*. 2015;.
- [22] Ratner AJ, De Sa CM, Wu S, et al. Data programming: Creating large training sets, quickly. In: *Advances in Neural Information Processing Systems*; 2016. p. 3567–3575.
- [23] Varma P, He BD, Bajaj P, et al. Inferring generative model structure with static analysis. In: *Advances in neural information processing systems*; 2017. p. 240–250.
- [24] Liu L, Ren X, Zhu Q, et al. Heterogeneous supervision for relation extraction: A representation learning approach. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*; 2017. p. 46–56.
- [25] Hoffmann R, Zhang C, Ling X, et al. Knowledge-based weak supervision for information extraction of overlapping relations. In: *Proceedings of the Forty-Ninth Annual Meeting of the Association for Computational Linguistics*; 2011.
- [26] Lin Y, Shen S, Liu Z, et al. Neural relation extraction with selective attention over instances. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*; Vol. 1; 2016. p. 2124–2133.
- [27] Halpern Y, Horng S, Choi Y, et al. Electronic medical record phenotyping using the anchor and learn framework. *Journal of the American Medical Informatics Association*. 2016;23(4):731–740.
- [28] Haghighi A, Klein D. Prototype-driven learning for sequence models. In: *Proceedings of the main conference on Human Language Technology Conference of the North American Chapter of the Association of Computational Linguistics*; Association for Computational Linguistics; 2006. p. 320–327.
- [29] Poon H. Grounded unsupervised semantic parsing. In: *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; 2013. p. 933–943.
- [30] Chang MW, Ratinov L, Roth D. Guiding semi-supervision with constraint-driven learning. In: *Proceedings of the 45th annual meeting of the association of computational linguistics*; 2007. p. 280–287.
- [31] Druck G, Mann G, McCallum A. Learning from labeled features using generalized expectation criteria. In: *Proceedings of the 31st annual international ACM SIGIR conference on Research and development in information retrieval*; 2008. p. 595–602.
- [32] Poon H, Domingos P. Joint unsupervised coreference resolution with markov logic. In: *Proceedings of the 2008 conference on empirical methods in natural language processing*; 2008. p. 650–659.
- [33] Ganchev K, Graça J, Gillenwater J, et al. Posterior regularization for structured latent variable models. *Journal of Machine Learning Research*. 2010;11(67):2001–2049.
- [34] Friedman N, Koller D. Being bayesian about network structure. a bayesian approach to structure discovery in bayesian networks. *Machine learning*. 2003;50(1-2):95–125.
- [35] Kok S, Domingos P. Learning the structure of markov logic networks. In: *Proceedings of the 22nd international conference on Machine learning*; 2005. p. 441–448.

- [36] Hearst MA. Automatic acquisition of hyponyms from large text corpora. In: Proceedings of the 14th conference on Computational linguistics-Volume 2; Association for Computational Linguistics; 1992. p. 539–545.
- [37] Carlson A, Betteridge J, Kisiel B, et al. Toward an architecture for never-ending language learning. In: Twenty-Fourth AAAI Conference on Artificial Intelligence; 2010.
- [38] Yarowsky D. Unsupervised word sense disambiguation rivaling supervised methods. In: 33rd annual meeting of the association for computational linguistics; 1995. p. 189–196.
- [39] Collins M, Singer Y. Unsupervised models for named entity classification. In: 1999 Joint SIGDAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora; 1999.
- [40] Varma P, Ré C. Snuba: automating weak supervision to label training data. Proceedings of the VLDB Endowment. 2018;12(3):223–236.
- [41] Tong S, Koller D. Active learning for structure in bayesian networks. In: International joint conference on artificial intelligence; Vol. 17; Citeseer; 2001. p. 863–869.
- [42] Galhotra S, Golshan B, Tan WC. Adaptive rule discovery for labeling text data. arXiv preprint arXiv:200506133. 2020;.
- [43] Poon H, Domingos P. Joint inference in information extraction. In: AAAI; Vol. 7; 2007. p. 913–918.
- [44] Poon H, Vanderwende L. Joint inference for knowledge extraction from biomedical literature. In: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics; Association for Computational Linguistics; 2010. p. 813–821.
- [45] Kimmig A, Bach S, Broecheler M, et al. A short introduction to probabilistic soft logic. In: Proceedings of the NIPS Workshop on Probabilistic Programming: Foundations and Applications; 2012. p. 1–4.
- [46] Rocktäschel T, Riedel S. End-to-end differentiable proving. Advances in Neural Information Processing Systems. 2017;30.
- [47] Manhaeve R, Dumancic S, Kimmig A, et al. Deepproblog: Neural probabilistic logic programming. Advances in Neural Information Processing Systems. 2018;31:3753–3760.
- [48] Qu M, Tang J. Probabilistic logic neural networks for reasoning. In: NeurIPS; 2019.
- [49] Kingma DP, Welling M. Auto-encoding variational bayes. arXiv preprint arXiv:13126114. 2013;.
- [50] Kingma DP, Mohamed S, Rezende DJ, et al. Semi-supervised learning with deep generative models. In: Advances in Neural Information Processing Systems; 2014. p. 3581–3589.
- [51] Johnson M, Duvenaud DK, Wiltchko A, et al. Composing graphical models with neural networks for structured representations and fast inference. In: Advances in neural information processing systems; 2016. p. 2946–2954.
- [52] Tran D, Hoffman MD, Saurous RA, et al. Deep probabilistic programming. arXiv preprint arXiv:170103757. 2017;.
- [53] Blum A, Mitchell T. Combining labeled and unlabeled data with co-training. In: Proceedings of the eleventh annual conference on Computational learning theory; 1998. p. 92–100.
- [54] Grechkin M, Poon H, Howe B. Ezlearn: Exploiting organic supervision in large-scale data annotation. arXiv preprint arXiv:170908600. 2017;.
- [55] Towell GG, Shavlik JW. Knowledge-based artificial neural networks. Artificial intelligence. 1994;70(1-2):119–165.
- [56] Hu Z, Ma X, Liu Z, et al. Harnessing deep neural networks with logic rules. In: Proceedings of the 2016 Conference on Association for Computational Linguistics; 2016.
- [57] Hu Z, Yang Z, Salakhutdinov R, et al. Deep neural networks with massive learned knowledge. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing; 2016. p. 1670–1679.
- [58] Pearl J. Probabilistic reasoning in intelligent systems: networks of plausible inference. Elsevier; 2014.
- [59] Reynolds SM, Bilmes JA. Part-of-speech tagging using virtual evidence and negative training. In: Proceedings of the conference on Human Language Technology and Empirical Methods in Natural Language Processing; Association for Computational Linguistics; 2005. p. 459–466.
- [60] Subramanya A, Bilmes J. Virtual evidence for training speech recognizers using partially labeled data. In: The Conference of the North American Chapter of the Association for Computational Linguistics; Association for Computational Linguistics; 2007. p. 165–168.
- [61] Li X. On the use of virtual evidence in conditional random fields. In: Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing: Volume 3-Volume 3; Association for Computational Linguistics; 2009. p. 1289–1297.
- [62] Parikh AP, Poon H, Toutanova K. Grounded semantic parsing for complex knowledge extraction. In:

- Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics; 2015. p. 756–766.
- [63] Ratner A, Hancock B, Dunnmon J, et al. Training complex models with multi-task weak supervision. In: Proceedings of the AAAI Conference on Artificial Intelligence; Vol. 33; 2019. p. 4763–4771.
 - [64] Mann GS, McCallum A. Generalized expectation criteria for semi-supervised learning of conditional random fields. Proceedings of ACL-08: HLT. 2008;;870–878.
 - [65] Murphy KP, Weiss Y, Jordan MI. Loopy belief propagation for approximate inference: An empirical study. In: Proceedings of the Fifteenth conference on Uncertainty in artificial intelligence; Morgan Kaufmann Publishers Inc.; 1999. p. 467–475.
 - [66] Keith K, Handler A, Pinkham M, et al. Identifying civilians killed by police with distantly supervised entity-event extraction. In: Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing; 2017. p. 1547–1557.
 - [67] CarøE CC, Schultz R. Dual decomposition in stochastic integer programming. Operations Research Letters. 1999;24(1-2):37–45.
 - [68] Domke J. Learning graphical model parameters with approximate marginal inference. IEEE transactions on pattern analysis and machine intelligence. 2013;35(10):2454–2467.
 - [69] Morgan AA, Lu Z, Wang X, et al. Overview of biocreative ii gene normalization. Genome biology. 2008; 9(2):S3.
 - [70] Kim JD, Ohta T, Pyysalo S, et al. Overview of bionlp'09 shared task on event extraction. In: Proceedings of the Workshop on Current Trends in Biomedical Natural Language Processing: Shared Task; Association for Computational Linguistics; 2009. p. 1–9.
 - [71] Quirk C, Choudhury P, Gao J, et al. MSR SPLAT, a language analysis toolkit. In: Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Demonstration Session; 2012.
 - [72] de Marneffe MC, MacCartney B, Manning CD. Generating typed dependency parses from phrase structure parses. In: Proceedings of the Fifth International Conference on Language Resources and Evaluation; 2006.
 - [73] Manning CD, Surdeanu M, Bauer J, et al. The Stanford CoreNLP natural language processing toolkit. In: Proceedings of the Fifty-Second Annual Meeting of the Association for Computational Linguistics: System Demonstrations; 2014.
 - [74] Dienstmann R, Jang IS, Bot B, et al. Database of genomic biomarkers for cancer drugs and clinical targetability in solid tumors. Cancer Discovery. 2015;5.
 - [75] Poon H, Quirk C, DeZiel C, et al. Literome: PubMed-scale genomic knowledge base in the cloud. Bioinformatics. 2014;30(19).
 - [76] Pyysalo S, Ginter F, Moen H, et al. Distributional semantics resources for biomedical text processing. In: Proceedings of LBM 2013; 2013. p. 39–44. Available from: <http://lbm2013.biopathway.org/lbm2013proceedings.pdf>.
 - [77] Wei CH, Kao HY, Lu Z. Gnormplus: an integrative approach for tagging genes, gene families, and protein domains. BioMed research international. 2015;2015.
 - [78] Hall MA. Correlation-based feature selection for machine learning. 1999;.
 - [79] Maas AL, Daly RE, Pham PT, et al. Learning word vectors for sentiment analysis. In: Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies-volume 1; Association for Computational Linguistics; 2011. p. 142–150.
 - [80] Socher R, Perelygin A, Wu J, et al. Recursive deep models for semantic compositionality over a sentiment treebank. In: Proceedings of the 2013 conference on empirical methods in natural language processing; 2013. p. 1631–1642.
 - [81] Zhang X, Zhao J, LeCun Y. Character-level convolutional networks for text classification. In: Advances in neural information processing systems; 2015. p. 649–657.
 - [82] Yang Z, Yang D, Dyer C, et al. Hierarchical attention networks for document classification. In: Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies; 2016. p. 1480–1489.

Chapter 15

Human-Centered Concept Explanations for Neural Networks

Chih-Kuan Yeh, Carnegie Mellon University Machine Learning Department

Been Kim, Google Brain

Pradeep Ravikumar, Carnegie Mellon University Machine Learning Department

15.1. Introduction

Deep neural networks have been instrumental in many modern Artificial Intelligence (AI) successes, with super-human performance across application areas such as vision recognition, speech recognition, and language understanding [1, 2, 3]. As the performance and complexity of deep neural networks improve, understanding how they operate has also become increasingly difficult. This has led to the burgeoning research area of explainable AI (XAI), which provides tools than enable us to better understand an AI model.

The initial set of XAI methods have focused on providing importance weights for either input features [4, 5, 6, 7], or for training samples [8, 9, 10]. While useful, these explanations may not necessarily effectively communicate why the model is making its predictions *at the right level of abstraction*. For example, provides importance weights to individual pixels in an image can only express which parts of that particular image is important to the model. However, human reasoning often comprises “concept-based thinking,” where we can loosely relate concepts to groupings of “similar” examples [11, 12, 13, 14]. Thus, we may obtain better human-centered explanations via *concept-based explanations*, where we explain AI models using human-centered “concepts”.

In this work, we review the emerging area of concept based explanations. We start by reviewing common classes of explanations, as well as so-called self-interpretable models that use latent variables as concepts. We then review the class of Concept Activation Vectors (CAV) which characterize concepts using vectors in appropriate spaces of neural activations. We next discuss properties of useful concepts, and approaches to measure the usefulness of concept vectors. We then discuss approaches to automatically extract concepts, and approaches to address some of their caveats. Finally, we discuss some case studies that showcase the utility of such concept-based explanations.

15.2. Related Work

While this work is mainly focused on human-centered concept explanations, we start with a brief overview of other classes of explanations. As we will discuss in the sequel, these alternative explanation approaches can in turn be used as a sub-routine within concept based explanations, for instance to explain the importance of individual concepts to the final model output. In Sec. 15.2.3, we discuss existing work on approaches that use “semantically meaningful” latent variables. These extracted latent variables are similar in spirit to concepts, but with the contrast that they are connected to bespoke generative models or specially designed neural networks, whereas concept based explanations [15] could in principle be provided for general neural network models.

Certain models are inherently interpretable e.g. small decision trees, but for others e.g. deep neural networks, we need to provide post-hoc explanations, which can in principle be applied to any given (or specific classes of) pre-trained models. Most post-hoc explanations fall under the following three categories (note that the categories are not necessarily disjoint): (a) feature-based explanation methods, that attribute the model output to input features, (b) example-based explanation methods, that attribute the model output to training samples, and (c) counterfactual explanations, which answer “what if” questions posed to the model.

15.2.1. Feature based Explanations

Feature based explanations attribute the model prediction to individual features. A large class of feature-based explanations are based on feature perturbations, and gauge the importance of features by perturbing them and measuring the prediction difference. In this line of work, [16, 17] use perturbations with grey patch occlusions on CNNs, while [18, 19, 20, 21, 22, 23] improve upon these perturbations via generative models and advanced smoothing designs. Gradient-based explanations can also be seen as a variant of perturbation based explanations via infinitesimal perturbations [24, 25, 26, 4], and which range from explicit gradients, to variants that modify back-propagation (such as ignoring negative gradients) to address some caveats with simple gradients. As shown in [27], many recent explanations such as ϵ -LRP [28], Deep LIFT [29], and Integrated Gradients [6] can also be seen as variants of gradient explanations. To reduce the noise in gradient saliency maps, [30] propose removing distractors from the image. Smooth-Grad [31] on the other hand generates artificially noisy images via additive Gaussian noise, and averages the gradient of the sampled images; due to the averaging, the resulting output is much less sensitive than vanilla gradients. [5] consider a local region characterized by local perturbations of the test input, and restricted to this region approximate the behavior of the given complex model by an locally linear interpretable model. This has been further specialized to different domains by [32, 33]. [34, 35] further compute the importance of a feature by taking the marginal contribution of the feature to any given subset of the set of all features, and computing a weighted average over all possible subsets; which has roots in cooperative game theory and revenue division.

Any such feature explanation in turn can be related to concept based explanations by simply treating the set of concepts as features to the model, and obtaining the importance weights of the concept-features. We discuss this further in Sec. 15.4.3

15.2.2. Sample based Explanations

Sample based explanations attribute the model prediction to individual training samples. Prototype selection methods [36, 37] explain a model by providing a set of “representative” samples from the training data set. [38] additionally provide criticism alongside the prototypes to explain what are not captured by prototypes. [8] provide tractable approximations to estimating *influential* training samples defined as those training examples that are the most helpful for reducing the model prediction loss. This was further accelerated by [39] by using a k-nearest-neighbors based selection over training samples. [40] use a graph over the training samples to select influential training samples. [9] use the Fisher kernel to select important training examples around the model’s decision boundary. [10] use a representer theorem for neural networks to decompose the neural network prediction as a sum of weighted kernel similarities between the test point and the training samples, which they then use to derive influential training samples. [41] propose another decomposition but of the loss of the model at the test point based on the NTK kernel between training points and the test point for different weights along the training trajectory.

Any such sample explanation in turn can be related to concept based explanations by associating clusters of the important training examples with concepts [42].

15.2.3. Semantically Meaningful Latent Variables and Self-Interpretable Concept Networks

A line of work focuses on deriving semantically meaningful latent variables or features, which can naturally be related to concepts. A popular class of these approaches use dimensionality reduction methods (such as variational encoders [43]) to derive latent features that can be connected to human-relatable concepts [44, 43], and which has shown great success in applications such as speech [45] and language [46, 47]. [48] however show that meaningful latent features cannot be derived in a completely unsupervised setting, and suggest the necessity of appropriate inductive biases.

Another line of work focuses on designing “self-interpretable” models that themselves have semantically-meaningful latent variables. [49] develop such a self-interpretable model for image classification by relating representative training patches to latent variables, while [50] develop ones for language data. [51] further use “concept networks” in their self-interpretable models. [52] extend such designs to improve their robustness. [53] propose to replace batch normalization in deep neural networks with a concept whitening module. The benefit of such self-interpretable models is that they do not make the common assumption that the concept vectors lie in a linear vector space of some neural network layer activations (which we will expand upon in the sequel). The caveat with these models on the other hand is that they are not applicable as a post-hoc explanation approach for arbitrary models.

15.3. Concept based Explanations: A Human-centered Approach

Suppose we denote the given model by $f : \mathbb{R}^d \mapsto \mathbb{R}^K$, so that given a test input $\mathbf{x} \in \mathbb{R}^d$, it outputs $f(\mathbf{x}) \in \mathbb{R}^K$ as the K scores for each of K classes. The class-specific output for

the k -th class is then given by $f_k(\mathbf{x}) \in \mathbb{R}$. We additionally use $f^{[\ell]}(\mathbf{x}) \in \mathbb{R}^{d^{[\ell]}}$ to denote the ℓ -th layer activation for test point $\mathbf{x}$. A concept can then be loosely defined as a vector that “represents” the collection of activation vectors (for the given model) corresponding to concept relevant inputs. In the next couple of sections, we discuss approaches that formalize this further.

15.3.1. CAV and TCAV

[15] introduce Concept Activation Vectors (CAV) motivated by a line of research that the linear vector space of neural activations may encode meaningful, insightful information [54, 55]. They define CAV as the normal vector to a hyperplane separating non-concept-example activations from concept-example activations. Such a hyperplane can be obtained by a binary classifier that discriminates between positive and negative examples with respect a given concept. More formally, suppose we are given set P of positive examples and a set N of negative examples with respect a given concept, for a total of n examples. Then the intermediate activations of positive examples is given by $\{f^{[\ell]}(\mathbf{x}) : \mathbf{x} \in P\}$, while those for negative examples is given by $\{f^{[\ell]}(\mathbf{x}) : \mathbf{x} \in N\}$. The concept activation vector c for the given concept can then be obtained as:

$$c = \arg \min_{c'} \min_b \sum_{j=1}^n L(y_j, f^{[\ell]}(\mathbf{x}_j) \cdot c + b),$$

where $y_j = 1$ if $\mathbf{x}_j \in P$ and $y_j = 0$ if $\mathbf{x}_j \in N$, and L is some loss function for binary classification. The concept vector thus acts as a linear discriminant in the activation vector space between positive and negative examples for a concept.

To quantify how relevant a given concept is to the prediction of the model, [15] propose the TCAV score, which is the fraction of training samples whose model classifier scores increase when the input is infinitesimally moved in the direction of the concept. To state this formally, for a concept c , class label k , and layer ℓ , and using $\mathcal{X}_k$ to denote the set of training samples with class k , we have:

$$TCAV_{c,k,\ell} = \left| \left\{ \mathbf{x} \in \mathcal{X}_k : \frac{df_k(\mathbf{x})}{df^{[\ell]}(\mathbf{x})} \cdot c > 0 \right\} \right| / |\mathcal{X}_k|.$$

A concept is considered to be related to a class label k if the TCAV score is significantly different from TCAV scores with random concepts (i.e., where concept examples are random examples).

Given *relative concepts* such finer-grained comparisons (e.g., brown hair vs. black hair), we can then obtain relative CAVs, and relative TCAV scores. These are most useful in determining which among set of concepts is most relevant to the model, in contrast to determining whether or not a concept is relevant to the model at all [56].

15.3.2. Interpretable Concept Basis

Similar to CAV, [57] define an *interpretable concept basis* based on the linear discriminant on an intermediate layer $f^{[\ell]}$, obtaining concept vectors

$$c = \arg \min_c \min_b \sum_{j=1}^n L(y_j, f^{[\ell]}(\mathbf{x}_j) \cdot c + b),$$

given binary concept labels $\{y_j\}_{j=1}^n$ for a set of inputs $\{\mathbf{x}_j\}_{j=1}^n$; following the same notation as in the earlier section. Here, they specify the intermediate layer $f^{[\ell]}$ to be the second to last layer just before the logit scores, which are thus given by:

$$f_{\text{logit}}(\mathbf{x}) = W f^{[\ell]}(\mathbf{x}) + b,$$

for a weight matrix W and bias vector b , The prediction logit for class k is then given by:

$$f_{\text{logit};k}(\mathbf{x}) = w_k \cdot f^{[\ell]}(\mathbf{x}) + b_k.$$

The goal of [57] is to then decompose the weight vectors w_k in terms of the concept vectors $\{c_j\}_{j=1}^m$, such that

$$w_k \approx \alpha_{k1}c_1 + \alpha_{k2}c_2 + \dots + \alpha_{km}c_m.$$

The weights $\alpha_k := \{\alpha_{kj}\}_{j=1}^m$ are further constrained to be non-negative since negative weights are harder to interpret; and are further constrained to have sparsity $s < m$. To address these desiderata, they propose to solve for weights $\alpha_k := \{\alpha_{kj}\}_{j=1}^m$ via the following optimization problem:

$$\arg \min_{\alpha_k > 0, |\alpha_k|_0 \leq s} \|w_k - \sum_{j=1}^m \alpha_{kj}c_j\|$$

Thus both interpretable concept bases [57] and CAVs [15] follow the same recipe for deriving concept vectors. The main distinction between them is that the former learns concepts vectors from the second-to-last layer, and focus on the use of these concept vectors as an interpretable basis for logit weights; while the latter can be applied to any user-chosen layer, and focuses on the use of these concept vectors to determine their importance to the model prediction via TCAV scores.

15.3.3. Some Empirical Evidence for Concept-Based Explanations Against Feature-Based Explanations

[15] propose an experiment based on a dataset of images, that have objects, such as cucumbers and cabs, along with a text caption, such as “cucumber” and “cab,” on the bottom of the image. By varying the noise in the image and the text caption, they obtain several models that each use only the text caption, or only the object in the image for classification. An example image is shown in Fig 1; here the model may classify this image as a car based either on the object in the image, or the text caption. The approximated ground truth of whether the model depends on the image object or text caption is determined by the performance of the model when only the object or text is provided. For instance, if the model performs well only when the image object is provided, but does not performs well when only text is provided, then the model is determined to depend on the image object.

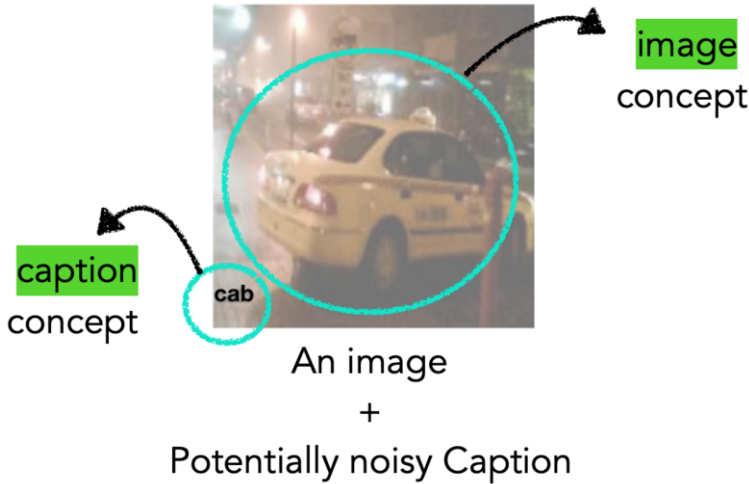


Figure 1. Example image containing both the image of a cab and the text caption of cab.

[15] then conducted a 50-person human study on Amazon Mechanical Turk, and asked users to predict whether the model depends on the text or image object for two models based on saliency map explanations [6, 31]. Of the two models, one model depends on text, and the other model depends on image object. Only 52% of users were able to predict the correct model source when given saliency map explanations, while they were almost always able to predict the correct model source when given TCAV score explanations. This is likely due to TCAV showing the importance of human-defined concepts, while saliency scores show the importance of raw features.

15.4. Properties and Evaluation Measures of Concepts

We next discuss some useful properties of concept vectors, as well as measures to evaluate the usefulness of such concept vectors.

15.4.1. When are Concepts Sufficient to Fully Explain the Model?

A limitation of TCAV scores is that they only answer the question of whether a concept is *related* to the model prediction. However, it may be unclear whether these concepts fully account for the model's prediction. For instance, several concepts may be mildly related to the model prediction, but the main concept critical to the model prediction may not have been discovered. Therefore, it would be useful to determine whether the concepts are also *sufficient* to explain the model prediction.

In this section, we discuss how to measure the sufficiency of concepts as proposed by [58]. The high level assumption here is that the concept scores for a sufficient set of concepts should be a sufficient statistic for the model output. The intuition being that if this does not hold, the model is likely basing its prediction on information *not captured* by the concepts, and thus the concepts are not “sufficient” to explain the model output.

Suppose we are analyzing a set of m concepts, with unit concept vectors $\mathbf{c} := \{\mathbf{c}_j\}_{j=1}^m$ that represent directions in the activation vector space $\text{span}(\text{range}(\phi)) \subseteq \mathbb{R}^d$. Suppose the overall model prediction can be written as $f(\mathbf{x}) = h(\phi(\mathbf{x}))$, where $h(\cdot)$ maps the activations to the model outputs. We also allow for the input $\mathbf{x} = (\mathbf{x}_t)_{t=1}^T$ to correspond to multiple parts (e.g. patches in an image, or some specified sets of input features)

We then define the part concept score for the input part $\mathbf{x}_t$ and concept vectors $\mathbf{c}$ as $v_{\mathbf{c}}(\mathbf{x}_t) := \langle \phi(\mathbf{x}_t), \mathbf{c}_j \rangle_{j=1}^m \in \mathbb{R}^m$, and the *concept score* for the entire input $\mathbf{x}$ as $v_{\mathbf{c}}(\mathbf{x}) = (v_{\mathbf{c}})_t^T \in \mathbb{R}^{T \cdot m}$. These concept scores thus capture similarities of (activations of) the input (parts) to the concept vectors.

We define “sufficient” concepts as those whose concept scores are sufficient statistics for the model output, so that we may evaluate the completeness of concepts by how well one can recover the prediction given the concept scores. Let $g : \mathbb{R}^{T \cdot m} \rightarrow \mathbb{R}^d$ denote any mapping from concept scores to the activation space. If concept scores $v_{\mathbf{c}}(\cdot)$ are sufficient statistics for the model output, then there exists some mapping g_f such that $h(g_f(v_{\mathbf{c}}(\mathbf{x}))) \approx f(\mathbf{x})$. Following this intuition, we define the **Completeness Score** $\eta_f(\mathbf{c})$ for a set of concept vectors $\mathbf{c} := \{\mathbf{c}_j\}_{j=1}^m$ as follows:

$$\eta_f(\mathbf{c}_1, \dots, \mathbf{c}_m) = \frac{\sup_g \mathbb{P}_{\mathbf{x}, y}[y = \arg \max_{y'} h_{y'}(g(v_{\mathbf{c}}(\mathbf{x})))] - a_r}{\mathbb{P}_{\mathbf{x}, y}[y = \arg \max_{y'} f_{y'}(\mathbf{x})] - a_r}, \quad (1)$$

where a_r is the accuracy of random prediction to equate the lower bound of completeness score to 0. The population accuracies above can be estimated by the empirical accuracies over validation data. Note that $\sup_g \mathbb{P}_{\mathbf{x}, y \sim V}[y = \arg \max_{y'} h_{y'}(g(v_{\mathbf{c}}(\mathbf{x})))]$ is the accuracy of the best possible classifier when only given concept scores $v_{\mathbf{c}}(\mathbf{x})$. In practice, we optimize over a flexible class of classifiers such as DNNs.

Note that the completeness score can also be used to measure how sufficient concepts are for the dataset itself independent of the model, by replacing $\phi(\cdot), h(\cdot)$ with identity functions, and the model prediction $f(\mathbf{x})$ with the label y . We provide an illustrative example below on the usefulness of the completeness score: Suppose the inputs $\mathbf{x} \in \mathbb{R}^m$, and the activation function ϕ is simply the identity function. Assume that the features $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m$ follow independent Bernoulli distributions with bias $p = 0.5$, and the model we attempt to explain is $f(\mathbf{x}) = \mathbf{x}_1 \text{ XOR } \mathbf{x}_2 \dots \text{ XOR } \mathbf{x}_m$. A natural set of m concepts $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m$ can be obtained by a one-hot encoding of each feature in $\mathbf{x}$. It can be seen that the full set of concepts $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m$ are sufficient for the model prediction, so that $\eta_f(\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m) = 1$. However, if we only have information on $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{m-1}$ but not on $\mathbf{c}_m$, then we can only do as well as random guessing in predicting the model output. In this case, $\eta_f(\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{m-1}) = 0$.

The completeness score allows us assess the “sufficiency” of the discovered concepts to “explain” the reasoning behind a model’s decision. After users manually label some set of concepts and extract concepts of interest by the TCAV score, one can then calculate the concept completeness score. If the completeness score is too low, users could then try to label additional concepts that might be relevant to the model.

15.4.2. Interventions on Concepts

Given the development of concept vectors as in the previous sections, a natural follow-up question is “how would the model prediction change if a concept were changed in some manner in the input?”.

To address this question, [59, 60] propose frameworks to estimate the effect on model output upon intervening on the concepts, with each making different assumptions on the data generative process. [60] assume the generative process follows the graphical model $\mathbf{c} \rightarrow \mathbf{x} \rightarrow y$, while [59] assume the generative process follows the graphical model $\mathbf{x} \rightarrow \mathbf{c} \rightarrow y$. Note that the latter explicitly entails that concepts scores are sufficient for the model prediction, which mirrors the sufficiency and completeness viewpoint concepts proposed by [58].

[60] define the Causal Concept Effect (CaCE) as the causal effect of a binary concept $\mathbf{c}$ on the output of the classifier f under the generative process above:

$$\text{CaCE}(\mathbf{c}, f) = \mathbb{E}[f(\mathbf{x}|\text{do}(\mathbf{c} = 1))] - \mathbb{E}[f(\mathbf{x}|\text{do}(\mathbf{c} = 0))].$$

The benefit of the causal concept effect over the TCAV score is that it captures the causal relationship between the concept and model output, while TCAV only captures the associative relationship between concept and the prediction. Consider the case where the model is solely dependent on the color of the animal (black and white). In the dataset, assume there is an equal number of cat and dogs, but 90% of the cats are white and 90% of the dogs are black. The TCAV model may easily learn that the concept “cat” is related to the model output, since the concept vectors of “cat” and “color” as estimated from this data may be close. However, in spite of this correlation, the causal concept effect may be able to learn that the concept “cat” is not causal to the model prediction. To obtain CaCE values, [60] utilize environments where intervening on concepts is possible, or to approximate these using flexible generative models such as conditional VAE models to generate input conditioning on the concept values.

[59] assume the model prediction score can be computed based solely on the concept scores. They thus train a new model where one of the intermediate layers is exactly the concepts scores for each of the concepts, and use this new model to make predictions based on the concept scores. The benefit of using this retrained model is that it is then possible to intervene on the concept scores, and answer questions such as “If the concept score was different for this input, will the model make a correct prediction?”. This also allows user to interact with models and correct certain concept scores for prediction. However, an implicit limitation is that if complete concepts are not provided, it is unclear whether the user would be able to interact with the concept scores as intended.

15.4.3. Importance Evaluations of Concepts

There are many ways to measure the “importance” of concepts for a set of concepts $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m$ and a model $f(\cdot)$. Here, we recap a set of quantitative measurements to evaluate the importance of each concept.

TCAV score [15]: The TCAV score measures the inner product between the concept vector $\mathbf{c}$ and the gradient of the model prediction for class k with respect to activations in an intermediate layer ℓ :

$$\text{TCAV}_{c,k,\ell} = \left| \left\{ \mathbf{x} \in \mathcal{X}_k : \frac{df_k(\mathbf{x})}{df^{[\ell]}(\mathbf{x})} \cdot \mathbf{c} > 0 \right\} \right| / |\mathcal{X}_k|.$$

Concept Causal Effect [60]: The concept causal effect of concept $\mathbf{c}$ measures the difference in prediction value of the model f when intervening on value of a binary concept value $\mathbf{c}$:

$$\text{CaCE}(\mathbf{c}, f) = \mathbb{E}[f(\mathbf{x}|\text{do}(\mathbf{c} = 1))] - \mathbb{E}[f(\mathbf{x}|\text{do}(\mathbf{c} = 0))].$$

ConceptSHAP [58]: Given a set of concepts $C_S = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m\}$, and a completeness score function η , the ConceptSHAP $\mathbf{s}_i$ for concept $\mathbf{c}_i$ is defined as

$$\mathbf{s}_i(\eta) = \sum_{S \subseteq C_S \setminus \mathbf{c}_i} \frac{(m - |S| - 1)!|S|!}{m!} [\eta(S \cup \{\mathbf{c}_i\}) - \eta(S)],$$

where each concept is treated as a player in a co-operative game. The Shapley value aggregates the contribution of each player based on its performance when co-operating with other players, and uniquely satisfies a natural set of game-theoretic axioms [61, 35].

TCAV^{ICS} [62]: [62] introduce $TCAV^{ICS}$, which combines TCAV scores with Integrated Gradient (IG) by projecting the IG on the intermediate layer $f^{[\ell]}(\mathbf{x})$ onto the concept direction $\mathbf{c}$. The full formulation can be written as:

$$TCAV_{\mathbf{c}, k, l, b}^{ICS} = \frac{1}{n} \sum_{i=1}^n (f^{[\ell]}(\mathbf{x}_i) - b)^T \mathbf{c} \int_{[b, f^{[\ell]}(\mathbf{x}_i)]} \mathbf{c} \cdot \frac{df_k(\mathbf{x}_i)}{df^{[\ell]}(\mathbf{x}_i)} df^{[\ell]}(\cdot).$$

Here, b is a baseline value which is set to be the activation after “removing” concept information, so that we could set $b = f^{[\ell]}(\mathbf{x}) - \lambda \mathbf{c}$, or $b = f^{[\ell]}(\mathbf{x}) - \text{projection}(f^{[\ell]}(\mathbf{x}), \mathbf{c})$.

Each of these concept importance scores can be seen to be related to methods of feature importance, where TCAV is related to the model gradient, CaCE is related to Leave-One-Out explanations (measuring effect on model output when leaving out a feature), ConceptSHAP is related to the Shapley value explanations, and $TCAV^{ICS}$ is related to the model Integrated Gradient. It would be interesting to see if other approaches for feature importance can be usefully extended to measure the importance of concepts.

15.5. Unsupervised Discovery of Concepts

One of the main caveats for CAVs is that we need to provide training examples for each of the concept. These typically have to be manually labeled, which however may often not be possible or too costly.

A natural question is whether can one discover concepts in the dataset that are human interpretable in an *unsupervised manner*? We briefly review a few approaches that address this question. [63] discover concepts in images by super-pixel segmentation followed by k-means clustering. [58] discover concepts in image and language data by learning concepts that are representative of clusters of training inputs. [64] find disentangled and diverse concepts (which they term a concept trajectory) by leveraging counterfactuals.

[63] propose the desiderata of “Meaningfulness”, “Coherency”, “Importance” for concept explanations. To obtain meaningful concepts, they use super-pixel segmentation

to mimic the process of humans finding semantically meaningful parts of the image data. To obtain meaningful and coherent parts of the data, they propose to use k-means clustering of the super-pixel segments. Finally, they use TCAV scores to determine if the k-means clusters are related to the model prediction for any specific class. [58] aim to find concepts that are “complete” with respect to the model, while also ensuring the concepts are coherent and meaningful by: (a) limiting each concept to be a contiguous part of the input data (a patch of image or a sub-sentence in language data), and (b) enforcing each concept to be close to its top-k nearest neighbors in the training data. The first constraint increases the meaningfulness of the concept, as contiguous parts of the data are more interpretable. The second constraints also increases human interpretability of the concepts as users can better understand the concept by observing the top-k nearest neighbor training examples. [64] aim to find k concepts by generating counterfactuals close to the image of interest $\mathbf{x}$. They use a generative model $G_{\mathbf{x}}(\alpha, c)$ they call DISSECT that generates counterfactuals of any test image $\mathbf{x}$ by only changing the concept c , and where the model prediction on the counterfactual is some predefined value α , so that $f(G_{\mathbf{x}}(\alpha, c)) = \alpha$. They then use this generative model to learn the concepts themselves.

A less ambitious approach might be to simply *propose* potential concepts in an unsupervised manner, and which can then be identified and labeled by human experts.[65] provide a visualization of random directions in the activation space in real-time, and which users can rotate around and label as concepts if they find a direction that is meaningful to them. Another approach is to use statistical test to find interesting directions. While not completely unsupervised, this significantly lowers the labeling cost for concept vectors.

15.5.1. Undesired Correlation and Dependence between Concepts and Labels

A caveat with the approaches above is that they might learn concepts that are predictive of the class label due to so-called spurious associations and correlations between the concepts and the label, but which are not causally related to the label [66, 67]. A natural reason this might occur is due to what is known as confounding. As an example, consider an MNIST digit classification task of predicting whether a digit is even or odd. Then concept scores for say specific digits of ‘4’ and ‘5’ are predictive of the class label, but are perhaps not the right concepts to be learnt. Note that even for a random concept via a random direction in the activation space, the concept scores which are the projection of the input onto the concept direction still encodes predictive information. From a generative process viewpoint, this implies that some confounding variables may exist between input data and concepts, and thus concept scores which are not causally related to the model prediction nonetheless have predictive information.

To account for the confounding variables between input data and concepts, [68] propose the following generative model. Let u denote the confounding variable between the input data $\mathbf{x}$ and the concepts $\mathbf{c}$. Let d denote the unconfounded (unbiased) concepts that are independent of the confounding variable u . The generative model is then given by:

$$d = f_1(y) + \epsilon_1, \quad (2)$$

$$c = d + h(u), \quad (3)$$

$$\mathbf{x} = f_2(u, d) + f_3(y) + \epsilon_2, \quad (4)$$

where $\varepsilon_1, \varepsilon_2$ are independent noise variables, and f_1, f_2, f_3 are deterministic functions. [68] show that debiased concepts from the generative model above can retrieve higher quality concepts in both simulation and real world datasets. Note that even if we use debiased concepts above, they never be fully complete in the sense of [58], as $\mathbf{x}$ may contain more predictive information (due to spurious correlations with the label) when compared to the debiased concepts.

A related idea to the debiasing of concepts is to “whiten” the concepts (so that the covariance of the concepts is the identity matrix). [53] suggest using concept whitening modules within a deep neural network which they suggest learn concepts that do not rely on the usual assumptions that concept vectors lie in the linear vector space of neural activations [15]. However, [66] show that there is information leakage even when you whiten the concept, since decorrelating concept representations does not remove all statistical dependence. They thus suggest minimizing the mutual information between concept scores to prevent information leakage of soft concept representations.

15.6. Evaluations of Concept Based Explanations

Given a set of concepts $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_m$, and a model $f(\cdot)$, how does one evaluate the quality of the set of discovered concepts? We first discuss the setting where the ground truth concepts are unknown, and then provide some case studies where we do know the underlying concepts.

15.6.1. When Ground Truth Concepts Are Unknown

While most work on concept explanations use specific datasets where ground truth concepts can be retrieved, we next discuss some evaluation measures that can be used even when the ground truth concepts are unknown.

Necessary and Sufficient Concepts [63]: One way to evaluate a set of concepts is to mask out (or only keep) the top k most important concepts in the training data. If the set of concepts is useful for prediction, removing (only keeping) the top- k concepts should result in a large drop (increase) of the model performance.

Completeness Score [58]: The evaluation of the completeness score can also be performed in the absence of knowledge of ground-truth concepts. It can be seen to be similar to the remove and retrain framework above, but where we only partially retrain the part of the model from the intermediate layer activation space to the final prediction. This is because the concept scores of complete concepts are assumed to be a sufficient statistic for the model prediction f .

Synthetic Datasets [69]: [69] introduces the BAM dataset where objects can be synthesized in different scenes. They then train two models, where one is trained with labels of the objects and one is trained with labels of the scene. They then proposes the model contrast scores (MCS), which measure the difference of concept scores of objects in the two models. The concept score of objects should be larger when the object model is used, and the concept score of scenes should be larger when the scene model is used.

One caveat with the first approach to evaluate concepts is that it involves perturbing the input with respect to the concepts, which may take us out of the data manifold.

With model contrast scores, the effect of true concepts could be correctly evaluated, but the concept space and the model space would be limited to those in the BAM dataset. Completeness scores can be used without perturbing images and without synthetic data. However, it does not take into account end-task performance.

15.6.2. Case Study: Building Post-Hoc Symbolic Explanations for Sequential Decision-Making Problems by User-Defined Concept

Concept-Based explanations have the benefit of speaking the language of users, and can also be used as a base unit to build symbolic explanations with user-defined concepts. We next describe an example of such a partially-symbolic model based on human-defined concepts [70]. In the game Montezuma's Revenge, a popular Atari game, the goal is to score points by gathering jewels and killing enemies along the way. Concepts labeled by humans include "skeleton on left", "left door closed", and "not on ladder". A symbolic approximation of an RL agent can then be learned from the concept scores to the model output with preconditions and cost functions similar to the STRIPS model [71]. The symbolic approximation model chooses an action if the current state satisfies the preconditions of the action, and if the action has the lowest cost function of all viable actions.

Based on this symbolic approximation model, [70] propose to generate contrastive explanations for why the model is not performing a "foil action," which is an alternate plan specified by users. Based on the symbolic action model, the model presents explanations of the form "the foil action A is not chosen because precondition B is not satisfied" or "the foil action A is not chosen because its predicted cost C is higher than the current plan with cost D". As an example, the model could present an explanation that the foil action "go left" is not chosen because precondition "skull not on left" is not satisfied, or that the foil action "attack" is not chosen because the cost of attack is at least 500 when "skull on left", but the cost of the chosen plan is 20. While this proposed algorithm relies on the assumption that it is possible to approximate the applicability of actions and cost functions in terms of high-level concepts and via a symbolic action model, which may be not satisfied for more complex environments, this nevertheless sheds light on how concept explanations can bridge the gap between black-box model and symbolic reasoning as explanations.

15.6.3. Broader Applications of Concept-Explanations

In addition to extensions in the machine learning community, concept-based explanations have been widely introduced to application areas beyond the machine learning community. In the medical domain, using concept-based explanations [72] show that nuclei texture is a relevant concept in detecting tumor tissue in breast lymph node samples. [73] show that ventricular ejection and filling rates concepts are crucial in cardiac MRI classification. [74] discover that radiomic features describing increased entropy, as well as those describing variations of intensity are useful concepts for the prediction of calcifications. In scientific domains, using concept-based explanations, [75] show that the "Eye" concept is important for the prediction of Category 4 tropical cyclones in a CNN model. [76] show that concepts such as antibiotics and one class of nephrotoxic drugs, non-steroidal anti-inflammatory drugs (NSAIDs), are significant for the prediction of Acute Kidney Injury in time series data.

The adaptation of human-centered explanations in such domains where efficiently communicating with highly skilled experts is crucial showcases the potential of human-centered explanation in real-world problems.

15.7. Conclusion and Discussion

We have provided an overview of current advances in concept-based explanations to explain complex machine learning models. Concepts can be seen as a way to bridge the gap between the reasoning of humans and machine learning models. Instead of having humans speak the language of machines, in terms of raw input features and training samples, concept-based explanations aim to speak the language of humans, via concepts. Concepts and their scores, as defined Sec. 15.4.3, can be used as quantitative tool for such translation. We have seen that concept-based methods can be combined with feature-based and sample-based explanations to leverage their strengths, and can also be used as a basis for contrastive explanations. The human-centered nature of concept explanations has also made concept explanation successful in several real-world applications, especially when highly skilled professionals with established domain concepts (e.g., medicine) are involved.

Despite this exciting progress, many interesting open questions remain. How can we discover unknown confounding concepts? Can we extend concept explanations to models such as tree models, where the assumption that concept vectors are directions in a linear vector space need no longer hold? How can we identify the intrinsic properties of what makes a concept understandable to humans? For instance, in the high dimension activation space in the neural network, why are some vectors/directions understandable to human, while some others are not? Can we teach humans new concepts by identifying training examples that can be discriminated by directions in the activation space? More generally, can we use the richness of concepts to convey knowledge from a super-human-performance model to humans?

References

- [1] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition; 2016. p. 770–778.
- [2] Hinton G, Deng L, Yu D, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal processing magazine*. 2012;29(6):82–97.
- [3] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: *Advances in neural information processing systems*; 2017. p. 5998–6008.
- [4] Selvaraju RR, Cogswell M, Das A, et al. Grad-cam: Visual explanations from deep networks via gradient-based localization. *International conference on computer vision*. 2017;.
- [5] Ribeiro MT, Singh S, Guestrin C. Why should i trust you?: Explaining the predictions of any classifier. In: *KDD*; ACM; 2016.
- [6] Sundararajan M, Taly A, Yan Q. Axiomatic attribution for deep networks. In: *Proceedings of the 34th International Conference on Machine Learning-Volume 70*; JMLR. org; 2017. p. 3319–3328.
- [7] Yeh CK, Hsieh CY, Suggala AS, et al. On the (in)fidelity and sensitivity of explanations. In: *NeurIPS*; 2019.
- [8] Koh PW, Liang P. Understanding black-box predictions via influence functions. In: *ICML*; 2017.
- [9] Khanna R, Kim B, Ghosh J, et al. Interpreting black box predictions using fisher kernels. In: *The 22nd International Conference on Artificial Intelligence and Statistics*; 2019. p. 3382–3390.

- [10] Yeh CK, Kim J, Yen IEH, et al. Representer point selection for explaining deep neural networks. In: NeurIPS; 2018.
- [11] Armstrong SL, Gleitman LR, Gleitman H. What some concepts might not be. *Cognition*. 1983;13(3):263 – 308.
- [12] Tenenbaum JB. A bayesian framework for concept learning [dissertation]. Massachusetts Institute of Technology; 1999.
- [13] Margolis E, Laurence S, et al. Concepts: core readings. Mit Press; 1999.
- [14] Rosch E. Reclaiming concepts. *Journal of consciousness studies*. 1999;6(11-12):61–77.
- [15] Kim B, Wattenberg M, Gilmer J, et al. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). In: ICML; 2018.
- [16] Zeiler MD, Fergus R. Visualizing and understanding convolutional networks. In: European conference on computer vision; Springer; 2014. p. 818–833.
- [17] Fong RC, Vedaldi A. Interpretable explanations of black boxes by meaningful perturbation. 2017 IEEE International Conference on Computer Vision (ICCV). 2017;:3449–3457.
- [18] Zintgraf LM, Cohen TS, Adel T, et al. Visualizing deep neural network decisions: Prediction difference analysis. In: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings; 2017. Available from: <https://openreview.net/forum?id=BJ5UeU9xx>.
- [19] Chang CH, Creager E, Goldenberg A, et al. Explaining image classifiers by counterfactual generation. In: International Conference on Learning Representations; 2019. Available from: <https://openreview.net/forum?id=B1MXz20cYQ>.
- [20] Fong R, Patrick M, Vedaldi A. Understanding deep networks via extremal perturbations and smooth masks. In: Proceedings of the IEEE/CVF International Conference on Computer Vision; 2019. p. 2950–2958.
- [21] Petsiuk V, Das A, Saenko K. Rise: Randomized input sampling for explanation of black-box models. arXiv preprint arXiv:180607421. 2018;.
- [22] Dabkowski P, Gal Y. Real time image saliency for black box classifiers. In: NeurIPS; 2017.
- [23] Fong R, Vedaldi A. Explanations for attributing deep neural network predictions. In: Explainable ai: Interpreting, explaining and visualizing deep learning. Springer; 2019. p. 149–167.
- [24] Baehrens D, Schroeter T, Harmeling S, et al. How to explain individual classification decisions. *Journal of Machine Learning Research*. 2010;11(Jun):1803–1831.
- [25] Simonyan K, Vedaldi A, Zisserman A. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:13126034. 2013;.
- [26] Springenberg JT, Dosovitskiy A, Brox T, et al. Striving for simplicity: The all convolutional net. arXiv preprint arXiv:14126806. 2014;.
- [27] Ancona M, Ceolini E, Oztireli C, et al. A unified view of gradient-based attribution methods for deep neural networks. International Conference on Learning Representations. 2018;.
- [28] Bach S, Binder A, Montavon G, et al. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PloS one*. 2015;10(7):e0130140.
- [29] Shrikumar A, Greenside P, Kundaje A. Learning important features through propagating activation differences. International Conference on Machine Learning. 2017;.
- [30] Kindermans PJ, Schütt KT, Alber M, et al. Patternnet and patternlrp—improving the interpretability of neural networks. International Conference on Learning Representations. 2018;.
- [31] Smilkov D, Thorat N, Kim B, et al. Smoothgrad: removing noise by adding noise. arXiv preprint arXiv:170603825. 2017;.
- [32] Ying R, Bourgeois D, You J, et al. Gnn explainer: A tool for post-hoc explanation of graph neural networks. arXiv preprint arXiv:190303894. 2019;.
- [33] Plumb G, Molitor D, Talwalkar AS. Model agnostic supervised local explanations. In: Advances in Neural Information Processing Systems; 2018. p. 2515–2524.
- [34] Datta A, Sen S, YZ. Algorithmic transparency via quantitative input influence: Theory and experiments with learning systems. In: Security and Privacy (SP), 2016 IEEE Symposium on; IEEE; 2016. p. 598–617.
- [35] Lundberg SM, Lee SI. A unified approach to interpreting model predictions. In: NeurIPS; 2017.
- [36] Bien J, Tibshirani R. Prototype selection for interpretable classification. *The Annals of Applied Statistics*. 2011;:2403–2424.
- [37] Kim B, Rudin C, Shah JA. The bayesian case model: A generative approach for case-based reasoning

- and prototype classification. In: *Advances in Neural Information Processing Systems*; 2014. p. 1952–1960.
- [38] Kim B, Khanna R, Koyejo OO. Examples are not enough, learn to criticize! criticism for interpretability. In: *NeurIPS*; 2016.
 - [39] Guo H, Rajani NF, Hase P, et al. Fastif: Scalable influence functions for efficient model interpretation and debugging. *arXiv preprint arXiv:201215781*. 2020;.
 - [40] Anirudh R, Thiagarajan JJ, Sridhar R, et al. Influential sample selection: A graph signal processing approach. *arXiv preprint arXiv:171105407*. 2017;.
 - [41] Pruthi G, Liu F, Kale S, et al. Estimating training data influence by tracing gradient descent. *Advances in Neural Information Processing Systems*. 2020;33.
 - [42] Ghorbani A, Wexler J, Zou J, et al. Towards automatic concept-based explanations. In: *Advances in Neural Information Processing Systems*; 2019.
 - [43] Kingma DP, Welling M. Auto-encoding variational bayes. *arXiv preprint arXiv:13126114*. 2013;.
 - [44] Chan TH, Jia K, Gao S, et al. Pcanet: A simple deep learning baseline for image classification? *IEEE transactions on image processing*. 2015;24(12):5017–5032.
 - [45] Chorowski J, Weiss RJ, Bengio S, et al. Unsupervised speech representation learning using wavenet autoencoders. *arXiv preprint arXiv:190108810*. 2019;.
 - [46] Radford A, Józefowicz R, Sutskever I. Learning to generate reviews and discovering sentiment. *arXiv:170401444*. 2017;.
 - [47] Grover S, Pulice C, Simari GI, et al. Beef: Balanced english explanations of forecasts. *IEEE Transactions on Computational Social Systems*. 2019;6(2):350–364.
 - [48] Locatello F, Bauer S, Lucic M, et al. Challenging common assumptions in the unsupervised learning of disentangled representations. *arXiv preprint arXiv:181112359*. 2018;.
 - [49] Chen C, Li O, Tao D, et al. This looks like that: deep learning for interpretable image recognition. In: *Advances in Neural Information Processing Systems*; 2019. p. 8928–8939.
 - [50] Shi T, Zhang X, Wang P, et al. Corpus-level and concept-based explanations for interpretable document classification. *arXiv preprint arXiv:200413003*. 2020;.
 - [51] Bouchacourt D, Denoyer L. Educe: Explaining model decisions through unsupervised concepts extraction. *arXiv preprint arXiv:190511852*. 2019;.
 - [52] Alvarez-Melis D, Jaakkola TS. Towards robust interpretability with self-explaining neural networks. In: *Proceedings of the 32nd International Conference on Neural Information Processing Systems*; 2018. p. 7786–7795.
 - [53] Chen Z, Bei Y, Rudin C. Concept whitening for interpretable image recognition. *Nature Machine Intelligence*. 2020;2(12):772–782.
 - [54] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:13013781*. 2013;.
 - [55] Pennington J, Socher R, Manning CD. Glove: Global vectors for word representation. In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*; 2014. p. 1532–1543.
 - [56] Cai CJ, Reif E, Hegde N, et al. Human-centered tools for coping with imperfect algorithms during medical decision-making. *CoRR*. 2019;abs/1902.02960. Available from: <http://arxiv.org/abs/1902.02960>.
 - [57] Zhou B, Sun Y, Bau D, et al. Interpretable basis decomposition for visual explanation. In: *ECCV*; 2018.
 - [58] Yeh CK, Kim B, Arik S, et al. On completeness-aware concept-based explanations in deep neural networks. *Advances in Neural Information Processing Systems*. 2020;33.
 - [59] Koh PW, Nguyen T, Tang YS, et al. Concept bottleneck models. *arXiv preprint arXiv:200704612*. 2020;.
 - [60] Goyal Y, Feder A, Shalit U, et al. Explaining classifiers with causal concept effect (cace). *arXiv preprint arXiv:190707165*. 2019;.
 - [61] Shapley LS. A value for n-person games; 1988. p. 31–40.
 - [62] Schrouff J, Baur S, Hou S, et al. Best of both worlds: local and global explanations with human-understandable concepts. *arXiv preprint arXiv:210608641*. 2021;.
 - [63] Ghorbani A, Wexler J, Zou J, et al. Towards automatic concept-based explanations. *NeurIPS*. 2019;.
 - [64] Ghandeharioun A, Kim B, Li CL, et al. Dissect: Disentangled simultaneous explanations via concept traversals. *arXiv preprint arXiv:210515164*. 2021;.
 - [65] Adrianna Janik KS. Discovering concepts in learned representations using statistical inference and in-

- teractive visualization. KDD. 2019;.
- [66] Mahinpei A, Clark J, Lage I, et al. Promises and pitfalls of black-box concept learning models. arXiv preprint arXiv:210613314. 2021;.
 - [67] Margeloiu A, Ashman M, Bhatt U, et al. Do concept bottleneck models learn as intended? arXiv preprint arXiv:210504289. 2021;.
 - [68] Bahadori MT, Heckerman D. Debiasing concept-based explanations with causal analysis. In: International Conference on Learning Representations; 2020.
 - [69] Yang M, Kim B. Benchmarking attribution methods with relative feature importance. arXiv. 2019; :arXiv-1907.
 - [70] Sreedharan S, Soni U, Verma M, et al. Bridging the gap: Providing post-hoc symbolic explanations for sequential decision-making problems with black box simulators. arXiv preprint arXiv:200201080. 2020; .
 - [71] Geffner H, Bonet B. A concise introduction to models and methods for automated planning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*. 2013;8(1):1–141.
 - [72] Graziani M, Andrearczyk V, Müller H. Regression concept vectors for bidirectional explanations in histopathology. In: *Understanding and interpreting machine learning in medical image computing applications*. Springer; 2018. p. 124–132.
 - [73] Clough JR, Oksuz I, Puyol-Antón E, et al. Global and local interpretability for cardiac mri classification. In: *International Conference on Medical Image Computing and Computer-Assisted Intervention*; Springer; 2019. p. 656–664.
 - [74] Yeche H, Harrison J, Berthier T. Ubs: A dimension-agnostic metric for concept vector interpretability applied to radiomics. In: *Interpretability of machine intelligence in medical image computing and multimodal learning for clinical decision support*. Springer; 2019. p. 12–20.
 - [75] Conner Sprague EBW, Guch I. Interpretable ai for deep learning- based meteorological applications. In: *American Meteorological Society Annual Meeting*. AMS; 2019.
 - [76] Mincu D, Loreaux E, Hou S, et al. Concept-based model explanations for electronic health records. In: *Proceedings of the Conference on Health, Inference, and Learning*; 2021. p. 36–46.

Chapter 16

Abductive Learning

Zhi-Hua Zhou, Nanjing University, Nanjing 210023, China

Yu-Xuan Huang, Nanjing University, Nanjing 210023, China

16.1. Introduction

To develop a unified framework which accommodates and enables machine learning and logical reasoning to work together effectively is a well-known holy grail problem in artificial intelligence. It is often claimed that advanced intelligent technologies could emerge when machine learning and logical reasoning can be seamlessly integrated as human beings generally perform problem-solving based on the leverage of perception and reasoning, where perception corresponds to a data-driven process that can be realized by machine learning whereas reasoning corresponds to a knowledge-driven process that can be realized by logical reasoning. In the history of artificial intelligence research, however, machine learning and logical reasoning are almost separately developed: Before the 1990s most attention was paid to logical reasoning (and knowledge engineering) whereas after the 1990s machine learning becomes mainstream while logical reasoning receives less attention.

Among the obstacles encumbering the connection of logical reasoning with machine learning lies in their very different representations. Generally, popular logical reasoning techniques are based on first-order logic representation, whereas popular machine learning techniques are based on attribute-value representation. The latter can be regarded as an equivalence of propositional logic representation, as the attribute-value pairs can be converted to truth tables; such a conversion, however, could not bridge machine learning and logical reasoning directly as there exist big gaps. For example, consider the issues that may emerge when one tries to convert a first-order logical clause into a set of attribute-value represented data: due to the existence of the universal quantifier $\forall$, one clause may lead to infinite instances; each predicate may actually express a kind of relation among instances, and therefore, the conversion may lead to constraints about possible value-takings of a subset of instances rather than definite attribute-values of an individual instance; etc.

Nevertheless, as the holy grail problem is fundamentally important and attractive, great efforts have been made in the past few decades. Roughly speaking, there are two general paradigms for the integration of logical reasoning and machine learning. The first paradigm attempts to introduce some elements of machine learning into logical rea-

soning, as represented by PLP (Probabilistic Logic Program) [1] where the first-order logic can be extended to accommodate probabilistic groundings such that probabilistic inference can be conducted. The second paradigm attempts to introduce some elements of logical reasoning into machine learning, as represented by SRL (Statistical Relational Learning) [2] where a probabilistic learning model can be constructed/initialized based on domain knowledge expressed in the first-order logic clauses.

The first paradigm can be described as a *heavy-reasoning light-learning* way, because the most workload is to be finished by logical reasoning though some elements of machine learning are introduced; e.g., probabilistic elements are introduced in the beginning but then the remaining problem-solving procedure relies almost totally on logical reasoning. In contrast, the second paradigm can be described as a *heavy-learning light-reasoning* way, because the most workload is to be finished by machine learning though some elements of logical reasoning are introduced; e.g., a graphical model is initialized based on the first-order logic clauses in the beginning but then the remaining problem-solving procedure relies almost totally on machine learning.

This chapter will introduce ABL (ABductive Learning) [3], a new paradigm which integrates machine learning and logical reasoning in a *balanced-loop* enabling them to work together in a mutually beneficial way, as shown in Figure 1.

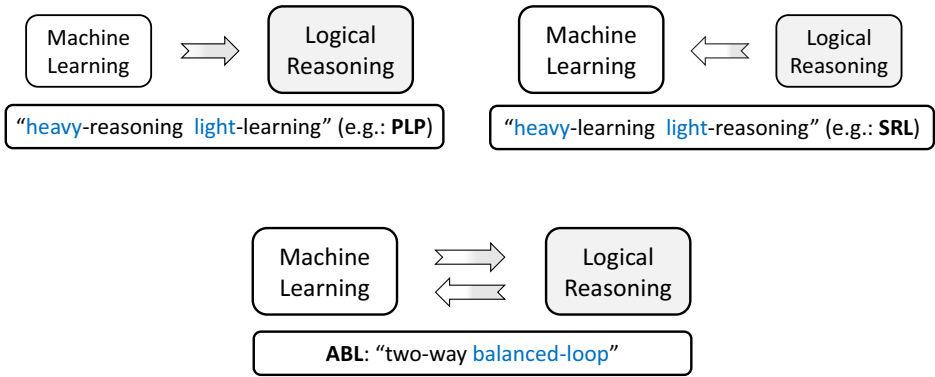


Figure 1. Three paradigms for integrating machine learning and logical reasoning.

The rest of this chapter is organized as follows. Section 16.2 introduces the ABL paradigm; Section 16.3 presents an example implementation; Section 16.4 presents a law-enforcement application; followed by conclusion.

16.2. The Paradigm

16.2.1. Motivating Story: Mayan Calender Decipherment

The Mayan calendar was used by the Mayan civilization in Central America [4, 5]. The system is remarkably intricate, consisting of three interlacing calendars, i.e.,

- *Long Count* which works by counting the number of days from the Creation in the Mayan legend. It appears as xx.xx.xx.xx.xx, like an IP address, which actually

Copyright © 2022, IOS Press, Incorporated. All rights reserved.

represents a day tallied in a modified base-20 scheme, where the second number from the right resets after only counting to 18; e.g., “1.16.5.4.12” represents the $((1 \times 20 + 16) \times 20 + 5) \times 18 + 4 \times 20 + 12 = 261,092$ day from the Creation.

- *Tzolk'in* which combines two parallel cycles of thirteen numbers and twenty names, e.g., “1 Ahau”, leading to 260 days per year.
- *Haab'* which has 360 days per year divided into 18 months of 20 days represented by a number in the month followed by the name of the month, e.g., “13 Mac”, plus a period of five “nameless” days at the end of the year.

All the numbers and names were represented by hieroglyphs. Due to many reasons such as the variance of handiwork, one number/name can be represented by hieroglyphs that look quite different. Figure 2 exhibits a Mayan calendar pillar discovered from the Palenque Temple of the Cross Complex [5], where the places corresponding to the *Long Count*, *Tzolk'in*, and *Haab'* calendars have been highlighted in three rectangles with different colors and line types. Some hieroglyphs have been deciphered, such as “18”, “5”, “0” in *Long Count*, “Ahau” in *Tzolk'in*, and “13” and “Mac” in *Haab'*. Now the task is to decipher the three unknown hieroglyphs marked by “?”.

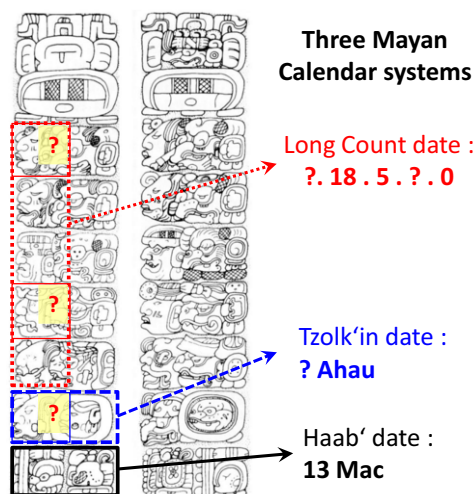


Figure 2. The task is to decipher the three unknown hieroglyphs.

Archaeologists started by trying to recognize the unknown hieroglyphs based on their previous experience. Unfortunately, they could only confirm that the two hieroglyphs in red boxes with dashed line in Figure 3 should be the same number, but it might be “1” or “8” or “9”, a kind of conjecture.

Then, based on the known hieroglyphs (“18”, “5”, “0”, “Ahau”, “13”, “Mac”) in Figure 2, archaeologists explored all possible conjectures, i.e., how if the hieroglyphs in red boxes with dashed line were “1” or “8” or “9”, as well as the conjectures about the hieroglyph in blue box with solid line. In total there were 16 possibilities as shown in Figure 3. Archaeologists’ knowledge offered an important cue, i.e., the dates specified by the three calendars must be consistent because they appeared together in the same pillar; in other words, the three calendars pointed to the same day. Actually, among the 16 rows

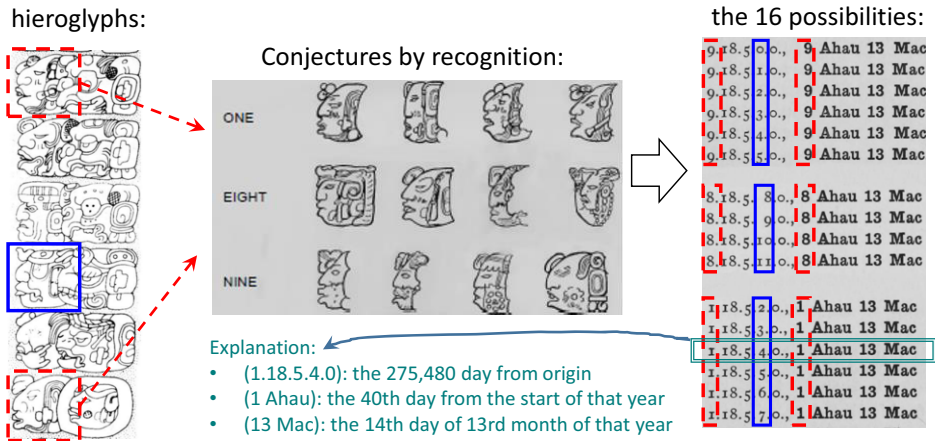


Figure 3. The decipherment procedure: A preliminary recognition conjectured that the hieroglyphs in red boxes with dashed line could be “1” or “8” or “9”; the knowledge instructed that the three calendars must be consistent, leading to the identification of the row which deciphered the unknown hieroglyphs to be “1” and “4”, respectively.

shown in Figure 3, there is one row whose three components are consistent, leading to the decipherment of the hieroglyphs in red boxes with dashed line/blue box with solid line as “1” and “4”, respectively. The problem solved.

Revisiting the whole procedure, one can see that the archaeologists were working with incomplete observed facts, i.e., some hieroglyphs were known but some were unknown; they had some knowledge about the task; the final result inferred the most likely explanation (deciphered numbers) to the unknown hieroglyphs. This reassembles *abduction*: Starting from incomplete observations, and proceeds to the likeliest possible explanation for the set.

More precisely, the abduction, also called *retro-production*, refers to the process of selectively inferring certain facts and hypotheses that explain phenomena and observations based on background knowledge [6]. Given observed facts and background knowledge expressed as first-order logic clauses, logical formalization of abductive reasoning, i.e., logical abduction, can abduce hypotheses as possible explanations to the observed facts.

Abduction [7] has been a recurring topic of interest in the field of AI, and in the history it had been tried to integrate with symbolic *induction* [8, 9]. Considering that modern machine learning is also a kind of induction, logical abduction has been exploited to introduce symbolic knowledge into statistical learning [10]. In fact, the abduction and learning should be naturally leveraged together, just like in the aforementioned abductive problem-solving process for the Mayan calendar decipherment problem where an induction component should be involved to enable archaeologists to recognize hieroglyphs based on their experience. With this understanding, ABL [3] attempts to leverage learning and abduction in a mutually beneficial loop, which exhibits a new paradigm towards integrating machine learning and logical reasoning in a unified framework.

16.2.2. The ABL Paradigm

In conventional supervised learning, e.g., binary classification, a set of training examples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ is given, where $\mathbf{x}_i \in \mathcal{X}$ is the i -th training instance, $y_i \in \mathcal{Y}$ is the corresponding ground-truth class label; the task is to learn a function $f: \mathcal{X} \mapsto \mathcal{Y}$ from the input space $\mathcal{X}$ to the output space $\mathcal{Y}$ to accurately predict the unseen data. As shown in Figure 4, the parts in black color with bold font are given in advance.

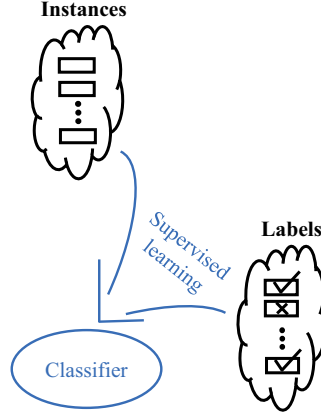


Figure 4. Conventional supervised learning.

In abductive learning [3], in addition to the training instances $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$, a knowledge base $\mathcal{KB}$ and an initial classifier $\mathcal{C}$ are available; the task is to learn a function $f: \mathcal{X} \mapsto \mathcal{Y}$ for making correct predictions on unseen data, and meanwhile, the logical facts grounded by $\{(\mathbf{x}_1, f(\mathbf{x}_1)), \dots, (\mathbf{x}_m, f(\mathbf{x}_m))\}$ should be compatible with $\mathcal{KB}$. Formally, given $\{\mathbf{x}_1, \dots, \mathbf{x}_i\}$, $\mathcal{C}$ and $\mathcal{KB}$, the task is to seek a function f such that

$$\{\mathbf{x}_1, \dots, \mathbf{x}_i\}, f \triangleright \mathcal{O} \quad (1)$$

$$\text{s.t.} \quad \mathcal{KB} \models \mathcal{O}, \text{ or} \quad (2)$$

$$\mathcal{KB} \models \Delta(\mathcal{O}), f \leftarrow \psi(f, \Delta(\mathcal{O})), \quad (3)$$

where $\mathcal{O}$ denotes the logical facts grounded by $\mathbf{x}_i$'s and f in Eq.(1). If $\mathcal{O}$ is consistent with $\mathcal{KB}$ as shown in Eq.(2), the current f and the corresponding $\mathcal{O}$ will be returned; otherwise, $\Delta(\mathcal{O})$ compatible with $\mathcal{KB}$ will be generated via logical abduction as specified by the first equation in Eq.(3), and f will then be updated based on $\Delta(\mathcal{O})$ by a process ψ . If $\mathcal{O}$ is not compatible with $\mathcal{KB}$, i.e., $\mathcal{KB} \not\models \mathcal{O}$ and there does not exist Δ enabling $\mathcal{KB} \models \Delta(\mathcal{O})$, the process terminates and return False.

Figure 5 presents a flowchart of ABL, where the given information is highlighted in black color with bold font, including the training instances, an initial classifier and a knowledge base. The initial classifier works as similar as the archaeologists' brains in the Mayan calendar decipherment task: to generate conjectures (i.e., *pseudo-labels*) about the observations (i.e., training instances). The pseudo-labels are converted to pseudo-grounded facts such as the $A, \neg B, \dots, \neg C$ shown in Figure 5, corresponding to $\mathcal{O}$ in Eq.(1). Then, a logical reasoning process is applied to verify whether these pseudo-

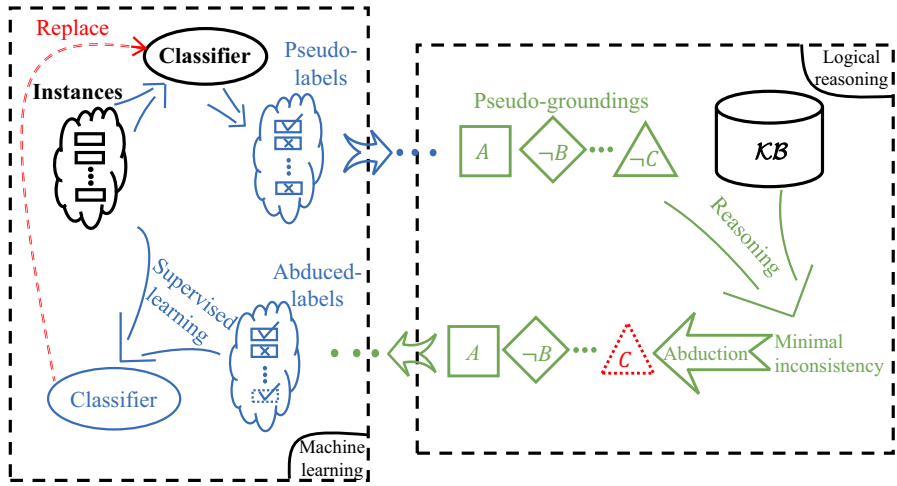


Figure 5. Abductive learning where a classifier and a knowledge base are available; the machine learning and logical reasoning components are shown in blue and green on the left and right, respectively. The given classifier helps generate pseudo-labels, leading to pseudo-groundings; then, the revisions to the pseudo-groundings (shown as a red triangle with dotted line) are generated via logical abduction based on minimizing the inconsistency with the knowledge base. The abduced-labels are used to train the classifier, which then replaces the original classifier in the next iteration.

groundings are consistent with the knowledge base. If the pseudo-groundings are inconsistent, a hypothetical revision to the pseudo-groundings based on minimizing the inconsistency with the knowledge base, will be generated via logical abduction. For example, suppose that the $\neg C$ in the pseudo-groundings in Figure 5 has been hypothetically suggested to change to C as the minimal revision, corresponding to $\Delta(\mathcal{O})$ in Eq.(3). Then, the revision of pseudo-groundings leads to the revision of some pseudo-labels to generate the *abduced-labels*. The abduced-labels will be used like ground-truth labels in conventional supervised learning to train a new classifier, which will then be adopted to replace the original classifier, corresponding to the ψ process in Eq.(3). The above process repeats, until the classifier is no longer updated, or the logical facts are compatible with the knowledge base.

The initial classifier in Figure 5 can be simple, such as preprocessing based on clusterings or nearest neighbor classification. In practice, it can be a pre-trained model, e.g., a deep learning model pre-trained from benchmark datasets, or a result of transfer learning from relevant tasks, or even a reusable learnware [11]. If abundant labeled data are given, the initial classifier is less required because the logical groundings can be directly generated by the labeled data; in this sense, labeled data can be regarded as an alternative to an initial classifier.

Though neither Eq.(1) nor Figure 5 relies on ground-truth labels, it does not mean that abductive learning cannot utilize ground-truth labels. Actually, if some ground-truth labels are available, the pseudo-grounded facts are expected to be more reliable and the logical abduction can be more effective, because the ground-truth labels can help prune the huge hypothesis space for abduction. Ground-truth labels can also be used to train the final classifier, leading to potential improvement of performance. In this case, abductive learning can be regarded as a special kind of weakly supervised learning [12], where

the supervision information comes not only from the ground-truth labels, but also from knowledge reasoning. Indeed, abductive learning can be viewed as enlarging the scope of weakly supervised learning: A nice model can be generated even when there are no sufficient labeled/unlabeled data if nice knowledge available.

In fact, from the view of machine learning, the logical reasoning part in Figure 5 provides not only additional supervision information, but also a new criterion for model selection, i.e., to select the model which is the most consistent with the knowledge base. On the other hand, from the view of logical reasoning, the machine learning part in Figure 5 provides additional control of termination and backtracking for knowledge reasoning. That is, if the machine learning model updated based on the abduced-labels exhibits worse performance, then the reasoning should be terminated, and the whole process should be backtracked to the previous state, enabling to restart the reasoning process and select another reasoning result.

Note that the given knowledge base in Figure 5 can often be incomplete in practice. For example, new classes that are not in the knowledge base may emerge in streaming data, in which case the knowledge base needs to be updated to adapt to the new situation. A possible solution is to introduce the predicate invention technique such as meta-interpretive learning [13].

More seriously, the given knowledge base in Figure 5 may be inaccurate in practice. To cope with this issue, not only the pseudo-groundings can be revised to minimize the inconsistency with the knowledge base, but also the knowledge base itself can be subject to refinement. For example, if the knowledge base becomes compatible with all pseudo-groundings by revising only one clause in the knowledge base, then it can be decided to make this revision. However, this often requires human participation to determine whether to accept the specific revision to the knowledge base. On the other hand, extending the revisable object from the pseudo-groundings to the whole knowledge base will make the abductive learning process more complex, and many technical challenges need to be addressed. Even if the knowledge base is assumed to be accurate and consistent, minimizing the inconsistency remains a challenge because it involves optimization on symbolic relations.

Actually, in abductive learning it is quite crucial to introduce symbolic optimization to augment logical reasoning. Previous logical reasoning techniques are mostly based on pure reasoning with definitely precise results. By introducing the inconsistency minimization, abductive learning exploits symbolic optimization in addition to reasoning. This allows the knowledge reasoning process to be more flexible, more adaptable to the less precise real world, and allows a smoother connection with machine learning which generally pursues approximate rather than definite results. Symbolic optimization, however, could not be accomplished by commonly used numerical optimization techniques such as gradient-based approaches. Fortunately, the recently developed derivative-free optimization [14] offers a feasible solution, as shown in the implementation of the next section.

16.3. An Implementation

Abductive learning is a general paradigm which has great flexibility. Different ABL models can be generated by choosing different options to realize the machine learning, logical reasoning, as well as conversion parts in Figure 5.

Table 1. Background knowledge about structure of equations.

```

% Define a single digit
digit(0).
digit(1).
% Recursively define digits
digits([D]) ----> [D], digit(D) .
digits([D|T]) ----> [D], !, digits(T), digit(D) .
digits(X) :- phrase(digits(X), X).

% Recursively define equations.
% Since the pseudo-labels may contain missing values (variables),
% we define eq_args as non operator symbols (including variables).
eq_arg([D]) ----> [D], not(D == '+'), not(D == '=') .
eq_arg([D|T]) ---->
    [D], !, eq_arg(T), not(D == '+'), not(D == '=') .
equation(eq(X, Y, Z)) ---->
    eq_arg(X), [+], eq_arg(Y), [=], eq_arg(Z).
parse_eq(Pseudo_Labels, Eq) :-
    phrase(equation(Eq), Pseudo_Labels).

```

cursively define that a digit is a sequence of “0” and “1”, and equations share the structure of $X+Y=Z$, though the length of X , Y and Z can be varied. The knowledge about bit-wise operations is a recursive logic program, as shown in Table 2, which reversely calculates $X+Y$, i.e., it operates on X and Y digit-by-digit and from the last digit to the first. Table 3 defines the ALP used in the logical reasoning part in Figure 7.

Given the initial classifier $\mathcal{C}$, the pseudo-labels “0”, “1”, “+” and “=” can be obtained for the observed pictures, which are then used as symbolic pseudo-groundings. Given the pseudo-groundings and the knowledge base $\mathcal{KB}$, ALP can abduce ground hypotheses as possible explanations to the observed facts, suggesting some pseudo-groundings to be revised, by minimizing the inconsistency (i.e., maximizing the consistency) between the revision $\Delta(\mathcal{O})$ and $\mathcal{KB}$. Suppose $\mathcal{O}$ is a set of groundings $\mathcal{O} = \mathcal{O}_1 \cup \mathcal{O}_2$, in which $\mathcal{O}_1$ are the pseudo-groundings of equations and $\mathcal{O}_2$ are the learned groundings of operation rules such as $op([0, 0], [0])$, $op([0, 1], [1])$, etc. ABL attempts to maximize the consistency between the revised groundings $\Delta(\mathcal{O})$ and background knowledge $\mathcal{KB}$, i.e.,

$$\max_{\Delta(\mathcal{O})} \text{Con}(\Delta(\mathcal{O}); \mathcal{KB}), \quad (4)$$

where $\text{Con}(\Delta(\mathcal{O}); \mathcal{KB})$ stands for the size of subset $\hat{\mathcal{O}} \subseteq \mathcal{O}$ which is consistent with $\mathcal{KB}$ after revision. It can be defined as

Table 2. Background knowledge about bit-wise calculation.

```

% Abductive bit-wise calculation with given pseudo-labels,
% this procedure abduces missing pseudo-labels together with unknown
% operation rules.
calc(Rules, Pseudo) :-
    calc([], Rules, Pseudo).
calc(Rules0, Rules1, Pseudo) :-
    parse_eq(Pseudo, eq(X,Y,Z)),
    bitwise_calc(Rules0, Rules1, X, Y, Z).

% Bit-wise calculation that handles carrying
bitwise_calc(Rules, Rules1, X, Y, Z) :-
    reverse(X, X1), reverse(Y, Y1), reverse(Z, Z1),
    bitwise_calc_r(Rules, Rules1, X1, Y1, Z1),
    maplist(digits, [X,Y,Z]).
% Recursively calculate back-to-front
bitwise_calc_r(Rs, Rs, [], Y, Y).
bitwise_calc_r(Rs, Rs, X, [], X).
bitwise_calc_r(Rules, Rules1, [D1|X], [D2|Y], [D3|Z]) :-
    % Abduces my_op/3 during the calculation.
    abduce_op_rule(my_op([D1],[D2],Sum), Rules, Rules2),
    % Handling carry
    ((Sum = [D3], Carry = []); (Sum = [C,D3], Carry = [C])),
    bitwise_calc_r(Rules2, Rules3, X, Carry, X-carried),
    bitwise_calc_r(Rules3, Rules1, X-carried, Y, Z).

```

$$\text{Con}(\Delta(\mathcal{O}); \mathcal{KB}) = \max_{\hat{\mathcal{O}} \subseteq \mathcal{O}} |\hat{\mathcal{O}}| \quad (5)$$

s.t. $\mathcal{KB} \models \Delta(\hat{\mathcal{O}})$.

ABL tries to correct the wrongly perceived pseudo-labels to achieve consistent abductions, to enable groundings in $\mathcal{O}$, as many as possible, to be consistent with $\mathcal{KB}$. Note that maximizing the size of subset $\hat{\mathcal{O}}$ is equivalent to maximizing the size of subset $\hat{\mathcal{O}}_1 \subseteq \mathcal{O}_1$, as the size of $\mathcal{O}_2$ is a constant in this task.

Denote the part of pseudo-groundings to be revised in Eq.(4) as $\delta[\mathcal{O}] \subseteq \mathcal{O}$, where δ is a heuristic function to estimate which pseudo-labels are perceived incorrectly by current machine learning model; this is analogous to the archaeologists' ability to identify misinterpreted hieroglyphs in Figure 3. After removing the incorrect pseudo-groundings marked by the δ function, ABL can abduce the candidate pseudo-groundings to revise $\delta[\mathcal{O}]$ by considering

Table 3. The ALP used for handwritten equation decipherment.

```

% Main predicate for performing abduction
% ‘‘Examples’’ are the pseudo-labels of a set of examples,
% ‘‘Delta_C’’ is the abduced operation rules.
abduce(Examples, Delta_C) :-
    abduce(Examples, [], Delta_C).
abduce([], Delta_C, Delta_C).
abduce([E|Examples], Delta_C0, Delta_C1) :-
    calc(Delta_C0, Delta_C2, E),
    abduce(Exs, Delta_C2, Delta_C1).

% Getting an existed (already abduced) operation rule from history.
abduce_op_rule(R, Rules, Rules) :-
    member(R, Rules).
% Abduce a new rule.
abduce_op_rule(R, Rules, [R|Rules]) :-
    op_rule(R),
    % integrity constraints.
    valid_rules(Rules, R).

% Integrity Constraints on operation rule set, forbidding
% redundant rules and inconsistent rules.
valid_rules([], _).
valid_rules([my_op([X1], [Y1], _) | Rs], my_op([X], [Y], Z)) :-
    not([X, Y] = [X1, Y1]),
    not([X, Y] = [Y1, X1]),
    valid_rules(Rs, my_op([X], [Y], Z)).
valid_rules([my_op([Y], [X], Z) | Rs], my_op([X], [Y], Z)) :-
    not(X = Y),
    valid_rules(Rs, my_op([X], [Y], Z)).

% Abducing single operation rule.
op_rule(my_op([X], [Y], [Z])) :- digit(X), digit(Y), digit(Z).
op_rule(my_op([X], [Y], [Z1, Z2])) :- digit(X), digit(Y), digits([Z1, Z2]).

```

$$\mathcal{KB} \models (\mathcal{O} - \delta[\mathcal{O}]) \cup \Delta(\delta[\mathcal{O}]), \quad (6)$$

where $\mathcal{O} - \delta[\mathcal{O}]$ are the remaining ‘‘correct’’ pseudo-groundings determined by δ , $\Delta(\delta[\mathcal{O}])$ are the abduced pseudo-groundings by ALP, and $(\mathcal{O} - \delta[\mathcal{O}]) \cup \Delta(\delta[\mathcal{O}])$ corresponds to $\Delta(\mathcal{O})$ in Eq.(3).

To restrict the exponentially large search space for the abduction and keep the link between logical reasoning and actual data, in this implementation, ABL chooses to restrict the revision $\Delta(\mathcal{O})$ to be not too far away from the perceived facts $\mathcal{O}$, by limiting $|\delta[\mathcal{O}]| \leq M$, where M defines the step-wise search space on the scale of the abduction and is sufficient to be set a small number. Therefore, the optimization problem of $\Delta(\mathcal{O})$ can be transformed into an optimization problem of function δ , and Eq.(4) can be reformulated as

$$\begin{aligned} \max_{\delta} \quad & \text{Con}(\Delta_{\delta}(\mathcal{O}); \mathcal{KB}), \\ \text{s.t.} \quad & |\delta[\mathcal{O}]| \leq M, \end{aligned} \quad (7)$$

where $\Delta_{\delta}(\mathcal{O}) = (\mathcal{O} - \delta[\mathcal{O}]) \cup \Delta(\delta[\mathcal{O}])$ are the abduced groundings defined by Eq.(6). Though this objective is still non-convex, optimizing δ instead of $\Delta(\mathcal{O})$ allows ABL to revise and improve the hypothesis even when the machine learning model is sub-optimal. Here, the heuristic function δ is solved by derivative-free optimization [14], which is a flexible framework for optimizing non-convex objectives, whereas the subset selection problem in Eq.(7) is solved by greedy. Note that the derivative-free optimization does not rely on gradient calculation, and therefore, it can be applied to symbolic optimization. Finally, the obtained $\Delta_{\delta}(\mathcal{O})$ are to be converted into pseudo-labels for updating the machine learning model.

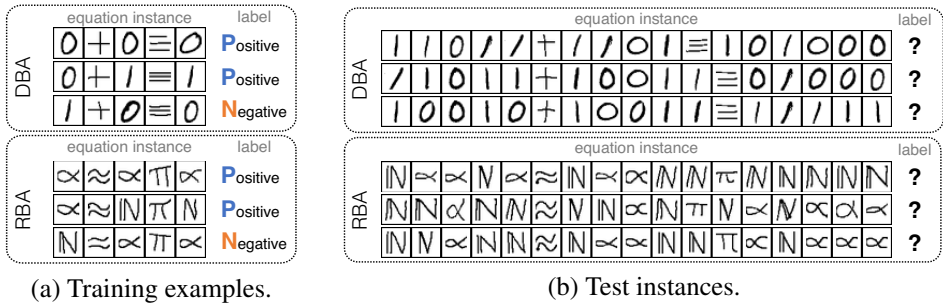


Figure 8. Data examples for the handwritten equations decipherment tasks.

For empirical study, two data sets are generated [15], as shown in Figure 8. The DBA (Digital Binary Additive) equations are generated from handwritten images [16, 19], whereas the more difficult RBA (Random Symbol Binary Additive) equations are constructed from the Omniglot dataset [20]. Two different settings of ABL have been studied: the ABL-all that uses all training data and the ABL-short that uses training equations of only lengths 5-8. Compared approaches include DNC (Differentiable Neural Computer) [21], Transformer networks [22] and BiLSTM (Bidirectional Long Short-Term Memory Network) [23]. Human volunteers have also been requested to do the same task for comparison.

Figure 9 shows that on both tasks, the ABL-based approaches significantly outperform the compared approaches. All approaches perform better on the DBA tasks than RBA, as the DBA tasks are much easier than RBA tasks. The performance of ABL-all and ABL-short do not have significant differences. The performance of all approaches become worse as the size of the testing equations grows, while the ABL-based ap-

proaches are apparently the best. It is interesting to observe that the ABL performance is even better than human performance on this task [15].

A further examination discloses that the ABL approaches often suffered from incorrectly perceived symbols. As the failure example in the RBA task shown in Figure 10, the ground-truth symbols in the equation are “110010+11100110=100011000” but the perceived results are “110010+11100110110001=000”. However, almost all the operation rules learned by ABL were correct. In other words, the failures of ABL approaches were mostly caused by the incorrectly perceived pseudo-labels. This is quite interesting because, in contrast, a further examination discloses that the failures of human volunteers were mostly caused by reasoning errors, i.e., the difficulties in finding consistent operation rules.

16.4. An Application

The ABL has been found useful in application to *theft judicial sentencing* [24], a law-enforcement task.

For the sake of justice and fairness, the high courts in China often examine the quality of sentences made by local courts. Manual examination requires tremendous human resources and is too time-consuming, and therefore, intelligent assistants are demanded.

This examination process relies on the judicial documents written by the judges of local courts. The most important parts of the judicial documents for this task are the amount of money involved, the sentencing elements, and the final sentence. The amount of money (e.g., 5,000 RMB), and the final sentence (e.g., 12 months imprisonment) are numerical values explicitly written in the documents. The sentencing elements, such as burglary, recidivism, etc., which are important factors affecting the sentencing decisions, however, are generally implicitly expressed in various case descriptions. Thus, a machine learning model trained from documents with labeled sentencing elements is employed. It is difficult to collect a large number of documents with labeled sentencing elements, and thus, this machine learning model is not very accurate.

In addition to data of previous judgment cases, in this task there is a knowledge base that mainly consists of first-order logical rules expressing law rules, matching rules and common sense rules. Taking sentencing elements as input, human judges are able to make the final sentence based on these rules. However, the rules are not easy to be applied directly by machines; this is because that there is space for discretion of judges. Roughly speaking, the law rules could not be exhaustive for all possible combinations of

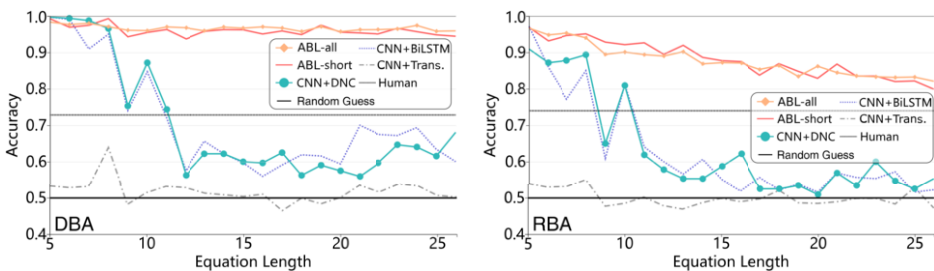


Figure 9. Experimental results on the DBA (left) and RBA (right) tasks.



Figure 10. An incorrectly perceived testing equation by ABL.

Domain knowledge for calculating the punishment in sentencing:

$$penalty(X,Z) \leftarrow base_penalty(X,Y_1) \wedge weight(X,Y_2) \wedge Z = Y_1(1+Y_2).$$
$$base_penalty(X,Z) \leftarrow money(X,m) \wedge Z = 0.7m + 5.7.$$
$$weight([],0) \leftarrow$$
$$weight([X|Xs],Z) \leftarrow element_weight(X,Y_1) \wedge weight(Xs,Y_2) \wedge Z = Y_1 + Y_2.$$

Punishment weights for different sentencing elements:

$$element_weight(recidivism,29\%). \quad element_weight(confession,-11\%).$$
$$element_weight(pickpocket,14\%). \quad element_weight(burglary,3\%).$$

Figure 11. Some example rules in the knowledge base.

sentencing elements, and thus, judges need to make their decision for the degree of severity of the punishment based on their own expertise and experience. If this is expressed as a kind of parameter, then the parameter value is unknown and needs to be learned from previous judgment cases. Figure 11 shows some example rules in the knowledge base.

Overall, this task aims to training a machine learning model that can recognize sentencing elements from textual criminal judgment documents, and learning the parameters for utilizing the law rules in knowledge base to make the final sentence, as illustrated in Figure 12.

The theft judicial sentencing task is somewhat similar to the Mayan calendar decipherment task in Figure 2. In Mayan calendar decipher task, the input data are the Mayan calendar pictures, where the numbers represented by hieroglyphs are unknown; in the theft judicial sentencing task, the input data are textual criminal judgment documents, where the labels of sentencing elements are unknown. In Mayan calendar decipher task, there is symbolic knowledge of the calendars, and the perceived numbers should be con-

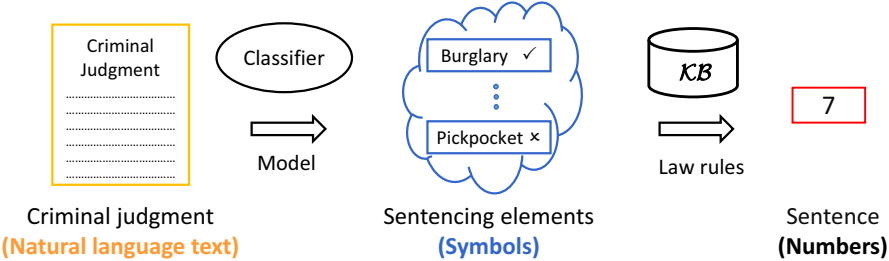


Figure 12. The theft judicial sentencing task. The classifier recognizes sentencing elements first, and then the knowledge base is utilized to make the final sentence.

Table 4. Comparison results on the theft judicial sentencing task, where F1 measures the sentencing elements prediction performance, whereas MAE and MSE measure the final sentencing performance.

Method	F1	MAE	MSE
BERT-10	0.811±0.010	0.867±0.032	1.204±0.123
PL-10	0.814±0.006	0.862±0.035	1.155±0.107
Tri-10	0.812±0.016	0.840±0.066	1.155±0.107
ABL-10	0.862±0.005	0.824±0.017	1.146±0.049
BERT-50	0.857±0.006	0.830±0.020	1.065±0.044
PL-50	0.858±0.010	0.832±0.035	1.045±0.154
Tri-50	0.861±0.007	0.810±0.021	0.994±0.046
ABL-50	0.865±0.007	0.788±0.027	0.959±0.091
BERT-100	0.863±0.003	0.821±0.014	1.011±0.031

sistent with the knowledge base; in the theft judicial sentencing task, there is symbolic knowledge of law rules, and the predicted sentencing elements should be consistent with the knowledge base. Moreover, in the theft judicial sentencing task, the machine learning model for recognizing sentencing elements is not very accurate due to the limit of labeled data, whereas the knowledge reasoning is also complex due to the unknown law rule parameters. Thus, it is desired to have the machine learning and knowledge reasoning to help each other, and therefore, abductive learning is naturally applicable.

Table 4 shows some results on a real application [24]. Here, multi-label BERT (Bidirectional Encoder Representations from Transformers) [25] was used as the classifier in ABL, with two types of baselines, i.e., BERT using only labeled data and SSL (Semi-Supervised Learning) approaches leveraging unlabeled data, including Pseudo-Label (PL) [26] and Tri-training (Tri) [27]. The F1 column shows the F1-score (micro-averaging) of the compared approaches on prediction of the sentencing elements. The MAE and MSE columns, respectively, show the Mean Absolute Error and Mean Square Error of the final sentencing performance. “10”, “50” and “100” mean that the approach uses 10%, 50% or 100% labeled data, respectively.

It can be seen that the performance of ABL on sentencing elements prediction is superior to other approaches. The performance of ABL-10 is even superior to BERT-50 which uses much more labeled data. The gain from utilizing knowledge and unlabeled data is larger when there are less labeled data. BERT-100 has a similar F1-score as ABL-50, but worse MAE and MSE for the final sentencing; this may be explained by the fact that the interlacing of machine learning and knowledge reasoning in ABL enables it to be more robust even when the ground-truth labels of the sentencing elements are not definitely accurate.

Table 5 shows some learned sentencing rules learned by ABL [24]. These highly interpretable rules can be examined by human judges, providing explanatory assistance to judicial sentencing.

Table 5. Some rules learned by ABL.

Base sentence=0.64m+6 (m: money involved in thousands)	
<i>burglary</i> → base +6%	<i>confession</i> → base -14%
<i>forgiveness</i> → base -15%	<i>juvenile</i> → base -15%
<i>no loss</i> → base -7%	<i>pickpocket</i> → base +15%
<i>surrender</i> → base -10%	<i>recidivism</i> → base +27%

16.5. Conclusion

This chapter introduces abductive learning, a new paradigm which integrates machine learning and logical reasoning in a *balanced* and mutually beneficial way. It is flexible and general enough to produce models with different capabilities through different implementations. From a psychological viewpoint, the perception and reasoning of human beings are entangled rather than separated; this is reassembled to some sense in abductive learning. Indeed, human perception is not always accurate, and some misperceptions can be corrected when the perceived patterns are involved in reasoning, wherein the reasoning process can provide a conjecture about correct patterns. Such conjectures are simulated by the abduced-labels in Figure 5. Human knowledge can be updated/refined when new facts are observed; this can be simulated via (potential) knowledge refinement in abductive learning. Actually, if the machine learning part in Figure 5 is regarded as a data-driven perception mechanism whereas the logical reasoning part is regarded as a knowledge-driven cognition mechanism, abductive learning exhibits how the perception can be guided by cognition, and how the cognition can be augmented/refined based on perception.

Acknowledgements

This research was supported by the National Science Foundation of China (61921006, 61751306) and the National Key R&D Program of China (2018YFB1004300).

References

- [1] Raedt LD, Frasconi P, Kersting K, et al., editors. Probabilistic inductive logic programming: Theory and applications. Berlin: Springer; 2008.
- [2] Getoor L, Taskar B, editors. Introduction to statistical relational learning. Cambridge, MA: The MIT Press; 2007.
- [3] Zhou ZH. Abductive learning: Towards bridging machine learning and logical reasoning. *Science China Information Sciences*. 2019;62(7):76101.
- [4] Houston SD, Mazariegos OC, Stuart D, editors. The decipherment of ancient maya writing. Norman, Oklahoma: University of Oklahoma Press; 2001.
- [5] Bowditch CP. The numeration, calendar systems and astronomical knowledge of the mayas. Cambridge, UK: Cambridge University Press; 1910.
- [6] Magnani L. Abductive cognition: The epistemological and eco-cognitive dimensions of hypothetical reasoning. Berlin: Springer; 2009.
- [7] Peirce SC. Abduction and induction. In: Buchler J, editor. Philosophical writings of peirce. New York, NY: Dover Publications; 1955. p. 150–156.
- [8] Mooney RJ. Integrating abduction and induction in machine learning. In: Flach P, Kakas AC, editors. Abduction and induction. Amsterdam, The Netherlands: Kluwer; 2000. p. 181–191.
- [9] Muggleton SH, Bryant CH. Theory completion using inverse entailment. In: Proceedings of the 10th International Conference on Inductive Logic Programming; London, UK; 2000. p. 130–146.
- [10] Dai WZ, Zhou ZH. Combining logic abduction and statistical induction: Discovering written primitives with human knowledge. In: Proceedings of the 31st AAAI Conference on Artificial Intelligence; San Francisco, CA; 2017. p. 2977–2983.
- [11] Zhou ZH. Learnware: On the future of machine learning. *Frontiers of Computer Science*. 2016; 10(4):589–590.
- [12] Zhou ZH. A brief introduction to weakly supervised learning. *National Science Review*. 2018;5(1):44–53.

- [13] Muggleton SH, Lin D, Tamaddoni-Nezhad A. Meta-interpretive learning of higher-order dyadic datalog: Predicate invention revisited. *Machine Learning*. 2015;100(1):49–73.
- [14] Yu Y, Qian H, Hu YQ. Derivative-free optimization via classification. In: *Proceedings of the 30th AAAI Conference on Artificial Intelligence*; Phoenix, AZ; 2016. p. 2286–2292.
- [15] Dai WZ, Xu Q, Yu Y, et al. Bridging machine learning and logical reasoning by abductive learning. In: *Advances in neural information processing systems 32*. Vancouver, Canada; 2019. p. 2811–2822.
- [16] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*. 1998;86(11):2278–2324.
- [17] Kakas AC, Kowalski RA, Toni F. Abductive logic programming. *Journal of Logic Computation*. 1992; 2(6):719–770.
- [18] d’Avila Garcez AS, Broda KB, Gabbay DM. *Neural-symbolic learning systems: Foundations and applications*. London, UK: Springer; 2009.
- [19] Thoma M. The HASYv2 dataset. *arXiv:170108380*. 2017;.
- [20] Lake BM, Salakhutdinov R, Tenenbaum JB. Human-level concept learning through probabilistic program induction. *Science*. 2015;350(6266):1332–1338.
- [21] Graves A, Wayne G, Reynolds M, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*. 2016;538(7626):471–476.
- [22] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: *Advances in neural information processing systems 30*. Long Beach, CA; 2017. p. 5998–6008.
- [23] Schuster M, Paliwal KK. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*. 1997;45(11):2673–2681.
- [24] Huang YX, Dai WZ, Yang J, et al. Semi-supervised abductive learning and its application to theft judicial sentencing. In: *Proceedings of the 20th IEEE International Conference on Data Mining*; Sorrento, Italy; 2020. p. 1070–1075.
- [25] Devlin J, Chang MW, Lee K, et al. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv:181004805*. 2018;.
- [26] Lee DH. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In: *ICML Workshop on Challenges in Representation Learning*; 2013.
- [27] Zhou ZH, Li M. Tri-training: Exploiting unlabeled data using three classifiers. *IEEE Transactions on Knowledge and Data Engineering*. 2005;17(11):1529–1541.

Chapter 17

Logic Tensor Networks: Theory and Applications

Luciano Serafini, FBK, Trento, Italy

Artur d'Avila Garcez, City, University of London, UK

Samy Badreddine, Sony AI, Japan

Ivan Donadello, Free University of Bozen-Bolzano, Italy

Michael Spranger, Sony AI, Sony CSL, Japan

Federico Bianchi, Bocconi Univesity, Milan, Italy

17.1. Introduction

The recent availability of large-scale data combining multiple data modalities such as image, text, audio and sensor data, has opened various research and commercial opportunities in Artificial Intelligence (AI). Machine Learning (ML) has achieved important results in this area mostly by adopting a sub-symbolic distributed representation of entities, referred to as *deep learning*, that is efficient and foregoes the use of an explicit model of the world. It is generally accepted now that such purely sub-symbolic approaches can be data inefficient and struggle at extrapolation and reasoning. By contrast, symbolic AI is based on rich, high-level representations based ideally on human-readable symbols. By rich representation, we refer to well-established logical systems which are more expressive than propositional logic and can express full first-order logic including universal and existential quantification ($\forall x$ and $\exists y$), arbitrary n -ary relations over variables, e.g. $R(x, y, z, \dots)$, and function symbols, e.g. $\text{fatherOf}(x)$, $x + y$, etc. Symbolic AI systems, despite being much more explainable than deep learning and having had success at theorem proving and reasoning, usually struggles when faced with incomplete knowledge or inaccurate, large-scale data, not being efficient in the presence of combinatorial knowledge or large multimodal data sets such as image, text and sensor data.

Neurosymbolic AI attempts to combine the strength of both approaches (see [1, 2, 3, 4] for recent surveys), that is, to combine reasoning with complex representation of knowledge (knowledge-bases, semantic networks, ontologies, trees and graphs) with efficient learning from data (images, time series, sensorimotor data, natural language,

etc). Consequently, neurosymbolic AI seeks to ground rich knowledge into efficient sub-symbolic representations, and to explain sub-symbolic representations typical of deep learning by offering high-level symbolic descriptions for such learning systems.

Logic Tensor Networks (LTN) are a neurosymbolic AI system for querying, learning and reasoning with rich data and abstract knowledge. LTN introduces Real Logic, a fully differentiable first-order language with concrete semantics such that every symbolic expression has an interpretation that is grounded onto real numbers in the domain. In particular, LTN converts Real Logic formulas into computational graphs that enable gradient-based optimization.

LTN and Real Logic were first introduced in [5], which illustrates the framework on knowledge completion tasks using Neural Tensor Networks [6] to ground the relational predicates (symbols) into a concrete interpretation (vectors and tensors). In [7], LTN was further formalized and applied to many examples of AI tasks, including semi-supervised learning, learning of embeddings and reasoning tasks; [7] also extends the LTN framework to include guarded quantification and better guidelines for gradient-based optimization.

LTN has been applied recently to many important AI tasks, including semantic image interpretation, ontology learning and reasoning, and reinforcement learning, which illustrates its use for supervised classification, data clustering, semi-supervised learning, embedding learning, reasoning or query answering [7, 8, 9, 10, 11]. In this chapter, we present an accessible overview of LTN (Section 17.2 and Section 17.3) and bring together some of its main recent applications to relevant AI problems (Section 17.4). We analyze results in the context of related work, and discuss the next steps for neurosymbolic AI and LTN-based AI models.

17.2. Real Logic

17.2.1. Syntax

Real Logic forms the basis of Logic Tensor Networks. Real Logic is defined on a first-order language $\mathcal{L}$ with a signature that contains a set $\mathcal{C}$ of constant symbols (objects), a set $\mathcal{F}$ of functional symbols, a set $\mathcal{P}$ of relational symbols (predicates), and a set $\mathcal{X}$ of variable symbols. $\mathcal{L}$ -formulas allow us to specify relational knowledge with variables, e.g. the atomic formula `is_friend(Alice, Bob)` may state that Alice is a friend of Bob, the formula $\forall x \forall y (\text{is_friend}(x, y) \rightarrow \text{is_friend}(y, x))$ states that the relation `is_friend` is symmetric, and the formula $\forall x (\exists y (\text{Italian}(x) \wedge \text{is_friend}(x, y)))$ states that every person who is Italian has a friend. Since we are interested in learning and reasoning in real-world scenarios where degrees of truth are often fuzzy and exceptions exist, formulas can be partially true, and therefore we adopt fuzzy logic semantics.

Objects can be of different types. Similarly, functions and predicates are typed. Therefore, we assume there exists a non-empty set of symbols $\mathcal{D}$ called *domain symbols*. To assign types to the elements of $\mathcal{L}$ we introduce the functions $\mathbf{D}$, $\mathbf{D}_{\text{in}}$ and $\mathbf{D}_{\text{out}}$ such that:

- $\mathbf{D}(x)$ and $\mathbf{D}(c)$, where $x \in \mathcal{X}$ and $c \in \mathcal{C}$, return the domain of a variable x or constant symbol c ,

- $\mathbf{D}_{\text{in}}(f)$ and $\mathbf{D}_{\text{in}}(p)$, where $f \in \mathcal{F}$ and $p \in \mathcal{P}$, return the domain of the arguments of a function or predicate symbol, respectively,
- $\mathbf{D}_{\text{out}}(f)$, where $f \in \mathcal{F}$, returns the range of a function symbol.

Real Logic may also contain propositional variables, as follows: if P is a 0-ary predicate with $\mathbf{D}_{\text{in}}(P) = \langle \rangle$ (the empty sequence of domains) then P is a propositional variable (an atom with truth-value in the interval $[0, 1]$).

Terms and formulas are constructed recursively in the usual way, given that functional and predicate symbols are applied to an appropriate number of terms with appropriate domains. A term is constructed recursively from constant symbols, variables and function symbols. An expression formed by applying a predicate symbol to an appropriate number of terms (with appropriate domains) is called an atomic formula. A formula evaluates to *true* or *false* in classical logic and to a number in $[0, 1]$ in the case of Real Logic. Complex formulas are constructed recursively using the connectives $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$ (negation, conjunction, disjunction, implication and bi-conditional, respectively) and the quantifiers $\{\forall, \exists\}$ (universal and existential, respectively).

Example 17.2.1. Let *Town* denote the domain of towns in the world and *People* denote the domain of living people. Suppose that $\mathcal{L}$ contains the constant symbols Alice, Bob and Charlie of domain *People*, and Rome and Seoul of domain *Town*. Let x be a variable of domain *People* and u be a variable of domain *Town*. The term x, u (i.e. the sequence x followed by u) has domain *People, Town* which denotes the Cartesian product between *People* and *Town*. Alice, Rome is interpreted as an element of the domain *People, Town*. Let *lives_in* be a predicate with input domain $\mathbf{D}_{\text{in}}(\text{lives_in}) = \text{People, Town}$. Then, *lives_in*(Alice, Rome) is a well-formed expression, whereas *lives_in*(Seoul, Bob) is not.

17.2.2. Semantics of Real Logic

17.2.2.1. Grounding the signature

The semantics of Real Logic departs from the standard abstract semantics of First-order Logic (FOL). In Real Logic, every individual denoted by constants, variables, and terms, is interpreted as a tensor of real values.¹ Domains are interpreted in coordinate spaces. Functions are interpreted as real functions or tensor operations. Predicates are interpreted as functions or tensor operations specifically projecting onto a value in the interval $[0, 1]$.

To emphasize that in Real Logic symbols are grounded onto real-valued features, we use the term *grounding*, denoted by $\mathcal{G}$, in the place of *interpretation*². $\mathcal{G}(t)$ associates real features to any term t of $\mathcal{L}$, and $\mathcal{G}(\phi)$ associates a value, a number in the interval $[0, 1]$, to any formula ϕ , which represents the system's degree of confidence in the truth of ϕ ; the higher the value, the higher the confidence.

A variable x is grounded as an explicit *sequence* of n_x individuals from a domain, with $0 < n_x < \infty$. This differs from a FOL variable, which is a placeholder for any individual from the domain. Consequently, a term $t(x)$ or a formula $P(x)$ constructed recursively with a free variable x , will be grounded to a sequence of n_x values too.

¹In the rest of the paper, we commonly use "tensor" to designate "tensor in the real field".

²Notice that this is different from the common use of the term *grounding* in logic, which indicates the operation of replacing the variables of a term or formula with constants or terms containing no variables. To avoid confusion, we use the synonym *instantiation* for this purpose.

Example 17.2.2. Suppose that $\mathcal{L}$ contains the variables x and y , the function f , the predicate p and the set of domains $\mathcal{D} = \{V, W\}$. Let $\mathbf{D}(x) = V$, $\mathbf{D}(y) = W$, $\mathbf{D}_{\text{in}}(f) = VW$, $\mathbf{D}_{\text{out}}(f) = W$ and $\mathbf{D}(p) = VW$. In what follows, an example of the grounding of $\mathcal{L}$ and $\mathcal{D}$ is shown on the left, and the grounding of some examples of possible terms and atomic formulas is shown on the right.

$$\begin{aligned}
\mathcal{G}(V) &= \mathbb{R}^+ & \mathcal{G}(f(x, y)) &= \begin{pmatrix} v_1 \cdot w_1 & v_1 \cdot w_2 \\ v_2 \cdot w_1 & v_2 \cdot w_2 \\ v_3 \cdot w_1 & v_3 \cdot w_2 \end{pmatrix} \\
\mathcal{G}(W) &= \mathbb{R}^- \\
\mathcal{G}(x) &= \langle v_1, v_2, v_3 \rangle \\
\mathcal{G}(y) &= \langle w_1, w_2 \rangle \\
\mathcal{G}(p) : x, y &\mapsto \sigma(x + y) & \mathcal{G}(p(x, f(x, y))) &= \begin{pmatrix} \sigma(v_1 + v_1 \cdot w_1) & \sigma(v_1 + v_1 \cdot w_2) \\ \sigma(v_2 + v_2 \cdot w_1) & \sigma(v_2 + v_2 \cdot w_2) \\ \sigma(v_3 + v_3 \cdot w_1) & \sigma(v_3 + v_3 \cdot w_2) \end{pmatrix} \\
\mathcal{G}(f) : x, y &\mapsto x \cdot y
\end{aligned}$$

It is worth noting the dimensions of the results of grounding: $\mathcal{G}(f(x, y))$ and $\mathcal{G}(p(x, f(x, y)))$ return $|\mathcal{G}(x)| \times |\mathcal{G}(y)| = 3 \times 2$ values, one for each combination of individuals that occur in the variables. For functions, one can have additional “feature” dimensions. Let us assume a different grounding such that $\mathcal{G}(\mathbf{D}_{\text{out}}(f)) = \mathbb{R}^m$. Then, the dimensions of $\mathcal{G}(f(x, y))$ would be $|\mathcal{G}(x)| \times |\mathcal{G}(y)| \times m$, where m corresponds to such feature dimensions. However, $\mathcal{G}(p(x, f(x, y)))$ will always return a tensor with the exact dimensions $|\mathcal{G}(x)| \times |\mathcal{G}(y)| \times 1$ because, under any grounding, a predicate always returns a value in $[0, 1]$.

17.2.2.2. Connectives and Quantifiers

The semantics of the connectives is defined according to the semantics of first-order fuzzy logic [12]. Definitions of some common fuzzy operators are presented in [7]. By comparing the gradients of different operators, [13] shows that the *product configuration* is the best symmetric configuration for gradient-based optimization (see Section 17.3). In the product configuration, conjunctions ($\wedge$) are grounded with the product t-norm T_{prod} , disjunctions ($\vee$) with the product t-conorm S_{prod} , implications ($\rightarrow$) with the Reichenbach implication I_R , and negations ($\neg$) with the standard fuzzy negation N_S , as follows.

$$T_{\text{prod}}(u, v) = uv \quad (1)$$

$$S_{\text{prod}}(u, v) = u + v - uv \quad (2)$$

$$I_R(u, v) = 1 - u + uv \quad (3)$$

$$N_S(u) = 1 - u \quad (4)$$

The semantics of the quantifiers ($\forall, \exists$) is defined with the use of aggregators. When optimizing with LTN, following the insights from [13], it is recommended to ground existential quantifiers ($\exists$) with the generalized mean M_p , with $p \geq 1$, and

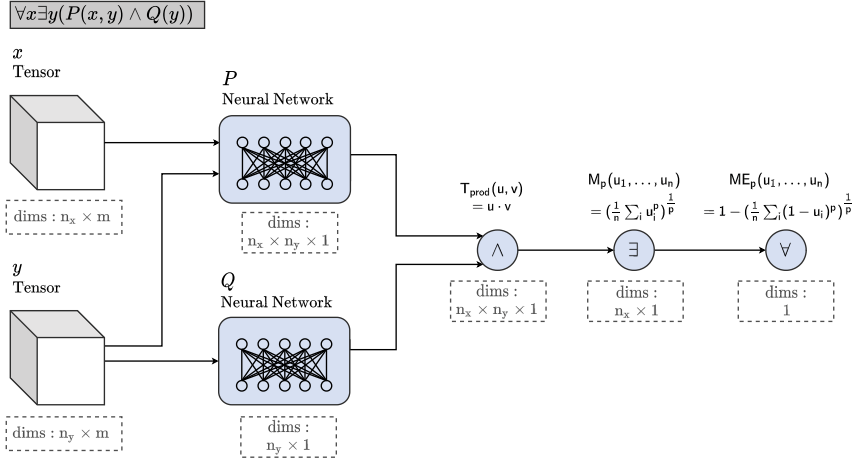


Figure 1. A Real Logic sentence compiled to a computational graph under some grounding $\mathcal{G}$. In this setting, $\mathcal{G}(x), \mathcal{G}(y) \in \mathbb{R}^m$, and $\mathcal{G}(P), \mathcal{G}(Q)$ are grounded with neural networks. Every node of the computational graph is annotated with the dimension of the intermediate result. Notice how $\wedge$ is applied element-wise and how $\exists y, \forall x$ aggregate the dimension that corresponds to the quantified variable.

universal quantifiers ($\forall$) with the generalized mean w.r.t the error values ME_p , with $p \geq 1$, s. t. $\lim_{p \rightarrow \infty} M_p(u_1, \dots, u_n) = \max\{u_1, \dots, u_n\}$ and $\lim_{p \rightarrow \infty} ME_p(u_1, \dots, u_n) = \min\{u_1, \dots, u_n\}$.

$$M_p(u_1, \dots, u_n) = \left(\frac{1}{n} \sum_{i=1}^n u_i^p \right)^{\frac{1}{p}} \quad (5)$$

$$ME_p(u_1, \dots, u_n) = 1 - \left(\frac{1}{n} \sum_{i=1}^n (1 - u_i)^p \right)^{\frac{1}{p}} \quad (6)$$

Despite being more suitable than most alternatives, this configuration has unstable gradients on edge cases of the domains. $T_{\text{prod}}(u, v)$ has vanishing gradients on the edge case $u = v = 0$; $S_{\text{prod}}(u, v)$ has vanishing gradients on the edge case $u = v = 1$; $I_R(u, v)$ has vanishing gradients on the edge case $u = 0, v = 1$; $M_p(u_1, \dots, u_n)$ has exploding gradients when $\sum_i (u_i)^p$ tends to 0; $ME_p(u_1, \dots, u_n)$ has exploding gradients when $\sum_i (1 - u_i)^p$ tends to 0. In [7], minor corrections are introduced to achieve numerical stability of these operators. To avoid that some value u is equal to 0, we use the transformation $u' = (1 - \varepsilon)u + \varepsilon$, where ε is an arbitrarily small positive value. For example, this transformation is used on both inputs for T_{prod} . To avoid that some value u is equal to 1, we use the transformation $u' = (1 - \varepsilon)u$. For example, this transformation is used on both inputs for S_{prod} .

The connective operators are applied element-wise to the tensors. The aggregators aggregate the dimension that corresponds to the quantified variable. This is illustrated in Figure 1.

17.2.2.3. Special forms of quantification

LTN also allows two special forms of quantification introduced in [7]. The first is called *diagonal quantification* (Diag). $\text{Diag}(x_1, \dots, x_h)$ quantifies over specific tuples such that the i -th tuple contains the i -th instance of each of the variables in the argument of Diag, under the assumption that all variables in the argument are grounded onto sequences with the same number of instances. Diagonal quantification intuitively performs a Python's `zip` over two or more variables before applying the aggregator.³ Notice that this is possible because variables are grounded as sequences and are ordered (unlike sets, for instance).

Example 17.2.3. Let x be a variable of People and u a variable of Town. Let $\mathcal{G}(x) = \langle \text{Alice}, \text{Bob}, \text{Charlie} \rangle$ and $\mathcal{G}(y) = \langle \text{Seoul}, \text{Rome}, \text{London} \rangle$. The closed formula $\forall \text{Diag}(x, u)(\text{lives_in}(x, u))$ aggregates the three pairs (Alice, Seoul), (Bob, Rome) and (Charlie, London) taken in order from $\mathcal{G}(x)$ and $\mathcal{G}(y)$. Without Diag, the expression would be evaluated for $3 \times 3 = 9$ combinations.

The second form of special quantification is called *guarded quantification*. In many situations, one may wish to quantify over a set of elements of a domain whose groundings satisfy some condition. In particular, one may wish to express such condition using formulas of the language of the form:

$$\forall y (\exists x : \text{age}(x) > \text{age}(y) (\text{parent}(x, y))) \quad (7)$$

where $\text{age}(x) > \text{age}(y)$ represents the condition, which we call a *mask*. Its grounding returns a crisp, boolean value. The mask restricts the quantification over the individuals with groundings that satisfy the boolean condition.

17.3. Learning, Reasoning, and Querying in Real Logic

In Real Logic, one can define the tasks of *learning*, *reasoning* and *query answering*. Given a Real Logic theory that represents the knowledge of an agent at a given time, *learning* is the task of making generalizations from specific observations obtained from data. This is often called inductive inference. *Reasoning* is the task of deriving what knowledge follows from the facts which are currently known. *Query answering* is the task of evaluating the truth value of a certain logical expression (called a query), or finding the set of objects in the data that evaluate a certain expression to *true*. In what follows, we define and exemplify each of these tasks. To do so, we first need to specify which types of knowledge can be represented in Real Logic.

17.3.1. Representing Knowledge with Real Logic

In traditional logic-based knowledge representation systems, knowledge is represented by logical formulas that represent propositions about a domain of interest. In Real Logic, knowledge is represented not only by logical formulas but also by the grounding $\mathcal{G}$ which explicitly connects symbols occurring in formulas and what concretely holds in the domain. The following types of knowledge can be represented in Real Logic.

³<https://docs.python.org/3/library/functions.html#zip>

Knowledge through symbol groundings: One can represent knowledge in $\mathcal{G}$ in the form of domain boundaries, explicit definitions, or parametric definitions obtained from data. For example, the grounding of Alice in real features that describe her height, age, etc., is knowledge. This grounding definition can be parametric if $\mathcal{G}(\text{Alice})$ is a set of embedded features to learn. In that case, we write $\mathcal{G}(\text{Alice} \mid \boldsymbol{\theta})$, where $\boldsymbol{\theta}$ is the set of trainable parameters. Consequently, learning knowledge in Real Logic is not limited to inferring new formulas, but also concerns learning the parameters of a grounding.

Knowledge through formulas: Knowledge about the properties of some or all of the objects of some domains can be specified using factual or generalized propositions. For example, suppose that it is known that img_1 is a number two and img_2 is a number nine ($\mathcal{G}(\text{img}_1) = \mathbf{2}$, $\mathcal{G}(\text{img}_2) = \mathbf{9}$). This can be represented by adding the following facts to the knowledge-base: $\text{is_two}(\text{img}_1)$, $\text{is_nine}(\text{img}_2)$. Additionally, general knowledge is specified using first-order logic formulas with quantified variables. For example, one can express the following constraints for a classification task: $\forall x (\text{is_nine}(x) \rightarrow \neg \text{is_two}(x))$, $\forall x (\text{is_nine}(x) \rightarrow \text{is_number}(x))$, etc.

Knowledge through operators: The grounding of a formula ϕ depends on the operators approximating the connectives and quantifiers that appear in ϕ . Different operators give different interpretations of the satisfaction associated with the formula. For instance, the operator $\text{ME}_p(a_1, \dots, a_n)$ that approximates universal quantification can be understood as a smooth minimum if $p \geq 1$. If $p = 1$ then $\text{ME}_p(a_1, \dots, a_n)$ corresponds to the arithmetic mean. As p increases, given the same input, the value of the universally quantified formula will decrease as ME_p converges to the min operator. To define how strictly the universal quantification should be interpreted in each proposition, one can use different values of p for different propositions of the knowledge base. For instance, a formula $\forall x P(x)$ where ME_p is used with a low value for p will in fact denote that P holds for *some* x , whereas a formula $\forall x Q(x)$ with a higher p may denote that Q holds for *most* x .

The definition that follows formalizes the different components of a Real Logic knowledge-base.

Definition 17.3.1 (Theory/Knowledge-base). A *theory* of Real Logic is a triple $\mathcal{T} = \langle \mathcal{K}, \mathcal{G}(\cdot \mid \boldsymbol{\theta}), \Theta \rangle$, where $\mathcal{K}$ is a set of closed first-order logic formulas defined on $\mathcal{L}$, $\mathcal{G}(\cdot \mid \boldsymbol{\theta})$ is a parametric grounding for all the symbols $\mathcal{L}$ and all the logical operators, and $\Theta = \{\Theta_s\}_{s \in \mathcal{S}}$ is the hypothesis space for each set of parameters $\boldsymbol{\theta}_s$ associated with symbol s .

17.3.2. Querying

Given a grounded theory $\mathcal{T} = (\mathcal{K}, \mathcal{G}_{\boldsymbol{\theta}})$, *query answering* allows one to check if a certain fact is true (or, more precisely, by how much it is true since in Real Logic truth-values are real numbers in the interval $[0, 1]$). There are various types of queries that can be asked of a grounded theory.

The first type of query is called *truth queries*. A truth query evaluates the truth value of a formula ϕ_q by computing its grounding, i.e. $\mathcal{G}_{\boldsymbol{\theta}}(\phi)$. Notice that, if ϕ_q is a closed formula, the answer is a scalar in $[0, 1]$ denoting the truth-value of ϕ_q according to $\mathcal{G}_{\boldsymbol{\theta}}$. If ϕ_q contains n free variables $x_1, \dots, x_n$, the answer to the query is a tensor of

order n such that the component indexed by $i_1 \dots i_n$ is the truth-value of ϕ_q evaluated in $\mathcal{G}_{\theta}(x_1)_{i_1}, \dots, \mathcal{G}_{\theta}(x_n)_{i_n}$.

The second type of query is called *value queries*. Any term t_q can be a value query. The answer to a value query t_q is a tensor of real numbers obtained by computing the grounding of the term, i.e. $\mathcal{G}_{\theta}(t_q)$. Analogously to truth queries, the answer to a value query is a “tensor of tensors” if t_q contains free variables. Using value queries, one can inspect how a constant or a term, more generally, is embedded in the manifold.

17.3.3. Learning

Learning and Reasoning in a Real Logic theory are both associated with optimizing the set of parameters $\theta \in \Theta$ to maximize the satisfaction of some objective. We use the term *grounded theory*, denoted by $\langle \mathcal{K}, \mathcal{G}_{\theta} \rangle$, to refer to a Real Logic theory with a specific set of learned parameter values. To define this optimization problem, we aggregate the truth-values of all the formulas in $\mathcal{K}$ by selecting a *formula aggregating operator* $\text{SatAgg} : [0, 1]^* \rightarrow [0, 1]$.

Definition 17.3.2. The *satisfiability* of a theory $\mathcal{T} = \langle \mathcal{K}, \mathcal{G}_{\theta} \rangle$ with respect to the *aggregating operator* SatAgg is defined as $\text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\theta}(\phi)$.

Learning is the process of searching for the set of parameter values θ^* that maximize the satisfiability of $\mathcal{T}$ w.r.t. a given aggregator:

$$\theta^* = \underset{\theta \in \Theta}{\operatorname{argmax}} \quad \underset{\phi \in \mathcal{K}}{\text{SatAgg}} \mathcal{G}_{\theta}(\phi)$$

Notice that with this general formulation, one can learn the grounding of constants, functions, and predicates. The learning of the grounding of constants corresponds to the learning of *embeddings*. The learning of the grounding of functions corresponds to the learning of *generative models* or a *regression* task. Finally, the learning of the grounding of predicates corresponds to a *classification* task in Machine Learning.

Because Real Logic grounds expressions in real and continuous domains, LTN attaches gradients to every sub-expression and consequently learns through gradient-descent optimization.

LTN can generalize and extrapolate when querying formulas grounded with unseen data (for instance, new individuals from a domain), using knowledge learned with previous groundings (for instance, re-using a trained predicate). As done customarily in ML, one can impose some regularization on the set of parameters or use other traditional approaches to reduce overfitting.

17.3.4. Reasoning

From a pure logic perspective, reasoning is the task of verifying if a formula is a logical consequence of a set of formulas. This can be achieved semantically using model theory ($\models$) or syntactically via a proof theory ($\vdash$). To characterize reasoning in Real Logic, we adapt the notion of logical consequence for fuzzy logic provided in [14]: A formula ϕ is a fuzzy logical consequence of a finite set of formulas Γ , in symbols $\Gamma \models \phi$ if for every fuzzy interpretation f , if all the formulas in Γ are true (i.e. evaluate to 1) in f then ϕ

is true in f . In other words, every model of Γ is a model of ϕ . A direct application of this definition to Real Logic is not practical since in most practical cases the level of satisfiability of a grounded theory $(\mathcal{K}, \mathcal{G}_\theta)$ will not be equal to 1. We therefore define an interval $[q, 1]$ with $\frac{1}{2} < q < 1$ and assume that a formula is true if its truth-value is in the interval $[q, 1]$. This leads to the following definition:

Definition 17.3.3. A closed formula ϕ is a logical consequence of a knowledge-base $(\mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta)$, in symbols $(\mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta) \models_q \phi$, if, for every grounded theory $(\mathcal{K}, \mathcal{G}_\theta)$, if $\text{SatAgg}(\mathcal{K}, \mathcal{G}_\theta) \geq q$ then $\mathcal{G}_\theta(\phi) \geq q$.

Logical consequence by direct application of Definition 17.3.3 requires querying the truth value of ϕ for a potentially infinite set of groundings. Instead, [7] proposes to reason by refutation and thus solve the dual problem:

$$\text{There exists } \theta \in \Theta \text{ such that } \mathcal{G}_\theta(\mathcal{K}) \geq q \text{ and } \mathcal{G}_\theta(\phi) < q, \quad (8)$$

by introducing an alternative objective function. If (8) is true, a counterexample to the logical consequence is found and it is disproved. If (8) is false, no counterexample to the logical consequence is found and it is proved. The reader is referred to [7] for more information.

17.4. Applications of LTN

We now turn to some recent relevant applications of LTN in the areas of semantic image interpretation, ontology reasoning and reinforcement learning.

17.4.1. Semantic Image Interpretation

Semantic Image Interpretation (SII) is the task of extracting scene graphs from images describing their semantic content: the nodes are the image objects, the edges the relations between objects [15]. This task presents a relational-hybrid domain as the objects are linked together and present both numeric and semantic features. In addition, background knowledge can be used to describe the object relations. Therefore, LTN is a suitable framework for SII. To this extent, LTN have been used for the detection of visual relationships: given two bounding boxes, LTN is used to predict their labels and the underlying relationship, i.e. the triple $\langle \text{subj_label}, \text{relation}, \text{obj_label} \rangle$. LTN achieves good results in such a task when using the PASCAL-Part [16] and the Visual Relationship Dataset (VRD) [9].

17.4.1.1. The LTN Framework for SII

The LTN **domains** are the sets $\text{BBoxesPpart}, \text{BBoxesVRD}$ that denote bounding boxes for the PASCAL-Part and the VRD, respectively. The **constants** are the sets $\mathcal{C}_{\text{Ppart}}$ and $\mathcal{C}_{\text{VRD}}$ of symbols for the PASCAL-Part and the VRD bounding boxes, respectively. The **PASCAL-Part predicates** are 20 unary predicates for whole objects and 39 unary predicates for part objects. The **VRD predicates** are 100 unary predicates about animals, vehicles and generic objects and 70 binary predicates for their relationships. Regarding the **axioms**, two knowledge bases have been created for each data set: $\mathcal{K}_{\text{full}}$ containing

factual propositions about the positive and negative examples given by the ground truth (c.f. [16, 9] for more detail) and $\mathcal{K}_{exmpl}$ with general propositions. The PASCAL-Part axioms in $\mathcal{K}_{exmpl}$ comprehend constraints about i) mutual exclusivity between P_i, P_j : $\forall x(P_i(x) \rightarrow \neg P_j(x))$; ii) partOf anti-symmetry and anti-reflexivity; iii) the domains and ranges of the parts of a whole object and the whole objects to which a part belongs. Examples are $\forall xy(\text{Motorcycle}(y) \wedge \text{partOf}(x, y) \rightarrow \text{Handlebar}(x) \vee \text{Headlight}(x) \vee \text{Saddle}(x) \vee \text{Wheel}(x))$ and $\forall xy(\text{Saddle}(x) \wedge \text{partOf}(x, y) \rightarrow \text{Motorcycle}(y) \vee \text{Bicycle}(y))$. The VRD $\mathcal{K}_{exmpl}$ contains negative domain and range constraints that exclude some predicates as domain or range, e.g. physical objects cannot sleep on anything: $\forall xy(\text{sleepon}(x, y) \rightarrow \neg \text{PhyObj}(x))$. The **grounding** of the domains are the RGB pixels of the bounding boxes. Given a bounding box constant b , $\mathcal{G}(b) = \mathbf{b} = \langle \text{score}(P_1, b), \dots, \text{score}(P_n, b), x_0(b), y_0(b), x_1(b), y_1(b) \rangle$, with $x_0(b), y_0(b), x_1(b), y_1(b)$ the coordinates of the top-left and bottom-right corners of b ; $\text{score}(P_i, b)$ is the classification score of an object detector for b according to the predicate P_i . The grounding of pairs $\langle b_1, b_2 \rangle$ is the concatenation of the groundings with additional features regarding joint geometrical properties: $\mathcal{G}(b_1, b_2) = \langle \mathbf{b}_1, \mathbf{b}_2, \mathbf{ef}_{b_1, b_2} \rangle$, where $\mathbf{ef}$ encodes, for example, the area of the intersection of b_1, b_2 or the Euclidean distance between the centroids of $\langle b_1, b_2 \rangle$ (see [9] for more detail). Following the first implementation of LTN [5], the grounding of the predicates are learnt with a *neural tensor network* [5]: $\mathcal{G}_\theta(P_i) : x \mapsto \text{sigmoid}(\text{NTN}_{P_i, \theta}(x))$ and $\mathcal{G}_\theta(R_i) : x, y \mapsto \text{sigmoid}(\text{NTN}_{R_i, \theta}(x, y))$ for each unary and binary predicates P_i, R_i . The **learning** phase has been performed on two different theories for each dataset, $\mathcal{T}_{full} = \langle \mathcal{K}_{full}, \mathcal{G}_\theta \rangle$ and $\mathcal{T}_{exmpl} = \langle \mathcal{K}_{exmpl}, \mathcal{G}_\theta \rangle$, to test the ability of LTN to learn in the presence of logical constraints. The semantics of the connectives was implemented using the Łukasiewicz t-norm, whereas for both the $\forall$ and the formula aggregation, the M_p with $p = -1$, which corresponds to the harmonic mean, was used.

17.4.1.2. Results for the PASCAL-Part data set

Figure 2 shows the results for the indoor objects. For both object types and the part-of relation classification, $\mathcal{T}_{full}$ has better performance than $\mathcal{T}_{exmpl}$. Moreover, background knowledge allows LTN to improve the performance of the Fast R-CNN (FRCNN) object detector with Area Under the Curve (AUC) of 0.800 in comparison with 0.75.

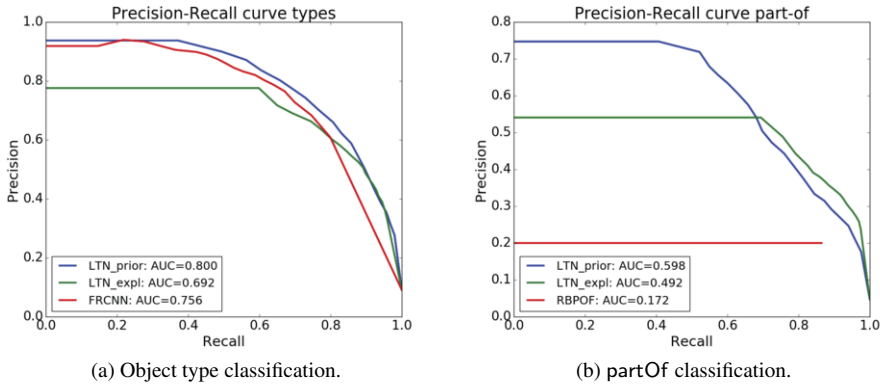


Figure 2. LTN applied to Semantic Image Interpretation: Precision-recall curves and Area Under the Curve (AUC). LTN achieves improvements in AUC for both object type and the part-of object relation classification.

Both $\mathcal{T}_{full}$ and $\mathcal{T}_{expl}$ outperform a rule-based approach (based on the ratio of overlapping between bounding boxes) in the detection of part-of relations, achieving AUC of 0.598 and 0.419 in comparison with 0.172. $\mathcal{T}_{expl}$ outperforms only the rule-based method for the part-of detection. However, this drop is compensated by the logical constraints in $\mathcal{T}_{full}$. For the other object types (animals and vehicles), LTN obtains results comparable to FRCNN: FRCNN beats $\mathcal{T}_{full}$ marginally by 0.05 and 0.037 AUC for animals and vehicles, respectively.

Results for Data Poisoning: Training sets can be affected by annotation noise, disagreements or data poisoning. The results in Figure 3 show the robustness of LTN models with respect to such label inconsistencies. An increasing amount of noise is added to the training labels and a measure of the performance degradation is taken. For an error rate $er \in \{0, 10, 20, 30, 40\}$, the $er\%$ of the bounding boxes in the training data have had their classification labels changed randomly. In addition, the value of the part-of relation label for a random $er\%$ of pairs of bounding boxes has been flipped. The results for $er = 0$ were described before. Then, two sets of LTN grounded theories, $\{\mathcal{T}_{expl}^{er}\}_{er=0}^{40}$ and $\{\mathcal{T}_{full}^{er}\}_{er=0}^{40}$, have been trained according to the values of er and evaluated.

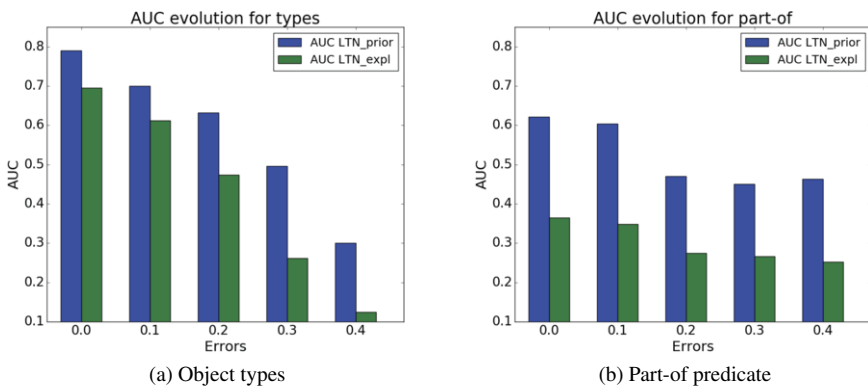


Figure 3. Results of adding noisy training labels: AUCs for indoor object types and part-of relations with increasing noise in the labels of the training data. The symbolic logic constraints used by LTN offer robustness to errors in the labels.

The symbolic logic constraints used by LTN offer robustness to increasing errors in the labels of the training set. In addition to the expected overall drop in performance, an increasing gap can be seen between the drop in performance of $\mathcal{T}_{expl}$ and $\mathcal{T}_{full}$. This drop is noticeably smaller for $\mathcal{T}_{full}$.

17.4.1.3. Results on Zero-Shot Learning

The VRD data set is more challenging than the PASCAL-Part as it contains more predicates and relations and involves more difficult detection tasks. The detection of *phrases* involves the detection of a single bounding box containing two labeled objects and their relation, whereas in *relationship detection*, two labeled bounding boxes and their relations need to be detected. In the *predicate detection* tasks, the two labeled bounding boxes are given and only their relationship has to be classified. In the zero-shot learning setting, the results are tested only on visual relationships never seen in the training set.

Table 1 contains the LTN results (recall@50 metric) compared with standard methods. For ablation studies and recall@100 results, the reader is referred to [9].

Method	Phrase Detect.	Relationship Detect.	Predicate Detect.
$\mathcal{T}_{exmpl}$	11.00 $\pm$ 0.34	10.01 $\pm$ 0.38	56.25 $\pm$ 0.75
$\mathcal{T}_{full}$	11.40 $\pm$ 0.31	10.47 $\pm$ 0.32	57.34 $\pm$ 0.78
LKD [17]	12.96	12.02	54.20
VRL [18]	9.17	7.94	-
Context-AwareVRD [19]	10.78	9.54	16.37

Table 1. The use of the logical constraints by LTN in $\mathcal{T}_{full}$ leads to the best results outperforming the other methods in the zero-shot learning predicate detection task.

The phrase and the relationship detection tasks involve the use of an object detector whose detected bounding boxes and labels are provided with the VRD [19, 18]. This allows for a fair comparison with the models starting from the same level of object detection error. Both $\mathcal{T}_{exmpl}$ and $\mathcal{T}_{full}$ models outperform these competitors showing that LTN can deal better with the object detection errors. The performance of LKD [17] is slightly better than LTN, however, LKD uses a more recent object detector with better performance than the one provided by VRD. On the other hand, the predicate detection task is independent from the object detection. Therefore, this allows for a fair comparison among all the methods. Here, $\mathcal{T}_{exmpl}$ has similar performance to LKD, but the use of logical constraints in $\mathcal{T}_{full}$ improves on the LKD classification results. This means that the negative domain and range constraints are effective at excluding some binary predicates for a bounding box with a given subject or object. These results confirm i) the ability of LTN in the visual relationship detection to generalize to never seen triples; ii) the ability of LTN to leverage logical constraints.

17.4.2. Complementing Logical Reasoning with Sub-symbolic Commonsense

In LTN, constants of the logical language are represented by vectors. One interesting question that can be asked is if these vectors can be initialized with some “knowledge” and if this can be shown useful for reasoning. We investigate an experiment [8, 20] that will bring evidence that combining distributional representation of entities [21] with LTN can be a promising approach to reasoning. First, we introduce the basic components of the combination between LTN and distributional embeddings and the motivation behind this.

17.4.2.1. Background

Distributional Entity Embeddings: With distributional embeddings, we refer to vector representations of terms - words, entities occurring in text - that have been created under the distributional hypothesis [22]. This hypothesis states that the meaning of a term is a function of the context in which the term appears. We expect words like “cat” and “kitten” to occur in similar contexts in text and thus to be similar.

Distributional embedding models such as word2vec [23] take inspiration from this hypothesis to generate vector representations of terms starting from an input corpus in which sequences of terms appear (e.g. sentences).

In the vector space generated by models like word2vec, each word is represented with a vector and the similarity in the space is a function that approximates the similarity between two words: “cat” and “kitten” will have similar vector representations based on the context in which they appear. These representations, often referred to as word embeddings, have been found to be cognitively plausible to some extent by psychologists [24] and we use them to represent commonsense knowledge.

However, for our objective of combining these representations with LTN, we cannot use pure word embeddings: words are ambiguous, and to represent logical constraints we need to use terms that do not have this problem. To solve this, we can annotate the text to find knowledge graph entities in text [25]. The words in text can be replaced with entity identifiers (such as done in <https://dbpedia.org/page/Cat>) that are not ambiguous. The final result of this process is that we obtain a corpus that contains sequences of entities. Thus, after applying distributional embedding algorithms like word2vec to the annotated text, we generate vector representations of entities - our entity embeddings.

LTN and Distributional Embeddings: Knowing that tigers and cats are similar (i.e. they have similar embeddings) can count as evidence to make common inferences about them: we might not know that tigers are mammals, however if we know that cats are mammals, than we might expect tigers to be mammals too. This is the key property that we would like our combination between embeddings and LTN to show: initializing LTN with distributional embeddings is supposed to bring an advantage when performing inference because the model can access more knowledge: explicit knowledge and knowledge by similarity.

17.4.2.2. Reasoning Experiments

We use the entity embeddings from [21]. These embeddings are generated using the well-known word embedding algorithm skip-gram [23] on a corpus of documents - Wikipedia abstracts - that have been annotated with the use of DBpedia Spotlight [26]. As already mentioned, the entity embeddings are used in the place of LTN constants which are not updated during training time.

The grounding of the predicates is done using Neural Tensor Networks [6] (with $k = 20$). We also use the fuzzy Lukasiewicz operators as described in [5]. All the material needed to replicate this experimental evaluation is available from https://github.com/vinid/logical_commonsense.

Reference Knowledge Base: For our experiment, we make use of a small knowledge base derived from the DBpedia Knowledge Graph.⁴ To collect this, we first use the DBpedia SPARQL endpoint to collect entities coming from the following classes: Mammal (0.38%), Fungus (0.17%), Bacteria (0.03%) and finally Plant (0.42%). We create the classes Animal, Eukaryote, and Species and we generate the transitive closure with respect to all the classes. For example, if from the Mammal class we collected the entity `dbr : cat`, this entity is going to be subclass of Mammal, Animal, Eukaryote, and Species. We also generate all the negative instantiated atoms such as `¬fungus(dbr : cat)`.

Considering all the positive and negative instantiated atoms, this knowledge base contains 35,133 elements. To better understand how effective it is to combine distributional embeddings with LTN, we divide this knowledge base into three small data sets:

⁴<https://www.dbpedia.org/>

- **D1.**

- * **Data:** In training, we have 1,400 positive atoms (e.g, $\text{mammal}(\text{dbr} : \text{cat})$). During testing the models have to find all the other 7,077 atoms that are exclusively related to the entities found in the 1,400 atoms; for example, models have to infer something about the instance “dbr:cat” even if the only atom that was present in the training set was $\text{species}(\text{dbr} : \text{cat})$.
- * **Objective:** This dataset has been created to evaluate the performance of the models in a task in which there are no negative examples.

- **D2.**

- * **Data:** In training, we have 7,026 atoms both positive and negative. During testing the models have to find all the other 20,890 atoms (positive and negative).
- * **Objective:** This dataset has been created to evaluate the performance in a task in which the models can access to both positive and negative atoms. Note that we force the fact that each entity in the test set has to appear also in the training set, in this way all the models can be used to make inferences.

- **D3.**

- * **Data:** In training, we have 1,756 atoms. The models are now asked to infer the value of the rest of the entire dataset 33,377 atoms (positive and negative).
- * **Objective:** This dataset has been created to evaluate the performance of the model in a task in which both positive and negative atoms are given, but in which the test set contains atoms of entities that were not present in the training set. The models will need to rely on the distributional embeddings to infer the values of predicates with respect to unseen entities.

To support LTN reasoning, we also define 22 universally quantified formulas that will be used during training (this will be our domain theory). Here, we show some of these formulas, the complete list is available in the online repository. These formulas are meant to give the model knowledge about the taxonomy of the data that we have defined.

- $\forall x(\text{plant}(x) \rightarrow \text{eukaryote}(x))$
- $\forall x(\text{mammal}(x) \rightarrow \text{animal}(x))$
- $\forall x(\text{plant}(x) \rightarrow \neg \text{mammal}(x))$
- $\forall x(\text{fungus}(x) \rightarrow \neg \text{animal}(x))$

Models: We will refer to the model that combines the embeddings of the entities and LTN as LTN_{EE} . This model is going to be compared with the following other models:

- A LTN model that is not initialized with the embeddings. This model is used to see if adding the distributional embeddings has some effect on the reasoning process. If LTN_{EE} shows better performance than the LTN we can conclude that the distributional embeddings are helpful.
- A Probabilistic Soft Logic [27] (PSL) model that can be trained on both atoms and universally quantified formulas. The tool that we used in this experiment can be found in the webpage of the project, <https://psl.linqs.org/>, where we use the default parameters of the model.

- A Deep Neural Network (DNN) that makes use of the entity embeddings and that is trained to classify the various atoms as 0 or 1 (we are thus treating the problem as a binary classification task). The network can not make use of the domain theory we have defined above. The DNN we use has the following structure: it uses the distributional representations of entities and uses a one-hot representation of predicates projected in 20 dimensions. These two elements are then concatenated and then projected into a 1-dimensional output layer. We use a sigmoid function on the output layer so that we can have a score in $[0, 1]$. We apply early stopping on this training procedure using a validation set created with 20% of the training samples.

17.4.2.3. Experimental Results

Figures 4, 5 and 6 show the F1 scores for each predicate for each model.

Experiments on D1: In this setting we compare LTN_{EE} with LTN and PSL . The DNN cannot be trained because we are using only positive atoms (i.e., all the examples have the same label for the DNN); we indeed expect the other models to use the universally quantified formulas to fill this gap. Results show that LTN_{EE} is the best performing model. The comparison between pure LTN and PSL shows instead that while the latter performs better their difference is negligible in this setting.

Experiments on D2: In this experiment we have added the negated axioms and we know that each entity in the test set appears at least once in the training set. PSL performance is more or less similar to the one shown on the D1 dataset. Instead the performance between LTN_{EE} and DNN is comparable. In this settings, the domain theory does not seem to provide increases in performance.

Experiments on D3: The results from this experiment indicate that LTN_{EE} generalizes slightly better than the DNN . This could be due to both the domain theory and to the fact that LTN_{EE} trains each predicate as a separate tensor layer. The F1 scores for Fungus and Bacteria shows a minor drop in performance with respect to the previous experiment. This might be due to the fact that the distributional representations of elements of the class Fungus are similar to those of the class Plant. On the other hand, the class Bacteria has only a few instances in this experiment, and this might make it more difficult for the model to learn a good representation. In general, DNN and LTN_{EE} show similar performances, but LTN_{EE} seems to be slightly better.

Nonetheless, the fact that LTN can be used for after training logical inferences gives more flexibility to the model and shows that the combination between embeddings and reasoning is promising.

17.4.3. Annotating RL Environments using Prior Knowledge

Recently, much of AI and ML has been concerned with end-to-end learning for solving complex decision tasks such as Atari Games and Go [28, 29]. Almost all recent progress in this field has been driven by applications of deep neural networks to Reinforcement Learning (RL) tasks – a subfield known as Deep Reinforcement Learning (DRL). While DRL has shown important progress over the past years, state-of-the-art algorithms still struggle to achieve human-like performance within limited training time. The results of the recent NetHack Challenge - <https://nethackchallenge.com/> - which took

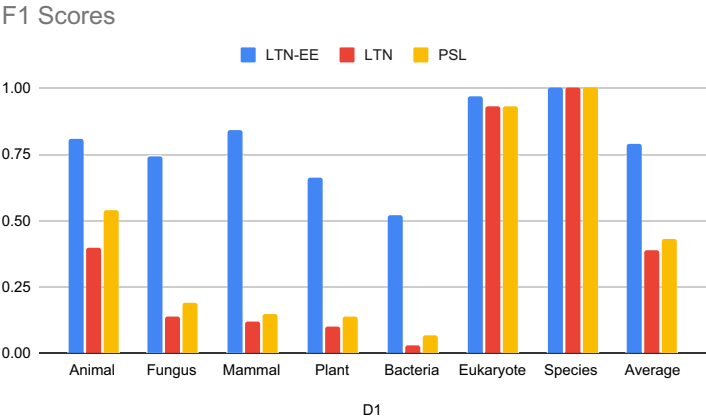


Figure 4. Results of the D1 dataset. F1 is shown per class.

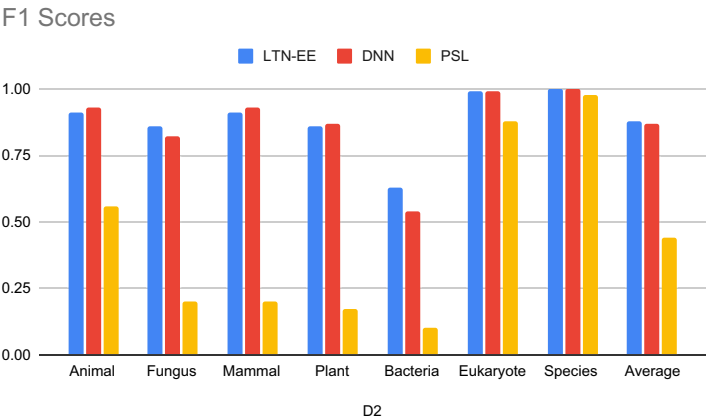


Figure 5. Results of the D2 dataset. F1 is shown per class.

place at NeurIPS 2021, also point to the need for more research combining DRL with knowledge, especially when rewards are known to be sparse.

Traditional DRL agents’ learning is slow and algorithms require lots of training data. Humans on the other hand can quickly understand and solve tasks they have never seen before in complex environments. Such efficiency is likely explained by the usage of prior knowledge on object and event semantics of the environment [30].

We propose LTN as a principled method for annotating the environment with prior knowledge, to use in DRL agents for transfer learning. More specifically, we focus on *object semantics priors* and describe high-level symbolic facts about objects in the environment –e.g. object x is an enemy, y a key, z a door, etc.– using a first-order language. The knowledge is provided by the human (as prior knowledge) using Real Logic and joined to the image describing the environment. A DRL algorithm is then trained on conjoint image and semantic data and can choose to exploit prior information if it helps

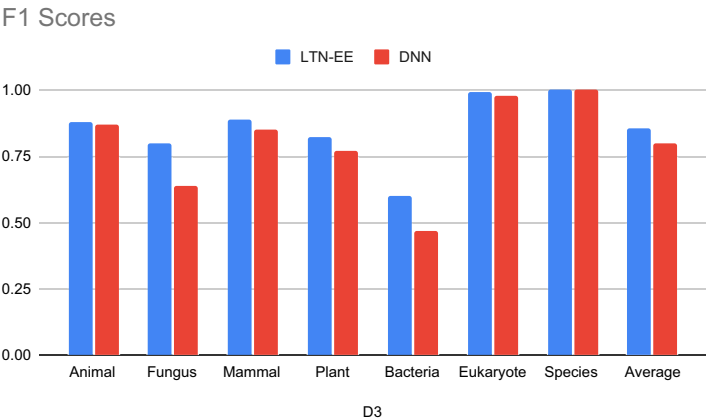


Figure 6. Results of the D3 dataset. F1 is shown per class.

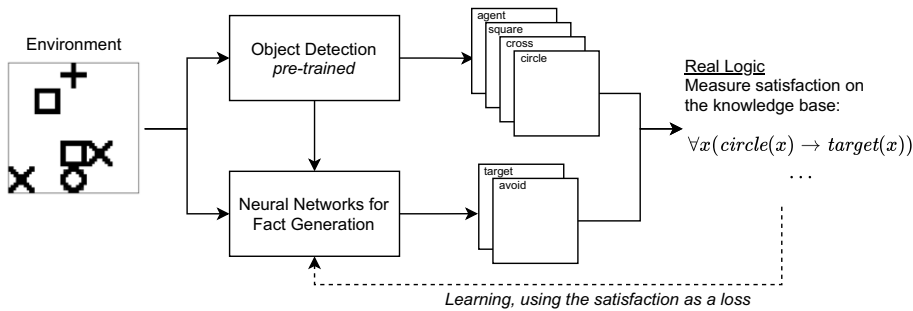


Figure 7. The environment consists of an agent (+ sign) moving on a grid and collecting objects of different types *circle*, *square* and *cross*. Some types –e.g. *circle*– are associated as *target* (+1 reward on collection), others are to *avoid* (-1 reward on collection). We train semantic segmentation networks to generate facts about the environment using a dataset of images sampled from the environment.

performance and learning. That is, the system can learn to take advantage of both the symbolic layer and the conventional layer in a single decision selection module.

17.4.3.1. RL Environment

We illustrate the approach in a simple grid environment inspired by previous work [31]. An agent moves on a 5 × 5 grid and collects objects of different types: *circle*, *square* and *cross* (see Figure 7). The task for the agent is to collect all objects of a particular *target type* – e.g. circles – and avoid all objects of *avoid type* – e.g. crosses and squares –. The agent receives a +1 reward upon collecting a *target type* object, a -1 reward upon collecting an *avoid type* object.

The environment is represented by a 50 × 50 pixel image. The agent can move in the environment with the following actions: *move-up*, *move-down*, *move-left*, *move-right*. An object is collected when the agent enters a field. Number and position of objects as well as the position of the agent are randomized for each trial. A trial ends when the agent has collected all target objects or after 50 steps.

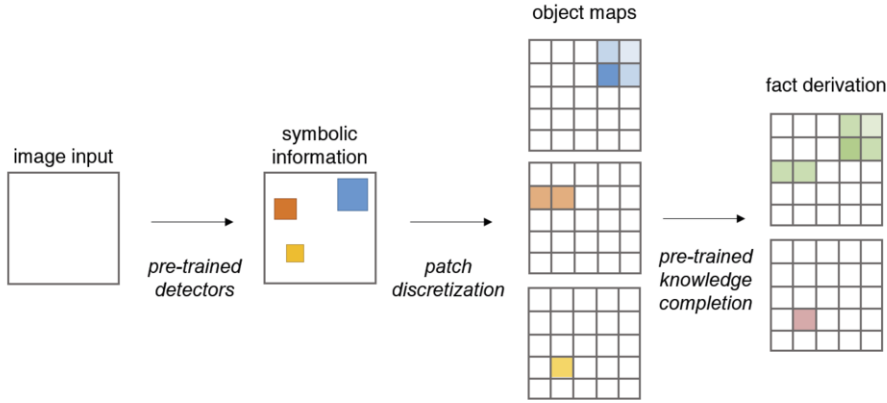


Figure 8. Domain of discourse: we discretize the images of the environment in patch representations, similar to feature maps. Each cell of the maps constitutes an individual of our first-order language. Each channel of the maps is associated with a type predicate. The maps are filled with the corresponding detection results, using interpolation methods when necessary.

17.4.3.2. Prior Knowledge Representation

We inject prior knowledge into the agent's model using a three step process (Figure 8).

- 1. Symbolic descriptions:** Symbolic reasoning on objects of an image is only possible if high-level symbolic descriptions are available to the agent. We extract these descriptions from the low-level pixel features in each image using pre-trained object detectors. We have classifiers for agent, circle, square, cross.
- 2. Object maps:** We discretize the environment in a patch representation similar to feature maps. Each cell of the maps constitutes an individual of our first-order language. That is, this step defines the domain of discourse of our language. Each channel of the maps is associated with a type predicate. The maps are filled with the corresponding detection results, using interpolation methods when necessary.
- 3. Fact derivation:** Facts about which objects to *avoid* and which to *goto* are computed based on background knowledge (c.f. Figure 7) to generate additional feature maps.

Note that the domain of objects in our real logic language is not the bounding boxes generated by the object detectors. The number of bounding boxes can vary from one frame to another; annotations associated directly with these objects would vary in dimension between each frame, making it difficult to incorporate in traditional DRL agents, which usually deal with input spaces of fixed dimensions. Instead, we use cell-level (or pixel-level) annotations.

The grounded theory on the grid environment consists of the following type predicates circle, square, square, agent and the following more general predicates avoid, target. The type predicates are grounded with pre-trained neural object detectors. The two general predicates are grounded with neural networks, trained using the following rules:

$$\forall x (\text{circle}(x) \leftrightarrow \text{target}(x)) \quad (9)$$

$$\forall x (\text{cross}(x) \vee \text{square}(x) \leftrightarrow \text{avoid}(x)) \quad (10)$$

17.4.3.3. Knowledge Injection in a DRL agent

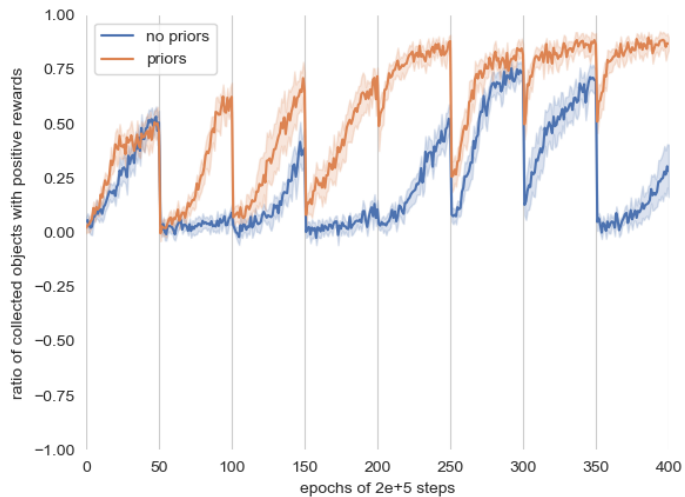
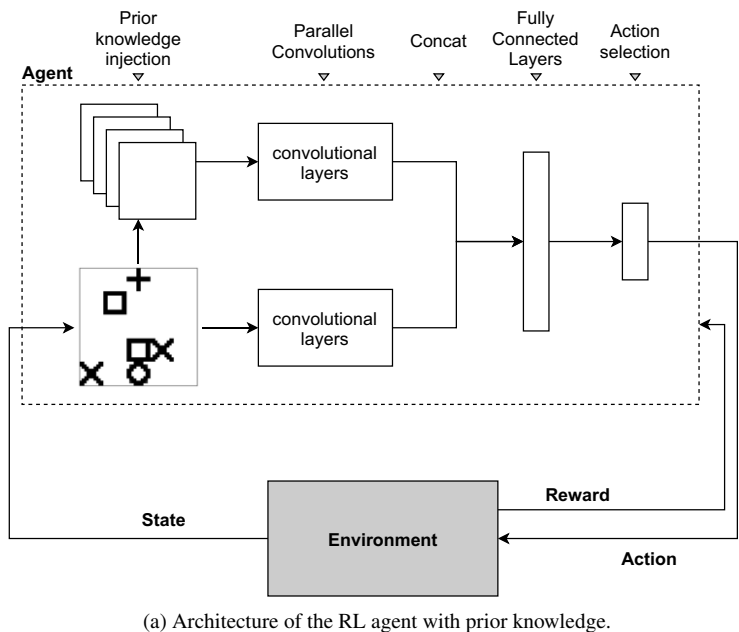


Figure 9. Comparison of a Deep Q-Network (DQN) training performance with and without prior knowledge. The version with prior knowledge uses LTN and learns faster than DQN. The drop in performance at each scenario transition is reduced over time with the use of LTN.

RL agents solve tasks by observing the environment through sensors, choosing to act accordingly, and occasionally receiving a reward for their actions. The goal of the agent

is to learn a policy that maximizes expected reward across trials. We propose the agent architecture in Figure 9a. The current image of the environment is input to the agent and processed with prior knowledge. Both the original input and the prior knowledge derived maps are input into parallel convolutional layers then concatenated into a single representation. The concatenation result is the input for a Deep Reinforcement Learning algorithm. For the environment discussed here, algorithms that can deal with image-like input spaces and discrete action spaces are appropriate. Hence, we apply a Deep Q-Network (DQN) architecture with experience replay. [28] The output is one of four possible actions (*move-up*, *move-down*, *move-left*, *move-right*).

We test the agent on the following experiment: from the basic grid environment, a more complex task is created where the agent is trained on the same game but rendered with different color settings.

Set 1. black objects, white background

Set 2. white objects, black background

Set 3. red objects, blue background

Set 4. blue objects, red background

The settings represent different environments where objects and backgrounds look visually different but interactions can be generalized, here by using a symbolic layer. We show the results in Figure 9b, for an average of 5 runs. Every 50 epochs, we change the color settings of the game. The agent should learn to generalize by relying on the symbolic part of the input and understand objects past their visual context.

The agents' network is trained every 200 timesteps. 100 training iterations are defined as an epoch. The agent is evaluated every epoch by measuring the collected rewards normalized by the potential maximum reward on the environment. Evaluations are averaged on 50 trajectories. The position of the agent and the position and number of objects are randomized at each trajectory.

The baseline algorithm *without prior* essentially relearns the task every time the visual settings changes. The performance *with priors* shows two trends vs the baseline. 1) The learning seems to be much faster. 2) The drop in performance at each scenario change becomes smaller and smaller. In other words, over time, the system learns to rely on the symbolic information and becomes immune to visual changes in the environment. In summary, object priors help the system to learn faster and to be able to transfer task performance between different scenarios.

17.5. Related Work

The past years have seen considerable work aiming to integrate symbolic systems and neural networks. We focus on work whose objective is to build computational models that integrate deep learning and logical reasoning into a so-called end-to-end (fully differentiable) architecture. We use the categorization defined in [7] as outlined in Figure 10.

17.5.1. Neural architectures for logical reasoning

These use neural networks to perform (probabilistic) inference on logical theories. Early work in this direction has shown correspondences between various logical-symbolic sys-

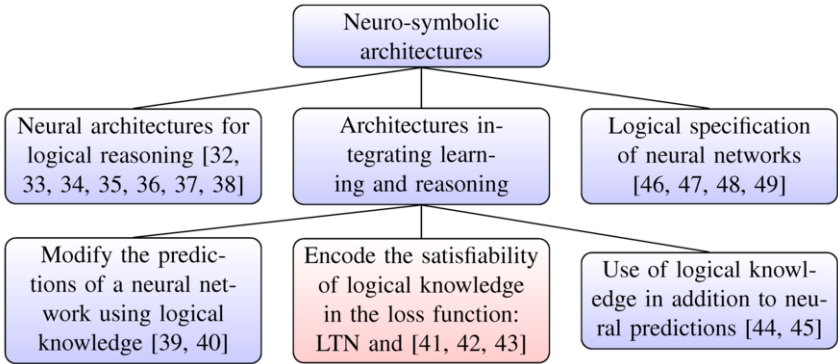


Figure 10. Three classes of neurosymbolic approaches with *Architectures Integrating Learning and Reasoning* further sub-divided into three sub-classes, with LTN belonging to the sub-class highlighted in red.

tems and neural network models [33, 50, 51, 52, 53]. Current neural networks (including deep learning) have been shown capable of representing propositional logic and fragments of first-order logic, but not full first-order or higher-order logic. Recently, there has been a resurgence of interest in the topic with many proposals emerging [32, 35, 36]. In [32], each clause of a Stochastic Logic Program is converted into a factor graph with reasoning becoming differentiable so that it can be implemented by deep networks. In [54], a differentiable unification algorithm is introduced with theorem proving sought to be carried out inside the neural network. Furthermore, in [55, 54] neural networks are used to learn reasoning strategies and logical rule induction.

17.5.2. *Logical specification of neural network architectures*

Here the goal is to use a logical language to specify the architecture of a neural network. Examples include [32, 46, 47, 48, 49]. In [47], the languages of extended logic programming (logic programs with negation by failure) and answer set programming are used as background knowledge to set up the initial architecture and set of weights of a recurrent neural network, which is subsequently trained from data using backpropagation. In [46], first-order logic programs in the form of Horn clauses are used to define a neural network that can solve Inductive Logic Programming tasks, starting from the most specific hypotheses covering the set of examples. Lifted relational neural networks [49] is a declarative framework where a Datalog program is used as a compact specification of a diverse range of existing advanced neural architectures, with a particular focus on Graph Neural Networks (GNNs) and their generalizations. In [48] a weighted Real Logic is introduced and used to specify neurons in a highly modular neural network that resembles a tree structure, whereby neurons with different activation functions are used to implement the different logic operators.

17.5.3. *Architectures for the integration of inductive learning and deductive reasoning*

These architectures seek to enable the integration of inductive and deductive reasoning in a unique fully differentiable framework. The systems that belong to this class combine a neural component with a logical component. The former consists of one or more

neural networks, the latter provides a set of algorithms for performing logical tasks such as model checking, satisfiability, and logical consequence. These two components are tightly integrated so that learning and inference in the neural component are influenced by reasoning in the logical component and vice versa. They can be further separated into three sub-classes:

1. Approaches that introduce additional layers to the neural network to encode logical constraints which modify the predictions of the network. This sub-class includes Deep Logic Models [40] and Knowledge Enhanced Neural Networks [39].
2. Approaches that integrate logical knowledge as additional constraints in the objective function or loss function used to train the neural network (LTN and [41, 42, 43]).
3. Approaches that apply (differentiable) logical inference to compute the consequences of the predictions made by a set of base neural networks. Examples of this sub-class are DeepProbLog [45] and Abductive Learning [44].

This latter category is the closest fit to LTN, though to some extent, it is also possible to specify neural architectures using logic (Section 17.5.2) in LTN. For example, if one defines a classifier $P(x, y)$ as the formula $P(x, y) = (Q(x, y) \wedge R(y)) \vee S(x, y)$, the resulting grounding is a computational graph that combines sub-architectures following the syntax of the logical formula. Furthermore, recent efforts to integrate reasoning with LTN [7, 56] are reminiscent of the category in Section 17.5.1.

17.6. Conclusions and Future Work

We have presented the theory of Logic Tensor Networks as a model and system for neurosymbolic AI and have illustrated its use in practice when applied to image interpretation, ontology reasoning from text, and reinforcement learning. This chapter therefore compiles the most important applications of LTN in the recent literature, showing that LTN is capable of combining approximate reasoning and deep learning from knowledge and data for diverse tasks such as semantic image interpretation, ontology learning and reasoning, and reinforcement learning applied to games. Integrative neural-symbolic approaches are known for either seeking to bring neurons into a symbolic AI model [45] or bring symbols into neurons [57]. LTN adopts the latter approach but maintaining a close link between the symbols and their grounding into the neural network. The discussion around these two options - neurons to symbols vs. symbols to neurons - is likely to take center stage in the debate around neurosymbolic AI in the next decade. LTN and related approaches are well placed to play an important role in this debate by offering a rich logical language tightly coupled with an efficient distributed implementation into TensorFlow computational graphs.

Finally, we argue that the close connection between first-order logic and its implementation into deep networks makes LTN especially suitable as a model for the neural-symbolic cycle [33, 58], which translates between neural and symbolic representations. Such translations take place based on the structure of the neural network given a symbolic language, as done in [33, 59], or at the level of the loss functions, as done in [32, 60, 40]. Of particular relevance to pressing issues around fairness and trust in AI, the neural-symbolic cycle may offer an important tool for reducing bias in AI via the iterative appli-

cation of symbolic reasoning alongside black box deep learning, as outlined e.g. in [59] also with the use of LTN.

References

- [1] d'Avila Garcez A, Gori M, Lamb LC, et al. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *FLAP*. 2019;6(4):611–632. Available from: <https://collegepublications.co.uk/ifcolog/?00033>.
- [2] d'Avila Garcez A, Lamb LC. Neurosymbolic AI: The 3rd wave ; 2020.
- [3] Lamb LC, d'Avila Garcez A, Gori M, et al. Graph neural networks meet neural-symbolic computing: A survey and perspective. In: Bessiere C, editor. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020* [scheduled for July 2020, Yokohama, Japan, postponed due to the Corona pandemic]. *ijcai.org*; 2020. p. 4877–4884. Available from: <https://doi.org/10.24963/ijcai.2020/679>.
- [4] Raedt LD, Dumančić S, Manhaeve R, et al. From statistical relational to neuro-symbolic artificial intelligence ; 2020.
- [5] Serafini L, d'Avila Garcez A. Learning and reasoning with logic tensor networks. In: *Conference of the Italian Association for Artificial Intelligence*; Springer; 2016. p. 334–348.
- [6] Socher R, Chen D, Manning CD, et al. Reasoning with Neural Tensor Networks for Knowledge Base Completion. In: *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 1*; USA. Curran Associates Inc.; 2013. p. 926–934; NIPS'13. Event-place: Lake Tahoe, Nevada; Available from: <http://dl.acm.org/citation.cfm?id=2999611.2999715>.
- [7] Badreddine S, d'Avila Garcez A, Serafini L, et al. Logic tensor networks. *Artificial Intelligence*. 2022;303:103649. Available from: <https://www.sciencedirect.com/science/article/pii/S0004370221002009>.
- [8] Bianchi F, Palmonari M, Hitzler P, et al. Complementing logical reasoning with sub-symbolic common-sense. In: *International Joint Conference on Rules and Reasoning*; Springer; 2019. p. 161–170.
- [9] Donadello I, Serafini L. Compensating supervision incompleteness with prior knowledge in semantic image interpretation. In: *2019 International Joint Conference on Neural Networks (IJCNN)*; IEEE; 2019. p. 1–8.
- [10] Badreddine S, Spranger M. Injecting Prior Knowledge for Transfer Learning into Reinforcement Learning Algorithms using Logic Tensor Networks. *arXiv:1906.06576* [cs, stat]. 2019 Jun; *ArXiv*: 1906.06576; Available from: <http://arxiv.org/abs/1906.06576>.
- [11] van Krieken E, Acar E, van Harmelen F. Semi-Supervised Learning using Differentiable Reasoning. *arXiv:1908.04700* [cs]. 2019 Aug; *ArXiv*: 1908.04700; Available from: <http://arxiv.org/abs/1908.04700>.
- [12] Hajek P. *Metamathematics of Fuzzy Logic*. Kluwer Academic Publishers; 1998.
- [13] van Krieken E, Acar E, van Harmelen F. Analyzing Differentiable Fuzzy Logic Operators. *arXiv:2002.06100* [cs]. 2020 Feb; *ArXiv*: 2002.06100; Available from: <http://arxiv.org/abs/2002.06100>.
- [14] Běhounek L, Cintula P, Hájek P. Introduction to mathematical fuzzy logic. In: Cintula P, Hájek P, Noguera C, editors. *Handbook of mathematical fuzzy logic, volume 1. (Studies in Logic, Mathematical Logic and Foundations; Vol. 37)*. College Publications; 2011. p. 1–102.
- [15] Donadello I, Serafini L. Mixing low-level and semantic features for image interpretation - A framework and a simple case study. In: *ECCV Workshops (2); (Lecture Notes in Computer Science; Vol. 8926)*. Springer; 2014. p. 283–298.
- [16] Donadello I, Serafini L, d'Avila Garcez A. Logic tensor networks for semantic image interpretation. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*; 2017. p. 1596–1602. Available from: <https://doi.org/10.24963/ijcai.2017/221>.
- [17] Yu R, Li A, Morariu VI, et al. Visual relationship detection with internal and external linguistic knowledge distillation. In: *ICCV*. IEEE Computer Society; 2017. p. 1068–1076.
- [18] Liang X, Lee L, Xing EP. Deep variation-structured reinforcement learning for visual relationship and attribute detection. In: *CVPR*. IEEE Computer Society; 2017. p. 4408–4417.
- [19] Zhuang B, Liu L, Shen C, et al. Towards context-aware interaction recognition for visual relationship detection. In: *ICCV*. IEEE Computer Society; 2017. p. 589–598.

- [20] Ebrahimi M, Eberhart A, Bianchi F, et al. Towards bridging the neuro-symbolic gap: Deep deductive reasoners. *Applied Intelligence*. 2021;:1–23.
- [21] Bianchi F, Palmonari M, Nozza D. Towards encoding time in text-based entity embeddings. In: *The Semantic Web – ISWC 2018*; Cham. Springer International Publishing; 2018. p. 56–71.
- [22] Harris ZS. Distributional structure. *Word*. 1954;10(2-3):146–162.
- [23] Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality. In: *Advances in neural information processing systems*; 2013. p. 3111–3119.
- [24] Lenci A. Distributional semantics in linguistic and cognitive research. *Italian journal of linguistics*. 2008; 20(1):1–31.
- [25] Federico B, Gaetano R, Luca C, et al. Knowledge graph embeddings and explainable ai. *Studies on the Semantic Web*. 2020;:49–72 Available from: <https://doi.org/10.3233/SSW200011>.
- [26] Mendes PN, Jakob M, García-Silva A, et al. Dbpedia spotlight: shedding light on the web of documents. In: *Proceedings of the 7th international conference on semantic systems*; 2011. p. 1–8.
- [27] Bach SH, Broecheler M, Huang B, et al. Hinge-loss markov random fields and probabilistic soft logic. *Journal of Machine Learning Research*. 2017;18:1–67.
- [28] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. *Nature*. 2015 Feb;518(7540):529–533. Available from: <http://www.nature.com/articles/nature14236>.
- [29] Silver D, Schrittwieser J, Simonyan K, et al. Mastering the game of go without human knowledge. *Nature*. 2017;550(7676):354.
- [30] Dubey R, Agrawal P, Pathak D, et al. Investigating Human Priors for Playing Video Games. *arXiv:180210217 [cs]*. 2018 Feb;ArXiv: 1802.10217; Available from: <http://arxiv.org/abs/1802.10217>.
- [31] Garnelo M, Arulkumaran K, Shanahan M. Towards Deep Symbolic Reinforcement Learning. *arXiv:160905518 [cs]*. 2016 Sep;ArXiv: 1609.05518; Available from: <http://arxiv.org/abs/1609.05518>.
- [32] Cohen WW, Yang F, Mazaitis K. Tensorlog: A probabilistic database implemented using deep-learning infrastructure. *J Artif Intell Res*. 2020;67:285–325. Available from: <https://doi.org/10.1613/jair.1.11944>.
- [33] d’Avila Garcez A, Lamb LC, Gabbay DM. *Neural-symbolic cognitive reasoning*. 1st ed. Springer Publishing Company, Incorporated; 2008.
- [34] Hohenecker P, Lukasiewicz T. Ontology reasoning with deep neural networks. *Journal of Artificial Intelligence Research*. 2020;68:503–540.
- [35] Marra G, Kuželka O. Neural markov logic networks. *arXiv preprint arXiv:190513462*. 2019;.
- [36] Qu M, Tang J. Probabilistic logic neural networks for reasoning. In: *Advances in Neural Information Processing Systems*; 2019. p. 7712–7722.
- [37] Rocktäschel T, Riedel S. End-to-end differentiable proving. In: *Advances in Neural Information Processing Systems*; 2017. p. 3788–3800.
- [38] Wang PW, Donti PL, Wilder B, et al. Satnet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver. *arXiv preprint arXiv:190512149*. 2019;.
- [39] Daniele A, Serafini L. Knowledge enhanced neural networks. In: *Pacific Rim International Conference on Artificial Intelligence*; Springer; 2019. p. 542–554.
- [40] Marra G, Giannini F, Diligenti M, et al. Integrating learning and reasoning with deep logic models. In: Brefeld U, Fromont É, Hotho A, et al., editors. *Machine Learning and Knowledge Discovery in Databases - European Conference, ECML PKDD 2019, Würzburg, Germany, September 16-20, 2019, Proceedings, Part II*; (Lecture Notes in Computer Science; Vol. 11907). Springer; 2019. p. 517–532. Available from: https://doi.org/10.1007/978-3-030-46147-8_31.
- [41] Fischer M, Balunovic M, Drachler-Cohen D, et al. DL2: Training and querying neural networks with logic. In: *International Conference on Machine Learning*; 2019. p. 1931–1941.
- [42] Hu Z, Ma X, Liu Z, et al. Harnessing deep neural networks with logic rules. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; Aug.; Berlin, Germany. Association for Computational Linguistics; 2016. p. 2410–2420.
- [43] Marra G, Giannini F, Diligenti M, et al. Lyrics: A general interface layer to integrate logic inference and deep learning. In: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*; Springer; 2019. p. 283–298.
- [44] Dai WZ, Xu Q, Yu Y, et al. Bridging machine learning and logical reasoning by abductive learning.

- In: Proceedings of the 33rd International Conference on Neural Information Processing Systems; USA. Curran Associates Inc.; 2019. NeurIPS'19.
- [45] Manhaeve R, Dumancie S, Kimmig A, et al. Deepproblog: Neural probabilistic logic programming. In: Proceedings of the 32nd International Conference on Neural Information Processing Systems; USA. Curran Associates Inc.; 2018. p. 3753–3763; NeurIPS'18. Available from: <http://dl.acm.org/citation.cfm?id=3327144.3327291>.
- [46] Franca M, Zaverucha G, d'Avila Garcez A. Fast relational learning using bottom clause propositionalization with artificial neural networks. Machine Learning. 2014 01;94:81–104.
- [47] d'Avila Garcez A, Gabbay DM, Broda KB. Neural-symbolic learning system: Foundations and applications. Berlin, Heidelberg: Springer-Verlag; 2002.
- [48] Riegel R, Gray A, Luus F, et al. Logical neural networks ; 2020.
- [49] Sourek G, Aschenbrenner V, Zelezny F, et al. Lifted relational neural networks: Efficient learning of latent relational structures. Journal of Artificial Intelligence Research. 2018;62:69–100.
- [50] Hölldobler S, Kurfess FJ. CHCL - A connectionist inference system. In: Fronhöfer B, Wrightson G, editors. Parallelization in Inference Systems, International Workshop, Dagstuhl Castle, Germany, December 17–18, 1990, Proceedings; (Lecture Notes in Computer Science; Vol. 590). Springer; 1990. p. 318–342. Available from: https://doi.org/10.1007/3-540-55425-4_17.
- [51] Pinkas G. Reasoning, nonmonotonicity and learning in connectionist networks that capture propositional knowledge. Artif Intell. 1995;77(2):203–247. Available from: [https://doi.org/10.1016/0004-3702\(94\)00032-V](https://doi.org/10.1016/0004-3702(94)00032-V).
- [52] Shastri L. Advances in SHRUTI-A neurally motivated model of relational knowledge representation and rapid inference using temporal synchrony. Appl Intell. 1999;11(1):79–108. Available from: <https://doi.org/10.1023/A:1008380614985>.
- [53] Smolensky P, Legendre G. The harmonic mind: From neural computation to optimality-theoretic grammar-volume i: Cognitive architecture (bradford books). The MIT Press; 2006.
- [54] Minervini P, Riedel S, Stenortorp P, et al. Learning reasoning strategies in end-to-end differentiable proving ; 2020.
- [55] Campero A, Pareja A, Klinger T, et al. Logical rule induction and theory learning using neural theorem proving. CoRR. 2018;abs/1809.02193. Available from: <http://arxiv.org/abs/1809.02193>.
- [56] Bianchi F, Hitzler P. On the capabilities of logic tensor networks for deductive reasoning. In: Proceedings of the AAAI 2019 Spring Symposium on Combining Machine Learning with Knowledge Engineering (AAAI-MAKE 2019) Stanford University, Palo Alto, California, USA, March 25–27, 2019.; 2019. Available from: <http://ceur-ws.org/Vol-2350/paper22.pdf>.
- [57] Schlag I, Smolensky P, Fernandez R, et al. Enhancing the transformer with explicit relational encoding for math problem solving. CoRR. 2019;abs/1910.06611. Available from: <http://arxiv.org/abs/1910.06611>.
- [58] Hammer B, Hitzler P, editors. Perspectives of neural-symbolic integration. (Studies in Computational Intelligence; Vol. 77). Springer; 2007.
- [59] Wagner B, d'Avila Garcez A. Neural-symbolic integration for fairness in AI. In: Martin A, Hinkelmann K, Fill H, et al., editors. Proceedings of the AAAI 2021 Spring Symposium on Combining Machine Learning and Knowledge Engineering (AAAI-MAKE 2021), Stanford University, Palo Alto, California, USA, March 22–24, 2021; (CEUR Workshop Proceedings; Vol. 2846). CEUR-WS.org; 2021. Available from: <http://ceur-ws.org/Vol-2846/paper5.pdf>.
- [60] Marra G, Giannini F, Diligenti M, et al. Constraint-based visual generation. In: Tetko IV, Kurková V, Karpov P, et al., editors. Artificial Neural Networks and Machine Learning - ICANN 2019: Image Processing - 28th International Conference on Artificial Neural Networks, Munich, Germany, September 17–19, 2019, Proceedings, Part III; (Lecture Notes in Computer Science; Vol. 11729). Springer; 2019. p. 565–577. Available from: https://doi.org/10.1007/978-3-030-30508-6_45.

NEURO-SYMBOLIC ARTIFICIAL INTELLIGENCE: THE STATE OF THE ART

Neuro-symbolic AI is an emerging subfield of Artificial Intelligence that brings together two hitherto distinct approaches. "Neuro" refers to the artificial neural networks prominent in machine learning, "symbolic" refers to algorithmic processing on the level of meaningful symbols, prominent in knowledge representation. In the past, these two fields of AI have been largely separate, with very little crossover, but the so-called "third wave" of AI is now bringing them together.

This book, *Neuro-Symbolic Artificial Intelligence: The State of the Art*, provides an overview of this development in AI. The two approaches differ significantly in terms of their strengths and weaknesses and, from a cognitive-science perspective, there is a question as to how a neural system can perform symbol manipulation, and how the representational differences between these two approaches can be bridged. The book presents 17 overview papers, all by authors who have made significant contributions in the past few years and starting with a historic overview first seen in 2016. With just seven months elapsed from invitation to authors to final copy, the book is as up-to-date as a published overview of this subject can be.

Based on the editors' own desire to understand the current state of the art, this book reflects the breadth and depth of the latest developments in neuro-symbolic AI, and will be of interest to students, researchers, and all those working in the field of Artificial Intelligence.



ISBN 978-1-64368-244-0 (print)
ISBN 978-1-64368-245-7 (online)
ISSN 0922-6389 (print)
ISSN 1879-8314 (online)